

Nullexit: Unified Project Document

Omar Alaaeldein

July 18, 2026

Contents

1	Project Directory Tree	3
2	README.md	5
3	agent.md	12
4	devref.md	19
5	diagrams.md	70
6	toggle.sh	75
7	recover.sh	103
8	setup.sh	112
9	scripts/setup-linux.sh	113
10	docker-compose.yml	114
11	scripts/common.sh	117
12	scripts/watcher.sh	131
13	scripts/crypto.sh	135
14	.aiignore	136
15	.claude/settings.local.json	136
16	.gitignore	136
17	Recover-Gateway.applescript	137
18	Sweep-Gateway.applescript	137
19	Toggle-Gateway.applescript	138
20	benchmarks/benchmark.sh	138
21	benchmarks/test_load.py	139
22	black_list.txt	140

23	cloud-init.yaml	141
24	docker/tor/Dockerfile	142
25	docker/tor/entrypoint.sh	142
26	launchd/com.nullexit.noise.plist	143
27	launchd/com.nullexit.wake-recovery.plist	144
28	plan.md	145
29	post-rules.txt	145
30	scripts/Dockerfile.routing-fix	146
31	scripts/Dockerfile.rule-compiler	146
32	scripts/Toggle-Gateway.bat	146
33	scripts/check_circuits.py	149
34	scripts/diagnose-host-leak.sh	150
35	scripts/dns-proxy.py	159
36	scripts/dns_anomaly_detector.py	160
37	scripts/fix-docker-bridge-collision.sh	166
38	scripts/fix-ssh-delay.sh	171
39	scripts/generate_tex.py	172
40	scripts/logger/go.mod	174
41	scripts/logger/main.go	174
42	scripts/noise.sh	178
43	scripts/pf.conf	181
44	scripts/reinstall-host-tailscale.sh	182
45	scripts/routing-fix.sh	191
46	scripts/rule-compiler/go.mod	195
47	scripts/rule-compiler/main.go	195
48	scripts/setup-common.sh	206
49	scripts/sweep.sh	212
50	scripts/unlock-files.sh	216
51	tor-bridges.txt	217
52	white_list.txt	217

1 Project Directory Tree

```
nullexit/
|-- .aignore
|-- .claude
|   '-- settings.local.json
|-- .gitignore
|-- LICENSE
|-- README.md
|-- Recover-Gateway.applescript
|-- Sweep-Gateway.applescript
|-- Toggle-Gateway.applescript
|-- agent.md
|-- benchmarks
|   |-- benchmark.sh
|   '-- test_load.py
|-- black_list.txt
|-- cloud-init.yaml
|-- devref.md
|-- diagrams.md
|-- docker
|   '-- tor
|       |-- Dockerfile
|       '-- entrypoint.sh
|-- docker-compose.yml
|-- launchd
|   |-- com.nullexit.noise.plist
|   '-- com.nullexit.wake-recovery.plist
|-- plan.md
|-- post-rules.txt
|-- recover.sh
|-- scripts
|   |-- Dockerfile.routing-fix
|   |-- Dockerfile.rule-compiler
|   |-- Toggle-Gateway.bat
|   |-- check_circuits.py
|   |-- common.sh
|   |-- crypto.sh
|   |-- diagnose-host-leak.sh
|   |-- dns-proxy.py
|   |-- dns_anomaly_detector.py
|   |-- fix-docker-bridge-collision.sh
|   |-- fix-ssh-delay.sh
|   |-- generate_tex.py
|   |-- logger
|   |   |-- go.mod
|   |   '-- main.go
|   |-- noise.sh
|   |-- pf.conf
|   |-- reinstall-host-tailscale.sh
|   |-- routing-fix.sh
|   |-- rule-compiler
|   |   |-- go.mod
|   |   '-- main.go
```

```
| |-- setup-common.sh
| |-- setup-linux.sh
| |-- sweep.sh
| |-- unlock-files.sh
| '-- watcher.sh
|-- setup.sh
|-- sweep-20260718T035359Z.txt
|-- sweep-20260718T040634Z.txt
|-- sweep-20260718T040833Z.txt
|-- toggle.sh
|-- tor-bridges.txt
'-- white_list.txt
```

2 README.md

```
1 # nullexit: Tailscale + Cloudflare WARP Docker Gateway
2
3 > **Last updated:** July 12, 2026 [Architecture & Flow Diagrams ](./diagrams.md) [Full Dev Reference
4 ](./devref.md)
5
6 **nullexit** is a chained network gateway that routes all Tailscale exit-node traffic through a
7 Cloudflare WARP VPN tunnel double-encrypting every packet, hiding your ISP metadata, and providing
8 network-wide DNS ad-blocking (AdGuard Home) and kernel-level IP threat blocking ('ipset'/'iptables')
9 for every device on your mesh.
10
11 ---
12
13 ## 1. Prerequisites
14 - Docker and Docker Compose installed.
15 - A Tailscale account.
16 - **macOS:** Install the standalone Tailscale CLI via Homebrew ('brew install tailscale') and register '
17 tailscaled' as a system service ('brew services start tailscale'). No '.app' GUI install required.
18 - **OS & Python:** Developed and tested against **macOS 26.5.2**. The project has zero external Python
19 dependencies (no 'requirements.txt') and explicitly targets the default Apple-provided **Python
20 3.9.6** ('/usr/bin/python3'). While the scripts could function on newer versions, 'nullexit'
21 explicitly avoids experimenting with or modifying the built-in system Python environment to prevent
22 unexpected OS-level side effects.
23
24 ## 2. Installation & Setup
25
26 Run the interactive setup script for your OS. It downloads 'wgcf', generates fresh Cloudflare WARP
27 WireGuard keys, prompts for a Tailscale Auth Key, and writes your '.env'.
28
29 ```bash
30 ./setup.sh # macOS
31 ./scripts/setup-linux.sh # Linux
32 ```
33
34 ### Reference '.env'
35 This is a quick reference of the common variables. For the **complete, authoritative list** of every '.
36 env' variable (including all optional/advanced settings), see 'devref.md 7'.
37
38 ```env
39 WIREGUARD_PRIVATE_KEY=<Generated>
40 WIREGUARD_PUBLIC_KEY=<Generated>
41 WIREGUARD_ADDRESSES=<Generated>
42 TS_AUTHKEY=tskey-auth-...
43 GATEWAY_RULE_PROFILE=medium # light | medium | heavy
44 # Safe MSS for double-tunneled traffic (Tailscale + WARP). Change to 1180 if you experience slow speeds
45 and have a healthy path.
46 GATEWAY_MSS=1120
47 # Set to false to run as a 'Headless Server' (Docker acts as an exit node, but the host Mac/Linux's own
48 internet is NOT hijacked).
49 GATEWAY_HIJACK_HOST=true
50 GATEWAY_USE_EXIT_NODE=true
51 WARP_FAIL_THRESHOLD=6 # consecutive warp=off polls before auto-shutdown (default 6 = 30s)
52 KILL_SWITCH=false # enforce strict PF lock that breaks SSH if VPN fails
53 # On campus/enterprise networks (WPA2-Enterprise / 802.1x), AP Client Isolation blocks direct
54 # LAN traffic between devices. Tailscale attempts a local P2P upgrade anyway, which causes
55 # macOS gproxy to SNAT-mangle the reply's source port poisoning the phone's WireGuard
56 # endpoint and blackholing 100% of its traffic. Setting this to false drops those local
57 # hole-punch packets so the phone safely falls back to Tailscale DERP relays.
58 # watcher.sh auto-detects 802.1x and AP isolation on every Wi-Fi change and overrides
59 # this setting automatically you only need to set this manually if auto-detection fails.
60 TAILSCALE_ALLOW_LAN_P2P=false # auto-managed; true only on trusted hotspots/home networks
61 ADGUARD_USER=admin # Username for AdGuard Home (defaults to admin if not set)
62 ADGUARD_PASSWORD=nullexit # Shared password for AdGuard Home and Tor ControlPort (defaults to
63 nullexit if not set)
64 BLOCKED_COUNTRIES="kp il" # dynamically block country IP ranges via ipdeny.com
65 TOR_EXCLUDE_EXIT_NODES="" # comma-separated country codes to exclude as Tor exit nodes (e.g. {us
66 },{gb})
67 DEBUG_TRACE=false # true = mirror a full command-by-command trace of the toggle to
68 output.log (see 9)
69 ```
70
71 ## 3. Deploy the Gateway
72
73 **macOS (Recommended):**
74
75 ```bash
76 ./toggle.sh
77 ```
78
79 Running it again stops the gateway and restores your network. Handles the full lifecycle: Colima VM,
80 containers, DNS hijacking, Tailscale exit-node routing, rule compilation, and sleep prevention.
```

```

62 **Linux / Manual:**
63 '''bash
64 docker compose up -d
65 '''
66
67 ## 4. Post-Deployment
68
69 ### Approve the Exit Node
70 1. Go to [Tailscale Admin Machines](https://login.tailscale.com/admin/machines).
71 2. Find the 'tailscale' device '...' **Edit route settings** enable as Exit Node.
72
73 ### Verify Traffic Flow
74 On any Tailscale client device, select this gateway as the exit node, then visit [api.ipify.org](https://api.ipify.org) or run:
75 '''bash
76 curl -s https://www.cloudflare.com/cdn-cgi/trace | grep -E '^(warp|ip|colo)='
77 # expect: warp=on
78 '''
79
80 ### AdGuard DNS Setup
81
82 To ensure client devices correctly route DNS through the AdGuard container (and do not bypass it via 'tailscaled's internal resolver), you **must** configure your Tailscale Admin Console:
83 1. Go to the **DNS** tab in the [Tailscale Admin Console](https://login.tailscale.com/admin/dns).
84 2. Enable **MagicDNS**.
85 3. Under **Nameservers**, add a **Custom** nameserver and enter the gateway's Tailscale IPv4 address (run 'tailscale ip -4' on the gateway machine to find it, e.g., '100.x.x.x').
86 4. Click the **three dots (...)** next to the nameserver you just added and enable **"Use with exit node".
87 5. Delete any other Global Nameservers (like Google or Cloudflare).
88 6. Toggle ON **Override local DNS**.
89
90 Traffic will now be correctly forced into the AdGuard sinkhole, and mobile devices will be blocked from bypassing it via encrypted DNS (DoH/DoT).
91
92 > [!WARNING]
93 > **Disable "Private DNS" on mobile devices.** Android's Private DNS (DoT/DoH) bypasses port 53 interception entirely. Go to 'Settings Connections More connection settings Private DNS Off'.
94
95 > [!WARNING]
96 > **Bandwidth doubling:** Streaming 1GB of video on a remote device consumes 1GB download + 1GB upload on the host's network. Be mindful on metered connections.
97
98 ### AdGuard Dashboard
99 Navigate to 'http://<gateway-tailscale-ip>:3000' credentials: **'admin' / 'nullexit'**.
100
101 ### Access Any Mesh Device From Anywhere (SFTP/SSH)
102
103 Once the gateway exit node is online, **any device on your Tailscale mesh can reach any other mesh device, anywhere in the world** as long as both devices are on the mesh and the target device has an SSH server running. No port forwarding, no public IPs, no firewall holes. You can SSH into any device using its MagicDNS hostname (e.g. 'ssh username@my-macbook') or its assigned Tailscale IP ('100.x.x.x').
104
105 > **Security Tip (Zero Local Attack Surface):** It is highly recommended to disable macOS's native "Remote Login" and "Screen Sharing" in System Settings to prevent exposing those ports (22, 5900) to physical Wi-Fi networks (which scanners can fingerprint). The gateway enforces 'tailscale up --ssh=true' and the 'pf' Kill-Switch automatically blocks local Wi-Fi access to these ports. Since both MagicDNS and Tailscale IPs route through the exact same encrypted tunnel, they are equally securebut **MagicDNS is recommended simply for ease of use.**
106
107 ### Extreme OPSEC: The Secret Tor-over-WARP Proxy
108 'nullexit' includes a completely isolated, headless Tor infrastructure proxy designed for terminals and hacking tools (e.g., 'curl', 'sqlmap', 'nmap').
109 To protect against local malware fingerprinting:
110 1. **Procedural Ports:** The proxy ports are randomized into the ephemeral range (10000-65000) on every single boot using a strict kernel socket collision-avoidance check ('netstat').
111 2. **Honey-Port Tripwire:** The gateway spawns a fake trap port. If any malware attempts a sequential port-scan on 'localhost' to find the proxy, it trips the Honey-Port, instantly triggering a macOS desktop notification and a critical log alert.
112
113 > [!NOTE]
114 > **Threat Model limitation:** Port randomization and the Honey-Port only defeat blind, generic port scanners. They do not protect against targeted local malware that runs 'lsof -i' or reads the '/tmp/nullexit-ports.env' file directly. To actually prevent malicious local processes from reaching the proxy, you must run a host-level outbound firewall (like LuLu 4.3.0+ or Little Snitch) with **localhost/loopback filtering explicitly enabled** to gate access by process identity.
115
116 To use the Tor proxy, run 'cat /tmp/nullexit-ports.env' to find your '$TOR SOCKS_PORT' for the current session, and point your hardened browser (LibreWolf/Tor Browser) or CLI tool to '127.0.0.1:$TOR SOCKS_PORT'.

```

```

117 > [!WARNING]
118 > **Preventing DNS Leaks (SOCKS5 vs. SOCKS5-Hostname):**
119 > When routing command-line tools through Tor, you must force the tool to delegate DNS resolution to
120 the Tor proxy. Using plain SOCKS5 (e.g., 'socks5://' or curl's '--socks5' flag) will resolve
    hostnames locally using the host's DNS settings (AdGuard/WARP) before establishing the connection.
    While this DNS traffic is encrypted inside the WARP tunnel, the destination domain name is still
    leaked to AdGuard and Cloudflare, undermining your anonymity.
121 >
122 > Use these correct protocols and configurations:
123 > - 'curl': Use '--socks5-hostname' or the 'socks5h://' scheme (e.g., 'curl -x socks5h://127.0.0.1:
    $TOR SOCKS_PORT https://check.torproject.org'). Do not use '--socks5' or 'socks5://'.
124 > - 'sqlmap': Pass the proxy URL using the 'socks5h://' scheme to ensure remote resolution: '--proxy
    =socks5h://127.0.0.1:$TOR SOCKS_PORT'.
125 > - 'nmap': Nmap's built-in '--proxies' option resolves hostnames locally. To safely scan targets
    without DNS leaks, tunnel it using 'proxychains-ng' configured with 'proxy_dns' enabled (add 'socks5
    127.0.0.1 <PORT>' to '/etc/proxychains.conf' or a local config, then run 'proxychains4 nmap <args
    >').
126
127
128 ##### Transparent .onion Browsing & Dark Web Access
129 In addition to the randomized SOCKS5 proxy port, 'nullexit' provides seamless, transparent '.onion'
    domain browsing for all devices on your Tailscale mesh:
130 - Zero-Config DNS: AdGuard Home automatically intercepts queries for '.onion' domains and resolves
    them to Tor's virtual mapping subnet.
131 - Tailscale Auto-Routing: The virtual subnet is mapped to the '198.18.0.0/15' benchmark range (RFC
    2544). Since this range is not a private IP subnet, Tailscale's Exit Node mode automatically routes
    it to the gateway (bypassing local LAN exclusions).
132 - Transparent NAT Proxying: Kernel firewall rules in the gateway redirect all TCP traffic destined
    for '198.18.0.0/15' directly into Tor's transparent port ('9040').
133 - Exit Node Exclusion: Supports the ability to exclude specific countries from being used as Tor exit
    nodes via the 'TOR_EXCLUDE_EXIT_NODES' environment variable in '.env'.
134
135 ##### Enable in Web Browsers
136 To prevent accidental DNS leaks, web browsers block '.onion' lookups by default. To browse dark web sites
    natively:
137 - Firefox: Go to 'about:config', search for 'network.dns.blockDotOnion', and set it to 'false'. Type
    any onion address (e.g. 'duckduckgogg42xjoc72x3sjasowarfbgcmvfimafitt6twagswzczad.onion') directly
    into the URL bar and it will load.
138 - Mobile Browsers (iOS/Android): Connect to your Tailscale exit node, open Firefox (with
    blockDotOnion disabled), and browse onion sites natively on the go.
139
140 ##### Hardening: Using obfs4 Tor Bridges
141 If you want to disguise your Tor traffic from the WARP exit node provider, you can optionally enable Tor
    bridges:
142 1. Obtain private 'obfs4' bridge lines from [bridges.torproject.org](https://bridges.torproject.org) or
    the official Telegram bot.
143 2. Create a file named 'tor-bridges.txt' in the root of the 'nullexit' folder and paste your bridge lines
    inside it (one per line).
144 3. Set 'TOR_USE_BRIDGES=true' in your '.env' file and restart the gateway. The proxy will refuse to start
    if the bridges file is empty or missing.
145
146 ##### Example: Browse your Android phone's files from your Mac
147
148
149 1. On your Android phone, install [Termux](https://termux.dev/) and [Termux:Boot](https://f-droid.org
    /packages/com.termux.boot/) from F-Droid.
150 2. In Termux, install and start the SSH server:
151     ''bash
152     pkg install openssh
153     sshd
154     ''
155     Termux defaults to port 8022 (ports below 1024 require root on Android).
156 3. Find your phone's Tailscale IP:
157     ''bash
158     tailscale ip -4 # e.g. 100.87.42.87
159     ''
160 4. Stop Android from killing Termux do ALL of these (Samsung One UI is especially aggressive):
161     - Settings Apps Termux Battery Unrestricted
162     - Device Care Battery Background usage limits Sleeping apps / Deep sleeping apps remove Termux
    if listed
163     - In Termux, run 'termux-wake-lock' to hold a persistent wakelock
164 5. On your Mac/PC, open [Cyberduck](https://cyberduck.io/) (or any SFTP client: WinSCP, FileZilla, FE
    File Explorer on iOS) and connect to:
165     - Server: '100.87.42.87' (your phone's Tailscale IP)
166     - Port: '8022'
167     - Protocol: SFTP (SSH File Transfer Protocol)
168     - Username: (your Termux username, usually 'u0_a123' run 'whoami' in Termux to check)
169     - Password: (your Termux password set with 'passwd' in Termux if you haven't already)
170

```

```

171 That's it you can now browse, download, and upload files to your phone from any device on the mesh, no
    matter where either device is physically located. This works identically for any mesh device: Mac
    Mac, Windows Linux, phone NAS, etc.
172
173 > **Troubleshooting:** If the connection hangs for ~30 seconds before failing, your phone's SSH server
    isn't running. Open Termux and run 'sshd' again. If Android keeps killing it despite Unrestricted
    battery, the 'termux-wake-lock' command is your fix.
174
175 ---
176
177 ## 5. Multi-Layer Firewall & Kill-Switch
178
179 ### Layer 1 DNS Sinkhole (AdGuard Home)
180 Blocks ads and tracking domains before they are downloaded. In automated Lighthouse tests on ad-heavy
    sites:
181 - **Time to Interactive:** ~22% faster (51.0s 41.8s)
182 - **Speed Index:** ~20% faster (15.3s 12.8s)
183
184 Customize blocking via 'black_list.txt' and 'white_list.txt'. Rule profiles ('light' / 'medium' / 'heavy
    ') are set in '.env'.
185
186 ### Layer 2 Kernel IP Firewall ('ipset'/'iptables')
187 On every startup, the 'rule-compiler' fetches threat-intelligence feeds (Spamhaus DROP, Feodo Tracker,
    Emerging Threats, CINS) and compiles **~16,700 unique malicious IPs/CIDRs** into the kernel 'FORWARD
    ' chain blocking both outbound C2 connections and inbound attack traffic. Zero configuration
    required.
188
189 ### Layer 3 Geo-IP Blocking
190 Dynamically blocks all traffic to and from specific countries using live IP ranges from 'ipdeny.com'.
191 To add countries to your blocklist, open your '.env' file and set the 'BLOCKED_COUNTRIES' variable with
    2-letter ISO country codes (e.g. 'BLOCKED_COUNTRIES="kp il cn ru"'), then restart the gateway.
192
193 ### Layer 4 macOS PF Kill-Switch
194 A native macOS Packet Filter ('pf') kill-switch that enforces a strict default-deny policy on your
    physical Wi-Fi interface ('en0'). When 'KILL_SWITCH=true' is set in your '.env', it drops all
    outgoing traffic except the Cloudflare WARP endpoints and Tailscale DERP relays.
195
196 - **Failsafe Design:** If the VPN tunnel crashes or Docker dies, your Mac completely loses internet
    access rather than leaking your real IP.
197 - **Remote-Access Safe:** Carefully designed to whitelist Tailscale's control plane ('192.200.0.0/24'), '
    utun*' tunnel traffic, and local LAN subnets. This guarantees that your remote SSH sessions and
    local AirDrop will survive even when the kill switch engages.
198 - **Setup Requirement:** The toggle scripts need permission to manipulate the network and firewall in the
    background without prompting you for a password. You must configure your passwordless sudo config
    by running exactly:
199 "bash
200 echo "$USER ALL=(root) NOPASSWD: /sbin/pfctl, /usr/sbin/networksetup, /usr/bin/dscacheutil, /usr/bin/
    killall, /usr/bin/pkill, /bin/kill, /sbin/route, /sbin/ifconfig, /usr/bin/true, /opt/homebrew/bin/
    brew, /usr/local/bin/brew, /usr/bin/python3 -I $PWD/scripts/dns-proxy.py, /opt/homebrew/bin/python3
    -I $PWD/scripts/dns-proxy.py, /usr/local/bin/python3 -I $PWD/scripts/dns-proxy.py" | sudo tee /etc/
    sudoers.d/nullexit
201 "
202
203 ---
204
205 ## 6. Toggle Scripts
206
207 ### macOS 'toggle.sh'
208 Fully automated lifecycle script. Also supports a native '.app' launcher:
209 "bash
210 osacompile -o "Toggle Gateway.app" Toggle-Gateway.applescript
211 "
212
213 Optional '.env' settings (common subset; the full canonical table lives in 'devref.md 7'):
214 - 'GATEWAY_BYPASS_PING=true' proceed even if pre-flight connectivity checks fail.
215 - 'GATEWAY_USE_EXIT_NODE=false' DNS-only mode, skip exit-node routing.
216 - 'GATEWAY_HIJACK_HOST=false' skip DNS hijacking on the host (provides VPN/adblocking only to external
    Tailscale peers).
217 - 'GATEWAY_MSS=1180' override the TCP MSS clamp for the double-tunnel (default '1120'; raise to '1180'
    for more speed on a healthy path).
218 - 'HOST_MTU=1200' override the host Tailscale interface MTU. Defaults to 1200 to fit inside the 1280-
    byte WARP tunnel.
219 - 'KILL_SWITCH=true' enforce strict network lock that breaks SSH if the VPN fails.
220 - 'STOP_COLIMA_ON_EXIT=true' fully shut down the Colima VM on toggle-off (saves battery on dedicated
    hosts).
221 - 'TAILSCALE_ALLOW_LAN_P2P=true' allow direct Tailscale LAN P2P connections between devices on the same
    network.
222 - **Default: 'false'. ** On campus or enterprise Wi-Fi (WPA2-Enterprise / 802.1x), AP Client Isolation
    blocks local device-to-device traffic. When Tailscale still attempts a local P2P upgrade, macOS's '
    gvproxy' layer SNAT-mangles the reply's source port. WireGuard's endpoint roaming treats this as a
    legitimate address change and updates the phone's outbound path to the newly-mangled port which has

```



```

    no listener. This blackholes 100% of the phone's traffic while the DERP relay path is abandoned.
    Setting this to 'false' preemptively drops all RFC1918-destined Tailscale UDP from the gateway so
    the phone receives a clean failure and safely falls back to DERP.
223 - **Auto-managed:** 'watcher.sh' automatically detects WPA2-Enterprise via 'system_profiler
    SPAirPortDataType' (the 'airport' binary was removed in macOS 14.4) and probes for AP isolation by
    pinging the default gateway ('route -n get 1.1.1.1') on every network change. It writes the detected
    value to '.lan_p2p_detected' in the repo root, which 'routing-fix.sh' re-reads every 30 seconds **
    no restart required.** Only set this manually if auto-detection fails. See 'devref.md 15.7.1' for
    the full SNAT endpoint poisoning analysis and 'devref.md 15.11.6' for the 'airport' removal
    background. *(Note: Direct P2P between the gateway container and the host Mac itself is always
    permitted via the Docker bridge to bypass DERP relays, regardless of this setting).*
224 - Set to 'true' on trusted home networks or Windows/mobile hotspots (no AP isolation) for a faster,
    lower-latency direct path.
225 - 'WARP_FAIL_THRESHOLD=3' number of consecutive failed checks before forcing a gateway teardown (default
    : 6 checks = 30s).
226 - 'WARP_ENDPOINT_1' / 'WARP_ENDPOINT_2' override the Cloudflare WARP WireGuard endpoints.
227
228 ### Linux 'toggle.sh' / 'recover.sh'
229 ```bash
230 ./toggle.sh # toggle ON/OFF (cross-platform, dispatches on $OSTYPE)
231 ./recover.sh # recovery tool (cross-platform)
232 ```
233
234 ### Sleep Prevention (macOS)
235 When the gateway is ON, 'caffeinate -i' runs in the background to prevent idle system sleep, keeping
    Docker and all services alive even on battery. The display can still sleep normally.
236
237 ### Auto-Recovery (macOS post-wake / post-roam)
238 Install the launchd LaunchAgent so the gateway self-heals after sleep/wake and Wi-Fi roams:
239 ```bash
240 cp ./launchd/com.nullexit.wake-recovery.plist ~/Library/LaunchAgents/
241 sed -i '' "s|__NULLEXIT_HOME__|$(pwd)|" ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
242 launchctl load -w ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
243 ```
244 See 'devref.md 15.5.1' for the full deep dive.
245
246 ### Cover traffic / dummy padding (macOS optional, opt-in)
247 'scripts/noise.sh' emits verifiable-random UDP padding toward the exit so a passive
248 on-path observer sees a continuously varying encrypted flow instead of your real
249 traffic envelope (a metadata / traffic-flow correlation defence). Unlike the rest
250 of the toolkit it is **active** it puts traffic on the wire so it stays inert
251 unless you opt in. **One switch controls everything:** set 'NOISE_ENABLED=true' in
252 '.env'. There is no separate "persistent vs one-shot" option installing the
253 LaunchAgent below simply means padding auto-starts whenever 'NOISE_ENABLED=true',
254 and the job is a clean no-op whenever it is 'false'.
255
256 ```bash
257 # One-off (this session only):
258 bash scripts/noise.sh verify # prove the noise is statistically random (entropy/monobit/chi-square)
259 bash scripts/noise.sh start # arm padding now; 'status' / 'stop' to manage
260
261 # Persistent (survives reboot/login) install the LaunchAgent once:
262 cp ./launchd/com.nullexit.noise.plist ~/Library/LaunchAgents/
263 sed -i '' "s|__NULLEXIT_HOME__|$(pwd)|" ~/Library/LaunchAgents/com.nullexit.noise.plist
264 launchctl load -w ~/Library/LaunchAgents/com.nullexit.noise.plist
265 # To stop persisting: launchctl bootout gui/$UID/com.nullexit.noise
266 ```
267
268 Tunables (rate, packet size, jitter, target) live in '.env' under the
269 'NOISE_PAD_*' keys; defaults are a modest 64 kbps toward the gateway. Honest
270 limits: this is one-way random padding it blurs uplink volume + timing, not a
271 full traffic-analysis-resistant shaper. See the 'noise.sh' / 'Cover Traffic &
272 Padding' notes in the knowledge graph.
273
274 ---
275
276 ## 7. Networking Notes
277
278 - **Port conflicts:** None. Tailscale uses dynamic UDP ports; WARP uses outbound UDP:2408. The gateway
    binds '0.0.0.0:41642/udp' on the host for Tailscale's WireGuard listener this is intentional and
    required to receive inbound hole-punch packets from peers on the internet. All other internal ports
    (AdGuard :5335, SOCKS5 :1080) are bound to '127.0.0.1' only and are not reachable from the LAN.
279 - **Tailscale P2P vs DERP:** On the same Wi-Fi, Tailscale establishes a fast P2P connection. On cellular
    (CGNAT), it always falls back to a DERP relay (~80-200ms). See 'devref.md 5.1'.
280 - **AirDrop / AirPlay:** Unaffected the gateway only touches standard Wi-Fi/Ethernet interfaces, not '
    awdl0'.
281 - **VPN coexistence:** Do **not** run a local VPN client (WARP, Mullvad, NordVPN) simultaneously with the
    exit node. Both fight for the default route and will blackhole your connection.
282 - **Banking & financial sites:** May intermittently block logins through WARP. Banks use anti-fraud
    databases that flag datacenter IP ranges (like Cloudflare's '104.28.x.x') as VPN/proxy traffic.
    Success fluctuates depending on which WARP IP you land on. If a site blocks you, either disable the
    exit node temporarily for that session or use the bank's mobile app (which often uses different

```

```

    fraud detection).
282 - **MSS clamping (Bufferbloat):** Double-tunnelling adds ~120-160 bytes of overhead per packet. This is
    now automatically handled natively via the macOS 'pf' firewall, which universally applies TCP MSS
    Clamping to all outbound packets to strictly prevent MTU fragmentation and bufferbloat. No manual '.
    env' configuration is required. Additionally, the host's Tailscale interface MTU is explicitly
    lowered to 1200 (configurable via 'HOST_MTU' in '.env') to guarantee packets fit inside the
    container's WARP tunnel without fragmenting.
283
284 ---
285
286 ## 8. Privacy & Security Hardening
287
288 The stack shifts trust across four layers: WARP hides traffic from your ISP, AdGuard blocks tracking at
    DNS, Tailscale encrypts device-to-device, and 'ipset' blocks known malicious IPs at the kernel.
289
290 For higher security, the gateway supports these drop-in upgrades:
291
292 | Layer | Default | Hardened |
293 |---|---|---|
294 | VPN Exit | Cloudflare WARP (US) | Mullvad (Swedish, no-logs, anonymous payment) |
295 | DNS Resolver | AdGuard via WARP | AdGuard via Mullvad DoH |
296 | Coordination | Tailscale (US) | Headscale (self-hosted) |
297 | Auth | Persistent OAuth keys | Ephemeral auth keys |
298 | Content | WireGuard asymmetric | WireGuard + Pre-Shared Keys (PSK) |
299
300 ##### Python Runtime Airgap (Isolated Mode)
301 **Never** install third-party 'pip' packages (like 'requests' or 'pyyaml') into the Python environment
    that 'nullexit' uses. All python scripts in this project (e.g., the 'dns-proxy.py' daemon) are
    strictly engineered to run on the zero-dependency Python Standard Library. To strictly isolate the
    environment from supply-chain attacks, 'nullexit' executes these scripts using **Python's Isolated
    Mode** ('python3 -I'). This deliberately blinds the execution environment to any malicious user-
    installed 'pip' packages on the host, permanently air-gapping the gateway from PyPI.
302
303 See 'devref.md 9' for the full threat model, Snowden programme analysis, and trust assumptions.
304
305 ---
306
307 ## 9. Debugging
308
309 - **'output.log'** All stderr from 'toggle.sh', 'recover.sh', 'setup.sh', and the rule-compiler is
    written here. The WARP Watcher ('WARP_FAIL_THRESHOLD') also writes its events here 'WARP DOWN', '
    WARP RECOVERED', and 'WARP SHUTDOWN' notices. Check this first.
310 - Every run now records lifecycle breadcrumbs regardless of 'DEBUG_TRACE': an 'EXEC: / 'EXIT <code>:'
    line brackets each heavyweight command ('colima start', 'docker compose up -d --build', 'docker
    compose down'), and an 'EXIT: code=<N> phase=<init|stop|start>' line is written when the script
    exits **for any reason** including a killed terminal, 'pkill', or OOM. This closes a prior blind
    spot where an interrupted **START** (e.g. a '--restart' race while Wi-Fi was re-associating) left
    the log ending abruptly after the teardown with no indication of what failed. If a toggle "does
    nothing" or the gateway ends up half-up, 'grep -E 'EXEC:|EXIT ' output.log | tail' shows the last
    command run and its result.
311 - **'DEBUG_TRACE=true'** (in '.env') Full command-by-command execution trace of 'toggle.sh', mirrored to
    'output.log' for deep post-mortems of intermittent failures. Each traced line is timestamped and
    tagged with 'file:line:function' (via a custom 'PS4'), so you can follow the *exact* code path a run
    took including subshells, Docker/Colima calls, and where it aborted. Off by default because it is
    verbose (the log grows quickly); enable it only while reproducing a specific problem, then set it
    back to 'false'. Implementation note: it prefers bash's 'BASH_XTRACEFD' (bash 4.1, e.g. most Linux)
    to send the trace to the log while keeping your terminal clean; on macOS's stock '/bin/bash' 3.2 it
    transparently falls back to tee'ing stderr to the log (trace also appears in the terminal). It never
    uses the dynamic-fd ('exec {var}>>') form, so it is safe on 3.2. To read a trace: 'grep -nE '~\+ '
    output.log'.
312 - **'bash scripts/diagnose-host-leak.sh'** One-shot host-routing diagnostic. Runs 8 checks and
    classifies your state into one of four known scenarios (System Extension conflict, SOCKS5 fallback,
    IPv6 leak, route-table freeze) or OK, and prints the exact fix command. Check 5 uses 'route -n get
    1.1.1.1' to ask the kernel which interface real traffic actually resolves through (more accurate
    than 'netstat -rn | grep default' on macOS). Pass '--fix' to apply the remediation automatically.
    Pass '--watch' (or '--watch 30') to run a full baseline diagnostic then continuously monitor warp/
    IPv6/default-route for leaks, alerting on any state change.
313 - **'bash scripts/sweep.sh'** One-shot, read-only gateway health sweep. Runs 6 checks containers up/
    healthy, double-tunnel/IP-leak (host egress == container egress with 'warp=on'), direct P2P to the
    gateway (no DERP relay), AdGuard compiled-rule counts + live DNS interception, PF kill-switch armed,
    and tailnet reachability (a 'tailscale ping' to every online peer) then rolls them into a PASS/
    WARN/FAIL verdict. Writes a timestamped 'sweep-<UTC>.txt' report to the repo root (diffable across
    runs) and exits non-zero on FAIL, so it can gate launchd hooks or CI. Pass '--quiet' for just the
    verdict line. It never mutates routing, PF, DNS, or containers safe to run against a live gateway
    at any time.
314 - **'bash scripts/fix-docker-bridge-collision.sh'** One-shot fix for Docker bridge IP conflicts. If your
    local Wi-Fi router assigns you an IP in the '172.17.0.0/16' range, it will violently collide with
    Docker's default internal bridge, permanently killing your internet. This script detects the
    collision and auto-patches your Colima VM to use a safe subnet (e.g. '10.200.0.1/24').
315 - **'devref.md'** Complete architecture reference, routing deep-dives, and full resolved-issues log.
    Read this before modifying any routing or firewall logic.

```

```

316 > [!CAUTION]
317 > **Two infinite routing loops are possible.** (1) If a hotspot device providing internet to the gateway
318 host is *also* using the gateway as its exit node, packets bounce forever between the two. (2) When
    the host's default route is redirected to 'utun*', WARP endpoint packets can loop back into the
    tunnel. Both are documented in 'devref.md 15.2.1' and '15.2.2' and are automatically handled by '
    toggle.sh'.
319 ---
320 ---
321
322 ## 10. Unified Project Document (Quine)
323
324 The project includes a 'generate_tex.py' script that compiles the entire source code, configurations, and
    documentation into a single, unified LaTeX document ('nullexit_unified.tex' / '.pdf').
325
326 Funnily enough, this document acts as a "project-level quine." It encapsulates the entirety of its own
    source code, its documentation, and the exact Python code required to generate itself. It serves as
    a self-contained, long-term archiving strategy providing everything needed to reconstruct and
    understand the 'nullexit' system from a single file.
327 ---
328 ---
329
330 ## 11. Acknowledgements
331 - **[SyameimaruKoa](https://github.com/SyameimaruKoa):** Dual-stack TCP MSS clamping, 'SIGHUP' state-
    tracking in the routing sidecar, and 'TS_AUTH_ONCE' integration.
332
333 ## 12. License
334 GNU Affero General Public License v3. See [LICENSE](./LICENSE).
335
336 ## 13. Changelog
337
338 - **July 17, 2026** 'scripts/sweep.sh' (the read-only automated health sweep added July 15, mirroring '
    sweep.md 1') gained a 6th check: **tailnet reachability** every Tailscale peer reported online is
    probed with 'tailscale ping' (TSMP, so it works even on phones that drop ICMP) and rolled into the
    PASS/WARN/FAIL verdict. Documented in 9 above.
339 - **July 12, 2026** Tor ControlPort dynamic password authentication (unified with 'ADGUARD_PASSWORD'),
    enabling live circuit inspection via '/sweep'. See 'devref.md 15.10.2'.
340 - **July 11, 2026** Tor container hardening: 'obfs4proxy' restored (base image 'debian:bookworm-slim'),
    robust bridge-file validation, and image pinning. See 'devref.md 15.10.1'.
341 - **July 10, 2026** Roaming/startup stabilization suite: fixed the Wi-Fi-roam host-IP leak (raw ISP
    '132.x') caused by the WARP Watcher racing post-wake recovery, plus a batch of pre-flight/
    concurrency/kill-switch/Tailscale-flag bugs and the macOS SNAT endpoint-poisoning P2P blackhole.
    Full post-mortems in the Incident Log see 'devref.md 15.2.7', '15.5.3', '15.5.5', '15.2.8', '15.2.9',
    '15.7.1', '15.7.3', '15.7.5', '15.12.18'.
342 - **July 6, 2026** Edge-case security and stability hardening: DNS Watcher interface independence,
    robust lock-file PID verification, fail-closed crypto integrity check, strict 'iptables' pinning for
    tailscale0tun0, Docker port binding to '127.0.0.1' to prevent LAN exposure, routing verification
    via 'netstat -nr'. See 'devref.md 17'.
343 - **July 4, 2026** Colima bridged networking, SOCKS5 proxy migrated natively to Tailscale, headless
    prompt crashes fixed, proxy bypass domains added to fix LAN P2P. Python dependencies completely
    eliminated: 'logger' and 'rule-compiler' ported to blazing-fast Go multi-stage Docker builds. Added
    'crypto.sh' to enforce cryptographic signature validation on all core bash scripts. Massively
    refactored 'toggle.sh' and 'recover.sh' to eliminate ~200 lines of duplicated code by migrating
    network and process logic to 'scripts/common.sh'. Added a concurrency mutex to 'toggle.sh', resolved
    unbound 'TS_AUTHKEY' check crashes on setup, and integrated Go validation to the CI pipeline.
344 - **July 3, 2026** **Critical Security Updates:** Addressed the overnight silent IP leak and improved
    geo-blocking. Introduced 'scripts/host-leak-probe.sh' (a sub-second host-egress leak detector) and a
    statistical threshold auto-shutdown watcher. Added 'unlock-files.sh' to safely reset file
    permissions. Resolved numerous startup regressions and system bugs.
345 - **July 2, 2026** Two infinite routing loops resolved ('devref.md 15.2.2'): host-VM tunnel loop (WARP
    bypass routes now via gateway IP + manual 'utun*' redirection) and Docker Compose subnet takeover
    ('10.200.1.0/24' lock). '--accept-routes=true' now explicit on all 'tailscale up --reset' calls.
346 - **July 2, 2026** Auto-recovery daemon documented in 6 above ('scripts/watcher.sh' + launchd plist).
    See 'devref.md 15.5.1'.
347 - **July 2, 2026** Linux scripts ('scripts/toggle-linux.sh', 'recover-linux.sh', 'setup-linux.sh')
    documented in 6.
348 - **July 1, 2026** 'recover.sh --post-wake' ships; wired to watcher daemon. Triggered on every sleep/
    wake or network-state change.
349 - **June 28, 2026** 8 internal scripts moved to 'scripts/'. User-facing files ('toggle.sh', 'recover.sh',
    'setup.sh', 'docker-compose.yml') stay at repo root.
350 - **June 28, 2026** All hardcoded install paths removed. 'SCRIPT_DIR'--relative resolution and '
    _NULLEXIT_HOME_' placeholder used throughout.
351
352 ### Cryptographic Integrity Verification
353
354 nullexit uses a built-in cryptographic integrity checker designed to provide tamper-*evidence* against
    accidental edits, logic bugs, or non-privileged malware. During setup, a 256-bit 'NULLEXIT_SEED' is
    injected into your '.env' file. The core endpoint scripts ('toggle.sh', 'recover.sh', etc.) are
    hashed using HMAC-SHA256, and their signatures are stored in '.signatures'.
355

```

```

356 *(Note: This is not a strict boundary against an attacker who already has write access to the repo, as
    they could simply read the seed and re-sign the scripts. It is designed as a strict footgun-catcher)
    .*
357
358 If any script is modified without authorization, the gateway will loudly fail to boot. If you make
    intentional edits to the bash scripts, you must re-sign them by running:
359 ``bash
360 ./scripts/crypto.sh --sign
361 ``

```

3 agent.md

```

1 # nullexit Detailed Agent Analysis
2
3 > Complete per-file breakdown of every function, constant, duplication, and bug found.
4 > **Note (July 2026):** 'sync-rules.py' and 'logger.py' have been fully ported to Go ('scripts/rule-
    compiler/main.go' and 'scripts/logger/main.go') to dramatically improve performance and reduce
    container footprint via multi-stage Alpine builds. The analysis below reflects their previous Python
    states.
5
6 ## Important Agent Instruction
7 - **NEVER use 'cat << EOF' or terminal redirections to write or modify files.** You must always use your
    native file-editing tools ('replace_file_content' or 'multi_replace_file_content'). Using 'cat EOF'
    leads to duplicated text, broken markdown headers, and structural corruption (which previously
    destroyed the section numbering in 'devref.md').
8 - **Always run 'bash scripts/crypto.sh --sign' after making any modifications to the core bash scripts.**
    This is required to update the cryptographic HMAC-SHA256 signatures; otherwise, the startup
    integrity checks will fail and block execution.
9 - **Always run 'git diff' and print the changes to the user *before* requesting or executing any git
    commit command.** The user must be shown the diff so they can review and understand the changes.
    Based on this diff, formulate a highly precise, detailed, and descriptive commit message (explaining
    exactly what was changed and why) rather than a generic summary, and present it to the user
    alongside the diff.
10 - **Always run 'git status' before 'git diff' to identify all modified and untracked files, ensuring no
    files (such as new scripts or configuration assets) are missed before formulating commit diffs and
    staging.**
11 - **Always use the '--restart' flag when asked to restart the toggle (e.g., run './toggle.sh --restart')
    .**
12 - **NEVER execute any 'sudo' commands (especially 'sudo route', 'sudo dscacheutil', or any network-
    modifying commands) without explicitly explaining to the user what the command does and why it is
    needed FIRST. Do not assume implicit permission. Wait for the user's approval before running it.**
13 - **When fixing an error and using logs to debug, always show the user the reasoning and the specific
    logs that support this theory.**
14 - **When the user tells you to "push", this means read git diffs and prepare commit messages that
    correspond to these changes (do not ignore any changes). If the changes can be atomized into
    multiple commits for easier understanding, do so.**
15 - **When the user says "latex this", it means you should run 'python3 scripts/generate_tex.py' to
    regenerate the unified LaTeX document.**
16 - **NEVER apply changes to running gateway containers directly via 'docker compose up' or 'docker compose
    restart'.** Recreating or restarting containers (especially the 'warp' container which hosts the
    network namespace) breaks the shared network stack for all other services ('adguardhome', 'tailscale',
    'routing-fix'), severing host connectivity and freezing macOS DNS. If you need to apply changes
    or rebuild containers, cleanly stop the gateway first using './toggle.sh', or trigger 'recover.sh --
    post-wake' to rebuild and re-verify the network stack safely in the correct dependency order. Do NOT
    run './toggle.sh --restart' or any network-rebuilding commands yourself if the execution could
    sever host network access; instead, explain the changes and ask the USER to run './toggle.sh --
    restart' (or './toggle.sh' stop and start) themselves to safely apply the modifications.
17
18 ## How to Verify Gateway is Working
19 Whenever modifications are made to the gateway scripts, routing, or containers, execute these
    verification checks in order:
20 1. **Container Health:** Run 'docker compose ps' to ensure all containers ('warp', 'adguardhome', '
    routing-fix', 'tailscale') are 'running' and 'healthy'.
21 2. **DNS Hijacking Status:** Run 'networksetup -getdnsservers "Wi-Fi"' (or appropriate network service)
    and ensure it is set exclusively to the gateway's Tailscale static IP (typically '100.100.21.8').
22 3. **DNS Query Interception:** Run 'dig @100.100.21.8 google.com' (using the gateway static IP from '
    gateway_ip') to ensure AdGuard Home is active, intercepting, and resolving queries.
23 4. **Double-Tunnel Egress:** Run 'curl -s https://www.cloudflare.com/cdn-cgi/trace | grep warp' to verify
    the egress is routed through Cloudflare WARP ('warp=on').
24 5. **Host Leak Monitoring:** Run 'tail -n 30 output.log' and verify the background watchers (DNS + WARP)
    report healthy status without 'LEAK' warnings. For a deeper on-demand audit, run 'bash scripts/
    diagnose-host-leak.sh' (add '--watch' to poll, '--fix' to remediate).
25
26 **Automated:** 'bash scripts/sweep.sh' runs a read-only 6-check health sweep covering the above plus the
    PF kill-switch and tailnet-wide peer reachability, writes a timestamped 'sweep-<UTC>.txt' report to
    the repo root, and exits non-zero on FAIL ('--quiet' for just the verdict). It never mutates routing
    /PF/DNS/containers, so it is safe against a live gateway. See its entry in Part 3.

```

```

27 ---
28 ---
29
30 ## Part 1: macOS Main Scripts
31
32 ### toggle.sh (2191 lines, unified macOS + Linux)
33
34 > **Refreshed 2026-07-15.** Post-unification ('$OSTYPE' switch at L20) + the July-2026 'common.sh'
    extraction, the tables below reflect the *current* file. Most former "toggle.sh functions" now live
    in 'scripts/common.sh'; the host-leak-probe pair was removed. For the ordered execution flow see **
    toggle.sh Execution Map & Critical Path*** above.
35
36 **Functions still defined in 'toggle.sh'** several exist in *both* halves (Linux L63848 / macOS L8782191
    ):
37
38 | Function | Linux L | macOS L | Purpose |
39 |-----|-----|-----|-----|
40 | '_log_exit_breadcrumb()' | 30 | 885 | Write lifecycle-phase breadcrumb on exit (post-mortem trail) |
41 | 'get_free_port()' | 67 | 944 | Pick a random unused port (self- + kernel-collision checked) |
42 | 'run_gui_cmd()' | 1033 | | Run a command as the logged-in console GUI user |
43 | 'write_gateway_active_marker()' | 1053 | | Write '/tmp/nullexit-gateway-active.marker' (macOS only) |
44 | 'clear_gateway_active_marker()' | 1059 | | Remove active marker + 'TUNNEL_FAILED_CLOSED.marker' |
45 | 'start_sleep_prevention()' | 145 | 1091 | Start 'caffeinate' w/ revert trap |
46 | 'stop_sleep_prevention()' | 188 | | Stop caffeinate via PID file |
47 | 'stop_dns_watcher()' | 202 | | Stop DNS watcher (macOS uses 'stop_pidfile_daemon') |
48 | 'start_warp_watcher()' | 1164 | | Background WARP liveness monitor 'recover.sh' after N fails |
49 | 'stop_warp_watcher()' | 1241 | | Stop WARP watcher via PID file |
50 | 'cleanup_handler()' | 215 | 1258 | Fail-closed teardown trap (guarded by 'SUCCESS_RUN') |
51 | 'restart_tailscaled_daemon()' | 327 | *(common)* | Restart tailscaled (Linux-half local copy; macOS
    uses common.sh) |
52 | 'disconnect_tailscale_host()' | 337 | *(common)* | 'tailscale down' w/ retry (Linux-half local copy;
    macOS uses common.sh) |
53 | 'setup_exit_node_routing()' | 1430 | | Override default route through the Tailscale utun interface |
54 | 'cleanup_network_state()' | 354 | 1464 | Nuclear cleanup: proxies, DNS cache, routes, Wi-Fi, sharing
    services |
55
56 **Delegated to 'scripts/common.sh'** (moved out of toggle.sh in the July-2026 refactor; line = common.sh)
    :
57
58 | Function | common.sh L | Function | common.sh L |
59 |-----|-----|-----|-----|
60 | 'run_with_timeout()' | 58 | 'enable_socks_proxy()' | 769 |
61 | 'is_gateway_active()' | 168 | 'disable_socks_proxy()' | 803 |
62 | 'gateway_state()' | 232 | 'reset_dns()' | 824 |
63 | 'get_active_service()' | 284 | 'stop_local_dns_proxy()' | 898 |
64 | 'restart_tailscaled_daemon()' | 338 | 'start_local_dns_proxy()' | 909 |
65 | 'disconnect_tailscale_host()' | 352 | 'force_dns_to_gateway()' | 988 |
66 | 'stop_pidfile_daemon()' | 393 | 'add_warp_bypass_routes()' | 484 |
67 | 'start_dns_watcher()' | 715 | 'remove_warp_bypass_routes()' | 551 |
68
69 **Removed:** 'start_host_leak_probe()' / 'stop_host_leak_probe()' and the whole 'HOST_LEAK_PROBE' prober
    subsystem no longer defined or called anywhere in 'toggle.sh', 'recover.sh', or 'common.sh', and '
    HOST_LEAK_PROBE' is no longer an env var (all verified). The only surviving host-leak tool is the
    standalone, on-demand 'scripts/diagnose-host-leak.sh' (there is **no** 'scripts/host-leak-probe.sh')
    .
70
71 **Constants / anchors (current):**
72
73 | Constant | Value | Line |
74 |-----|-----|-----|
75 | 'LOG_FILE' | '$PWD/output.log' | L22 (Linux) / L863 (macOS) |
76 | 'LOCK_FILE' | '/tmp/nullexit-toggle.lock' | L1020 |
77 | 'PID_FILE' (caffeinate) | '/tmp/nullexit-caffeinate.pid' | L142 |
78 | 'DNS_WATCHER_PID_FILE' | '/tmp/nullexit-dns-watcher.pid' | L200 |
79 | 'WARP_WATCHER_PID_FILE' | '/tmp/nullexit-warp-watcher.pid' | L1144 |
80 | Gateway active marker | '/tmp/nullexit-gateway-active.marker' | L1054 (write) |
81 | TUNNEL_FAILED_CLOSED marker | '<repo>/TUNNEL_FAILED_CLOSED.marker' | L1061 |
82 | 'SOCKS_PROXY_PORT' | random per-toggle, '1080' fallback | L91/L107 (Linux) L968/L986 (macOS) |
83 | DNS search domain | 'ts.net' | L763 |
84 | 'PATH' (via 'setup_standard_path', common.sh) | Homebrew + system bins | L1330 |
85 | WARP endpoints (via 'get_warp_endpoint_1/2', common.sh) | '162.159.192.1' / '162.159.193.1' | common.sh
    L335-336 |
86 | DNS fallback | '1.1.1.1' (in 'reset_dns', common.sh) | common.sh L824 |
87
88 ---
89
90 ### toggle.sh Execution Map & Critical Path (verified 2026-07-15)
91
92 > Traced from the current unified 'toggle.sh' (2191 lines). The file carries **two halves** selected by '
    $OSTYPE' at **L20** ('if [[ "$OSTYPE" != "darwin"* ]]' runs the self-contained **Linux path L63848
    ** then 'exit 0' at ~L845; darwin falls through to the **macOS path L8782191**). Line numbers below

```

```

    are the **macOS path** (the deployment target). The halves are *similar in intent but not identical
    * see "Linux path where it diverges" below. '~Line' = current, approximate.
93
94 ##### Entry & dispatch
95
96 '''
97 ./toggle.sh [ (no-arg) | --restart | restart ]
98 |
99 | -- verify HMAC signatures (crypto.sh --verify) --fail--> exit 1      (L857)
100 | -- rotate output.log if > 1GB
101 | -- arm teardown trap: ERR/INT/TERM/HUP -> cleanup_handler            (L1317-1320)
102 | -- generate random ports (Tor / SOCKS / DNS)                          (L940)
103 |
104 | -- --restart --> gateway_state? running -> run STOP, then START      (L999)
105 |
106 | -- no-arg --> gateway_state?                                         (L1535)
107 |     running -> STOP branch (L1538-1584)
108 |     stopped -> START branch (L1590-2191)
109 '''
110
111 'gateway_state' is driven by '/tmp/nullexit-gateway-active.marker' (persisted intent), falling back to a
    live container probe.
112
113 ##### START   synchronous critical path (all must complete)
114
115 | # | Step | ~Line | Effect if it fails / is interrupted |
116 | ---|-----|-----|-----|
117 | 1 | 'disconnect_tailscale_host' (clean slate) | 1595 | host left mesh |
118 | 2 | 'colima_start_until_ready' (readiness poll, 15.8.7) | 1631 | no Docker VM abort |
119 | 3 | honey-port reap ('pkill') + arm ('nc -l', disowned) | 1695 | tripwire missing (non-fatal) |
120 | 4 | rule-compile (hash-gated, OFF critical path 15.8.8) | 1708 | reuse prior compiled rules |
121 | 5 | build-gate routing-fix / tor images | 1743 | rebuild only on change |
122 | 6 | 'docker compose up -d' warp, tailscale, routing-fix, adguard, tor | 1758 | no gateway |
123 | 7 | wait tailscaled reachable | 1907 | host tailscale unusable |
124 | 8 | **host 'tailscale set --exit-node=TS_IP'** | 2046 | **host not routed via gateway REAL-IP LEAK**
125 | 9 | 'add_warp_bypass_routes' + 'setup_exit_node_routing' | 2048 | WARP endpoint unreachable / wrong
    route |
126 | 10 | **'force_dns_to_gateway'** (+ 'start_local_dns_proxy') | 2078 | host DNS not hijacked |
127 | 11 | 'enable_socks_proxy' + verify SOCKSWARP | 2100 | SOCKS off |
128 | 12 | print **"STARTED in Ns"** (timing marker) | 2127 | cosmetic only NOT the commit point |
129 | 13 | 'start_sleep_prevention' (caffeinate) | 2130 | Mac may sleep |
130 | 14 | 'start_dns_watcher' (re-hijack DNS every 30s) | 2134 | no roam DNS recovery |
131 | 15 | 'start_warp_watcher' (warp liveness recover.sh) | 2138 | no WARP self-heal |
132 | 16 | **'write_gateway_active_marker'** | 2146 | watcher believes gateway is DOWN |
133 | 17 | **'SUCCESS_RUN=true'** disarms the teardown trap | 2164 | *(true commit point)* |
134 | 18 | print "You can close this terminal window now." | 2191 | |
135
136 ##### STOP   critical path
137
138 'disconnect_tailscale_host' (1543) 'docker compose down --remove-orphans' (1552) 'colima stop' (1558)
    'reset_dns' (1568) 'stop_local_dns_proxy' (1571) stop caffeinate (1574) stop DNS watcher (1575)
    'stop_warp_watcher' (1576) 'clear_gateway_active_marker' (1577) "STOPPED in Ns" (1584).
139
140 ##### Long-lived background processes spawned by START
141
142 | Process | Started | PID / reap | Role |
143 | ---|-----|-----|-----|
144 | honey-port 'nc -l localhost $PORT' (disowned) | L1695 | 'pkill -f "nc -l localhost"' | port-scan
    tripwire |
145 | 'caffeinate' | L2130 | '/tmp/nullexit-caffeinate.pid' | prevent system sleep |
146 | DNS watcher loop | L2134 | '/tmp/nullexit-dns-watcher.pid' | re-hijack DNS every 30s (Wi-Fi roam) |
147 | WARP watcher loop | L2138 | '/tmp/nullexit-warp-watcher.pid' | monitor warp; fire recover.sh after N
    fails |
148 | local DNS proxy 'dns-proxy.py' | L2089 | (killed by 'stop_local_dns_proxy') | UDP:53 gateway DNS
    fallback |
149
150 Separately, the **launchd 'scripts/watcher.sh'** daemon (post-wake / post-roam recovery) runs
    independently of toggle and invokes 'recover.sh --post-wake'; toggle does **not** spawn it.
151
152 ##### 'cleanup_handler' the fail-closed teardown trap
153
154 Armed on 'ERR/INT/TERM/HUP' (L1317-1320), guarded by 'SUCCESS_RUN'. While 'SUCCESS_RUN=false' it runs: '
    reset_dns', 'disconnect_tailscale_host', 'stop_local_dns_proxy', stop watchers, '
    clear_gateway_active_marker', 'docker compose down' (+ 'colima stop' on 'ERR'). This is the **fail-
    closed guarantee**: a half-built gateway is torn down, not left leaking.
155
156 ##### Correctness findings (verified against source)
157
158 1. **"STARTED in Ns" (L2127) is NOT the commit point 'SUCCESS_RUN=true' (L2164) is.** The marker write (
    L2146) and watcher starts (L2130-2138) sit *between* them. **Any TERM/INT including a wrapping tool

```



```

's timeout in the L2127L2164 window fires 'cleanup_handler' a FULL teardown** (containers + colima
+ host tailscale), leaving the marker ABSENT and the host on its raw ISP link, *despite* the
printed success. This is exactly the 2026-07-15 incident. (The Linux half has the same shape but a *
much* narrower window: STARTED L814 'SUCCESS_RUN=true' L820, and it maintains no marker.)
159 2. **Never pipe 'toggle.sh' through a reader** ('| tail', '| grep'). The disowned children (honey-port,
watchers, dns-proxy, caffeinate) inherit stdout, so the pipe never EOFs and the reader hangs long
after toggle has exited which then trips the wrapper's timeout SIGTERM finding #1. Run detached
with a file redirect instead: 'nohup ./toggle.sh --restart > /tmp/t.log 2>&1 &', then poll the file.
160 3. **Host wiring is correctly on the synchronous critical path** (steps 8 & 10 precede "STARTED"), so a *
completed* START cannot leave the host unrouted/leaking the only way to that state is an
interruption per finding #1.
161 4. **Namespacing ordering:** step 6's 'docker compose up' creates warp's network namespace that 'tailscale
'/'routing-fix'/'adguard' share (compose 'depends_on: warp healthy' enforces order). Never 'docker
compose restart warp' on a live gateway it detaches the shared netns (see Agent Instruction, L16).
162
163 ##### Linux path where it diverges (L63848)
164
165 The Linux half is intent-similar but **not** a line-for-line mirror of macOS. Verified differences:
166
167 | Aspect | macOS (L8782191) | Linux (L63848) |
168 |-----|-----|-----|
169 | VM layer | Colima ('colima_start_until_ready', L1631) | none native Docker |
170 | Host exit-node assert | '$TS_BIN set --exit-node="$TS_IP"' (L2046) | '$TS_BIN up --reset --exit-node
="$TS_IP"' (L735) |
171 | gateway-active marker | written (L2146) | **not maintained** (L119) 'gateway_state' degrades to a live
container probe |
172 | Commit window | STARTED L2127 'SUCCESS_RUN' L2164 | STARTED L814 'SUCCESS_RUN' L820 |
173 | Port gen tooling | 'jot' / 'netstat' | 'shuf'/'$RANDOM' / 'ss' |
174
175 The 'up --reset' vs 'set' distinction matters: '--reset' re-applies *all* prefs and forces a route re-
eval, whereas 'set' mutates only the exit-node pref the same asymmetry 'recover.sh --post-wake'
relies on (devref 15.12.21).
176
177 ---
178
179 ### recover.sh (566 lines, 23KB)
180
181 **Functions (7 total):**
182
183 | # | Function | Lines | Purpose |
184 |---|---|---|---|
185 | 1 | 'step()' | L40 | Print bold step header |
186 | 2 | 'ok()' | L41 | Print green success message |
187 | 3 | 'warn()' | L42 | Print yellow warning message |
188 | 4 | 'fail()' | L43 | Print red failure message |
189 | 5 | 'die()' | L44 | Print red error and exit |
190 | 6 | 'run_with_timeout()' | L5674 | Run command with timeout (bash-native, no GNU coreutils) |
191 | 7 | 'restart_tailscaled_daemon()' | L118129 | Restart tailscaled daemon via brew services |
192
193 **Constants:**
194
195 | Constant | Value | Line |
196 |-----|-----|-----|
197 | Color codes | 'RED', 'GREEN', 'YELLOW', 'BOLD', 'NC' | L37-38 |
198 | 'SCRIPT_DIR' | '${dirname "${BASH_SOURCE[0]}"}' | L50 |
199 | 'PATH' export | '/opt/homebrew/bin:/usr/local/bin:$PATH' | L77 |
200 | 'PID_FILE' | '/tmp/nullexit-caffeinate.pid' | L387 |
201 | 'DNS_WATCHER_PID_FILE' | '/tmp/nullexit-dns-watcher.pid' | L407 |
202 | WARP endpoints | '162.159.192.1', '162.159.193.1' | L329-336 |
203 | TUNNEL_FAILED_CLOSED marker | '$SCRIPT_DIR/TUNNEL_FAILED_CLOSED.marker' | L51 |
204
205 **Duplicated logic from toggle.sh:**
206 *(Note: As of July 2026, **ALL** of the below duplicated logic has been successfully extracted to '
scripts/common.sh'!)*
207 - 'run_with_timeout()' simpler subshell version vs toggle's PID-tracking version
208 - 'restart_tailscaled_daemon()' near-identical (uses warn()/ok() vs echo)
209 - Active network service detection inline at L217-233 (toggle has 'get_active_service()' function)
210 - EN0 service resolution inline at L230-233 (same as toggle L515-516)
211 - WARP bypass routes inline at L329-337 (toggle has 'add_warp_bypass_routes()')
212 - Exit node routing setup inline at L340-345 (toggle has 'setup_exit_node_routing()')
213 - DNS cache flushing L296-302 (same as toggle L611-616)
214 - Wi-Fi power-cycling L442-461 (similar to toggle L626-637)
215 - Proxy disabling L282-288 (same as toggle L604-608)
216 - Sharing services reset L586 (same as toggle L642)
217 - PID file stop pattern L387-399, L407-419, L423-435 (same as toggle L157-167, L193-203, L312-322)
218 - KILL_SWITCH check L194 (same as toggle L141, L469)
219 - .gateway_ip reading L138-142, L209-213 (same pattern as toggle L1080)
220
221 ---
222
223 ### setup.sh (410 lines, 18KB)

```

```

224
225 **Functions (4 total):**
226
227 | # | Function | Lines | Purpose |
228 |---|-----|-----|-----|
229 | 1 | 'step()' | L10 | Print bold blue step header |
230 | 2 | 'ok()' | L11 | Print green success message |
231 | 3 | 'warn()' | L12 | Print yellow warning message |
232 | 4 | 'die()' | L13 | Print red error and exit |
233
234 **Constants:**
235
236 | Constant | Value | Line |
237 |-----|-----|-----|
238 | Color codes | 'RED', 'GREEN', 'YELLOW', 'BLUE', 'BOLD', 'NC' | L7-8 |
239 | 'RECOMMENDED_TS_VERSION' | '1.98.5' | L71 |
240 | 'ADGUARD_PASSWORD' | 'nullexit' | L215 |
241 | Default .env values | 'GATEWAY_RULE_PROFILE=medium', 'GATEWAY_MSS=1120', etc. | L240-248 |
242
243 ---
244
245 ## Part 2: Linux Scripts
246
247 ### toggle.sh Linux path (unified into the root cross-platform script)
248
249 The Linux toggle is no longer a separate file. 'scripts/toggle-linux.sh' was
250 merged into the repo-root 'toggle.sh', which dispatches on '$OSTYPE': an early
251 'if [[ "$OSTYPE" != darwin* ]]' block runs the self-contained Linux toggle
252 (shuf / ss / nmcli / ip / systemd) and exits before the macOS body. The shared
253 DNS/proxy/daemon primitives it used were already moved to 'common.sh' in the
254 '313dc32' unification, so both branches call them by the same names.
255
256 ---
257
258 ### recover.sh Linux path (unified into the root cross-platform script)
259
260 Linux recovery is no longer a separate file. 'scripts/recover-linux.sh' was
261 merged into the repo-root 'recover.sh', which dispatches on '$OSTYPE': the Linux
262 branch runs a self-contained teardown (resolvectl / nmcli / ip) and exits before
263 the macOS dual-mode body. All formatting/timeout helpers come from 'common.sh'.
264
265 ---
266
267 ### scripts/setup-linux.sh (416 lines, 18KB)
268
269 **Functions (4 total):**
270
271 | # | Function | Line | Purpose |
272 |---|-----|-----|-----|
273 | 1 | 'step()' | L10 | Print formatted step header |
274 | 2 | 'ok()' | L11 | Print green success message |
275 | 3 | 'warn()' | L12 | Print yellow warning message |
276 | 4 | 'die()' | L13 | Print red error + exit 1 |
277
278 **~95% identical to macOS setup.sh.** Only differences:
279 1. Desktop shortcuts (L366-391 Linux '.desktop' files vs macOS '.app' via 'osacompile')
280 2. Final instructions text
281 3. .env template: macOS adds 'GATEWAY_USE_EXIT_NODE', 'WARP_FAIL_THRESHOLD', 'KILL_SWITCH' Linux omits
282 these
283
284 ---
285
286 ## Part 3: Utility Scripts
287
288 ### scripts/diagnose-host-leak.sh (722 lines, 38KB)
289
290 **Purpose:** Comprehensive host-routing diagnostic. 8 checks classify host IP leaks. Supports '--fix' and
291 '--watch' modes.
292
293 **Duplicated logic:**
294 - 'resolve_active_service()' identical pattern as toggle.sh, recover.sh, fix-docker-bridge-collision.sh
295 - 'run_with_timeout()' reimplemented timeout
296 - Color codes RED/GREEN/YELLOW/BOLD/NC with tty detection
297 - WARP check via 'cloudflare.com/cdn-cgi/trace'
298 - 'export PATH="/opt/homebrew/bin:/usr/local/bin:$PATH"'
299 - .gateway_ip reading pattern
300
301 ### scripts/fix-docker-bridge-collision.sh (402 lines, 18KB)
302
303 **Purpose:** Fixes Docker bridge subnet collisions with '10.200.1.0/24'.

```



```

303 **Duplicated logic:**
304 - 'resolve_active_iface()' same interface detection as diagnose-host-leak.sh
305 - Color codes, step()/ok()/warn()/fail()/die() formatting helpers
306
307 ### scripts/fix-ssh-delay.sh (63 lines, 2.7KB)
308
309 **Purpose:** One-shot fix for ~20s SSH delay. Standalone, minimal duplication (uses hardcoded / instead
    of color functions).
310
311 ### scripts/reinstall-host-tailscale.sh (740 lines, 31KB)
312
313 **Purpose:** Complete uninstall + reinstall of host Tailscale.
314
315 **Duplicated logic:**
316 - 'with_timeout()' yet another bash timeout implementation (polling-based, different from diagnose and
    toggle versions)
317 - Color codes + logging functions
318 - System Extension detection logic (similar to diagnose-host-leak.sh)
319
320 ### scripts/routing-fix.sh (266 lines, 10.5KB)
321
322 **Purpose:** Runs INSIDE warp container. Maintains routing table 200, FORWARD rules, country blocking, IP
    blocklists, tunnel health checks.
323
324 **Duplicated logic:**
325 - (Fixed) WARP_ENDPOINT defaults are now centralized in common.sh
326 - WARP health check via cdn-cgi/trace (another instance)
327 - iptables dual-backend pattern (for-loop over 'iptables iptables-legacy')
328
329 ### scripts/sweep.sh (331 lines, 18KB)
330
331 **Purpose:** Read-only post-restart health sweep answers "after a toggle/restart/wake, is the gateway
    actually healthy end-to-end?" Never mutates routing, PF, DNS, or containers, so it is safe to run
    against a live gateway at any time (including from a remote session). 'errexit' is deliberately OFF:
    probes are *expected* to fail on an unhealthy gateway and each exit status is recorded as the
    diagnostic signal.
332
333 **The 6 checks** (each recorded PASS/WARN/FAIL; exit 0 only when nothing FAILs, so it can gate CI/launchd
    /post-restart hooks):
334 1. **Containers** 'docker compose ps': warp/adguardhome/tailscale/tor/routing-fix up (warp + adguardhome
    additionally 'healthy').
335 2. **Double-tunnel/leak** container WARP egress IP == host egress IP, both 'warp=on' (gluetun ships wget
    not curl, so the in-container trace falls back to wget).
336 3. **Hostcontainer path** the gateway peer line in 'tailscale status' shows 'direct <ip:port>'; WARNs on
    a DERP relay.
337 4. **AdGuard filtering** 'compiled_rules.txt' block counts present + live interception ('dig @' the
    gateway IP from '.gateway_ip').
338 5. **PF kill-switch** 'pfctl' reports Enabled AND anchor 'com.apple/nullexit' carries rules (needs
    passwordless sudo, else WARN).
339 6. **Tailnet reachability** (added 2026-07-17) every peer 'tailscale status' reports online (self and '
    offline' peers skipped; a '-' status means online-but-idle and is kept) must answer 'tailscale ping
    --timeout=3s --c 1 --until-direct=false' (TSMP probes the WireGuard data path even on devices that
    drop ICMP). All pong PASS; some unreachable WARN; none FAIL. DERP-relayed pongs count as
    reachable (direct-only enforcement stays in check 3, which covers the gateway).
340
341 **Output:** coloured stdout summary + a timestamped plain-text report 'sweep-<TIMESTAMP>.txt' in the repo
    root (one file per run, so runs can be diffed); '--quiet' prints only the verdict. Signed in '.
    signatures' re-run 'bash scripts/crypto.sh --sign' after editing it.
342
343 ### scripts/watcher.sh (269 lines, 13KB)
344
345 **Purpose:** Long-running launchd daemon for post-wake/post-roam recovery.
346
347 **Key function 'detect_lan_p2p_mode()':**
348 Added July 10, 2026. Called on startup and on every network state change (Listener 2 NET: events).
    Determines whether the current Wi-Fi network safely allows direct Tailscale LAN P2P connections:
349 1. **WPA2-Enterprise check:** 'system_profiler SPAirPortDataType' reads the 'Security:' field for the
    current association. Enterprise = 802.1x = always AP-isolated sets 'allow=false'.
350 2. **AP isolation probe:** 'route -n get 1.1.1.1' finds default gateway IP, then 'ping -c 3 -t 1 -q "$gw
    "' if 0 responses on a non-empty network, AP isolation is active sets 'allow=false'.
351 3. **Output:** Writes 'true' or 'false' to '.lan_p2p_detected' in the repo root.
352 4. **Consumer:** 'routing-fix.sh' reads this file every 30 seconds and enforces/relaxes the RFC1918 DROP
    rule accordingly no restart required.
353
354 **Duplicated logic:**
355 - PATH export (similar to toggle.sh/recover.sh)
356 - Marker file pattern ('/tmp/nullexit-gateway-active.marker')
357
358 ### scripts/dns-proxy.py (92 lines, 2.9KB) Clean, standalone
359
360 ### scripts/logger.py (144 lines, 5.9KB) Clean, standalone

```

```

361 ---
362 ---
363
364 ## Part 4: Cross-Cutting Issues
365
366 ### Functions Identical Across macOS Linux
367
368 | Function | macOS toggle | macOS recover | macOS setup | Linux toggle | Linux recover | Linux setup |
369 |-----|:-----|:-----|:-----|:-----|:-----|:-----|
370 | 'run_with_timeout()' | | | identical | identical | |
371 | 'stop_local_dns_proxy()' | | | identical | | |
372 | 'start_local_dns_proxy()' | | | ~95% similar | | |
373 | 'is_gateway_active()' | | | ~80% similar | | |
374 | 'step/ok/warn/die' | | | identical | identical |
375
376 ### Platform-Adapted Functions (same logic, different OS commands)
377
378 | Function | Linux Command | macOS Command |
379 |-----|-----|-----|
380 | 'get_active_service()' | 'ip route get 1.1.1.1 \\\ awk '{print $5}' ' | 'route get default' + '
    networksetup -listnetworkserviceorder' |
381 | 'reset_dns()' | 'resolvectl revert' | 'networksetup -setdnsservers' |
382 | 'cleanup_network_state()' | 'ip link set dev down/up' | 'networksetup -setairportpower off/on' + proxy
    disable |
383 | 'force_dns_to_gateway()' | 'resolvectl dns' + 'resolvectl domain ~ts.net' | 'networksetup -
    setdnsservers' + '-setsearchdomains ts.net' |
384 | 'start_sleep_prevention()' | 'systemd-inhibit --what=sleep' | 'caffeinate -i' |
385 | 'enable/disable_socks_proxy()' | Stub (no-op on Linux) | 'networksetup -setsocksfirewallproxy' |
386 | DNS cache flush | 'resolvectl flush-caches' | 'dscacheutil -flushcache' + 'killall -HUP mDNSResponder'
    |
387
388 ### Features Missing From Linux Scripts
389
390 These exist in 'toggle.sh's macOS branch but NOT its Linux branch:
391
392 1. 'write_gateway_active_marker()' / 'clear_gateway_active_marker()'
393 2. 'restart_tailscaled_daemon()'
394 3. 'start_warp_watcher()' / 'stop_warp_watcher()' WARP liveness monitor
395 5. 'add_warp_bypass_routes()' / 'remove_warp_bypass_routes()' / 'setup_exit_node_routing()'
396 6. 'LOG_FILE' structured logging (timestamps to output.log)
397 7. MSS clamping injection (step 4c)
398 8. Rule count reporting
399 9. '--post-wake' mode in recover
400 10. Container teardown on failure in cleanup_handler
401
402 ---
403
404
405
406 ## Part 5: Constant Duplication Map
407
408 | Value | Files |
409 |-----|-----|
410 | '162.159.192.1' (WARP endpoint) | docker-compose.yml, routing-fix.sh (now dynamically loaded via .env/
    common.sh!) |
411 | '5335' (AdGuard DNS port) | docker-compose.yml, AdGuardHome.yaml, post-rules.txt |
412 | '5354' (mapped DNS port) | docker-compose.yml, dns-proxy.py |
413 | '41642' (Tailscale WireGuard) | docker-compose.yml, routing-fix.sh, post-rules.txt |
414 | '1080' (SOCKS5 port) | docker-compose.yml, toggle.sh |
415 | '1120' (MSS clamp) | post-rules.txt, .env |
416 | 'admin:nullexit' (AdGuard creds) | AdGuardHome.yaml (bcrypt) |
417 | '10.200.1.0/24' (Docker subnet) | (Fixed) no longer duplicated |
418 | 'America/New_York' (timezone) | docker-compose.yml (4 services) |
419 | '/tmp/nullexit-caffeinate.pid' | (Fixed) centralized in common.sh |
420 | '/tmp/nullexit-dns-watcher.pid' | (Fixed) centralized in common.sh |
421 | '/opt/homebrew/bin:...' (PATH) | (Fixed) centralized in common.sh |
422
423 ---
424
425 ## Part 6: Refactoring Outcome (Implemented July 2026)
426
427 **Decision: "Scripts-only" architecture**
428 Instead of introducing a new 'lib/' directory, all shared code was moved into the existing 'scripts/'
    directory to keep the project hierarchy flat and simple.
429
430 ### 1. 'scripts/common.sh' (288 lines, 10KB)
431 Extracts the fundamental primitives and networking logic used across orchestrator scripts:
432 - **Formatters:** 'step()', 'ok()', 'warn()', 'fail()', 'die()'
433 - **Timeout Wrapper:** 'run_with_timeout()' (Canonical version with PID tracking, graceful SIGTERM, and
    SIGKILL fallback)

```

```

434 - **Daemon Lifecycle:** 'restart_tailscaled_daemon()', 'stop_pidfile_daemon()' (collapsed caffeinate, DNS
    watcher, and WARP watcher teardown logic)
435 - **Network Interface/Routing:** 'get_active_service()', 'get_en0_service()', 'bounce_wifi_interfaces()',
    'add_warp_bypass_routes()', 'remove_warp_bypass_routes()'
436 - **Proxy/DNS Cleanup:** 'disable_all_proxies()', 'flush_dns_cache()', 'reset_sharing_services()'
437 - **Configuration Helpers:** 'read_adguard_ip()', 'is_kill_switch_enabled()', 'get_warp_endpoint_1()', '
    get_warp_endpoint_2()' (which centralizes the hardcoded 162.159.192.1 / .193.1 into .env logic)
438
439 ### 2. 'scripts/setup-common.sh' (~350 lines)
440 Extracts the heavy, duplicated installation logic shared between 'setup.sh' (macOS) and 'scripts/setup-
    linux.sh':
441 - Docker and Colima prerequisite checks
442 - 'wgcf' binary download and WARP key generation
443 - '.env' configuration file generation
444 - Interactive Tailscale Auth Key and AdGuard Rule Profile prompts
445
446 ### Sourcing Pattern
447 - macOS root scripts ('toggle.sh', 'recover.sh', 'setup.sh') source via:
448   'source "$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/scripts/common.sh"'
449 - Scripts under 'scripts/' ('watcher.sh', 'diagnose-host-leak.sh', etc.) source via:
450   'source "$(dirname "${BASH_SOURCE[0]}")/common.sh"'
451
452 ## Agent Verbs / Protocols
453
454 ### '/sweep' (or 'sweep the gateway')
455 When the user invokes this verb, the agent MUST perform a full end-to-end connectivity, health, security,
    and performance audit of the gateway:
456 1. **WARP Validation:** Run 'curl -s https://www.cloudflare.com/cdn-cgi/trace | grep warp=' (must equal '
    on').
457 2. **Tor Proxy & Control Validation:**
458   - Source the dynamically generated ports from '/tmp/nullexit-ports.env' (or identify them using '
    docker compose ps').
459   - **SOCKS5 Check:** Run 'curl -s --socks5-hostname 127.0.0.1:$TOR SOCKS_PORT https://check.torproject.
    org/api/ip' to confirm the SOCKS5 proxy successfully connects and routes through the Tor network.
460   - **Control Port Check:** Query the Tor Control Port using netcat: 'echo "PROTOCOLINFO" | nc
    127.0.0.1:$TOR_CONTROL_PORT'. Verify it returns a standard '250-PROTOCOLINFO' response, confirming
    the Control Port is active and communicating.
461   - **Circuit & Node Lookup:** Query the Tor Control Port for the active path ('GETINFO circuit-status')
    . For each active built circuit, parse the relay fingerprints, lookup their IP addresses ('GETINFO
    ns/id/<fingerprint>'), and resolve their country codes using Tor's local GeoIP database ('GETINFO ip
    -to-country/<IP>'). Include the list of active circuits, showing the role (Guard/Middle/Exit),
    nickname, IP, and country of each hop in the final sweep report.
462   - **Transparent Onion Validation:** Run 'nslookup
    duckduckgogg42xjoc72x3sjasowoarfbgcmvfimaftt6twagswzczad.onion' to verify it resolves to an IP
    within '198.18.0.0/15' benchmark range. Verify transparent dark web routing by running 'curl -sI -H
    "Host: duckduckgogg42xjoc72x3sjasowoarfbgcmvfimaftt6twagswzczad.onion" http://<resolved-ip>/' (
    should return HTTP 301/200).
463 3. **DNS Leak Validation:**
464   - Audit the upstream resolver currently utilized by the host. Run 'curl -s https://edns.ip-api.com/
    json' to fetch details of the DNS resolver processing the client's requests.
465   - Verify that the resolver IP and organization returned belong to Cloudflare (e.g., AS13335) or match
    the WARP tunnel's exit range, and that **no** DNS traffic leaks to the user's raw physical ISP DNS.
466 4. **Performance Benchmark Audit:**
467   - Run the python benchmark script 'python3 benchmarks/test_load.py'.
468   - Report the overall load speed and the ratio of successfully fetched vs. blocked/failed subresources
    for the targeted domains to ensure ad-blocking and MTU encapsulation are performing correctly.
469 5. **Tailscale Mesh Validation:**
470   - Run 'ping -c 1 100.100.21.8' to ensure the gateway is reachable.
471   - Run 'tailscale status' to identify all active/online nodes in the mesh.
472   - For every online peer returned in the status output, execute 'tailscale ping --timeout=3s --c 1 --
    until-direct=false <peer-ip>' (TSMP probes the WireGuard data path and works even on phones that
    drop ICMP) to verify connectivity and confirm healthy routing to all active mesh devices. 'bash
    scripts/sweep.sh' automates this as its check 6/6 and reports each peer's path (direct endpoint vs '
    DERP(...)') and latency.
473 6. **Log Audit:** Run 'tail -n 100 output.log | grep -iE "(error|fail|warn)"' to catch any underlying
    component failures or warnings.
474
475 ### '/latex' (or 'generate latex')
476 When the user invokes this verb, the agent MUST run 'python3 -I scripts/generate_tex.py' to regenerate
    the documentation source code ('nullexit_unified.tex'). The agent MUST NOT attempt to compile the '.
    tex' file into a PDF (the user handles PDF compilation locally).

```

4 devref.md

```

1 # nullexit Development Reference & Resolved Issues
2

```

```

3 > **Last updated:** July 12, 2026
4 > **Purpose:** Provide any LLM or developer with complete project understanding, debugging history, and
   resolved issues so they can make informed changes without re-reading every file.
5 > **Diagrams:** See ['diagrams.md'](/diagrams.md) for system architecture, toggle.sh flowcharts,
   monitoring layer, traffic sequence, recover.sh decision tree, and the full failuresself-healing map.
6
7 This reference is organized into four parts:
8 > - **Part I Architecture & Reference** (18)
9 > - **Part II Design Decisions & Threat Model** (914)
10 > - **Part III Incident Log & Resolved Issues** (15)
11 > - **Part IV Observations, Changelog & TODO** (1618)
12
13 ---
14
15 # Part I Architecture & Reference
16
17 ## 1. What Is nullexit?
18
19 nullexit is a **Tunnel-in-Tunnel VPN gateway** that chains **Tailscale** (mesh VPN) through **Cloudflare
   WARP** (exit VPN). It runs on a macOS host inside Docker containers managed by **Colima** (
   lightweight Linux VM). The goal: any device on the user's Tailscale mesh can use this gateway as an
   exit node, and all traffic exits through Cloudflare WARP achieving double encryption, ISP
   invisibility, and network-wide ad-blocking via **AdGuard Home**.
20
21 ### Traffic Flow
22 ```
23 Phone/Laptop Tailscale mesh (WireGuard) Mac host Docker container
24 Tailscale decrypts Gluetun re-encrypts WARP tun0 Cloudflare Internet
25 ```
26
27 ### SOCKS5 Failover Path (if exit node is unavailable)
28 ```
29 DNS: Browser host 127.0.0.1:53 Python proxy TCP:5354 AdGuard WARP DNS
30 TCP: Apps macOS SOCKS5 proxy (127.0.0.1:1080) container tun0 WARP Internet
31 ```
32
33 ---
34
35 ## 2. Architecture
36
37 ### Containers (all share 'warp's network namespace via 'network_mode: service:warp')
38 | Container | Image | Role |
39 |-----|-----|-----|
40 | 'warp' | 'qmcgaw/gluetun:v3.41.1' | Gluetun WireGuard client Cloudflare WARP. Owns the network
   namespace. Strict firewall. |
41 | 'tailscale' | 'tailscale/tailscale:v1.98.4' | Advertises as exit node on the mesh. Also provides the
   built-in SOCKS5 proxy fallback via 'TS_SOCKS5_SERVER=:1080'. |
42 | 'routing-fix' | 'alpine:3.20' | Sidecar that maintains routing tables + iptables rules every 30 seconds
   . |
43 | 'rule-compiler' | 'golang:1.22-alpine' / 'alpine:3.20' | One-shot startup container that compiles 500k+
   DNS block rules in <2s and immediately exits. |
44 | 'adguardhome' | 'adguard/adguardhome:v0.107.77' | DNS sinkhole for ads/trackers. Listens on port 5335.
   Upstream DNS: '127.0.0.1:53' (through WARP). |
45
46 ### Port Mappings (on host via Colima)
47 | Host Port | Container Port | Protocol | Purpose |
48 |-----|-----|-----|-----|
49 | 5354 | 5335 | TCP+UDP | AdGuard DNS |
50 | (Tailscale IP):3000 | 3000 | TCP | AdGuard web UI (Internal to Mesh) |
51 | 41641 | 41641 | UDP | Tailscale WireGuard direct |
52 | 1080 | 1080 | TCP | SOCKS5 proxy |
53
54 ### Host Components
55 - **Colima VM** Linux VM running Docker (600MB RAM + 400MB swap)
56 - **tailscaled** Standalone Tailscale daemon via 'brew install tailscale' + 'brew services start
   tailscale' (NOT the GUI '.app')
57 - **macOS network settings** DNS, SOCKS proxy, routing all manipulated by 'toggle.sh'
58
59 ---
60
61 ## 3. Key Files
62
63 | File | Lines | Purpose |
64 |-----|-----|-----|
65 | 'toggle.sh' | ~2109 | **Main script (macOS + Linux).** Dispatches on '$OSTYPE' an early Linux block (
   shuf/ss/nmcli/ip, systemd) runs and exits before the macOS body (jot/netstat/networksetup, launchd/
   pf). Detects state, toggles gateway ON/OFF. Handles DNS hijacking, Tailscale exit node, SOCKS proxy,
   sleep prevention, timeouts, cleanup. |
66 | 'setup.sh' | 406 | **One-time setup.** Installs deps (Docker, Tailscale, wgcf), generates WARP keys,
   writes '.env', configures AdGuard via API, starts containers. |
67 | 'docker-compose.yml' | 194 | Service definitions for all 5 containers. |

```

```

68 | 'scripts/routing-fix.sh' | 201 | Maintains routing tables (table 200 for SOCKS5, table 52 for CGNAT)
    | and FORWARD iptables rules in both nftables and legacy backends. Loads and enforces the IP blocklist
    | via 'ipset'. Runs in a 30-second loop. |
69 | 'post-rules.txt' | 18 | Gluetun iptables rules loaded at container start. FORWARD accept for tailscale0
    | , NAT MASQUERADE on tun0, DNS redirect to port 5335, IPv6 drop, TCP MSS clamping. |
70
71 | 'scripts/dns-proxy.py' | 91 | Local DNS proxy. UDP:53 TCP:5354 with proper 2-byte length prefix.
    | Fallback when exit node is unavailable. |
72 | 'scripts/rule-compiler/' | ~800 | Unified threat compiler (ported to Go from Python). **DNS pipeline:**
    | fetches remote blocklists, deduplicates against AdGuard native filters, optimizes subdomains (~50%
    | reduction), compiles to AdGuard syntax. **IP pipeline:** fetches threat-intel feeds (Spamhaus, Feodo
    | , ET, CINS), strips private/Tailscale ranges, outputs atomic 'ipset' restore file. Built dynamically
    | in Docker multi-stage build. |
73 | '**adguard/work/**' | | **Bind-mounted container data folder. Off-limits from outside Docker READ-
    | ONLY-EXCEPT-VIA-'sync-rules'. See README 6.** |
74 | 'recover.sh' | ~742 | Nuclear recovery script (**macOS + Linux** dispatches on '$OSTYPE'; the Linux
    | branch runs a self-contained teardown via 'resolvectl'/'nmcli'/'ip' and exits, macOS falls through
    | to the dual-mode body). Gain: use '--post-wake' (macOS) for non-destructive refresh after sleep/wake
    | or Wi-Fi roam keeps the gateway live while still re-hijacking DNS + refreshing Tailscale exit-node
    | + force-recreating warp if unhealthy. Resets DNS on ALL services, disables proxies, flushes routes,
    | stops containers, kills sleep prevention, power-cycles Wi-Fi. Cryptographically verified. |
75 | 'scripts/diagnose-host-leak.sh' | 580+ | **One-shot host-routing diagnostic.** Runs 8 checks (Tailscale
    | , SOCKS5, CDN-cgi/trace, IPv6 leak, default route, output.log hints, gateway IP, Tailscale System
    | Extension conflict), classifies into ONE of three known leak scenarios (A=SOCKS5 fallback, B=IPv6
    | leak, C=route freeze) or OK, writes a timestamped report to 'host-leak-diagnostic-<UTC>.txt', and
    | prints a ready-to-run fix on stdout. Pass '--fix' to apply the matched remediation and re-verify
    | egress. Pass '--watch' (or '--watch 30') to run a full baseline then continuously monitor warp/IPv6/
    | default-route every N seconds, alerting on any state change. See 15.6.1. |
76 | 'scripts/host-leak-probe.sh' | | **REMOVED (2026-07-15, verified deleted/untracked).** Was a
    | continuous sub-second host-egress probe auto-launched via 'HOST_LEAK_PROBE'. Its fail-closed role
    | is now covered by the **PF kill-switch** ('enable_killswitch' in 'common.sh') + the in-container **
    | WARP Watcher** (15.6.2); on-demand auditing is 'scripts/diagnose-host-leak.sh'. See 15.6.1 (retained
    | as historical rationale). |
77 | 'scripts/fix-docker-bridge-collision.sh' | ~290 | **One-shot fix for 15.2.3 (Docker bridge subnet
    | collision).** Detects Docker's '172.17.0.0/16' bridge colliding with the host Wi-Fi subnet (the
    | symptom that produces 'default 172.17.0.1 UGScg en0' in 'netstat -rn'), picks a non-colliding
    | 'docker.bip' from a small candidate set, idempotently edits '/.colima/default/colima.yaml' with a
    | timestamped backup, restarts Colima, rebinds Wi-Fi DHCP, verifies the new default route is no longer
    | Docker's bridge, and re-runs 'toggle.sh' + 'scripts/diagnose-host-leak.sh'. Supports '--dry' and
    | '--skip-toggle'. |
78 | 'scripts/watcher.sh' | 194 | **Long-running post-wake + post-roam daemon.** Launched by launchd
    | LaunchAgent; subscribes to 'com.apple.powermanagement' events via 'log stream' and to network-state
    | changes via 'scutil n.watch'. Both fire 'recover.sh --post-wake'. Single-instance lock + SCRIPT_DIR-
    | relative resolve of 'RECOVER'. See 15.5.1. |
79 | 'scripts/common.sh' | ~40 | **Shared script primitives.** Centralizes formatted echo statements ('step
    | ', 'ok', 'warn', 'die') and the safe background 'run_with_timeout' execution wrapper. Sourced by all
    | orchestrator scripts across macOS and Linux. |
80 | 'scripts/setup-common.sh' | ~350 | **Shared installation logic.** Centralizes logic for verifying
    | Docker, installing Tailscale/wgcf, generating '.env', and prompting for auth keys. Sourced by 'setup
    | .sh' and 'setup-linux.sh'. |
81 | 'scripts/setup-linux.sh' | ~80 | **Linux sibling of 'setup.sh'.** Sources 'setup-common.sh'. Distro
    | detection (apt/dnf/pacman), installs Linux-specific dependencies, creates '.desktop' shortcuts. |
82 | 'scripts/Toggle-Gateway.bat' | ~300 | **Windows Toggle helper.** Moved from root to 'scripts/'. Helps
    | invoke the framework on Windows if applicable. |
83 | 'scripts/reinstall-host-tailscale.sh' | ~740 | **Nuke-and-reinstall tool for host Tailscale.**
    | Completely purges Tailscale, its settings, macOS Background Items ('.btm' database), and stale
    | System Extensions. Wipes ALL Login Items via 'sfltool reset-login-items' (requires sudo). Useful for
    | crisis recovery but destructive to other login items. |
84 | 'scripts/fix-ssh-delay.sh' | ~100 | **Fixes SSH connection delays.** One-shot script to address SSH
    | delays caused by DNS or routing issues, useful in crisis situations. |
85 | 'scripts/logger/' | ~100 | **Internal logger piped from 'scripts/routing-fix.sh' inside the 'warp'
    | container. Ported to Go from Python. Built via Docker multi-stage build. |
86 | 'black_list.txt' | 71 | Custom domains to block (ads, trackers, telemetry). Supports '$important'
    | modifier. |
87 | 'white_list.txt' | 222 | Domains to force-allow (YouTube, Apple services, etc.). Always wins over
    | blocks. |
88 | '.env' | ~14 | WARP WireGuard keys, Tailscale auth key, rule profile. **Contains secrets &
    | NULLEXIT_SEED.** |
89 | '.gateway-ip' | 1 | Static gateway Tailscale IP (fallback for dynamic resolution). |
90 | 'scripts/unlock-files.sh' | ~20 | **One-shot stale-permission fix.** Uses atomic rename ('cp' + 'mv')
    | to replace locked inodes (mode '000'/'0444' from old 'chmod' decisions) with fresh writable ones no
    | 'chmod' called. |
91 | 'scripts/crypto.sh' | ~50 | **Cryptographic integrity enforcement.** Uses 'NULLEXIT_SEED' from '.env'
    | to sign core bash scripts (HMAC-SHA256) and strictly verify them on start to prevent manipulation.
    | Pass '--sign' or '--verify'. |
92 | 'scripts/pf.conf' | 35 | **macOS native firewall ruleset.** Enforces the default-deny kill-switch,
    | allows local LAN / Tailscale traffic, and applies TCP MSS Clamping ('max-mss 1160') to prevent
    | WireGuard MTU fragmentation. |
93
94 ---
95

```

```

96 ## 4. toggle.sh Flow
97
98 ### State Detection
99 'is_gateway_active()' returns true if EITHER containers are running OR host DNS is not '1.1.1.1'. This
    dual check prevents the script from starting when it should stop (e.g., containers crashed but DNS
    is still hijacked).
100
101 ### START Path (gateway OFF ON)
102 1. **Reset DNS to 1.1.1.1** Prevents deadlocks during startup
103 2. **Disconnect host Tailscale** 'tailscale down' (prevents exit-node routing during container boot)
104 3. **Compile DNS rules** 'docker compose run --rm rule-compiler' (Go binary inside multi-stage Docker
    build)
105 4. **Boot Colima VM** (if not running) via 'colima_start_until_ready()', which returns as soon as Docker
    is reachable (~12s) instead of blocking on Colima's ~2-min post-provision hang (15.8.7). The
    networking mode is **gated on the LAN-P2P signal** ('.lan_p2p_detected', falling back to '
    TAILSCALE_ALLOW_LAN_P2P' in '.env'): a trusted home network requests '--network-address --network-
    mode bridged' (or 'shared' on WPA2-Enterprise), while the common AP-isolated / untrusted case boots
    plain fast user-mode networking ('colima start --memory 0.6 --vm-type vz').
106 - **Why Bridged Networking (when enabled)?** The '--network-mode bridged' and '--network-address'
    flags force the VM to receive its own IP directly from your local router's DHCP server, placing it
    on the same physical subnet as the host. This bypasses macOS network isolation and is what lets **
    external LAN devices** (phones, laptops on the same LAN), mDNS, and local service discovery reach
    across the container boundary. It requires the 'socket_vmnet' helper and is only worthwhile on a
    trusted LAN, which is why it is gated rather than always requested. **It is not* required for
    direct hostcontainer P2P** that path (the gateway its own Mac) works even in user-mode networking
    via the routing-fix '.host_ips' RETURN-rule carve-out; see 15.3.5 and 15.8.7.
107 - **Enterprise Wi-Fi Fallback & Dynamic Roaming:** 'toggle.sh' dynamically queries 'system_profiler
    SPAirPortDataType' on boot. If it detects a **WPA2-Enterprise** connection (802.1X), it forces
    Colima to fall back to '--network-mode shared'. This prevents aggressive network intrusion systems
    on corporate/university Wi-Fi switches from terminating the host's Wi-Fi connection due to "MAC
    spoofing" (detecting multiple MAC addresses originating from a single authenticated port). P2P
    connections are impossible on these networks anyway due to AP Client Isolation.
108 - **Dynamic Roam Downgrading:** 'toggle.sh' writes its selected network mode to '/tmp/
    nullexit_colima_mode.txt'. When roaming between networks (e.g., Home to University), 'recover.sh'
    actively checks this state against the new network's security requirement ('.lan_p2p_detected'). If
    a mismatch is detected (e.g., bridged Colima on an Enterprise network), 'recover.sh' automatically
    forces a full 'toggle.sh --restart' in the background to safely transition the Virtual Machine's
    network mode and protect the host from de-authentication.
109 5. **Configure VM swap** 400MB swap file inside the VM to prevent OOM
110 6. **Clean corrupted AdGuard config** Remove empty 'AdGuardHome.yaml'
111 7. **Start containers** 'docker compose up -d'
112 8. **Wait for gateway Tailscale** Poll 'tailscale status' for "offers exit node" (up to 60s, abort at 40
    consecutive NoState)
113 9. **Resolve gateway IP** From '.gateway_ip' or 'docker compose exec tailscale tailscale ip -4'
114 10. **Connect host to mesh** Verify 'tailscaled' is running (auto-start if needed), then 'tailscale up
    --reset --ssh=true --accept-dns=false --accept-routes=true --exit-node=' ('--accept-routes=true'
    must be explicit '--reset' reverts it to 'false' otherwise, silently preventing the default route
    from switching to 'utun')
115 11. **Pre-flight checks** [1/3] 'tailscale ping' gateway, [2/3] 'dig +tcp' AdGuard DNS, [3/3] WARP
    container internet
116 12. **If all pass** Set exit node + hijack DNS to gateway IP (single server, no fallback)
117 13. **If any fail** Enable SOCKS5 proxy + local DNS proxy as fallback
118 14. **Final DNS re-force** 'force_dns_to_gateway' called again after exit-node transition (tailscaled
    clobbers DNS briefly)
119 15. **Enable sleep prevention** 'caffeinate -i' in background to prevent idle sleep
120
121 ### STOP Path (gateway ON OFF)
122 1. Disconnect host Tailscale ('tailscale down')
123 2. 'docker compose down -t 5'
124 3. Reset DNS to 1.1.1.1
125 4. Stop local DNS proxy
126 5. **Stop sleep prevention** Kill caffeinate process
127 6. Full network state cleanup (proxies off, DNS cache flush, route flush, Wi-Fi power cycle)
128
129 ### Safety Mechanisms
130 - 'run_with_timeout()' Pure-bash watchdog for every system command (15s/120s)
131 - 'cleanup_handler()' Trap on ERR/INT/TERM/HUP restores DNS to 1.1.1.1, kills caffeinate, and tears down
    Tailscale
132 - 'force_dns_to_gateway()' Sets DNS and VERIFIES via 'networksetup -getdnsservers' (3 attempts with
    backoff)
133 - 'SUCCESS_RUN' flag Cleanup only runs if script didn't complete successfully
134 - 'start_sleep_prevention()' / 'stop_sleep_prevention()' Manages 'caffeinate -i' via PID file at '/tmp/
    nullexit-caffeinate.pid'
135
136 ---
137
138 ## 5. Routing & Firewall Architecture (Inside Container)
139
140 ### IP Rules ('ip rule show')
141 | Priority | Match | Table | Purpose |
142 |-----|-----|-----|-----|

```



```

143 | 99 | 'to 100.64.0.0/10' | 52 | CGNAT range Tailscale routes (injected by routing-fix) |
144 | 100 | 'from 172.18.0.2' | 200 | Container's own traffic (SOCKS5 proxy) tun0 |
145 | 101 | all | 51820 | Gluetun's WireGuard fwmark tun0 |
146
147 ### Table 200 (SOCKS5 proxy traffic)
148 - '162.159.192.1 via $DOCKER_GW dev eth0' WARP endpoint exception (prevents tunnel loop)
149 - 'default dev tun0' Everything else through WARP
150
151 ### iptables (TWO separate stacks!)
152 - **nftables backend** ('iptables') Used by Gluetun's 'post-rules.txt'. FORWARD policy DROP.
153 - **legacy backend** ('iptables-legacy') Used by Tailscale's 'ts-forward'. FORWARD policy ACCEPT.
154 - Both stacks apply to every packet. FORWARD RELATED,ESTABLISHED must exist in BOTH.
155
156 ### post-rules.txt (loaded by Gluetun)
157 '''
158 FORWARD: ACCEPT for tailscale0 in/out
159 INPUT: ACCEPT for tailscale0
160 NAT POSTROUTING: MASQUERADE on tun0
161 MANGLE: TCP MSS clamp to 1180
162 NAT PREROUTING: Redirect DNS (53) 5335 on tailscale0 and eth0
163 iptables: DROP FORWARD on tailscale0
164 '''
165
166 ---
167
168 ### 5.1 Tailscale Local Network Discovery (DERP Bypass)
169 By default, Gluetun's strict NAT swallows all of Tailscale's UDP hole-punching packets, forcing Tailscale
    to fall back to incredibly slow DERP relay servers (often adding 500ms+ latency).
170
171 To fix this, we use 'FIREWALL_OUTBOUND_SUBNETS=192.168.0.0/16,172.16.0.0/12,10.0.0.0/8' in 'docker-
    compose.yml' (on the 'warp' container). This creates 'ip rule' bypasses (Priority 99, Table 199)
    that route all packets destined for private IP ranges *outside* the WARP tunnel directly to the host
    's 'eth0'.
172 - **172.16.0.0/12** is especially critical because many modern Wi-Fi routers (and Docker itself) assign
    IPs in this block (e.g., '172.17.x.x').
173 - This bypass allows Tailscale's UDP packets to reach devices on the same Wi-Fi directly, establishing a
    fast peer-to-peer connection while the actual web traffic inside the tunnel remains fully routed
    through WARP.
174
175 ---
176
177 ### 5.2 Firewalling & Per-Device Access Control
178 Because every mesh device's traffic passes through the 'warp' container's network namespace, this
    namespace serves as the ultimate choke point for network-wide firewall rules.
179
180 **Where to Put Rules**
181 The right place to add firewall rules is 'scripts/routing-fix.sh'. It runs a 30-second loop inside the
    namespace and already handles re-injecting rules that Gluetun resets on reconnect. **Do not use '
    post-rules.txt' for dynamic rules**, as Gluetun flushes and resets it upon VPN reconnects.
182
183 **The Dual iptables Backend Constraint (CRITICAL)**
184 The container runs two simultaneous iptables backends: 'iptables' (for the nftables backend used by
    Gluetun) and 'iptables-legacy' (for Tailscale). Every rule **must** be added to both stacks, or it
    will silently fail for certain traffic.
185
186 **Avoiding Duplication**
187 Because 'scripts/routing-fix.sh' runs in a loop, you must always check for a rule's existence with '-C'
    before appending with '-A'. Otherwise, your rules will duplicate every 30 seconds.
188
189 **Per-Device Identification & Filtering**
190 Each mesh device has a stable '100.x.x.x' Tailscale IP, which is visible as the 'src' IP in the 'FORWARD'
    chain. This is how you identify devices for filtering.
191 * **Domain-level:** AdGuard Home (port 3000, credentials 'admin/nullexit') provides a REST API that
    supports per-client blocking rules based on their Tailscale IP.
192 * **IP-level:** Use 'iptables' in 'scripts/routing-fix.sh' (e.g., 'iptables -I FORWARD -s 100.x.x.x -d <
    blocked_ip> -j DROP').
193 * **Country-level:** Add 'ipset' to the 'routing-fix' apk install step in 'docker-compose.yml', and load
    CIDR ranges from 'ipdeny.com' into an ipset, then block that set in the 'FORWARD' chain. See 5.3 (
    Geo-IP Blocking) for the full configuration flow.
194
195 ### 5.3 Geo-IP Blocking
196 To block traffic to and from specific countries, the system dynamically downloads country-specific IP
    ranges from [ipdeny.com](http://www.ipdeny.com/ipblocks/data/countries/) and drops them via iptables
    'FORWARD' rules.
197
198 To add a new country to the blocklist:
199 1. Find the 2-letter ISO country code (e.g., 'cn' for China, 'ru' for Russia, 'il' for Israel, 'kp' for
    North Korea).
200 2. Open your '.env' file.
201 3. Update the 'BLOCKED_COUNTRIES' variable: e.g., 'BLOCKED_COUNTRIES="kp il cn ru"'.

```

```

202 4. Run 'toggle.sh' to restart the gateway and apply the new environment variables to the routing-fix
    container.
203
204 > [!NOTE]
205 > **Limitations of Geo-IP Blocking:** IP-based blocking is highly effective at blocking servers, direct
    apps, and raw infrastructure physically located and registered in a specific country (e.g., 'www.gov
    .il'). However, it will **not** block high-profile websites (e.g., 'jpost.com') that route their
    traffic through global Content Delivery Networks (CDNs) like Google Cloud, Cloudflare, or Fastly.
    Because the CDN's IP addresses are registered in the US or globally, the network traffic physically
    never routes to the blocked country, allowing it to bypass the Geo-IP firewall.
206
207 ## 6. macOS Quirks to Remember
208
209 1. **Colima SSH tunnel is TCP-only** UDP ports (like 5354/udp) are NOT accessible from host. Use 'dig +
    tcp' or the Python DNS proxy (UDPTCP converter).
210 2. **docker compose exec -T injects '\r'** Always 'tr -d '\r'' when capturing output for comparisons
    or 'networksetup' commands.
211 3. **brew services' can get stuck** Fix with 'sudo brew services restart tailscale'. Common state: '
    brew services list' shows 'started' but 'tailscale status' shows "Tailscale is stopped."
212 4. **No 'timeout' command** macOS lacks GNU 'timeout'. Use the 'run_with_timeout()' bash function.
213 5. **sudo -v' prompts even with NOPASSWD** Use 'sudo -n' (non-interactive) everywhere.
214 6. **DNS is scoped to en0** macOS scutil DNS resolver is scoped to en0 (Wi-Fi). DNS changes on other
    interfaces (like USB Ethernet) are ignored unless en0's service is also updated. The script always
    sets DNS on BOTH '$ACTIVE_SERVICE' and '$ENO_SERVICE'.
215 7. **tailscaled clobbers DNS during exit-node transition** 'force_dns_to_gateway' is called a second
    time at the end of the ENABLE branch to counteract this.
216 8. **tailscale up --reset** resets all unspecified flags to defaults Every '--reset' call MUST also
    pass '--accept-dns=false' explicitly or tailscaled re-enables DNS management.
217 9. **docker compose ps' CWD pitfall** 'docker compose' commands require a 'docker-compose.yml' in the
    current working directory. Use 'docker ps' for raw container listing from any CWD; 'docker compose
    ps' requires the project directory.
218
219 ---
220
221 ## 7. Environment Variables
222
223 ### '.env' File
224 | Variable | Purpose |
225 |-----|-----|
226 | 'WIREGUARD_PRIVATE_KEY' | WARP WireGuard private key (from 'wgcf generate') |
227 | 'WIREGUARD_PUBLIC_KEY' | WARP WireGuard public key |
228 | 'WIREGUARD_ADDRESSES' | WARP WireGuard address (e.g., '172.16.0.2/32') |
229 | 'TS_AUTHKEY' | Tailscale auth key for the container |
230 | 'NULLEXIT_SEED' | 256-bit seed for HMAC-SHA256 script integrity signing (generated by 'setup.sh') |
231 | 'GATEWAY_RULE_PROFILE' | Rule compilation tier: 'light', 'medium', 'heavy' |
232 | 'GATEWAY_BYPASS_PING' | (Optional) 'true' to proceed even if pre-flight checks fail |
233 | 'GATEWAY_USE_EXIT_NODE' | (Optional) 'false' to skip exit node, DNS-only mode |
234 | 'GATEWAY_HIJACK_HOST' | (Optional) 'false' to skip DNS hijacking on the host (VPN/adblocking for
    Tailscale peers only) |
235 | 'GATEWAY_MSS' | (Optional) TCP MSS clamp value (default 1120); 1180 for speed on healthy paths |
236 | 'WARP_FAIL_THRESHOLD' | (Optional) Consecutive 'warp=off' polls before auto-shutdown (default 6 = 30s;
    3 = 15s) |
237 | 'WARP_ENDPOINT_1' | (Optional) Override Cloudflare WARP WireGuard endpoint IP 1 (default
    '162.159.192.1') |
238 | 'WARP_ENDPOINT_2' | (Optional) Override Cloudflare WARP WireGuard endpoint IP 2 (default
    '162.159.193.1') |
239 | 'KILL_SWITCH' | (Optional) 'true' to enforce strict PF lock drops all host traffic if VPN fails.
    Breaks SSH if VPN dies. |
240 | 'STOP_COLIMA_ON_EXIT' | (Optional) 'true' to fully shut down the Colima VM on toggle-off (saves battery
    on dedicated hosts) |
241 | 'ADGUARD_USER' | (Optional) AdGuard Home web UI username (default: 'admin') |
242 | 'ADGUARD_PASSWORD' | (Optional) Shared password for AdGuard Home and Tor ControlPort (default: '
    nullexit') |
243 | 'BLOCKED_COUNTRIES' | Space-separated 2-letter ISO codes to block via ipdeny.com CIDR ranges (e.g. "kp
    il cn ru") |
244
245 ---
246
247 ## 8. Diagnostic Commands
248
249 ### Container Health
250 ''bash
251 docker ps --format '{{.Names}} {{.Status}}'
252 docker compose exec -T tailscale tailscale status
253 docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace
254 ''
255
256 ### SOCKS5 Proxy
257 ''bash
258 curl --socks5-hostname 127.0.0.1:1080 --max-time 5 -s https://www.cloudflare.com/cdn-cgi/trace
259 ''

```



```

260
261 ### AdGuard DNS
262 ```bash
263 dig +tcp @127.0.0.1 -p 5354 google.com +short +timeout=5
264 ```
265
266 ### Firewall & Routing
267 ```bash
268 # nftables FORWARD chain
269 docker compose exec -T warp iptables -L FORWARD -v --line-numbers
270 # Legacy FORWARD chain
271 docker compose exec -T warp iptables-legacy -L FORWARD -v --line-numbers
272 # IP rules + routing tables
273 docker compose exec -T warp ip rule show
274 docker compose exec -T warp ip route show table 199
275 docker compose exec -T warp ip route show table 200
276 ```
277
278 ### Host macOS
279 ```bash
280 tailscale status
281 brew services list | grep tailscale
282 networksetup -getdnsservers Wi-Fi
283 networksetup -getsocksfirewallproxy Wi-Fi
284 sysctl net.inet.ip.forwarding
285 ```
286
287 ---
288
289 # Part II Design Decisions & Threat Model
290
291 ## 9. Privacy Architecture & Threat Model
292
293 ### Layered Defense
294 The gateway stacks four layers targeting different attack surfaces:
295
296 - **Layer 1 ISP-Level Surveillance (WARP):** Your ISP sees only an encrypted WireGuard tunnel to a
297   Cloudflare IP. In the US, ISPs can legally sell your browsing metadata and are subject to secret
298   NSLs (National Security Letters). WARP removes their visibility entirely.
299 - **Layer 2 DNS Tracking (AdGuard Home):** DNS is the phone book of the internet. By default, queries go
300   to your ISP's resolver, giving them a complete log of your traffic. AdGuard intercepts all DNS from
301   mesh devices, blocks trackers, and resolves queries through the WARP tunnel.
302 - **Layer 3 Device Identity & IP Exposure (Tailscale):** On untrusted networks (coffee shops, public Wi-
303   Fi), your device's IP is exposed. Tailscale creates an encrypted WireGuard mesh between your devices
304   , routing all traffic to your gateway exit node.
305 - **Layer 4 Kernel-Level IP Blocking (ipset/iptables):** Sophisticated malware bypasses DNS entirely
306   with hardcoded IPs. The 'rule-compiler' fetches ~16,700 threat-intelligence IPs/CIDRs (Spamhaus,
307   Feodo Tracker, Emerging Threats, CINS) and enforces them as 'DROP' rules in the kernel 'FORWARD'
308   chain blocking both outbound C2 connections and inbound attack traffic.
309
310 ### The Documented Threat: Why This Matters
311 This architecture is calibrated against documented mass-surveillance programs revealed in the Snowden
312 disclosures:
313 - **PRISM:** Bulk collection of communication content from major tech providers.
314 - **UPSTREAM:** Tapping physical backbone fiber, collecting metadata and content in bulk.
315 - **XKeyscore:** Retroactive search of unencrypted traffic.
316 - **MUSCULAR:** Infiltration of unencrypted internal datacenter interconnect links.
317
318 Passive bulk collection is not paranoia it is a documented reality. By double-encrypting and routing
319 traffic, nullexit prevents your ISP from harvesting your metadata.
320
321 ### Trust Assumptions & Shifting
322 Every component shifts trust rather than eliminating it:
323 1. **Physical Device Integrity:** You trust your physical devices are not compromised.
324 2. **Cloudflare Integrity:** You trust Cloudflare not to correlate your WARP tunnel with exit traffic.
325 3. **Tailscale Integrity:** You trust Tailscale's coordination server not to MITM WireGuard keys.
326
327 The goal: no single provider has a complete picture. Your ISP sees an encrypted tunnel. WARP sees traffic
328 with no account identity. Tailscale sees node topology but not content.
329
330 ## 9.1 Exit Node IP Rotation (Design Decision)
331
332 A common question for privacy architectures is whether the system should aggressively rotate its public
333 exit IP (e.g., every 5 minutes, like a Tor circuit or a proxy rotator).
334
335 'nullexit' uses Cloudflare WARP via Gluetun. WARP provides **anonymity in a crowd** rather than
336 aggressive rotation. Thousands of users share the same Cloudflare Edge IP simultaneously, making
337 your traffic indistinguishable from the herd. The IP may occasionally rotate on its own (
338 historically surfaced as a 'ROTATE' event in 'output.log'), but it remains generally stable.
339
340
341 **Why aggressive IP rotation was intentionally excluded:**

```

```

325 1. **Broken TCP Connections:** Every rotation instantly drops all long-lived TCP sessions (SSH, large
    downloads, WebSockets, gaming, Zoom).
326 2. **CAPTCHA & 2FA Hell:** Modern web security (e.g., Cloudflare, Akamai) aggressively flags IP-hopping
    as bot behavior. Aggressive rotation guarantees the user will be bombarded with "Verify you are
    human" CAPTCHAs and "New Login Detected" 2FA emails constantly.
327 3. **WireGuard Architecture:** WireGuard is stateless and does not natively support IP hopping on the fly
    . To rotate an IP, the VPN client must actively drop the tunnel, request a new endpoint, and re-
    handshake. This introduces latency gaps where the kill-switch would repeatedly trigger and blackhole
    the user's internet.
328
329 **Verdict:** For a daily-driver secure gateway, shared-IP crowding (WARP) is superior to aggressive
    rotation. As an added benefit, Cloudflare WARP IPs are actually highly static (tied to the
    persistent WireGuard public key generated for your device identity). Because your IP stays static
    and is part of Cloudflare's own trusted network, it builds a high "trust score," virtually
    eliminating CAPTCHA loops while still hiding you within the massive crowd of other users in that
    data center. All mesh devices routing through your gateway will reliably share this exact same
    trusted IP.
330
331 *(Note: The only exception where aggressive IP rotation is genuinely required is for specialized
    offensive security taskslike high-volume web scraping or dark web researchwhere avoiding IP bans or
    active tracking overrides the need for TCP stability.)*
332
333 ### 9.2 Cloudflare WARP Privacy & Logging
334
335 Because 'nullexit' uses Cloudflare WARP (via Gluetun) as its upstream exit node, the system inherits
    Cloudflare's consumer privacy policy.
336
337 **What Cloudflare DOES NOT log (Strict No-Logs Policy):**
338 * **Browsing History:** They do not log the domains or URLs you visit.
339 * **Traffic Destinations:** They do not log the IP addresses of the servers you connect to.
340 * **Content:** They do not inspect or log the payload/data of your traffic.
341 * **Data Brokering:** They explicitly guarantee they will never sell or rent your data to advertisers.
342
343 **What Cloudflare DOES log (Minimal Telemetry):**
344 * **Data Transfer Volume:** Total bandwidth used (required to enforce 10GB/mo quotas if you are not using
    a paid WARP+ key).
345 * **Performance Metrics:** Anonymized connection speeds and latency to optimize their network routing.
346 * **Aggregate Volume:** Total traffic volumes to popular destinations in aggregate (e.g., "datacenter X
    sent 50TB to Netflix"), stripped of all user IDs.
347 * **Device ID (Public Key):** They store the persistent WireGuard public key generated by your container
    to authorize the connection.
348
349 **Privacy Verdict:**
350 Cloudflare WARP provides excellent **historical privacy**. Because they do not log your destinations,
    they cannot comply with historical subpoenas asking what websites you visited yesterday. However, it
    is **not absolute anonymity**. They still know your real ISP IP address while you are actively
    connected. For daily-driver privacy against ISPs, hackers, and corporate trackers, it is top-tier.
    For nation-state evasion, use Tor (see 11).
351
352 ### 9.3 OPSEC: Python Runtime Airgap (Isolated Mode)
353 'nullexit' explicitly relies on the host's native system Python 3 runtime for critical components like '
    scripts/dns-proxy.py'. Because this script runs with elevated privileges ('sudo -n') to bind to the
    privileged UDP/53 socket, modifying or polluting this Python environment is strictly forbidden.
354
355 **The Zero-Dependency Rule & Isolated Mode ('-I')**
356 You must **NEVER** install third-party 'pip' packages (e.g., 'requests', 'pyyaml') into the system Python
    environment that 'nullexit' uses.
357 - All Python scripts in this repository must be written using *only* the Python Standard Library ('socket
    ', 'urllib', 'json', 'ssl').
358 - Third-party packages introduce a massive supply-chain attack vector. A compromised 'pip' package
    installed globally could allow local malware to hook into the Python runtime and exploit the 'sudo'
    privilege of the DNS proxy to gain full root access.
359 - To enforce strict immunity against user-installed malicious packages, 'toggle.sh' explicitly invokes
    the proxy using **Python's Isolated Mode** ('python3 -I'). This flag forces the interpreter to
    completely ignore 'PYTHONPATH', 'PYTHONHOME', and the user's 'site-packages' directory, executing
    exclusively from the read-only, Apple-signed system framework.
360
361 ### 9.4 Recommended Upgrades
362
363 | Layer | Default | Upgraded |
364 |---|---|---|
365 | VPN Exit | Cloudflare WARP (US) | Mullvad (Swedish, proven no-logs, anonymous payment) |
366 | DNS Resolver | AdGuard via WARP | AdGuard via Mullvad DoH (Cloudflare-blind) |
367 | Coordination | Tailscale (US, OAuth) | Headscale (self-hosted, no third-party) |
368 | Auth | OAuth / persistent keys | Ephemeral auth keys (no identity persistence) |
369 | Content | WireGuard asymmetric | WireGuard + PSKs (coordination server blind) |
370
371 #### Mullvad (Replacing WARP)
372 Swedish-based. Police raided their offices in 2023 and left empty-handed no logs exist to seize.
    Accounts are random numbers. Payment by cash or Monero. Replace 'VPN_SERVICE_PROVIDER=custom' with '
    VPN_SERVICE_PROVIDER=mullvad' in 'docker-compose.yml'.

```

```

373 ##### Mullvad DoH Upstreams
374 Point AdGuard's upstream resolvers to 'https://adblock.dns.mullvad.net/dns-query'. Cloudflare WARP only
375 sees encrypted HTTPS to Mullvad's IPs completely blind to your DNS queries.
376
377 ##### Headscale
378 Self-hosted Tailscale coordination server. Eliminates Tailscale the company, keeping all node topology
379 under your control.
380
381 ##### Ephemeral Auth Keys
382 Generate in the Tailscale admin panel. Used once to authenticate; node disappears from the admin panel
383 after disconnect. Breaks persistent identity linkage.
384
385 ##### Pre-Shared Keys (PSKs)
386 Add a WireGuard PSK between peer nodes. Adds a symmetric encryption layer that Tailscale's coordination
387 server has no knowledge of, blinding it from reading transit content.
388
389 ### 9.5 Structural Limitations
390 - **Traffic Analysis / Metadata:** Passive fiber tapping (UPSTREAM) can observe timestamps, data volumes,
391 and IP ranges without reading content. Defeating traffic analysis requires mixing networks (Tor) or
392 continuous dummy packet padding both heavily degrade performance.
393 - **Targeted State-Level Adversaries:** Consumer-grade VPNs are not a complete shield against a nation-
394 state actively targeting you. This stack is designed to defeat bulk passive surveillance and
395 corporate dragnet collection.
396
397 ---
398
399 ## 10. Per-Device Access Control
400
401 Because every mesh device routes internet-bound traffic through the 'warp' container's 'FORWARD' chain,
402 they all appear with their stable '100.x.x.x' Tailscale IP as the 'src' address. This gives two
403 powerful enforcement surfaces:
404
405 ### IP & Port Level ('scripts/routing-fix.sh')
406 Write raw 'iptables' rules targeting specific devices. The 30-second idempotent loop auto-survives
407 Gluetun reconnects:
408
409 ```bash
410 # Block a specific device from a target subnet
411 iptables -I FORWARD -s 100.x.x.x -d 203.0.113.0/24 -j DROP
412
413 # Restrict a device to only HTTP/HTTPS
414 iptables -I FORWARD -s 100.x.x.x -p tcp ! --dport 3000 -j DROP
415 iptables -I FORWARD -s 100.x.x.x -p tcp ! --dport 443 -j DROP
416
417 # Block a device from internet egress entirely (mesh only)
418 iptables -I FORWARD -s 100.x.x.x -j DROP
419
420 # Time-based (e.g., cut off 10 PM 7 AM)
421 iptables -I FORWARD -s 100.x.x.x -m time --timestart 22:00 --timestop 07:00 -j DROP
422 ```
423
424 ### DNS Level (AdGuard REST API)
425 AdGuard Home supports per-client rules keyed by Tailscale IP:
426
427 ```bash
428 curl -X POST http://localhost:80/control/clients/add \
429   -H "Content-Type: application/json" \
430   -d '{
431     "name": "kids-ipad",
432     "ids": ["100.x.x.x"],
433     "blocked_services": ["youtube", "tiktok", "instagram"],
434     "safesearch_enabled": true
435   }'
436 ```
437
438 ---
439
440 ## 11. Tor & OPSEC Architecture
441
442 ### 11.1 Tor-over-WARP Topology
443
444 When extreme anonymity is required, integrating the Tor network into 'nullexit' provides a powerful Tor-
445 over-VPN topology:
446 * **The University/ISP** sees only an encrypted Cloudflare WireGuard tunnel, rendering them completely
447 blind to the fact that you are using Tor (effectively bypassing Enterprise firewall Tor blocks).
448 * **Cloudflare** sees an encrypted TCP stream heading to a Tor Guard Node. They cannot see your DNS
449 lookups, the destination website, or the content of your traffic.
450 * **The Tor Exit Node** sees your traffic, but not your real IP (only the Cloudflare IP).
451
452 ##### The "Transparent Proxy" Trap (What NOT to do)

```

440 It is tempting to build a system-wide "Transparent Tor Proxy" that forces all host traffic (e.g.,
 441 standard Chrome/Safari browsers, background iCloud syncs) through Tor automatically. ****This is a
 massive OPSEC trap and destroys anonymity.****

442 Modern operating systems and browsers will instantly leak your identity through background syncing (e.g.,
 443 Apple Mail, Google Drive) and browser fingerprinting (canvas fingerprints, WebRTC, screen
 444 resolution). The Tor network protects your *IP*, but if your payload contains identifying cookies or
 445 unique fingerprints, the Tor Exit Node (which could be run by malicious actors) will instantly tie
 446 your session to your real identity.

447 **#### The nullexit Implementation: Isolated Procedural Proxy**

448 To safely integrate Tor without triggering the transparent proxy trap, 'nullexit' implements an ****
 Isolated Tor SOCKS5 Proxy**** with ****Procedurally Generated Ports****:

- 449 1. ****Isolated Container:**** A lightweight 'tor' Docker container runs securely inside the 'warp' network
 namespace. It does not hijack system traffic.
- 450 2. ****Opt-In Usage:**** You must consciously configure a highly-hardened browser (like the official Tor
 Browser or a dedicated Firefox profile) to use the proxy. This prevents accidental background sync
 leaks.
- 451 3. ****Procedurally Generated Ports:**** Rather than hardcoding the standard proxy ports (like
 '127.0.0.1:9050' or '1080'), 'toggle.sh' randomizes all internal proxy ports on every boot into the
 ephemeral range (10000-65000) and silently writes them to '/tmp/nullexit-ports.env'. This completely
 defeats static fingerprinting by malware.
- 452 4. ****Kernel-Level Collision Avoidance:**** To prevent the randomizer from picking a port already in use by
 a root daemon or Apple service (which would crash the gateway), the script directly queries the
 macOS kernel socket table ('netstat -an -p tcp') to verify the port is absolutely free before
 assigning it.
- 453 5. ****The Honey-Port Tripwire:**** While procedural ports defeat static fingerprinting, advanced malware
 might attempt a full sequential port scan of 'localhost' to find the hidden proxy. To counter this,
 'toggle.sh' spawns a "Honey-Port" (a fake random port with a netcat listener attached). If any local
 application attempts to connect to the Honey-Port, it instantly logs a critical security alert to '
 output.log' and fires a macOS desktop notification, serving as an early-warning Intrusion Detection
 System (IDS) for local malware.

454 > ****Threat Model note:**** The Tor SOCKS proxy and Honey-Port Tripwire only bind to '127.0.0.1' (loopback).
 Port randomization and the Honey-Port only defeat blind, generic port-scanners they do NOT protect
 455 against a targeted local process that reads '/tmp/nullexit-ports.env', greps process memory, or
 456 runs 'lsuf -i'. The proper mitigation is a host-level loopback-filtering firewall (LuLu/Little
 Snitch). See 15.9 (Threat Model: Local Proxy Discovery) for the full analysis and the rogue-binary
 verification test.

457 **### 11.2 Hardening: Tor Bridges and obfs4 Pluggable Transports**

458 For users operating under severe Deep Packet Inspection (DPI), 'nullexit' supports configuring Tor to
 459 connect exclusively through private bridges using the 'obfs4' pluggable transport.

460 When 'TOR_USE_BRIDGES=true' is set:

- 461 - ****What it does:**** It hides the entry IP from being matched against the public Tor consensus list, and
 disguises the traffic's protocol fingerprint from the WARP exit provider (Cloudflare).
- 462 - ****What it does NOT do:**** It does ***not*** add any extra layers of anonymity beyond what Tor already
 provides. It is purely a censorship-circumvention and obfuscation mechanism.
- 463 - ****Limitations:**** 'obfs4' is highly effective against passive DPI, but it is not guaranteed to defeat
 active-probing-resistant detection by a highly sophisticated state-level adversary.

464 > ****Architecture Rule:**** The 'nullexit' gateway deliberately does NOT bundle, hardcode, or automatically
 465 fetch bridge lines. Bridge lines are highly sensitive and expire. Users must obtain their own bridge
 466 lines from 'https://bridges.torproject.org' and manually populate the 'tor-bridges.txt' file. If
 bridges are enabled but the file is missing or empty, the Tor container will intentionally crash and
 fail to start rather than silently falling back to public guard relays.

467 ****The Bootstrapping Paradox (Out-of-Band Key Exchange)****

468 Automating the bridge acquisition process (e.g., scripting the gateway to log into the Telegram '
 469 @GetBridgesBot' via MTProto API) introduces catastrophic OPSEC flaws:

- 470 1. ****Identity Leaks:**** Baking a Telegram session into the container explicitly links the "anonymous"
 gateway to a real-world physical phone number.
- 471 2. ****The Chicken & Egg Deadlock:**** In heavily censored regimes, Telegram itself is usually blocked. If
 the container requires Telegram to fetch bridges to bypass the firewall, it will fail because it
 cannot bypass the firewall to reach Telegram.
- 472 3. ****Bridge Exhaustion & CAPTCHAs:**** Automated scraping behaves identically to state-sponsored scrapers,
 triggering Tor's anti-bot CAPTCHAs which leads to silent proxy failures and exhausts the limited
 public bridge pool.

473 Instead, users should rely on a less secure layer to manually reach Telegram, obtain the bridges, and
 474 populate 'tor-bridges.txt'. This can be done via the standard 'nullexit' WARP tunnel, a mobile phone
 on cellular data, Mullvad VPN, or a secure remote VPS. ***(Note: We plan on fully integrating Mullvad
 and VPS architectures directly into 'nullexit' in the future, but have not had the time to build it
 yet).*** This intentionally "air-gaps" the cryptographic key exchange from the proxy infrastructure,
 leaving zero API tokens and zero network traces for an adversary to exploit.

475 **### 11.3 Tor Container Kill-Switch Inheritance & Fail-Closed Egress Verification**

476 **#### Context**

477 Unlike the host's SOCKS5 fallback path (whose kill-switch interaction is documented in 15.7), the 'tor'
 478 container's connection status and fail-closed behavior under VPN tunnel drops is not governed by

```

macOS 'pf' rules. Instead, it relies on network namespace inheritance inside Docker Compose.
475
476 ##### Mechanics
477 1. **Shared Network Namespace:** The 'tor' service is configured with 'network_mode: service:warp' in '
docker-compose.yml'. This shares the network namespace of the 'warp' (Gluegun) container.
478 2. **Inherited Egress Rules:** All socket communication within the namespace is bound to the same
networking stack. Gluegun manages this stack by redirecting traffic through 'tun0' (the WireGuard/
WARP tunnel) and applying 'iptables' rules that explicitly drop outbound traffic originating on '
eth0' (the physical/bridge interface), except for permitted local subnets or the VPN server's
endpoint.
479 3. **Tunnel Fail-Closed:** If the WARP connection drops or the 'tun0' interface is brought down, the
routing table entries for 'tun0' become invalid or disappear. Because the 'iptables' rules in the
shared namespace continue to drop any outgoing internet traffic attempting to exit via 'eth0', the '
tor' container cannot leak raw traffic to the host's underlying networks. Egress fails closed.
480
481 ##### Verification / Manual Test Case
482 To manually verify that the Tor container fails closed and does not leak traffic when the tunnel dies:
483
484 1. **Verify active path:** Ensure the gateway is active ('toggle.sh start') and fetch the Tor port ('
$TOR SOCKS_PORT' in '/tmp/nullexit-ports.env'). Verify Tor traffic routes correctly:
485 'bash
486 curl --socks5-hostname 127.0.0.1:$TOR SOCKS_PORT https://check.torproject.org/api/ip
487 '
488 *(This should output a Tor exit node IP).*
489
490 2. **Simulate tunnel drop:** Bring down the virtual interface inside the shared network namespace:
491 'bash
492 docker compose exec warp ip link set dev tun0 down
493 '
494
495 3. **Re-run the egress check:**
496 'bash
497 curl --socks5-hostname 127.0.0.1:$TOR SOCKS_PORT https://check.torproject.org/api/ip
498 '
499 * **Expected Result:** The request must hang and eventually time out, or immediately fail with a proxy
error (e.g., 'curl: (97) SOCKS5: connection failed' or 'curl: (7) Failed to connect').
500 * **Leaked Egress Check:** Check the host's Wi-Fi router or firewall logs. No outbound packets from
the Tor container should route directly over the bridge network to the internet.
501
502 ### 11.4 Transparent .onion Routing via RFC 2544 Subnet (198.18.0.0/15)
503
504 ##### Context
505 To enable seamless dark web browsing across the entire mesh, the gateway requires a mechanism to
dynamically map '.onion' domains to IP addresses that the host operating system and network layers
can route.
506
507 ##### IP Address Conflict & Solution
508 1. **Initial Conflict:** Initially, the Tor virtual address network was configured to map '.onion' sites
within the '10.192.0.0/10' range. However, the Colima Docker daemon dynamically allocated
'10.200.1.0/24' to the containers. Because this Docker subnet mathematically falls within the
'10.192.0.0/10' block, a routing loop occurred: all host packets destined for the Docker containers
on ports like '9050' or '9051' were intercepted by the transparent proxy NAT rules and dropped,
causing proxy failures.
509 2. **Tailscale Private IP Exclusion:** Shifting the subnet to another private range like '10.123.0.0/16'
resolved the port conflict, but led to connection timeouts. On macOS, Tailscale's Exit Node routing
actively excludes RFC 1918 private subnets (e.g. '10.0.0.0/8', '172.16.0.0/12', '192.168.0.0/16') to
maintain accessibility to local LAN resources (like printers/routers). Thus, packets targeting
'10.123.x.x' were dropped locally by Tailscale before reaching the gateway tunnel.
510 3. **The Benchmarking Subnet Solution:** To bypass these private IP exclusions without manual static
routing tables, Tor's virtual network was changed to '**198.18.0.0/15**' (reserved by RFC 2544 for
benchmarking). Since this is not a private RFC 1918 range, Tailscale exit-node routing automatically
encapsulates and routes all traffic destined for '198.18.0.0/15' through the tunnel to the gateway.
511 4. **Transparent NAT Redirection:** The 'routing-fix' sidecar applies NAT rules in the shared network
namespace:
512 'bash
513 iptables -t nat -I PREROUTING -d 198.18.0.0/15 -p tcp -j REDIRECT --to-ports 9040
514 '
515 This intercepts all incoming TCP traffic targeting the mapped '.onion' range and transparently proxies
it through Tor's transparent port ('9040').
516
517 ##### Exit Node Exclusions
518 The gateway supports the ability to exclude specific countries from being used as exit nodes (e.g. to
avoid certain jurisdictions or nodes with poor network latency). This is configured globally via the
'TOR_EXCLUDE_EXIT_NODES' environment variable in '.env', which gets dynamically appended to Tor's '
ExcludeExitNodes' and strictly enforced with 'StrictNodes 1'.
519
520 ---
521
522 ## 12. Censorship-Resistant Transport & Egress Compartmentalization
523
524 ### 12.1 Why this is needed

```

525 WARP can be blocked from deployment countries with strict internet controls. The most likely cause isn't that "encrypted traffic is blocked" in general it's that WireGuard has a recognizable handshake signature, and 'WIREGUARD_ENDPOINT_IP=162.159.192.1' on 'WIREGUARD_ENDPOINT_PORT=2408' is a fixed, easily-enumerable target (Cloudflare's known WARP range). That combination is exactly what IP/ASN-based and DPI-based blocking is good at catching.

526

527 **### 12.2 Important correction: Gluetun's 'SHADOWSOCKS=on' is the wrong direction**

528 Gluetun's built-in Shadowsocks option runs a Shadowsocks ****server**** inside the already-connected VPN tunnel, so other LAN devices can reach out through Gluetun's ***existing*** connection via the SS protocol. It is not a way to make Gluetun itself connect ****outbound**** through a Shadowsocks server. Gluetun only speaks OpenVPN or WireGuard as its actual transport. There is no 'VPN_TYPE=shadowsocks'. Confirmed against Gluetun's own maintainer/community discussion: people have asked for a "Shadowsocks-client" mode specifically to chain through Gluetun, and the standing answer is "run a separate Shadowsocks client container."

529

530 **### 12.3 Diagnose before building anything**

531 The right fix depends entirely on what's actually being blocked:

532 - Point Gluetun at a ****different provider/IP/ASN**** (any other Gluetun-supported WireGuard/OpenVPN provider, or a self-hosted WireGuard server) and test. If that gets through, the block is Cloudflare/WARP-specific, not a general WireGuard block ****no new component needed****, just swap 'VPN_SERVICE_PROVIDER' / the endpoint.

533 - If WireGuard fails regardless of endpoint, the block is protocol-level (DPI fingerprinting the handshake) proceed to obfuscation.

534

535 **### 12.4 Path A: Swap WARP for a different Gluetun-native transport**

536 Simplest fix, only applies if 12.3 shows the block is Cloudflare-specific. Change 'VPN_SERVICE_PROVIDER' / 'VPN_TYPE' / endpoint in 'docker-compose.yml' and in the now-centralized 'WARP_ENDPOINT_*' values in '.env'. Nothing else in the stack needs to change.

537

538 **### 12.5 Path B: Add an obfuscation hop**

539 Needed if WireGuard itself is being fingerprinted/blocked. Two ways to slot it in:

540

541 ****B1 Wrap (recommended):**** Run a Shadowsocks (or obfs4/v2ray) client as a new sidecar container. Point Gluetun's 'WIREGUARD_ENDPOINT_IP' at '127.0.0.1:<local-port>' instead of Cloudflare directly; the sidecar relays that traffic to a Shadowsocks server you run yourself abroad. Keeps Gluetun's kill-switch, healthchecks, and the entire rest of the stack (Tailscale, AdGuard, ipset, 'routing-fix.sh') untouched, since none of them know the transport changed underneath 'warp'.

542

543 ****B2 Replace:**** Swap the 'warp' service entirely for a Shadowsocks-client + 'tun2socks' container that owns the network namespace instead of Gluetun. More invasive loses Gluetun's built-in kill-switch/healthcheck, which would need to be rebuilt by hand (see Section 12.9 for the pluggable architecture to support this natively).

544

545 ****Recommendation: B1.**** Smaller surface area, preserves the self-healing architecture already built and documented.

546

547 **### 12.6 Fingerprinting caveat**

548 Plain Shadowsocks can still be caught by active-probing DPI in more aggressive censorship environments. If plain SS gets blocked too, the next step up is an obfuscation plugin ('v2ray-plugin', 'Cloak') or a newer protocol designed against active probing (VLESS+Reality, Hysteria2). Test plain SS first don't over-build before confirming it's needed.

549

550 **### 12.7 AmneziaWG (The High-Speed Alternative)**

551 If you want to maintain the bare-metal speeds of WireGuard while bypassing DPI (e.g. in Egypt or Russia), the best alternative to Shadowsocks/V2Ray is AmneziaWG. AmneziaWG is a fork of WireGuard that pads the handshake packets with randomized "junk" data, entirely destroying the 148-byte DPI signature that firewalls look for.

552 - ****Pros:**** Because it runs entirely as UDP without TCP-over-TCP overhead, it completely avoids "TCP meltdown" latency spikes associated with Shadowsocks/V2Ray wrappers.

553 - ****Cons:**** You cannot use Cloudflare WARP. Furthermore, it requires completely ripping Gluetun out of the 'nullexit' stack and building a custom Docker container, meaning you lose Gluetun's built-in kill-switches and health-check logic.

554

555 **### 12.8 Infrastructure Costs & Privacy**

556 To use either Shadowsocks (Path B) or AmneziaWG, you cannot use the free Cloudflare WARP infrastructure, because Cloudflare servers only speak standard WireGuard. The software for both custom protocols is 100% free and open-source, but you must host the remote server infrastructure yourself.

557 - ****Free Tier (Oracle Cloud):**** You can host your remote server on Oracle Cloud's "Always Free" tier for \$0. However, this routes your highly sensitive, censorship-evading traffic directly through Oracle's massive corporate data broker. If privacy is the core objective, handing all your metadata to Oracle defeats the purpose.

558 - ****Paid Tier (Hetzner / Mullvad):**** The recommended path is to rent a standard ~\$4/month Virtual Private Server (VPS) in a free country (e.g., Hetzner in Germany, subject to strict EU GDPR privacy laws) and self-host the server. Alternatively, Mullvad VPN (5/month) provides native v2ray/Shadowsocks bridges built into their network, allowing you to bypass censorship with a strict zero-logs policy. It is highly recommended to pay with untrackable payment methods to maintain complete anonymity, which these providers typically support without requiring local residency.

559

560 **### 12.9 The Future: Egress Compartmentalization (Pluggable Architecture)**

561 To natively support invasive replacement paths like ****Path B2**** or ****AmneziaWG**** (Section 12.7) without breaking the core 'nullexit' routing and monitoring logic, the architecture must be refactored into


```

a modular, "pluggable" design.
562
563 1. **The Egress Plugin System:** Currently, WARP-specific logic (like 'cdn-cgi/trace' health checks and
hardcoded interface names) is scattered across 'toggle.sh' and 'scripts/routing-fix.sh'. This should
be abstracted into a 'scripts/plugins/' directory, where each egress method ('warp.sh', 'v2ray.sh')
implements standardized functions: 'get_egress_interface()', 'check_health()', and 'get_bypass_ips
()''.
564
565 2. **Dynamic Docker Compose:** Based on an 'EGRESS_TYPE' variable in '.env', dynamic compose file
generation would spin up only the required egress containers (e.g., skipping the 'warp' container
when 'EGRESS_TYPE=v2ray' is set).
566
567 3. **Agnostic Routing & Watchers:** 'routing-fix.sh' would dynamically read the 'EGRESS_INTERFACE' from
the active plugin to apply 'iptables' rules, and the background watchers in 'toggle.sh' would become
an agnostic 'Egress Watcher' that invokes 'check_health()' periodically.
568
569 This compartmentalization transforms 'nullexit' into a universal, censorship-resistant gateway where the
entire egress layer can be swapped by changing a single '.env' variable and running './toggle.sh --
restart'.
570
571 ---
572
573 ## 13. Roaming & Recovery Design: How nullexit Mimics Commercial VPNs
574
575 The 'nullexit' roaming/network recovery subsystem replicates the exact engineering mechanics used by
commercial, premium VPN clients (like Mullvad or NordVPN) to transition across Wi-Fi networks safely
and seamlessly. This is the design overview; the specific bugs encountered and fixed along the way
live in the Incident Log (15.5 Roaming/Sleep-Wake, 15.7 Tailscale P2P/SNAT).
576
577 ### The 4-Step Transition Cycle
578
579 When your Mac disconnects from one Wi-Fi and associates with another, 'nullexit' coordinates the
following events:
580
581 1. **Firewall Lock (Kill-Switch):** The macOS Packet Filter (PF) anchor 'com.apple/nullexit' remains
locked on the physical interface ('en0'), blocking all direct outbound IPv4 and IPv6 traffic. This
guarantees that your real ISP IP or unencrypted DNS requests **never leak** onto the new network
while the connection is rebuilding.
582
583 2. **System Event Interception:** A launchd LaunchAgent ('watcher.sh') listens to the macOS System
Configuration framework for the 'State:/Network/Global/IPv4' key changes, spawning 'recover.sh --
post-wake' in under 500ms when a change is detected.
584
585 3. **Bypass Routing:** The script dynamically resolves the new physical network's IP gateway (e.g.
'192.168.137.1'), cleans up stale bypass routes from the previous network, and adds static bypass
routes pointing to Cloudflare's WARP endpoints and the Tailscale control plane directly via the new
gateway. This allows the VPN client to negotiate its handshake outside of the tunnel.
586
587 4. **Hole-Punching / NAT Re-negotiation:** The script runs a target check on the 'warp' container's
tunnel. If the WireGuard connection is wedged by the network switch, it force-recreates the
container to establish a fresh NAT binding to Cloudflare, restoring connection in ~15 seconds.
588
589 ---
590
591 ## 14. Deployment, Packaging & Known Limitations
592
593 ### 14.1 Packaging nullexit as a macOS Application (.dmg)
594
595 nullexit can be packaged into a native '.app' bundle, but this introduces nested virtualization
requirements.
596
597 nullexit uses **Colima** (Apple Virtualization Framework 'vz') to run a Linux VM hosting Docker.
Installing the '.app' inside a virtualized macOS environment (Parallels, cloud Mac) means running a
Linux VM inside a macOS VM requiring hardware-level **nested virtualization**.
598
599 #### Hardware Support
600
601 - **Supported:** Apple Silicon M3/M4 (macOS Sequoia 15+), Intel Macs with VMX passthrough.
602
603 - **Unsupported:** M1, M2, A18 Pro. Colima crashes silently; the terminal ('setup.sh') surfaces crash
logs.
604
605 #### Build Steps
606
607 ```bash
608
609 # 1. Verify nested virtualization support
610 sysctl -a | grep hv_nested_virt_supported # expect: 1
611
612 # 2. Compile the AppleScript launcher
613 osacompile -o "Nullexit.app" "Toggle Gateway.applescript"
614
615 # 3. Embed scripts into app resources
616 mkdir -p "Nullexit.app/Contents/Resources/scripts"
617 cp toggle.sh setup.sh recover.sh docker-compose.yml "Nullexit.app/Contents/Resources/"
618
619 # 4. Package into DMG
620 hdiutil create -volname "Nullexit Installer" -srcfolder "./Nullexit.app" -ov -format UDZO Nullexit.dmg
621
622 # 5. Bypass Gatekeeper (unsigned app)
623 xattr -cr /Applications/Nullexit.app

```



```

614 '''
615
616 Without a paid Apple Developer certificate ($99/yr), users see an "App is damaged" Gatekeeper warning and
    need to run step 5.
617
618 ### 14.2 Moonlight/Sunshine + Tailscale Race Condition
619
620 When streaming from a remote Windows host running Sunshine over a Tailscale mesh (e.g., while the client
    is on a cellular hotspot), a race condition can occur on boot:
621
622 1. Both Tailscale and Sunshine are set to start automatically on boot.
623 2. Tailscale takes a few seconds to fully initialize its '100.x.x.x' virtual adapter and establish a
    connection.
624 3. **Sunshine starts too fast.** It launches before Tailscale is fully ready. Since Sunshine only binds
    to network interfaces that are active at the exact moment of startup, it misses the Tailscale
    adapter entirely.
625 4. Moonlight clients (even when configured with the correct Tailscale IP) will fail to connect because
    Sunshine isn't listening on that interface.
626
627 **The "Magic Toggle" Symptom:**
628 If the user manually enables or disables Wi-Fi on the host PC, it triggers a global Windows network
    change event. Sunshine listens for these events, re-enumerates all network adapters, and says, "Oh,
    there's a Tailscale adapter here!" and finally binds to it. Suddenly, Moonlight can connect.
629
630 **The Permanent Fix:**
631 On the Windows host, open 'services.msc', locate the **Sunshine Service**, and change its Startup type
    from **Automatic** to **Automatic (Delayed Start)**. This forces Windows to wait a minute or two
    after booting before launching Sunshine, ensuring Tailscale's virtual adapter is fully initialized
    first.
632
633 ### 14.3 Chrome Remote Desktop Performance (UDP NAT)
634
635 ##### Observation (July 11, 2026)
636 When the user connects to their host Mac using Chrome Remote Desktop (CRD) while 'nullexit' is active,
    the connection suffers from massive latency, lag, and degraded video quality. AdGuard Home's '
    blocked.log' shows zero blocked DNS requests for Google, WebRTC, or 'chromoting' endpoints,
    indicating this is not a DNS sinkhole issue. *(See also 15.11 for the separate CRD-on-remote-device
    connection-failure quirk.)*
637
638 ##### Root Cause
639 This is a fundamental limitation of tunneling all host traffic through a strict commercial NAT (
    Cloudflare WARP).
640 1. Chrome Remote Desktop attempts to use **STUN (UDP hole-punching)** to establish a lightning-fast,
    direct peer-to-peer WebRTC connection between the client device and the host Mac.
641 2. Because all non-Tailscale traffic is forcefully routed through the 'warp' container, the STUN packets
    exit the network from a Cloudflare IP.
642 3. Cloudflare's enterprise NAT infrastructure strictly drops unsolicited inbound UDP packets. The UDP
    hole-punch fails.
643 4. Because a direct P2P connection cannot be established, CRD falls back to **Relay Mode** (TURN),
    bouncing all video data back and forth through Google's cloud servers instead of sending it directly
    over the local or mesh network. This relaying introduces massive latency.
644
645 ##### Workaround / Fix
646 This lag is the accepted "tax" for maintaining absolute cryptographic anonymity; the only way to restore
    CRD speed would be to leak its UDP traffic outside the tunnel (violating zero-trust).
647 To achieve low-latency remote access, users should bypass CRD entirely and use macOS's built-in **Screen
    Sharing (VNC)** directly over the Tailscale IP. Tailscale's sophisticated custom DERP/WireGuard
    architecture successfully UDP hole-punches through strict NATs, providing a direct, encrypted, high-
    speed connection without relying on external cloud relays.
648
649 ### 14.4 Dynamic IP Fetch vs. MagicDNS
650
651 ##### Observation
652 The 'nullexit' gateway uses a dynamically fetched IP address (cached in '.gateway_ip') to configure host
    routing and the 'pf' kill-switch, rather than utilizing Tailscale's user-friendly MagicDNS hostname
    (e.g., 'nullexit-gateway').
653
654 ##### Why MagicDNS is Not Used
655 1. **The "Chicken and Egg" Bootstrapping Problem:**
656 macOS's Packet Filter ('pf') and low-level routing commands ('route add') operate strictly at Layer 3
    and require raw IP addresses. If 'toggle.sh' or 'recover.sh' attempted to use 'nullexit-gateway',
    the host Mac would need to perform a DNS lookup to resolve it. However, the very first step of the
    gateway's initialization is to completely hijack the host's DNS and point it *at the gateway itself
    *. The Mac cannot resolve the gateway's MagicDNS name if it needs the gateway's IP to know where to
    send the DNS query!
657 2. **Dynamic Self-Healing (Avoiding Stale Cache):**
658 To solve the bootstrapping problem without causing bugs if the node is deleted and re-authenticated on
    the Tailscale Admin Console, 'toggle.sh' explicitly runs 'docker compose exec ... tailscale ip -4'
    during boot to fetch the absolute freshest IP dynamically. It saves this to '.gateway_ip' so that
    fast-roaming scripts like 'recover.sh' can instantly retrieve the routing target without incurring
    the latency of querying Docker.

```

```

659
660 ### 14.5 Battery & Power Measurement Quirks
661
662 When trying to optimize this framework (or any daemon) for battery life, developers often look for a way
to programmatically measure the exact Wattage consumed by a specific process (e.g., "How many Watts
is 'toggle.sh' using?").
663
664 **This is a hardware impossibility on macOS and Linux.**
665
666 ##### Why Activity Monitor is Lying to You
667 Your machine's logic board only has physical power sensors for the *entire* CPU package, the GPU, and the
RAM. It knows the CPU is pulling 5W total, but it has no physical hardware capability to know
whether Docker is using 3W and Chrome is using 2W.
668
669 The "Energy Impact" score you see in macOS Activity Monitor is a **synthetic, heuristic score**
calculated by 'powerd'. It penalizes apps for waking up the CPU (idle wakes), doing disk I/O, or
keeping the screen awake. It is a relative score, not actual Wattage.
670
671 ##### The Proper A/B Testing Methodology
672 If you want to truly know how many Watt-hours or mAh your background daemons ('routing-fix.sh', the DNS/
WARP watchers, etc.) are costing you, you must perform a hardware-level A/B drain test:
673
674 1. Unplug the laptop, run the gateway, and note your exact battery mAh using:
675     ``bash
676     ioreg -l | grep "AppleRawCurrentCapacity"
677     ``
678 2. Leave the laptop idle for exactly 1 hour.
679 3. Run the command again to see exactly how many mAh were drained.
680 4. Turn the gateway off and repeat the test for another hour.
681
682 The delta in mAh drained between the two tests is the true, exact cost of running the software. When
optimizing polling loops (e.g., changing 'sleep 5' to 'sleep 30'), this is the only reliable way to
measure the actual battery life returned to the user.
683
684 ### 14.6 Host Lockdown Mode (Why It's Impossible on macOS)
685
686 A common privacy feature request is a "Host Lockdown Mode" (Mode 3): A mode where the Docker exit node
remains perfectly functional for remote devices, but the host machine itself is completely blocked
from accessing the internet (to prevent even encrypted host traffic from leaking metadata).
687
688 **Why this is impossible on macOS:**
689 On Linux (e.g., a Raspberry Pi), Docker runs natively. The host's traffic traverses the 'OUTPUT' iptables
chain, while Docker traffic traverses the 'FORWARD' chain. We could easily drop all 'OUTPUT'
traffic to kill the host's internet, and Docker would continue routing traffic perfectly.
690
691 However, Docker on macOS runs inside a Linux Virtual Machine (via 'qemu' or Apple 'Virtualization.
framework'). This VM runs as a standard macOS user-space application. If you configure the macOS
firewall ('pf') to drop all host outbound traffic, it will indiscriminately drop the VM application's
traffic as well, instantly killing the Cloudflare WARP tunnel and the exit node.
692
693 Writing complex macOS 'pf' rules to "allow the VM app, allow Cloudflare WARP IPs, allow Tailscale DERP
IPs, but block everything else" is highly fragile and heavily discouraged. Since the current 'toggle
.sh' default mode already fully encrypts the host's traffic via the WARP tunnel, the host is already
protected from ISP leaks.
694
695 ---
696
697 # Part III Incident Log & Resolved Issues
698
699 This is the single, deduplicated log of every incident, bug, and resolved issue. Entries are organized
into subsystem groups (15.115.12). Each entry follows a **Symptom / Root Cause / Fix** shape where
applicable; packet-level analyses are preserved as clearly-labeled **Deep dive** blocks. The terse
dated changelog lives in 17; this section holds the full post-mortems.
700
701 ## 15. Incident Log
702
703 ### 15.1 Exit-Node Return Path & SOCKS5 Failover
704
705 ##### 15.1.1 Exit Node Return Path ('rx 0') & The SOCKS5 Failover RESOLVED
706 **Symptom:** Gateway showed 'tx 936 rx 0' transmitted but never received return traffic. For days,
Tailscale's exit node refused to return traffic back to the client, leading to the assumption that
Tailscale's userspace forwarding was bypassing 'iptables' and ignoring the WARP 'tun0' interface. In
a desperate attempt to fix this, a **SOCKS5 proxy** ('scripts/socks5-proxy.py') was introduced as a
replacement.
707
708 **Root Cause:** Tailscale's userspace forwarding was *not* the problem. 'docker-compose.yml' included '
FIREWALL_OUTBOUND_SUBNETS=100.64.0.0/10'. Gluetun created a strict routing rule ('ip rule add to
100.64.0.0/10 lookup 199', priority 99) that forced all Tailscale CGNAT traffic to bypass the VPN
via the unencrypted Docker bridge ('eth0'). Return packets were sent to the macOS host instead of
back through 'tailscale0', blackholing all return packets back to the client.
709

```

```

710 **Fix:** Removed 'FIREWALL_OUTBOUND_SUBNETS' entirely, immediately restoring the Tailscale exit node. '
       scripts/routing-fix.sh' now injects 'lookup 52', and return packets correctly flow back into '
       tailscale0'. Instead of removing the SOCKS5 proxy, we repurposed it into a **bulletproof failover**
       for when restrictive Wi-Fi networks block Tailscale UDP ports.
711
712 Architecture after the fix:
713 ```
714 PRIMARY (Exit Node):
715 DNS/TCP/UDP: Apps  Tailscale Exit Node  gateway container  tun0  WARP  internet
716
717 FAILOVER (SOCKS5 - if Tailscale is blocked by local Wi-Fi):
718 DNS: Browser 127.0.0.1:53 Python proxy TCP:5354 AdGuard WARP
719 TCP: Apps SOCKS5:1080 gateway container tun0 WARP internet
720 Mesh: SSH/ping 100.x.x.x utun5 WireGuard peers (independent of proxy)
721 ```
722
723 **Deep dive: Exit Node Return Path Analysis (June 26, 2026).**
724 This preserves the detailed packet-level analysis that led to identifying and fixing the 'rx 0' bug.
725
726 *The exit node was probably never going to work in this container topology.* Here's the packet path
       traced by reading 'ip rule show' and the routing tables live:
727
728 **Outbound (Phone-1 internet):**
729 1. Phone-1 sends packet to gateway via Tailscale mesh
730 2. Gateway's tailscale0 (kernel TUN, 'TS_USERSPACE=false') decrypts packet enters FORWARD chain
731 3. Packet has src='100.87.42.87' (Phone-1), dst=some internet IP
732 4. FORWARD chain (nftables): Rule 1 matches ('-i tailscale0 -j ACCEPT')
733 5. Routing decision for the forwarded packet. Which ip rule matches?
734   - Rule 98: 'to 172.18.0.0/16' no (dst is internet)
735   - Rule 99: 'to 100.64.0.0/10' no (dst is internet)
736   - Rule 100: 'from 172.18.0.2' **NO** (src is '100.87.42.87', not the container IP!)
737   - Rule 101: 'not fwmark 0xca6c lookup 51820' **YES** (no fwmark on forwarded packets)
738 6. Table 51820: 'default dev tun0' packet goes out through WARP
739 7. NAT POSTROUTING: 'MASQUERADE on tun0' src becomes WARP IP
740 8. Packet exits through WARP to internet (in theory)
741
742 **Return (internet Phone-1):**
743 1. Return packet arrives on tun0, conntrack de-MASQUERADES dst back to '100.87.42.87'
744 2. Routing decision for dst='100.87.42.87':
745   - Rule 99: 'to 100.64.0.0/10 lookup 199' **YES**
746 3. Table 199: '100.64.0.0/10 via 172.18.0.1 dev eth0' **sends it to the Docker gateway!**
747 4. **This is wrong.** The packet should go to tailscale0 (to be re-encrypted and sent to Phone-1), but
       table 199 sends it out eth0 to the Docker bridge host.
748
749 **Table 199 is the smoking gun.** It has one route: '100.64.0.0/10 via 172.18.0.1 dev eth0'. This sends
       ALL Tailscale CGNAT return traffic out the Docker bridge to the host not back through tailscale0.
       The return packet leaves the container's network namespace entirely, hits the host's routing stack,
       and gets dropped because the host has no idea what to do with a raw packet for '100.87.42.87'.
750
751 *Why table 199 exists and why it's wrong for exit node traffic.* Table 199 + the CGNAT rule (pref 99) was
       designed so that the **container itself** can reach other Tailscale peers (e.g., for mesh
       management, DERP relay). For that use case, routing CGNAT traffic via the Docker bridge to the host
       (which has its own tailscaled) makes sense. But for **forwarded exit node traffic**, the return
       packet needs to go back through tailscale0 inside the container not out to the host. The CGNAT rule
       intercepts the return packet before it can be routed back to tailscale0 through the main table or
       Tailscale's own routing (table 52, which is at pref 5270).
752
753 *The SOCKS5 path works because it dodges all of this.* SOCKS5 proxy creates connections **from the
       container's own IP' ('172.18.0.2'). So:
754 - ip rule 100 ('from 172.18.0.2 lookup 200') matches
755 - Table 200 has 'default dev tun0'
756 - Return traffic comes back to '172.18.0.2' on tun0, conntrack de-NATs, proxy sends response to client
757
758 No FORWARD chain involved. No CGNAT rule involved. No table 199. It just works.
759
760 *Summary of findings:*
761
762 | Finding | Status | Evidence |
763 |-----|-----|-----|
764 | Table 199 CGNAT route hijacks exit node return traffic | **Fixed** | Changed 'lookup 199' to 'lookup
       52' in 'scripts/routing-fix.sh'. Return packets now route via tailscale0. |
765 | FORWARD RELATED, ESTABLISHED missing | **Fixed** | Installed 'iptables' via apk in 'docker-compose.yml'.
       Rule now injects successfully. |
766 | DOCKER_GW variable parsing error | **Fixed** | Added 'head -1' in 'scripts/routing-fix.sh'. |
767 | 'FIREWALL_OUTBOUND_SUBNETS' was the root cause | **Fixed** | Removed from 'docker-compose.yml'. Gluetun
       no longer hijacks CGNAT routing. |
768 | Host tailscaled error state from aggressive 'tailscale down' | **Mitigated** | Toggle script now
       detects and auto-restarts. |
769
770 #### 15.1.2 Container Default Route Bypasses WARP

```

```

771 **Problem:** Inside the WARP container's network namespace, the main routing table's default route was
    via 'eth0' (Docker bridge), not 'tun0' (WARP tunnel). While gluetun uses policy routing (rule 101
    table 51820 'tun0') for most traffic, traffic originating from the container's own IP
    ('172.18.0.2') hits rule 100 ('from 172.18.0.2 lookup 200'), whose default was also via 'eth0'. This
    caused the SOCKS5 proxy's outbound connections to bypass WARP.
772
773 **Fix:** Updated the 'routing-fix' container to:
774 1. Change the main table's default route to 'default dev tun0'
775 2. Change table 200's default route to 'default dev tun0'
776 3. Add a specific route for the WARP WireGuard endpoint ('162.159.192.1') through 'eth0' via the Docker
    gateway in table 200, preventing a tunnel loop
777 4. Re-assert all routes every 30 seconds (gluetun may reset them on health checks)
778 5. Dynamically detect the Docker subnet from 'ip route show dev eth0' instead of hardcoding
    '172.18.0.0/16'
779
780 ##### 15.1.3 Hardcoded Docker Subnet in Routing Fix
781 **Problem:** The 'routing-fix' container hardcoded '172.18.0.0/16' and '172.18.0.1' for the Docker bridge
    subnet and gateway. If Docker's bridge network changed (after 'docker network prune', Colima
    restart, or on a different machine), these routes would be wrong.
782
783 **Fix:** Detection is now dynamic using 'ip route show':
784 - 'DOCKER_NET=$(ip route show dev eth0 | grep -v default | head -1 | awk '{print $1}')"
785 - 'DOCKER_GW=$(ip route show default | awk '{print $3}')"
786
787 This extracts the actual subnet and gateway directly from the routing table, working with any subnet size
    ('/16', '/24', '/20', etc.).
788
789 ### 15.2 Routing Loops & Subnet Collisions
790
791 ##### 15.2.1 The Infinite Recursive Routing Loop (Hotspot Paradox)
792 **Problem:** A critical network topology crash occurs if the host Mac connects to a Mobile Hotspot (e.g.,
    a Windows PC or Phone) that is also connected to the Tailscale mesh and has "Use Exit Node"
    enabled. Because the Mac is hosting the Exit Node, it routes its internet traffic to the Hotspot.
    The Hotspot intercepts the Mac's traffic on its way to the cellular tower and routes it back to
    the Exit Node (the Mac) via Tailscale. The Mac receives it, tries to send it out again, and the
    Hotspot intercepts it again. This creates an infinite, recursive encryption loop that instantly
    destroys network bandwidth and drops all connections.
793
794 **Fix:** This is a physical routing paradox that cannot be resolved in software. The upstream physical
    router providing internet to the Mac must be allowed to route its traffic directly to the
    physical WAN interface. The user must manually disable "Use Exit Node" on the upstream device
    providing the hotspot.
795
796 **Advanced Workaround:** If the upstream hotspot is a Windows machine and the user explicitly wants it to
    remain connected to the Exit Node, a permanent static route can be injected into the Windows
    routing kernel to forcefully bypass the Tailscale WFP interception for WARP packets. By binding the
    route to the physical adapter's Interface Index (IF), it becomes a permanent, roaming-aware bypass.
797
798 On Windows (Admin Command Prompt):
799 ```cmd
800 route print (find Interface List, note the IF number for the active Wi-Fi adapter, e.g. 15)
801 route -p add 162.159.192.1 mask 255.255.255.255 0.0.0.0 IF 15
802 route -p add 162.159.193.1 mask 255.255.255.255 0.0.0.0 IF 15
803 ```
804
805 On Linux (Root Terminal):
806 ```bash
807 ip route add 162.159.192.1 dev wlan0
808 ip route add 162.159.193.1 dev wlan0
809 ```
810
811 On macOS (Root Terminal):
812 ```bash
813 route add -host 162.159.192.1 -interface en0
814 route add -host 162.159.193.1 -interface en0
815 ```
816
817 *(Note: If the upstream router is an iPhone or Android device, you cannot inject static routes without
    jailbreak/root. You must simply disable "Use Exit Node" in the mobile Tailscale app).*
818
819 This punches a tiny hole straight through the paradox, letting the Mac's WARP packets escape to the
    physical internet while keeping the rest of the upstream router's traffic securely trapped in the
    Tailscale Exit Node.
820
821 ##### 15.2.2 Infinite Egress Routing Loops & Subnet Takeover Paradoxes
822 **Problem:** Two distinct network loops can break host egress connectivity entirely when the exit node is
    active:
823
824 1. **The Recursive Host-VM Tunnel Loop:**
825 - **Mechanism:** The host Mac's default route points to the Tailscale exit node ('utun*'). The exit
    node container sends its encrypted tunnel packets (destined for the WARP endpoint '162.159.192.1')

```

```

back to the host via the VM NAT. Without a static route exception on the host Mac, the host Mac
routes those packets right back into the exit node ('utun*'), creating an infinite loop that
blackouts all host network egress.
826 - **ARP-Scoping Pitfall:** Simply binding the bypass route to the interface ('route add -host
162.159.192.1 -interface en0') fails when the default route is deleted or redirected to 'utun*'.
Since '162.159.192.1' is not local to the physical link, macOS fails to resolve it via ARP, dropping
all tunnel packets.
827 - **Fix:** Dynamically resolve the active physical gateway IP (e.g. '172.17.0.1') on startup, and add
the static host routes for the WARP endpoints via the gateway IP directly ('route add -host
162.159.192.1 <gateway_ip>'). Additionally, because standalone 'tailscaled' on macOS does not
automatically modify the system default route via the CLI, we manually redirect the default gateway
to 'utun*' using 'setup_exit_node_routing'.
828
829 2. **The Docker Compose Subnet Takeover Collision:**
830 - **Mechanism:** To avoid collisions with the host's campus network, Colima's default bridge ('docker.
bip') is moved (e.g. to '10.200.0.1/24'). However, because '172.17.0.0/16' is now free inside the VM
, Docker Compose's IPAM dynamically takes it over for the project's custom network ('
nullexit_default'). Because the containers receive IPs on '172.17.0.0/16', the host Mac cannot send
local peer discovery packets (e.g., Tailscale 'magicsock' packets on '172.17.0.2:41641') directly.
The host routing table sends them out 'en0' to the physical Wi-Fi, returning 'host is down' or 'no
route to host'.
831 - **Fix:** Explicitly configure a custom default network subnet '10.200.1.0/24' in 'docker-compose.yml
' to prevent Docker Compose from ever reclaiming the conflicting '172.17/172.18' ranges.
832
833 > See also 15.2.1 (The Hotspot Paradox) for the related infinite loop that occurs when the physical
upstream router is also a Tailscale exit-node client.
834
835 ##### 15.2.3 Docker Default Bridge vs Host Wi-Fi Subnet Collision (The 172.17.0.0/16 Subnet Overlap)
836 **Context:** When attempting to ping a local Windows client ('172.17.22.187') or the default gateway
('172.17.0.1') directly over Wi-Fi, the Mac host returns 'No route to host' (displaying a 'REJECT' /
'!' flag in the routing table). Tailscale on the client also drops offline shortly after starting
when using the exit node, failing to establish direct P2P connections and falling back to slow DERP
relays.
837
838 **Root Cause & Subnet Overlap Mechanism:**
839 1. **Docker Default Subnet:** By default, the Docker daemon initializes its default bridge network ('
docker0') on the **172.17.0.0/16** IP range.
840 2. **Subnet Conflict:** The host's physical Wi-Fi network happens to be assigned the exact same
**172.17.0.0/16** range by the local network router.
841 3. **Routing Clash:** Colima launches a Linux VM on the Mac host to run the Docker engine. Because Docker
inside that VM creates 'docker0' and claims the '172.17.0.0/16' subnet, any packet sent to '172.17.
x.x' from within the containers (such as Tailscale local discovery) is captured by the VM's internal
virtual bridge interface (which is down/empty) and is blackholed. On the Mac host, macOS
dynamically marks routes as 'REJECT' ('!') when local ARP resolution fails, causing local pings to
immediately abort with 'No route to host'.
842 4. **Tailscale Relay Saturation:** Because local peer-to-peer discovery is blackholed, Tailscale falls
back to public DERP relay servers (e.g. 'relay "tor"' in Toronto). When exit-node routing is enabled
, all client web traffic is routed through the relay. Public relays impose strict rate limits and
throttle high-volume traffic, resulting in packet drops. This saturation drops Tailscale's control
packets, causing the connection to time out and showing the client as offline.
843
844 **Fix:**
845 The canonical script for this is 'scripts/fix-docker-bridge-collision.sh'. **One command applies the
entire fix:**
846
847 bash
848 bash scripts/fix-docker-bridge-collision.sh          # apply
849 bash scripts/fix-docker-bridge-collision.sh --dry    # preview changes without applying
850 bash
851
852 It auto-detects the host's actual Wi-Fi subnet (no more guessing whether the campus DHCP handed out '172.
x', '10.x', or '192.168.x'), picks a non-conflicting 'docker.bip' from a small candidate set (
defaulting to '172.26.0.1/24' if no overlap is matched), backs up '~/.colima/default/colima.yaml'
with an ISO-8601 timestamp, edits the file **in place** (idempotent replaces an existing 'docker.
bip' if present, appends under an existing 'docker:' block, or creates a fresh block), restarts
Colima, rebinds Wi-Fi DHCP, verifies the new default route is no longer Docker's bridge, and re-runs
both './toggle.sh' and 'scripts/diagnose-host-leak.sh' to print the post-fix verdict.
853
854 For reference, the underlying manual fix is: edit the Colima configuration file ('~/.colima/default/
colima.yaml') on your Mac to define:
855 yaml
856 docker:
857   bip: 172.26.0.1/24    # any /24 that does NOT overlap your Wi-Fi subnet
858 
859 Then, restart Colima to apply the new subnet inside the VM:
860 bash
861 colima restart
862 
863 This forces Docker to use '172.26.x.x' for its default bridge, freeing the physical '172.17.x.x' range
and letting pings and direct Tailscale peer-to-peer connections flow normally.
864

```

```

865 ##### 15.2.4 July 4, 2026: The "Network Unreachable" Host Leak & Post-Wake Container Destructions
866 **Symptom:**
867 1. Running 'docker compose config' with dummy keys corrupted the user's '.env' file by appending invalid
   WireGuard keys. This caused the 'warp' container to become unhealthy on boot, which triggered a full
   teardown of the gateway by 'toggle.sh'.
868 2. After fixing '.env', 'toggle.sh' successfully booted the gateway, but the host's internet started
   bypassing the tunnel entirely (a massive host leak). The Cloudflare trace returned 'warp=off' and
   exposed the host's physical ISP IP.
869 3. Running 'recover.sh --post-wake' would violently force-recreate all gateway containers and drop the
   host's internet connection.
870
871 **Root Cause:**
872 - **Bug 1 (The Host Leak):** In 'toggle.sh', the logic to find the Tailscale 'utun' interface used '
   ifconfig | grep -B4 "inet 100."'. Due to the 4 lines of before-context, this regex was accidentally
   capturing the interface *above* the actual Tailscale interface (e.g. grabbing 'utun4' instead of '
   utun5'). Because 'utun4' had no IP address, the subsequent 'sudo route add -net 0.0.0.0/1 -interface
   utun4' silently failed with "Network is unreachable", leaving the host's default route exposed.
873 - **Bug 2 (The Post-Wake Destructions):** The health check in 'recover.sh --post-wake' relied on 'docker
   compose exec -T warp curl -sf https://www.cloudflare.com/cdn-cgi/trace'. However, the newer 'qmcgaw/
   gluetun' Alpine-based Docker image no longer ships with 'curl' pre-installed. The health check
   failed with 'executable file not found in $PATH', tricking 'recover.sh' into thinking the tunnel was
   completely dead and needed to be forcefully recreated via 'docker compose up -d --force-recreate'.
874 - **Bug 3 (Restart Flags):** Previous refactors incorrectly permitted 'toggle.sh restart' instead of
   strictly enforcing 'toggle.sh --restart'.
875
876 **Resolution:**
877 - Cleaned the '.env' file of duplicate dummy entries.
878 - Replaced the brittle 'grep -B4' interface parsing in 'toggle.sh' with a strict AWK parser: 'awk '/^[a-
   z0-9]+/{iface=$1} /inet 100\./{print iface; exit}' | tr -d ':''. This mathematically isolates the
   exact interface possessing the Tailscale '100.x.x.x' IP address, preventing the '0.0.0.0/1' route
   from silently failing.
879 - Changed the health check in 'recover.sh' to use 'wget -q0 - --timeout=3' which is natively installed in
   the Gluetun image. This stopped the unnecessary destructive recreations of the containers during
   post-wake events.
880 - Reverted the 'restart' alias in 'toggle.sh' to strictly require '--restart'.
881
882 ##### 15.2.5 July 4-5, 2026: Tailscale Data-Plane Loop (The DERP Relay Deadlock)
883 **Symptom:**
884 After bypassing the Tailscale control plane ('192.200.0.0/16') to fix the "offline" mesh status, the Mac
   appeared online. However, external peer devices (like the user's phone on cellular) could not
   successfully ping or SSH into the Mac. 'tailscale netcheck' reported 'Nearest DERP: unknown (no
   response to latency probes)'.
885
886 **Root Cause:**
887 While bypassing the control plane kept the daemon connected to the coordination servers, Tailscale's
   actual peer-to-peer data plane relies on STUN (for NAT traversal) and DERP relays. These relays span
   ~80 dynamically allocated IP addresses across multiple cloud providers. Because we were forcing
   '0.0.0.0/1' through the 'utun*' exit node, the Mac's own outbound connection attempts to DERP relays
   were being routed back into the tunnel. To prevent an infinite loop, Tailscale's daemon dropped its
   own packets when it saw them returning on the TUN interface. This effectively left the daemon deaf
   and blind to peer connections.
888
889 **Resolution:**
890 - Modified 'add_warp_bypass_routes' in 'scripts/common.sh' to execute a live API fetch ('curl -s https://
   login.tailscale.com/derpmap/default') during startup.
891 - The script parses out all active DERP relay IPv4 addresses (~80 IPs) and adds a physical host bypass
   route for every single one of them.
892 - These IPs are written to a temporary file ('/tmp/nullexit-derp-ips.txt') so they can be cleanly un-
   routed by 'remove_warp_bypass_routes' when the gateway shuts down. Peer connectivity (ping, SSH,
   SFTP) was fully restored.
893
894 ##### 15.2.6 July 5, 2026: Hardcoded WARP Endpoints in docker-compose
895 **Symptom:**
896 The bash scripts allowed overriding the default Cloudflare WARP IP endpoints via '.env' variables ('
   WARP_ENDPOINT_1' and 'WARP_ENDPOINT_2') to establish bypass routes. However, 'docker-compose.yml'
   statically hardcoded '162.159.192.1' for the 'warp' container's 'WIREGUARD_ENDPOINT_IP'.
897
898 **Root Cause & Risk:**
899 If a user set a custom endpoint in '.env', the script would successfully bypass the custom IP on the host
   level. However, Gluetun would still attempt to connect to the hardcoded '162.159.192.1'. Since
   '162.159.192.1' was no longer explicitly bypassed, its packets would be sucked into the 'utun*' exit
   node, causing an infinite WireGuard routing loop that instantly breaks the gateway.
900
901 **Resolution:**
902 Modified 'docker-compose.yml' to dynamically read the environment variable via '${WARP_ENDPOINT_1
   :-162.159.192.1}', ensuring both the host routing scripts and the container use the exact same
   endpoint.
903
904 ##### 15.2.7 Incident Post-Mortem: WARP Watcher Race vs Post-Wake (July 10, 2026)
905 **Symptom:** After switching Wi-Fi networks, the host IP appeared as the raw ISP IP ('132.x.x.x') instead
   of a Cloudflare/WARP IP. The gateway appeared to complete post-wake recovery successfully in the

```



```

logs, but nuclear shutdown fired immediately after and killed everything.
906
907 **Root Cause:** A fatal race condition between two concurrent processes:
908
909 1. **Wi-Fi roam** 'watcher.sh' fires 'recover.sh --post-wake'
910 2. **Post-wake** finds the 'warp' container missing/dead calls 'docker compose up -d --force-recreate'
   (~35 seconds)
911 3. **In parallel**, the WARP Watcher (running as a child of 'toggle.sh') polls the warp container every 5
   s during failures
912 4. While the container is being recreated it returns 'warp=off' for those 35 seconds 7 consecutive
   failures **WARP SHUTDOWN fires**, calling nuclear 'recover.sh'
913 5. Nuclear 'recover.sh' tears down Tailscale, resets DNS to 1.1.1.1, stops all containers **gateway dead
   , IP exposed**
914 6. The post-wake 'docker compose up' finishes at ~35s mark, container healthy but the gateway is already
   gone
915
916 The evidence in 'output.log':
917 ```
918 [2026-07-10T06:07:27Z] NET: ... bash recover.sh --post-wake
919 [2026-07-10T06:07:47Z] WARP DOWN cdn-cgi/trace reports warp=off watcher starts counting
920 ...
921 add net 0.0.0.0: gateway utun5 post-wake successfully re-routes at 02:08:22
922 ...
923 [2026-07-10T06:09:04Z] WARP SHUTDOWN 6 consecutive failures watcher fires nuclear
924 ```
925
926 **Fix (July 10, 2026):** Added a **WARP Watcher inhibit marker** ('/tmp/nullexit-warp-inhibit.marker'):
927 - 'recover.sh --post-wake' writes the marker at startup **before** any container operations
928 - The WARP Watcher loop checks for the marker at the top of every iteration; if present it resets '
   consec_off=0' and sleeps 5s (skips the poll entirely)
929 - 'recover.sh' removes the marker at the very end (on both success and failure via 'trap cleanup_inhibit
   EXIT')
930 - This guarantees the watcher never counts failures during the intentional container-down window of a
   post-wake force-recreate
931
932 The marker file path is hardcoded to '/tmp/nullexit-warp-inhibit.marker' in both 'toggle.sh' and 'recover
   .sh'.
933
934 #### 15.2.8 Incident Post-Mortem: Concurrency Collision between Nuclear Teardown and Post-Wake (July 10,
   2026)
935 **Symptom:** When the background WARP Watcher triggers a nuclear shutdown (due to a real or simulated
   tunnel failure), the gateway is completely torn down. However, the network configuration changes
   made during the teardown trigger a 'State:/Network/Global/IPv4' network change event. The
   LaunchAgent-based network watcher ('watcher.sh') intercepts this event and concurrently spawns '
   recover.sh --post-wake' to heal/restore the gateway. This causes 'docker compose down' (from nuclear
   teardown) and 'docker compose up' (from post-wake) to run in parallel, resulting in container
   errors like 'dependency failed to start: container warp exited (0)' and the gateway becoming wedged.
936
937 **Root Cause:** A lack of coordination/locking between the lifecycle control scripts. While 'toggle.sh'
   used '/tmp/nullexit-toggle.lock' to prevent concurrent 'toggle.sh' runs, 'recover.sh' (both nuclear
   and '--post-wake' modes) did not utilize or check any lock files.
938
939 **Fix (July 10, 2026):** Implemented a shared lifecycle lock file ('/tmp/nullexit-toggle.lock') across
   both scripts:
940 - **'recover.sh' (all modes):** Checks '/tmp/nullexit-toggle.lock' at startup. If the lock is held by
   another running lifecycle script ('toggle.sh' or 'recover.sh'):
941 - In '--post-wake' mode, it logs a skip message to 'output.log' and exits cleanly ('exit 0'). This
   safely prevents the auto-recovery agent from recreating containers during a teardown.
942 - In nuclear recovery mode, it exits with an error.
943 - **'recover.sh' Lock Cleanup:** Cleans up the lock file upon completion using 'trap cleanup_recover EXIT
   ' (which checks if the lock file belongs to the current PID).
944 - **'toggle.sh' update:** The lock check in 'toggle.sh' was updated to look for both 'toggle.sh' and '
   recover.sh' in the running processes to enforce proper exclusion.
945
946 #### 15.2.9 Incident Post-Mortem: Infinite Auto-Recovery Loop after WARP Watcher Nuclear Shutdown (July
   10, 2026)
947 **Symptom:** When the background WARP Watcher triggered a nuclear shutdown (due to a tunnel outage), it
   successfully executed 'recover.sh' (nuclear). However, immediately after the teardown completed, the
   system started spawning 'recover.sh --post-wake' and rebuilding the containers again. This created
   an infinite loop of tearing down and auto-restarting the gateway, keeping the host's internet route
   permanently wedged.
948
949 **Root Cause:** An active-state marker file leak. The LaunchAgent-based network watcher ('watcher.sh')
   only fires 'recover.sh --post-wake' when '/tmp/nullexit-gateway-active.marker' is present on disk.
   While 'toggle.sh' (STOP path) correctly deleted this marker, the nuclear 'recover.sh' script did not
   . When the WARP Watcher ran 'recover.sh' (nuclear), the containers were stopped but the active-state
   marker was left on disk. The network changes from the teardown then triggered 'watcher.sh', which
   saw the marker, believed the gateway was still supposed to be active, and triggered '--post-wake' to
   start it back up.
950

```


951 ****Fix (July 10, 2026):**** Modified the start of 'recover.sh' to explicitly delete '/tmp/nullexit-gateway-active.marker' if 'POST_WAKE' is 'false' (nuclear mode). This ensures that any subsequent network configuration changes made during the teardown are correctly ignored by 'watcher.sh' because the marker file is gone.

952

953 **### 15.3 MTU, Fragmentation & Throughput**

954

955 **#### 15.3.1 Network Bufferbloat & MTU Optimizations (Double-VPN UDP Crash)**

956 ****Symptom:**** When running aggressive UDP bandwidth tests (such as macOS 'networkQuality', which uses QUIC/HTTP3), the 'nullexit' tunnel completely crashes or exhibits severe lag (6,000ms+ bufferbloat).

957

958 ****Root Cause:**** The bug is a cascading MTU mismatch. Tailscale generates a UDP packet. Cloudflare WARP (also WireGuard) receives the packet, wraps it in another layer of encryption, pushing the packet size beyond the standard 1500 byte limit. Unlike TCP, UDP packets cannot be resized dynamically via MSS negotiation, resulting in massive IP packet fragmentation. The Apple Virtualization framework ('vz'), which Colima uses for bridged networking, silently drops heavily fragmented UDP packets under high load. This blackholes the entire UDP upload stream, instantly dropping the connection.

959

960 ****Fix (Partial):**** To prevent MTU fragmentation for standard web traffic, ****TCP MSS Clamping**** is strictly enforced via the macOS 'pf' firewall. By adding 'scrub out all max-mss 1160' to 'scripts/pf.conf', macOS unilaterally shrinks all outgoing TCP headers to fit comfortably inside the double-encrypted tunnel, bypassing Path MTU Discovery blackholes. 'TS_DEBUG_MTU=1200' was also added to 'docker-compose.yml' to minimize internal fragmentation.

961 ****Unresolved:**** The only way to solve the UDP crash bug natively is to artificially cap the maximum tunnel bandwidth using SQM ('fq_codel'). Because a static bandwidth cap would artificially throttle high-speed networks, 'nullexit' leaves the bandwidth uncapped, optimizing perfectly for TCP while accepting UDP fragmentation under extreme edge-case load tests.

962 ***Why not Dynamic SQM (Aurate)?*** Implementing an EMA/PID-controlled aurate daemon (like 'sqm-aurate') requires a constant 100ms polling loop to measure the kernel packet queue, which severely degrades laptop battery life by preventing deep C-state CPU idling. Furthermore, macOS's native firewall ('dumynet') only supports 'fq_codel' and does not support the newer Linux 'CAKE' algorithm, which is the only algorithm that offers kernel-native, event-driven 'aurate-ingress' (zero polling penalty). Therefore, dynamic SQM is computationally hostile to battery-powered macOS hosts.

963

964 **#### 15.3.2 Double-Tunneling MSS Clamping (Stalling Bug)**

965 ****Problem:**** Traffic double-wrapped through Tailscale (WireGuard) and WARP (WireGuard) suffered 120 bytes of MTU overhead (60 + 60). The default MSS clamp of 1180 was still slightly too large, causing large packets to fragment or drop, which resulted in mysterious web page stalls on strict-MSS endpoints.

966

967 ****Fix:**** Lowered the TCP MSS clamp in 'post-rules.txt' to '1120' to guarantee double-tunneled payloads fit safely within standard internet MTUs.

968

969 **#### 15.3.3 Wi-Fi Edge Packet Loss via QUIC (UDP) Fragmentation**

970 ****Problem:**** Applications that heavily utilize HTTP/3 (QUIC) over UDP such as Facebook, Instagram, and Google services experience massive stuttering and stalled image loading when the client device is far away from the router (weak Wi-Fi signal). The issue did not occur when close to the router.

971

972 ****Root Cause:**** The 'nullexit' gateway uses a double-encrypted tunnel (Tailscale + WARP). To prevent packets from fragmenting when entering these tunnels, a firewall rule in 'post-rules.txt' uses '--set-mss 1120' to safely clamp TCP packets. However, because QUIC runs entirely over UDP, it bypasses the TCP MSS handshake completely. QUIC blasts maximum-MTU (1280 byte) UDP packets. The inner 'tailscale0' interface (MTU 1200) forces these large UDP packets to fragment into two pieces. When the client is on the edge of Wi-Fi range (where 5-10% packet loss is common), losing *either* UDP fragment destroys the *entire* image frame. This exponential amplification of packet loss stalled QUIC streams.

973

974 ****Fix & Trade-offs:**** Appended an 'iptables' rule to 'post-rules.txt' that explicitly blocks 'UDP --dport 443' (QUIC). When modern web apps detect QUIC is blocked, they instantly fall back to HTTP/2 (TCP 443). Because TCP is routed through the MSS clamp, the packets are resized *before* they leave, entirely eliminating IP fragmentation and restoring high-speed loading over weak Wi-Fi.

975 ***Consequences:*** This adds a minor latency penalty (TCP 3-way handshake) compared to QUIC's zero-RTT connection. It may also force WebRTC video calls (Google Meet/Discord) to fall back to TCP TURN servers, slightly inflating real-time video latency. However, in a constrained double-tunnel mesh, the absolute stability gained from eliminating fragmentation vastly outweighs these trade-offs.

976

977 **#### 15.3.4 Cellular Networks & PMTUD Blackholes (The 1280 Byte Limit)**

978 ****Experiment:**** Conducted a live packet sweep from the PC-1 host to Phone-1 connected via a cellular network + Tailscale DERP relay using 'ping -D -s <size>'.

979

980 ****Result:****

981 - Packets with a payload of '1252' bytes (+ 28 bytes IP/ICMP headers = **1280 bytes total**) succeeded perfectly with ~60ms latency.

982 - Packets with a payload of '1253' bytes (**1281 bytes total**) and above resulted in a silent timeout.

983

984 ****Analysis:**** The cellular network (or the Tailscale DERP relay) enforces a strict MTU of 1280 bytes. Critically, it acts as a "PMTUD Blackhole" it silently drops packets larger than 1280 bytes instead of returning an ICMP "Fragmentation Needed" warning. If the exit node tries to send a standard 1500-byte internet packet to the phone over this link, it vanishes, causing web pages to stall permanently.

985

```

986 **Conclusion:** This empirically validates the absolute necessity of the TCP MSS clamping rule in 'post-
rules.txt'. By artificially clamping the MSS to '1120' (or '1180'), we ensure the TCP payload +
headers + double-WireGuard overhead never exceeds the strict 1280-byte ceiling of the cellular mesh
link.
987
988 #### 15.3.5 Low Gateway Throughput (DERP Relay & MTU Fragmentation) Fixed
989 **Problem:** The host Mac's internet throughput through the gateway dropped significantly (e.g., from 34
Mbps raw to 2.1Mbps via gateway). This was caused by a combination of two bottlenecks:
990
991 **1. Tailscale DERP Relay Bottleneck:**
992 Because the host Mac and the Docker container operate behind the same public IP, standard hole-punching
for Tailscale P2P failed, forcing traffic through Tailscale's NYC DERP relay. Normally, 'routing-fix
.sh' explicitly drops container Tailscale UDP packets destined for local subnets (when '
TAILSCALE_ALLOW_LAN_P2P=false' in '.env' or auto-detected). We nuke these P2P connections on purpose
to prevent the severe macOS 'gvproxy' SNAT endpoint poisoning issue we faced earlier (see 15.7 SNAT
Endpoint Poisoning). However, this strict policy unintentionally blocked direct communication
between the container and the Mac host itself via the Docker bridge network.
993
994 **Fix:** Tailscale's control plane registers the Mac host using its physical IP (e.g., '172.17.52.223'),
not the Docker bridge IP. If this physical IP falls within the dropped local subnets (like
'172.16.0.0/12'), the P2P handshake is dropped. To solve this natively, 'scripts/common.sh' now
features a 'write_host_ips()' function that actively grabs all physical IPv4 addresses on the Mac
and writes them to '.host_ips'. 'routing-fix.sh' dynamically reads this file every 30 seconds and
injects priority 'RETURN' rules for those exact physical IPs. This guarantees direct container-to-
host P2P over the local bridge (bypassing DERP) while continuing to block external LAN devices (like
phones) to prevent the 15.7 SNAT poisoning bug.
995
996 **2. MTU Double-Encapsulation Fragmentation:**
997 The host's Tailscale interface ('utun5') defaulted to an MTU of 1280. The WARP tunnel inside the
container also uses an MTU of 1280. When a full 1280-byte Tailscale packet from the host entered the
container and was encapsulated by WARP (adding ~60 bytes of overhead), the resulting packet
exceeded 1280 bytes, leading to severe fragmentation and performance degradation.
998
999 **Fix:** In 'toggle.sh's 'setup_exit_node_routing' function, the host's Tailscale 'utun' interface MTU
is explicitly lowered to '1200' (configurable via 'HOST_MTU' in '.env'). This ensures that host-
originated packets can comfortably fit inside the container's WARP tunnel encapsulation without
fragmenting.
1000
1001 > **Design Note Is direct hostcontainer P2P a Docker bug, or are we "exploiting a VM/host relationship
"?**
1002 > Neither. It is expected, well-defined behavior, and the RETURN-rule carve-out is a *defensive de-
restriction*, not an exploit. The reasoning:
1003 > - **On native Linux**, the container and host share one kernel; the Docker bridge is a first-class
local interface and hostcontainer is *trivially* direct. Nobody calls that an exploit. Our RETURN
rules simply reproduce that same local reachability on macOS.
1004 > - **The RETURN rules add nothing new; they *stop blocking* something legitimate.** 'routing-fix.sh'
broadly DROPS containerLAN Tailscale UDP to prevent the real 15.7 'gvproxy' SNAT endpoint-poisoning
bug caused by *external* LAN peers (e.g. a phone on another AP). Tailscale's control plane registers
the Mac host by its *physical* LAN IP (e.g. '172.17.52.223'), which falls inside those DROPPed
ranges so the host, the one peer that is literally the same physical machine, became collateral
damage. The '.host_ips' RETURN rules are the minimal, precise exception that re-permits exactly that
same-machine path and nothing else.
1005 > - **The genuinely VM-specific wart is 'gvproxy' SNAT poisoning (15.7), and we route *around* it, not
through a hole in it.** Keeping hostcontainer traffic on the Docker bridge (direct) avoids letting
it get NATed through the userspace 'gvproxy' endpoint rewriter that confuses Tailscale's peer-
endpoint learning. That is the opposite of exploiting the VM boundary it is respecting it.
1006 > - **What genuinely *doesn't* work on a VM host is UDP hole-punching to arbitrary *external* peers** (
see 15.13). That is a real limitation of userspace VM networking, and it is why remote peers fall
back to DERP unless bridged mode is enabled on a trusted LAN. Direct hostcontainer P2P is the one
case that works regardless because both endpoints live on the same physical machine, no hole-
punching is needed at all. It was verified 'direct 172.17.52.223' even in plain user-mode networking
(no '--network-address').
1007 >
1008 > In short: the DROP is the deliberate policy, the RETURN is a surgical exception for same-machine
traffic, and the whole thing is standard bridge networking not a Docker bug and not an exploit.
1009
1010 ### 15.4 DNS Interception & Proxy
1011
1012 #### 15.4.1 DNS Proxy: TCP Wire Format Mismatch (socat / dnsmasq)
1013 **Problem:** When the Tailscale data plane is unavailable (DERP relay only), the script falls back to a
local DNS proxy that forwards queries from host UDP:53 to Docker's TCP:5354 (AdGuard). The 'socat'
and 'dnsmasq' approaches both failed.
1014
1015 - **socat** forwards raw bytes it does not prepend the 2-byte length prefix that DNS-over-TCP requires.
AdGuard parses the stream incorrectly and returns garbage.
1016 - **dnsmasq** tries UDP first to reach the upstream ('127.0.0.1#5354'), but Colima's SSH tunnel only
forwards TCP. With a single upstream server, dnsmasq doesn't fall back to TCP even with '--timeout
=1'.
1017
1018 **Solution:** Replaced both with a **Python DNS proxy** ('scripts/dns-proxy.py', ~25 lines) that properly
handles the DNS-over-TCP wire format: reads a UDP query from the host, prepends the 2-byte length

```

```

prefix, sends it over TCP to AdGuard, strips the prefix from the response, and sends it back over
UDP.
1019
1020 *Architectural Note: Why is the DNS proxy in Python while the SOCKS5 proxy was moved to a compiled Go
    Docker container?
1021 The SOCKS5 proxy handles heavy data streaming (e.g., video, downloads). Python introduces massive CPU/
    battery overhead for raw socket streaming, which is why we rely on the compiled Go implementation
    built directly into the 'tailscaled' daemon via 'TS_SOCKS5_SERVER=:1080'.
1022 Conversely, the DNS proxy only handles intermittent UDP queries (a few bytes per minute) generated solely
    by the local host machine. The Python overhead for this is effectively 0.001 Watts. We deliberately
    kept 'dns-proxy.py' in Python because it runs natively on the macOS/Linux host rewriting it in Go
    would force users to install a Go compiler ('brew install go') on their host machine for zero
    noticeable battery gain. (Note: If this architecture is ever adapted to serve as a public DNS
    resolver for thousands of users or for massive automated web-scraping clusters, 'dns-proxy.py' must
    be rewritten in Go to survive the concurrent packet flood).
1023
1024 ##### 15.4.2 Tailscale MagicDNS Bypassing the AdGuard PREROUTING Intercept
1025 **Context:** The 'routing-fix.sh' script applies an iptables NAT rule ('PREROUTING -i tailscale0 -p udp
    --dport 53 -j REDIRECT') to intercept all incoming DNS queries from connected Tailscale peers and
    force them into the AdGuard container on port 5335.
1026 **Problem:** When remote clients (like Android or iOS) have "Use Tailscale DNS settings" turned on, they
    query Tailscale's MagicDNS IP ('100.100.100.100'). This packet enters the exit node, but the '
    tailscaled' daemon running *inside* the gateway container intercepts the '100.100.100.100' packet
    internally. 'tailscaled' then generates a brand new, local DNS request to the upstream DNS server to
    resolve the query. Because this new request is generated *locally* by the daemon (not arriving
    externally via the 'tailscale0' interface), it bypasses the 'PREROUTING' chain completely. The
    request glides straight out the WARP tunnel to Cloudflare/Google, entirely bypassing the AdGuard
    sinkhole.
1027 **Fix:** The only way to fix this blindspot is to force 'tailscaled' to use AdGuard as its upstream. In
    the Tailscale Admin Console, the gateway's Tailscale IPv4 address (e.g., '100.x.x.x') must be added
    as the sole Custom Global Nameserver, and "Override local DNS" must be enabled. Additionally, to
    block Android devices from bypassing this via Private DNS (DNS-over-TLS), outbound Port 853 traffic
    is explicitly REJECTED in both 'routing-fix.sh' and 'post-rules.txt'.
1028
1029 ##### 15.4.3 Seamless Wi-Fi Roaming & The macOS DNS Wipeout Loophole
1030 **Problem:** The Tailscale, WireGuard, and Colima NAT stacks are natively designed for connectionless
    roaming and will seamlessly survive when the macOS host switches Wi-Fi networks (e.g., roaming from
    home Wi-Fi to a cellular hotspot). However, the internet completely breaks upon network transition.
    This occurs because macOS intentionally wipes out all custom DNS settings ('networksetup -
    setdnsservers') and reverts to the new network's default DHCP nameserver whenever the active Wi-Fi
    BSSID changes. Since the Mac is locked to the exit node, its DNS requests are swallowed by WARP and
    dumped onto the public internet, which cannot route private DHCP IPs. This results in a total DNS
    failure.
1031
1032 **Solution:** Implementing a silent background "DNS Watcher" daemon ('nullexit-dns-watcher') in 'toggle.
    sh'. When the gateway starts, it spawns a background polling loop that checks the active Wi-Fi DNS
    every 30 seconds. If macOS resets the DNS during a network change, the watcher instantly detects the
    drift and forcefully re-injects '100.100.21.8' via 'networksetup'. When the gateway stops, the
    watcher process is cleanly killed. This allows true, seamless roaming across physical networks
    without ever dropping the gateway state.
1033
1034 ##### 15.4.4 docker compose exec '\r' Injection
1035 **Fix:** All 'docker compose exec' output piped through 'tr -d '\r''.
1036
1037 ##### 15.4.5 Stderr Invisibly Swallowed
1038 **Symptom:** Failures were silent due to stderr redirected to 'output.log'.
1039
1040 **Fix:** Tailscale wait loop now shows output. Critical commands show errors inline.
1041
1042 ### 15.5 Roaming, Sleep/Wake & Auto-Recovery
1043
1044 ##### 15.5.1 Gateway Breakage on Lid Close / Wi-Fi Roam Post-Wake + Post-Roam Auto-Recovery
1045 **Context / Symptom:** Closing the lid (sleep/wake) OR losing + reconnecting Wi-Fi (e.g. in an elevator,
    caf, or roaming between APs) leaves the gateway in a partly-dead state. Symptoms range from a ~30s
    window of dead DNS to a permanent outage that only full 'recover.sh' or 'toggle.sh' fixes. The root
    cause is that nullexit has **no daemon watching macOS sleep/wake or network-state-change events**
    the existing DNS Watcher inside 'toggle.sh' only runs in-process and is suspended along with the
    rest of the user shell on lid close, so the gateway cannot self-heal after either event.
1046
1047 **Diagnosis:** Two distinct failure modes share the same root gap.
1048
1049 * ** (A) Lid close sleep wake. ** 'caffeinate -i' (used by 'toggle.sh') blocks *idle* sleep but **does
    not block forced sleep from lid close**. Closing the lid hard-suspends Colima VM + every container +
    every Tailscale-quit-shells-except-tailscaled. On wake the VM clock needs ~5-30s to resync via NTP,
    during which TLS cert validation fails: Cloudflare WARP '162.159.192.1:2408' rejects the handshake
    gluetun's healthcheck ('interval: 2s, retries: 15') eventually flips unhealthy Docker 'unless-
    stopped' restarts the 'warp' container (~30s gap). 'mDNSResponder's in-memory cache still holds pre-
    sleep entries (stale A records for tailnet hosts) so the first dozen DNS queries after wake time
    out. 'tailscaled' on the host often keeps its stale DERP relay preference and routes packets through
    a dead relay for another 30-60s.

```

```

1050 * **(B) Wi-Fi roam / loss in elevators, captive portals.** WARP is a UDP tunnel to Cloudflare's anycast
    edge. Carrier NATs and captive-portal networks silently invalidate the UDP binding on roam and on
    hotspot-bound re-association. macOS 'networksetup' *should* preserve DNS settings on the same Wi-Fi
    service across a roam, but most captive-portal networks DHCP-replace DNS to the captive-portal
    resolver **during the captive-portal dance itself**, before the user has authenticated so even DNS
    hijack survives a clean roam and quietly breaks on captive-portal handoff.

1051
1052 **Resolution.** A single **launchd LaunchAgent** ('com.nullexit.wake-recovery') runs a long-running shell
    daemon ('scripts/watcher.sh') that listens for both event sources and, on each fire, executes 'bash
    recover.sh --post-wake' a *lighter-touch* recovery mode that's the opposite of the existing '--
    nuclear' semantics: it keeps the gateway live while still refreshing every stale subsystem.

1053
1054 **Deep dive: Apple Power Management Primer (for the unfamiliar).**
1055 macOS exposes several relevant surfaces; the right primitive depends on what you actually need to detect.
    None of these are obvious and none of them have a standard splash screen in the docs.

1056
1057 * ***caffeinate <flags>** creates I/O Kit power assertions via 'IOPMAssertionCreateWithName'. It does
    NOT block forced sleep (lid close, low-battery shutdown, sleep timer firing on a per-set Energy
    Settings policy). Flags:
1058 * '-i' PreventIdleSleep (the only one 'toggle.sh' uses; protects against the OS going idle while the
    user is active but a clamshell close still hard-puts it to sleep)
1059 * '-s' PreventSystemSleep (only honored on AC power; the user may be on battery)
1060 * '-u' DeclareUserActive (resets the idle timer every 30s by default; equivalent to keeping the cursor
    moving)
1061 * '-d' PreventDisplaySleep
1062 * '-m' PreventDiskIdleSleep
1063 * There is NO '-l' (lid-block) flag in any modern macOS. Lid close is a kernel-firmware-level event
    that ignores user-space assertions.
1064 * To prove any of this on live hardware: 'pmset -g assertions' lists every active assertion with type /
    process / reason / timeout.
1065 * ***pmset** is the read-side of the same system:
1066 * 'pmset -g log | grep -E 'Sleep|Wake|DarkWake'' shows the recent timeline.
1067 * 'pmset -g history' is the per-day summary.
1068 * 'pmset -g assertions' is the live assertion list (matches '-i -s -d -u' to processes).
1069 * ***launchd** does NOT have a native "on-wake" hook. **Not in any version of macOS as of Sonoma. The
    canonical recipe is 'StartCalendarInterval' with a sub-minute cadence and let launchd queue missed
    intervals for replay-on-wake, or use a third-party daemon (Sleepwatcher). For our use case a long-
    running shell daemon listening to the unified log is more responsive than a polling agent.
1070 * ***com.apple.system.powermanagement.** Darwin notifications **('willSleep', 'didWake', 'didDim', '
    didUndim') are the C-level signal. From a shell, they are best reached through the unified log with
    'log stream --predicate' see the watcher implementation.
1071 * ***scutil n.watch** is the canonical CLI for live-following network-state changes. Pairs with 'n.add
    State:/Network/Global/IPv4' to get a single tap that fires on **any** IPv4 link/route/DNS/address
    change across **all** interfaces. This is what 'scripts/watcher.sh's network-change listener uses.
1072 * ***com.apple.networkChange** Darwin notification **is the C-level equivalent but has no shell-friendly
    listener; use 'scutil' instead.
1073 * ***SCNetworkReachability** is the Objective-C API used by SOCKS5/HTTP clients to detect route changes
    per target. Never a fit for shell scripts.

1074
1075 **The Fix (component-by-component).**
1076
1077 1. **'recover.sh --post-wake** (new flag, **non-destructive** by design). Adding '--post-wake' to the
    existing nuclear recovery script inverts the semantics. The default mode still tears down Tailscale,
    resets DNS to empty, stops caffeinate, runs 'docker compose down', power-cycles Wi-Fi. The new mode
    does:
1078 * Skip Tailscale disconnect (we WANT it to keep the mesh connection)
1079 * **Re-hijack** DNS to the gateway Tailscale IP read from '.gateway_ip' (instead of resetting to empty)
    applies to both 'ACTIVE_SERVICE' and 'ENO_SERVICE' because the system resolver is scoped to en0
1080 * **Refresh** the exit-node preference with 'tailscale up --reset --ssh=true --accept-dns=false --exit-
    node=\"$TS_IP\" --exit-node-allow-lan-access=true' (the exact flags 'toggle.sh' START uses). This re
    -asserts DERP mapping without dropping the mesh
1081 * Inspect 'docker inspect --format '{{.State.Health.Status}}' nullexit-warp-1' and **only** 'docker
    compose up -d --force-recreate warp' if gluetun is unhealthy this single targeted recreate nudges
    the UDP NAT binding back to Cloudflare without disturbing Tailscale or AdGuard
1082 * Run the sharingd-reset step (existing fix from 'toggle.sh' START path; see 15.11) so AirDrop doesn't
    freeze on the new IP lease
1083 * Verify the gateway with 'dig +tcp @127.0.0.1 -p 5354 google.com' (AdGuard via TCP through Colima's
    SSH tunnel) and a 4-second 'tailscale ping' to the gateway Tailscale IP, instead of the default mode
    's direct-to-1.1.1.1 check
1084 * Crucially: **skip 'sudo -v** because the LaunchAgent has no TTY to prompt on; all privileged calls
    use 'sudo -n' and lean on '/etc/sudoers.d/nullexit' NOPASSWD entries ('networksetup', 'dnsmasq', '
    dscacheutil', 'killall', 'pkill').
1085 2. **'toggle.sh** writes and clears '/tmp/nullexit-gateway-active.marker' (an ISO-8601 UTC timestamp) at
    the bottom of the START path and the top of the STOP path / 'cleanup_handler'. Two new helpers: '
    write_gateway_active_marker' and 'clear_gateway_active_marker'. The watcher uses this file as its **
    only signal** that the gateway is live: if 'toggle.sh' was stopped (or never started), the watcher
    no-ops. This prevents waking-up-only-on-counterfactual events from churning DNS.
1086 3. **'scripts/watcher.sh** (long-running daemon, sibling-style source file). Two backgrounded pipelines:
1087 * **Wake listener**: 'log stream --predicate 'composedMessage CONTAINS[c] \"didWake\" OR composedMessage
    CONTAINS[c] \"Wake from Sleep\" OR composedMessage CONTAINS[c] \"Waking from\" OR composedMessage
    CONTAINS[c] \"DarkWake\"' --style compact --info' piped through 'while read; do run_recover \"WAKE:

```

```

..."; done'. '[c]' makes the predicate case-insensitive (the unified log writes phrases in mixed
case across versions).
1088 * **Network listener**: '(echo "n.add State:/Network/Global/IPv4"; echo "n.watch"; while true; do sleep
86400; done) | scutil 2>/dev/null' piped through a 'case' match on 'n.state*' / 'SCEventUpdate*'.
1089 * **'run_recover'** checks the marker, debounces via '/tmp/nullexit-watcher.last-recovery' (default 10s
, configurable via 'NULLEXIT_DEBOUNCE_SECONDS'), and shells out to 'bash recover.sh --post-wake'.
The 10s debounce prevents a single wake event (which typically emits 2-3 log lines) from triggering
multiple recoveries.
1090 * 'trap ... TERM INT' 'kill 0' cleanly tears down the process group on launchctl 'bootout' so the '
sleep 86400' inside the scutil pipe also dies.
1091 * 'wait' blocks indefinitely while both pipelines feed it.
1092 4. **'launchd/com.nullexit.wake-recovery.plist'** (LaunchAgent in the user domain runs as the console
user at every login without needing sudo).
1093 * 'Label: com.nullexit.wake-recovery'
1094 * 'ProgramArguments: ["/bin/bash", "<absolute-path-to-your-nullexit-install>/scripts/watcher.sh"]'
1095 * 'RunAtLoad: true' starts immediately on 'launchctl load' (no need to log out and back in)
1096 * 'KeepAlive: { SuccessfulExit: false, Crashed: false }' **don't** restart on clean SIGTERM exit (so '
launchctl bootout' doesn't get stuck in a relaunch loop). Also **don't** restart on hard crash: we
observed that macOS Sonoma's sleep/wake suspend-resume path accumulates orphan watcher.sh PIDs
because SIGCONT does not reliably clean up the prior instance's listener grandchildren, and a '
KeepAlive.Crashed=true'-driven relaunch actively races with the still-live prior process. The script
enforces single-instance on its own via a 'flock'-style PID-file lock, so a relaunch-on-crash would
be skipped by the lock and produce 0 listener coverage until the live instance happened to die.
Trade-off: a real crash leaves the user without auto-recovery until they run 'launchctl load -w'
manually acceptable because (a) gateway still works without auto-recovery, (b) a relaunch wouldn't
fix whatever caused the crash, and (c) the gateway-recovery itself is observably broken (no DNS, no
exit-node) if it does go down.
1097 * 'ProcessType: Background' hint to App Nap not to suspend us.
1098 * 'StandardOutPath/StandardErrorPath: /tmp/nullexit-watcher.{out,err}.log' separates launchd-level
diagnostics from the script's own '/tmp/nullexit-watcher.log' (which 'exec >>' redirects).
1099
1100 **Install (one-time).** The plist lives in the repo at **'launchd/com.nullexit.wake-recovery.plist'** for
version control. The installed (live) copy must land in '~/Library/LaunchAgents/' so launchd picks
it up on the user session.
1101
1102 ```bash
1103 # Copy the plist to the user-launchd location (run from repo root)
1104 cp ./launchd/com.nullexit.wake-recovery.plist \
1105 ~/Library/LaunchAgents/
1106
1107 # Substitute __NULLEXIT_HOME__ with your local install absolute path.
1108 # The plist uses WorkingDirectory + a relative program arg, so this
1109 # single substitution is the only place it references your machine.
1110 sed -i '' "s|__NULLEXIT_HOME__|$(pwd)|" \
1111 ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
1112
1113 # Validate the XML
1114 plutil -lint ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
1115
1116 # Load + persist this user session + every future login
1117 launchctl load -w ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
1118
1119 # Confirm it actually started
1120 launchctl list | grep nullexit
1121 ```
1122
1123 To **stop** the watcher (without uninstalling):
1124
1125 ```bash
1126 launchctl bootout gui/$UID/com.nullexit.wake-recovery
1127 # or:
1128 launchctl unload -w ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
1129 ```
1130
1131 To **uninstall** completely:
1132
1133 ```bash
1134 launchctl bootout gui/$UID/com.nullexit.wake-recovery 2>/dev/null || true
1135 rm ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
1136 ```
1137
1138 **Why this fix was preferred over alternatives.**
1139
1140 * **Polling with 'StartCalendarInterval'** would have given ~30-60s detection latency per wake. The
unified-log stream is sub-second.
1141 * **Sleepwatcher** would have covered wake events but not network changes. We'd still need a separate '
scutil' listener, and introducing a third-party dependency for what amounts to ~150 lines of shell
is unjustified.
1142 * **Forcing 'caffeinate -l'** doesn't exist and the closest equivalent ('caffeinate -s') only works on AC
power. The whole point of the gateway is to stay usable on battery while traveling lid close must
NOT silently kill the gateway.

```



```

1143 * **A kernel extension** is impossible (no entitlement) and out of scope.
1144
1145 **Reproduction recipe.** Test both events in isolation to confirm the fix actually closes the gap.
1146
1147 * **(A) Lid-only repro**: with the gateway up, run in a terminal: 'pmset displaysleepnow && sleep 30 &&
    pmset wake' (without physically closing the lid). Watch the wake wake-fires the watcher post-wake
    routine runs login should still work in <5s after wake. Compare to baseline pre-fix: pre-fix the
    gateway would be dead for 30-90s.
1148 * **(B) Roam-only repro**: with the gateway up: 'networksetup -setairportpower off && sleep 30 &&
    networksetup -setairportpower on' (or simply walk out of Wi-Fi range and back). The captive-portal
    WILL repave DHCP DNS for a few seconds; the watcher fires on the second 'State:/Network/Global/IPv4'
    change (after the Wi-Fi re-associates) and the post-wake routine re-hijacks DNS within 2-3s.
1149
1150 **Operative files.**
1151
1152 * 'recover.sh' added arg parsing; wrapped destructive sections behind 'POST_WAKE'; added re-hijack DNS /
    refresh exit-node / force-recreate-if-unhealthy path.
1153 * 'toggle.sh' added 'write_gateway_active_marker' / 'clear_gateway_active_marker' called at the END of
    START and TOP of STOP / cleanup_handler.
1154 * 'scripts/watcher.sh' new long-running daemon.
1155 * 'launchd/com.nullexit.wake-recovery.plist' launchd LaunchAgent descriptor.
1156 * '~/Library/LaunchAgents/com.nullexit.wake-recovery.plist' the installed copy (one-time copy).
1157
1158 **Honest assessment (read before testing).** This fix has two asymmetric halves:
1159
1160 * **Network / roam / captive-portal path RELIABLE.** 'scutil n.watch' on 'State:/Network/Global/IPv
    {4,6}' catches every Wi-Fi roam, captive-portal DHCP-rebind, hotspot handoff, and VPN change with
    zero missed events. To validate, drop Wi-Fi for 30 s and re-add it; '/tmp/nullexit-watcher.log' will
    record a 'NET:' trigger and 'recover.sh --post-wake' will run within ~10 s of the network coming
    back.
1161 * **Lid close / wake path BEST-EFFORT on macOS Sonoma+.** Apple *suspends* LaunchAgents process-state
    during forced sleep and *resumes* them on wake; it does NOT re-launch them every wake. As a result:
    (a) 'log stream' is a live subscription events emitted during the agent's suppressed window are
    DROPPED, not buffered, so the wake event that prompted the resume is invisible to 'log stream' on
    resume; (b) the "fire on startup" guard runs once on 'launchctl load -w' and after a 'KeepAlive'
    crash-recovery, NOT on every successive wake; (c) 'subsystem == "com.apple.powermanagement"' will
    catch FUTURE emissions (display dim/restore, manual sleep/wake) but not the lid-close/open wake
    directly.
1162
1163 In practice the lid-wake gap is short, because 'toggle.sh's existing DNS Watcher already polls every 5 s
    and re-hijacks DNS (so DNS recovers within 5 s of any roam or wake), 'tailscaled' self-heals via
    DERP fallback within 3060 s, and 'gluetun' reconnects via its healthcheck retry within 30 s. The
    user-visible pain on lid open is ~30 s of wonky DNS, not a permanent outage.
1164
1165 **Test the ROAM path FIRST.** If ROAM works in your testing, the lid-wake gap is acceptable. If lid-wake
    handling is critical, the two clean follow-ups are:
1166 1. **Sleepwatcher** (one canonical install): 'brew install sleepwatcher'; drop 'recover.sh --post-wake'
    into '~/.sleep' and '~/.wake' so Sleepwatcher invokes it directly on every wake without the launchd
    propagation problem.
1167 2. **Move the watcher to a LaunchDaemon** (root) and use 'IOPMSchedulePowerEvent' via a tiny C wrapper
    not portable to shell.
1168
1169 A truly reliable shell-only solution to "wake events from inside a LaunchAgent" does not exist on macOS
    Sonoma+.
1170
1171 **Empirical lessons from deployment.** Two findings came up while deploying this fix end-to-end;
    recording so the next operator doesn't lose an evening to each.
1172
1173 1. **Bash function-call-before-definition gotcha.** While introducing the gateway-active marker helpers,
    the defensive 'clear_gateway_active_marker' call was inserted at the very top of 'toggle.sh' (right
    after 'set -e'), but the function definition lived near 'PID_FILE="/tmp/nullexit-cafeinate.pid'
    ~90 lines down. Bash parses top-to-bottom and resolves names at first invocation, so the call site
    exited with code 127 ('clear_gateway_active_marker: command not found'). The gateway stayed
    unreachable from the START path even though 'bash -n' passed (the parser doesn't catch order-of-
    resolution errors). Rule: define functions BEFORE any block that calls them. Verify with 'grep -n '^
    name()\s*{' followed by 'grep -n '^name\s*{' the latter must appear on a line number AFTER the
    former.
1174 2. **networksetup -setairportpower off+on' is not a faithful roam reproduction.** Synthetic Wi-Fi radio-
    cycling ('sudo networksetup -setairportpower Wi-Fi off' for 30 s, then back on) is expected to
    produce a 'NET:' trigger in 'scutil n.watch' but produced ZERO during testing because '
    setairportpower' powers the radio at the driver level while leaving the network SERVICE in the "up
    " state from scutil's perspective, so no 'n.state State:/Network/Global/IPv4' event is generated. A
    real roam (macOS briefly loses the AP and re-associates into a different one) DOES fire the event
    because the link actually changes.
1175 Better reproduction recipes in priority order:
1176 * Walk between two real SSIDs via 'networksetup -switchtolocation' (or actual physical station
    movement) guaranteed to fire the scutil event.
1177 * Force a DHCP rebind on the same SSID with 'sudo ipconfig set en0 DHCP' triggers 'State:/Network/
    Global/IPv4' to update.
1178 * Trust the existing 30-second DNS Watcher inside 'toggle.sh' for the DNS path: it re-hijacks DNS
    automatically. The post-wake watcher adds clear value mostly for tailscale exit-node re-assertion

```

```

and warp gluetun force-recreate, both of which self-heal within ~30 s anyway.
1179
1180 ##### 15.5.2 July 8, 2026: Network Watcher Event Mismatch, Sudo-in-Background Crash, and Loop-Prevention
1181 **Symptom:**
1182 1. Switching Wi-Fi or waking the Mac from sleep did not trigger a post-wake recovery ('recover.sh --post-
wake') automatically.
1183 2. The network connection would eventually get completely sinkholed/blocked. If the user tried to restart
or wait, it did not self-heal.
1184 3. During the recovery attempt, checking the public IP on the host would return the unencrypted campus/
ISP IP (starting with '132.') instead of the encrypted tunnel IP.
1185
1186 **Root Cause:**
1187 - **Bug 1 (Network Watcher Mismatch):** In 'scripts/watcher.sh', the network change listener watched '
State:/Network/Global/IPv4' changes via 'scutil n.watch' but matched them against '*n.state|*
SCEventUpdate*'. On macOS, the actual output is 'changedKey [0] = State:/Network/Global/IPv4'.
Because the pattern did not match, all network switches were silently ignored.
1188 - **Bug 2 (Sudo Caching Background Crash):** When the background WARP Watcher eventually detected the
broken tunnel (after 6 failures/30s), it triggered the default nuclear 'recover.sh' (with 'POST_WAKE
=false'). Because there was no active TTY in the background context, the 'sudo -v' caching check
aborted instantly with a terminal required error. Due to 'set -e', 'recover.sh' died immediately
before resetting DNS, disabling the packet-filter killswitch, or stopping containers, leaving the
host network completely sinkholed.
1189 - **Bug 3 (Post-Wake Infinite Loop):** Once the watcher patterns were fixed, 'recover.sh --post-wake'
triggered successfully on network changes. However, because the script deleted the default routing
entries at the start, spent ~15 seconds rebuilding 88 DERP relay bypass routes, and re-added the
default routes at the end, this execution time exceeded the watcher's 10-second debounce threshold.
Furthermore, the watcher recorded the *start* timestamp in the debounce file. Consequently, the
route changes made by the script itself triggered the watcher again immediately after completion,
trapping the system in an infinite loop of recovery runs. During this loop, the VPN routes were
constantly deleted, resulting in the host leaking traffic through the real ISP interface (an IP
starting with '132.') for 15s out of every cycle.
1190
1191 **Resolution:**
1192 - **Fixed Watcher Matching:** Updated 'scripts/watcher.sh' to match '*changedKey*' and '*State:/Network/
Global/IPv4*' to correctly catch network changes.
1193 - **Bypassed Background Sudo:** Added '--auto'/'--non-interactive' flags to 'recover.sh' and added a TTY
check '[ -t 0 ]' to skip interactive 'sudo -v' authentication when running headlessly in the
background. Updated the WARP Watcher call in 'toggle.sh' to pass '--auto'.
1194 - **Prevented Infinite Loops:** Modified 'scripts/watcher.sh' to write the **completion** timestamp of '
recover.sh' to 'DEBOUNCE_FILE' (instead of only the start timestamp). This ensures all buffered
network config events generated during route reconstruction are evaluated against the end-of-run
timestamp and successfully debounced, breaking the infinite loop.
1195 - **Centralized & Timestamped Logs:** Redirected 'watcher.sh' stdout/stderr from '/tmp/nullexit-watcher.
log' to the repository's main 'output.log' and updated 'scripts/common.sh's 'step', 'ok', 'warn', '
fail', and 'die' helpers to prefix logs with a local '[YYYY-MM-DD HH:MM:SS]' timestamp.
1196
1197 ##### 15.5.3 Incident Post-Mortem: Startup Pre-Flight Check Tailscale Race (July 10, 2026)
1198 **Symptom:** Immediately after startup via 'toggle.sh', the pre-flight connectivity check '[1/3] Gateway
reachable via Tailscale' returned 'FAIL', forcing 'toggle.sh' to skip the full exit-node activation
and fall back to SOCKS5/local DNS proxy mode.
1199
1200 **Root Cause:** A timing race condition during startup. In 'toggle.sh', the host joins the mesh using '
tailscale up --reset' (Phase A) right before performing the pre-flight check. Because 'tailscale up'
resets and reconfigures the host's 'tailscaled' daemon, the local daemon must fetch the latest
netmap asynchronously from the coordination servers. This network map propagation and route
establishment takes a few seconds. Running 'tailscale ping' immediately after 'tailscale up' returns
(with no delay) causes the check to fail because the local daemon has not yet discovered the
gateway node '100.100.21.8'.
1201
1202 **Fix (July 10, 2026):** Upgraded pre-flight Check 1 in both 'toggle.sh' and 'scripts/toggle-linux.sh' to
use a 5-attempt retry loop with a 1-second delay between checks. This allows up to 5 seconds for
local tailnet routing paths to settle, ensuring the gateway is correctly reached and a pong is
received before deciding whether to enable exit-node routing.
1203
1204 ##### 15.5.4 Incident Post-Mortem: Pre-Flight Check False Negatives due to VPN Firewall STUN Blocks (July
10, 2026)
1205 **Symptom:** Immediately after a cold boot or full restart of the gateway, the pre-flight connectivity
check '[1/3] Gateway reachable via Tailscale' failed, causing 'toggle.sh' to skip the full exit-node
configuration and fall back to SOCKS5/local DNS proxy mode. However, manually running 'tailscale
ping 100.100.21.8' immediately after startup succeeded in 1ms.
1206
1207 **Root Cause:** An asynchronous Tailscale handshake delay caused by firewall restrictions. Inside the
container network namespace, the 'warp' container (gluetun) implements a strict firewall that blocks
all outbound UDP traffic to the public internet except to the designated WireGuard endpoint.
Because of this, the 'tailscale' container's STUN requests to public STUN servers are blocked,
causing the local daemon to report 'UDP is blocked, trying HTTPS'. Without UDP STUN capability,
Tailscale is forced to fall back to a slower DERP-assisted handshake (relayed via HTTPS/TCP). This
negotiation and direct-path punch-through takes roughly 30 to 35 seconds to establish on cold boots
(after a full 'tailscale up --reset'). Since the pre-flight check only waited 15 seconds (15
attempts with 1-second sleeps), the check timed out and aborted before the path completed.
1208

```



```

1209 **Fix (July 10, 2026):** Increased the pre-flight ping retry limit from 15 to 45 attempts (45 seconds) in
    both 'toggle.sh' and 'scripts/toggle-linux.sh'. This provides sufficient time for the DERP-assisted
    handshake to complete on cold boots, while still passing immediately (in 1-2 seconds) on warm
    starts or once the path is established.
1210
1211 #### 15.5.5 Incident Post-Mortem: Docker Image Build DNS Failures on Immediate Restart (July 10, 2026)
1212 **Symptom:** When executing 'toggle.sh --restart', the script successfully stops the gateway but then
    immediately fails during startup during 'docker compose build' with DNS resolution errors:
1213 'failed to do request: Head "https://registry-1.docker.io/...": dial tcp: lookup registry-1.docker.io on
    192.168.5.1:53: no such host'.
1214
1215 **Root Cause:** A race condition between physical link negotiation and virtual machine startup. The STOP
    path of the restart command invokes 'cleanup_network_state' which power-cycles the host's physical
    Wi-Fi interface ('en0') to flush stale routing states. Immediately after, 'toggle.sh' starts the
    gateway boot sequence. Because 'toggle.sh' did not verify the status of the physical interface, it
    proceeded to boot Colima and build Docker containers while 'en0' was still negotiating a DHCP lease.
    Consequently, the host (and by extension, the VM bridge) had no active internet gateway/DNS path,
    causing Docker's registry check to fail.
1216
1217 **Fix (July 10, 2026):**
1218 1. Created a unified 'wait_for_dhcp_settle' helper function in 'scripts/common.sh' that polls 'ipconfig
    getpacket' and 'ifconfig' until a valid IP and router are assigned. To handle typical macOS Wi-Fi
    reassociation and DHCP negotiation latency (which commonly takes 15-20 seconds after a power-cycle),
    this loop was configured to poll up to 60 times (30 seconds total).
1219 2. Injected a call to 'wait_for_dhcp_settle' at the beginning of the 'START' action in 'toggle.sh' (right
    before checking Colima status) to block container builds until the host's physical network is
    online.
1220 3. Refactored 'recover.sh' to reuse the unified 'wait_for_dhcp_settle' function.
1221
1222 #### 15.5.6 Watcher.sh Suppressed Exit Code Bug (Fixed)
1223 **Observation:** The 'scripts/watcher.sh' script is a long-running daemon that invokes 'recover.sh --post
    -wake' when it detects system wake or network roam events. Previously, it contained the following
    bug:
1224 ```bash
1225 bash "$RECOVER" --post-wake
1226 local exit_code=$?
1227 ...
1228 return $exit_code || true
1229 ```
1230 The '|| true' mistakenly swallowed the actual '$exit_code' returned by 'recover.sh', causing 'run_recover
    ()' to always return '0' (success), masking any underlying failures in the wake recovery process.
1231
1232 **Fix:** The '|| true' statement was removed from the return line:
1233 ```bash
1234 return $exit_code
1235 ```
1236 Since 'watcher.sh' explicitly omits 'set -e' (to prevent pipe breakages from killing the daemon),
    removing '|| true' safely allows the underlying exit code to propagate without risking daemon
    termination. The watcher daemon was then reloaded ('launchctl kickstart -k') to pick up the script
    changes in memory.
1237
1238 #### 15.5.7 Network Readiness Race Condition on '--restart' (Fixed)
1239 **Problem:** When running './toggle.sh --restart', the script tears down the gateway and immediately
    begins the startup sequence. On systems with Wi-Fi (or any wireless interface), the OS takes a
    moment to re-associate with the network after the teardown's DNS/routing changes. If the startup
    sequence ran before the OS had a default route, 'get_active_service' would return nothing, routing
    bypass targets would be wrong, and DNS/WARP setup would silently fail resulting in a partially
    configured gateway that self-heals only once the 'watcher.sh' re-runs.
1240
1241 **Root cause:** The original code had no gate before the first network-dependent call ('ACTIVE_SERVICE=$(
    get_active_service)' at the top of the start branch). This was a pure timing race.
1242
1243 **First fix (Wi-Fi specific):** A polling loop on 'ipconfig getifaddr en0' was added to wait for an IPv4
    address on 'en0' before proceeding. This worked but was brittle it only handled Wi-Fi and would
    fail for Ethernet, USB-C adapters, or iPhone USB tethering.
1244
1245 **Generalized fix:** Replaced the 'en0' check with 'route get default | grep 'gateway:'. This detects a
    valid default gateway on *any* interface, covering all connection types. The loop polls every 2
    seconds with a 60-second hard timeout (then proceeds with a warning rather than hanging forever).
    The output prints the detected gateway IP so failures are easy to diagnose in logs.
1246
1247 ```bash
1248 # In toggle.sh, before ACTIVE_SERVICE=$(get_active_service)
1249 while true; do
1250     if route get default 2>/dev/null | grep -q 'gateway:'; then
1251         _gw=$(route get default 2>/dev/null | awk '/gateway:/{print $2}')
1252         echo " Network ready (default gateway: $_gw)."
1253         break
1254     fi
1255     ...
1256 done

```

```

1257 '''
1258
1259 **Why not ping?** Pinging an IP (e.g. '1.1.1.1') during restart is unreliable because DNS may still be
    hijacked from the previous session, and the exit node routing may not yet be cleared, causing the
    ping to loop back through the tunnel. 'route get default' is a pure local kernel query no packets
    sent.
1260
1261 ### 15.6 WARP Tunnel Liveness & Host Leaks
1262
1263 #### 15.6.1 Host-Side Traffic Leaking Past the Exit Node (with Remote Clients Routing Correctly)
1264 **Symptom.** 'toggle.sh' reports success. Remote tailnet clients (e.g. an S24 phone) correctly egress
    through Cloudflare WARP and see a Cloudflare IP on 'whatismyip.com'. But on the **HOST Mac running
    the gateway, 'whatismyip.com' reports the underlying physical ISP's ASN (e.g. 'campus_isp' for a
    university campus Wi-Fi). The gateway container itself is healthy; the leak is specifically on the
    host.
1265
1266 **Root causes.** Three distinct mechanisms can each produce this exact symptom on the host alone, while
    leaving remote clients unaffected. They are mutually rankable so the diagnostic picks the active one
    deterministically.
1267
1268 * **(A) SOCKS5 fallback lane active.** 'toggle.sh's three Phase-B pre-flight checks ('tailscale ping'
    AdGuard via 'dig +tcp' WARP internet, toggle.sh ~lines 750-820) sometimes flake during the first
    minute. When ANY of the three fails, the script sets 'SKIP_EXIT_NODE=true' (toggle.sh:849) and
    instead activates the SOCKS5 fallback path (toggle.sh:894). The SOCKS5 fallback hijacks DNS to
    '127.0.0.1' (so AdGuard filtering still works) but **modern browsers on macOS do NOT honor the
    system SOCKS5 proxy** they ignore 'networksetup -setsocksfirewallproxy'. Result: DNS gets ad-
    filtered but actual HTTP/S traffic exits direct through the campus ISP. This branch is invisible
    from a remote tailnet client because the client's tunnel is unaffected.
1269
1270 * **(B) IPv6 leak over campus Wi-Fi.** The Tailscale exit node and WARP tunnel are both IPv4-only (
    gluetun + 'post-rules.txt' drop all IPv6 forwarded traffic; see 15.12 IPv6 Exit-Node Leak). But many
    university and enterprise APs are dual-stack the host happily accepts AAAA DNS responses and sends
    IPv6 traffic straight out 'en0'. macOS happily encrypts that traffic through Tailscale ONLY if
    Tailscale has IPv6 advertised; with an IPv4-only exit node, IPv6 traffic skips Tailscale entirely.
    This branch is invisible from a phone because iOS/Android Tailscale apps actively sinkhole IPv6 in
    their VPN profile, but the macOS host's 'tailscaled' does not.
1271
1272 * **(C) Route-table freeze and '--accept-routes' reset after 'tailscaled' wake/toggle.** Running '
    tailscale up --reset' clears the exit-node preference and reverts '--accept-routes' back to its
    default ('false'). Without explicitly passing '--accept-routes=true', 'tailscaled' will not attempt
    to install the default route through the 'utun*' tunnel interface even if the exit node preference
    is reconciled. Additionally, macOS occasionally freezes and fails to apply route-table changes (see
    15.11 Standalone Tailscale Daemon Freeze). In both cases, 'netstat -rn' shows the default route
    still pointing to the physical Wi-Fi gateway (e.g. '172.17.0.1') instead of the Tailscale 'utun*'
    interface, causing host traffic to exit direct while remote clients route correctly.
1273
1274 **Why remote clients don't see this.** Each remote phone/laptop uses its OWN Tailscale tunnel to reach
    the container's '100.x.x.x:443' over the mesh they're end-to-end encrypted regardless of how the
    host Mac itself is configured. Only the HOST's own egress sockets (HTTP requests from the host's
    browser, curl, etc.) are affected.
1275
1276 **Resolution.** A single script: 'scripts/diagnose-host-leak.sh'. It runs the 8 checks, classifies into
    one of A / B / C / OK, prints a verdict + a ready-to-run fix command on stdout, AND writes the full
    report to 'host-leak-diagnostic-<UTC>.txt' so the user can paste the file back instead of scrolling-
    and-copying from the terminal. With '--fix', the matched remediation is applied and the two checks
    that could change (warp + ipv6) are re-verified. With '--watch' (or '--watch 30'), it runs the full
    baseline diagnostic then enters a continuous loop checking warp/IPv6/default-route every N seconds,
    alerting only on state changes logs to 'host-leak-watch.log'.
1277
1278 '''bash
1279 # Diagnose only:
1280 bash scripts/diagnose-host-leak.sh
1281
1282 # Diagnose + apply matched fix + re-verify:
1283 bash scripts/diagnose-host-leak.sh --fix
1284 '''
1285
1286 **What it prints.** A 7-section report ('tailscale status', SOCKS5 state, 'cdn-cgi/trace' warp=, IPv6
    leak probe, host default route, last toggle.sh output.log hints, resolved gateway Tailscale IP),
    followed by a classification block ('Scenario: A | B | C | OK') with the exact command(s) the user
    needs to paste into their terminal they don't have to figure out which 'tailscale up' flags match
    their environment.
1287
1288 **What it does NOT do.** It does not touch 'toggle.sh' or 'toggle.sh's pre-flight logic, does not modify
    '.env' (except '--fix' mode appends 'GATEWAY_BYPASS_PING=true' when fixing Scenario A), and does
    not restart Docker. It is a read-only diagnostic with an opt-in remediation flag. Safe to run on a
    healthy gateway the worst case is a 30-line report that says "OK".
1289
1290 **Why cdn-cgi/trace over whatismyip.com for the verdict.** 'whatismyip.com's ISP database routinely
    mislabels campus IPv6 ranges as residential ISPs (that's why the local ISP's ASN appears even when
    you're routing through Cloudflare). 'cdn-cgi/trace' is hosted by Cloudflare ITSELF and reports 'warp

```

```

=on|off' plus the exact edge colo serving the request. It is the only public endpoint that can
truthfully confirm whether the WARP tunnel is in use.
1291
1292 **Common false alarms.**
1293
1294 * **'warp=on' but whatismyip.com still shows the local ISP.** 'whatismyip.com's ISP-detection database
is unreliable on academic networks. Trust 'cdn-cgi/trace's 'warp=on' over 'whatismyip.com's ISP
string. Use 'https://api.ipify.org' (returns just the IP, no ISP DB lookup) as a third confirmatory
check.
1295 * **'tailscale status' shows the host online but exit node preference is missing.** 'tailscale up --reset
--exit-node=' (empty value) clears any stale preference; 'tailscale up --reset --exit-node=<100.x.x
.x>' re-establishes it. The diagnostic prints the exact command with the resolved TS_IP filled in.
1296 * **Both IPv6 leak AND route-freeze flags set simultaneously.** Scenario B outranks Scenario C fixing
IPv6 makes IPv4 the only viable egress, after which the route-freeze usually self-resolves within
~30s (or one more 'toggle.sh' cycle).
1297
1298 **Deep dive: Host Leak Probe Continuous Sub-Second Egress Monitor ('scripts/host-leak-probe.sh').**
1299
1300 > ** STATUS: REMOVED (historical).** 'scripts/host-leak-probe.sh' and the 'HOST_LEAK_PROBE' env flag no
longer exist the file is deleted/untracked and nothing launches it (verified 2026-07-15). Its fail-
closed role was superseded by the **PF kill-switch** ('enable_killswitch' in 'common.sh') plus the
in-container **WARP Watcher** (15.6.2); the only surviving host-leak tool is the on-demand 'scripts/
diagnose-host-leak.sh'. The text below is kept for design rationale only treat every present-tense
claim (auto-launch, PID file, 'HOST_LEAK_PROBE=false' toggle) as past tense.
1301
1302 *What it is.* 'scripts/host-leak-probe.sh' is a lightweight background daemon (~1.4 MB RSS, ~01% CPU)
that polls 'https://www.cloudflare.com/cdn-cgi/trace' every 300ms directly from the host via 'curl'
not via 'docker compose exec'. This is a fundamentally different vantage point from the WARP
Watcher (15.6.2): the WARP Watcher asks the WARP *container* whether it sees 'warp=on'; the Host
Leak Probe asks what a real browser request leaving the host's physical NIC looks like. The two
blind spots it covers that the WARP Watcher cannot:
1303
1304 1. **Host-side flash-leaks** a Cloudflare anycast edge re-route, a browser reusing a stale pooled
connection across a tunnel state change, or a split-second routing gap during a Gluetun healthcheck
restart (see 15.12.13 Routing Fix 30-second polling acknowledged 14s window).
1305 2. **Host egress while container is healthy** the container can report 'warp=on' while the host's own
traffic exits direct (the three-scenario leak class documented above in 15.6.1).
1306
1307 *Lifecycle.* Started by 'toggle.sh's START path ('start_host_leak_probe') immediately after '
start_warp_watcher'. Controlled by:
1308
1309 | Signal path | What happens |
1310 |---|---|
1311 | 'toggle.sh' STOP | 'stop_host_leak_probe()' SIGTERM + PID file cleanup |
1312 | 'toggle.sh' 'cleanup_handler' (error/INT/TERM/HUP) | Same |
1313 | 'recover.sh' (default, non '--post-wake') | 6d block kill by PID file |
1314 | 'recover.sh --post-wake' | Left running the gateway is still live |
1315
1316 PID file: '/tmp/nullexit-host-leak-probe.pid'. Toggle with 'HOST_LEAK_PROBE=false' in '.env' to disable
at next gateway start.
1317
1318 *Output format.* All events are written to 'output.log' (repo root) with UTC timestamps. Only state *
changes* are logged the probe is silent during normal healthy operation.
1319
1320 ```
1321 [09:10:26] LEAK warp=off ip=45.23.1.1 prev_warp=on prev_ip=104.28.246.50
1322 [09:10:26] ROTATE warp=on ip=104.28.248.11 prev_ip=104.28.246.50
1323 [09:10:26] HOST-PROBE failed/timeout (count=1)
1324 ```
1325
1326 curl transport errors ('TLS handshake failed', 'Connection refused', etc.) are also appended to 'output.
log' via '2>>output.log' nothing is silenced to '/dev/null'.
1327
1328 *Grep cheatsheet.*
1329
1330 ```bash
1331 grep LEAK output.log # real warp=off events from the host
1332 grep ROTATE output.log # Cloudflare anycast edge rotations (harmless)
1333 grep 'HOST-PROBE' output.log # curl failures / timeouts
1334 ```
1335
1336 *Resource budget.* ~1.4 MB RSS, ~0% CPU at rest, ~3 HTTPS requests/second to Cloudflare. Safe to leave
running for hours. Back off the polling interval ('bash scripts/host-leak-probe.sh 1.0') if you
observe probe-failure pile-ups indicating Cloudflare rate-limiting.
1337
1338 *Detection vs prevention.* This script detects leaks; it does not prevent them. A sub-300ms flash-leak
can still slip between two polls undetected. If 'grep LEAK output.log' shows confirmed leak events,
the next step is a kill-switch (prevention) an 'iptables' rule on the host's 'en0' that drops non-
Tailscale, non-local egress whenever the gateway is active. That is a separate engineering task not
yet implemented.
1339

```

```

1340 ##### 15.6.2 In-Flight WARP Tunnel Liveness Monitor & Statistical Auto-Shutdown
1341 **Why.** nullexit's core promise is that your IP always appears as Cloudflare. If the WARP tunnel
    silently dies due to a UDP NAT timeout, a Cloudflare edge rotation, a Colima VM clock skew causing
    TLS cert rejection, or any of a dozen other failure modes the host silently falls back to egressing
    through the physical ISP. The user sees the gateway as "up" (containers running, Tailscale
    connected, DNS hijacked) but their real IP is exposed. This is the exact class of failure that Ingo
    Blechschmidt documented in his chilling 2024 Linux kernel post-mortem: for over two years, LUKS disk
    encryption on suspend was silently broken because a refactored kernel syscall returned success
    while doing nothing ([source](https://mathstodon.xyz/@iblech/116769502749142438)). The lesson: ***
    security mechanism that fails silently is worse than no mechanism at all*** it creates a false sense
    of safety that prevents the user from taking corrective action.
1342
1343 **Design principle.** The WARP Watcher ('start_warp_watcher()' in 'toggle.sh') is a background daemon
    that polls 'cdn-cgi/trace' every 30 seconds and applies a **statistically calibrated threshold**
    before triggering any safety response. This threshold distinguishes transient network blips (which
    self-heal within seconds) from genuine outages (which require intervention). A single 'warp-off'
    reading is meaningless Docker might be slow, the network might be congested, a container
    healthcheck might be restarting. But 6 consecutive off readings (30 continuous seconds of downtime)
    has vanishingly low probability of being a false positive: even at an unrealistically high 10% false
    -positive rate per poll,  $P(6 \text{ consecutive}) = 0.1 \cdot 0.0001\%$ . In practice, the per-poll false-positive
    rate is far lower than 10%, making the real probability astronomically small.
1344
1345 **What it does.**
1346
1347 1. **Always logs.** Every state transition (onoff, offon) is timestamped to 'output.log'. The user can '
    grep WARP output.log' to audit the tunnel's entire lifetime. This is the "never fail silently"
    mandate.
1348
1349 2. **Tracks consecutive failures.** A counter increments on each 'off' reading and resets to 0 on any 'on'
    reading. This ensures the watcher never fires on intermittent flapping.
1350
1351 3. **Auto-shuts down at threshold.** When the counter reaches 'WARP_FAIL_THRESHOLD' (default 6,
    configurable in '.env'), the watcher declares a statistically significant outage and runs 'recover.
    sh' the nuclear recovery script that disconnects Tailscale, stops Docker containers, resets DNS to
    '1.1.1.1', flushes routes, and power-cycles Wi-Fi. This instantly reverts the host to normal (un-
    encrypted) internet, guaranteeing no traffic leaks through a dead tunnel while the user thinks they'
    re protected. A trigger file at '/tmp/nullexit-warp-shutdown-triggered' prevents double-firing.
1352
1353 4. **Exits cleanly.** After shutdown, the watcher exits (the gateway is dead, there's nothing left to
    monitor). 'stop_warp_watcher()' is also called by 'toggle.sh's STOP path and 'cleanup_handler' to
    terminate the daemon cleanly during normal teardown.
1354
1355 **Configuration.** Set 'WARP_FAIL_THRESHOLD=3' in '.env' for a more aggressive 15s timeout, or '
    WARP_FAIL_THRESHOLD=12' for a conservative 60s window. The variable is parsed from '.env' using the
    same 'grep' pattern as 'GATEWAY_MSS' and 'GATEWAY_BYPASS_PING'.
1356
1357 **Log sample.** After a real outage (or a forced test via 'docker compose pause warp'):
1358
1359 ```
1360 [2026-07-03T21:15:17Z] WARP DOWN cdn-cgi/trace reports warp=off
1361 [2026-07-03T21:15:47Z] WARP SHUTDOWN 6 consecutive failures (threshold=6). Running recover.sh to kill
    the gateway and restore normal internet. Your IP is NO LONGER Cloudflare.
1362 [2026-07-03T21:15:52Z] WARP SHUTDOWN recover.sh completed, watcher exiting. Your IP is now your real ISP
    IP.
1363 ```
1364
1365 Or, if WARP self-recovers before the threshold:
1366
1367 ```
1368 [2026-07-03T21:15:17Z] WARP DOWN cdn-cgi/trace reports warp=off
1369 [2026-07-03T21:15:32Z] WARP RECOVERED warp=on after 3 consecutive off readings (threshold was 6)
1370 ```
1371
1372 **Relationship to other monitors.** The WARP Watcher is the **innermost safety layer** it runs as a
    child of 'toggle.sh' and only while the gateway is active. It complements (does not replace) the '
    scripts/watcher.sh' daemon (15.5.1, which handles sleep/wake and Wi-Fi roam) and 'scripts/diagnose-
    host-leak.sh' (15.6.1, which is a manual diagnostic tool). The WARP Watcher is the only component
    that actively verifies end-to-end tunnel liveness and takes autonomous action when it fails.
1373
1374 ##### 15.6.3 Incident: The Overnight Silent IP Leak (July 2026)
1375 **The Problem:** The host leaked traffic over its raw, unencrypted 'eth0' IP continuously for roughly 8
    hours, completely bypassing the VPN. The 'toggle.sh' state and the container liveness checks all
    reported the gateway as healthy and active.
1376
1377 **The Investigation:**
1378 1. **Sleep/Wake Checks:** We initially suspected macOS sleep cycles broke the routing table. A check of '
    pmset -g log' confirmed the laptop literally never slept (0 sleep events recorded since boot),
    ruling out all sleep/wake code paths.
1379 2. **Keepalive Expiry:** We discovered 'docker-compose.yml' was missing '
    WIREGUARD_PERSISTENT_KEEPLIVE_INTERVAL'. Without a keepalive, standard router NAT tables (which
    typically garbage collect UDP after 30 to 120 seconds of silence) quietly expired the WireGuard port

```

```

mapping overnight.
1380 3. **Container State Masking:** Because WireGuard is stateless, the 'warp' container didn't immediately
1381 crash. It stayed 'Up', which meant Docker's healthchecks and our 'toggle.sh' liveness checks never
1382 noticed the tunnel inside it was totally dead.
1383
1384 **The Solution:**
1385 1. Added 'WIREGUARD_PERSISTENT_KEEPLIVE_INTERVAL=25s' (the WireGuard standard to beat standard 30s NAT
1386 timeouts) to keep the UDP tunnel permanently pinned open through the router.
1387 2. Injected a **Fail-Closed Kill-Switch** into 'routing-fix.sh': A 30-second loop now actively verifies
1388 tunnel health by running 'curl' over 'tun0' to Cloudflare's trace endpoint. If it fails 3
1389 consecutive times, it immediately rewrites the default route to 'blackhole', severing all traffic
1390 instead of letting it leak through 'eth0'.
1391 3. Added a listener in 'watcher.sh' that detects this fail-closed event and instantly pushes a native
1392 macOS UI notification to the desktop, alerting the user to manually intervene.
1393
1394 > [!NOTE]
1395 > **Why dual failure systems?**
1396 > The system now relies on two independent failure handlers that cover each other's blind spots:
1397 > 1. **Automated Self-Healing (WARP Watcher + 'recover.sh')** Triggered when the 'warp' container logs '
1398 warp=off'. This happens when the server actively kicks the client. Because the container *knows* it
1399 is disconnected, it triggers 'recover.sh' to safely reboot Docker and reconnect.
1400 > 2. **Fail-Closed Kill-Switch ('routing-fix.sh' + 'watcher.sh')** Triggered when a silent network
1401 failure occurs (e.g., NAT timeouts). The container incorrectly believes it is connected ('warp=on'),
1402 so auto-recovery never fires. Instead of auto-recovering (which risks falling back to raw Wi-Fi mid-
1403 process while the user isn't looking), this kill-switch actively blackholes the internet and forces
1404 a manual intervention via a desktop notification.
1405
1406 ### 15.7 Tailscale P2P, SNAT & Control Plane
1407
1408 #### 15.7.1 macOS SNAT Endpoint Poisoning & The P2P Blackhole (July 10, 2026)
1409 **Symptom:** When a Tailscale client (like an S24) connected to the exact same Wi-Fi network as the macOS
1410 gateway, it completely lost internet connectivity. However, if the S24 connected to a Windows PC's
1411 hotspot on the same network, the tunnel worked flawlessly.
1412
1413 **Root Cause:** Claude's critique correctly dismantled the port collision theory: the gateway daemon wasn't
1414 failing to bind, and macOS wasn't load-balancing ports. The real culprit was **macOS SNAT Source
1415 Port Mangling poisoning Tailscale's path discovery**.
1416
1417 Here is the exact step-by-step of the "infinite loop" blackhole:
1418 1. **The P2P Handshake:** The S24 and the Mac are on the same Wi-Fi ('192.168.1.x'). The S24 discovers
1419 the Mac's local IP and sends a UDP hole-punch packet to '192.168.1.5:41641'. Colima successfully
1420 forwards this to the gateway container.
1421 2. **The Bypass Rule:** The gateway replies. Because 'routing-fix.sh' forces all Tailscale UDP traffic
1422 out 'eth0' to bypass WARP, the reply goes to the macOS host.
1423 3. **macOS SNAT Mangling:** macOS routes the reply out 'en0' to the S24. Because the packet crosses from
1424 the Colima VM bridge to the Wi-Fi interface, macOS applies Source NAT (SNAT). Critically, macOS **
1425 randomizes the source port** (e.g., changes it from '41641' to '58291').
1426 4. **Endpoint Poisoning:** The S24 receives the authenticated Tailscale packet from '192.168.1.5:58291'.
1427 Tailscale's 'magicsock' is designed to handle NAT dynamically, so it assumes the Mac has roamed to
1428 port '58291'! The S24 updates its endpoint and starts sending all its encrypted internet traffic to
1429 '192.168.1.5:58291'.
1430 5. **The Blackhole:** macOS has no port forward for '58291'. It drops 100% of the S24's outbound data
1431 packets. The connection is completely dead.
1432
1433 **Why did the Hotspot work?** When the S24 connected to the PC hotspot, it was behind the PC's NAT. The
1434 S24 sent the packet, PC NATted it, and the Mac replied (mangled to '58291'). When the reply hit the
1435 PC, the PC's strict NAT dropped it because the source port ('58291') didn't match the destination
1436 port it originally sent to ('41641'). Because the mangled packet was dropped, the S24 never poisoned
1437 its endpoint! It safely gave up on P2P, fell back to the 100% reliable TCP DERP relay, and the
1438 internet worked perfectly.
1439
1440 **Fix & Resolution (July 10, 2026):**
1441
1442 **Phase 1 Port disambiguation:** Changed the gateway container's Tailscale listen port from the default
1443 '41641' to '41642' across 'docker-compose.yml', 'post-rules.txt', and 'routing-fix.sh'. This
1444 prevents any bind conflict with the macOS host's native Tailscale daemon.
1445
1446 **Phase 2 RFC1918 DROP rules:** Updated 'routing-fix.sh's 'add_tailscale_p2p_bypass()' to explicitly
1447 drop all outgoing Tailscale UDP packets destined for private subnets ('192.168.0.0/16',
1448 '10.0.0.0/8', '172.16.0.0/12') when 'TAILSCALE_ALLOW_LAN_P2P=false'. This surgically kills the local
1449 P2P hole-punch attempt *before* it can trigger the macOS gvpnproxy SNAT mangling. The S24 receives a
1450 clean failure and immediately locks onto the public DERP relay with 0% packet loss.
1451
1452 **Phase 3 Automatic network detection:** Because 'TAILSCALE_ALLOW_LAN_P2P=false' breaks P2P on trusted
1453 hotspots that genuinely don't have AP isolation, a '.env' toggle and a full auto-detection system
1454 were implemented:
1455
1456 - **'.env' toggle:** 'TAILSCALE_ALLOW_LAN_P2P=true/false' static fallback, used only if auto-detection
1457 cannot run.
1458 - **'watcher.sh' auto-detection ('detect_lan_p2p_mode')** On every network change, 'watcher.sh' runs two
1459 checks:

```


1419 1. ****WPA2-Enterprise detection:**** 'airport -I' is used to read the current Wi-Fi 'link auth' field. If
the auth type does not contain '-psk' (e.g., it is plain 'wpa2' = 802.1x), the network is classified
as enterprise and P2P is forced off. *(Note: the 'airport' binary was removed in macOS Sonoma 14.4
the detection now uses 'system_profiler SPAirPortDataType'; see 15.11 for the full migration.)*

1420 2. ****AP Isolation probe:**** For WPA2-Personal networks, a short 'ping' is sent to the default gateway.
If the gateway responds (it always should on a real home/hotspot network), P2P is allowed. If not,
AP isolation is inferred and P2P is forced off.

1421 - The result ('true' or 'false') is written to '.lan_p2p_detected' in the repo root.

1422 - ****'routing-fix.sh' dynamic reading:**** 'add_tailscale_p2p_bypass()' re-reads '.lan_p2p_detected' on
every 30-second loop iteration and atomically adds or removes the RFC1918 DROP rules. ****No container
restart is needed when switching networks.****

1423

1424 ****Known Unknowns (Pending Verification):****

1425 - The gvpnproxy SNAT source port mangling theory is mechanistically sound and backed by observed behavior (gvpnproxy's UDP proxy changelog has documented edge cases in this exact code path), but has not yet
been confirmed via 'tcpdump -i en0 udp portrange 40000-60000' while triggering a P2P handshake from
the phone. A packet capture cross-referenced with 'tailscale ping' endpoint reports on the phone
side would confirm the theory definitively.

1426 - Claude's recommendation: run 'tcpdump -i en0 udp port 41642 or portrange 40000-60000' on the Mac while
triggering the handshake from the phone to see whether the reply leaves with a different source port
than '41642'.

1427 - ****Client-Side VPN Cycling (Roaming Recovery):**** An observation (July 10, 2026) suggests that when a
hung DNS probe state is triggered by roaming between Access Points (APs) on a WPA2-Enterprise
network, simply toggling the VPN connection (Tailscale) off and on locally on the client phone
immediately resolves the issue. You do not need to cycle the phone's physical Wi-Fi. This reinforces
the theory that roaming on Enterprise networks leaves a stale/poisoned P2P endpoint in the phone's
Tailscale magicsock state, which gets instantly flushed upon a local VPN restart. (Status: Possible
but not formally verified).

1428

1429 ##### 15.7.2 July 6, 2026: Packet Filter (pfctl) Defaulting Local LAN Rules to TCP-Only

1430 ****Symptom:**** Devices on the exact same local Wi-Fi network as the gateway were experiencing ~500ms
latency to the gateway instead of the expected ~80ms.

1431

1432 ****Root Cause:**** When writing the 'pf.conf' rules to whitelist local LAN traffic ('pass out quick on
\$ext_if to { 192.168.0.0/16, ... }'), the rule omitted an explicit protocol. The macOS 'pfctl'
compiler implicitly applies state tracking to all 'pass' rules. However, because state tracking ('
keep state') natively evaluates TCP sessions, 'pfctl' automatically appended the 'flags S/SA'
modifier to the compiled rule in the kernel. This silently restricted the whitelist to ****TCP only****,
causing all local UDP traffic to be dropped by the default-deny kill-switch. Since Tailscale relies
on UDP port 41641 for direct P2P connections, the local devices were unable to hole-punch and were
forced to fall back to routing traffic through a remote Tailscale DERP relay, massively inflating
latency.

1433

1434 ****Resolution:**** Split the single implicit rule into explicit 'proto tcp', 'proto udp', and 'proto icmp'
rules in 'scripts/pf.conf' to guarantee the macOS compiler allows all three core transport protocols
on the local subnet without mutating the rule into a TCP-only lock.

1435

1436 ##### 15.7.3 Incident Post-Mortem: PF Kill-Switch Bricks Host Internet in SOCKS5 Fallback Mode (July 10,
2026)

1437 ****Symptom:**** When the pre-flight checks fail (e.g. because host Tailscale was unauthenticated/logged out)
, the script falls back to SOCKS5 proxy mode but the host immediately loses all internet
connectivity and tailscaled reports 'You are logged out... connect: no route to host'. Even running
'sudo tailscale up' to log in fails because the connection to the control plane times out.

1438

1439 ****Root Cause:**** An architectural mismatch. The macOS Packet Filter (PF) Kill-Switch works at the IP level
: it blocks all outbound traffic on the physical interface ('en0') except to explicitly whitelisted
VPN endpoints, expecting all other traffic to go through the virtual VPN interface ('utun*'). In
SOCKS5 fallback mode, the exit-node ('utun*') is NOT enabled; instead, only specific applications (like
browsers) use the localhost SOCKS5 proxy, while all other host traffic (including the
'tailscaled' daemon's UDP packets) continues to go through 'en0'. Because the script still enabled
the PF Kill-Switch during SOCKS5 fallback, it blocked all non-SOCKS5 traffic on 'en0', completely
bricking the host's internet and locking the Tailscale daemon out of the control plane.

1440

1441 ****Fix (July 10, 2026):**** Modified 'toggle.sh' to remove the 'enable_killswitch' call from the SOCKS5
fallback path. The firewall kill-switch is now strictly reserved for exit-node VPN mode, ensuring
SOCKS5 fallback leaves the host's direct internet route open for system daemons to authenticate.

1442

1443 ##### 15.7.4 Incident Post-Mortem: Host Tailscale Logouts due to Aggressive reset Flags (July 10, 2026)

1444 ****Symptom:**** Every time the user runs 'toggle.sh' or 'recover.sh' which disconnects/reconnects the host
client, the host client randomly gets logged out, forcing them to re-run 'sudo tailscale up' to
authenticate.

1445

1446 ****Root Cause:**** An overly aggressive configuration reset. The host-side 'tailscale up' calls (used in
'disconnect_tailscale_host' and the startup/enable exit node paths) all passed the '--reset' flag. On
macOS, '--reset' resets the entire configuration state and authentication token cache of the daemon
. Since the script does not pass a 'TS_AUTHKEY' to the host's commands (which are meant for the user's
private interactive client), the '--reset' flag effectively logged the user out of their tailnet,
requiring interactive re-login.

1447

1448 ****Fix (July 10, 2026):**** Removed the '--reset' flag from all host-side 'tailscale up' invocations in
'toggle.sh' and 'scripts/common.sh'. This ensures that exit-node state transitions (enabling/

disabling) are handled safely while preserving the host client's authenticated session.

15.7.5 Incident Post-Mortem: macOS Tailscale Flag Requirement Abort (July 10, 2026)

Symptom: When the 'toggle.sh' stop trap or the roaming watcher triggered, the gateway failed to shut down or disconnect properly. The 'output.log' showed the error: 'Error: changing settings via 'tailscale up' requires mentioning all non-default flags. To proceed, either re-run your command with --reset...'

Root Cause: In a previous attempt to stop host-side logouts (15.7.4), we removed the '--reset' flag from all 'tailscale up' invocations. However, if the Tailscale daemon on macOS is already running with non-default flags (like '--ssh' or '--accept-routes'), invoking 'tailscale up' with only a subset of flags (like '--exit-node=') triggers a strict safety check in the Tailscale CLI that aborts the command completely. Because the command aborted, exit-node routing remained stuck.

Fix (July 10, 2026): Migrated all specific preference changes in 'toggle.sh' and 'scripts/common.sh' from 'tailscale up' to a two-step sequence:

1. 'tailscale up' (with no flags) to safely ensure the interface is brought up using the current authenticated state.
2. 'tailscale set --exit-node=...' to modify the specific target preference cleanly without triggering the missing flags error or requiring '--reset'.

15.8 Docker / Colima Lifecycle

15.8.1 Colima VM Memory / Swap Thrashing Latency

Symptom: After running nullexit flawlessly for over 24 hours straight to test stability, the 0.5GB VM RAM configuration began experiencing severe network latency on the Tailscale mesh later in the day.

Root Cause: Services (like AdGuard, Tailscale, Docker logs) slowly accumulate cache and state over extended periods. Under the tight 512MB physical RAM limit, this slow creep forced the Linux kernel to aggressively swap memory to the 512MB SSD swap file. The constant disk I/O thrashing blocked process execution, causing DNS resolutions and packet forwarding to delay significantly (making the internet feel "very slow").

Proof (Empirical Observation): While in this OOM-thrashing state, direct DNS queries through the Tailscale mesh ('dig @100.100.21.8 google.com') would intermittently hang or take several seconds to resolve. After restarting with the 600MB configuration, the exact same query immediately dropped to a lightning-fast **41ms** response time. (Note: Querying the mapped host port via '127.0.0.1:5354' will always time out due to Gluetun's strict leak-prevention firewall blocking non-VPN ingress traffic).

Fix: Increased VM base physical RAM to 600MB ('--memory 0.6') and reduced the swap file to 400MB. This provides enough native RAM headroom for long-running state caches without forcing heavy I/O swapping. Subdomain deduplication still reduces blocklists by ~60%. Memory profiles ('light'/'medium'/'heavy') let users trade off coverage for memory.

15.8.2 Missing iptables in routing-fix Container

Root Cause: 'alpine:3.20' does not have iptables installed. Commands failed silently with '2>/dev/null || true'.

Fix: Updated 'docker-compose.yml' to 'apk add --no-cache iptables iproute2' before 'scripts/routing-fix.sh'.

15.8.3 DOCKER_GW Variable Parsing Bug

Root Cause: 'ip route show default' printed two lines; 'awk '{print \$3}'' picked up both, causing route commands to fail.

Fix: Added 'head -1' to the parsing chain.

15.8.4 Hard Reboots Leave Stale Docker Socket in Colima VM

Problem: If the macOS host machine is subjected to a hard reboot, sudden power loss, or a kernel panic, the Colima VM running in the background is abruptly killed. Upon next boot, running 'toggle.sh' resulted in the contradictory output:

'Colima is already running.' followed by 'Cannot connect to the Docker daemon at unix:///..colima/default/docker.sock.'

Root Cause: The hard crash prevents the inner Docker daemon from executing its graceful shutdown routines. It leaves behind a corrupted 'docker.sock' file inside the VM. On boot, the outer VM spins up (hence "already running"), but the inner Docker daemon sees the stale socket, assumes another instance is running, and crashes.

Fix: Added an Auto-Healing routine directly into 'toggle.sh' (Step 4b). The script now explicitly tests the Docker daemon with 'docker ps'. If the daemon is unresponsive despite Colima reporting as active, it assumes a stale socket and automatically runs 'colima restart' in the background to cleanly rebuild the VM and socket before proceeding.

15.8.5 Cloudflare WARP 24-Second Startup Delay (UDP Session Rate-Limiting)

Context: When running 'toggle.sh' to turn the gateway ON, 'docker compose up -d' often hangs for exactly 24-25 seconds waiting for the 'warp' container to become 'healthy'. Users might mistakenly blame the Go 'rule-compiler' processing 400k+ rules for this delay, as Docker prints 'rule-compiler Exited 24.3s' at the end of the wait.

1490 ****Root Cause:**** The 'rule-compiler' actually finishes its work in <2 seconds. The 24-second blockage is entirely due to Cloudflare WARP's edge server rate-limiting. WireGuard is a stateless protocol with no "disconnect" packet. When the toggle is turned OFF, the old UDP session state remains active on Cloudflare's backend for 20-30 seconds. When the toggle is instantly turned back ON, Docker starts a new WireGuard connection from a new ephemeral UDP source port but uses the ****same static cryptographic key**** (from '.env'). Cloudflare detects this as a potential replay attack or key-sharing abuse, and intentionally drops all packets (rate-limits) for the new connection until the old session's ghost state fully expires (~20-25 seconds).

1491 ****Result:**** Gluetun's healthcheck fails repeatedly every 2 seconds until Cloudflare lifts the penalty box, at which point traffic flows and Docker unblocks the rest of the startup sequence. This is a known, expected behavior of reusing static keys rapidly on Cloudflare's network and cannot be bypassed.

1492

1493 **#### 15.8.6 July 4, 2026: Multi-stage Docker Builds for Go Ports & Teardown Hang Fix**

1494 ****Symptom:**** The Python implementations of 'logger.py' and 'sync-rules.py' were slow and required a heavy python runtime inside Alpine containers. Also, 'routing-fix' container teardown was hanging for 30s during 'toggle.sh' shutdown.

1495 ****Root Cause:****

1496 - Python is slower than Go and installing it dynamically via 'apk add' added overhead and complexity to the containers.

1497 - Docker containers ignore 'SIGTERM' if the init process doesn't handle it, causing a full 30s timeout on shutdown.

1498 ****Resolution:****

1499 - Ported 'logger' and 'sync-rules' to Go using Goroutines for massive concurrent speedups.

1500 - Implemented ****Multi-stage Docker builds**** (using 'golang:1.22-alpine' as builder) to compile the static binaries inside Docker. The final containers are raw Alpine with zero dependencies.

1501 - Added '--build' to 'docker compose up -d' in 'toggle.sh' to ensure updates are consistently applied on boot.

1502 - Added 'stop_grace_period: 1s' to 'routing-fix' in 'docker-compose.yml' which eliminates the 30-second teardown hang instantly.

1503 - Implemented a cryptographic integrity checker ('scripts/crypto.sh') using HMAC-SHA256 and 'NULLEXIT_SEED' in '.env' to prevent tampering of bash scripts. (See also 15.9.4.)

1504

1505 **#### 15.8.7 Colima 'start' Post-Provision Hang (~2 min) & the Docker-Readiness Poll (July 14, 2026)**

1506 ****Symptom:**** 'toggle.sh' took ~197s of wall-clock time to bring the gateway up, and almost the entire delay lived in a single step: 'colima start'. Timestamps in 'output.log' showed the VM reaching 'READY' and creating the 'colima' Docker context in ~8-12 seconds, after which 'colima start' simply ***did not return*** for ~110 more seconds until the 'run_with_timeout 120' watchdog 'SIGKILL'ed it. The rest of the boot (containers, routing) then proceeded normally.

1507

1508 ****Root Cause:**** A colima-internal hang that occurs ***after*** 'Successfully created context colima' is printed. It is ****mode-independent****. It was initially misattributed to '--network-address' / '--network-mode bridged' needing the 'socket_vmmnet' helper, but a controlled test with plain user-mode networking ('colima start --memory 0.6 --vm-type vz', no '--network-address') reproduced the identical ~120s hang (04:40:34 04:42:34, watchdog-killed) while Docker was fully usable the entire time. Crucially, killing the hung 'colima start' process does ****not**** stop the VM and does ****not**** corrupt state 'colima status' correctly reports 'running' afterwards and 'docker' keeps working via the 'colima' context. The ~110s was therefore pure dead time waiting on a CLI that had already done its real work.

1509

1510 ****Fix:**** 'scripts/common.sh' now provides 'colima_start_until_ready()'. It launches 'colima start "\$@"' in the background and polls 'docker --context colima info' every 3s. The moment Docker answers (~12s in practice) it stops waiting, reaps the (usually still-hung) starter with 'kill', runs 'docker context use colima' to guarantee the default context is set, and returns 0. If Docker never answers within a 90s cap it returns non-zero and 'toggle.sh' continues to the existing Step 4b 'docker ps' auto-heal (15.8.4). Both Colima start call sites in 'toggle.sh' the LAN-P2P bridged/shared path and the user-mode fast path now go through this helper instead of 'run_with_timeout 120 colima start'.

1511

1512 ****Result:**** The Colima phase dropped from ~120s to ~12s and total 'toggle.sh --restart' wall time from ~197s to ~106s, with the double-tunnel and direct hostcontainer P2P confirmed intact after the change. The key insight: treat "Docker is reachable" not "the 'colima start' CLI returned" as the definition of ***started***.

1513

1514 **#### 15.8.8 Toggle-On Startup Optimizations: Rule-Compiler Off the Critical Path (July 14, 2026)**

1515 ****Context:**** A genuine cold toggle-on (VM bounced for RAM, WARP down) measured ****95s****. Profiled per-phase with the now-fixed 'DEBUG_TRACE' (15.11.8-A): Colima cold start ****19s**** (near floor), 'docker compose up' ****45s****, tailscale connect+poll ****21s****, final checks ****7s****.

1516

1517 ****Two wrong hypotheses first (documented so they aren't re-tried):**** (1) that BuildKit '--build' was the ~30s cost it wasn't; every layer was already 'CACHED', so gating '--build' saved only ~2s. (2) that 'docker compose exec' was slow in the poll loop it isn't (0.18s steady-state vs 0.06s for 'docker exec'); the ~5s/iteration was 'tailscale status' ****blocking during cold connect**** and hitting the 'run_with_timeout 5' cap.

1518

1519 ****Real bottleneck (evidence-backed):**** the 45s 'compose up' was ****not**** WARP. On a genuine cold start WARP goes healthy fast (tor starts right with it no 15.8.5 same-key penalty; that only bites rapid offon). The long pole is the ****rule-compiler re-deduping ~340k rules inside the 600MB VM (~28s), every toggle**** 'adguardhome' started 28s after warp because it waited on 'rule-compiler: service_completed_successfully', even though all 16 remote lists loaded from cache unchanged.

1520

1521 ****Fix** hash-gated, off the critical path (kept in-container):

```

1522 - 'rule-compiler' is now a compile-profiled service, excluded from 'docker compose up'. '
      adguardhome' and 'routing-fix' no longer 'depends_on' it they boot instantly on the 'compiled_rules
      .txt' / 'ip_blocklist.ipset' already in './adguard/work'.
1523 - 'toggle.sh' runs it on-demand via 'docker compose run --build --rm rule-compiler', gated on '
      compute_rules_hash()'** = sha256 of 'black_list.txt' + 'white_list.txt' (the lists the user controls
      ). Recompile only when that hash changes, when an artifact is missing, or when the compiled file is
      older than 'RULES_MAX_AGE_DAYS' (default 7, so the remote blocklists still refresh occasionally). A
      compile failure is non-fatal we keep the previous compiled rules rather than bricking the
      tunnel (ad-block the kill-switch).
1524 - Also trimmed the tailscale-connect poll timeouts (status '5s3s', 'ip -4' '10s5s') to reduce overshoot.
1525
1526 Why NOT run the compiler natively on the Mac host (the tempting idea): rejected on purpose. There is
      no Go on the host, so "native" means either 'brew install go' (a host toolchain dependency that
      drifts the project from *portable+secure-on-any-device* toward *faster-on-mine*) or committing a
      prebuilt arch-specific binary blob you'd then have to trust/sign. Worse, it would make the host
      write into the './adguard/work' bind-mount violating the deliberate "single allowed writer"
      is the rule-compiler container" invariant (hostcontainer UID/perms mismatches corrupt AdGuard's
      BoltDB store). Staying in-container preserves both properties; the gate delivers the speed.
1527
1528 Result: unchanged toggle 'compose up' 45s 9s, total '--restart' 95106s 63s, with AdGuard
      still serving the full 342,519 rules (it reuses the existing compiled file) and the double-tunnel +
      direct hostcontainer P2P intact. Editing 'black_list.txt'/'white_list.txt' flips the hash one-off
      recompile new rules take effect. Both lists were added to the 'crypto.sh' signed set (they're
      security inputs), so editing them requires a re-sign which is the same moment you'd want the
      recompile. Local state files '.build_hash' / '.rules_hash' are gitignored.
1529
1530 15.9 Permissions, Crypto & Security Hardening
1531
1532 15.9.1 Sudo Credential Caching Removed
1533 Problem: The script used 'sudo -v' at startup to cache sudo credentials, plus a background '
      SUDO_KEEPER_PID' loop refreshing them every 60 seconds. This required interactive password entry
      even with NOPASSWD sudoers configured, because 'sudo -v' itself prompts.
1534
1535 Fix: Removed the entire 'sudo -v' + 'SUDO_KEEPER_PID' section. All privileged commands now use 'sudo
      -n' (non-interactive), which works silently because the user has NOPASSWD rules in '/etc/sudoers.d/
      nullexit' for the specific commands needed ('networksetup', 'python3', 'dscacheutil', 'killall', '
      pkill', 'route', 'ifconfig').
1536
1537 15.9.2 Background Sudo Failures
1538 Context: The '--post-wake' script is intentionally designed to skip the 'sudo -v' interactive prompt
      because it is launched by 'watcher.sh' running via 'launchd' in the background (which has no TTY and
      cannot accept password input). Thus, it relies entirely on 'sudo -n' (non-interactive sudo) for all
      its internal routing changes.
1539 Problem: If the user has not configured the '/etc/sudoers.d/nullexit' file properly, any automated
      background wake event will silently fail: 'sudo -n' commands drop out, the WARP bypass routes fail
      to apply, and the 'warp' container loses internet connectivity.
1540
1541 15.9.3 Tailscale Hijacking the Default Route (PHYSICAL_GW Bug)
1542 Problem: When '--post-wake' runs, it clears the routing table using 'sudo route -n flush' to ensure a
      clean slate after the Wi-Fi card roams or wakes up. Because it skips bouncing the Wi-Fi interface (
      to make the reconnect faster), macOS's DHCP client does not automatically rebuild the physical
      default route. To counteract this, the script must manually restore the physical default route.
      Originally, the script tried to capture the physical gateway via 'route get default'. However,
      because the gateway is already running and Tailscale is active, the default route is often already
      pointing to 'utunX'. When this happens, 'route get default' does not output a 'gateway:' field (it
      only outputs the 'interface:'), causing the captured 'PHYSICAL_GW' variable to be empty.
1543 When 'PHYSICAL_GW' is empty, the physical default route is never restored, causing the 'en0' interface to
      be isolated from the physical network. The WARP bypass routes then fall back to targeting '-
      interface en0' (instead of routing via the gateway), which breaks WireGuard's remote connections
      completely.
1544 Fix: The script now bypasses the OS routing table entirely and directly queries the hardware DHCP
      lease ('ipconfig getpacket en0') to reliably extract the physical router IP, ensuring the physical
      route is correctly restored even when Tailscale is active.
1545
1546 15.9.4 July 4, 2026: Consolidation of Core Bash Scripts (Refactoring)
1547 Symptom: Over 200 lines of bash logic were directly duplicated across 'toggle.sh' and 'recover.sh' (e
      .g. 'restart_tailscaled_daemon', 'stop_sleep_prevention', WARP bypass routes). This caused drift
      when one script was updated without the other.
1548 Root Cause: Historical separation of responsibilities allowed 'recover.sh' to grow complex alongside
      'toggle.sh'.
1549 Resolution:
1550 - Massively refactored both scripts by extracting all duplicated networking logic, PID lifecycle
      management, and environmental configuration down to 'scripts/common.sh'.
1551 - Specifically, the 'stop_sleep_prevention', 'stop_dns_watcher', 'stop_host_leak_probe', and '
      stop_warp_watcher' functions were collapsed into a single 'stop_pidfile_daemon()' abstraction.
1552 - Proxy disable loops were abstracted to 'disable_all_proxies()'.
1553 - The '162.159.192.1/193.1' WARP edge IPs are now dynamically resolved from '.env' instead of hardcoded.
1554
1555 15.9.5 Threat Model: Local Proxy Discovery
1556 The Tor SOCKS proxy and Honey-Port Tripwire only bind to '127.0.0.1' (loopback). The real security
      boundary here is "can an unprivileged local process reach the loopback interface," not "does the

```

```

malware know the port number."
1557
1558 The port-randomization and the Honey-Port trap only catch blind or generic port-scanners that do not
already know where to look. They **do not protect** against a process that directly reads the '/tmp/
nullexit-ports.env' file, greps the 'toggle.sh' memory space, or runs 'lsof -i' to inspect open
socket bindings.
1559
1560 Because modifying file ownership on '/tmp/nullexit-ports.env' via 'chown'/'chmod' to restrict read-access
introduces an unavoidable Time-Of-Check to Time-Of-Use (TOCTOU) race condition (the file is created
user-owned before privilege escalation, meaning file descriptors can be snatched), it is
deliberately left as a standard user-owned file.
1561
1562 The actual, proper mitigation for preventing a malicious local process from reaching the proxy is not
hiding the portit is running a host-level outbound firewall with strict localhost/loopback filtering
enabled (e.g., LuLu 4.3.0+ or Little Snitch). These firewalls gate access by cryptographic process
identity (binary signature) at the kernel/network-extension level, rather than by port secrecy.
1563
1564 **Verifying Localhost Filtering (The Rogue Binary Test).** To empirically validate that the host firewall
properly intercepts local proxy hijacking, you can compile and execute a completely unknown, un-
whitelisted "rogue" binary to attempt a connection to the proxy port:
1565
1566 '''c
1567 // lulu-test.c
1568 #include <stdio.h>
1569 #include <stdlib.h>
1570 #include <arpa/inet.h>
1571
1572 int main(int argc, char *argv[]) {
1573     int port = atoi(argv[1]);
1574     int sock = socket(AF_INET, SOCK_STREAM, 0);
1575     struct sockaddr_in server = { .sin_family = AF_INET, .sin_port = htons(port) };
1576     server.sin_addr.s_addr = inet_addr("127.0.0.1");
1577
1578     printf("Rogue binary attempting connection to 127.0.0.1:%d...\n", port);
1579     if (connect(sock, (struct sockaddr *)&server, sizeof(server)) < 0) {
1580         printf("Connection BLOCKED by firewall.\n");
1581         return 1;
1582     }
1583     printf("Connection SUCCESSFUL (Firewall bypassed!).\n");
1584     return 0;
1585 }
1586 '''
1587 Compile with 'gcc -o lulu-test lulu-test.c'. When executed against the '$TOR SOCKS_PORT', a properly
configured host firewall (with "Allow Loopback" disabled in LuLu Preferences) will immediately pause
the socket execution and generate a desktop alert for the unrecognized binary signature. This
physically stops targeted local malware from exfiltrating data through the Tor tunnel, proving the
threat model mitigation.
1588
1589 #### 15.9.6 Unlocking Mode-Locked Output Files Without 'chmod' ( 'unlock-files.sh')
1590 **The one-command fix:** 'bash scripts/unlock-files.sh' replaces every known locked inode in the project
with a fresh writable one. No 'chmod'. Covers '.env' (mode '000'), 'adguard/work/userfilters/
compiled_rules.txt', 'ip_blocklist.ipset', 'cache/*.txt', 'cache/ip/*.txt', and 'adguard/work/data/
filters/*.txt' (mode '0444'). Run it once after setup or after pulling an old repo; 'toggle.sh' and
'sync-rules.py' will then work without permission errors.
1591
1592 **Context:** The nullexit project has a strict project-wide policy of 'no chmod from scripts' (see README
6). This rule exists because every prior 'chmod 0444' (post-write tamper-proof lock) and 'chmod
000' (post-toggle '.env' lock) call from 'scripts/sync-rules.py' / 'toggle.sh' could leave a file
permanently unreadable on disk if the script exited early, was killed mid-flight, or simply set a
restrictive mode without ever being re-flipped. We've stripped every 'chmod' from the codebase. But
files **written by older versions of those scripts** are still on disk in those restrictive modes
('0444' for 'compiled_rules.txt', 'ip_blocklist.ipset', 'data/filters/<id>.txt'; '000' for '.env'
after end-of-toggle lock). Re-running 'toggle.sh' or 'scripts/sync-rules.py' against those leftovers
hits 'PermissionError: [Errno 13] Permission denied' immediately.
1593
1594 **Why the stale permissions also block 'mv'/'rm' of the parent folder:** The restrictive modes ('000' on
'.env', '0444' on output files) don't just block writes they prevent the kernel from unlinking the
file's dentry during a 'mv' or 'rm' of the **containing directory** ('adguard/work/'). macOS
enforces this at the VFS layer: you own the directory, but you can't delete a directory that
contains entries you can't write to. This made the entire 'adguard/work/' tree unmovable/undeletable
until the locked inodes inside it were replaced. 'unlock-files.sh' resolves this permanently by
swapping each locked inode for a fresh '0644' one via atomic rename ('cp' + 'mv'), which operates on
the directory entry rather than the file contents no 'chmod' called, no permission bypass needed.
1595
1596 **Problem:** You (the owner) cannot 'cat', 'cp', 'rm', '>' redirect, or any normal POSIX write against a
'mode=000' file even though you own it the basic mode bits block ALL access for owner, group, and
other. The script does not call 'chmod' and you want a permanent fix that never relies on a chmod
from any future run either.
1597
1598 **Solution:** Replace the locked inode with a fresh readable one. 'mv' is atomic; mode bits live in the
inode, not the directory entry. Two flavors cover every case we hit on a real machine:

```

```

1599
1600 **Flavor A locked file is mode '000' (owner can't even READ it):**
1601 NOPASSWD sudoers ('/etc/sudoers.d/nullexit') already includes '/usr/bin/python3'. So we read via 'sudo
python3' (which has the DAC override root has) and pipe the bytes around as base64 in user space,
where the redirect happens with the user's normal umask = '0644':
1602
1603 '''bash
1604 sudo -n /usr/bin/python3 -c "import base64,sys; sys.stdout.write(base64.b64encode(open('<FILE>','rb')).
read()).decode())" > /tmp/snap.b64
1604 base64 -d < /tmp/snap.b64 > <FILE>.new
1605 mv -f <FILE>.new <FILE>
1606 ls -la <FILE> # mode is now 0644
1607 '''
1608 The old 'mode=000' inode is unlinked by 'mv'; the new 'mode=0644' inode (created by base64-decode via the
calling user's umask) takes the path. No chmod ever called. The parent directory just needs to be
writable by you (it always is, since you own it).
1609
1610 **Flavor B locked file is mode '0444' (you can READ but cannot WRITE; 'scripts/sync-rules.py' needs to
re-write):**
1611 You own the file and can read it, you just cannot truncate/overwrite it. The simplest path is to rename
it aside and let the writer create a fresh inode from scratch (which gets the umask-default '0644'):
1612 '''bash
1613 mv <FILE> /tmp/old.<FILE>.$$
1614 python3 scripts/sync-rules.py # now writes <FILE> cleanly at mode 0644
1615 '''
1616 Or apply Flavor A if you'd prefer to preserve the contents while resetting the mode.
1617
1618 **Why this is the canonical recovery, not a hack:**
1619 - Mode bits live in the inode, not the path. 'mv -f' replaces the path's inode reference; the old locked
inode drops out of the directory entirely (dentry eviction) and loses its last link, so the kernel
reclaims it. The new inode (whatever it is: a freshly-written one, or a base64-decoded copy) gets
the path.
1620 - The only prerequisite to apply Flavor B's 'mv' is: you own the parent directory (so you can unlink
entries from it) AND you have a NOPASSWD-capable binary that can read the byte stream if mode
prohibits user read.
1621 - This pattern generalizes. Any time a script ends up producing a "locked file that the next run can't
update" situation, the same trick applies without a single chmod anywhere.
1622
1623 **Empirical verification (user machine, July 1, 2026):**
1624 - '.env' was 'mode=000' (owner $USER, i.e. you): Flavor A unblocked it in 3 commands, no chmod.
1625 - Before: '----- 1 $USER staff 899 Jun 28 07:07 .env'
1626 - After: '-rw-r--r--@ 1 $USER staff 899 Jul 1 20:39 .env'
1627 - 'docker compose config' immediately picked up 'TS_AUTHKEY' + 'WIREGUARD_PRIVATE_KEY' substitution.
1628 - 'compiled_rules.txt', 'ip_blocklist.ipset', 'data/filters/1782645604.txt' were 'mode=0444': Flavor B ('
mv'-aside) unblocked all three.
1629 - scripts/sync-rules.py then ran clean: 279587 block rules + 171 allow rules + 16710 IP entries
compiled; new files came out at '0644'.
1630 - 'toggle.sh' then went end-to-end in 48s (warp / routing-fix / adguardhome / tailscale all healthy,
gateway mesh-joined, DNS hijack verified single-server).
1631
1632 **When this trick is appropriate vs. inappropriate:**
1633 - You own the parent directory and the file (typical desktop scenario).
1634 - The lock is a leftover from a removed 'chmod' call that you want off disk forever.
1635 - NOPASSWD sudo includes at least one interpreter ('python3' on this system) that can be used to read
mode-0 files. If it does not, add it first: '<USER> ALL=(ALL) NOPASSWD: /usr/bin/python3'.
1636 - The file is owned by another user (root, another account) and the lock is intentional respect it; do
not bypass another user's security boundary.
1637 - The parent directory is owned/writable only by another user escalate properly via sudo, or stop and
ask the user.
1638 - You do not actually own the file and there is a legitimate reason for its lock state restore the file
via a trusted source rather than circumventing the perms.
1639
1640 **Reusable cookbook:**
1641
1642 '''bash
1643 # Quick diagnostic: figure out which flavor you need.
1644 WHOAMI=$(whoami)
1645 ls -la <FILE>
1646 stat -f 'mode=%Sp owner=%Su group=%Sg' <FILE>
1647 # mode=----- or mode=---r----- : you cannot even read it Flavor A.
1648 # mode=r--r--r-- but writer fails : you are blocked from write but can read Flavor B.
1649 # mode=rw-r--r-- : not actually locked at all; unrelated write failure.
1650
1651 # Flavor A (mode=000):
1652 sudo -n /usr/bin/python3 -c "import base64,sys; sys.stdout.write(base64.b64encode(open('<FILE>','rb')).
read()).decode())" > /tmp/snap.b64
1653 base64 -d < /tmp/snap.b64 > <FILE>.new && mv -f <FILE>.new <FILE> && rm -f /tmp/snap.b64
1654
1655 # Flavor B (mode=0444, file is readable):
1656 mv <FILE> /tmp/old.<FILE>.$(date +%s)
1657 # (let the original writer recreate <FILE>; new file lands at mode 0644)
1658

```



```

1659 # Verify after either flavor:
1660 ls -la <FILE> # mode should be 0644
1661 <your normal validation command>
1662 ''
1663
1664 ### 15.10 Tor Container
1665
1666 ##### 15.10.1 July 11, 2026: Tor Container Hardening & obfs4proxy Restoration
1667 **Symptom:**
1668 1. The Tor container entered a fatal crash loop upon boot, throwing 'exec: "obfs4proxy": executable file
    not found in $PATH' when 'TOR_USE_BRIDGES=true'.
1669 2. When the user pasted Tor bridge lines containing comments ('#') or blank lines into 'tor-bridges.txt',
    the container failed to validate the file properly and crashed.
1670
1671 **Root Cause:**
1672 - **Bug 1 (Missing Pluggable Transports):** The Tor Dockerfile was built on 'debian:bullseye-slim', which
    had silently dropped or failed to resolve the 'obfs4proxy' package in its latest apt repositories.
1673 - **Bug 2 (Fragile Validation):** The 'entrypoint.sh' bridge validation check merely checked '[ -s tor-
    bridges.txt ]' (file size > 0). It did not verify if the file actually contained valid, uncommented
    bridge lines. Passing empty lines or comments tricked the validation script into appending invalid '
    Bridge' directives to the 'torrc', causing the Tor daemon to instantly exit.
1674
1675 **Resolution:**
1676 - **Upgraded Base Image:** Shifted the Tor Dockerfile base image to 'debian:bookworm-slim', restoring
    access to the 'obfs4proxy' pluggable transport package.
1677 - **Robust Bridge Validation:** Replaced the fragile '[ -s ]' check with a strict 'grep -vE '\s*(#|$\)'
    command that strips comments and empty lines. If no valid lines remain, the container safely falls
    back to standard public guard relays instead of crashing, ensuring Tor remains available even with a
    misconfigured bridge file.
1678 - **Image Pinning:** Explicitly pinned 'image: nullexit-tor:v1.0.0' in 'docker-compose.yml' to prevent
    arbitrary local builds from drifting to ':latest'.
1679
1680 ##### 15.10.2 July 12, 2026: Tor ControlPort Dynamic Password Authentication & User Config
1681 **Symptom:** Querying the Tor Control Port (e.g., via the '/sweep' script or 'check_circuits.py')
    returned empty responses or connection errors. Logs showed that Tor automatically closed its
    ControlPort:
1682 'You have a ControlPort set to accept unauthenticated connections from a non-local address. ... That's so
    bad that I'm closing your ControlPort for you.'
1683
1684 **Root Cause:** Docker port mapping (especially under Colima on macOS) requires container sockets to bind
    to '0.0.0.0' to be reachable from the host. However, when Tor's ControlPort is bound to '0.0.0.0' (
    non-local) with no authentication, Tor closes it by default as a safety precaution. Sourcing SOCKS5/
    ControlPort to '127.0.0.1' inside the container was not an option as it broke the Docker bridge
    routing path.
1685
1686 **Resolution:**
1687 - **Dynamic Password Authentication:** Updated the Tor container's 'entrypoint.sh' to read 'TOR_PASSWORD'
    and dynamically generate its HMAC hash via 'tor --hash-password', adding 'HashedControlPassword <
    hash>' to 'torrc'. This secures the ControlPort on '0.0.0.0' so Tor keeps it open.
1688 - **Unified Credentials in '.env':** Exposed 'ADGUARD_USER=admin' and 'ADGUARD_PASSWORD=nullexit' in '.
    env' to serve as the unified gateway credentials. Sourced 'TOR_PASSWORD' directly from '
    ADGUARD_PASSWORD' in 'docker-compose.yml'.
1689 - **Client Authentication:** Updated 'scripts/check_circuits.py' to fetch 'TOR_PASSWORD' (defaulting to '
    ADGUARD_PASSWORD' or 'nullexit') and authenticate with the ControlPort using 'AUTHENTICATE "<
    password>"' to regain access.
1690
1691 > See also 11.3 (Tor Container Kill-Switch Inheritance & Fail-Closed Egress Verification) for the design-
    level fail-closed guarantee.
1692
1693 ### 15.11 macOS Platform Quirks
1694
1695 ##### 15.11.1 macOS Application Firewall Silently Blocking AirDrop & Continuity
1696 **Problem:** Users of complex network extensions (Tailscale, LuLu, Docker, etc.) often run into an issue
    where AirDrop, AirPlay, and Universal Clipboard silently fail, even with the gateway inactive and Wi-
    Fi/Bluetooth correctly configured. The internal Apple Wireless Direct Link ('awdl0') interface
    appears healthy, but connections never establish.
1697
1698 **Root Cause:** The built-in macOS Application Firewall has a specific list of explicitly blocked
    applications. Apple's own core networking services (e.g., '/usr/libexec/sharingd', '/usr/libexec/
    rapportd', '/usr/libexec/avconferenced') can be silently added to the "Block incoming connections"
    list either through accidental "Deny" clicks on permission prompts, execution of aggressive privacy-
    hardening scripts, or a known macOS bug that retains strict blocks even after toggling the "Block
    all incoming connections" setting off. Since these are background daemons, macOS provides no visible
    error.
1699
1700 **Fix:** The firewall rules must be cleared via the terminal (or System Settings > Network > Firewall >
    Options):
1701 ''bash
1702 sudo /usr/libexec/ApplicationFirewall/socketfilterfw --unblockapp /usr/libexec/sharingd
1703 sudo /usr/libexec/ApplicationFirewall/socketfilterfw --unblockapp /usr/libexec/rapportd
1704 ''

```


1705 Once unblocked, 'sharingd' immediately resumes broadcasting mDNS/Bonjour discovery packets, and AirDrop
restores instantly without requiring a reboot.

1706

1707 ##### 15.11.2 Standalone Tailscale Daemon Freeze after macOS Sleep/Wake

1708 ****Problem:**** When the macOS host goes to sleep and wakes up, the network interfaces and routing tables
are rebuilt. The standalone 'tailscaled' daemon (installed via Homebrew) occasionally fails to
handle this transition and becomes completely frozen/unresponsive. In this state, any command like '
tailscale down' or 'tailscale status' hangs indefinitely, and all DNS requests sent to the local
Tailscale resolver ('100.100.21.8') time out, causing a total internet blackout on the host.

1709

1710 ****Fix:**** Enhanced the Tailscale verification and teardown logic in 'toggle.sh' and 'recover.sh':

1711 1. ****Unresponsive Daemon Detection:**** Instead of assuming the daemon is healthy just because the Unix
socket '/var/run/tailscaled.socket' exists, the scripts now actively run a status check with a 5-
second timeout ('run_with_timeout 5 tailscale status'). If the check times out (exit code 143), the
daemon is classified as unresponsive.

1712 2. ****Auto-Recovery Loop:**** If the daemon is unresponsive, the script now automatically attempts to
restart the service using 'brew services restart tailscale' (falling back to 'sudo -n brew services
restart tailscale').

1713 3. ****Graceful Retry:**** Once restarted, the script pauses for 3 seconds for the daemon to initialize
before proceeding with connection or teardown commands.

1714

1715 ##### 15.11.3 macOS Sharing Services (AirDrop/AirPlay) Freeze on Network Transitions

1716 ****Problem:**** When the gateway is turned on or off, the scripts perform network configuration cleanups
which include flushing the routing tables ('route -n flush'), restarting the 'mDNSResponder' daemon
(('killall -HUP mDNSResponder'), and power-cycling the Wi-Fi interface ('setairportpower' or '
ifconfig en0 down/up'). While AirDrop BLE discovery continues to function, the actual file transfers
stall at "Waiting..." indefinitely.

1717

1718 ****Root Cause:**** The rapid tear-down and rebuild of the local routing table and Wi-Fi interface causes
Apple's core sharing and connection daemons ('sharingd' and 'rapportd') to lose their socket
bindings to the mDNS/Bonjour discovery interface. Instead of reconnecting gracefully, they enter a
wedged state and try to route peer-to-peer (AWDL) IPv6/TCP transfer traffic into the default gateway
(the Tailscale interface 'utun') instead of the direct wireless link, causing the TLS verification
handshake to time out.

1719

1720 ****Fix:**** Integrated an automatic sharing services reset into both 'toggle.sh' and 'recover.sh':

1721 1. The script now automatically runs 'sudo -n killall sharingd rapportd' at the end of both the ****START****
path and the ****STOP**** path (as well as inside 'recover.sh').

1722 2. Killing these daemons forces macOS to immediately relaunch them, binding them fresh to the newly
initialized network interfaces and routing tables, allowing AirDrop to work seamlessly while the
gateway is active.

1723

1724 ##### 15.11.4 Chrome Remote Desktop Connection Failures (Tailscale on Remote Device)

1725 ****Context:**** When attempting to access a remote machine via Chrome Remote Desktop (CRD), the connection
fails or hangs if Tailscale is active ****on the remote device****. The connection works fine even if
Tailscale (and the exit node) is active ****on the local client/viewer device****, but only if Tailscale
is disabled on the remote machine. *(This is distinct from the CRD-through-WARP latency limitation
in 14.3.)*

1726

1727 ****Why:**** Having Tailscale active on the remote machine creates virtual network interfaces and DNS/routing
modifications that conflict with Chrome Remote Desktop's WebRTC bindings and signaling traffic.

1728

1729 ****Workaround:****

1730 - ****Use Tailscale Native RDP/RustDesk (Recommended):**** Connect directly to the remote device's Tailscale
IP ('100.X.Y.Z') via an RDP or RustDesk client. This is faster and avoids third-party relay
conflicts.

1731 - ****Disable Tailscale on the Remote Device:**** If you must use CRD, disable Tailscale on the remote
machine before connecting.

1732

1733 ##### 15.11.5 tailscaled Daemon Broken State (brew services)

1734 ****Symptom:**** 'brew services list' shows 'started' but 'tailscale status' shows "stopped".

1735

1736 ****Fix:**** 'sudo brew services restart tailscale'. Toggle script now detects and auto-restarts.

1737

1738 ##### 15.11.6 Apple Silently Removed the 'airport' Binary in macOS Sonoma 14.4 (March 2024)

1739 ****What happened:**** 'watcher.sh's 'detect_lan_p2p_mode()' was written to use the 'airport' CLI tool at:
'
'
'/System/Library/PrivateFrameworks/Apple80211.framework/Versions/Current/Resources/airport
'
'

1742 This path returned 'exit 127: no such file or directory'. The function silently fell back to 'reason="no-
wifi"' even while the Mac was actively connected to WPA2 Enterprise Wi-Fi.

1744

1745 ****Why Apple removed it:**** 'airport' was never a real public binary it lived inside a ****private framework**
****** and was always technically unsupported. Apple deprecated it quietly years ago and finally removed
it in ****macOS Sonoma 14.4 (March 2024)****. The removal is part of a broader privacy clampdown on Wi-
Fi metadata:

1746 - Starting in ****macOS 14.5****, BSSIDs and other Wi-Fi association details are ****redacted**** ('<redacted>')
in most tools, including the official replacement 'wdutil'

1747 - Apple's official recommendation is to use the ****CoreWLAN framework**** (Swift/Objective-C) with proper
entitlements useless for a bash script

```

1748 - The official CLI replacement 'wdutil' requires 'sudo' for most useful output also useless for a
      background LaunchAgent
1749
1750 **Fix:** Replaced 'airport -I | awk '/link auth/' with:
1751 ''bash
1752 system_profiler SPAirPortDataType | awk '/Current Network Information:/{found=1} found && /Security:/{
      print $NF; exit}'
1753 ''
1754 This returns human-readable strings like 'WPA2 Enterprise', 'WPA2 Personal', or 'None' for the currently
      associated network without 'sudo', and without any removed private binary.
1755
1756 **Confirmed working output on this machine:**
1757 ''
1758 P2P detect: security='Enterprise' allow=false (reason: 802.1x-enterprise)
1759 ''
1760
1761 > ** Risk: 'system_profiler' may not survive forever either.** 'system_profiler SPAirPortDataType' still
      works as of macOS Sequoia (15.x) without sudo and without redaction. However, given Apple's
      trajectory on Wi-Fi privacy, this could be locked down in a future release. If 'security' comes back
      empty on a future macOS version while the Mac is actively on Wi-Fi, the function will silently fall
      back to 'allow=false' (safe default), but the detection will be broken again. The long-term fix
      would be a small Swift helper using CoreWLAN that we compile once and bundle in the repo. *(This
      forward-looking caveat is also tracked as an observation in 16.)*
1762
1763 #### 15.11.7 Changing macOS Local Hostname
1764 **Context:** Users may wish to change their Mac's computer name/hostname in macOS System Settings.
1765 **Effect:** Changing the Mac's hostname will **not** break Nullexit (which relies on internal routing/
      localhost). However, Tailscale will automatically detect this change and update the machine name on
      the Tailnet.
1766 - The underlying Tailscale '100.x.x.x' IPv4 address will remain exactly the same.
1767 - The **MagicDNS name** will change to match the new hostname. Users must remember to update their SFTP/
      SSH clients to use the new MagicDNS address.
1768
1769 #### 15.11.8 Shell-Language Landmines: macOS '/bin/bash' 3.2 + 'set -e' (Field Notes)
1770
1771 This project's lifecycle scripts ('toggle.sh', 'recover.sh', 'common.sh', ) run under '#!/bin/bash',
      which on macOS is **GNU bash 3.2.57 (2007)** Apple has never shipped a newer bash because bash 4+
      is GPLv3. (Homebrew's bash 5 lives at '/opt/homebrew/bin/bash' but is *not* the shebang target, and
      cannot be assumed on PATH a bare 'bash --version' on a stock Mac reports 3.2.) Every script here
      also runs 'set -e' (exit-on-error) with an 'ERR'/'INT'/'TERM'/'HUP' trap. That combination **
      ancient bash + 'set -e' + traps + 'set -x' xtrace redirection** is a minefield. The traps below
      were all hit and fixed during development; document them so they are not re-discovered.
1772
1773 **A. 'set -e' + 'if then else fi' can abort at the 'else' (bash 3.2 heisenbug).**
1774 The nastiest one. A construct as innocent as:
1775 ''bash
1776 if [ -f "$MARKER_FILE" ]; then
1777   X="yes"
1778 else
1779   X="no"
1780 fi
1781 ''
1782 aborted 'toggle.sh' at init under 'set -e' whenever the file was **absent** the false status of the '[ -
      f ]' test leaked through the compound and 'set -e' killed the script ('EXIT: code=1 phase=init'),
      before any real work ran. Two things made this vicious: (1) it is intermittent/context-dependent it
      did **not** reproduce in isolated 'bash -c '' snippets of the identical block; it only manifested
      inside the full script with the 'set -x' xtrace fd-redirect ('exec 2> >(tee)') active. We only
      proved causation by **bisecting the live script** (replace the block with a trivial assignment the
      script sailed past init). (2) Static tools ('bash -n', 'shellcheck') cannot see it it is a runtime
      interaction. **Safe pattern:** default-assign, then an 'if' with **no 'else'** (an 'if' whose false
      condition has no else-branch yields status 0):
1783 ''bash
1784 X="no"
1785 if [ -f "$MARKER_FILE" ]; then X="yes"; fi
1786 ''
1787 **Confirmed root cause (2026-07-13, reproduced head-to-head).** The trigger is specifically **set -x'
      plus a 'PS4' that contains a command substitution** here the 'DEBUG_TRACE' prompt 'PS4='+ [$(date)
      ] '>'. Forcing the false-condition/else path ('gateway_state' 'stopped', so 'if [ "$(gateway_state)"
      = "running" ]' takes the else/START branch) on stock '/bin/bash' 3.2.57: with the **default 'PS4'**
      it survives (3/3, exit 0); with toggle's **$(date)' 'PS4'** the 'ERR' trap fires on the else
      branch ('rc=1' from the false condition, 3/3, exit 42). The '$(date)' runs a subshell on **every*
      traced command; while tracing the 'if'-condition it perturbs '$?' so the condition's non-zero status
      leaks past the normal 'if'-exemption and trips the 'ERR' trap. This is a true **heisenbug**
      enabling tracing ('DEBUG_TRACE=true') is what **causes* the abort, which is exactly why it's
      invisible with 'DEBUG_TRACE=false' (the default) and why every isolated repro on the **default* 'PS4'
      passes (this misled an entire debugging session into briefly "disproving" the landmine). It aborted
      the gateway START decision ('toggle.sh:689') under 'DEBUG_TRACE=true'; with 'DEBUG_TRACE=false' the
      gateway starts cleanly (verified: 192 s, all five containers healthy). **Fixed (2026-07-14).** '
      DEBUG_TRACE's 'PS4' now uses the subshell-free '${SECONDS}' (elapsed seconds since the shell
      started) instead of '$(date +%H:%M:%S)', removing the per-traced-command subshell that perturbed '
      $?'. bash 3.2 has no subshell-free **wall-clock** token for 'PS4' ('$EPOCHSECONDS' is 5.0+, 'printf

```

'%()T' is 4.2+), so the per-line stamp is now elapsed-seconds the absolute start time is still logged once in the 'DEBUG_TRACE enabled' header line, so wall-clock is recoverable. Bonus: this also drops a 'date' fork on *every* traced line (previously thousands). **Verified head-to-head on stock '/bin/bash' 3.2.57:** 'toggle.sh --restart' with 'DEBUG_TRACE=true' now completes 'code=0' with tracing active straight through the exact decision that aborted before the trace shows 'gateway_state stopped' ('common.sh:229') feeding the false 'if ["\$(gateway_state)" = "running"]', then the else branch reaching 'Action: STARTUP GATEWAY' (1436 traced lines, no 'ERR' trap firing, all five containers healthy). 'DEBUG_TRACE' is now safe to use on the STOP/START decision path.

More generally, any command whose "failure" is normal control flow ('grep -q', '[]', 'diff', 'kill -0') that ends up as the *last* command of a function/branch/script under 'set -e' is a landmine; guard it ('|| true') or restructure so a non-zero status can't be the tail value.

B. 'BASH_XTRACEFD' and the 'exec {var}>>file' dynamic-fd form are bash 4.1+ (hard-fail on 3.2).
Our 'DEBUG_TRACE' feature wants xtrace ('set -x') sent to 'output.log' while keeping the terminal clean the clean way is 'exec {BASH_XTRACEFD}>>"\$LOG_FILE"'. On 3.2 this is a **syntax/runtime error**: 'exec: {BASH_XTRACEFD}: not found' (dynamic-fd allocation '{var}>>' didn't exist until 4.1, and 'BASH_XTRACEFD' itself is 4.1+; on 3.2 it's just an ordinary variable that 'set -x' ignores). Left unguarded it would abort the script at startup i.e. the debug feature would break the very thing it's meant to observe. **Safe pattern:** branch on '\${BASH_VERSINFO[0]}.\${BASH_VERSINFO[1]}' on 4.1 use 'exec 9>>"\$LOG_FILE"; export BASH_XTRACEFD=9'; on 3.2 fall back to 'exec 2>>(tee -a "\$LOG_FILE" >&2)' (xtrace goes to stderr, which we duplicate to the log; the terminal also shows it, which is acceptable). Never use the 'exec {var}>>' form unless you *know* you're on 4.1.

C. 'trap ERR' + '\$LINENO' misattributes the failing line on 3.2.
When a command fails under 'set -e', our trap runs 'cleanup_handler ERR \$LINENO'. On bash 3.2, '\$LINENO' inside an 'ERR' trap frequently reports the line of the **enclosing 'fi'/closing brace of a compound statement**, not the true failing line. During the STOP/START-decision bug this produced 'Script failed at line 1202' where 1202 is a bare 'fi' actively misleading. **Mitigation:** don't trust the trap's line number as gospel on macOS; corroborate with the 'EXEC:/EXIT:' lifecycle breadcrumbs and the 'DEBUG_TRACE' xtrace (which prints 'file:line:function' via a custom 'PS4'), which pin the *actual* last-executed command.

D. 'set -e' is not inherited by command substitutions / subshells the way people expect, and is suppressed inside 'if/`&&`/||' conditions.
This cuts both ways and is a frequent source of "why didn't it fail?" and "why *did* it fail?": a non-zero command used as an 'if' condition (or left/right of '&&`/||') is *exempt* from 'set -e' (by design), but the moment that same call is a bare statement it aborts. 'is_gateway_active()' relies on this (it's always called as 'if is_gateway_active; then '), which is precisely why moving the decision to a marker file instead of a probe that must be 'if'-guarded to be safe is more robust (see 15.12.19). Also note 'local X=\$(cmd)' masks 'cmd's exit status (the 'local' builtin's status wins) a 'shellcheck' SC2155 we see repeatedly; harmless when we don't check '\$?', but a real footgun if you do.

E. Process substitution ('<()','<()') is a bashism, and its lifecycle is loose.
Our 3.2 xtrace fallback ('exec 2>>(tee -a "\$LOG_FILE" >&2)') uses process substitution fine under bash, but (1) it is **not** POSIX 'sh', so these scripts must never be invoked as 'sh script.sh'; and (2) the background 'tee' it spawns is reaped asynchronously, so the very last few lines written just before exit can occasionally be truncated in the log. Acceptable for a debug trace; do not rely on it for critical last-gasp logging (use a direct '>>"\$LOG_FILE"' append for must-capture lines, as the 'EXIT' breadcrumb does).

F. GNU-vs-BSD userland divergence (not bash itself, but same class of pain).
macOS ships BSD variants of the core tools, so idioms differ from Linux: 'sed -i '' (BSD requires the empty backup-suffix arg) vs 'sed -i' (GNU); 'stat -f%z' (BSD) vs 'stat -c%s' (GNU); 'date' format flags; 'jot' (BSD) vs 'shuf' (GNU) for random port generation; 'route'/'networksetup'/'dscacheutil' (macOS-only) vs 'ip'/'resolvectl' (Linux). This is *why* the codebase branches on '[["\$OSTYPE" == "darwin"*]]' everywhere 'toggle.sh' and 'recover.sh' are each single cross-platform scripts that dispatch on '\$OSTYPE' (an early Linux block runs and exits before the macOS body). 'timeout(1)' doesn't exist on stock macOS at all hence the pure-bash 'run_with_timeout' in 'common.sh'.

Working rules distilled from the above:

- Assume **bash 3.2** semantics for anything under '#!/bin/bash' on macOS; gate any 4.0+ feature ('BASH_XTRACEFD', dynamic '{var}' fds, associative arrays, '\${var^^}' case-conversion, 'mapfile'/'readarray', negative array indices) behind a 'BASH_VERSINFO' check or avoid it.
- Under 'set -e', never let a "benign non-zero" command ('[]', 'grep -q', 'kill -0', 'diff') be the tail statement of a branch/function/script; use no-'else' 'if's, '|| true', or explicit '\$?' handling.
- Prefer **persisted state (marker files)** over **live probes** for control-flow decisions a probe forces an 'if'-guard and can hang/abort; a file read cannot (see 15.12.19).
- 'bash -n' and 'shellcheck' catch syntax and many smells but **cannot** catch 'set -e'/trap runtime interactions a *live run is mandatory* to validate lifecycle-script changes; the 'DEBUG_TRACE' + breadcrumb instrumentation exists precisely to make those live runs diagnosable.
- Never run these scripts as 'sh' they are bashisms end to end.

15.12 Misc Resolved

15.12.1 Gluetun Resets nftables FORWARD Rules

Symptom: FORWARD RELATED, ESTABLISHED rule disappears after Gluetun health check.

```

1818 **Fix:** 'scripts/routing-fix.sh' re-adds to both iptables backends every 30 seconds. Legacy backend (
      policy ACCEPT) acts as safety net.
1819
1820 #### 15.12.2 Native SSH & SFTP Security (Zero Local Attack Surface)
1821 **Context:** macOS "Remote Login" ('ssh') and "File Sharing" ('smbd') open ports on the local network (e
      .g. '172.x.x.x'), exposing the host to brute-force attacks on public Wi-Fi.
1822
1823 **Implementation:** The script strictly enforces 'tailscale up --ssh=true' whenever the mesh state is
      reset. This empowers the user to completely disable native macOS Remote Login and File Sharing. This
      provides an incredible zero-trust security advantage: even if an attacker steals your Mac username
      and password, they **cannot** access your files remotely. Disabling the native services completely
      closes the listening ports on the local Wi-Fi interface ('172.x.x.x'), rendering the Mac
      inaccessible to local network logins. Meanwhile, Tailscale intercepts port 22 traffic exclusively
      over the encrypted mesh and authenticates cryptographically based on your Tailscale SSO identity.
      Stolen Mac passwords are mathematically useless without first bypassing your Tailscale Two-Factor
      Authentication and registering a device onto the mesh.
1824
1825 **Cross-Platform File Exchange:** To easily browse and exchange files securely, simply use an SFTP client
      and connect to your Mac's MagicDNS name on port 22. Recommended clients: **WinSCP** or **FileZilla**
      (Windows), **Cyberduck** (macOS), and **FE File Explorer** or **Solid Explorer** (iOS/Android).
1826
1827 #### 15.12.3 Logging Architecture
1828 For debugging, logs are strictly segmented based on the component's lifecycle:
1829 - **'output.log' (Host-side):** Contains all standard error ('stderr') and verbose output from the host
      scripts ('toggle.sh', 'recover.sh', 'setup.sh'). Since the 'rule-compiler' container is ephemeral
      and deleted after running, its logs (and any Python errors from 'scripts/sync-rules.py') are
      extracted and appended to this file before deletion.
1830 - **Log Rotation Policy:** On startup, 'toggle.sh' checks if 'output.log' exceeds **1GB**
      ('1,073,741,824' bytes). If it does, it performs a metadata-only rename ('mv output.log output.log.
      old'), preserving history while resetting the active log to 0 bytes. This rename-based rotation
      avoids the heavy disk I/O and CPU overhead of rewriting a massive file line-by-line. The threshold
      is intentionally large because with 'DEBUG_TRACE=true' the log accumulates a full command-by-command
      execution trace (effectively the framework's entire compilation/warning history), which is valuable
      to retain for post-mortems; a smaller cap would rotate that history away too quickly.
1831 - **Egress Probe Logging:** The background host-egress leak probe checks if stdout is a TTY ('[ -t 1
      ]') and will only print live status updates (using carriage return '\r') or duplicate console
      warnings in interactive mode. It automatically detects macOS/BSD vs. Linux environments to
      dynamically construct timestamps with date and time (omitting milliseconds on macOS to prevent high
      CPU utilization from spawning sub-second processes).
1832 - **'docker logs <container>' (Guest-side):** The persistent containers ('warp', 'tailscale', 'routing-
      fix', 'adguardhome') use the Docker 'json-file' logging driver with a strict 'max-size' (1m-10m) to
      prevent VM disk exhaustion. Use standard 'docker logs' to view them.
1833
1834 #### 15.12.4 Pre-Flight Check 1: Wrong 'tailscale ping' Flag
1835 **Problem:** Check 1 of the pre-flight connectivity checks used '--c 1' (double dash) but 'tailscale ping
      ' accepts '-c 1' (single dash). The invalid flag caused the command to exit with code 1, marking the
      check as FAIL even though the gateway was reachable.
1836
1837 **Fix:** Changed '--c 1' to '-c 1'.
1838
1839 #### 15.12.5 Pre-Flight Check 1: DERP Relay Exit Code
1840 **Problem:** Even with the correct '-c 1' flag, 'tailscale ping' exits with code 1 when the connection
      goes through a DERP relay ('direct connection not established') even though a pong was successfully
      received (28ms via DERP(tor)). The check treated relayed connections as failures.
1841
1842 **Fix:** Changed the check from relying on exit code to piping stdout through 'grep -q "pong"'. This
      accepts relayed connections as valid (they work fine for the exit node use case), while still
      failing on true timeouts or unreachable peers.
1843
1844 #### 15.12.6 IPv6 Exit-Node Leak
1845 **Problem:** Tailscale supports IPv6, but WARP/luetun is IPv4-only. IPv6 packets forwarded through the
      exit node would leak through the Docker bridge unencrypted.
1846
1847 **Fix:** Added 'ip6tables -A FORWARD -i tailscale0 -j DROP' to 'post-rules.txt', blocking all IPv6
      forwarded traffic from the Tailscale interface. *(This is the fix referenced by the IPv6-leak
      scenario in 15.6.1.)*
1848
1849 #### 15.12.7 SOCKS5 Proxy Threading Race Condition
1850 **Problem:** The 'forward' function in 'scripts/socks5-proxy.py' called 'select.select([src, dst], [],
      [])' which raised 'ValueError: file descriptor cannot be a negative integer (-1)' when one thread
      closed a socket while the other thread was selecting on it.
1851
1852 **Fix:** Added 'ValueError' exception handling around 'select.select()' and 'recv()' calls. When a file
      descriptor becomes negative, the thread breaks out of the forwarding loop cleanly instead of
      crashing.
1853
1854 #### 15.12.8 Docker Healthcheck: Multi-line Python in CMD Array
1855 **Problem:** The socks-proxy healthcheck used a 'CMD' array with a multi-line Python string. YAML
      collapsed the newlines, causing the '#' comment to comment out the rest of the code and the 'assert'
      statement to be mangled into 'as'.
1856

```

```

1857 **Fix:** Changed from ["CMD", "python3", "-c", "..."] to ["CMD-SHELL", "python3 -c '...'] with a
      single-line Python expression. Uses a real SOCKS5 handshake (sends [5, 1, 0], expects [5, 0])
      instead of a simple TCP port check.
1858
1859 #### 15.12.9 Graceful Shutdowns Leave DNS Hijacked
1860 **Problem:** The gateway overrides the macOS host's DNS settings (via 'networksetup') to route queries to
      the AdGuard container. Because macOS persists 'networksetup' changes to disk, shutting down the Mac
      while the gateway is running causes the Mac to boot up later with hijacked DNS. Since the Colima VM
      and Docker containers don't auto-start on boot, the user would have no internet access until they
      manually run the toggle script.
1861
1862 **Root Cause:** The 'toggle.sh' script exits after the gateway starts. There was no background daemon
      listening for macOS system shutdown signals to revert the network settings.
1863
1864 **Fix:** Wrapped the background 'caffeinate' process (used to prevent system sleep) in a bash 'trap'
      command. The trap listens for 'SIGTERM', 'SIGINT', and 'SIGHUP'. When the user initiates a graceful
      shutdown, macOS sends 'SIGTERM' to the background trap, which instantly executes 'recover.sh' to
      flush the host DNS back to '1.1.1.1' right before the machine powers off.
1865 *Note: We explicitly use 'recover.sh' instead of 'toggle.sh' because during a shutdown, the OS limits
      process cleanup time and is already independently sending termination signals to Docker and Colima.
      Trying to run 'docker compose down' inside a shutdown trap creates a race condition that can hang
      the script until macOS forcefully 'SIGKILL's it. 'recover.sh' abandons container management and
      surgically flushes the macOS network settings in milliseconds, guaranteeing completion before the OS
      pulls the plug.
1866
1867 #### 15.12.10 Linux Native Wi-Fi Bouncing
1868 **Problem:** The generated 'scripts/recover-linux.sh' incorrectly retained the macOS 'networksetup'
      commands, which would crash and fail to bounce Wi-Fi on Linux hosts.
1869
1870 **Fix:** Swapped the macOS logic for native Linux commands. The script now attempts 'nmcli radio wifi off
      /on' first, and gracefully falls back to 'ip link set wlan0 down/up' if NetworkManager is
      unavailable.
1871
1872 #### 15.12.11 TS_AUTH_ONCE Cloud Footgun & Ephemeral Keys
1873 **Decision:** The compose file uses 'TS_AUTH_ONCE=true' to prevent authentication loops. However, on
      headless cloud VPS deployments, if a standard auth key expires (usually 90 days), the container will
      silently drop off the mesh. We documented this trade-off explicitly in 'docker-compose.yml' and
      recommended the use of **Tailscale Ephemeral Keys** for cloud deployments, which automatically clean
      themselves up and bypass this issue.
1874
1875 #### 15.12.12 Hardcoded AdGuard Credentials
1876 **Decision:** AdGuard is deployed with the hardcoded credentials 'admin / nullexit'. This is perfectly
      safe behind a NAT on a local macOS laptop. However, if deployed on a cloud host with exposed ports,
      this becomes a severe security risk. We added a stark warning comment in 'docker-compose.yml' to
      ensure cloud users change this before deployment.
1877
1878 #### 15.12.13 Routing Fix: 30-second polling vs Netlink Socket
1879 **Decision:** Instead of building a complex Netlink socket listener ('ip monitor') to react instantly to
      iptables and routing flushes from Gluetun, we retained the 5-second 'sleep' loop in 'scripts/routing
      -fix.sh'.
1880 - **Reasoning:** Building a netlink listener in pure bash on Alpine is incredibly brittle and complex (
      especially for iptables events). The 30-second polling loop is bulletproof. The only trade-off is a
      potential 1-4 second stutter if Gluetun reconnects, which was deemed an acceptable edge case for
      absolute reliability.
1881
1882 #### 15.12.14 Cache Poisoning and URL Rot in scripts/sync-rules.py
1883 **Problem:** If a remote blacklist URL went permanently offline (404 Not Found), the Python 'urllib'
      request would return the 404 HTML string. The script would aggressively regex-parse this HTML, find
      no domains, and overwrite the healthy local cache with a 0-domain file. This effectively disabled ad
      -blocking until the URL was fixed.
1884
1885 **Fix:** Implemented a defensive programming sanity check in 'scripts/sync-rules.py'. Before overwriting
      the cache, the script now verifies that the compiled domain count is greater than 10 (and 1 for IP
      feeds). If it falls below this threshold (indicating a catastrophic 404 failure across multiple URLs
      ), it aborts the cache overwrite, raises a 'ValueError', and keeps the previous day's healthy cache
      intact.
1886
1887 #### 15.12.15 Gluetun nf_tables Parser Crash (Bash Interpolation Failure)
1888 **Problem:** Attempting to make the TCP MSS dynamically configurable by using a standard bash variable
      inside 'post-rules.txt' ('iptables ... --set-mss ${GATEWAY_MSS:-1120}') caused the Gluetun container
      to catastrophically crash during startup. Gluetun's internal parser executes 'post-rules.txt' line
      by line without a shell, meaning the variable was never expanded. It literally attempted to inject
      the string '${GATEWAY_MSS:-1120}' into iptables, resulting in a fatal 'bad value for option "--set-
      mss"' error.
1889
1890 **Fix:** Removed the bash variable from 'post-rules.txt' and reverted it to a pure hardcoded integer. To
      retain dynamic 'env' configuration, we moved the interpolation logic to 'toggle.sh'. Right before
      Docker starts, the host script parses 'GATEWAY_MSS' from the 'env' file and uses a cross-platform '
      sed' command to dynamically overwrite the integer directly inside 'post-rules.txt'. This perfectly
      mimics manual configuration and completely bypasses Gluetun's strict parser.
1891

```


1892 ##### 15.12.16 AdGuard Filter Redundancy & Memory Optimization
1893 ****Problem:**** AdGuard Home does not perform cross-list deduplication in memory. When it loads its own native subscription filters (e.g., 'AdGuard DNS filter') alongside our massive 'compiled_rules.txt', overlapping rules are loaded twice, wasting significant RAM in the Colima VM. The UI would show ~500k rules, representing the sum of all lists rather than unique domains.

1894
1895 ****Fix:**** Updated 'scripts/sync-rules.py' to intelligently cross-reference AdGuard's configuration and deduplicate our list against it.

1896 1. The script now reads 'adguard/conf/AdGuardHome.yaml' to dynamically fetch the exact URLs of any enabled native AdGuard filters.

1897 2. The parsing engine was upgraded to normalize basic AdGuard syntax ('||domain~') back into raw base domains during the build process.

1898 3. The script subtracts the native AdGuard domains from our custom blocklist *before* compiling, immediately purging ~83,000 completely redundant rules.

1899 4. The normalized base domains then pass through our subdomain optimizer, which squashes an additional ~50% of the remaining rules.

1900
1901 This reduces the final compiled output from ~325k to ~281k rules on the 'heavy' profile, saving ~45k rules from being loaded into memory twice and reducing the overall RAM footprint of the 'adguardhome' container.

1902 ##### 15.12.17 Kernel-Level IP Blocklist (Threat Intelligence Firewall)
1903 ****Problem:**** DNS sinkholing cannot stop malware or botnets that bypass DNS entirely by hardcoding direct IP addresses (a common technique for C2 communication). These connections pass straight through AdGuard unseen.

1904
1905 ****Fix:**** Added a second compilation pipeline to 'scripts/sync-rules.py' that runs on every startup alongside the DNS pipeline.

1906 1. The compiler concurrently fetches four curated threat-intelligence feeds: ****Feodo Tracker**** (abuse.ch botnet C2 IPs), ****Spamhaus DROP**** (IPs allocated to criminal organizations), ****Emerging Threats**** (Proofpoint active C2 and scanners), and ****CINS**** (active brute-force sources).

1907 2. All entries are normalized via Python's 'ipaddress' module. Any IP or CIDR that overlaps with RFC1918 private ranges, loopback, link-local, or the Tailscale CGNAT range ('100.64.0.0/10') is stripped to prevent accidentally locking users out of their own LAN or mesh.

1908 3. The cleaned list (16,721 unique IPs/CIDRs) is written as an 'ipset restore' file using an ****atomic swap pattern****: 'create_new populate swap with live destroy_new'. This guarantees the live 'block_malicious' ipset is never empty during a reload.

1909 4. 'scripts/routing-fix.sh' watches the file's mtime every 30 seconds. On change it runs 'ipset restore' and idempotently re-injects 'FORWARD DROP' rules for both 'src' and 'dst' into both iptables backends (nftables + legacy).

1910 5. 'docker-compose.yml' mounts 'adguard/work/userfilters/' into 'routing-fix' as '/userfilters:ro' and adds 'rule-compiler: service_completed_successfully' to its 'depends_on', eliminating the race condition where routing-fix could start before the file existed.

1911
1912 ****Memory cost:**** The entire 16,721-entry ipset costs only ~1.6 MiB of kernel memory negligible in the 600MB VM budget.

1913
1914 ##### 15.12.18 Incident Post-Mortem: Discarded Docker Exec Errors in logs (July 10, 2026)
1915 ****Symptom:**** When the 'warp' container is stopped, dead, or failing to connect to its endpoints, the 'toggle.sh' and 'recover.sh' connectivity health checks fail, but no details are printed to 'output.log'. The logs only show generic 'FAIL' outputs, making it hard to see if the failure was a network timeout, Docker daemon error, or a missing binary.

1916
1917 ****Root Cause:**** Overly broad error silencing. The 'docker compose exec' commands in the health checks (Check 3 in 'toggle.sh' and the traffic check in 'recover.sh') both redirected stderr to '/dev/null' ('&>/dev/null' and '2>/dev/null' respectively), completely throwing away any diagnostic error messages produced by Docker or 'wget'.

1918
1919 ****Fix (July 10, 2026):**** Redirected stderr of the 'docker compose exec' check commands to 'output.log' ('2>> output.log'). This ensures that any command execution failures or connection timeout details are logged.

1920
1921 ##### 15.12.19 Incident Post-Mortem: Toggle Decided STOP/START From a Live Probe, Aborting When Docker Was Down (July 13, 2026)
1922 ****Symptom:**** Running './toggle.sh' (or '--restart') while the Colima VM / Docker daemon was ****not**** running caused the script to hang for ~15 seconds and then abort during the START phase without ever booting Colima. With 'DEBUG_TRACE' the lifecycle breadcrumb showed 'toggle.sh EXIT: code=1 phase=start', and the trace showed execution jumping straight from the START-branch entry to the 'ERR' trap ('cleanup_handler ERR') with the START body never executing. Historically this silent failure occurred 57 in 'output.log'. Spam-clicking the toggle during the resulting window (each retry rejected by the lock guard, then finally succeeding) is what produced the earlier "pile of stacked toggle processes" and Wi-Fi-bounce confusion.

1923
1924 ****Root Cause:**** 'toggle.sh' decided whether to STOP or START by calling 'is_gateway_active()' ('scripts/common.sh'), which ****live-probes**** the running system its first step is 'run_with_timeout 15 docker compose ps --status running'. When '/var/run/docker.sock' is absent (Colima down), that call blocks for the full 15 s timeout and returns non-zero. Combined with the script's global 'set -e' and 'ERR' trap, a non-zero result evaluated at the 'if is_gateway_active; then else fi' boundary aborted the whole script *before* the 'else' (START) branch body ran precisely the branch whose job is to start Colima. Design-wise the deeper flaw is that a ****disable**** should not depend on whether the gateway is ****currently**** healthy; it should act on whether the gateway is ****supposed**** to be running (

persisted intent). The correct signal already existed 'MARKER_FILE=/tmp/nullexit-gateway-active.marker', written at the end of a successful START and cleared on STOP but 'toggle.sh' only wrote/cleared it and never read it for this decision. **Note:** this was a long-standing latent bug, not a regression from the July 2026 'common.sh' platform-function unification or the 'DEBUG_TRACE' work; those changes only made the previously-silent abort *visible* (via the 'EXEC:/' 'EXIT:' breadcrumbs and xtrace).

Fix (July 13, 2026): Introduced 'gateway_should_be_running()' in 'common.sh' and routed all four decision sites through it ('toggle.sh' main decision + '--restart' pre-check; 'scripts/toggle-linux.sh' main decision + '--restart' pre-check). It decides from persisted intent:

1. 'toggle.sh' captures the marker's existence into 'GATEWAY_MARKER_AT_STARTUP' **before** the top-of-script "defensive stale-marker clear" wipes it, and the helper honors that captured value (falling back to the live marker file for callers that run before the clear, e.g. the '--restart' pre-check).
2. Marker present STOP path; marker absent START path. This is instant and cannot hang or abort.
3. Only when the marker is absent **and** the Docker socket is actually reachable ('/var/run/docker.sock' or '/.colima/default/docker.sock' on macOS) does it consult 'is_gateway_active' as a non-fatal reconciliation fallback (covers a marker lost to a '/tmp' wipe while containers are genuinely up). A dead socket short-circuits to "stopped" with no probe, so 'set -e' can never be tripped.
4. The STOP path's 'docker compose down' is now '|| true'-guarded so a disable always completes even with Docker down; the START path's 'docker compose up' is deliberately left unguarded (a failed start *should* trigger cleanup). On Linux the marker isn't maintained, so the helper degrades to the 'is_gateway_active' fallback fast there because native Docker has no Colima VM to hang.

How we got here (methodology note): This one is worth recording because the *process* mattered as much as the patch. The failure had been happening silently for a long time the log simply ended after a teardown with no error, so there was nothing to chase. We first attacked the *observability gap* rather than guessing at the cause: we wrapped the heavyweight lifecycle commands ('colima start', 'docker compose up/down') so their output and exit codes always land in 'output.log', added an 'EXIT: code= phase=' breadcrumb that fires even when the process is killed, and put a full opt-in xtrace behind 'DEBUG_TRACE'. Only *then* did we reproduce the failure and this time the trace said, unambiguously, 'EXIT: code=1 phase=start' with the START body never entered. The instrumentation didn't fix anything, but it converted an invisible abort into a precise, reproducible fact. (Lesson, consistent with 15.12.18 and the LUKS post-mortem referenced in 15.11: *a mechanism that fails silently is worse than one that fails loudly* so we make it fail loudly first.)

The root-cause leap, though, came from a *design-smell* intuition, not the stack trace: the observation that it is nonsensical for a *disable* to first check whether the gateway is *currently online* a teardown should act on whether the gateway is *supposed* to be running, not probe a subsystem to find out. That instinct turned out to name the bug exactly. The smell was a *conflated responsibility* with an inverted dependency: to decide whether to *start* the gateway, the code first had to *talk to* the gateway (via Docker) the very thing START is responsible for bringing up so the check was guaranteed to be unavailable in precisely the cold-start case that mattered. A second tell was *false symmetry*: enable (builds state) and disable (tears it down) are not symmetric operations, yet both ran the identical expensive probe; whenever two operations that should differ share suspiciously identical machinery, the code is usually lying about the structure of the problem, and reality eventually calls the bluff. The fix invented nothing the honest signal ('MARKER_FILE', literally "the gateway is supposed to be up") already existed and was maintained in all the right places; it was simply never consulted for the decision it was made for. We just pointed the decision at the answer the codebase already knew. General principle worth keeping: *design smells* here are load-bearing they tend to *be* latent bugs, not merely cosmetic, and are worth chasing on intuition even before a trace confirms them.

Acceptance verified (static + live): with marker present STOP (instant, even with Docker down); with marker absent + Docker down START (0 s, no 'set -e' abort); a real run now proceeds past init to the gateway-status decision. 'bash -n' clean on all three scripts; no new 'shellcheck' findings; re-signed '.signatures' ('toggle.sh'/'common.sh'/'toggle-linux.sh' are in the signed set) and 'crypto.sh --verify' exits 0.

Follow-up (same day) a 'set -e' regression the fix introduced, and why only a live run caught it. The first cut of the marker capture wrote the intent as a full 'if [-f "\$MARKER_FILE"]; then GATEWAY_MARKER_AT_STARTUP="yes"; else GATEWAY_MARKER_AT_STARTUP="no"; fi'. On macOS '/bin/bash' 3.2, with 'set -e' active and the 'DEBUG_TRACE' xtrace redirect ('exec 2> >(tee)') in effect, that construct *aborted* the script at *init* the moment the marker was absent the false '[-f]' status leaked through the compound and 'set -e' killed the run ('EXIT: code=1 phase=init'), before the gateway logic ran at all. Notably this did *not* reproduce in isolated 'bash -c' snippets of the same block it only surfaced in the script's real runtime context so we had to bisect against the actual 'toggle.sh' (swap the block for a trivial assignment the script sailed past init) to prove causation. Fix: write it as a default assignment plus an 'if' *without* an 'else' ('GATEWAY_MARKER_AT_STARTUP="no" then 'if [-f "\$MARKER_FILE"]; then GATEWAY_MARKER_AT_STARTUP="yes"; fi'), which yields status 0 when the condition is false. Lesson reinforced: 'set -e' on legacy bash is genuinely treacherous around conditionals, and the observability work paid for itself again the breadcrumb pinned the abort to init instantly, and a live run (not static checks) was the only thing that exposed a heisenbug invisible to isolated reproduction.

15.12.20 Honey-Port Tripwire Accumulation (July 14, 2026)

Symptom: After many toggle-ons, 'ps' showed a pile of idle 'bash ./toggle.sh' subshells, each blocked on an 'nc -l localhost <port>' listener one leaked per toggle-on, never reaped.

Root Cause: The Honey-Port Tripwire arms a fresh random port each START via a disowned '(nc -l) &' subshell. 'nc -l' blocks until a connection that (by design) almost never comes, so each toggle-on leaks one idle listener; because the port is random, successive listeners never collide and simply

stack up over time.

1945

1946 ****Fix:**** 'toggle.sh' now reaps the previous tripwire before arming a new one 'pkill -f "nc -l localhost" (macOS) / 'pkill -f "nc -l .*.127.0.0.1" + "nc -l localhost" (Linux) immediately before 'HONEY_PORT=\$(get_free_port)' at both arming sites. No 'sudo' needed (the listeners are user-owned). Net effect: at most one tripwire alive at a time instead of one per toggle. (The same 'pkill' pattern cleared the backlog that this session's ~15 test toggles produced.)

1947

1948 **####** 15.12.21 Post-Wake Self-Feedback Loop: The Watcher Re-Fires On The Route Change It Caused (July 14, 2026)

1949 ****Symptom:**** After a container restart (or wake), 'output.log' showed 'recover.sh --post-wake' launching ~6 times in a row, one launch roughly every ~40s, each exiting 'exit=0' recovery *worked* every time yet kept re-firing "until it worked." Every launch was triggered by 'NET: changedKey State:/Network/Global/IPv4'.

1950

1951 ****Root Cause** a self-feedback loop, NOT PF.** 'scripts/watcher.sh' Listener 2 watches 'State:/Network/Global/IPv4' via 'scutil n.watch' and launches 'recover.sh --post-wake' on any change. But post-wake re-asserts the exit node ('recover.sh' ~L367, 'tailscale set --exit-node'), and even though 'set' is deliberately used instead of '--reset' to minimise churn (recover.sh ~L360-366), **re-asserting the exit node re-writes the host default route and the default route IS 'State:/Network/Global/IPv4'.** So each run mutates the exact key the watcher triggers on. The self-triggered change lands ~15-45s after the run finishes; 'run_recover()'s old 10s debounce (reset to the run's *finish* time at the tail of the function) had long expired by then, so the watcher re-fired. The loop self-terminates only once 'tailscale set' becomes a no-op (route already correct) no route change no re-trigger.

1952

1953 ****Why the debounce couldn't stop it:**** the loop period (~40s = one full post-wake run + settle) far exceeds the 10s debounce, which therefore only ever caught macOS's rapid *duplicate* scutil notifications for a single change (the '+0.2s' doublets), not the self-trigger.

1954

1955 ****Not PF:**** post-wake never calls 'pfctl' (the kill-switch is armed once, in 'scripts/common.sh', not per cycle); no PF-enable line falls inside the loop window. The 'No ALTQ support in kernel' warning is normal macOS pfctl noise (Apple ships pfctl without ALTQ compiled in), and 'Could not capture PF reference token' is a separate benign ref-count bookkeeping wart both unrelated to the loop.

1956

1957 ****Fix:**** 'scripts/watcher.sh' 'run_recover()' now arms a ****post-recovery settle cooldown**** ('/tmp/nullexit-watcher.settle-until', 'NULLEXIT_POSTWAKE_SETTLE_SECONDS' default 45s) *after* each post-wake run, and skips launching while 'now < settle-until'. Because it is armed only after a recovery runs, a genuine first wake/roam is still handled immediately; only the Global/IPv4 change the run caused itself is absorbed. A real new roam during the window is delayed at most 'POSTWAKE_SETTLE_SECONDS', an acceptable trade for killing the loop. Watcher-side only 'recover.sh' is unchanged. Severity was low (self-correcting, healthy every iteration, no leak/outage; cost was wasted ~28s runs + log noise).

1958

1959 **####** 15.12.22 FaceTime (and Real-Time P2P Media) Over the Double-Tunnel: Why It Breaks + the Split-Route Plan (July 15, 2026)

1960 ****Symptom:**** A FaceTime call placed while nullexit is up fails to connect.

1961

1962 ****Diagnosis** nullexit is NOT blocking it; every layer passes the traffic (all verified live):**

1963 - ****DNS:**** no Apple/FaceTime domain is sinkholed 'dig' via the gateway ('100.100.21.8') returns real Apple A/CNAME records, none '0.0.0.0'/'127.0.0.1'. The '<no answer>' domains ('gateway.push.apple.com', 'fmn.apple.com', 'identityservices.apple.com', 'fdr.apple.com') return empty on '1.1.1.1' too CNAME/non-A, not our block.

1964 - ****routing-fix 'FORWARD':**** the exit-path rule '-i tailscale0 -o tun0 -j ACCEPT' carries ~203K pkts / 73 M bytes and accepts *everything*.

1965 - ****Country / threat blocklists**** ('block_il', 'block_kp', 'block_malicious'): ****0**** packets dropped.

1966 - ****WARP** (gluetun) egress:** '-A OUTPUT -o tun0 -j ACCEPT' + '-A POSTROUTING -o tun0 -j MASQUERADE' all outbound UDP (any port) leaves and is SNAT'd through WARP.

1967

1968 ****Root cause** double symmetric NAT ending at Cloudflare WARP.** Path: 'host Tailscale (MASQUERADE) WARP (MASQUERADE) Cloudflare shared egress'. WARP is a CGNAT-style shared egress with ****no inbound UDP mappings****, and the double-MASQUERADE presents as ****symmetric NAT**** the exact condition that defeats FaceTime's STUN/ICE UDP hole-punching. No P2P path can form; the Apple-relay fallback is unreliable through the stacked tunnel, and the 12801200 MTU squeeze degrades media. This is inherent to routing real-time P2P media through WARP, ****not**** a firewall rule. (Zoom/WhatsApp *calls* hit the same wall.)

1969

1970 ****Latent finding** (separate bug, NOT yet fixed):** the 'FORWARD' chain's '-i tailscale0 -p udp --dport 443' (QUIC) and '--dport 853' (DoT) 'REJECT' rules are ****shadowed**** by the earlier '-i tailscale0 -o tun0 -j ACCEPT' (first-match wins) verified ****0 pkts**** on each. So the intended QUIC-block / force-through-AdGuard (block DoT) on the exit path is ****not enforced****: a mesh client can use DoT (853) to bypass AdGuard filtering entirely. To actually enforce, those port-'REJECT' rules must be inserted ****before**** the 'tailscale0 tun0' ACCEPT (or scoped '-o tun0'). ****TODO.****

1971

1972 ****Fix plan** (chosen: Option 2 narrow, always-on, opt-in split-route). DESIGNED / STAGED, not yet implemented in code:**

1973 - ****Mechanism:**** add more-specific host routes for FaceTime's Apple '/16's via the ****physical**** gateway ('172.17.0.1'/'en0'). Longest-prefix match beats Tailscale's exit-node '0.0.0.0/1' + '128.0.0.0/1' routes, so only those '/16's leave ****direct**** while everything else stays double-tunneled. Mirrors 'add_warp_bypass_routes'.

```

1974 - **Wiring:** 'add_apple_split_routes()' / 'remove_apple_split_routes()' in 'common.sh'; called in toggle
      START (after exit-node assert) + 'recover.sh --post-wake' (survives roam) + removed on STOP;
      guarded by an **opt-in '.env' flag (default OFF)** so the privacy posture is unchanged unless
      enabled.
1975 - **Privacy tradeoff:** whatever '/16's go direct see the **real ISP IP** (not WARP). Apple already ties
      traffic to the Apple ID, so Apple-direct is the accepted scope; user chose **narrow** (FaceTime-
      infra '/16's only) over all-'17.0.0.0/8' to minimise the surface.
1976 - **CIDR list to be finalised from an empirical capture (PENDING, deferred at user request):** host-side
      'sudo tcpdump -n -i <exit-utun, e.g. utun6> net 17.0.0.0/8' during a real call, then extract
      distinct '/16's. Known-likely from DNS today: '17.57/16' (APNs push/ringing), '17.248/16' (iCloud/
      FaceTime gateway), '17.253/16' (aapling push CDN), '17.157/16' (GSA auth). **The relay/STUN '/16's
      the load-bearing media path are call-time-discovered and MUST come from the capture;** routing
      signaling-only will leave media broken. Signed scripts are untouched until the capture yields the
      media ranges. See P2P note 16.4.
1977 ---
1978
1979
1980 # Part IV Observations, Changelog & TODO
1981
1982 ## 16. Unverified Observations
1983
1984 ### 16.1 AdGuard Returns Two Different Sinkhole IPs (Unverified)
1985
1986 > **Status: Observed but not formally verified. Needs a deeper audit of the rule-compiler sources to
      confirm.**
1987
1988 **Observation (July 10, 2026):** When querying AdGuard Home for blocked ad domains, some domains resolve
      to '0.0.0.0' and others resolve to '127.0.0.1'. Both are blocked, both cause connection failure, but
      the different responses were unexpected.
1989
1990 ```
1991 doubleclick.net          0.0.0.0      (blocked)
1992 ads.google.com           127.0.0.1   (blocked)
1993 pagead2.googlesyndication.com 0.0.0.0     (blocked)
1994 scorecardresearch.com    127.0.0.1   (blocked)
1995 ```
1996
1997 **Likely Explanation (Unverified):** There are three historical schools of thought on the "correct" DNS
      sinkhole response, and AdGuard faithfully preserves whichever the source blocklist used:
1998
1999 | Sinkhole IP | Convention | Origin |
2000 |---|---|---|
2001 | '0.0.0.0' | AdBlock-style lists ('\\|\\|domain^') | Modern DNS-level blocking; AdGuard's native format.
      Fails fast OS doesn't attempt a TCP connection. Works for both IPv4 and IPv6 without a separate
      rule. |
2002 | '127.0.0.1' | Hosts-file-style lists ('127.0.0.1 domain') | 90s-era '/etc/hosts' tradition. Steven
      Black's hosts list (500k+ domains) still uses this. |
2003 | 'NXDOMAIN' | Purist / Pi-hole default | Returns "domain doesn't exist." Semantically most accurate but
      requires browsers to handle NXDOMAIN gracefully. |
2004
2005 The dual-response behavior in this project is because 'rule-compiler' pulls from both AdBlock-format
      sources (Hagezi, uBlock origin lists) and hosts-format sources (Spamhaus DROP, Steven Black, Feodo
      Tracker), and AdGuard returns whatever sinkhole IP the matching rule specifies.
2006
2007 **To Verify:** Run 'docker compose exec tailscale cat /etc/hosts' to confirm no hosts-file rules are
      being injected at the container level, and check which specific AdGuard rule matched each domain via
      the AdGuard query log at 'http://100.100.21.8:3000/#logs'.
2008
2009 ### 16.2 AGY CLI Appears to Generate Native macOS Notification Pop-ups (Unverified)
2010
2011 > **Status: Observed but source unverified. macOS security logs ruled out system-level origin.**
2012
2013 **Observation (July 10, 2026):** While running a DNS blocklist verification via the Antigravity CLI ('agy
      '), a command was proposed that included 'malware.wicar.org' (a known-safe DNS blocklist test domain
      ) in a 'dig' loop. Shortly after, a macOS-style notification popup appeared describing a "malicious"
      or "blocked" event. The AGY CLI terminal session then closed.
2014
2015 **Investigation:**
2016 - **macOS XProtect / Gatekeeper logs:** Empty. Zero events in the 30-minute window around the incident.
2017 - **macOS Endpoint Security / System Policy logs:** Empty.
2018 - **Broad 'log show' search for "malicious", "malware", "blocked":** Empty.
2019 - **Conclusion:** macOS itself did not generate the notification. No system security subsystem fired.
2020
2021 **Likely Explanation (Unverified):** The notification appears to have originated from **AGY CLI's own
      internal safety system**. AGY CLI is a native macOS application (not just a terminal process) and
      has access to the macOS Notification Center API ('UNUserNotificationCenter'). When its guardrails
      detected a domain containing the word "malware" ('malware.wicar.org') in a proposed shell command,
      it likely:
2022 1. Blocked the command from executing
2023 2. Posted a native macOS-style notification via the Notification Center
2024

```

2025 This is surprising behavior for a CLI tool most terminal programs don't send GUI notifications but AGY
CLI appears to bridge both worlds: it runs in a terminal but is packaged as a native app with full
system API access. The notification would be indistinguishable from a system security alert at a
glance.

2026

2027 ****Notes:****

2028 - 'malware.wicar.org' is the DNS/AV equivalent of the EICAR test file a deliberately benign domain that
triggers blocklist/AV systems to verify they work. It was included in the test set to confirm the
AdGuard blocklist was catching it. AGY's safety system is not aware of this distinction.

2029 - For future DNS blocklist testing, use only ad/tracking domains (e.g., 'doubleclick.net', 'adnxs.com')
to avoid triggering AGY's guardrails.

2030

2031 **### 16.3 'system_profiler SPAirPortDataType' Longevity (Wi-Fi Detection)**

2032 The Wi-Fi security detection used by 'watcher.sh's 'detect_lan_p2p_mode()' currently relies on '
system_profiler SPAirPortDataType' (after the 'airport' binary removal full history in 15.11.6).
This works as of macOS Sequoia (15.x) without sudo and without redaction, but given Apple's ongoing
Wi-Fi privacy clampdown, it may be locked down in a future release. If 'security' comes back empty
on a future macOS version while on Wi-Fi, detection silently falls back to 'allow=false' (a safe
default). The long-term fix would be a small Swift CoreWLAN helper compiled and bundled in the repo.

2033

2034 **### 16.4 Tailscale LAN P2P Succeeds Cross-AP But Fails Same-AP (Client Isolation + No NAT Hairpin)**

2035

2036 ****Observation (July 15, 2026):**** On the residence Wi-Fi, a Galaxy S24 in the lobby reception 6 floors
away, associated to a *different* AP establishes a **direct, low-ping** Tailscale P2P session to an
iPhone 11. The *same* pair on the *same* AP / same floor / close range does **not** go direct.
Counterintuitively, **farther apart = P2P works**. The effect shows for the iPhone 11 (a plain
client) but **not** for S24macOS or S24the gateway container.

2037

2038 ****Mechanism:****

2039 - ****Same AP:**** the AP enforces **client isolation** (L2 devicedevice block on one radio). Both devices
then share one public IP; STUN returns identical external mappings, so reaching each other would
need the residence router to **NAT-hairpin** which most campus/consumer gear does not do direct
P2P fails and falls back to a DERP relay.

2040 - ****Different AP (floors away):**** large residence networks **segment per AP/area into distinct VLANs/
subnets**, so the two devices sit on different L3 networks routed through the campus core, where
client isolation does not apply and the STUN path works **without** hairpin clean direct P2P.

2041 - ****Why macOS / the container are immune:**** nullexit pins a **direct WireGuard endpoint** (Tailscale port
'41642', 'direct 172.x'), so those peers already hold a negotiated direct path and do not depend on
same-AP L2 discovery. A plain iPhone 11 has no pinned endpoint, so its path is entirely at the
mercy of the Wi-Fi topology which is exactly why *it* exposes the isolation effect.

2042

2043 ****Relevance:**** refines 'watcher.sh's 'detect_lan_p2p_mode()' (which probes same-subnet AP isolation via
a broadcast ping). The probe correctly flags same-AP isolation, but this observation adds that
isolation is **per-AP** and cross-VLAN P2P can succeed even when same-AP fails so a single "LAN P2P
allowed?" boolean under-describes a multi-AP network. No code change; documents expected behavior.
Related: the FaceTime NAT analysis in 15.12.22 (same STUN/hairpin physics, different symptom).

2044

2045 **### 16.5 Working Theory iPhone Cross-Device Visibility Reaches Other Phones, Not macOS (Instagram Auto-
Open, July 15, 2026)**

2046

2047 ****Observation:**** Opening Instagram on a Galaxy S24 (specifically via **Chrome** on the S24, not Samsung
Internet) auto-opens a tab on a nearby MacBook, but **only when the S24 is physically close** to the
Mac it does not happen when the phone is away. **Chrome on the S24 has zero granted Android
permissions** (no Bluetooth, no location, no nearby-devices) this rules out Chrome itself
performing any BLE/Nearby proximity scan, since Android enforces those permissions at the app level
regardless of what the browser ships. This pushes the likely actor down to **Google Play Services**,
which holds system-level Bluetooth/nearby-device access independent of per-app grants and could be
doing the proximity detection on Chrome's behalf (e.g. via Android's "Nearby" APIs) then handing
the result to Chrome only for the actual tab-open. A purely local, short-range channel is still
required to explain the proximity gating regardless of which component performs it; this also rules
out any cloud-mediated mechanism (Chrome cloud tab sync, FCM push, Tailscale/WAN P2P per 16.4) since
those are distance-independent. Live probes during the session ('system_profiler
SPBluetoothDataType' for a paired device, 'dns-sd -B _services._dns-sd._udp' for 8s) found **no
paired Bluetooth device and no Android/Google/Instagram mDNS service** only the Mac's own Bonjour
services (AirPlay, RFB, '_companion-link', Spotify Connect). Inconclusive: a BLE-advertisement-only
channel (no formal pairing) would not show up in either probe.

2048

2049 ****User's working theory:**** the iPhone (also on hand, on the same network/Apple ID as the Mac) makes *
other devices connected to it* even an Android phone visible to nearby devices, but this
visibility channel reaches **other phones more readily than it reaches macOS**. I.e., the iPhone may
be acting as a discovery bridge/relay between the S24 and the Mac rather than the S24 and Mac
talking directly. Combined with the Play Services angle above, a refined version is: **Play Services
on the S24 detects proximity to an Apple mesh (iPhone and/or Mac both signed into iCloud/AWDL-
visible) via some cross-ecosystem discovery Google has built**, rather than the S24 and Mac
negotiating directly.

2050

2051 ****Cross-ecosystem refinement:**** this isn't Google-only. Apple runs an equivalent OS-level background BLE
scanning layer for its own Continuity stack (Handoff, Universal Clipboard, AirDrop discovery, Find
My the latter using *every nearby Apple device*, not just the owner's, as a passive BLE relay),
implemented as system daemons ('bluetoothd'/'sharingd'/'rapportd', already implicated in the
unrelated AWDL wedge bug at 15.10) rather than anything gated by app-level permission. The key

mechanism that makes the two ecosystems mutually visible: ****BLE advertisement packets are broadcast in the clear over open air**** Apple's Continuity BLE frames are semi-proprietary but unencrypted at the advertisement layer, so any BLE radio (including Android's, via Play Services once Android's own "Wi-Fi & Bluetooth scanning" system-level toggle is on) can passively observe "an Apple device is here" and react, without pairing or explicit permission from Apple's side. So both ecosystems maintain a BLE mesh-discovery layer that sits below their own per-app permission model, and each side's layer is passively readable by the other's radio which is a plausible bridge for a Play-Services-mediated Chrome trigger reacting to iPhone/Mac Continuity broadcasts.

****Status:**** Unverified. Not yet reproduced under an active capture the July 15 session ran the mDNS browse **before** confirming the phone was actively broadcasting, so it's not a clean negative result. To confirm: capture `'tcpdump -i en0 udp port 5353'` (mDNS) ****and a raw BLE advertisement capture**** (e.g. 'nRF Connect' or similar BLE scanner app on a third device, or 'sudo btmon'-equivalent on Linux macOS does not expose raw BLE advertisement capture without a hardware sniffer) at the exact moment Instagram is opened on the S24 while close, with the iPhone present vs. absent, to isolate whether the iPhone's Continuity broadcasts are the trigger or the S24Mac channel exists independently of it. On the Android side, checking 'Settings Location Location Services Wi-Fi & Bluetooth scanning' and whether ****Google Play Services**** (not Chrome) holds Bluetooth/nearby-device/location permissions would confirm or kill the Play-Services-as-actor theory. Related: 16.4 (same physical-proximity family of P2P/discovery quirks, different mechanism and inverted distance relationship).

17. Changelog / Recent Updates

Reflects the current branch's state. Each entry is a one-line summary + commit hash; read the commit message (and the linked Incident Log entry) for full detail. Full post-mortems live in 15.

- ****July 10, 2026 'fix(race): suppress WARP Watcher during post-wake force-recreate**** Fixed a race where switching Wi-Fi caused the host IP to leak as the raw ISP IP ('132.x') on every roam; 'recover.sh --post-wake''s warp force-recreate tripped the WARP Watcher into a nuclear shutdown. Fixed via '/tmp/nullexit-warp-inhibit.marker'. See 15.2.7.
- ****July 10, 2026 startup/roaming stabilization suite**** Pre-flight Tailscale race (15.5.3), STUN-block cold-boot false negative (15.5.4), restart DNS-build race via 'wait_for_dhcp_settle' (15.5.5), nuclear/post-wake concurrency lock (15.2.8), infinite auto-recovery loop marker leak (15.2.9), PF kill-switch bricking SOCKS5 fallback (15.7.3), '--reset' host logouts (15.7.4), missing-flags abort (15.7.5), discarded docker-exec errors (15.12.18), and the SNAT endpoint poisoning fix (15.7.1).
- ****July 6, 2026 'fix: resolve edge-case vulnerabilities and system state leaks**** Security-review hardening suite:
 1. ****DNS Watcher Interface Independence:**** 'start_dns_watcher' now detects the active interface via 'get_active_service()' instead of hardcoding a 'grep' for "Wi-Fi".
 2. ****Robust Lock Verification:**** 'toggle.sh' lock-file logic uses 'ps -p' verification to prevent permanent lockouts from a recycled PID.
 3. ****Fail-Closed Crypto:**** 'crypto.sh' integrity check switched from '[-x]' to '[-f]' so it runs even if git strips the '+x' bit.
 4. ****Strict 'iptables' Pinning:**** 'post-rules.txt' FORWARD-chain rules explicitly pin 'tailscale0' 'tun0' both directions, preventing escape via 'eth0' during WARP drops.
 5. ****Docker Local Network Isolation:**** 'docker-compose.yml' binds exposed WARP ports ('1080', '5354') to '127.0.0.1' to prevent LAN access.
 6. ****Routing Verification & Sudoers:**** Added a 'netstat -nr' default-route override check and ensured '/usr/bin/python3' is permitted in the sudoers template.
- ****July 4, 2026 'cc789c5**** Concurrency mutex on 'toggle.sh' ('/tmp/nullexit-toggle.lock'); defensive 'TS_AUTHKEY' init under 'set -u'; Go 'go vet'/'go build' steps in CI; AdGuard log capping; macOS BSD 'grep' fixes in 'setup-common.sh'; 'recover-linux.sh' quoting/PID fixes; removed redundant bypass-routes block in 'recover.sh'.
- ****July 4, 2026 'fix: post-wake routing loop & destructive recovery**** (1) 'add_warp_bypass_routes' now accepts explicit interface args so post-wake stops routing WARP traffic back into 'utun*' (was an infinite loop killing internet on every wake). (2) Replaced 'sudo route -n flush' with targeted deletions of '0.0.0.0/1'/'128.0.0.0/1' + WARP bypass routes, so recovery no longer destroys loopback/multicast routes and wedges 'configd'/'mDNSResponder' until reboot.
- ****July 4, 2026 'feat: secure execution**** Restricted the blanket '/usr/bin/python3' NOPASSWD sudo to exactly '/usr/bin/python3 -I .../scripts/dns-proxy.py'; locked 'dns-proxy.py' with 'chown root:wheel' + 'chmod 755'; added native auth prompts to 'Toggle Gateway.app'/'Recover Gateway.app'; 'toggle.sh' now captures 'warp' logs on failure before teardown.
- ****July 4, 2026 Go ports + refactor**** 'logger' and 'rule-compiler' ported to Go multi-stage Docker builds (15.8.6); ~200 lines of duplicated bash extracted to 'scripts/common.sh' (15.9.4); 'crypto.sh' HMAC-SHA256 script integrity added.
- ****July 3, 2026 'fix: resolve numerous system regressions**** Fixed truncated '/etc/sudoers.d/nullexit' line; temporary '.tmp'+os.replace()' atomic writes in 'sync-rules.py' (later reverted, see below); 'recover.sh' 'ts_args' quoting; ANSI '%b'/'-e' output in 'diagnose-host-leak.sh'/'fix-docker-bridge-collision.sh'; 'toggle-linux.sh' using 'resolvectl dns' instead of macOS 'networksetup'; AdGuard REST refresh POST targeting port '3000'; restored 'START_GATEWAY=true'; purged stale port '80' mapping; verified 'output.log' in '.gitignore'.
- ****July 3, 2026 'unlock-files.sh' + revert atomic writes**** Reverted all 5 '.tmp'+os.replace()' sites in 'sync-rules.py' to direct writes; created 'scripts/unlock-files.sh' to permanently fix stale '0444'/'000' file permissions via atomic rename (no 'chmod'). Covers '.env', 'compiled_rules.txt', 'ip_blocklist.ipset', 'cache/*.txt', 'cache/ip/*.txt', 'data/filters/*.txt'. See 15.9.6 + 3.
- ****July 3, 2026 Overnight leak + geo-blocking**** Introduced 'scripts/host-leak-probe.sh' (sub-second host-egress detector, 15.6.1) and the statistical-threshold WARP auto-shutdown watcher (15.6.2); fixed the overnight silent IP leak (15.6.3).
- ****July 2, 2026 '8c5fae1' + '181e1e1**** Two infinite routing loops fixed: host-VM tunnel loop (WARP bypass routes via physical gateway IP + 'setup_exit_node_routing' forcing default to 'utun*') and

Docker Compose subnet takeover ('10.200.1.0/24' lock). '--accept-routes=true' added explicitly to all 'tailscale up --reset' calls. See 15.2.2.

2076 - **July 2, 2026 auto-recovery daemon** 'scripts/watcher.sh' + launchd LaunchAgent documented. See 15.5.1.

2077 - **July 1, 2026 'c5b92c0' (refactor: personal-leak cleanup)** Eliminated every residual personal-username leak across source + config; launchd plist uses 'WorkingDirectory' + relative program arg with the '\$_NULLEXIT_HOME_' sentinel (install recipe gains a single 'sed -i ' ' "s|\$_NULLEXIT_HOME_|\$(pwd)|" ' step); 8 devref prose sites genericized to '~/colima/...', '~/Library/LaunchAgents/...', '\$USER', '<USER>'.

2078 - **July 1, 2026 'a5e2a5a' (refactor: script-relative paths)** Replaced 6 hardcoded '/Users/<user>/.../nullexit/...' literals with script-relative resolution. 'recover.sh'/'watcher.sh' inject 'SCRIPT_DIR=\$(cd "\$(dirname "\${BASH_SOURCE[0]}")" && pwd)"; 'RECOVER="\${SCRIPT_DIR}/../recover.sh"' resolves without launch-path dependency.

2079 - **June 28, 2026 'f8f8e4e' (M1: scripts/ move)** 8 internal scripts consolidated into 'scripts/'; the 4 user-facing/orchestrator/config files ('toggle.sh', 'recover.sh', 'setup.sh', 'docker-compose.yml') stayed at repo root.

2080 - **June 28, 2026 '0875ef3' (ci: glob)** CI 'py_compile' switched to 'python3 -m py_compile scripts/*.py' after M1 left root with zero '.py' files.

2081 - **June 28, 2026 '25b37a1' (docs: scripts/ prefix)** 'devref.md' + 'README.md' updated to the 'scripts/' prefix for all 8 moved files (~31 patches).

2082 ### End-to-end verification (commit 'c5b92c0')

2083 Manual START STOP cycle ran clean on macOS after path relativization + personal-leak cleanup:

2084 - 'bash toggle.sh' START: exit 0, ~135s (cold Colima boot); marker present ('2026-07-02T03:41:41Z'), all 5 containers healthy, host DNS hijacked to '100.100.21.8' (no fallback), exit node selected, Cloudflare trace through WARP succeeds.

2086 - 'bash toggle.sh' STOP: exit 0, ~12s; marker cleared, all nullexit containers gone, host DNS restored to '1.1.1.1', Tailscale disconnected.

2087 ## 18. TODO & Future Work

2088

2089 ### 18.1 TODO

2090

2091 * **FaceTime / real-time P2P split-route (Option 2, narrow):** Implement 'add_apple_split_routes()' / 'remove_apple_split_routes()' in 'common.sh' (route FaceTime's Apple '/16's direct via the physical gateway, opt-in '.env' flag, default OFF), wired into toggle START/STOP + 'recover.sh --post-wake'. **Blocked on** an empirical 'tcpdump' capture of the relay/STUN '/16's during a real call (user-triggered). Full design + diagnosis in 15.12.22.

2092 * **Fix the shadowed exit-path port REJECTs (DoT/QUIC bypass):** The 'FORWARD' '-i tailscale0 -p udp --dport 853'/'--dport 443' REJECT rules are shadowed by the earlier 'tailscale0 tun0' ACCEPT (0 pkts), so the intended block-DoT / force-AdGuard and QUIC-block are **not enforced** a mesh client can bypass AdGuard via DoT (853). Re-order the REJECTs before the ACCEPT (or scope '-o tun0'). See 15.12.22.

2093 * **Decide toggle STOP/START from persisted *intent*, not a live probe:** '~toggle.sh' decided STOP-vs-START by calling 'is_gateway_active()', which live-probes the running system ('docker compose ps' + DNS + SOCKS checks). When Colima/Docker was down the 'docker compose ps' blocked for the full 15 s 'run_with_timeout' window and, under 'set -e' + the 'ERR' trap, aborted the script *before* the START body ran* so the path that would boot Colima never executed ('EXIT: code=1 phase=start', seen 57 historically).~~ **Closed** added 'gateway_should_be_running()' in 'common.sh', which reads persisted intent ('MARKER_FILE') captured into 'GATEWAY_MARKER_AT_STARTUP' before the defensive stale-marker clear, and only falls back to 'is_gateway_active' when the marker is absent *and* the Docker socket is actually reachable (so a dead-socket probe can neither hang nor trip 'set -e'). All four decision sites ('toggle.sh' main + '--restart', 'toggle-linux.sh' main + '--restart') now use it, and the STOP 'docker compose down' is '|| true'-guarded so a disable always completes even with Docker down. See 15.12 Misc Resolved.

2094 * **Verify Wi-Fi Roaming:** Investigate and test whether switching Wi-Fi networks now successfully and smoothly recovers the gateway end-to-end without any tailscale flag errors or dead tunnels. (Pending user verification).

2095 * **Fix Diagnostic Script Check 5/8:** ~Investigate why 'diagnose-host-leak.sh' check 5/8 ('Host default route') sometimes reports "default route goes via physical Wi-Fi Tailscale route assertion failed" on macOS even though the "warp=on" egress checks confirm that traffic is successfully tunneling.~~ **Closed** fixed by switching check 5 from 'netstat -rn | grep default' to 'route -n get 1.1.1.1'. macOS Tailscale uses interface-scoped routes and does not replace the DHCP default route entry. See 15.7.1 Known Unknowns.

2096 * **Expose Tor Control Port (9051):** ~Consider mapping the Tor Control port to localhost and adding 'nyx' (the Tor terminal status monitor) to allow users to inspect their entry, middle, and exit node IPs in real-time.~~ **Closed** mapped the Tor Control Port to a procedurally generated, localhost-bound port 'TOR_CONTROL_PORT', configured 'ControlPort' and disabled cookie authentication in 'docker/tor/entrypoint.sh', and integrated it into the '/sweep' verification protocol.

2097 * **Host-Only Tor Mode (Pluggable Egress):** Implement an 'EGRESS_TYPE=tor_host_only' mode for users in heavily censored (DPI-restricted) environments. This mode would skip booting 'warp' and 'tailscale', boot 'adguardhome' and 'tor' (using Telegram 'obfs4' bridges), set AdGuard's upstream to Tor's 'DNSPort', and use the 'pf' kill-switch to force all host Mac traffic strictly through the Tor network. This provides a 100% free, DPI-resistant evasion path without requiring an external VPS.

2098 * **The Technical Realities (What You Need to Watch Out For):**

2099 * **The UDP Problem:** Tor only transports TCP traffic. It does not support UDP (aside from DNS requests passed directly to its DNSPort). macOS is incredibly chatty over UDP (FaceTime, mDNS, QUIC protocols). Your pf rules defined in 'scripts/pf.conf' must be configured to aggressively and silently DROP all host UDP traffic (except for local DNS queries to AdGuard). If you don't drop it cleanly, macOS services might hang indefinitely while waiting for timeouts.

2100 * **The "Daily Driver" Friction:** Routing a whole host OS through Tor is brutal on performance. Background daemons (iCloud sync, macOS software updates, Spotlight indexing) will blindly attempt to


```

2101     download gigabytes of data over Tor, which could saturate the circuits or time out completely.
2102 * **Bridge Rot:** State firewalls actively hunt and block obfs4 bridges. When a user's bridges burn
2103 out, they will lose all internet access due to the pf kill-switch. You will need to ensure the user
2104 has a frictionless way to update the 'tor-bridges.txt' file and restart the Tor container without
2105 exposing their IP.
2106 * **Exit Node Discrimination:** Because all traffic will exit from known Tor nodes, the user will
2107 face an onslaught of Cloudflare CAPTCHAs, and many banking or streaming services will flat-out
2108 reject the connections.
2109
2110 ### 18.2 Future Work
2111 - **Direct P2P on VM Hosts:** Non-Linux hosts run Docker inside a VM, making true UDP hole-punching **to
2112 arbitrary external peers** impossible; those peers fall back to the DERP relay unless bridged mode
2113 is enabled on a trusted LAN. The only fully general solution is a native Linux host (Raspberry Pi,
2114 Intel NUC) where Docker runs without VM hypervisor translation. *(The one case that works regardless
2115 is direct **hostcontainer** P2P the gateway and its own Mac, the same physical machine re-
2116 permitted via routing-fix 'host_ips' RETURN rules and confirmed 'direct' even in plain user-mode
2117 networking; see 15.3.5 and 15.8.7.)*
2118 - **Native Linux Deployment:** Benchmark on a Raspberry Pi native container footprint is ~75MB without
2119 macOS hypervisor overhead.
2120 - **Mesh-Wide Filesystem (SFTP over Tailscale):** Any mesh device passively exposes its filesystem over
2121 SFTP. Android via Termux + OpenSSH + wakelock; iOS is client-only due to background process limits.
2122 - **Post-Quantum Cryptography (PQC):** Tailscale uses Curve25519, theoretically vulnerable to "harvest
2123 now, decrypt later" quantum attacks. A future iteration could replace the Tailscale container with
2124 raw WireGuard + [Rosenpass](https://rosenpass.eu/) to negotiate post-quantum PSKs.
2125 - **Egress Compartmentalization (Pluggable Architecture):** See 12.9 for the full plan to make the entire
2126 egress layer swappable via a single '.env' variable.

```

5 diagrams.md

```

1 # nullexit Flow Diagrams & Architecture
2
3 ---
4
5 ## 1. System Architecture What's Running & Where
6
7 ``mermaid
8 graph TB
9     subgraph INTERNET[" Internet / Cloudflare Edge"]
10         CF["Cloudflare WARP\n(Exit Point)\nwarp=on"]
11     end
12
13     subgraph TAILNET[" Tailscale Mesh (100.x.x.x)"]
14         PHONE[" Phone / Laptop\n(Tailnet Client)"]
15     end
16
17     subgraph MACHOST[" macOS Host"]
18         direction TB
19         PFFIREWALL["macOS pf Firewall\n(Kill-Switch & TCP MSS Clamping)"]
20         TSDAEMON["tailscaled\n(brew service)\nhost Tailscale"]
21         HOSTDNS["Host DNS\nGateway IP\n(hijacked by toggle.sh)"]
22         HOSTSOCKS["Host SOCKS5\n127.0.0.1:1080\n(fallback only)"]
23         HOSTTOR["Host Tor SOCKS5\n127.0.0.1:$TOR SOCKS_PORT\n(randomized)"]
24         HOSTTORCTRL["Host Tor Control\n127.0.0.1:$TOR_CONTROL_PORT\n(randomized)"]
25
26         subgraph MONITORS[" Background Daemons (toggle.sh children)"]
27             WARPWATCH["WARP Watcher\n(every 30s, via docker exec)\noutput.log"]
28             DNSWATCHER["DNS Watcher\n(re-hijacks if drift detected)\noutput.log"]
29             CAFFEINATE["caffeinate -i\n(sleep prevention)"]
30         end
31     end
32
33     subgraph COLIMAVM[" Colima VM (Linux, 600MB RAM)"]
34         subgraph NETNS["warp container network namespace (shared by ALL containers)"]
35             GLUETUN["warp / Gluetun\nWireGuard tun0\n(owns the namespace)"]
36             TS["tailscale container\nAdvertises exit node\ntailscale0 interface"]
37             SOCKS["tailscale\nSOCKS5\nport 1080"]
38             ADGUARD["adguardhome\nDNS sinkhole\nport 5335"]
39             ROUTINGFIX["routing-fix sidecar\nGeo-IP Blocking & Go Logger\n(writes blocked.log)"]
40             TOR["tor container\nSOCKS5: port 9050\nControl: port 9051\nTransparent Proxy: port 9040\nnDNSPort: port 5353"]
41         end
42     end
43
44     subgraph LAUNCHD[" launchd (always-on)"]
45         WATCHER["scripts/watcher.sh\n(sleep/wake + Wi-Fi roam\n+ LAN P2P auto-detection)"]
46     end

```

```

47 subgraph RECOVERY[" Recovery"]
48     RECOVER["recover.sh\n(nuclear reset)"]
49     RECOVERPOST["recover.sh --post-wake\n(light refresh)"]
50 end
51
52 %% Traffic flow
53 PHONE -->|"WireGuard / Tailscale mesh"| TSDAEMON
54 TSDAEMON -->|"exit-node route utun*"| TS
55 TS -->|"FORWARD tun0 (MASQUERADE)"| GLUETUN
56 GLUETUN -->|"WireGuard UDP (macOS Host Egress)"| PFFIREWALL
57 PFFIREWALL -->|"TCP MSS Clamped / Kill-Switch check"| CF
58 TS -->|"198.18.0.0/15 PREROUTING redirect"| TOR
59 TOR -->|"internal path / exit nodes via tun0"| GLUETUN
60
61 %% DNS
62 HOSTDNS -->|"UDP:53 TCP:5354"| ADGUARD
63 ADGUARD -->|"DNS upstream via tun0"| CF
64 ADGUARD -->|"onion queries localhost:5353"| TOR
65
66 %% Monitoring
67 WARPWATCH -->|"docker exec warp wget cdn-cgi/trace"| GLUETUN
68 DNSWATCHER -->|"networksetup -getdnsservers"| HOSTDNS
69
70 %% Logging
71 ROUTINGFIX -->|"writes"| BLOCKEDLOG["blocked.log\n(Host-side)"]
72 HOSTTOR -->|"port mapping"| TOR
73 HOSTTORCTRL -->|"port mapping"| TOR
74
75 %% Recovery triggers
76 WARPWATCH -->|"6 consecutive warp=off"| RECOVER
77 WATCHER -->|"sleep/wake / roam"| RECOVERPOST
78 RECOVERPOST -->|"if gateway unhealthy"| RECOVER
79 WATCHER -->|"writes .lan_p2p_detected"| ROUTINGFIX
80
81 style MONITORS fill:#1a1a2e,color:#e0e0ff
82 style COLIMAVM fill:#0d2137,color:#e0e0ff
83 style NETNS fill:#0a1628,color:#e0e0ff
84 style RECOVERY fill:#2d0a0a,color:#ffe0e0
85 style INTERNET fill:#0a2d1a,color:#e0ffe0
86 style TAILNET fill:#1a2d0a,color:#e8ffe0
87 style LAUNCHD fill:#2d1a2d,color:#ffe0ff
88
89 ...
90
91 ---
92
93 ## 2. toggle.sh Full START Flow
94
95 ``mermaid
96 flowchart TD
97     START(["./toggle.sh"])
98     CHECK{"is_gateway_active?\n(containers running OR\nDNS 1.1.1.1)"}
99
100     START --> CHECK
101     CHECK -->|"YES already ON"| STOPFLOW[" See STOP Flow"]
102     CHECK -->|"NO start it"| S1
103
104     S1["1. Reset DNS 1.1.1.1\n(prevent deadlocks)"]
105     S2["2. tailscale down\n(prevent exit-node routing during boot)"]
106     S3["3. Check Colima VM\n(colima start --memory 0.6 --vm-type vz --network-address --network-mode bridged)"]
107     S4["4. Configure VM swap\n(400MB, prevent OOM)"]
108     S5["5. Clean corrupted AdGuard config\n(prevent container crash loop)"]
109     S6["6. docker compose up -d --build\n(compile Go threat rules & boot containers)"]
110     S7["7. Poll tailscale status\n(wait for 'offers exit node'\nup to 60s)"]
111     TSREADY{"container\nTailscale ready?"}
112     ABORT([" ABORT\n cleanup_handler"])
113     S8["8. Resolve gateway Tailscale IP\n(.gateway_ip or docker exec)"]
114     S9["9. Verify tailscaled running\n(auto-start if needed)"]
115     S10["10. tailscale up --reset\n--exit-node=\n--accept-routes=true\n--ssh=true --accept-dns=false"]
116
117     PF1["Pre-flight 1/3\ntailscale ping gateway"]
118     PF2["Pre-flight 2/3\ndig +tcp AdGuard DNS"]
119     PF3["Pre-flight 3/3\nWARP internet check"]
120     ALLPASS["All 3 pass?"]
121
122     EXITPATH[" EXIT NODE PATH\n tailscale up --exit-node=\n force_dns_to_gateway (3 retry)\n SOCKS5 disabled"]
123     SOCKSPATH[" SOCKS5 FALLBACK PATH\n macOS SOCKS5 127.0.0.1:1080\n DNS proxy 127.0.0.1:53\n No exit node (SKIP_EXIT_NODE=true)"]
124

```

```

125 DAEMONS["Start background daemons\n caffeineate -i (sleep prevention)\n DNS Watcher\n WARP Watcher\n(
all write output.log)"]
126
127 RULECOMPILE["Rule compiler status\n(log rule/IP counts)"]
128 WRITEMARKER["write_gateway_active_marker\n(/tmp/nullexit-gateway-active.marker)"]
129 DONE([" Gateway UP"])
130
131 S1 --> S2 --> S3 --> S4 --> S5 --> S6 --> S7
132 S7 --> TSREADY
133 TSREADY -->|"NO (40 NoState)"| ABORT
134 TSREADY -->|"YES"| S8
135 S8 --> S9 --> S10
136 S10 --> PF1 --> PF2 --> PF3
137 PF1 & PF2 & PF3 --> ALLPASS
138 ALLPASS -->|"YES"| EXITPATH
139 ALLPASS -->|"NO"| SOCKSPATH
140 EXITPATH --> DAEMONS
141 SOCKSPATH --> DAEMONS
142 DAEMONS --> RULECOMPILE --> WRITEMARKER --> DONE
143
144 style ABORT fill:#5c1010,color:#fff
145 style EXITPATH fill:#0a3d1a,color:#c0ffc0
146 style SOCKSPATH fill:#3d2d00,color:#ffe0a0
147 style DAEMONS fill:#0a1f3d,color:#c0d8ff
148 style DONE fill:#0a3d1a,color:#fff
149
150
151 ---
152
153 ## 3. toggle.sh STOP Flow
154
155 ```mermaid
156 flowchart TD
157 STOP(["./toggle.sh\n(gateway already ON)"])
158
159 ST1["1. tailscale down\n(disconnect from mesh)"]
160 ST2["2. docker compose down -t 5\n(stop all containers)"]
161 ST3["3. Reset DNS 1.1.1.1\n(networksetup on all services)"]
162 ST4["4. stop_local_dns_proxy\n(kill Python UDP proxy)"]
163 ST5["5. stop_pidfile_daemon\n(kill caffeineate, rm PID)"]
164 ST6["6. stop_pidfile_daemon\n(kill DNS Watcher, rm PID)"]
165 ST7["7. stop_warp_watcher\n(kill WARP Watcher, rm PID)"]
166 ST8["8. clear_gateway_active_marker\n(rm /tmp/nullexit-gateway-active.marker)"]
167 ST9["9. cleanup_network_state\n disable_all_proxies\n Flush DNS cache (dscacheutil)\n Flush stale
routes\n Power-cycle Wi-Fi (offon)"]
168
169 DONE([" Gateway DOWN\nInternet restored"])
170
171 STOP --> ST1 --> ST2 --> ST3 --> ST4
172 ST4 --> ST5 --> ST6 --> ST7 --> ST8 --> ST9 --> DONE
173
174 style DONE fill:#0a3d1a,color:#fff
175
176
177 ---
178
179 ## 4. Monitoring Layer The 3 Background Daemons
180
181 ```mermaid
182 graph LR
183 subgraph DAEMONS["Background Daemons (all start with gateway, stop on teardown)"]
184 direction TB
185
186 subgraph D1[" Sleep Prevention"]
187 CAFF["caffeinate -i\nPID: /tmp/nullexit-caffeinate.pid\n\nKeeps Mac awake while\ngateway is
active"]
188 end
189
190 subgraph D2[" DNS Watcher"]
191 DNSW["Polls networksetup every ~30s\nPID: /tmp/nullexit-dns-watcher.pid\n\nIf DNS drifts from
gateway IP\n re-hijacks automatically\n output.log"]
192 end
193
194 subgraph D3[" WARP Watcher"]
195 WARPW["Polls every 30s via:\ndocker compose exec warp wget cdn-cgi/trace\nPID: /tmp/nullexit-
warp-watcher.pid\n\nCounts consecutive warp=off\n WARP_FAIL_THRESHOLD (default 6=30s)\n runs recover
.sh (nuclear)\n output.log"]
196 end
197
198 end
199

```

```

200 subgraph BLIND["What each monitor can/can't see"]
201     B1["WARP Watcher sees:\n Container WARP tunnel state\n Host NIC traffic\n Flash leaks < 5s"]
202     B2["Host-NIC blind spot covered by:\n PF kill-switch (kernel, fail-closed)\n on-demand: diagnose-
host-leak.sh\n(--watch to monitor) / sweep.sh"]
203 end
204
205 D3 -->|"covers"| B1
206 B1 -->|"gap closed by"| B2
207
208 style D1 fill:#1a2d0a
209 style D2 fill:#0a1a2d
210 style D3 fill:#2d0a2d
211 style BLIND fill:#1a1a1a,color:#aaa
212 '''
213
214 ---
215
216 ## 5. Traffic Flow Data Path (Normal Operation)
217
218 '''mermaid
219 sequenceDiagram
220 participant PHONE as Phone<br/>(Tailnet client)
221 participant TS_HOST as tailscaled<br/>(macOS host)
222 participant TS_CONTAINER as tailscale<br/>(container)
223 participant GLUETUN as Gluetun/warp<br/>(container)
224 participant CF as Cloudflare<br/>WARP Edge
225 participant INTERNET as Internet
226
227 Note over PHONE,INTERNET: Normal EXIT NODE path (all 3 pre-flights passed)
228
229 PHONE->>TS_HOST: WireGuard packet<br/>(encrypted, UDP 41641)
230 TS_HOST->>TS_CONTAINER: route via utun* tailscale0
231 TS_CONTAINER->>GLUETUN: FORWARD chain<br/>(iptables MASQUERADE on tun0)
232 GLUETUN->>TS_HOST: WireGuard tunnel (tun0) egresses macOS Host
233 Note over TS_HOST: macOS pf Firewall applies<br/>Kill-Switch & TCP MSS Clamping
234 TS_HOST->>CF: re-encrypted UDP packet (MSS 1160)
235 CF->>INTERNET: Plain HTTPS from<br/>Cloudflare IP
236
237 Note over PHONE,INTERNET: DNS Resolution path
238 PHONE->>TS_HOST: DNS query
239 TS_HOST->>TS_CONTAINER: UDP:53 TCP:5354 AdGuard :5335
240 TS_CONTAINER->>GLUETUN: AdGuard upstream DNS via tun0
241 GLUETUN->>CF: Encrypted DNS query
242 CF->>GLUETUN: DNS response
243 GLUETUN->>TS_CONTAINER: response
244 TS_CONTAINER->>TS_HOST: response
245 TS_HOST->>PHONE: DNS answer
246 '''
247
248 ---
249
250 ## 6. recover.sh Decision Tree
251
252 '''mermaid
253 flowchart TD
254     INVOKE(["recover.sh called"])
255     MODE{"--post-wake\nflag?"}
256
257     subgraph POSTWAKE["--post-wake (light refresh, gateway stays UP)"]
258         PW1["Re-hijack DNS gateway IP"]
259         PW2["Flush DNS cache"]
260         PW3["Refresh tailscale exit-node"]
261         PW4["Force-recreate warp container\n(if Gluetun healthcheck failed)"]
262         PW5["Leave: containers, Wi-Fi,\ncaffeinate, DNS watcher untouched"]
263     end
264
265     subgraph NUCLEAR["Default (nuclear gateway torn down)"]
266         NO["Sudo keep-alive loop (background)"]
267         N1["1. tailscale down\n(disconnect from mesh)"]
268         N2["2. Reset DNS 1.1.1.1\n(ALL network services)"]
269         N3["3. disable_all_proxies\n(all interfaces)"]
270         N4["4. Flush DNS cache\n(dscacheutil -flushcache)"]
271         N5["5. Flush stale routes\n(WARP endpoint routes)"]
272         N6a["6a. docker compose down -t 30\n(stop all containers)"]
273         N6b["6b. stop_pidfile_daemon\n(kill caffeinate)"]
274         N6c["6c. stop_pidfile_daemon\n(kill DNS Watcher)"]
275         N7["7. Power-cycle Wi-Fi\n(networksetup offsleep 2on)"]
276         N8["8. Verify internet working\n(curl ifconfig.io)"]
277     end
278
279     DONE_PW(["Gateway refreshed\n(still running)"])

```

```

280 DONE_NUC([" Internet restored\n(gateway fully stopped)"])
281
282 INVOKE --> MODE
283 MODE -->|"yes"| PW1
284 PW1 --> PW2 --> PW3 --> PW4 --> PW5 --> DONE_PW
285 MODE -->|"no"| NO
286 NO --> N1 --> N2 --> N3 --> N4 --> N5
287 N5 --> N6a --> N6b --> N6c --> N7 --> N8 --> DONE_NUC
288
289 style POSTWAKE fill:#0a2d1a,color:#c0ffc0
290 style NUCLEAR fill:#2d0505,color:#ffc0c0
291 style DONE_PW fill:#0a3d1a,color:#fff
292 style DONE_NUC fill:#3d1a00,color:#ffe0c0
293
294
295 ---
296
297 ## 7. Failure Paths & Self-Healing
298
299 'mermaid
300 flowchart LR
301     subgraph EVENTS["Failure / Event"]
302         E1["WARP tunnel drops\n(warp=off)"]
303         E2["Mac wakes from sleep\nor Wi-Fi roams"]
304         E3["DNS drifts from\ngateway IP"]
305         E4["toggle.sh crashes /\nCtrl-C / SIGTERM"]
306         E5["Host-side leak\n(non-tunnel HOST NIC egress)"]
307         E6["Silent NAT timeout\n(ping stops working)"]
308     end
309
310     subgraph DETECTORS["Detector"]
311         D1["WARP Watcher\n(30s poll, docker exec)"]
312         D2["scripts/watcher.sh\n(launchd, always on)"]
313         D3["DNS Watcher\n(30s poll, networksetup)"]
314         D4["cleanup_handler\n(ERR/INT/TERM/HUP trap)"]
315         D5["PF kill-switch\n(kernel, always-on) +\ndiagnose-host-leak.sh --watch\n(on-demand)"]
316         D6["routing-fix.sh\n(30s curl over tun0)"]
317     end
318
319     subgraph ACTIONS["Response"]
320         A1["threshold consecutive off\nrecover.sh (nuclear)"]
321         A2["recover.sh --post-wake\n(gentle refresh)"]
322         A3["Re-hijack DNS\n(networksetup setdnsservers)"]
323         A4["Stop all daemons\nReset DNS 1.1.1.1\nTear down Tailscale"]
324         A5["Non-tunnel egress blocked\nfail-closed at kernel;\n--watch alerts on state change"]
325         A6["Blackhole internet traffic\n+ Drop TUNNEL_FAILED_CLOSED.marker"]
326         A7["Push macOS Desktop Notification\n(via watcher.sh listener)"]
327     end
328
329     E1 --> D1 --> A1
330     E2 --> D2 --> A2
331     E3 --> D3 --> A3
332     E4 --> D4 --> A4
333     E5 --> D5 --> A5
334     E6 --> D6 --> A6
335     A6 -->|"marker detected"| D2
336     D2 --> A7
337
338     style EVENTS fill:#2d1010,color:#ffcccc
339     style DETECTORS fill:#0a1a2d,color:#cce0ff
340     style ACTIONS fill:#0a2d0a,color:#ccffcc
341
342
343 ---
344
345 ## 8. output.log Who Writes What
346
347 All events converge into a single 'output.log' at the repo root.
348
349 | Writer | Event Types | Format |
350 |-----|-----|-----|
351 | 'toggle.sh' | All startup/shutdown steps, errors, pre-flight results | Plain text with timestamps |
352 | 'recover.sh' | Every recovery step (nuclear or post-wake) | Plain text |
353 | **WARP Watcher** | 'WARP DOWN', 'WARP RECOVERED', 'WARP SHUTDOWN' | '[UTC timestamp]' prefix |
354 | **DNS Watcher** | DNS re-hijack events | Appended inline |
355 | 'docker compose' | Container logs on failure (last 100 lines of warp) | Dumped on ERR path |
356 | 'routing-fix.sh' | (via 'nullexit-logger' inside container) | Structured |
357
358 **Grep cheatsheet:**
359 'bash
360 grep 'WARP DOWN' output.log # Container-side tunnel drops

```

```

361 grep 'WARP SHUTDOWN' output.log # Auto-recovery triggered
362 grep 'WARP RECOVERED' output.log # Self-healed before threshold
363 grep -E 'EXEC|EXIT' output.log # Lifecycle breadcrumbs (last command + exit)
364 ''

```

6 toggle.sh

```

1  #!/bin/bash
2  # toggle.sh nullexit gateway toggle script (macOS + Linux)
3  # Automatically handles Docker containers, Colima, DNS hijacking, and Tailscale exit node routing.
4  #
5  # One file, both OSes: the dispatch below runs the self-contained Linux
6  # implementation (shuf / ss / nmcli / ip / systemd) and exits; on macOS it
7  # falls through to the macOS implementation (jot / netstat / networksetup,
8  # launchd / pf).
9
10 set -e
11
12 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
13
14 #
15 # LINUX TOGGLE self-contained; runs the full toggle then exits before the
16 # macOS body below. Native Linux tooling (shuf / ss / nmcli / ip / systemd).
17 # (Body inlined verbatim at column 0 it contains multiline 'bash -c "'
18 # strings that must NOT be re-indented.)
19 #
20 if [[ "$OSTYPE" != "darwin"* ]]; then
21 # Define log file (used by the lifecycle breadcrumb + run_logged helper)
22 LOG_FILE="$PWD/output.log"
23
24 # --- LIFECYCLE PHASE TRACKING + EXIT BREADCRUMB ---
25 # TOGGLE_PHASE is updated as the script moves through STOP/START so that if the
26 # process is killed (terminal closed, pkill, OOM) or exits on error, the EXIT
27 # trap records the final exit code AND the phase it died in making an
28 # interrupted START traceable in output.log instead of ending abruptly.
29 TOGGLE_PHASE="init"
30 _log_exit_breadcrumb() {
31     local rc=$?
32     echo "[$(date +%Y-%m-%d %H:%M:%S')] toggle.sh EXIT: code=$rc phase=$TOGGLE_PHASE (pid $$)" >> "
33         $LOG_FILE" 2>/dev/null || true
34 }
35 trap '_log_exit_breadcrumb' EXIT
36
37 # --- DEBUG TRACE (opt-in via .env: DEBUG_TRACE=true) ---
38 # When enabled, mirror a full command-by-command xtrace to output.log. Off by
39 # default. Read directly from .env because common.sh (read_env_var) isn't
40 # sourced yet. Branch on bash version: BASH_XTRACEFD needs >= 4.1 (Linux
41 # usually has bash 5); fall back to teeing stderr on older bash. We never use
42 # the 'exec {var}>>' dynamic-fd form (4.1+ only).
43 DEBUG_TRACE=$(grep -E '^DEBUG_TRACE=' .env 2>/dev/null | cut -d '=' -f2- | tr -d '\\"' | tr '[:upper:]' '
44     [:lower:]' | tr -d '[:space:]')
45 if [ "$DEBUG_TRACE" = "true" ]; then
46     echo "[$(date +%Y-%m-%d %H:%M:%S')] DEBUG_TRACE enabled xtrace mirrored to output.log (bash ${
47         BASH_VERSIONINFO[0]}.${BASH_VERSIONINFO[1]})" >> "$LOG_FILE"
48     export PS4='+ [${SECONDS}s] ${BASH_SOURCE##*/}:${LINENO}:${FUNCNAME[0]:-main}: ' # subshell-free clock
49     : a $(() in PS4 trips the bash 3.2 set -x/ERR heisenbug (devref 15.11.8)
50     if [ "${BASH_VERSIONINFO[0]}" -gt 4 ] || { [ "${BASH_VERSIONINFO[0]}" -eq 4 ] && [ "${BASH_VERSIONINFO[1]}" -ge
51         1 ]; }; then
52         exec 9>>"$LOG_FILE"
53         export BASH_XTRACEFD=9
54         set -x
55     else
56         # bash < 4.1: plain stderrfile redirect. NOT a process substitution under
57         # set -e, xtrace flowing through a backgrounded 'tee' can abort the script
58         # (failed write/SIGPIPE trips set -e). See devref 15.11.8.
59         exec 2>>"$LOG_FILE"
60         set -x
61     fi
62 fi
63
64 # --- PROCEDURAL PORT GENERATION ---
65 # To prevent static port fingerprinting by malware, we randomize exposed ports
66 # on every boot and persist them to a hidden environment file.
67 PORTS_FILE="/tmp/nullexit-ports.env"
68 if [[ "$1" == "" || "$1" == "--restart" || "$1" == "restart" ]]; then

```



```

65 # Collision-avoidance function
66 GENERATED_PORTS=""
67 get_free_port() {
68     local port
69     while true; do
70         # Linux uses shuf or $RANDOM instead of jot
71         port=$(shuf -i 10000-65000 -n 1 2>/dev/null || echo $((10000 + RANDOM % 55000)))
72         # 1. Prevent self-collision within this loop
73         if [[ "$GENERATED_PORTS" == *"$port"* ]]; then
74             continue
75         fi
76         # 2. Prevent collision with ANY process on the machine (root daemons, terminal apps, etc)
77         # We use ss to read the kernel socket table directly.
78         if ! ss -lnt | awk '{print $4}' | grep -q ":$port$"; then
79             echo "$port"
80             return
81         fi
82     done
83 }
84
85 export TOR SOCKS_PORT=$(get_free_port)
86 GENERATED_PORTS="$GENERATED_PORTS $TOR SOCKS_PORT"
87
88 export TOR_CONTROL_PORT=$(get_free_port)
89 GENERATED_PORTS="$GENERATED_PORTS $TOR_CONTROL_PORT"
90
91 export SOCKS_PROXY_PORT=$(get_free_port)
92 GENERATED_PORTS="$GENERATED_PORTS $SOCKS_PROXY_PORT"
93
94 export DNS_PROXY_PORT=$(get_free_port)
95
96 echo "export TOR SOCKS_PORT=$TOR SOCKS_PORT" > "$PORTS_FILE"
97 echo "export TOR_CONTROL_PORT=$TOR_CONTROL_PORT" >> "$PORTS_FILE"
98 echo "export SOCKS_PROXY_PORT=$SOCKS_PROXY_PORT" >> "$PORTS_FILE"
99 echo "export DNS_PROXY_PORT=$DNS_PROXY_PORT" >> "$PORTS_FILE"
100 elif [ -f "$PORTS_FILE" ]; then
101     # On STOP, source the existing ports so docker-compose down matches
102     source "$PORTS_FILE"
103 else
104     # Fallbacks if file is lost
105     export TOR SOCKS_PORT=9050
106     export TOR_CONTROL_PORT=9051
107     export SOCKS_PROXY_PORT=1080
108     export DNS_PROXY_PORT=5354
109 fi
110
111 # Start execution timer
112 TOGGLE_START_TIME=$SECONDS
113
114 if [[ "$1" == "restart" ]] || [[ "$1" == "--restart" ]]; then
115     echo "Executing Gateway Restart Sequence..."
116     # We must source common.sh here temporarily to check gateway state before we proceed
117     source "$SCRIPT_DIR/scripts/common.sh"
118     # Use persisted intent via gateway_state (echoes a string, never a return code;
119     # set -e-safe under set -x see devref 15.11.8). On Linux the marker is not
120     # maintained, so this degrades to the is_gateway_active fallback (fast native Docker).
121     if [ "$(gateway_state)" = "running" ]; then
122         echo "Gateway is currently running. Stopping it first..."
123         bash "$0"
124         echo "Gateway stopped. Now starting it..."
125     else
126         echo "Gateway is already stopped. Starting it..."
127     fi
128     bash "$0"
129     exit 0
130 fi
131
132 # Global flag to track if the script completed successfully
133 SUCCESS_RUN=false
134
135 # Global variable to track the currently running background command PID
136 CURRENT_BG_PID=""
137
138 # Source common bash functions
139 source "$SCRIPT_DIR/scripts/common.sh"
140
141 PID_FILE="/tmp/nullexit-cafeinate.pid"
142
143 # Start macOS caffeinate to prevent sleep while gateway is running
144 start_sleep_prevention() {

```

```

146 if ! command -v systemd-inhibit >/dev/null 2>&1; then
147     echo " [Warning] systemd-inhibit tool not found. Sleep prevention unavailable."
148     return 1
149 fi
150
151 # Stop any existing sleep prevention process first to avoid duplicates
152 stop_sleep_prevention
153
154 echo " Enabling system sleep prevention (systemd-inhibit)..."
155 # Run systemd-inhibit wrapped in a bash trap to prevent idle system sleep.
156 # Critically, this traps shutdown signals (SIGTERM) and automatically
157 # flushes hijacked DNS back to normal right before the PC powers off.
158 # We do not use recover.sh here because it is a heavy recovery script
159 # which takes too long and gets terminated before it can reset the DNS. Instead,
160 # we surgically and instantly restore normal DNS settings here.
161 nohup bash -c "
162     trap '
163         echo \"[Shutdown Trap] Reverting network settings...\\" >> \"\$PWD/output.log\" 2>&1
164         kill \$! 2>/dev/null || true
165         sudo -n resolvectl domain \"\$ACTIVE_SERVICE\" \"\" >> \"\$PWD/output.log\" 2>&1 || true
166         sudo -n resolvectl revert \"\$ACTIVE_SERVICE\" >> \"\$PWD/output.log\" 2>&1 || true
167         resolvectl domain \"\$ACTIVE_SERVICE\" \"\" >> \"\$PWD/output.log\" 2>&1 || true
168         resolvectl revert \"\$ACTIVE_SERVICE\" >> \"\$PWD/output.log\" 2>&1 || true
169         if command -v tailscale >/dev/null 2>&1; then
170             tailscale up --accept-dns=false --exit-node= >> \"\$PWD/output.log\" 2>&1 &
171             TS_DOWN_ARGS=\"\"
172             if [ \"\$KILL_SWITCH\" = \"true\" ]; then TS_DOWN_ARGS=\"--accept-risk=lose-ssh\"; fi
173             tailscale down \"\$TS_DOWN_ARGS >> \"\$PWD/output.log\" 2>&1 &
174         fi
175         sleep 1
176         echo \"[Shutdown Trap] Cleanup complete.\\" >> \"\$PWD/output.log\" 2>&1
177         exit 0
178     ' SIGTERM SIGINT SIGHUP
179     systemd-inhibit --what=sleep --who=nullexit --why=\"gateway running\" --mode=block sleep infinity &
180     wait \$!
181     \" >> output.log 2>&1 &
182     local caffe_pid=$!
183     echo \"$caffe_pid\" > \"$PID_FILE"
184     echo " Sleep prevention active (PID $caffe_pid). Your PC won't sleep while the gateway is running."
185 }
186
187 # Stop sleep prevention when gateway is stopped
188 stop_sleep_prevention() {
189     if [ -f "$PID_FILE" ]; then
190         local caffe_pid
191         caffe_pid=$(cat "$PID_FILE")
192         if [ -n "$caffe_pid" ] && kill -0 "$caffe_pid" 2>/dev/null && ps -p "$caffe_pid" -o command= 2>/dev/
193             null | grep -q -E "systemd-inhibit|recover.sh"; then
194             echo " Stopping system sleep prevention (systemd-inhibit wrapper PID $caffe_pid)..."
195             kill "$caffe_pid" 2>/dev/null || true
196         fi
197         rm -f "$PID_FILE"
198     fi
199 }
200
201 DNS_WATCHER_PID_FILE="/tmp/nullexit-dns-watcher.pid"
202
203 stop_dns_watcher() {
204     if [ -f "$DNS_WATCHER_PID_FILE" ]; then
205         local watcher_pid
206         watcher_pid=$(cat "$DNS_WATCHER_PID_FILE")
207         if [ -n "$watcher_pid" ] && kill -0 "$watcher_pid" 2>/dev/null; then
208             echo " Stopping background DNS Watcher (PID $watcher_pid)..."
209             kill -9 "$watcher_pid" 2>/dev/null || true
210         fi
211         rm -f "$DNS_WATCHER_PID_FILE"
212     fi
213 }
214
215 # Cleanup handler to restore DNS to 1.1.1.1 on error or user interrupt (Ctrl+C / SIGTERM / SIGHUP)
216 cleanup_handler() {
217     local exit_code=$?
218     local trigger_type="$1"
219     local line_no="$2"
220
221     if [ "$SUCCESS_RUN" = "false" ]; then
222         echo -e "\n[Self-Correction] Script interrupted or failed ($trigger_type). Restoring host DNS to
223         1.1.1.1..."
224         if [ -n "$ACTIVE_SERVICE" ]; then
225             reset_dns
226         fi
227     fi

```

```

225
226 if [ -n "$CURRENT_BG_PID" ]; then
227     echo "Terminating active command (PID $CURRENT_BG_PID)..."
228     kill -15 "$CURRENT_BG_PID" 2>> output.log || true
229 fi
230
231 # Disconnect host Tailscale and close the GUI application on failure
232 disconnect_tailscale_host
233
234 # Stop local DNS proxy if running
235 stop_local_dns_proxy
236
237 # Stop sleep prevention
238 stop_sleep_prevention
239 stop_dns_watcher
240
241 # Nuke leftover network state (proxies, routes, DNS cache, Wi-Fi)
242 if [ -n "$ACTIVE_SERVICE" ]; then
243     cleanup_network_state
244 fi
245
246
247 if [ "$trigger_type" = "ERR" ]; then
248     echo -e "\n====="
249     echo "ERROR: Script failed at line $line_no with exit code $exit_code."
250     echo "====="
251     echo "Troubleshooting steps:"
252     echo "1. Run 'colima status' to verify the VM is active."
253     echo "2. Run 'docker compose ps' to check container health."
254     echo "3. Run 'docker compose logs' to view application logs."
255     echo "4. Run 'tailscale status' to check host Tailscale status."
256     echo "====="
257 fi
258 fi
259 }
260
261 trap 'cleanup_handler ERR $LINENO' ERR
262 trap 'cleanup_handler INT' INT
263 trap 'cleanup_handler TERM' TERM
264 trap 'cleanup_handler HUP' HUP
265
266 # NOPASSWD sudo
267 # The user has NOPASSWD in /etc/sudoers.d/nullexit for the specific commands
268 # needed (resolvectl, ip, systemctl, systemd-inhibit, pkill, and
269 # python3 scripts/dns-proxy.py for the fallback DNS proxy).
270 # All privileged calls below use 'sudo -n' which will run without prompting.
271 # No credential caching loop is needed.
272
273 # Add Homebrew and standard paths
274 export PATH="/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:$PATH"
275
276 # Check for local DNS proxy tools (used when tailnet data plane is unavailable)
277 # Uses a Python DNS proxy (handles TCP wire format correctly).
278 # Python3 is built into macOS no external dependencies.
279 DNS_PROXY_BIN=""
280 if command -v python3 >/dev/null 2>&1; then
281     DNS_PROXY_BIN="python3 -I"
282 elif command -v python >/dev/null 2>&1; then
283     DNS_PROXY_BIN="python -I"
284 fi
285
286 # Path to the DNS proxy script (sibling of this script)
287 DNS_PROXY_SCRIPT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/dns-proxy.py"
288
289 # Resolve Tailscale CLI on host. We rely on the standalone Homebrew formula
290 # ('brew install tailscale' + 'systemctl start tailscaled') NOT the
291 # Tailscale.app GUI so there is no .app bundle or Network Extension sandbox
292 # to launch, no menu-bar click to perform, and no scutil Network Service to
293 # start manually. tailscaled runs as a per-user LaunchAgent (or LaunchDaemon
294 # if you ran the install with sudo) and the control socket is always available.
295 TS_BIN=""
296 if command -v tailscale >> output.log 2>&1; then
297     TS_BIN="tailscale"
298 fi
299
300 # Baseline: disable Tailscale DNS management at the daemon level.
301 # 'tailscale set' writes a persistent preference. However, 'tailscale up --reset'
302 # (used in steps 7 and 9) RESETS all unspecified flags to defaults, which would
303 # re-enable '--accept-dns=true' and nullify this 'set'.
304 # Therefore, EVERY 'tailscale up --reset' call in this file MUST also pass
305 # '--accept-dns=false' explicitly. See steps 7 and 9 for the fix.

```

```

306 #
307 # This initial 'set' is still useful as a baseline for non-reset operations
308 # (e.g. if the user runs 'tailscale up' manually without --reset) and covers
309 # the brief window between this line and the first '--reset' call.
310 #
311 # Runs once at script start while tailscaled is still connected (if it was
312 # left running from a previous toggle), so the pref takes effect before any
313 # disconnection or reconnection logic.
314 if [ -n "$TS_BIN" ]; then
315     $TS_BIN set --accept-dns=false >> output.log 2>&1 || true
316 fi
317
318 # Disconnect host Tailscale from the mesh. With the standalone daemon, this is the
319 # *entire* teardown no need to mess with system network managers.
320 # tailscaled keeps running (as a systemd service), which is intentional:
321 # the next 'tailscale up' then reconnects in seconds rather than waiting for a
322 # slow daemon launch. The 'tailscale down' call also retains the user's
323 # auth state, so we don't force a browser re-login every cycle.
324 #
325 # Defined early (before the if/else branches and referenced by cleanup_handler's
326 # ERR trap) so that any failure before the main logic can still tear down Tailscale.
327 restart_tailscaled_daemon() {
328     echo " [Tailscale Recovery] tailscaled daemon appears to be wedged/unresponsive."
329     echo " [Tailscale Recovery] Attempting to restart tailscaled service via systemctl..."
330     if run_with_timeout 15 sudo -n systemctl restart tailscaled >> output.log 2>&1; then
331         echo " [Tailscale Recovery] Successfully restarted tailscaled service."
332     else
333         echo " [Tailscale Recovery] WARNING: Failed to restart tailscaled. Manual intervention may be needed"
334     fi
335     sleep 3
336 }
337 disconnect_tailscale_host() {
338     if [ -n "$TS_BIN" ]; then
339         echo "Disconnecting host Tailscale from mesh (tailscaled stays running as system service)..."
340         local ts_args=""
341         if is_kill_switch_enabled; then ts_args="--accept-risk=lose-ssh"; fi
342         if ! run_with_timeout 10 $TS_BIN down $ts_args >> output.log 2>&1; then
343             restart_tailscaled_daemon
344             run_with_timeout 10 $TS_BIN down $ts_args >> output.log 2>&1 || true
345         fi
346     fi
347 }
348
349 ACTIVE_SERVICE=$(get_active_service)
350
351 # Helper to nuke leftover network state after teardown (proxy settings, routing
352 # table, DNS cache, Wi-Fi). Prevents the "had to write a whole recovery script"
353 # problem where stale state kills internet after the gateway stops.
354 cleanup_network_state() {
355     echo -e "\nCleaning up network state (proxies, routes, DNS cache, Wi-Fi)..."
356
357     # 1. Disable any leftover proxy settings (needs root on many macOS versions)
358     # (Skipped for Linux since we don't set global SOCKS proxy)
359     echo " Proxies disabled."
360
361     # 2. Flush DNS cache
362     if command -v resolvectl >> output.log 2>&1; then
363         sudo -n resolvectl flush-caches 2>> output.log || true
364     elif command -v systemd-resolve >> output.log 2>&1; then
365         sudo -n systemd-resolve --flush-caches 2>> output.log || true
366     fi
367     echo " DNS cache flushed."
368
369     # 3. Flush stale routing table entries
370     if command -v ip >> output.log 2>&1; then
371         sudo -n ip route flush cache >> output.log 2>&1 || true
372     fi
373     echo " Routing table flushed."
374
375     # 4. Power-cycle Wi-Fi to clear any lingering interface state
376     echo " Restarting network interface..."
377     sudo -n ip link set dev "$ACTIVE_SERVICE" down 2>> output.log || true
378     sleep 1
379     sudo -n ip link set dev "$ACTIVE_SERVICE" up 2>> output.log || true
380     echo " Interface power-cycled ($ACTIVE_SERVICE)."
381     sleep 3
382
383     echo "Network state cleanup complete."
384 }
385

```

```

386 # Ensure DNS is set to 1.1.1.1 immediately at script start to prevent any DNS deadlocks/hangs
387 # during status checks, container startup, VM boot, or teardown. We *also* clear any leftover
388 # 'ts.net' search domain from a previous 'tailscale up --accept-dns=true' run, otherwise macOS
389 # would prepend it to every lookup and most public DNS queries would resolve to NXDOMAIN.
390 # Capture current DNS before resetting so we can check if it was hijacked
391 INITIAL_DNS=$(resolvectl dns "$ACTIVE_SERVICE" 2>/dev/null | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' |
    head -1 || true)
392
393 echo "Initializing DNS to 1.1.1.1 to ensure reliable internet access..."
394 reset_dns
395
396
397 # Local Network Surveillance Check
398 echo -e "\n"
399 echo "Local Network Surveillance Check"
400 echo ""
401 # Count populated ARP cache entries to detect network scanning or high congestion
402 if command -v ip >/dev/null 2>&1; then
403     ARP_COUNT=$(ip neigh | grep -iv 'incomplete' | wc -l | tr -d ' ')
404 else
405     # Using '-an' avoids hanging on DNS reverse resolution when the network state is in transition.
406     ARP_COUNT=$(arp -an 2>/dev/null | grep -iv 'incomplete' | wc -l | tr -d ' ')
407 fi
408 if [ "$ARP_COUNT" -gt 15 ]; then
409     echo -e " \033[1;33m[] WARNING: High ARP activity detected ($ARP_COUNT devices in cache).\033[0m"
410     echo -e " \033[1;33m[] This Wi-Fi network may be heavily congested or actively scanned (e.g., arp-
411 scan).\033[0m"
412     echo -e " [] Your traffic remains fully encrypted and invisible.\n"
413 else
414     echo -e " [] Local network looks quiet ($ARP_COUNT devices). No aggressive scanning detected.\n"
415 fi
416 echo "Checking Gateway Status..."
417 # Decide STOP vs START from persisted intent via gateway_state, which echoes a
418 # string (never a return code) to stay set -e-safe under set -x. See devref 15.11.8.
419 # On Linux this falls back to is_gateway_active (fast native Docker).
420 if [ "$(gateway_state)" = "running" ]; then
421     TOGGLE_PHASE="stop"
422     echo -e "\n=====
423 echo "Gateway is RUNNING. Stopping it now..."
424     echo -e "=====
425
426 # 1. Disconnect host Tailscale cleanly first before stopping the exit-node container
427 disconnect_tailscale_host
428 echo ""
429
430 echo "Stopping Docker containers..."
431 # '|| true': a disable must always complete even if the Docker daemon is down.
432 run_logged docker compose down -t 5 || true
433
434 echo -e "\nDocker daemon handles container teardown automatically."
435
436 # The host's DNS was hijacked to the gateway IP during ENABLE; now that the
437 # gateway is down, restore DNS to 1.1.1.1 immediately so subsequent lookups
438 # don't stall for the macOS DNS timeout before the 1.1.1.1 fallback engages.
439 echo -e "\nRestoring host DNS to 1.1.1.1 (gateway is gone)... "
440 reset_dns
441
442 # 3. Stop local DNS proxy if running
443 stop_local_dns_proxy
444
445 # Stop sleep prevention
446 stop_sleep_prevention
447 stop_dns_watcher
448
449 # 4. Nuke leftover network state so internet actually works after teardown
450 cleanup_network_state
451
452 ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
453 echo -e "\nGateway has been successfully STOPPED in ${ELAPSED} seconds."
454 else
455     TOGGLE_PHASE="start"
456     echo -e "\n=====
457     echo "Gateway is STOPPED. Starting it now..."
458     echo -e "=====
459
460 # 1. Prevent host exit-node deadlock during VM / Container startup
461 disconnect_tailscale_host
462
463 echo -e "\nUsing native Linux Docker..."
464

```

```

465 # 4. Clean up corrupted AdGuardHome configurations
466 if [ -f "adguard/conf/AdGuardHome.yaml" ] && [ ! -s "adguard/conf/AdGuardHome.yaml" ]; then
467     rm -f "adguard/conf/AdGuardHome.yaml"
468     echo "Removed corrupted empty AdGuardHome.yaml to prevent container crash loop."
469 fi
470
471
472
473 # 5. Start compose services
474 write_host_ips
475 # Procedurally generated ports have already been exported at the top of the script.
476 echo " [Security] Procedurally randomized proxy ports generated to evade fingerprinting."
477
478 # --- HONEY-PORT TRIPWIRE ---
479 # Generate a random trap port. If any malware does a sequential port scan on localhost,
480 # it will inevitably hit this port. When it does, we instantly fire a desktop alert.
481 # Reap the honey-port from any previous toggle before arming a new one, so the
482 # 'nc -l' tripwire never accumulates one idle listener per toggle-on (15.12.20).
483 pkill -f "nc -l .*127.0.0.1" 2>/dev/null || true
484 pkill -f "nc -l localhost" 2>/dev/null || true
485 HONEY_PORT=$(get_free_port)
486 (
487     if nc -l -p "$HONEY_PORT" 127.0.0.1 >/dev/null 2>&1 || nc -l localhost "$HONEY_PORT" >/dev/null 2>&1;
488     then
489         echo "[$(date +%Y-%m-%d %H:%M:%S)] [CRITICAL SECURITY ALERT] PORT SCAN DETECTED! An unknown
490         application just pinged the local Honey-Port ($HONEY_PORT)!" >> "$PWD/output.log"
491         if command -v notify-send >/dev/null 2>&1; then
492             notify-send -u critical "nullexit OPSEC Alert" "An unknown application is aggressively scanning
493             your local ports. Malware port-scan suspected."
494         fi
495     fi
496 ) &
497 disown %1 2>/dev/null || true
498 echo " [Security] Honey-Port Tripwire armed on random port to detect malware port-scans."
499
500 echo -e "\nStarting Docker containers..."
501 run_logged docker compose up -d
502
503 # Clean up the one-off rule compiler container so it doesn't clutter the Docker UI
504 docker compose rm -s -f rule-compiler >> output.log 2>&1
505
506 # 6. Wait for the gateway container's Tailscale connection to be ready
507 BYPASS_PING=$(read_env_var GATEWAY_BYPASS_PING | tr '[:upper:]' '[:lower:]')
508 USE_EXIT_NODE=$(read_env_var GATEWAY_USE_EXIT_NODE | tr '[:upper:]' '[:lower:]')
509
510 echo "Waiting for gateway container's Tailscale to connect to the tailnet..."
511 echo " (This can take 30-60s Tailscale goes through: NoState Starting Running Online)"
512 connected=false
513 consecutive_failures=0
514 for i in {1..60}; do
515     # Run tailscale status and capture its output so the user can see what's happening.
516     # Previously this was hidden behind 2>> output.log, making it impossible to debug hangs.
517     ts_output=$(run_with_timeout 3 docker compose exec -T tailscale tailscale status 2>&1 || true)
518
519     if echo "$ts_output" | grep -q "offers exit node"; then
520         connected=true
521         break
522     fi
523
524     # Track consecutive failures to detect a stuck state.
525     # If we see 40+ consecutive 'NoState' responses, give up early
526     # the daemon is alive but can't connect to the coordination server.
527     if echo "$ts_output" | grep -q "NoState"; then
528         consecutive_failures=$((consecutive_failures + 1))
529         if [ "$consecutive_failures" -ge 40 ]; then
530             echo ""
531             echo " [attempt $i/60] Stuck in 'NoState' for $consecutive_failures checks."
532             echo " Tailscale daemon is running but can't reach the coordination server."
533             break
534         fi
535     else
536         consecutive_failures=0
537     fi
538
539     # Print a condensed status every 10 seconds so the user can see progress
540     if [ $((i % 10)) -eq 0 ]; then
541         ts_short=$(echo "$ts_output" | head -1 | tr -d '\r\n')
542         echo " [attempt $i/60] $ts_short"
543     elif [ $((i % 5)) -eq 0 ]; then
544         echo -n "$i"
545     else

```



```

543     echo -n "."
544 fi
545 sleep 1
546 done
547 echo ""
548
549 if [ "$connected" = "true" ]; then
550     echo "Gateway container is online on the Tailscale mesh."
551 elif [ "$BYPASS_PING" = "true" ]; then
552     echo "[Warning] Gateway container check timed out. Proceeding anyway (GATEWAY_BYPASS_PING is true)..."
553 else
554     echo "ERROR: Gateway container failed to initialize Tailscale (timed out after 60s)." >&2
555     echo "The container was seen in the following states during the wait:" >&2
556     echo "$ts_output" | sed 's/^/ /' >&2
557     echo "" >&2
558     echo "This could mean:" >&2
559     echo "1. Tailscale auth key has expired check TS_AUTHKEY in .env" >&2
560     echo "2. Network issue inside the container check: docker compose logs tailscale" >&2
561     echo "3. gluetun VPN tunnel not connecting check: docker compose logs warp | tail -20" >&2
562     echo "" >&2
563     echo "Diagnose manually:" >&2
564     echo "docker compose exec -T tailscale tailscale status" >&2
565     echo "docker compose exec -T tailscale tailscale netcheck" >&2
566 cleanup_handler ERR $LINENO
567 exit 1
568 fi
569
570 # 6b. Resolve the gateway's Tailscale IP.
571 # Dynamically query the container for the IP (handles IP changes after re-auth)
572 # and cache it in .gateway_ip for fast retrieval by recover.sh during Wi-Fi roaming.
573 #
574 # CRITICAL: 'docker compose exec -T' on macOS/Colima injects carriage
575 # returns (\r) into captured output. If $TS_IP ends with \r, then
576 # 'networksetup -setdnsservers' silently rejects the malformed IP and
577 # DNS stays at 1.1.1.1 the primary bug this project was hitting.
578 # 'tr -d \r' strips the CR; 'awk 'NR==1{print $1; exit}'' takes only
579 # the first field of the first line.
580 TS_IP=""
581 if [ "$connected" = "true" ]; then
582     TS_IP=$(run_with_timeout 5 docker compose exec -T tailscale tailscale ip -4 2>> output.log | tr -d '\r' | awk 'NR==1{print $1; exit}' || true)
583     if [ -n "$TS_IP" ]; then
584         echo "Resolved gateway Tailscale IP dynamically: $TS_IP"
585         echo "$TS_IP" > .gateway_ip
586     else
587         echo "[!] Failed to resolve gateway IP dynamically. Trying fallback cache..." >&2
588         TS_IP=$(read_adguard_ip || true)
589     fi
590 else
591     TS_IP=$(read_adguard_ip || true)
592 fi
593
594 # 7. Connect host Mac to Tailscale mesh.
595 # With the standalone tailscaled daemon (registered as a per-user LaunchAgent or
596 # system LaunchDaemon via 'systemctl start tailscaled'), the previous five-
597 # phase flow collapses into two trivial CLI calls. There is no .app GUI to launch,
598 # no menu bar item to click, and no Accessibility permission required.
599 #
600 # CRITICAL: If tailscaled is not running, 'tailscale up' will silently fail.
601 # We auto-start it before proceeding, and SKIP the rest of the Tailscale setup
602 # if it can't be started (DNS hijack to a 100.x.x.x IP and exit-node routing
603 # are impossible without the host on the mesh).
604 #
605 # We connect in two phases:
606 # Phase A Join mesh WITHOUT exit node (so pre-flight checks can run)
607 # Phase B Set exit node only after pre-flight checks confirm the gateway works
608 HOST_ON_MESH=false
609 SKIP_EXIT_NODE=false
610 if [ -n "$TS_BIN" ]; then
611     echo -n "Verifying tailscaled is reachable"
612     daemon_ready=false
613     for i in {1..10}; do
614         # 'tailscale status' returns exit code 1 when the daemon is alive but
615         # disconnected ("Tailscale is stopped."). We check the daemon socket
616         # directly as a fallback if the socket exists, tailscaled is running
617         # even if the host isn't connected to the mesh yet.
618         if run_with_timeout 5 $TS_BIN status >> output.log 2>&1 || [ -S /var/run/tailscaled.socket ]; then
619             daemon_ready=true
620             break
621         fi

```

```

622     echo -n "."
623     sleep 1
624 done
625 echo ""
626
627 if [ "$daemon_ready" != "true" ]; then
628     echo -e "\033[0;33m[!] tailscaled daemon is not running. Attempting to start it...\033[0m"
629
630     # Try system-wide daemon (with timeout sudo may prompt for password)
631     if [ "$daemon_ready" != "true" ]; then
632         if run_with_timeout 10 sudo systemctl start tailscaled 2>> output.log; then
633             echo "Started tailscaled as system LaunchDaemon."
634             for i in {1..15}; do
635                 if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
636                     daemon_ready=true
637                     break
638                 fi
639                 sleep 1
640             done
641         fi
642     fi
643 fi
644
645 if [ "$daemon_ready" = "true" ]; then
646     echo "tailscaled is responsive (running as a system service)."
647
648     # Phase A: Join mesh without exit node
649     echo "Connecting host to Tailscale mesh (no exit node yet)..."
650     if $TS_BIN up --reset --ssh=true --accept-dns=false --exit-node=; then
651         HOST_ON_MESH=true
652         echo "Host is on Tailscale mesh."
653     else
654         echo -e "\033[0;33m[!] tailscale up failed even though the daemon is running.\033[0m"
655         echo "This usually means the host hasn't authenticated with your tailnet yet."
656         echo "Run this command in a terminal to authenticate:"
657         echo ""
658         echo "    sudo tailscale up"
659         echo ""
660         echo "A browser will open. Log in to your Tailscale account, then re-run toggle.sh."
661     fi
662 else
663     echo -e "\033[0;31m[!] tailscaled could NOT be started automatically.\033[0m"
664     echo "Run this in a terminal to start and authenticate Tailscale:"
665     echo ""
666     echo "    sudo systemctl start tailscaled"
667     echo "    sudo tailscale up"
668     echo ""
669     echo "Then re-run toggle.sh."
670 fi
671 else
672     echo -e "\n[Warning] tailscale CLI not found on PATH. Skipping host Tailscale configuration..."
673 fi
674
675 # Phase B: Pre-flight checks + enable exit node only if safe
676 if [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" != "false" ] && [ -n "$TS_IP" ]; then
677     echo -e "\n"
678     echo "Pre-flight connectivity check"
679     echo ""
680
681     # Check 1: Can we reach the gateway via Tailscale (uses tailscale ping, not ICMP)?
682     # NOTE: tailscale ping exits 1 when the connection goes through a DERP relay
683     # ("direct connection not established") even though the pong was received.
684     # We check for 'pong' in the output (stderr discarded) to accept relayed connections.
685     echo -n " [1/3] Gateway reachable via Tailscale... "
686     ping_ok=false
687     # The host tailscaled needs a few seconds to propagate the new network map and
688     # establish a connection route after 'tailscale up --reset'. We retry up to 45 times.
689     for attempt in {1..45}; do
690         if $TS_BIN ping --until-direct=false -c 2 --timeout 2s "$TS_IP" 2>/dev/null | grep -q "pong"; then
691             ping_ok=true
692             break
693         fi
694         sleep 1
695     done
696     if [ "$ping_ok" = "true" ]; then
697         echo "PASS"
698     else
699         echo "FAIL"
700         SKIP_EXIT_NODE=true
701     fi
702 fi

```

```

703 # Check 2: Can we reach AdGuard via localhost (exposed port ${DNS_PROXY_PORT})?
704 echo -n " [2/3] AdGuard DNS via localhost... "
705 if command -v dig >/dev/null 2>&1; then
706     if dig +tcp @127.0.0.1 -p ${DNS_PROXY_PORT} google.com +short +timeout=5 >/dev/null 2>&1; then
707         echo "PASS"
708     else
709         echo "FAIL"
710         SKIP_EXIT_NODE=true
711     fi
712 elif command -v nslookup >/dev/null 2>&1; then
713     if nslookup -port=${DNS_PROXY_PORT} google.com 127.0.0.1 >/dev/null 2>&1; then
714         echo "PASS"
715     else
716         echo "FAIL"
717         SKIP_EXIT_NODE=true
718     fi
719 else
720     echo "SKIP (dig/nslookup not available)"
721 fi
722
723 # Check 3: Does the WARP container have external internet?
724 echo -n " [3/3] WARP container internet... "
725 if docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace &>/dev/
726 null; then
727     echo "PASS"
728 else
729     echo "FAIL"
730     SKIP_EXIT_NODE=true
731 fi
732
733 # Act based on check results
734 if [ "$SKIP_EXIT_NODE" != "true" ]; then
735     echo -e "\nAll checks passed. Enabling exit node $TS_IP..."
736     if $TS_BIN up --reset --ssh=true --accept-dns=false --exit-node="$TS_IP" --exit-node-allow-lan-
737 access=true; then
738         echo "Exit node enabled."
739     else
740         echo "[Warning] Failed to set exit node."
741         SKIP_EXIT_NODE=true
742     fi
743 fi
744
745 if [ "$SKIP_EXIT_NODE" = "true" ]; then
746     echo -e "\n\033[0;33m[!] Pre-flight checks failed EXIT NODE + DNS HIJACK SKIPPED.\033[0m"
747     echo " Host stays on Tailscale mesh but your internet routes through your normal connection."
748     echo " Troubleshoot with:"
749     echo "     docker compose logs warp | tail -20"
750     echo "     docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/
751 trace"
752     echo ""
753     echo " Falling back to SOCKS5 proxy for traffic routing..."
754 fi
755
756 elif [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" = "false" ]; then
757     echo "USE_EXIT_NODE is false. Skipping exit node and DNS hijack."
758     SKIP_EXIT_NODE=true
759 fi
760
761 # 8. Apply DNS Hijacking.
762 # If exit node is enabled: hijack DNS to gateway's Tailscale IP (routes through tailnet).
763 # Otherwise, leave DNS at 1.1.1.1 (already set by reset at the top).
764 # AdGuard is also available at localhost:${DNS_PROXY_PORT} for manual configuration in apps.
765 if [ -n "$TS_IP" ] && [ "$HOST_ON_MESH" = "true" ] && [ "$SKIP_EXIT_NODE" != "true" ]; then
766     echo -e "\nHijacking host DNS to point ONLY at AdGuard Home ($TS_IP)..."
767     echo "Setting MagicDNS search domain 'ts.net' so tailnet hostnames resolve..."
768     if force_dns_to_gateway "$TS_IP"; then
769         echo "DNS hijack verified: single server = $TS_IP."
770         start_dns_watcher "$TS_IP"
771     else
772         echo "[Warning] DNS hijack could not be verified."
773         echo " If DNS is broken, run manually:"
774         echo "     networksetup -setdnsservers \"\$ACTIVE_SERVICE\" \"$TS_IP"
775         echo "     networksetup -setsearchdomains \"\$ACTIVE_SERVICE\" ts.net"
776     fi
777 fi
778
779 elif [ -n "$TS_IP" ]; then
780     echo -e "\n[Info] Exit node not enabled (pre-flight checks failed)."
781     echo " Trying local DNS proxy for ad-blocking..."
782     if start_local_dns_proxy; then
783         echo " Ad-blocking active via local DNS proxy through AdGuard."
784     else
785         echo " Local DNS proxy unavailable. DNS stays at 1.1.1.1."
786         echo " AdGuard available at http://localhost:80 (port ${DNS_PROXY_PORT} for DNS via TCP)."
787     fi
788 fi

```

```

781 fi
782
783 # Enable SOCKS5 proxy for traffic routing through WARP
784 # (enabled whenever exit node is skipped, regardless of HOST_ON_MESH)
785 echo -e "\n Enabling SOCKS5 proxy for TCP traffic routing through gateway WARP tunnel..."
786 echo " (SOCKS5 proxy runs inside gateway container, routes through tun0 -> WARP)"
787 enable_socks_proxy "$ACTIVE_SERVICE"
788
789 # Verify the SOCKS5 proxy works through WARP
790 echo -n " Verifying SOCKS5 proxy routes through WARP... "
791 if curl --socks5-hostname 127.0.0.1:$SOCKS_PROXY_PORT --max-time 10 -s https://www.cloudflare.com/cdn
-cgi/trace 2>/dev/null | grep -q "warp=on"; then
792     echo "ok (warp=on)."
793     echo " All TCP traffic now encrypted through Cloudflare WARP tunnel."
794 else
795     echo "FAILED (proxy not routing through WARP)."
796     echo " Disabling SOCKS5 proxy internet stays on direct connection."
797     disable_socks_proxy
798     echo " SOCKS proxy available at localhost:$SOCKS_PROXY_PORT for manual configuration."
799 fi
800 else
801     echo -e "\n[Warning] Could not resolve gateway Tailscale IP."
802 fi
803
804 # 9. Verify host connectivity
805 if [ "$HOST_ON_MESH" = "true" ] && [ -n "$TS_BIN" ]; then
806     if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
807         echo "Host is online on the Tailscale mesh."
808     else
809         echo "[Warning] Host is not responding on Tailscale."
810     fi
811 fi
812
813 ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
814 echo -e "\nGateway has been successfully STARTED in ${ELAPSED} seconds."
815
816 # Prevent system from going to sleep while gateway is running
817 start_sleep_prevention
818 fi
819
820 SUCCESS_RUN=true
821
822 # Final DNS state summary
823 if [ -n "$ACTIVE_SERVICE" ]; then
824     FINAL_DNS=$(resolvectl dns "$ACTIVE_SERVICE" 2>/dev/null | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' |
825     tr ' ' '\n' || true)
826     echo -e "\n"
827     echo -e "DNS STATE: $FINAL_DNS"
828     echo -e ""
829 fi
830
831 if [ "$START_GATEWAY" = "true" ] && command -v docker >/dev/null 2>&1; then
832     if docker compose logs rule-compiler 2>/dev/null | grep -q "Warning: Failed to fetch"; then
833         echo -e "\n[Warning] One or more blocklist URLs failed to download (404/Offline)."
834         echo " The gateway gracefully fell back to yesterday's cached blocklist."
835         echo " (Run 'docker logs rule-compiler' later to investigate which link died.)"
836     fi
837 fi
838
839 echo -e "\n"
840 read -rp "Press [r] and Enter to instantly reverse state, or just press Enter to exit: " USER_CHOICE
841 if [[ "${USER_CHOICE}" == "r" || "${USER_CHOICE}" == "R" ]]; then
842     echo "Reversing gateway state..."
843     exec bash "$0"
844 fi
845
846 echo -e "\nYou can close this terminal window now."
847
848 exit 0
849 fi
850
851 #
852 # macOS TOGGLE jot / netstat / networksetup, launchd / pf.
853 #
854
855 # Enforce Cryptographic Script Integrity
856 if [ -f "scripts/crypto.sh" ]; then
857     if ! bash scripts/crypto.sh --verify; then
858         exit 1
859     fi

```

```

860 fi
861
862 # Define log file
863 LOG_FILE="$PWD/output.log"
864
865 # Rotate log file if it exceeds 1GB (1073741824 bytes).
866 # Sized large on purpose: with DEBUG_TRACE the log holds a full command-by-command
867 # trace effectively the entire compilation/warning history of the framework
868 # which is valuable to keep for post-mortems. Raised from 50MB so always-on
869 # tracing doesn't churn that history away prematurely.
870 if [ -f "$LOG_FILE" ]; then
871     log_size=$(stat -f%z "$LOG_FILE" 2>/dev/null || stat -c%s "$LOG_FILE" 2>/dev/null || echo 0)
872     if [ "$log_size" -gt 1073741824 ]; then
873         mv "$LOG_FILE" "${LOG_FILE}.old" 2>/dev/null || rm -f "$LOG_FILE"
874         echo "[$(date '+%Y-%m-%d %H:%M:%S')] Log file exceeded 1GB; rotated to output.log.old" > "$LOG_FILE"
875     fi
876 fi
877
878 # --- LIFECYCLE PHASE TRACKING + EXIT BREADCRUMB ---
879 # TOGGLE_PHASE is updated as the script moves through STARTUP/STOP/START so that
880 # if the process is killed (window closed, kill, OOM) or exits with an error,
881 # the EXIT trap records the final exit code AND the phase it died in. Previously
882 # an interrupted START left output.log ending abruptly after the teardown with no
883 # indication of what failed this makes every exit traceable.
884 TOGGLE_PHASE="init"
885 _log_exit_breadcrumb() {
886     local rc=$?
887     echo "[$(date '+%Y-%m-%d %H:%M:%S')] toggle.sh EXIT: code=$rc phase=$TOGGLE_PHASE (pid $$)" >> "
888         $LOG_FILE" 2>/dev/null || true
889 }
890 trap '_log_exit_breadcrumb' EXIT
891
892 # --- DEBUG TRACE (opt-in via .env: DEBUG_TRACE=true) ---
893 # When enabled, mirror a full command-by-command xtrace to output.log for a
894 # complete post-mortem trail (e.g. a race during --restart while Wi-Fi is
895 # re-associating). Off by default. Read directly from .env here because
896 # common.sh (read_env_var) isn't sourced yet.
897 # BASH_XTRACEFD (send xtrace to its own fd, keeping the terminal clean) needs
898 # bash >= 4.1. macOS ships /bin/bash 3.2, so we branch:
899 # - bash >= 4.1 (e.g. Linux): dedicate fd 9 to the log and point xtrace there;
900 #   the terminal stays clean and normal stderr is untouched.
901 # - bash 3.2 (stock macOS): BASH_XTRACEFD is unsupported, so xtrace always goes
902 #   to stderr. We redirect stderr *straight to the log file* with a plain
903 #   'exec 2>>"$LOG_FILE"'. This deliberately does NOT use a process
904 #   substitution ('2>>(tee)'): under 'set -e', stderr flowing through the
905 #   backgrounded 'tee' proved to abort the script mid-START on 3.2 (a failed
906 #   write / SIGPIPE in a pipeline returned non-zero and tripped 'set -e', with
907 #   $LINENO mis-blaming the enclosing 'fi' see devref 15.11.8-A/E). The
908 #   tradeoff: on 3.2 the terminal no longer shows live progress while tracing
909 #   (everything is in output.log instead). Acceptable for an opt-in debug mode.
910 # We NEVER use the 'exec {var}>>' dynamic-fd form (4.1+ only), which hard-fails on 3.2.
911 DEBUG_TRACE=$(grep -E '^DEBUG_TRACE=' .env 2>/dev/null | cut -d '=' -f2- | tr -d "\"' " | tr '[:upper:]' '
912     [:lower:]' | tr -d '[:space:]')
913 if [ "$DEBUG_TRACE" = "true" ]; then
914     echo "[$(date '+%Y-%m-%d %H:%M:%S')] DEBUG_TRACE enabled xtrace mirrored to output.log (bash ${
915         BASH_VERSION[0]}.${BASH_VERSION[1]})" >> "$LOG_FILE"
916     # Timestamped, location-aware xtrace prompt (works on 3.2 and 4.x).
917     export PS4='+ [${SECONDS}s] ${BASH_SOURCE##*/}:${LINENO}:${FUNCNAME[0]:-main}: ' # subshell-free clock
918     : a $(()) in PS4 trips the bash 3.2 set -x/ERR heisenbug (devref 15.11.8)
919     if [ "${BASH_VERSION[0]}" -gt 4 ] || { [ "${BASH_VERSION[0]}" -eq 4 ] && [ "${BASH_VERSION[1]}" -ge
920         1 ]; }; then
921         exec 9>>"$LOG_FILE"
922         export BASH_XTRACEFD=9
923         set -x
924     else
925         # bash 3.2 (stock macOS): plain stderrfile redirect (no process substitution).
926         exec 2>>"$LOG_FILE"
927         set -x
928     fi
929 fi
930
931 # --- MAIN EXECUTION LOGGING ---
932 echo -e "\n===== " >> "$LOG_FILE"
933 echo "[$(date '+%Y-%m-%d %H:%M:%S')] toggle.sh invoked" >> "$LOG_FILE"
934 echo "===== " >> "$LOG_FILE"
935
936 # (Previously this script chmod 600'd .env at start to "unlock" it for docker compose,
937 # and chmod 000'd it on exit. That pattern broke docker compose on macOS/Colima when
938 # the umask produced 0600 the script removed its own ability to read .env before it could even proceed.)

```

```

936 # --- PROCEDURAL PORT GENERATION ---
937 # To prevent static port fingerprinting by malware, we randomize exposed ports
938 # on every boot and persist them to a hidden environment file.
939 PORTS_FILE="/tmp/nullexit-ports.env"
940 if [[ "$1" == "" || "$1" == "--restart" ]]; then
941
942     # Collision-avoidance function
943     GENERATED_PORTS=""
944     get_free_port() {
945         local port
946         while true; do
947             port=$(jot -r 1 10000 65000)
948             # 1. Prevent self-collision within this loop
949             if [[ "$GENERATED_PORTS" == *"$port"* ]]; then
950                 continue
951             fi
952             # 2. Prevent collision with ANY process on the Mac (root daemons, terminal apps, etc)
953             # We use netstat instead of lsof because netstat reads the kernel socket table
954             # directly and doesn't require sudo to see ports bound by other users/root.
955             if ! netstat -an -p tcp | awk '{print $4}' | grep -q "\.$port$"; then
956                 echo "$port"
957                 return
958             fi
959         done
960     }
961
962     export TOR SOCKS_PORT=$(get_free_port)
963     GENERATED_PORTS="$GENERATED_PORTS $TOR SOCKS_PORT"
964
965     export TOR_CONTROL_PORT=$(get_free_port)
966     GENERATED_PORTS="$GENERATED_PORTS $TOR_CONTROL_PORT"
967
968     export SOCKS_PROXY_PORT=$(get_free_port)
969     GENERATED_PORTS="$GENERATED_PORTS $SOCKS_PROXY_PORT"
970
971     export DNS_PROXY_PORT=$(get_free_port)
972
973     echo "export TOR SOCKS_PORT=$TOR SOCKS_PORT" > "$PORTS_FILE"
974     echo "export TOR_CONTROL_PORT=$TOR_CONTROL_PORT" >> "$PORTS_FILE"
975     echo "export SOCKS_PROXY_PORT=$SOCKS_PROXY_PORT" >> "$PORTS_FILE"
976     echo "export DNS_PROXY_PORT=$DNS_PROXY_PORT" >> "$PORTS_FILE"
977     ADGUARD_PASS=$(grep -E "^ADGUARD_PASSWORD=" .env 2>/dev/null | cut -d'=' -f2- | tr -d '\n')
978     echo "export TOR_PASSWORD=${ADGUARD_PASS:-nullexit}" >> "$PORTS_FILE"
979 elif [ -f "$PORTS_FILE" ]; then
980     # On STOP, source the existing ports so docker-compose down matches
981     source "$PORTS_FILE"
982 else
983     # Fallbacks if file is lost
984     export TOR SOCKS_PORT=9050
985     export TOR_CONTROL_PORT=9051
986     export SOCKS_PROXY_PORT=1080
987     export DNS_PROXY_PORT=5354
988 fi
989
990 # Start execution timer
991 TOGGLE_START_TIME=$SECONDS
992
993 if [[ "$1" == "--restart" ]]; then
994     echo "Executing Gateway Restart Sequence..."
995     # We must source common.sh here temporarily to check gateway state before we proceed
996     source "$SCRIPT_DIR/scripts/common.sh"
997     # Use persisted intent (marker) via gateway_state, which echoes a string (never
998     # a return code) to stay set -e-safe under set -x. See devref 15.11.8.
999     if [ "$(gateway_state)" = "running" ]; then
1000         echo "Gateway is currently running. Stopping it first..."
1001         bash "$0"
1002         echo "Gateway stopped. Now starting it..."
1003     else
1004         echo "Gateway is already stopped. Starting it..."
1005     fi
1006     bash "$0"
1007     exit 0
1008 fi
1009
1010 # Global flag to track if the script completed successfully
1011 SUCCESS_RUN=false
1012
1013 # Global variable to track the currently running background command PID
1014 CURRENT_BG_PID=""
1015
1016 # Source common bash functions

```



```

1017 source "$SCRIPT_DIR/scripts/common.sh"
1018
1019 # Prevent concurrent execution of lifecycle scripts (toggle.sh / recover.sh)
1020 LOCK_FILE="/tmp/nullexit-toggle.lock"
1021 if [ -f "$LOCK_FILE" ]; then
1022     LOCK_PID=$(cat "$LOCK_FILE" 2>/dev/null || echo "")
1023     if [ -n "$LOCK_PID" ] && [ "$LOCK_PID" != "$$" ] && kill -0 "$LOCK_PID" 2>/dev/null; then
1024         if ps -p "$LOCK_PID" -o args= 2>/dev/null | grep -q -E "toggle\.sh|recover\.sh"; then
1025             die "Another instance of toggle.sh or recover.sh (PID $LOCK_PID) is already running."
1026         fi
1027     fi
1028 fi
1029 echo "$$" > "$LOCK_FILE"
1030
1031 # Helper function to run a GUI command as the logged-in console user
1032 # (Prevents permission failures if the script is run with sudo)
1033 run_gui_cmd() {
1034     local console_user
1035     console_user=$(stat -f '%Su' /dev/console 2>> output.log || echo "$SUDO_USER")
1036     if [ -z "$console_user" ] || [ "$console_user" = "root" ]; then
1037         console_user=$(logname 2>> output.log || echo "$USER")
1038     fi
1039
1040     if [ -n "$console_user" ] && [ "$console_user" != "root" ] && [ "$EUID" -eq 0 ]; then
1041         sudo -u "$console_user" "$@"
1042     else
1043         "$@"
1044     fi
1045 }
1046
1047 # Active-state marker: written at end of START path so external watchers
1048 # (see launchd/com.nullexit.wake-recovery.plist + scripts/watcher.sh +
1049 # devref 10.29) know the gateway is currently up. They only fire
1050 # recover.sh --post-wake when this file is present. Cleared at the top
1051 # of the STOP path and inside cleanup_handler on any error/signal path
1052 # so a half-broken gateway never re-triggers post-wake refreshes.
1053 write_gateway_active_marker() {
1054     date -u +%FT%TZ > /tmp/nullexit-gateway-active.marker
1055     # Set the watcher debounce timestamp to now to prevent a redundant post-startup recovery run.
1056     date +%s > /tmp/nullexit-watcher.last-recovery 2>/dev/null || true
1057 }
1058
1059 clear_gateway_active_marker() {
1060     rm -f /tmp/nullexit-gateway-active.marker
1061     rm -f "$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/TUNNEL_FAILED_CLOSED.marker"
1062 }
1063
1064 # Capture whether the gateway-active marker exists BEFORE the defensive clear
1065 # below wipes it. The STOP-vs-START decision (gateway_should_be_running) reads
1066 # this captured value persisted *intent* instead of a live Docker probe, so
1067 # a disable is instant and works even when Colima/Docker is down. Must be read
1068 # before clear_gateway_active_marker or it would always look "stopped".
1069 #
1070 # Written as default-assign + 'if' WITHOUT an 'else': under 'set -e' (with the
1071 # DEBUG_TRACE xtrace redirect active) an 'if [ -f ]; then ; else ; fi' whose
1072 # condition is false was aborting the script here at init on macOS /bin/bash 3.2
1073 # the false '[ -f ]' status leaked through the compound. An 'if' with no 'else'
1074 # yields status 0 when the condition is false, so nothing leaks.
1075 GATEWAY_MARKER_AT_STARTUP="no"
1076 if [ -f "$MARKER_FILE" ]; then
1077     GATEWAY_MARKER_AT_STARTUP="yes"
1078 fi
1079
1080 # Defensive: clear any stale marker from a prior crashed/aborted run. If a
1081 # previous toggle.sh wrote the marker but was OOM-killed in the middle of the
1082 # START block (before the END-of-START write) or if the user rebooted mid
1083 # session this prevents the watcher from firing post-wake against a
1084 # non-running gateway on every wake event. Placed AFTER the function
1085 # definitions so bash resolves the call at parse time.
1086 clear_gateway_active_marker
1087
1088 PID_FILE="$PID_CAFFEINATE"
1089
1090 # Start macOS caffeinate to prevent sleep while gateway is running
1091 start_sleep_prevention() {
1092     if ! command -v caffeinate >/dev/null 2>&1; then
1093         echo " [Warning] caffeinate tool not found. Sleep prevention unavailable."
1094         return 1
1095     fi
1096
1097     # Stop any existing sleep prevention process first to avoid duplicates

```

```

1098 stop_pidfile_daemon "/tmp/nullexit-cafeinate.pid" "system sleep prevention"
1099
1100 echo " Enabling system sleep prevention (caffeinate)..."
1101 # Run caffeinate wrapped in a bash trap to prevent idle system sleep.
1102 # Critically, this traps macOS shutdown signals (SIGTERM) and automatically
1103 # flushes hijacked DNS back to normal right before the Mac powers off.
1104 # We do not use recover.sh here because it is a heavy recovery script (managing
1105 # containers, restarting tailscaled, etc.) which takes too long and gets
1106 # terminated by macOS before it can reset the DNS. Instead, we surgically and
1107 # instantly restore normal DNS and proxy settings here.
1108 nohup bash -c "
1109     trap '
1110         echo \"[Shutdown Trap] Reverting network settings...\" >> \"\$PWD/output.log\" 2>&1
1111         kill \$! 2>/dev/null || true
1112         sudo -n networksetup -setdnsservers \"\$ACTIVE_SERVICE\" 1.1.1.1 >> \"\$PWD/output.log\" 2>&1 || true
1113         sudo -n networksetup -setdnsservers \"\$ENO_SERVICE\" 1.1.1.1 >> \"\$PWD/output.log\" 2>&1 || true
1114         sudo -n networksetup -setsearchdomains \"\$ACTIVE_SERVICE\" \"Empty\" >> \"\$PWD/output.log\" 2>&1 ||
1115         true
1116         sudo -n networksetup -setsearchdomains \"\$ENO_SERVICE\" \"Empty\" >> \"\$PWD/output.log\" 2>&1 ||
1117         true
1118         sudo -n networksetup -setsocksfirewallproxystate \"\$ACTIVE_SERVICE\" off >> \"\$PWD/output.log\"
1119         2>&1 || true
1120         sudo -n networksetup -setsocksfirewallproxystate \"\$ENO_SERVICE\" off >> \"\$PWD/output.log\" 2>&1
1121         || true
1122         sudo -n networksetup -setwebproxystate \"\$ACTIVE_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
1123         sudo -n networksetup -setwebproxystate \"\$ENO_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
1124         sudo -n networksetup -setsecurewebproxystate \"\$ACTIVE_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 ||
1125         true
1126         sudo -n networksetup -setsecurewebproxystate \"\$ENO_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 ||
1127         true
1128         if command -v tailscale >/dev/null 2>&1; then
1129             tailscale up >> \"\$PWD/output.log\" 2>&1 || true
1130             tailscale set --accept-dns=false --exit-node= >> \"\$PWD/output.log\" 2>&1 || true
1131             TS_DOWN_ARGS=\"\"
1132             if [ \"\$KILL_SWITCH\" = \"true\" ]; then TS_DOWN_ARGS=\"--accept-risk=lose-ssh\"; fi
1133             tailscale down \"\$TS_DOWN_ARGS\" >> \"\$PWD/output.log\" 2>&1 &
1134         fi
1135         sleep 1
1136         echo \"[Shutdown Trap] Cleanup complete.\" >> \"\$PWD/output.log\" 2>&1
1137         exit 0
1138     ' SIGTERM SIGINT SIGHUP
1139     caffeinate -i &
1140     wait \$!
1141     \" >> output.log 2>&1 &
1142     local caffe_pid=$!
1143     echo \"\$caffe_pid\" > \"\$PID_FILE\"
1144     echo \" Sleep prevention active (PID \$caffe_pid). Your Mac won't sleep while the gateway is running.\"
1145 }
1146
1147 DNS_WATCHER_PID_FILE=\"$PID_DNS_WATCHER\"
1148 WARP_WATCHER_PID_FILE=\"/tmp/nullexit-warp-watcher.pid\"
1149
1150 # Background WARP tunnel liveness monitor. Polls cdn-cgi/trace every 30s
1151 # while healthy, but accelerates to polling every 5s if a failure is detected.
1152 # Always logs state transitions to output.log. When warp=off persists for
1153 # WARP_FAIL_THRESHOLD consecutive polls (default 6 = 30s of downtime), the watcher
1154 # triggers a nuclear recovery: runs recover.sh to tear down the entire
1155 # gateway disconnecting Tailscale, stopping containers, resetting DNS,
1156 # and power-cycling Wi-Fi. Note: This minimizes exposure window, but
1157 # only the pf kill switch guarantees absolutely zero leakage on drop.
1158 #
1159 # The threshold (default 6) is statistically chosen: even at an
1160 # unrealistically high 10% false-positive rate per poll, P(6 consecutive
1161 # false positives) = 0.1-6 0.0001%. Real-world false-positive rates
1162 # are far lower, making 30s of continuous downtime overwhelmingly likely
1163 # to be a genuine outage.
1164 #
1165 # Configurable via .env: WARP_FAIL_THRESHOLD=3 (15s, more aggressive)
1166 start_warp_watcher() {
1167     stop_warp_watcher
1168     # Parse threshold from .env (same pattern as GATEWAY_MSS, GATEWAY_BYPASS_PING, etc.)
1169     local threshold
1170     threshold=$(read_env_var WARP_FAIL_THRESHOLD)
1171     if [ -z \"$threshold\" ]; then
1172         threshold=\"${WARP_FAIL_THRESHOLD:-6}\"
1173     fi
1174     local trigger_file=\"/tmp/nullexit-warp-shutdown-triggered\"

```

```

1173 rm -f "$trigger_file"
1174 echo " Starting background WARP Watcher (polling every 30s [accelerating to 5s on failure], shutdown
after ${threshold} consecutive failures)..."
1175 nohup bash -c "
1176 trap 'exit 0' SIGTERM SIGINT SIGHUP
1177 last_state='on'
1178 consec_off=0
1179 threshold='${threshold}'
1180 trigger_file='${trigger_file}'
1181 out_log='${PWD}/output.log'
1182 recover_bin='${PWD}/recover.sh'
1183 inhibit_file='/tmp/nullexit-warp-inhibit.marker'
1184 while true; do
1185     # Inhibit check
1186     # recover.sh --post-wake writes this marker while force-recreating the
1187     # warp container. During that window, the container is intentionally
1188     # down don't count it as a failure or we'll fire nuclear recover.sh
1189     # and kill a gateway that was in the process of healing itself.
1190     if [ -f "\${inhibit_file}" ]; then
1191         consec_off=0
1192         sleep 5
1193         continue
1194     fi
1195
1196     state='off'
1197     if docker compose exec -T warp wget -q0- --timeout=5 \
1198         https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null \
1199         | grep -q 'warp=on'; then
1200         state='on'
1201     fi
1202
1203     # State-transition logging
1204     if [ "\${state}" != "\${last_state}" ]; then
1205         if [ "\${state}" = 'off' ]; then
1206             echo "\[${date -u +%FT%TZ}\] WARP DOWN cdn-cgi/trace reports warp=off\" >> \"\${out_log}\"
1207         fi
1208         last_state="\${state}"
1209     fi
1210
1211     # Consecutive-failure tracking + auto-shutdown
1212     if [ "\${state}" = 'off' ]; then
1213         consec_off=$((consec_off + 1))
1214         if [ "\${consec_off}" -ge "\${threshold}" ] && [ ! -f "\${trigger_file}" ]; then
1215             touch "\${trigger_file}"
1216             echo "\[${date -u +%FT%TZ}\] WARP SHUTDOWN \${consec_off} consecutive failures (threshold=\
${threshold}). Running recover.sh to kill the gateway and restore normal internet. Your IP is NO
LONGER Cloudflare.\" >> \"\${out_log}\"
1217             # Nuclear recovery: tear down the entire gateway.
1218             # recover.sh resets DNS 1.1.1.1, disconnects Tailscale,
1219             # stops Docker containers, flushes routes, power-cycles Wi-Fi.
1220             bash "\${recover_bin}" --auto >> \"\${out_log}\" 2>&1 || true
1221             echo "\[${date -u +%FT%TZ}\] WARP SHUTDOWN recover.sh completed, watcher exiting. Your IP is
now your real ISP IP.\" >> \"\${out_log}\"
1222             exit 0
1223         fi
1224     else
1225         if [ "\${consec_off}" -gt 0 ]; then
1226             echo "\[${date -u +%FT%TZ}\] WARP RECOVERED warp=on after \${consec_off} consecutive off
readings (threshold was \${threshold})\" >> \"\${out_log}\"
1227             fi
1228             consec_off=0
1229         fi
1230
1231         if [ "\${state}" = 'off' ]; then
1232             sleep 5
1233         else
1234             sleep 30
1235         fi
1236     done
1237 " >> output.log 2>&1 &
1238 echo $! > "$WARP_WATCHER_PID_FILE"
1239 }
1240
1241 stop_warp_watcher() {
1242     if [ -f "$WARP_WATCHER_PID_FILE" ]; then
1243         local wp
1244         wp=$(cat "$WARP_WATCHER_PID_FILE")
1245         if [ -n "$wp" ] && kill -0 "$wp" 2>/dev/null; then
1246             echo " Stopping background WARP Watcher (PID $wp)..."
1247             kill "$wp" 2>/dev/null || true
1248         fi

```

```

1249     rm -f "$WARP_WATCHER_PID_FILE"
1250 fi
1251 }
1252
1253
1254
1255
1256
1257 # Cleanup handler to restore DNS to 1.1.1.1 on error or user interrupt (Ctrl+C / SIGTERM / SIGHUP)
1258 cleanup_handler() {
1259     local exit_code=$?
1260     local trigger_type="$1"
1261     local line_no="$2"
1262
1263     if [ "$SUCCESS_RUN" = "false" ]; then
1264         echo -e "\n[Self-Correction] Script interrupted or failed ($trigger_type). Restoring host DNS to 1.1.1.1..."
1265         if [ -n "$ACTIVE_SERVICE" ]; then
1266             reset_dns
1267         fi
1268
1269         if [ -n "$CURRENT_BG_PID" ]; then
1270             echo "Terminating active command (PID $CURRENT_BG_PID)..."
1271             kill -15 "$CURRENT_BG_PID" 2>> output.log || true
1272         fi
1273
1274         # Disconnect host Tailscale and close the GUI application on failure
1275         disconnect_tailscale_host
1276
1277         # Stop local DNS proxy if running
1278         stop_local_dns_proxy
1279
1280         # Stop sleep prevention
1281         stop_pidfile_daemon "/tmp/nullexit-cafeinate.pid" "system sleep prevention"
1282         stop_pidfile_daemon "/tmp/nullexit-dns-watcher.pid" "background DNS Watcher"
1283         stop_warp_watcher
1284         clear_gateway_active_marker
1285
1286         # Capture warp logs on failure for debugging before teardown
1287         if [ "$trigger_type" = "ERR" ]; then
1288             echo -e "\n--- WARP FAILURE LOGS ---" >> output.log
1289             docker compose logs warp --tail=100 >> output.log 2>&1 || true
1290             echo "-----" >> output.log
1291         fi
1292
1293         # Best-effort container teardown on failure
1294         docker compose down --remove-orphans -t 30 2>> output.log || true
1295
1296         # Nuke leftover network state (proxies, routes, DNS cache, Wi-Fi)
1297         if [ -n "$ACTIVE_SERVICE" ]; then
1298             cleanup_network_state
1299         fi
1300
1301
1302         if [ "$trigger_type" = "ERR" ]; then
1303             echo -e "\n=====
1304             echo "ERROR: Script failed at line $line_no with exit code $exit_code."
1305             echo "=====
1306             echo "Troubleshooting steps:"
1307             echo "1. Run 'colima status' to verify the VM is active."
1308             echo "2. Run 'docker compose ps' to check container health."
1309             echo "3. Run 'docker compose logs' to view application logs."
1310             echo "4. Run 'tailscale status' to check host Tailscale status."
1311             echo "=====
1312         fi
1313         rm -f "${LOCK_FILE:-/tmp/nullexit-toggle.lock}"
1314     fi
1315 }
1316
1317 trap 'cleanup_handler ERR $LINENO' ERR
1318 trap 'cleanup_handler INT' INT
1319 trap 'cleanup_handler TERM' TERM
1320 trap 'cleanup_handler HUP' HUP
1321
1322 # NOPASSWD sudo
1323 # The user has NOPASSWD in /etc/sudoers.d/nullexit for the specific commands
1324 # needed (networksetup, pfctl, route, ifconfig, dscacheutil, killall, pkill,
1325 # and python3 -I scripts/dns-proxy.py for the fallback DNS proxy).
1326 # All privileged calls below use 'sudo -n' which will run without prompting.
1327 # No credential caching loop is needed.
1328

```

```

1329 # Add Homebrew and standard paths
1330 setup_standard_path
1331
1332 # Check for local DNS proxy tools (used when tailnet data plane is unavailable)
1333 # Uses a Python DNS proxy (handles TCP wire format correctly).
1334 # Python3 is built into macOS no external dependencies.
1335 DNS_PROXY_BIN=""
1336 if command -v python3 >> output.log 2>&1; then
1337     DNS_PROXY_BIN="python3 -I"
1338 elif command -v python >> output.log 2>&1; then
1339     DNS_PROXY_BIN="python -I"
1340 fi
1341
1342 # Path to the DNS proxy script (sibling of this script)
1343 DNS_PROXY_SCRIPT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/scripts/dns-proxy.py"
1344
1345 # Resolve Tailscale CLI on host. We rely on the standalone Homebrew formula
1346 # ('brew install tailscale' + 'brew services start tailscale') NOT the
1347 # Tailscale.app GUI so there is no .app bundle or Network Extension sandbox
1348 # to launch, no menu-bar click to perform, and no scutil Network Service to
1349 # start manually. tailscaled runs as a per-user LaunchAgent (or LaunchDaemon
1350 # if you ran the install with sudo) and the control socket is always available.
1351 TS_BIN=""
1352 if command -v tailscale >> output.log 2>&1; then
1353     TS_BIN="tailscale"
1354 fi
1355
1356 # Baseline: disable Tailscale DNS management at the daemon level.
1357 # 'tailscale set' writes a persistent preference. However, 'tailscale up --reset'
1358 # (used in steps 7 and 9) RESETS all unspecified flags to defaults, which would
1359 # re-enable '--accept-dns=true' and nullify this 'set'.
1360 # Therefore, EVERY 'tailscale up --reset' call in this file MUST also pass
1361 # '--accept-dns=false' explicitly. See steps 7 and 9 for the fix.
1362 #
1363 # This initial 'set' is still useful as a baseline for non-reset operations
1364 # (e.g. if the user runs 'tailscale up' manually without --reset) and covers
1365 # the brief window between this line and the first '--reset' call.
1366 #
1367 # Runs once at script start while tailscaled is still connected (if it was
1368 # left running from a previous toggle), so the pref takes effect before any
1369 # disconnection or reconnection logic.
1370 if [ -n "$TS_BIN" ]; then
1371     $TS_BIN set --accept-dns=false >> output.log 2>&1 || true
1372 fi
1373
1374 # Disconnect host Tailscale from the mesh. With the standalone daemon, this is the
1375 # *entire* teardown no scutil tunnel stop, no Tailscale.app GUI quit, no pkill.
1376 # tailscaled keeps running (as a LaunchAgent / LaunchDaemon), which is intentional:
1377 # the next 'tailscale up' then reconnects in seconds rather than waiting for a
1378 # slow .app launch + Network Extension activation. The 'tailscale down' call also
1379 # retains the user's auth state, so we don't force a browser re-login every cycle.
1380 #
1381 # Defined early (before the if/else branches and referenced by cleanup_handler's
1382 # ERR trap) so that any failure before the main logic can still tear down Tailscale.
1383
1384
1385 # Wait for Network Readiness
1386 # On --restart, the gateway tears down while the host network interface may
1387 # still be reconnecting (Wi-Fi, Ethernet, USB tethering, etc.). If we proceed
1388 # too early, get_active_service returns nothing, routing targets the wrong
1389 # interface, and DNS/WARP setup silently fails.
1390 #
1391 # We use 'route get default' to detect a valid default gateway rather than
1392 # checking a specific interface (e.g. en0) this generalizes across all
1393 # connection types: Wi-Fi, Ethernet, USB-C adapters, iPhone tethering, etc.
1394 # The moment the OS has any routable default gateway, we proceed.
1395 _net_wait_secs=0
1396 _net_timeout=60
1397 echo "Waiting for network connectivity before starting..."
1398 while true; do
1399     if route get default 2>/dev/null | grep -q 'gateway:~'; then
1400         _gw=$(route get default 2>/dev/null | awk '/gateway:/{print $2}')
1401         echo "Network ready (default gateway: $_gw)."
1402         break
1403     fi
1404     if [ "$_net_wait_secs" -ge "$_net_timeout" ]; then
1405         echo " [Warning] No default gateway after ${_net_timeout}s proceeding anyway."
1406         break
1407     fi
1408     sleep 2
1409     _net_wait_secs=$(($_net_wait_secs + 2))

```

```

1410 done
1411
1412 ACTIVE_SERVICE=$(get_active_service)
1413
1414 # Resolve the service name for en0 (usually "Wi-Fi") macOS scutil DNS resolver
1415 # is commonly scoped to en0, so per-service DNS changes on other interfaces are
1416 # ignored unless en0's service is also updated.
1417 ENO_SERVICE=$(get_en0_service)
1418
1419 # Helper to add host-side bypass routes for the WARP WireGuard endpoints.
1420 # This prevents an infinite recursive routing loop (tunnel loop) where
1421 # the container's WireGuard packets exit the VM, reach the host, match the
1422 # default route (the exit node), and get routed back into the tunnel.
1423 # We route via the gateway IP (not interface) to ensure packets are routed
1424 # properly even when the host's default route changes to the Tailscale interface.
1425
1426 # Helper to manually override the host's default route to point through the
1427 # Tailscale utun* interface. Standalone tailscaled on macOS (Homebrew version)
1428 # lacks the platform-specific integration to override the default gateway
1429 # automatically when --exit-node is used.
1430 setup_exit_node_routing() {
1431     local ts_iface
1432     ts_iface=$(ifconfig | awk '/^[a-z0-9]+:/{iface=$1} /inet 100\./{print iface; exit}' | tr -d ':')
1433
1434     if [ -n "$ts_iface" ]; then
1435         echo "Re-routing default gateway to $ts_iface using 0.0.0.0/1 split..."
1436         local host_mtu
1437         host_mtu=$(read_env_var HOST_MTU)
1438         host_mtu=${host_mtu:-1200}
1439
1440         # Lower the host Tailscale MTU (defaults to 1200) to prevent fragmentation when passing
1441         # through the container's WARP tunnel (which also has an MTU of 1280).
1442         sudo -n ifconfig "$ts_iface" mtu "$host_mtu" >> output.log 2>&1 || true
1443
1444         # DO NOT delete the physical default route! It crashes Colima.
1445         # Use the /1 trick to mathematically override the default route.
1446         sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
1447         sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
1448         sudo -n route add -net 0.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
1449         sudo -n route add -net 128.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
1450
1451         if netstat -nr | grep -E -q "^(0\.\0\.\0/1|0/1)[[:space:]]+.*$ts_iface"; then
1452             echo "    [] Default route successfully overridden via $ts_iface."
1453         else
1454             echo "    [!] Failed to verify exit node routing on $ts_iface."
1455         fi
1456     else
1457         echo "[Warning] Could not detect Tailscale utun interface for host routing."
1458     fi
1459 }
1460
1461 # Helper to nuke leftover network state after teardown (proxy settings, routing
1462 # table, DNS cache, Wi-Fi). Prevents the "had to write a whole recovery script"
1463 # problem where stale state kills internet after the gateway stops.
1464 cleanup_network_state() {
1465     echo -e "\nCleaning up network state (proxies, routes, DNS cache, Wi-Fi)..."
1466
1467     # 1. Disable any leftover proxy settings (needs root on many macOS versions)
1468     for svc in "$ACTIVE_SERVICE" "$ENO_SERVICE"; do
1469         disable_all_proxies "$svc"
1470     done
1471     echo "    Proxies disabled."
1472
1473     # 2. Flush DNS cache
1474     if command -v dscacheutil >> output.log 2>&1; then
1475         sudo -n dscacheutil -flushcache 2>> output.log || true
1476     fi
1477     sudo -n killall -HUP mDNSResponder 2>> output.log || true
1478     echo "    DNS cache flushed."
1479
1480     # 3. Flush stale routing table entries
1481     if command -v route >> output.log 2>&1; then
1482         sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
1483         sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
1484     fi
1485     remove_warp_bypass_routes
1486     remove_apple_split_routes
1487     disable_killswitch
1488     echo "    Routing table and firewall flushed."
1489
1490     # 4. Power-cycle Wi-Fi to clear any lingering interface state

```



```

1491 WIFI_PORT=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: Wi-Fi/{getline;
1492   print $2}')
1493 if [ -n "$WIFI_PORT" ]; then
1494     sudo -n networksetup -setairportpower "$WIFI_PORT" off 2>> output.log || true
1495     sleep 2
1496     sudo -n networksetup -setairportpower "$WIFI_PORT" on 2>> output.log || true
1497     echo " Wi-Fi power-cycled (interface $WIFI_PORT)."
1498     sleep 3
1499 else
1500     echo " Wi-Fi interface not detected; bouncing en0..."
1501     sudo -n ifconfig en0 down 2>> output.log || true
1502     sudo -n ifconfig en0 up 2>> output.log || true
1503 fi
1504 # Force restart sharing services to prevent AirDrop / AirPlay discovery freezes
1505 # after network interface/DNS changes.
1506 echo " Resetting macOS sharing services (AirDrop/AirPlay)..."
1507 reset_sharing_services
1508
1509 echo "Network state cleanup complete."
1510 }
1511
1512 SOCKS_PROXY_PORT=${SOCKS_PROXY_PORT}
1513
1514 # Ensure DNS is set to 1.1.1.1 immediately at script start to prevent any DNS deadlocks/hangs
1515 # during status checks, container startup, VM boot, or teardown. We *also* clear any leftover
1516 # 'ts.net' search domain from a previous 'tailscale up --accept-dns=true' run, otherwise macOS
1517 # would prepend it to every lookup and most public DNS queries would resolve to NXDOMAIN.
1518
1519
1520 echo "Initializing DNS to 1.1.1.1 to ensure reliable internet access..."
1521 reset_dns
1522
1523
1524
1525
1526 echo "Checking Gateway Status..."
1527 HIJACK_HOST=$(read_env_var GATEWAY_HIJACK_HOST | tr '[:upper:]' '[:lower:]')
1528
1529 # Decide STOP vs START from persisted intent (marker), not a live Docker probe.
1530 # This makes *disable* instant and robust even when Colima/Docker is down.
1531 #
1532 # CRITICAL (set -e + set -x on bash 3.2): gateway_state ECHOES "running"/"stopped"
1533 # and always returns 0, so we branch on the STRING never on a function's exit
1534 # code. A helper that 'return 1's trips a spurious 'set -e' abort under 'set -x'
1535 # (DEBUG_TRACE=true) even when guarded, hence the echo contract. See devref 15.11.8.
1536 if [ "$(gateway_state)" = "running" ]; then
1537     TOGGLE_PHASE="stop"
1538     echo -e "\n=====
1539     echo "Gateway is RUNNING. Stopping it now..."
1540     echo -e "=====
1541
1542 # 1. Disconnect host Tailscale cleanly first before stopping the exit-node container
1543 if [[ "$HIJACK_HOST" != "false" ]]; then
1544     disconnect_tailscale_host
1545     echo ""
1546 fi
1547
1548 echo "Stopping Docker containers..."
1549 # '|| true': a disable must always complete its teardown even if the Docker
1550 # daemon / Colima VM is already down (compose down would exit non-zero and,
1551 # under set -e, abort the STOP). The START path intentionally does NOT guard
1552 # its 'docker compose up' a failed start SHOULD trigger cleanup.
1553 run_logged docker compose down --remove-orphans -t 30 || true
1554
1555 # Only stop Colima if the user explicitly opted in (false by default) to prevent breaking their other
1556 # Docker dev projects
1557 STOP_COLIMA=$(read_env_var STOP_COLIMA_ON_EXIT | tr '[:upper:]' '[:lower:]')
1558 if [[ "$STOP_COLIMA" == "true" ]]; then
1559     echo -e "\nStopping Colima VM to free up host RAM and battery..."
1560     run_with_timeout 30 colima stop >> output.log 2>&1 || echo "Warning: Failed to stop Colima gracefully"
1561     .
1562 else
1563     echo -e "\nLeaving Colima running. (Set STOP_COLIMA_ON_EXIT=true in .env to change this)"
1564 fi
1565
1566 if [[ "$HIJACK_HOST" != "false" ]]; then
1567     # The host's DNS was hijacked to the gateway IP during ENABLE; now that the
1568     # gateway is down, restore DNS to 1.1.1.1 immediately so subsequent lookups
1569     # don't stall for the macOS DNS timeout before the 1.1.1.1 fallback engages.
1570     echo -e "\nRestoring host DNS to 1.1.1.1 (gateway is gone)..."

```

```

1569     reset_dns
1570
1571     # 3. Stop local DNS proxy if running
1572     stop_local_dns_proxy
1573
1574     # Stop background daemons
1575     stop_pidfile_daemon "/tmp/nullexit-cafeinate.pid" "system sleep prevention"
1576     stop_pidfile_daemon "/tmp/nullexit-dns-watcher.pid" "background DNS Watcher"
1577     stop_warp_watcher
1578     clear_gateway_active_marker
1579
1580     # 4. Nuke leftover network state so internet actually works after teardown
1581     cleanup_network_state
1582 fi
1583
1584 ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
1585 echo -e "\nGateway has been successfully STOPPED in ${ELAPSED} seconds."
1586 else
1587     TOGGLE_PHASE="start"
1588     START_GATEWAY=true
1589     echo "[$(date '+%Y-%m-%d %H:%M:%S')] Action: STARTUP GATEWAY" >> "$LOG_FILE"
1590     echo -e "\n=====
1591     echo "Gateway is STOPPED. Starting it now..."
1592     echo -e "=====
1593
1594     # 1. Prevent host exit-node deadlock during VM / Container startup
1595     if [[ "$HIJACK_HOST" != "false" ]]; then
1596         disconnect_tailscale_host
1597     fi
1598
1599     # 2. Wait for physical DHCP lease to settle (crucial for restarts/wake-up/roaming)
1600     wait_for_dhcp_settle
1601
1602     # 3. Boot Colima VM if it is not already running
1603     echo -e "\nChecking Colima VM status..."
1604     if ! run_with_timeout 15 colima status >> output.log 2>&1; then
1605         # Colima networking is gated on the SAME LAN-P2P signal routing-fix uses
1606         # (.lan_p2p_detected overrides TAILSCALE_ALLOW_LAN_P2P in .env). A LAN-reachable
1607         # VM (--network-address + bridged/shared) needs the socket_vmnet helper and is
1608         # only worthwhile on a trusted home network, so we request it only when P2P is
1609         # allowed; otherwise AP-isolated / enterprise, the common case we boot plain
1610         # user-mode networking. Either way host<->container P2P is UNAFFECTED: it rides
1611         # routing-fix's Docker-gateway / .host_ips RETURN rules, not the VM network mode
1612         # (see devref 15.3.5). colima_start_until_ready returns as soon as Docker answers
1613         # (~15s) instead of blocking on colima's ~2min post-provision hang (see 15.8.7);
1614         # that hang is colima-internal and mode-independent (it also occurs in user mode).
1615         allow_p2p=$(read_env_var "TAILSCALE_ALLOW_LAN_P2P")
1616         [ -z "$allow_p2p" ] && allow_p2p="false"
1617         if [ -f "$SCRIPT_DIR/.lan_p2p_detected" ]; then
1618             allow_p2p=$(tr -d '[:space:]' < "$SCRIPT_DIR/.lan_p2p_detected" 2>/dev/null || echo "false")
1619         fi
1620
1621         if [ "$allow_p2p" = "true" ]; then
1622             colima_network_mode="bridged"
1623             if [[ "$OSTYPE" == "darwin"* ]]; then
1624                 security_type=$(system_profiler SPAirPortDataType 2>/dev/null | awk '/Current Network Information
1625                 :/{found=1} found && /Security:/{print $NF; exit}')
1626                 if echo "$security_type" | grep -qi "Enterprise"; then
1627                     echo -e "\n    [!] WPA2-Enterprise Wi-Fi detected! Bridged mode will cause MAC-spoofing
1628                     deauthentication."
1629                     echo "        Falling back to '--network-mode shared' for Colima."
1630                     colima_network_mode="shared"
1631                 fi
1632             fi
1633             echo "Colima is not running. Starting Colima (600MB RAM, vz VM, LAN P2P on: --network-address
1634             $colima_network_mode)..."
1635             colima_start_until_ready --memory 0.6 --vm-type vz --network-address --network-mode "
1636             $colima_network_mode" \
1637             || echo "    [!] Colima Docker not confirmed ready; continuing (downstream docker-ps auto-heal will
1638             retry)."
1639         else
1640             colima_network_mode="user"
1641             echo "Colima is not running. Starting Colima (600MB RAM, vz VM, LAN P2P off: fast user-mode
1642             networking)..."
1643             colima_start_until_ready --memory 0.6 --vm-type vz \
1644             || echo "    [!] Colima Docker not confirmed ready; continuing (downstream docker-ps auto-heal will
1645             retry)."
1646         fi
1647         echo "$colima_network_mode" > /tmp/nullexit_colima_mode.txt
1648     else
1649         echo "Colima is already running."

```

```

1643 fi
1644
1645 # 3b. Auto-detect and fix Docker Subnet Collisions (e.g. if the user travels to a 172.17.x.x Wi-Fi)
1646 if [ -f "scripts/fix-docker-bridge-collision.sh" ]; then
1647     if bash scripts/fix-docker-bridge-collision.sh --dry | grep -q "collision detected"; then
1648         echo -e "\n[!] WARNING: Docker Subnet Collision Detected with your current Wi-Fi!"
1649         echo -e "    Auto-fixing Colima config to prevent internet jitter..."
1650         bash scripts/fix-docker-bridge-collision.sh --skip-toggle || true
1651     fi
1652 fi
1653
1654 # 3c. Configure swap file inside the VM to prevent OOM on low-memory limits
1655 # We also set vm.swappiness=10 so the Linux kernel strictly prefers physical RAM
1656 # and avoids unnecessarily wearing out the SSD with proactive background swapping.
1657 if ! run_with_timeout 15 colima ssh -- grep -q 'swapfile' /proc/swaps >> output.log 2>&1; then
1658     echo "Configuring 400MB swap file inside the VM to prevent OOM..."
1659     run_with_timeout 30 colima ssh -- sudo sh -c "if [ ! -f /swapfile ]; then dd if=/dev/zero of=/
        swapfile bs=1M count=400 status=none && mkswap /swapfile; fi && swapon /swapfile && sysctl vm.
        swappiness=10" >> output.log 2>&1 || echo "Warning: Failed to enable swap file inside the VM."
1660 fi
1661
1662 # 4. Clean up corrupted AdGuardHome configurations
1663 if [ -f "adguard/conf/AdGuardHome.yaml" ] && [ ! -s "adguard/conf/AdGuardHome.yaml" ]; then
1664     rm -f "adguard/conf/AdGuardHome.yaml"
1665     echo "Removed corrupted empty AdGuardHome.yaml to prevent container crash loop."
1666 fi
1667
1668 # 4b. Auto-heal stale Docker socket from hard reboots
1669 if ! run_with_timeout 15 docker ps >/dev/null 2>&1; then
1670     echo "Docker daemon is unresponsive (likely a stale socket from a crash). Auto-healing Colima..."
1671     run_with_timeout 120 colima restart >> output.log 2>&1
1672 fi
1673
1674 # 4c. Inject TCP MSS clamp from .env into post-rules.txt before starting
1675 if [ -f .env ] && grep -q "^GATEWAY_MSS=" .env; then
1676     GATEWAY_MSS=$(read_env_var GATEWAY_MSS)
1677     if [[ "$GATEWAY_MSS" =~ ^[0-9]+$ ]]; then
1678         # macOS sed syntax
1679         sed -i '' "s/--set-mss [0-9]*/--set-mss ${GATEWAY_MSS}/g" post-rules.txt 2>/dev/null || \
1680         # Linux sed fallback
1681         sed -i "s/--set-mss [0-9]*/--set-mss ${GATEWAY_MSS}/g" post-rules.txt 2>/dev/null
1682     fi
1683 fi
1684
1685 # 5. Start compose services
1686 write_host_ips
1687
1688 # Procedurally generated ports have already been exported at the top of the script.
1689 echo "    [Security] Procedurally randomized proxy ports generated to evade fingerprinting."
1690
1691 # --- HONEY-PORT TRIPWIRE ---
1692 # Generate a random trap port. If any malware does a sequential port scan on localhost,
1693 # it will inevitably hit this port. When it does, we instantly fire a macOS alert.
1694 # Reap the honey-port from any previous toggle before arming a new one, so the
1695 # 'nc -l' tripwire never accumulates one idle listener per toggle-on (15.12.20).
1696 pkill -f "nc -l localhost" 2>/dev/null || true
1697 HONEY_PORT=$(get_free_port)
1698 (
1699     if nc -l localhost "$HONEY_PORT" >/dev/null 2>&1; then
1700         echo "[$(date '+%Y-%m-%d %H:%M:%S')] [CRITICAL SECURITY ALERT] PORT SCAN DETECTED! An unknown
            application just pinged the local Honey-Port ($HONEY_PORT)!" >> "$LOG_FILE"
1701         osascript -e 'display notification "An unknown application is aggressively scanning your local
            ports. Malware port-scan suspected." with title "nullexit OPSEC Alert" sound name "Basso" ' 2>/dev/
            null || true
1702     fi
1703 ) &
1704 disown %1 2>/dev/null || true
1705 echo "    [Security] Honey-Port Tripwire armed on random port to detect malware port-scans."
1706
1707 echo -e "\nStarting Docker containers..."
1708
1709 # --- Ad-block rule compilation: hash-gated one-off, off the critical path (15.8.8) ---
1710 # The rule-compiler (a 'compile'-profiled container, the SOLE writer of adguard/work)
1711 # re-dedupes ~340k rules inside the 600MB VM (~28s) far too slow to run every toggle.
1712 # It is gated on a hash of the lists YOU control (black_list.txt + white_list.txt):
1713 # recompile only when they change, when compiled_rules.txt / ip_blocklist.ipset are
1714 # missing, or when they exceed RULES_MAX_AGE_DAYS (default 7, so the remote blocklists
1715 # still refresh occasionally). Otherwise AdGuard/routing-fix boot on the existing
1716 # artifacts and we skip the ~28s they no longer 'depends_on' the rule-compiler
1717 # container (see docker-compose.yml). A compile failure is non-fatal: we keep the
1718 # previous compiled rules rather than bricking the tunnel (ad-block != the kill-switch).

```

```

1719 RULES_HASH_FILE="$SCRIPT_DIR/.rules_hash"
1720 rules_hash=$(compute_rules_hash)
1721 stored_rules_hash=""
1722 [ -f "$RULES_HASH_FILE" ] && stored_rules_hash=$(cat "$RULES_HASH_FILE" 2>/dev/null)
1723 rules_max_age_days=$(read_env_var RULES_MAX_AGE_DAYS)
1724 [ -z "$rules_max_age_days" ] && rules_max_age_days=7
1725 need_compile=false
1726 if [ ! -f adguard/work/userfilters/compiled_rules.txt ] || [ ! -f adguard/work/userfilters/ip_blocklist
    .ipset ]; then
1727     need_compile=true
1728 elif [ -z "$rules_hash" ] || [ "$rules_hash" != "$stored_rules_hash" ]; then
1729     need_compile=true
1730 elif [ -n "$(find adguard/work/userfilters/compiled_rules.txt -mtime +"$rules_max_age_days" 2>/dev/null
    )" ]; then
1731     need_compile=true
1732 fi
1733 if [ "$need_compile" = "true" ]; then
1734     echo " Ad-block lists changed (or artifacts missing/stale) compiling rules (one-off, ~28s)..."
1735     if run_logged docker compose run --build --rm rule-compiler >> output.log 2>&1; then
1736         if [ -n "$rules_hash" ]; then echo "$rules_hash" > "$RULES_HASH_FILE"; fi
1737     else
1738         echo " [!] Rule compile failed continuing with existing compiled ad-block rules."
1739     fi
1740 else
1741     echo " Ad-block lists unchanged reusing compiled rules (skipping the ~28s recompile)."
1742 fi
1743
1744 # --- Gate '--build' for the long-running images (routing-fix, tor) ---
1745 # BuildKit re-checking these on the 600MB VM costs ~30s even when every layer is
1746 # CACHED (15.8.8). Only pass --build when the hashed inputs change or an image is
1747 # missing; otherwise plain 'up -d' reuses cached images. 'up' starts everything
1748 # except the 'compile'-profiled rule-compiler.
1749 BUILD_HASH_FILE="$SCRIPT_DIR/.build_hash"
1750 build_inputs_hash=$(compute_build_inputs_hash)
1751 stored_build_hash=""
1752 [ -f "$BUILD_HASH_FILE" ] && stored_build_hash=$(cat "$BUILD_HASH_FILE" 2>/dev/null)
1753 local_images_present=true
1754 for _img in nullexit-routing-fix:v1.0.0 nullexit-tor:v1.0.0; do
1755     docker image inspect "$_img" >> output.log 2>&1 || local_images_present=false
1756 done
1757 if [ -n "$build_inputs_hash" ] && [ "$build_inputs_hash" = "$stored_build_hash" ] && [ "
    $local_images_present" = "true" ]; then
1758     echo " Build inputs unchanged skipping image build (reusing cached images)."
1759     run_logged docker compose up -d
1760 else
1761     echo " Build inputs changed or a local image is missing building images..."
1762     run_logged docker compose up -d --build
1763     # Record the hash only after a successful build (set -e aborts above on failure).
1764     if [ -n "$build_inputs_hash" ]; then echo "$build_inputs_hash" > "$BUILD_HASH_FILE"; fi
1765 fi
1766
1767 RULE_COUNT=$(grep "! Total Block Rules:" adguard/work/userfilters/compiled_rules.txt 2>/dev/null | awk
    -F': ' '{print $2}' || echo "0")
1768 NATIVE_COUNT=$(grep "! Native AdGuard Rules:" adguard/work/userfilters/compiled_rules.txt 2>/dev/null |
    awk -F': ' '{print $2}' || echo "0")
1769
1770 if [ "$RULE_COUNT" != "0" ] && [ "$RULE_COUNT" != "" ]; then
1771     if [ "$NATIVE_COUNT" != "0" ] && [ "$NATIVE_COUNT" != "" ]; then
1772         TOTAL_COUNT=$((RULE_COUNT + NATIVE_COUNT))
1773         # Format with commas for readability (e.g. 441,578)
1774         if command -v printf &> /dev/null; then
1775             FORMATTED_RULE=$(printf "%'d" "$RULE_COUNT")
1776             FORMATTED_TOTAL=$(printf "%'d" "$TOTAL_COUNT")
1777             echo " DNS: Compiled $FORMATTED_RULE unique custom rules (Total active in AdGuard: ~
    $FORMATTED_TOTAL rules)."
1778         else
1779             echo " DNS: Compiled $RULE_COUNT unique custom rules (Total active in AdGuard: ~$TOTAL_COUNT
    rules)."
1780         fi
1781     else
1782         echo " DNS: Compiled and loaded $RULE_COUNT optimized rules."
1783     fi
1784 fi
1785
1786 # Report IP blocklist compilation results
1787 IP_COUNT=$(grep "^# Entries:" adguard/work/userfilters/ip_blocklist.ipset 2>/dev/null | awk -F': ' '{
    print $2}' || echo "0")
1788 if [ "$IP_COUNT" != "0" ] && [ "$IP_COUNT" != "" ]; then
1789     if command -v printf &> /dev/null; then
1790         FORMATTED_IP=$(printf "%'d" "$IP_COUNT")
1791         echo " IP: Loaded $FORMATTED_IP threat intelligence IPs/CIDRs into kernel firewall."
    
```

```

1792     else
1793         echo " IP: Loaded $IP_COUNT threat intelligence IPs/CIDRs into kernel firewall."
1794     fi
1795 fi
1796
1797 # 6. Wait for the gateway container's Tailscale connection to be ready
1798 BYPASS_PING=$(read_env_var GATEWAY_BYPASS_PING | tr '[:upper:]' '[:lower:]')
1799 USE_EXIT_NODE=$(read_env_var GATEWAY_USE_EXIT_NODE | tr '[:upper:]' '[:lower:]')
1800
1801 echo "Waiting for gateway container's Tailscale to connect to the tailnet..."
1802 echo " (This can take 30-60s Tailscale goes through: NoState Starting Running Online)"
1803 connected=false
1804 consecutive_failures=0
1805 for i in {1..60}; do
1806     # Run tailscale status and capture its output so the user can see what's happening.
1807     # Previously this was hidden behind 2>> output.log, making it impossible to debug hangs.
1808     ts_output=$(run_with_timeout 3 docker compose exec -T tailscale tailscale status 2>&1 || true)
1809
1810     if echo "$ts_output" | grep -q "offers exit node"; then
1811         connected=true
1812         break
1813     fi
1814
1815     # Track consecutive failures to detect a stuck state.
1816     # If we see 40+ consecutive 'NoState' responses, give up early
1817     # the daemon is alive but can't connect to the coordination server.
1818     if echo "$ts_output" | grep -q "NoState"; then
1819         consecutive_failures=$((consecutive_failures + 1))
1820         if [ "$consecutive_failures" -ge 40 ]; then
1821             echo ""
1822             echo " [attempt $i/60] Stuck in 'NoState' for $consecutive_failures checks."
1823             echo " Tailscale daemon is running but can't reach the coordination server."
1824             break
1825         fi
1826     else
1827         consecutive_failures=0
1828     fi
1829
1830     # Print a condensed status every 10 seconds so the user can see progress
1831     if [ $((i % 10)) -eq 0 ]; then
1832         ts_short=$(echo "$ts_output" | head -1 | tr -d '\r\n')
1833         echo " [attempt $i/60] $ts_short"
1834     elif [ $((i % 5)) -eq 0 ]; then
1835         echo -n "[$i]"
1836     else
1837         echo -n "."
1838     fi
1839     sleep 1
1840 done
1841 echo ""
1842
1843 if [ "$connected" = "true" ]; then
1844     echo "Gateway container is online on the Tailscale mesh."
1845 elif [ "$BYPASS_PING" = "true" ]; then
1846     echo "[Warning] Gateway container check timed out. Proceeding anyway (GATEWAY_BYPASS_PING is true)..."
1847 else
1848     echo "ERROR: Gateway container failed to initialize Tailscale (timed out after 60s)." >&2
1849     echo " The container was seen in the following states during the wait:" >&2
1850     echo "$ts_output" | sed 's/^/ /' >&2
1851     echo "" >&2
1852     echo " This could mean:" >&2
1853     echo " 1. Tailscale auth key has expired check TS_AUTHKEY in .env" >&2
1854     echo " 2. Network issue inside the container check: docker compose logs tailscale" >&2
1855     echo " 3. gluetun VPN tunnel not connecting check: docker compose logs warp | tail -20" >&2
1856     echo "" >&2
1857     echo " Diagnose manually:" >&2
1858     echo " docker compose exec -T tailscale tailscale status" >&2
1859     echo " docker compose exec -T tailscale tailscale netcheck" >&2
1860     cleanup_handler ERR $LINENO
1861     exit 1
1862 fi
1863
1864 # 6b. Resolve the gateway's Tailscale IP.
1865 # Dynamically query the container for the IP (handles IP changes after re-auth)
1866 # and cache it in .gateway_ip for fast retrieval by recover.sh during Wi-Fi roaming.
1867 #
1868 # CRITICAL: 'docker compose exec -T' on macOS/Colima injects carriage
1869 # returns (\r) into captured output. If $TS_IP ends with \r, then
1870 # 'networksetup -setdnsservers' silently rejects the malformed IP and
1871 # DNS stays at 1.1.1.1 the primary bug this project was hitting.

```

```

1872 # 'tr -d '\r'' strips the CR; 'awk 'NR==1{print $1; exit}'' takes only
1873 # the first field of the first line.
1874 TS_IP=""
1875 if [ "$connected" = "true" ]; then
1876     TS_IP=$(run_with_timeout 5 docker compose exec -T tailscale tailscale ip -4 2>> output.log | tr -d '\r' | awk 'NR==1{print $1; exit}' || true)
1877     if [ -n "$TS_IP" ]; then
1878         echo "Resolved gateway Tailscale IP dynamically: $TS_IP"
1879         echo "$TS_IP" > .gateway_ip
1880     else
1881         echo " [!] Failed to resolve gateway IP dynamically. Trying fallback cache..." >&2
1882         TS_IP=$(read_adguard_ip || true)
1883     fi
1884 else
1885     TS_IP=$(read_adguard_ip || true)
1886 fi
1887
1888 # 7. Connect host Mac to Tailscale mesh.
1889 # With the standalone tailscaled daemon (registered as a per-user LaunchAgent or
1890 # system LaunchDaemon via 'brew services start tailscale'), the previous five-
1891 # phase flow collapses into two trivial CLI calls. There is no .app GUI to launch,
1892 # no menu bar item to click, and no Accessibility permission required.
1893 #
1894 # CRITICAL: If tailscaled is not running, 'tailscale up' will silently fail.
1895 # We auto-start it before proceeding, and SKIP the rest of the Tailscale setup
1896 # if it can't be started (DNS hijack to a 100.x.x.x IP and exit-node routing
1897 # are impossible without the host on the mesh).
1898 #
1899 # We connect in two phases:
1900 # Phase A Join mesh WITHOUT exit node (so pre-flight checks can run)
1901 # Phase B Set exit node only after pre-flight checks confirm the gateway works
1902 HOST_ON_MESH=false
1903 SKIP_EXIT_NODE=false
1904 if [ "$HIJACK_HOST" = "false" ]; then
1905     echo -e "\n[Info] HIJACK_HOST is false (Headless Mode). Host networking will remain untouched."
1906     SKIP_EXIT_NODE=true
1907 elif [ -n "$TS_BIN" ]; then
1908     echo -n "Verifying tailscaled is reachable"
1909     daemon_ready=false
1910     status_exit=0
1911     for i in {1..10}; do
1912         # Test if the daemon responds to status check.
1913         # If status returns non-zero, check if it was a normal "stopped/disconnected" state
1914         # (exit code 1 is normal for disconnected). If the command exited quickly (not killed by timeout),
1915         # and the error is just "Tailscale is stopped", then the daemon is alive and ready.
1916         # But if it timed out (exit code 143) or is completely unresponsive, it's wedged.
1917         status_exit=0
1918         if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
1919             daemon_ready=true
1920             break
1921         else
1922             status_exit=$?
1923             if [ "$status_exit" -ne 143 ] && [ -S /var/run/tailscaled.socket ]; then
1924                 daemon_ready=true
1925                 break
1926             fi
1927         fi
1928         echo -n "."
1929         sleep 1
1930     done
1931     echo ""
1932
1933     # If the daemon was unresponsive or socket check failed, attempt auto-restart
1934     if [ "$daemon_ready" != "true" ]; then
1935         if [ "$status_exit" -eq 143 ]; then
1936             warn "tailscaled daemon is wedged/unresponsive. Restarting it..."
1937         else
1938             warn "tailscaled daemon is not running. Attempting to start it..."
1939         fi
1940         restart_tailscaled_daemon
1941
1942         # Re-verify
1943         if run_with_timeout 5 $TS_BIN status >> output.log 2>&1 || [ -S /var/run/tailscaled.socket ]; then
1944             daemon_ready=true
1945         fi
1946     fi
1947
1948     if [ "$daemon_ready" = "true" ]; then
1949         echo "tailscaled is responsive (running as a system service)."
1950
1951         # Phase A: Join mesh without exit node

```



```

1952     echo "Connecting host to Tailscale mesh (no exit node yet)..."
1953     if $TS_BIN up; then
1954         $TS_BIN set --ssh=true --accept-dns=false --accept-routes=true --exit-node= || true
1955         HOST_ON_MESH=true
1956         echo "Host is on Tailscale mesh."
1957     else
1958         warn "tailscale up failed even though the daemon is running."
1959         echo "  This usually means the host hasn't authenticated with your tailnet yet."
1960         echo "  Run this command in a terminal to authenticate:"
1961         echo ""
1962         echo "      sudo tailscale up"
1963         echo ""
1964         echo "  A browser will open. Log in to your Tailscale account, then re-run toggle.sh."
1965     fi
1966 else
1967     fail "tailscaled could NOT be started automatically."
1968     echo "  Run this in a terminal to start and authenticate Tailscale:"
1969     echo ""
1970     echo "      sudo brew services start tailscale"
1971     echo "      sudo tailscale up"
1972     echo ""
1973     echo "  Then re-run toggle.sh."
1974 fi
1975 else
1976     echo -e "\n[Warning] tailscale CLI not found on PATH. Skipping host Tailscale configuration..."
1977 fi
1978
1979 # Phase B: Pre-flight checks + enable exit node only if safe
1980 if [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" != "false" ] && [ -n "$TS_IP" ]; then
1981     echo -e "\n"
1982     echo "Pre-flight connectivity check"
1983     echo ""
1984
1985     # Check 1: Can we reach the gateway via Tailscale (uses tailscale ping, not ICMP)?
1986     # NOTE: tailscale ping exits 1 when the connection goes through a DERP relay
1987     # ("direct connection not established") even though the pong was received.
1988     # We check for 'pong' in the output (stderr discarded) to accept relayed connections.
1989     echo -n " [1/3] Gateway reachable via Tailscale... "
1990     ping_ok=false
1991     # The host tailscaled needs a few seconds to propagate the new network map and
1992     # establish a connection route after 'tailscale up --reset'. We retry up to 45 times.
1993     for attempt in {1..45}; do
1994         if $TS_BIN ping --until-direct=false -c 2 --timeout 2s "$TS_IP" 2>/dev/null | grep -q "pong"; then
1995             ping_ok=true
1996             break
1997         fi
1998         sleep 1
1999     done
2000     if [ "$ping_ok" = "true" ]; then
2001         echo "PASS"
2002     else
2003         echo "FAIL"
2004         SKIP_EXIT_NODE=true
2005     fi
2006
2007     # Check 2: Can we reach AdGuard via localhost (exposed port ${DNS_PROXY_PORT})?
2008     # Colima's SSH tunnel only forwards TCP (not UDP), so we use +tcp.
2009     echo -n " [2/3] AdGuard DNS via localhost... "
2010     if command -v dig &>/dev/null; then
2011         if dig +tcp @127.0.0.1 -p ${DNS_PROXY_PORT} google.com +short +timeout=5 &>/dev/null; then
2012             echo "PASS"
2013         else
2014             echo "FAIL"
2015             SKIP_EXIT_NODE=true
2016         fi
2017     elif command -v nslookup &>/dev/null; then
2018         if nslookup -port=${DNS_PROXY_PORT} google.com 127.0.0.1 &>/dev/null; then
2019             echo "PASS"
2020         else
2021             echo "FAIL"
2022             SKIP_EXIT_NODE=true
2023         fi
2024     else
2025         echo "SKIP (dig/nslookup not available)"
2026     fi
2027
2028     # Check 3: Does the WARP container have external internet?
2029     echo -n " [3/3] WARP container internet... "
2030     tmp_err=$(mktemp)
2031     if docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace >/dev/
        null 2>"$tmp_err"; then

```

```

2032     echo "PASS"
2033 else
2034     echo "FAIL"
2035     if [ -s "$tmp_err" ]; then
2036         err_msg=$(cat "$tmp_err" | tr -d '\r')
2037         echo "[$(date +%Y-%m-%d %H:%M:%S)] [Check 3] WARP container check failed: $err_msg" >> output.
2038     log
2039     fi
2040     SKIP_EXIT_NODE=true
2041 fi
2042 rm -f "$tmp_err"
2043
2044 # Act based on check results
2045 if [ "$SKIP_EXIT_NODE" != "true" ]; then
2046     echo -e "\nAll checks passed. Enabling exit node $TS_IP..."
2047     $TS_BIN up || true
2048     if $TS_BIN set --ssh=true --accept-dns=false --accept-routes=true --exit-node="$TS_IP" --exit-node-allow-lan-access=true; then
2049         echo "Exit node enabled."
2050         add_warp_bypass_routes
2051         setup_exit_node_routing
2052         enable_killswitch
2053         # Opt-in FaceTime split-route (no-op unless FACETIME_SPLIT_ROUTE=true).
2054         # After the kill-switch so the com.apple/nullexit anchor + table exist.
2055         add_apple_split_routes
2056     else
2057         echo "[Warning] Failed to set exit node."
2058         SKIP_EXIT_NODE=true
2059     fi
2060 fi
2061
2062 if [ "$SKIP_EXIT_NODE" = "true" ]; then
2063     warn "Pre-flight checks failed EXIT NODE + DNS HIJACK SKIPPED."
2064     echo "Host stays on Tailscale mesh but your internet routes through your normal connection."
2065     echo "Troubleshoot with:"
2066     echo "    docker compose logs warp | tail -20"
2067     echo "    docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace"
2068     echo ""
2069     echo "Falling back to SOCKS5 proxy for traffic routing..."
2070 fi
2071 elif [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" = "false" ]; then
2072     echo "USE_EXIT_NODE is false. Skipping exit node and DNS hijack."
2073     SKIP_EXIT_NODE=true
2074 fi
2075
2076 # 8. Apply DNS Hijacking.
2077 # If exit node is enabled: hijack DNS to gateway's Tailscale IP (routes through tailnet).
2078 # Otherwise, leave DNS at 1.1.1.1 (already set by reset at the top).
2079 # AdGuard is also available at localhost:${DNS_PROXY_PORT} for manual configuration in apps.
2080 if [ -n "$TS_IP" ] && [ "$HOST_ON_MESH" = "true" ] && [ "$SKIP_EXIT_NODE" != "true" ]; then
2081     echo -e "\nHijacking host DNS to point ONLY at AdGuard Home ($TS_IP)..."
2082     echo "Setting MagicDNS search domain 'ts.net' so tailnet hostnames resolve..."
2083     if force_dns_to_gateway "$TS_IP"; then
2084         echo "DNS hijack verified: single server = $TS_IP."
2085     else
2086         echo "[Warning] DNS hijack could not be verified."
2087         echo "If DNS is broken, run manually:"
2088         echo "    networksetup -setdnsservers \"$ACTIVE_SERVICE\" \"$TS_IP"
2089         echo "    networksetup -setsearchdomains \"$ACTIVE_SERVICE\" ts.net"
2090     fi
2091 elif [ -n "$TS_IP" ] && [ "$HIJACK_HOST" != "false" ]; then
2092     echo -e "\n[Info] Exit node not enabled (pre-flight checks failed)."
2093     echo "Trying local DNS proxy for ad-blocking..."
2094     if start_local_dns_proxy; then
2095         echo "Ad-blocking active via local DNS proxy through AdGuard."
2096     else
2097         echo "Local DNS proxy unavailable. DNS stays at 1.1.1.1."
2098         echo "AdGuard available at http://localhost:80 (port ${DNS_PROXY_PORT} for DNS via TCP)."
2099     fi
2100
2101 # Enable SOCKS5 proxy for traffic routing through WARP
2102 # (enabled whenever exit node is skipped, regardless of HOST_ON_MESH)
2103 echo -e "\nEnabling SOCKS5 proxy for TCP traffic routing through gateway WARP tunnel..."
2104 echo " (SOCKS5 proxy runs inside gateway container, routes through tun0 -> WARP)"
2105 enable_socks_proxy "$ACTIVE_SERVICE"
2106
2107 # Verify the SOCKS5 proxy works through WARP
2108 echo -n "Verifying SOCKS5 proxy routes through WARP..."
2109 if curl --socks5-hostname 127.0.0.1:$SOCKS_PROXY_PORT --max-time 10 -s https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null | grep -q "warp=on"; then

```

```

2109     echo "ok (warp=on)."
```

```

2110     echo "  All TCP traffic now encrypted through Cloudflare WARP tunnel."
```

```

2111     else
```

```

2112     echo "FAILED (proxy not routing through WARP)."
```

```

2113     echo "  Disabling SOCKS5 proxy internet stays on direct connection."
```

```

2114     disable_socks_proxy
```

```

2115     echo "  SOCKS proxy available at localhost:${SOCKS_PROXY_PORT} for manual configuration."
```

```

2116     fi
```

```

2117     else
```

```

2118     echo -e "\n[Warning] Could not resolve gateway Tailscale IP."
```

```

2119     fi
```

```

2120
```

```

2121 # 9. Verify host connectivity
```

```

2122 if [ "$HOST_ON_MESH" = "true" ] && [ -n "$TS_BIN" ]; then
```

```

2123     if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
```

```

2124         echo "Host is online on the Tailscale mesh."
```

```

2125     else
```

```

2126         echo "[Warning] Host is not responding on Tailscale."
```

```

2127     fi
```

```

2128 fi
```

```

2129
```

```

2130 ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
```

```

2131 echo -e "\nGateway has been successfully STARTED in ${ELAPSED} seconds."
```

```

2132
```

```

2133 # Prevent system from going to sleep while gateway is running
```

```

2134 start_sleep_prevention
```

```

2135
```

```

2136 if [[ "$HIJACK_HOST" != "false" ]]; then
```

```

2137     if [ -n "$TS_IP" ] && [ "$TS_IP" != "1.1.1.1" ]; then
```

```

2138         start_dns_watcher "$TS_IP"
```

```

2139     fi
```

```

2140
```

```

2141     # Start WARP liveness monitor (logs to output.log on warp flip)
```

```

2142     start_warp_watcher
```

```

2143 fi
```

```

2144
```

```

2145 # Reset sharing services after network configuration to prevent AirDrop freezes
```

```

2146 echo "Resetting macOS sharing services (AirDrop/AirPlay)..."
```

```

2147 reset_sharing_services
```

```

2148
```

```

2149 # Tell external watchers (post-wake / network-change) the gateway is up.
```

```

2150 write_gateway_active_marker
```

```

2151
```

```

2152 # Local Network Surveillance Check
```

```

2153 echo -e "\n"
```

```

2154 echo "Local Network Surveillance Check"
```

```

2155 echo ""
```

```

2156 # Count populated ARP cache entries to detect network scanning or high congestion
```

```

2157 # Using '-an' avoids hanging on DNS reverse resolution when the network state is in transition.
```

```

2158 ARP_COUNT=$(arp -an 2>/dev/null | grep -iv 'incomplete' | wc -l | tr -d ' ')
```

```

2159 if [ "$ARP_COUNT" -gt 15 ]; then
```

```

2160     warn "High ARP activity detected ($ARP_COUNT devices in cache)."
```

```

2161     warn "This Wi-Fi network may be heavily congested or actively scanned (e.g., arp-scan)."
```

```

2162     echo -e "  [] Your traffic remains fully encrypted and invisible.\n"
```

```

2163 else
```

```

2164     echo -e "  [] Local network looks quiet ($ARP_COUNT devices). No aggressive scanning detected.\n"
```

```

2165 fi
```

```

2166 fi
```

```

2167
```

```

2168 SUCCESS_RUN=true
```

```

2169
```

```

2170 # Final DNS state summary
```

```

2171 if [ -n "$ACTIVE_SERVICE" ]; then
```

```

2172     FINAL_DNS=$(networksetup -getdnsservers "$ACTIVE_SERVICE" 2>> output.log || true)
```

```

2173     echo -e "\n"
```

```

2174     echo -e "DNS STATE: $FINAL_DNS"
```

```

2175     echo -e ""
```

```

2176 fi
```

```

2177
```

```

2178 if [ "$START_GATEWAY" = "true" ] && command -v docker >/dev/null 2>&1; then
```

```

2179     if docker compose logs rule-compiler 2>/dev/null | grep -q "Warning: Failed to fetch"; then
```

```

2180         echo -e "\n[Warning] One or more blacklist URLs failed to download (404/Offline)."
```

```

2181         echo "  The gateway gracefully fell back to yesterday's cached blacklist."
```

```

2182         echo "  (Run 'docker logs rule-compiler' later to investigate which link died.)"
```

```

2183     fi
```

```

2184 fi
```

```

2185
```

```

2186 rm -f "${LOCK_FILE:-/tmp/nullexit-toggle.lock}"
```

```

2187 if [ -t 0 ]; then
```

```

2188     read -rp "Press [r] and Enter to instantly reverse state, or just press Enter to exit: " USER_CHOICE
```

```

2189     if [[ "${USER_CHOICE}" == "r" || "${USER_CHOICE}" == "R" ]]; then
```

```

2190     echo "Reversing gateway state..."
2191     exec bash "$0"
2192 fi
2193 fi
2194
2195 echo -e "\nYou can close this terminal window now."

```

7 recover.sh

```

1  #!/bin/bash
2  # recover.sh  nullexit KILL-SWITCH recovery script (macOS + Linux)
3  # Fixes internet when toggle.sh leaves you stranded.
4  #
5  # This is a NUCLEAR option: it undoes EVERYTHING toggle.sh does by:
6  #   1. Disconnecting host Tailscale from the mesh (exit-node off)
7  #   2. Resetting DNS to default on ALL network services
8  #   3. Disabling rogue proxy settings (SOCKS, web, secure web)
9  #   4. Flushing the DNS cache
10 #   5. Flushing stale routing table entries
11 #   6. Stopping gateway Docker containers
12 #   7. Power-cycling Wi-Fi
13 #   8. Verifying internet is working
14 #
15 # One file, both OSes: the dispatch below runs the self-contained Linux
16 # recovery and exits; on macOS it falls through to the richer dual-mode
17 # implementation (default teardown / --post-wake refresh).
18 #
19 # Usage:  double-click Recover-Gateway.applescript
20 #         OR  ./recover.sh
21 #         OR  ./recover.sh --post-wake    (macOS only  lighter refresh, keeps gateway live)
22
23 set -e
24
25 # Resolve script directory (used for path-relative refs)
26 # recover.sh lives at the repo root; SCRIPT_DIR lets us launch toggle.sh
27 # (and reference any other repo-root file) from the same directory no
28 # matter where the user invoked recover.sh from.
29 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
30 cd "$SCRIPT_DIR" || exit 1
31
32 #
33 # LINUX RECOVERY  self-contained; runs the full teardown then exits before the
34 # macOS body below. Native Linux tooling (resolvectl / nmcli / ip) throughout.
35 #
36 if [[ "$OSTYPE" != "darwin"* ]]; then
37     # Source common formatting and helper functions
38     source "$SCRIPT_DIR/scripts/common.sh"
39
40     # Always add Homebrew path
41     export PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:$PATH"
42
43     echo ""
44     echo -e "${RED}${BOLD}${NC}"
45     echo -e "${RED}${BOLD}      Nullexit Gateway  RECOVERY MODE      ${NC}"
46     echo -e "${RED}${BOLD}${NC}"
47     echo ""
48     echo "This script will tear down the gateway and restore normal internet."
49     echo ""
50
51     # Cache sudo credentials upfront
52     # Prevents the script from hanging halfway through when it needs to run
53     # privileged commands (route flush, Wi-Fi power cycle, etc.).
54     echo -e "${YELLOW}${BOLD}Authentication required for network recovery...${NC}"
55     sudo -v
56     (
57         while true; do
58             sudo -n true 2>/dev/null
59             sleep 60
60         done
61     ) &
62     SUDO_KEEPER_PID=$!
63     trap 'kill $SUDO_KEEPER_PID 2>/dev/null' EXIT
64
65     # 1. Disconnect host Tailscale
66     step "Disconnecting host Tailscale from mesh"
67     if command -v tailscale >> output.log 2>&1; then

```

```

68 # Check if actually connected first (with timeout tailscaled could be wedged)
69 if run_with_timeout 5 tailscale status >> output.log 2>&1; then
70     # Also explicitly reset any exit-node so it doesn't linger.
71     if run_with_timeout 10 tailscale up --reset --ssh=true --accept-dns=false --exit-node= >> output.
log 2>&1; then
72         ok "Exit-node preference cleared"
73     else
74         warn "tailscale up --reset didn't respond (tailscaled may be wedged)"
75     fi
76     ts_args=""
77     if is_kill_switch_enabled; then ts_args="--accept-risk=lose-ssh"; fi
78     if run_with_timeout 10 tailscale down $ts_args >> output.log 2>&1; then
79         ok "Tailscale disconnected"
80     else
81         warn "tailscale down didn't respond"
82     fi
83 else
84     warn "tailscaled not reachable skipping (timeout after 5s)"
85 fi
86 else
87     warn "tailscale CLI not found skipping"
88 fi
89
90 # 2. Reset DNS to default on ALL network services
91 step "Resetting DNS to default on active interface"
92
93 ACTIVE_SERVICE=$(ip route get 1.1.1.1 2>/dev/null | awk '{print $5; exit}')
94 if [ -n "$ACTIVE_SERVICE" ]; then
95     sudo resolvectl revert "$ACTIVE_SERVICE" 2>> output.log || true
96     ok "DNS reset on $ACTIVE_SERVICE"
97 else
98     warn "Could not determine active network interface"
99 fi
100
101 # 3. Disable rogue proxy settings
102 step "Disabling proxy settings (SOCKS, web, secure web)"
103 echo " (Linux global SOCKS proxy configuration skipped.)"
104 ok "Proxy settings cleared"
105
106
107 # 4. Flush DNS cache
108 step "Flushing DNS cache"
109 if command -v resolvectl >> output.log 2>&1; then
110     sudo resolvectl flush-caches 2>> output.log || true
111     ok "DNS cache flushed"
112 else
113     warn "resolvectl not found"
114 fi
115
116 # 5. Clear stale routing table
117 step "Flushing stale routing table entries"
118 sudo ip route flush cache >> output.log 2>&1 || true
119 ok "Routing table flushed"
120
121 # 6. Stop gateway Docker containers
122 step "Stopping gateway Docker containers"
123 if command -v docker >> output.log 2>&1 && docker info >> output.log 2>&1; then
124     if [ -f "docker-compose.yml" ]; then
125         docker compose down -t 5 2>> output.log && ok "Containers stopped" || warn "No containers were
running"
126     else
127         warn "docker-compose.yml not found in current directory"
128     fi
129 else
130     warn "Docker not available skipping"
131 fi
132
133 # 6b. Stopping sleep prevention (systemd-inhibit)
134 step "Stopping sleep prevention"
135 PID_FILE="/tmp/nullexit-cafeinate.pid"
136 if [ -f "$PID_FILE" ]; then
137     INHIBIT_PID=$(cat "$PID_FILE")
138     if [ -n "$INHIBIT_PID" ] && kill -0 "$INHIBIT_PID" 2>/dev/null; then
139         kill "$INHIBIT_PID" 2>/dev/null || true
140         ok "Sleep prevention stopped (PID $INHIBIT_PID)"
141     else
142         warn "Sleep prevention process not running, cleaning up stale PID file"
143     fi
144     rm -f "$PID_FILE"
145 else
146     ok "Sleep prevention was not active"

```

```

147 fi
148
149 # 7. Power-cycle Wi-Fi
150 step "Power-cycling Wi-Fi"
151 if command -v nmcli &>/dev/null; then
152     sudo nmcli radio wifi off 2>> output.log || true
153     sleep 2
154     sudo nmcli radio wifi on 2>> output.log || true
155     ok "Wi-Fi bounced via nmcli"
156     sleep 3
157 else
158     warn "nmcli not found. Falling back to ip link..."
159     WIFI_IFACE=$(ip link | grep -E '[0-9]+: (wl[a-zA-Z0-9]+|wlan[0-9]+):' | awk -F': ' '{print $2}')
160     if [ -n "$WIFI_IFACE" ]; then
161         sudo ip link set "$WIFI_IFACE" down 2>> output.log || true
162         sleep 2
163         sudo ip link set "$WIFI_IFACE" up 2>> output.log || true
164         ok "Wi-Fi bounced via ip link (interface $WIFI_IFACE)"
165         sleep 3
166     else
167         warn "No Wi-Fi interface detected."
168     fi
169 fi
170
171 # 8. Verify internet connectivity
172 verify_internet_connectivity
173
174 # Summary
175 echo ""
176 echo -e "${BOLD}${NC}"
177 echo -e "${BOLD}RECOVERY SUMMARY${NC}"
178 echo -e "${BOLD}${NC}"
179
180 # Show final DNS state
181 FINAL_DNS=$(resolvectl dns "$ACTIVE_SERVICE" 2>> output.log | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+'
182 | tr '\n' ' ' || echo "N/A")
183 echo " DNS ($ACTIVE_SERVICE): $FINAL_DNS"
184
185 echo ""
186
187 if [ "$INTERNET_OK" = "true" ]; then
188     echo -e " ${GREEN}${BOLD} Internet is working.${NC}"
189 else
190     echo -e " ${YELLOW}${BOLD} Could not verify internet connectivity.${NC}"
191     echo " This might mean:"
192     echo " - Your Wi-Fi is disconnected from the router"
193     echo " - Tailscale is still interfering (try restarting tailscaled:"
194     echo "   systemctl restart tailscaled)"
195     echo " - You need to re-connect to Wi-Fi in the menu bar"
196     echo " - Try toggling Wi-Fi off/on in the menu bar"
197     echo ""
198     echo " Or try manually:"
199     echo "   resolvectl revert $ACTIVE_SERVICE"
200     extra_args=""
201     if is_kill_switch_enabled; then extra_args="--accept-risk=lose-ssh"; fi
202     echo "   tailscale down $extra_args"
203     echo "   sudo ip route flush cache"
204     echo "   systemctl restart tailscaled"
205 fi
206
207 echo ""
208 echo -e "${GREEN}${BOLD}Recovery complete.${NC}"
209 echo ""
210
211 exit 0
212 fi
213
214 # macOS RECOVERY (dual-mode: default teardown / --post-wake refresh)
215 #
216
217 # Enforce Cryptographic Script Integrity
218 if [ -f "scripts/crypto.sh" ]; then
219     if ! bash scripts/crypto.sh --verify; then
220         exit 1
221     fi
222 fi
223
224 # Argument parsing
225 # --post-wake: lightweight refresh after sleep/wake or Wi-Fi roam.
226 # Keeps the gateway live while HUPing DNS caches, refreshing the

```



```

227 #           Tailscale exit-node leak, re-hijacking host DNS, and force-
228 #           recreating the warp container if its gluettun healthcheck failed.
229 #           Does NOT tear down Docker, Tailscale, sleep prevention, or Wi-Fi.
230 #           Called from scripts/watcher.sh on every wake / network-change event
231 #           while /tmp/nullexit-gateway-active.marker is present (i.e. the
232 #           gateway is currently up). See devref.md 10.29 for the why.
233 POST_WAKE=false
234 AUTO_MODE=false
235 for arg in "$@"; do
236     if [ "$arg" = "--post-wake" ]; then
237         POST_WAKE=true
238     elif [ "$arg" = "--auto" ] || [ "$arg" = "--non-interactive" ]; then
239         AUTO_MODE=true
240     fi
241 done
242
243 rm -f "$SCRIPT_DIR/TUNNEL_FAILED_CLOSED.marker"
244
245 # Prevent concurrent execution of lifecycle scripts (toggle.sh / recover.sh)
246 LOCK_FILE="/tmp/nullexit-toggle.lock"
247 if [ -f "$LOCK_FILE" ]; then
248     LOCK_PID=$(cat "$LOCK_FILE" 2>/dev/null || echo "")
249     if [ -n "$LOCK_PID" ] && [ "$LOCK_PID" != "$$" ] && kill -0 "$LOCK_PID" 2>/dev/null; then
250         if ps -p "$LOCK_PID" -o args= 2>/dev/null | grep -q -E "toggle\.sh|recover\.sh"; then
251             if [ "$POST_WAKE" = "true" ]; then
252                 echo "[$(date -u +%FT%TZ)] POST-WAKE skipped: another lifecycle script (PID $LOCK_PID) is running"
253                 exit 0
254             else
255                 echo "Error: Another instance of toggle.sh or recover.sh (PID $LOCK_PID) is already running." >&2
256                 exit 1
257             fi
258         fi
259     fi
260 fi
261 echo "$$" > "$LOCK_FILE"
262
263 # Clear the active-state marker in nuclear recovery mode since the gateway is being
264 # completely torn down. This prevents scripts/watcher.sh from triggering post-wake
265 # recoveries in response to network changes caused by the teardown itself.
266 if [ "$POST_WAKE" = "false" ]; then
267     rm -f "/tmp/nullexit-gateway-active.marker"
268 fi
269
270 # WARP Watcher inhibit marker: written immediately in --post-wake mode so the
271 # background WARP Watcher (in toggle.sh) skips failure counting while we are
272 # intentionally tearing down / recreating the warp container. Without this, the
273 # watcher accumulates 6 consecutive warp=off readings during force-recreate (~35s)
274 # and fires nuclear recover.sh, killing a gateway that would have been healthy.
275 WARP_INHIBIT_FILE="/tmp/nullexit-warp-inhibit.marker"
276 if [ "$POST_WAKE" = "true" ]; then
277     touch "$WARP_INHIBIT_FILE"
278     echo "[$(date -u +%FT%TZ)] POST-WAKE started WARP Watcher inhibited to prevent spurious shutdown"
279     echo "during warp recreate." >> output.log
280 fi
281
282 # Ensure the inhibit marker and lock file are always removed when the script exits (success or error)
283 cleanup_recover() {
284     rm -f "$WARP_INHIBIT_FILE"
285     if [ -f "$LOCK_FILE" ] && [ "$(cat "$LOCK_FILE" 2>/dev/null)" = "$$" ]; then
286         rm -f "$LOCK_FILE"
287     fi
288 }
289 trap cleanup_recover EXIT
290
291 # Source common formatting and helper functions
292 source "$SCRIPT_DIR/scripts/common.sh"
293
294 # Always add Homebrew path
295 setup_standard_path
296
297 echo ""
298 if [ "$POST_WAKE" = "true" ]; then
299     echo -e "${YELLOW}${BOLD}${NC}"
300     echo -e "${YELLOW}${BOLD} Nullexit POST-WAKE / POST-ROAM LIGHT ${NC}"
301     echo -e "${YELLOW}${BOLD} (keeps the gateway live, refreshes) ${NC}"
302     echo -e "${YELLOW}${BOLD}${NC}"
303     echo ""
304     echo "Re-hijacking DNS, refreshing Tailscale exit-node, force-recreating"
305     echo "warp if unhealthy, and resetting sharing services. The gateway stays"
306     echo "up. docker compose / Wi-Fi / caffeinate are NOT touched."

```

```

306     echo ""
307 else
308     echo -e "${RED}${BOLD}${NC}"
309     echo -e "${RED}${BOLD}      Nullexit Gateway  RECOVERY MODE      ${NC}"
310     echo -e "${RED}${BOLD}${NC}"
311     echo ""
312     echo "This script will tear down the gateway and restore normal internet."
313     echo ""
314 fi
315
316 # Cache sudo credentials upfront (default mode ONLY)
317 # Skip in --post-wake: the LaunchAgent-launched watcher has no TTY, so 'sudo -v'
318 # would block forever waiting for a password. All post-wake commands use 'sudo -n'
319 # which is non-blocking and relies on /etc/sudoers.d/nullexit NOPASSWD entries.
320 SUDO_KEEPER_PID=""
321 if [ "$POST_WAKE" = "false" ] && [ "$AUTO_MODE" = "false" ] && [ -t 0 ]; then
322     echo -e "${YELLOW}${BOLD}Authentication required for network recovery...${NC}"
323     sudo -v
324     (
325         while true; do
326             sudo -n true 2>/dev/null
327             sleep 60
328         done
329     ) &
330     SUDO_KEEPER_PID=$!
331     trap 'kill $SUDO_KEEPER_PID 2>/dev/null' EXIT
332 fi
333
334 # Resolve physical default interface and gateway upfront
335 PHYSICAL_IFACE=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: (Wi-Fi|Ethernet
336 )/{getline; print $2; exit}')
337 [ -z "$PHYSICAL_IFACE" ] && PHYSICAL_IFACE="en0"
338
339 if [ "$POST_WAKE" = "true" ]; then
340     wait_for_dhcp_settle
341 else
342     # Quick resolution in non-post-wake mode (non-blocking)
343     PHYSICAL_GW=$(ipconfig getpacket "$PHYSICAL_IFACE" 2>> output.log | awk -F'[{}]' '/router/{print $2}')
344 fi
345
346 # 1. Disconnect host Tailscale (default) / Refresh exit-node (post-wake)
347
348 if [ "$POST_WAKE" = "true" ]; then
349     step "Refreshing host Tailscale exit-node routing (post-wake)"
350     if command -v tailscale >> output.log 2>&1; then
351         # Re-issue the SAME exit-node up command toggle.sh START uses. This refreshes
352         # the DERP relay mapping and re-asserts the exit-node preference without
353         # dropping the host's mesh connection (which 'tailscale down' would).
354         TS_IP=""
355         for src in .gateway_ip; do
356             [ -f "$src" ] || continue
357             TS_IP=$(cat "$src" 2>> output.log | tr -d '\r' | awk 'NR==1{print $1; exit}' || true)
358             [ -n "$TS_IP" ] && break
359         done
360         if [ -n "$TS_IP" ]; then
361             step "Re-establishing exit node $TS_IP via 'tailscale set --exit-node'"
362             # 'tailscale set' only mutates the named preference (exit-node here).
363             # Crucially it does NOT call --reset, which would re-apply ALL flags and
364             # trigger a sub-second route-table re-evaluation cycle each post-wake.
365             # On already-connected nodes, the lighter 'set' keeps the existing tunnel
366             # alive while only changing the exit-node preference.
367             if run_with_timeout 15 tailscale set --exit-node="$TS_IP" \
368                 --exit-node-allow-lan-access=true \
369                 >> output.log 2>&1; then
370                 ok "Exit-node $TS_IP re-asserted via 'tailscale set'"
371             else
372                 warn "tailscale set --exit-node=$TS_IP didn't respond; falling back to mesh-only"
373                 run_with_timeout 10 tailscale set --exit-node= \
374                     >> output.log 2>&1 || true
375             fi
376         else
377             warn ".gateway_ip missing cannot re-assert exit node"
378             run_with_timeout 10 tailscale up --reset --ssh=true --accept-dns=false --exit-node= \
379                 >> output.log 2>&1 || true
380         fi
381     else
382         warn "tailscale CLI not found"
383     fi
384 else
385     step "Disconnecting host Tailscale from mesh"

```

```

386     disconnect_tailscale_host "tailscale"
387 fi
388
389 # 2. Reset DNS to default on ALL network services (default) / Re-hijack gateway DNS (post-wake)
390 if [ "$POST_WAKE" = "true" ]; then
391     step "Re-hijacking host DNS to gateway IP (post-wake / post-roam)"
392     TS_IP=""
393     for src in .gateway_ip; do
394         [ -f "$src" ] || continue
395         TS_IP=$(cat "$src" 2>> output.log | tr -d '\r' | awk 'NR==1{print $1; exit}' || true)
396         [ -n "$TS_IP" ] && break
397     done
398
399     if [ -n "$TS_IP" ]; then
400         # Detect the active network service (using helper in common.sh)
401         ACTIVE_SVC=$(get_active_service)
402         ENO_SVC=$(get_en0_service)
403
404         networksetup -setsearchdomains "$ACTIVE_SVC" "ts.net" 2>> output.log || true
405         networksetup -setdnsservers "$ACTIVE_SVC" "$TS_IP" 2>> output.log || true
406         networksetup -setsearchdomains "$ENO_SVC" "ts.net" 2>> output.log || true
407         networksetup -setdnsservers "$ENO_SVC" "$TS_IP" 2>> output.log || true
408
409         HITS=$(networksetup -getdnsservers "$ACTIVE_SVC" 2>> output.log | grep -Fx "$TS_IP" || true)
410         if [ -n "$HITS" ]; then
411             ok "DNS re-hijacked to $TS_IP on services: $ACTIVE_SVC, $ENO_SVC"
412         else
413             warn "networksetup accepted but DNS read-back didn't include $TS_IP"
414             fi
415         else
416             warn ".gateway_ip missing cannot re-hijack DNS (user must run toggle.sh START)"
417             fi
418     else
419         step "Resetting DNS to default on all network services (empty = DHCP)"
420
421         # Get ALL network services (handles spaces in names, skips disabled ones)
422         SERVICES=$(networksetup -listallnetworkservices 2>> output.log | grep -v "^An asterisk" | grep -v "^$"
423             || true)
424
425         if [ -z "$SERVICES" ]; then
426             # Fallback to common names
427             SERVICES="Wi-Fi Ethernet Thunderbolt Ethernet USB 10/100 LAN"
428             fi
429
430         RESET_COUNT=0
431         while IFS= read -r service; do
432             # Strip leading asterisk (disabled services)
433             service_clean=$(echo "$service" | sed 's/^\*//')
434             [ -z "$service_clean" ] && continue
435
436             # Check if this service has a hardware port (skip VPN/service-only entries)
437             if networksetup -listallhardwareports 2>> output.log | grep -A1 "Port: $service_clean$ | grep -q "
438                 Device:" || [ "$service_clean" = "Wi-Fi" ]; then
439                 (
440                     networksetup -setsearchdomains "$service_clean" "Empty" 2>> output.log
441                     # Set to "empty" so macOS uses DHCP-assigned DNS (not hardcoded 1.1.1.1)
442                     sudo networksetup -setdnsservers "$service_clean" "empty" 2>> output.log
443                 ) && RESET_COUNT=$((RESET_COUNT + 1)) || true
444             fi
445         done <<< "$SERVICES"
446
447         ok "DNS reset on $RESET_COUNT service(s)"
448     fi
449
450 # 3. Disable rogue proxy settings (default only gateway uses proxy in non-exit-node path)
451 if [ "$POST_WAKE" = "false" ]; then
452     step "Disabling proxy settings (SOCKS, web, secure web)"
453     for svc in $(networksetup -listallnetworkservices 2>> output.log | grep -v "^An asterisk" | grep -v "^$
454         " || echo "Wi-Fi"); do
455         svc_clean=$(echo "$svc" | sed 's/^\*//')
456         [ -z "$svc_clean" ] && continue
457         disable_all_proxies "$svc_clean"
458     done
459     ok "Proxy settings cleared"
460 else
461     step "Proxy settings: leaving untouched (post-wake keeps gateway SOCKS wiring alive)"
462 fi
463
464 # 4. Flush macOS DNS cache (always safe and useful in both modes)
465 step "Flushing DNS cache"
466 if command -v dscacheutil >> output.log 2>&1; then

```

```

464     flush_dns_cache
465     ok "DNS cache flushed (mDNSResponder HUP)"
466 else
467     warn "dscacheutil not found"
468 fi
469
470 # 5. Clear stale routing table (always safe in both modes)
471 step "Flushing stale routing table entries"
472 # Resolve physical default gateway IP before flushing
473 # We cannot trust 'route get default' here because Tailscale may have already hijacked
474 # the default route to utunX. We must read the actual DHCP router assignment from the hardware.
475 # PHYSICAL_IFACE and PHYSICAL_GW resolved upfront
476
477 # Instead of full route -n flush (which destroys macOS loopback/multicast and wedges the OS until reboot)
478 # we cleanly delete the Tailscale overrides and Cloudflare WARP bypass routes in ALL modes.
479 # This ensures tailscaled isn't blocked from reaching its control plane when waking from sleep.
480 sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
481 sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
482 remove_warp_bypass_routes
483 ok "Routing overrides cleared"
484
485 if [ "$POST_WAKE" = "false" ]; then
486     disable_killswitch
487     ok "Routing table flush completed via targeted deletes and firewall disabled"
488 else
489     ok "Routing overrides cleared (post-wake mode to preserve Colima bridge100)"
490 fi
491
492 # In post-wake mode we don't bounce Wi-Fi, so we MUST manually restore the physical default route
493 if [ "$POST_WAKE" = "true" ] && [ -n "$PHYSICAL_GW" ]; then
494     # Ensure physical gateway is set just in case it was dropped
495     sudo -n route add default "$PHYSICAL_GW" >> output.log 2>&1 || true
496 fi
497
498 # CONDITIONAL BYPASS ROUTE RESTORATION:
499 # If this was a post-wake recovery and the exit node is active, the route flush
500 # just wiped the static bypass routes for the WARP endpoints. We must re-add
501 # them immediately to prevent a routing loop when Tailscale starts sending traffic.
502 if [ "$POST_WAKE" = "true" ]; then
503     if [ -z "${ACTIVE_IF:-}" ]; then
504         ACTIVE_IF=$(route get default 2>> output.log | awk '/interface:/{print $2; exit}')
505         if [ -z "$ACTIVE_IF" ] || [[ "$ACTIVE_IF" =~ ^utun ]] || [[ "$ACTIVE_IF" =~ ^tun ]]; then
506             for i in en0 en1 en2 en3; do
507                 if ifconfig "$i" 2>> output.log | grep -q 'status: active'; then
508                     ACTIVE_IF="$i"; break
509                 fi
510             done
511         fi
512         [ -z "$ACTIVE_IF" ] && ACTIVE_IF="en0"
513     fi
514
515     echo "Re-adding host bypass routes for Cloudflare WARP endpoints..."
516     remove_warp_bypass_routes
517     if [ -n "${PHYSICAL_GW:-}" ]; then
518         add_warp_bypass_routes "$PHYSICAL_GW"
519     else
520         add_warp_bypass_routes "$ACTIVE_IF"
521     fi
522
523     # Re-assert the opt-in FaceTime split-route so it survives roam/wake
524     # (no-op unless FACETIME_SPLIT_ROUTE=true). Physical gateway is known here.
525     add_apple_split_routes "${PHYSICAL_GW:-}"
526
527     # Also re-apply the default route pointing to the Tailscale interface
528     ts_iface=$(ifconfig 2>> output.log | grep -B4 "inet 100." | grep -E '[a-z0-9]+' | cut -d: -f1 | head -
529 n 1)
530     if [ -n "$ts_iface" ]; then
531         echo "Re-routing default gateway to $ts_iface using 0.0.0.0/1 split..."
532         # DO NOT delete the physical default route! It crashes Colima.
533         # Use the /1 trick to mathematically override the default route.
534         sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
535         sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
536         sudo -n route add -net 0.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
537         sudo -n route add -net 128.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
538         enable_killswitch
539     fi
540 fi
541
542 # 6. Stop gateway Docker containers (default) / Force-recreate warp if unhealthy (post-wake)
543 if [ "$POST_WAKE" = "true" ]; then
544     # 6a. Check for Roaming Network Mismatch (Bridged vs Enterprise Wi-Fi)

```

```

543 if [ -f "/tmp/nullexit_colima_mode.txt" ] && [ -f "$SCRIPT_DIR/../../lan_p2p_detected" ]; then
544     current_colima_mode=$(cat /tmp/nullexit_colima_mode.txt)
545     p2p_allowed=$(cat "$SCRIPT_DIR/../../lan_p2p_detected")
546     required_colima_mode="bridged"
547     [ "$p2p_allowed" == "false" ] && required_colima_mode="shared"
548
549     if [ "$current_colima_mode" != "$required_colima_mode" ]; then
550         warn "Network roaming mismatch detected!"
551         warn "Colima is running in '$current_colima_mode' mode but new network requires '$required_colima_mode'."
552         warn "Executing full gateway restart to protect against MAC-spoofing deauthentication..."
553         echo "[$(date +%Y-%m-%d %H:%M:%S)] [Roam] Mismatch ($current_colima_mode vs $required_colima_mode)
554             ). Triggering toggle.sh --restart." >> output.log
555         nohup bash "$SCRIPT_DIR/../../toggle.sh" --restart >/dev/null 2>&1 &
556         exit 0
557     fi
558 fi
559
560 step "Checking Colima VM state"
561 colima_status=$(colima status 2>&1)
562 if echo "$colima_status" | grep -qi "running"; then
563     ok "Colima VM is running"
564     echo "[$(date +%Y-%m-%d %H:%M:%S)] [Docker/Colima] Colima status check passed." >> output.log
565 else
566     warn "Colima VM is NOT running or unresponsive"
567     echo "[$(date +%Y-%m-%d %H:%M:%S)] [Docker/Colima] Colima is unhealthy or dead! Output:
568         $colima_status" >> output.log
569     echo "[$(date +%Y-%m-%d %H:%M:%S)] [Docker/Colima] Attempting to restart Colima..." >> output.log
570     colima restart >> output.log 2>&1 || warn "Failed to restart Colima"
571 fi
572
573 step "Checking warp container health (gluetun UDP tunnel)"
574 echo "[$(date +%Y-%m-%d %H:%M:%S)] [Docker/Colima] Inspecting warp container state..." >> output.log
575 # The warp container is the most failure-prone link in the chain at wake/roam:
576 # its UDP NAT binding to Cloudflare's edge (162.159.192.1:2408) silently dies
577 # during a Wi-Fi roam. We verify the tunnel is actually passing traffic via curl.
578 WARP_STATE=$(docker inspect --format '{{.State.Health.Status}}' warp 2>> output.log || echo "missing")
579 echo "[$(date +%Y-%m-%d %H:%M:%S)] [Docker/Colima] warp container status: $WARP_STATE" >> output.log
580
581 if [ "$WARP_STATE" = "missing" ]; then
582     warn "warp container is missing leaving gateway containers alone (full relaunch requires \"
583         $SCRIPT_DIR/toggle.sh\")"
584     echo "[$(date +%Y-%m-%d %H:%M:%S)] [Docker/Colima] warp container missing from Docker engine.
585         Requires full toggle.sh launch." >> output.log
586 else
587     tmp_err=$(mktemp)
588     if docker compose exec -T warp wget -q0- --timeout=3 https://www.cloudflare.com/cdn-cgi/trace 2>
589         "$tmp_err" | grep -q "warp=on"; then
590         ok "warp container is healthy and actively tunneling traffic (no recreate needed)"
591         echo "[$(date +%Y-%m-%d %H:%M:%S)] [Docker/Colima] warp container passed live traffic healthcheck
592             (Cloudflare trace successful)." >> output.log
593     else
594         warn "warp container tunnel is dead (Docker status: $WARP_STATE) force-recreating all gateway
595             containers to restore network namespace"
596         echo "[$(date +%Y-%m-%d %H:%M:%S)] [Docker/Colima] warp container is DEAD or STUCK. Forcing
597             recreation of gateway stack..." >> output.log
598         if [ -s "$tmp_err" ]; then
599             err_msg=$(cat "$tmp_err" | tr -d '\r')
600             echo "[$(date +%Y-%m-%d %H:%M:%S)] [Docker/Colima] Healthcheck failure details: $err_msg" >>
601             output.log
602         fi
603         docker compose up -d --force-recreate >> output.log 2>&1 || warn "force-recreate failed"
604
605         NEW_STATE=$(docker inspect --format '{{.State.Health.Status}}' warp 2>> output.log || echo "missing")
606         ok "warp container health after recreate: $NEW_STATE"
607         echo "[$(date +%Y-%m-%d %H:%M:%S)] [Docker/Colima] warp container health after recreation:
608             $NEW_STATE" >> output.log
609     fi
610     rm -f "$tmp_err"
611 fi
612 else
613     step "Stopping gateway Docker containers"
614     if command -v docker >> output.log 2>&1 && docker info >> output.log 2>&1; then
615         if [ -f "docker-compose.yml" ]; then
616             docker compose down -t 5 2>> output.log && ok "Containers stopped" || warn "No containers were
617                 running"
618         else
619             warn "docker-compose.yml not found in current directory"
620         fi
621     fi
622 else

```

```

611     warn "Docker not available skipping"
612 fi
613 fi
614
615 # 6b. Stopping sleep prevention (caffeinate) default only
616 if [ "$POST_WAKE" = "false" ]; then
617     step "Stopping sleep prevention"
618     stop_pidfile_daemon "$PID_CAFFEINATE" "system sleep prevention"
619 else
620     step "Sleep prevention: leaving untouched (caffeinate already re-acquired by parent shell)"
621 fi
622
623 # 6c. Stopping DNS Watcher default only
624 if [ "$POST_WAKE" = "false" ]; then
625     step "Stopping DNS Watcher"
626     stop_pidfile_daemon "$PID_DNS_WATCHER" "background DNS Watcher"
627 else
628     step "DNS Watcher: leaving untouched (it keeps re-hijacking DNS on every roam)"
629 fi
630
631 # 7. Power-cycle Wi-Fi default only
632 if [ "$POST_WAKE" = "false" ]; then
633     step "Power-cycling Wi-Fi"
634     bounce_wifi_interfaces
635 else
636     step "Wi-Fi: leaving untouched (post-wake keeps the current Wi-Fi association)"
637 fi
638
639 # 8. Verify (default checks direct internet; post-wake verifies the gateway)
640 if [ "$POST_WAKE" = "true" ]; then
641     step "Verifying gateway is healthy after post-wake refresh"
642
643     # Wait a moment for tailscale + warp healthcheck to settle
644     sleep 3
645
646     GATEWAY_OK=false
647     if command -v dig >> output.log 2>&1; then
648         # Source procedural ports if available
649         if [ -f "/tmp/nullexit-ports.env" ]; then
650             source "/tmp/nullexit-ports.env"
651         fi
652         DNS_PORT=${DNS_PROXY_PORT:-5354}
653
654         # AdGuard is exposed at localhost:${DNS_PORT}. If this resolves, the warp tunnel
655         # is functioning properly because AdGuard routes upstream to warp.
656         if dig +tcp @127.0.0.1 -p "$DNS_PORT" google.com +short +timeout=5 >> output.log 2>&1; then
657             ok "Gateway DNS (AdGuard via localhost:${DNS_PORT}) works"
658             GATEWAY_OK=true
659         else
660             warn "Gateway DNS not responding yet"
661         fi
662     fi
663
664     # Direct host gateway check via Tailscale ping (relayed pong counts as success)
665     if command -v tailscale >> output.log 2>&1; then
666         TS_IP=""
667         [ -f .gateway_ip ] && TS_IP=$(read_adguard_ip || true)
668         if [ -n "$TS_IP" ] && tailscale ping --until-direct=false -c 1 --timeout 4s "$TS_IP" 2>> output.log |
669             grep -q pong; then
670             ok "Gateway $TS_IP reachable via Tailscale"
671             GATEWAY_OK=true
672         else
673             warn "Tailscale ping to gateway did not pong exit-node may still be re-establishing"
674         fi
675     fi
676
677     if [ "$GATEWAY_OK" = "true" ]; then
678         echo -e " ${GREEN}${BOLD} Gateway is healthy after post-wake refresh.${NC}"
679     else
680         echo -e " ${YELLOW}${BOLD} Gateway recovery is in-progress.${NC}"
681         echo " The route may need another 30s before queries succeed."
682         echo " If it stays broken for >60s, run: bash \"${SCRIPT_DIR}/toggle.sh\""
683     fi
684     verify_internet_connectivity
685 fi
686
687 # Summary
688 echo ""
689 echo -e "${BOLD}${NC}"
690 if [ "$POST_WAKE" = "true" ]; then

```



```

691     echo -e "${BOLD}POST-WAKE SUMMARY${NC}"
692 else
693     echo -e "${BOLD}RECOVERY SUMMARY${NC}"
694 fi
695 echo -e "${BOLD}${NC}"
696
697 # Show final DNS state
698 FINAL_DNS=$(networksetup -getdnsservers "Wi-Fi" 2>/dev/null || echo "N/A")
699 echo "  DNS (Wi-Fi):  $FINAL_DNS"
700
701 FINAL_DNS2=$(networksetup -getdnsservers "Ethernet" 2>/dev/null || true)
702 if [[ -z "$FINAL_DNS2" || "$FINAL_DNS2" == *"not a recognized"* || "$FINAL_DNS2" == *"Error"* ]]; then
703     FINAL_DNS2="N/A (Not configured)"
704 fi
705 echo "  DNS (Ethernet): $FINAL_DNS2"
706
707 echo ""
708
709 if [ "$POST_WAKE" = "false" ]; then
710     if [ "$INTERNET_OK" = "true" ]; then
711         echo -e "  ${GREEN}${BOLD} Internet is working.${NC}"
712     else
713         echo -e "  ${YELLOW}${BOLD} Could not verify internet connectivity.${NC}"
714         echo "    This might mean:"
715         echo "    - Your Wi-Fi is disconnected from the router"
716         echo "    - Tailscale is still interfering (try restarting tailscaled:"
717         echo "      brew services restart tailscale)"
718         echo "    - You need to re-connect to Wi-Fi in the menu bar"
719         echo "    - Try toggling Wi-Fi off/on in the menu bar"
720         echo ""
721         echo "    Or try manually:"
722         echo "      networksetup -setdnsservers Wi-Fi empty"
723         echo "      tailscale down"
724         echo "      sudo route delete -net 0.0.0.0/1"
725         echo "      sudo route delete -net 128.0.0.0/1"
726         echo "      brew services restart tailscale"
727     fi
728 fi
729
730 # Reset sharing services to prevent AirDrop freezes after interface changes
731 # This is the SAME step in BOTH modes (post-wake inherits the previous fix from 10.27).
732 echo -e "  Resetting macOS sharing services (AirDrop/AirPlay)..."
733 reset_sharing_services
734
735 echo ""
736 if [ "$POST_WAKE" = "true" ]; then
737     echo -e "${YELLOW}${BOLD}Post-wake refresh complete.${NC}"
738     # Remove the WARP Watcher inhibit marker now that the gateway is fully healthy.
739     # The watcher will resume normal liveness monitoring on its next 5s poll.
740     rm -f "$WARP_INHIBIT_FILE"
741     echo "[$(date -u +%FT%TZ)] POST-WAKE complete  WARP Watcher inhibit released." >> output.log
742 else
743     echo -e "${GREEN}${BOLD}Recovery complete.${NC}"
744 fi
745 echo ""

```

8 setup.sh

```

1  #!/bin/bash
2  # setup.sh nullexit one-shot setup script (macOS)
3  # Configures Cloudflare WARP + Tailscale + AdGuard Home from scratch.
4  set -euo pipefail
5
6  # Source common formatting and helper functions
7  source "$(dirname "$0")/scripts/common.sh"
8
9  # Run common installation logic
10 source "$(dirname "$0")/scripts/setup-common.sh"
11
12 # Compile macOS Shortcuts
13 echo -e "\n${BOLD}Compiling macOS Desktop Shortcuts...${NC}"
14 if command -v osacompile &> /dev/null; then
15     osacompile -o "Toggle Gateway.app" "Toggle-Gateway.applescript" >/dev/null 2>&1
16     osacompile -o "Recover Gateway.app" "Recover-Gateway.applescript" >/dev/null 2>&1
17     echo "    Created 'Toggle Gateway.app' and 'Recover Gateway.app'"
18 fi

```

```

19
20 # Done
21 echo ""
22 echo -e "${GREEN}${BOLD}${NC}"
23 echo -e "${GREEN}${BOLD}                Setup complete!                ${NC}"
24 echo -e "${GREEN}${BOLD}${NC}"
25 echo ""
26 echo -e "${BOLD}One manual step remaining  approve the exit node:${NC}"
27 echo "    https://login.tailscale.com/admin/machines"
28 echo "    Find your gateway  '...'  'Edit route settings'  enable Exit Node"
29 echo ""
30
31 echo -e "${YELLOW}${BOLD}Sudoers / Background Execution Requirements:${NC}"
32 echo "    To allow silent, non-interactive execution of the firewall and network routes (especially during
33 sleep/wake recovery), you must configure passwordless sudo."
34 echo "    Run this exact command in your terminal (grants are scoped to least privilege):"
35 echo "        echo \"\$USER ALL=(root) NOPASSWD: /sbin/pfctl, /usr/sbin/networksetup, /sbin/route, /sbin/
36 ifconfig, /usr/bin/dscacheutil -flushcache, /usr/bin/killall -HUP mDNSResponder, /usr/bin/killall
37 sharingd rapportd, /usr/bin/pkill -9 tailscaled, /usr/bin/pkill -f dns-proxy.py, /bin/kill, /usr/bin
38 /true, /opt/homebrew/bin/brew services * tailscale, /opt/homebrew/bin/brew uninstall tailscale, /usr
39 /local/bin/brew services * tailscale, /usr/local/bin/brew uninstall tailscale, /usr/bin/python3 -I \
40 $PWD/scripts/dns-proxy.py, /opt/homebrew/bin/python3 -I \$PWD/scripts/dns-proxy.py, /usr/local/bin/
41 python3 -I \$PWD/scripts/dns-proxy.py\" | sudo tee /etc/sudoers.d/nullexit >/dev/null && sudo visudo
42 -cf /etc/sudoers.d/nullexit"
43
44 echo ""
45
46 if [[ -n "${TS_IP:-}" ]]; then
47     echo -e "${BOLD}AdGuard Home dashboard:${NC}    http://${TS_IP}:3000    (username: admin)"
48     echo ""
49 fi
50
51 echo -e "${BOLD}To toggle the gateway on/off:${NC}"
52 echo "    macOS:    double-click the 'Toggle Gateway' app icon!"
53 echo "    Windows: double-click Toggle-Gateway.bat"
54 echo ""
55
56 if [[ "$OSTYPE" == "darwin"* ]]; then
57     # LuLu localhost filtering check
58     if [ -d "/Applications/LuLu.app" ] || systemextensionsctl list 2>/dev/null | grep -q "com.objective-
59 see.lulu"; then
60         echo -e "${YELLOW}${BOLD}LuLu Firewall Detected:${NC}"
61         echo "    LuLu's localhost (loopback) filtering is OFF by default. To properly protect the"
62         echo "    local nullexit proxy ports from malicious local processes, you must manually enable"
63         echo "    'Allow Loopback' (or block it selectively) in LuLu's Preferences -> Rules."
64         echo ""
65     fi
66
67     echo -e "${YELLOW}${BOLD}Note on macOS Tailscale Sandboxing:${NC}"
68     echo "    The Tailscale GUI app (tailscale-app) installed on macOS is sandboxed."
69     echo "    While you cannot run a Tailscale SSH server on this Mac host, you can"
70     echo "    still SSH outbound to other devices on the mesh using their MagicDNS"
71     echo "    addresses directly (e.g. ssh user@device.tailnet-name.ts.net)."

```

9 scripts/setup-linux.sh

```

1 #!/bin/bash
2 # scripts/setup-linux.sh  nullexit one-shot setup script (Linux)
3 # Configures Cloudflare WARP + Tailscale + AdGuard Home from scratch.
4 set -euo pipefail
5
6 # Source common formatting and helper functions
7 source "${dirname "$0"}/common.sh"
8
9 # Run common installation logic
10 source "${dirname "$0"}/setup-common.sh"
11
12 # Compile Linux Desktop Shortcuts
13 echo -e "\n${BOLD}Creating Linux Desktop Shortcuts...${NC}"
14 DIR="$(pwd)"
15 cat <<EOF > "Toggle Gateway.desktop"
16 [Desktop Entry]
17 Version=1.0
18 Name=Toggle Gateway
19 Comment=Start or Stop Nullexit Gateway

```

```

20 Exec=bash -c "cd '$DIR' && ./toggle.sh; read -p 'Press Enter to close...' dummy"
21 Icon=utilities-terminal
22 Terminal=true
23 Type=Application
24 Categories=Network;Security;
25 EOF
26
27 cat <<EOF > "Recover Gateway.desktop"
28 [Desktop Entry]
29 Version=1.0
30 Name=Recover Gateway
31 Comment=Emergency DNS and Network Recovery
32 Exec=bash -c "cd '$DIR' && ./recover.sh; read -p 'Press Enter to close...' dummy"
33 Icon=utilities-terminal
34 Terminal=true
35 Type=Application
36 Categories=Network;Security;
37 EOF
38 echo "    Created 'Toggle Gateway.desktop' and 'Recover Gateway.desktop'"
39
40 # Done
41 echo ""
42 echo -e "${GREEN}${BOLD}${NC}"
43 echo -e "${GREEN}${BOLD}                Setup complete!                ${NC}"
44 echo -e "${GREEN}${BOLD}${NC}"
45 echo ""
46 echo -e "${BOLD}One manual step remaining  approve the exit node:${NC}"
47 echo "    https://login.tailscale.com/admin/machines"
48 echo "    Find your gateway  '...'  'Edit route settings'  enable Exit Node"
49 echo ""
50
51 if [[ -n "${TS_IP:-}" ]]; then
52     echo -e "${BOLD}AdGuard Home dashboard:${NC}  http://${TS_IP}:3000  (username: admin)"
53     echo ""
54 fi
55
56 echo -e "${BOLD}To toggle the gateway on/off:${NC}"
57 echo "    Linux: double-click the 'Toggle Gateway' desktop icon!"
58 echo "    (or run ./toggle.sh in terminal)"
59 echo ""

```

10 docker-compose.yml

```

1 services:
2   warp:
3     image: qmcgaw/gluetun:v3.41.1
4     container_name: warp
5     hostname: nullexit-gateway
6     cap_add:
7       - NET_ADMIN
8     devices:
9       - /dev/net/tun:/dev/net/tun
10    ports:
11      - "127.0.0.1:${DNS_PROXY_PORT:-5354}:5335/udp" # AdGuard DNS: host binds 5354 (the mapped DNS-
12        proxy port); 5335 is AdGuard's in-container port
13      - "127.0.0.1:${DNS_PROXY_PORT:-5354}:5335/tcp" # AdGuard DNS (TCP)
14      - "41642:41642/udp" # Tailscale WireGuard (direct connection to host)
15      - "127.0.0.1:${SOCKS_PROXY_PORT:-1080}:1080/tcp" # SOCKS5 proxy (routed through WARP tunnel)
16      - "127.0.0.1:${TOR SOCKS PORT:-9050}:9050/tcp" # Tor SOCKS5 proxy (procedurally generated)
17      - "127.0.0.1:${TOR_CONTROL_PORT:-9051}:9051/tcp" # Tor Control port (procedurally generated)
18    environment:
19      - TZ=America/New_York
20      - VPN_SERVICE_PROVIDER=custom
21      - VPN_TYPE=wireguard
22      - WIREGUARD_PRIVATE_KEY=${WIREGUARD_PRIVATE_KEY}
23      - WIREGUARD_PUBLIC_KEY=${WIREGUARD_PUBLIC_KEY}
24      - WIREGUARD_ADDRESSES=${WIREGUARD_ADDRESSES}
25      - WIREGUARD_ENDPOINT_IP=${WARP_ENDPOINT_1:-162.159.192.1}
26      - WIREGUARD_ENDPOINT_PORT=2408
27      # Set to 25s (WireGuard default) to beat standard 30s hardware NAT/firewall UDP timeouts
28      - WIREGUARD_PERSISTENT_KEEPLIVE_INTERVAL=25s
29      # MTU Fix (Critical): Prevent packet fragmentation from double-encryption
30      - WIREGUARD_MTU=1280
31      # Give WARP more time to establish the connection before internal restart (default is 6s which is
32        too short)
33      - HEALTH_VPN_DURATION_INITIAL=30s

```

```

32 # Use plaintext DNS resolvers instead of DNS-over-TLS (DoT) to prevent startup failures on networks
   # where port 853 is blocked
33 - DNS_UPSTREAM_RESOLVER_TYPE=plain
34 - DNS_UPSTREAM_PLAIN_ADDRESSES=1.1.1.1:53,8.8.8.8:53
35 # Allow local network traffic to bypass WARP so Tailscale can establish fast, direct peer-to-peer
   # connections (no DERP relays) on the LAN
36 - FIREWALL_OUTBOUND_SUBNETS=192.168.0.0/16,172.16.0.0/12,10.0.0.0/8
37 # Built-in DNS blocking for ads, malicious domains, and tracking
38 # NOTE: This is the massive "Hagezi-style" built-in blocklist.
39 # You can turn these to 'on' for maximum security if you have spare RAM
40 # or are running this on a cloud exit node with adequate autonomous resources (requires approx 1-2
   # GB of RAM).
41 # Disabled intentionally: AdGuard Home provides the DNS filtering layer instead.
42 # If AdGuard fails, there is no fallback filtering, but the healthcheck
43 # dependency chain ensures Tailscale won't come up if AdGuard is down.
44 - BLOCK_MALICIOUS=off
45 - BLOCK_SURVEILLANCE=off
46 - BLOCK_ADS=off
47 dns:
48 - 1.1.1.1
49 sysctls:
50 - net.ipv4.ip_forward=1
51 volumes:
52 - ./post-rules.txt:/iptables/post-rules.txt:ro
53 restart: unless-stopped
54 healthcheck:
55   test: ["CMD-SHELL", "/gluetun-entrypoint healthcheck"]
56   interval: 2s
57   timeout: 5s
58   retries: 15
59   start_period: 60s
60 logging:
61   driver: "json-file"
62   options:
63     max-size: "10m"
64     max-file: "3"
65
66 tailscale:
67   image: tailscale/tailscale:v1.98.4
68   container_name: tailscale
69   network_mode: service:warp
70   depends_on:
71     warp:
72       condition: service_healthy
73     adguardhome:
74       condition: service_healthy
75   cap_add:
76     - NET_ADMIN
77     - NET_RAW
78     - SYS_MODULE
79   sysctls:
80     - net.ipv4.ip_forward=1
81     - net.ipv6.conf.all.forwarding=1
82   environment:
83     - TZ=America/New_York
84     - TS_EXTRA_ARGS=--advertise-exit-node
85     - PORT=41642
86     - TS_USERSPACE=false
87     - TS_AUTHKEY=${TS_AUTHKEY}
88     - TS_STATE_DIR=/var/lib/tailscale
89     # (Note: For headless/cloud deployments, you can bypass this footgun by
90     # using Tailscale Ephemeral Keys which auto-clean themselves).
91     - TS_AUTH_ONCE=true
92     # MTU Optimization: Force Tailscale packets to be smaller than WARP's 1280 MTU
93     # to prevent WireGuard-in-WireGuard fragmentation, completely eliminating bufferbloat
94     - TS_DEBUG_MTU=1200
95     # Enable Tailscale's built-in SOCKS5 proxy to route out of the WARP tunnel
96     - TS_SOCKS5_SERVER=:1080
97   devices:
98     - /dev/net/tun:/dev/net/tun
99   volumes:
100     - tailscale_state:/var/lib/tailscale
101   restart: unless-stopped
102   logging:
103     driver: "json-file"
104     options:
105       max-size: "10m"
106       max-file: "3"
107
108 routing-fix:
109   image: nullexit-routing-fix:v1.0.0

```

```

110 build:
111   context: ./scripts
112   dockerfile: Dockerfile.routing-fix
113   container_name: routing-fix
114   network_mode: service:warp
115   # NOTE: Deliberately NOT sharing tailscale's PID namespace the shared
116   # namespace caused routing-fix to be killed whenever tailscale restarted
117   # (e.g. during gluetun health-check flaps), which dropped all routing
118   # table changes and left traffic leaking through eth0 instead of tun0.
119   cap_add:
120     - NET_ADMIN
121     - SYSLOG
122   depends_on:
123     warp:
124       condition: service_healthy
125       # rule-compiler dependency removed: it is now a hash-gated one-off run by
126       # toggle.sh before 'up' (15.8.8). This service boots on the existing
127       # compiled_rules.txt / ip_blocklist.ipset already present in ./adguard/work.
128   environment:
129     - TZ=America/New_York
130     - WARP_ENDPOINT=${WARP_ENDPOINT_1:-162.159.192.1}
131     - BLOCKED_COUNTRIES=${BLOCKED_COUNTRIES}
132     - TAILSCALE_ALLOW_LAN_P2P=${TAILSCALE_ALLOW_LAN_P2P:-false}
133   volumes:
134     - ./scripts/routing-fix.sh:/routing-fix.sh:ro
135     # SECURITY: ./adguard/work/ is bind-mounted into this container.
136     # Never write to it from the host or another container. The
137     # single allowed writer is 'rule-compiler' container manual
138     # edits corrupt the AdGuard cache + BoltDB session store.
139     - ./adguard/work/userfilters:/userfilters:ro
140     - ./adguard/work:/adguard_work:ro
141     - ./app
142   restart: unless-stopped
143   stop_grace_period: 1s
144   logging:
145     driver: "json-file"
146     options:
147       max-size: "1m"
148       max-file: "1"
149
150 adguardhome:
151   # WARNING: AdGuard is pre-configured with hardcoded credentials (admin/nullexit).
152   # This is fine for a local Mac running behind a NAT, but if you are deploying
153   # this on a public cloud VPS with port 80/3000 exposed, change this immediately!
154   image: adguard/adguardhome:v0.107.77
155   container_name: adguardhome
156   network_mode: service:warp
157   depends_on:
158     warp:
159       condition: service_healthy
160       # rule-compiler dependency removed: it is now a hash-gated one-off run by
161       # toggle.sh before 'up' (15.8.8). This service boots on the existing
162       # compiled_rules.txt / ip_blocklist.ipset already present in ./adguard/work.
163   environment:
164     - TZ=America/New_York
165   healthcheck:
166     test: ["CMD-SHELL", "nc -z 127.0.0.1 5335 || exit 1"]
167     interval: 2s
168     timeout: 5s
169     retries: 10
170   volumes:
171     # SECURITY: ./adguard/work/ is bind-mounted into this container.
172     # Never write to it from the host or another container. The
173     # single allowed writer is 'rule-compiler' container manual
174     # edits corrupt the AdGuard cache + BoltDB session store.
175     - ./adguard/work:/opt/adguardhome/work
176     - ./adguard/conf:/opt/adguardhome/conf
177   restart: unless-stopped
178   stop_grace_period: 2s
179   logging:
180     driver: "json-file"
181     options:
182       max-size: "10m"
183       max-file: "3"
184
185 rule-compiler:
186   image: nullexit-rule-compiler:v1.0.0
187   build:
188     context: ./scripts
189     dockerfile: Dockerfile.rule-compiler
190   container_name: rule-compiler

```

```

191 # 'compile' profile: excluded from 'docker compose up'. toggle.sh runs it
192 # on-demand via 'docker compose run --build --rm rule-compiler', hash-gated on
193 # black_list.txt/white_list.txt, so ~340k rules are re-deduped only when a list
194 # actually changes instead of every toggle (~28s saved on the common path, 15.8.8).
195 profiles: ["compile"]
196 volumes:
197   - ../app
198
199 tor:
200   build: ../docker/tor
201   image: nullexit-tor:v1.0.0
202   container_name: tor
203   network_mode: service:warp
204   depends_on:
205     warp:
206       condition: service_healthy
207   environment:
208     - TZ=America/New_York
209     - TOR_USE_BRIDGES=${TOR_USE_BRIDGES:-false}
210     - TOR_BRIDGE_LINES_FILE=${TOR_BRIDGE_LINES_FILE:-../tor-bridges.txt}
211     - TOR_PASSWORD=${ADGUARD_PASSWORD:-nullexit}
212     - TOR_EXCLUDE_EXIT_NODES=${TOR_EXCLUDE_EXIT_NODES:-{us}}
213   working_dir: /nullexit
214   volumes:
215     - ../nullexit:ro
216     - ../output.log:/output.log:rw
217   restart: unless-stopped
218   logging:
219     driver: "json-file"
220     options:
221       max-size: "10m"
222       max-file: "3"
223
224 volumes:
225   tailscale_state:
226
227 networks:
228   default:
229     ipam:
230       config:
231         - subnet: 10.200.1.0/24

```

11 scripts/common.sh

```

1 #!/bin/bash
2 # common.sh shared bash utilities for nullexit scripts
3
4 # Formatting
5 RED='\033[0;31m'
6 GREEN='\033[0;32m'
7 YELLOW='\033[1;33m'
8 BLUE='\033[0;34m'
9 BOLD='\033[1m'
10 NC='\033[0m'
11
12 # Constants & Environment
13 export MARKER_FILE="/tmp/nullexit-gateway-active.marker"
14 export PID_CAFFEINATE="/tmp/nullexit-caffeinate.pid"
15 export PID_DNS_WATCHER="/tmp/nullexit-dns-watcher.pid"
16 export DEFAULT_WARP_ENDPOINT_1="162.159.192.1"
17 export DEFAULT_WARP_ENDPOINT_2="162.159.193.1"
18
19 setup_standard_path() {
20   export PATH="/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:$PATH"
21 }
22
23 step() { echo -e "\n[ $(date '+%Y-%m-%d %H:%M:%S') ] ${BLUE}${BOLD} ${*}${NC}"; }
24 ok() { echo -e "[ $(date '+%Y-%m-%d %H:%M:%S') ] ${GREEN} ${*}${NC}"; }
25 warn() { echo -e "[ $(date '+%Y-%m-%d %H:%M:%S') ] ${YELLOW} ${*}${NC}"; }
26 fail() { echo -e "[ $(date '+%Y-%m-%d %H:%M:%S') ] ${RED} ${*}${NC}"; }
27 die() { echo -e "\n[ $(date '+%Y-%m-%d %H:%M:%S') ] ${RED} ${*}${NC}\n"; exit 1; }
28
29 # Helper Functions
30
31 # Append a timestamped line to output.log (best-effort; never fails the caller).
32 # Used to record lifecycle breadcrumbs (EXEC/EXIT markers, phase changes) so a

```



```

33 # START/STOP that is killed or window-closed still leaves a retraceable trail.
34 log_line() {
35     echo "[$(date +%Y-%m-%d %H:%M:%S)] $*" >> "${LOG_FILE:-output.log}" 2>/dev/null || true
36 }
37
38 # Run a lifecycle command with FULL observability: log the command, stream its
39 # combined stdout+stderr to the terminal AND append it to output.log via tee,
40 # then log the exit code explicitly. Returns the command's real exit code (not
41 # tee's). This closes the blind spot where 'docker compose up' / 'colima start'
42 # printed only to the terminal so when they fail (or the run is interrupted),
43 # output.log shows exactly what the last command was and whether it succeeded.
44 #
45 # Usage: run_logged docker compose up -d --build
46 run_logged() {
47     local logf="${LOG_FILE:-output.log}"
48     log_line "EXEC: $*"
49     # PIPESTATUS[0] is the command's exit code; tee (PIPESTATUS[1]) is ignored.
50     "$@" 2>&1 | tee -a "$logf"
51     local rc=${PIPESTATUS[0]}
52     log_line "EXIT $rc: $*"
53     return "$rc"
54 }
55
56 # Pure-bash timeout (no dependency on GNU coreutils' timeout)
57 # Runs a command with a safety cutoff so a wedged daemon can't hang the script.
58 run_with_timeout() {
59     local timeout_sec="$1"
60     shift
61     if [ $# -eq 0 ]; then return 1; fi
62
63     "$@" &
64     local cmd_pid=$!
65     CURRENT_BG_PID="$cmd_pid"
66
67     (
68         sleep "$timeout_sec"
69         if kill -0 "$cmd_pid" 2>> output.log; then
70             echo -e "\n[Timeout] Command '$*' exceeded $timeout_sec seconds. Terminating..." >&2
71             kill -15 "$cmd_pid" 2>> output.log || true
72             sleep 2
73             if kill -0 "$cmd_pid" 2>> output.log; then
74                 kill -9 "$cmd_pid" 2>> output.log || true
75             fi
76         fi
77     ) &
78     local watcher_pid=$!
79
80     # Disable set -e temporarily to capture exit status of wait
81     set +e
82     wait "$cmd_pid" 2>> output.log
83     local exit_status=$?
84     set -e
85
86     # Kill the watcher since the command finished
87     kill "$watcher_pid" 2>> output.log && wait "$watcher_pid" 2>> output.log || true
88
89     CURRENT_BG_PID=""
90     return "$exit_status"
91 }
92
93 # Start Colima and return the moment Docker is actually reachable, instead of
94 # blocking on colima 0.10.3's post-provision hang. Observed on macOS/vz: the VM
95 # reaches READY and creates the 'colima' docker context in ~10-15s, then
96 # 'colima start' frequently fails to return for ~110s (until a watchdog kills it)
97 # even though Docker is fully usable the entire time. We poll the colima docker
98 # context and proceed as soon as it answers, then reap the (usually hung) starter.
99 # The VM keeps running and 'colima status' stays accurate. $@ = colima start args.
100 # Returns 0 once Docker responds, 1 if it never does within the cap.
101 colima_start_until_ready() {
102     local max_wait=90 waited=0 ready=false start_pid
103     echo "[$(date +%Y-%m-%d %H:%M:%S)] EXEC(bg): colima start $*" >> output.log
104     colima start "$@" >> output.log 2>&1 &
105     start_pid=$!
106     while [ "$waited" -lt "$max_wait" ]; do
107         if run_with_timeout 5 docker --context colima info >/dev/null 2>&1; then
108             ready=true
109             break
110         fi
111         # If colima start exited on its own (genuine finish or hard failure), stop waiting.
112         if ! kill -0 "$start_pid" 2>/dev/null; then
113             run_with_timeout 5 docker --context colima info >/dev/null 2>&1 && ready=true

```

```

114     break
115 fi
116 sleep 3
117 waited=$((waited + 3))
118 done
119 # Reap the starter (harmless if it already exited); this does NOT stop the VM.
120 kill "$start_pid" 2>/dev/null || true
121 wait "$start_pid" 2>/dev/null || true
122 if [ "$ready" = "true" ]; then
123     docker context use colima >/dev/null 2>&1 || true
124     echo "[$(date '+%Y-%m-%d %H:%M:%S')] Colima Docker ready after ${waited}s (starter reaped)." >>
125     output.log
126     return 0
127 fi
128 echo "[$(date '+%Y-%m-%d %H:%M:%S')] Colima Docker NOT ready after ${max_wait}s." >> output.log
129 return 1
130 }
131 # Hash the files that feed the long-running locally-built images that 'docker
132 # compose up' starts (routing-fix, tor). toggle.sh uses this to skip
133 # 'docker compose --build' when nothing changed: BuildKit re-evaluating those
134 # images on the 600MB Colima VM costs ~30s per toggle even when every layer is
135 # CACHED (15.8.8). Includes the Dockerfiles, everything they COPY (routing-fix.sh,
136 # tor/entrypoint.sh, the logger/ Go source), and docker-compose.yml. The
137 # rule-compiler image is NOT here it is a 'compile'-profiled one-off, built+run
138 # by the gated compile step (15.8.8). Prints a sha256 hex digest, or empty.
139 compute_build_inputs_hash() {
140     local root="${SCRIPT_DIR:-.}" f
141     {
142         for f in \
143             "$root/scripts/Dockerfile.routing-fix" \
144             "$root/scripts/routing-fix.sh" \
145             "$root/docker/tor/Dockerfile" \
146             "$root/docker/tor/entrypoint.sh" \
147             "$root/docker-compose.yml"; do
148             printf ':%s:.' "$f"; cat "$f" 2>/dev/null
149         done
150         find "$root/scripts/logger" -type f 2>/dev/null \
151             | LC_ALL=C sort | while IFS= read -r f; do printf ':%s:.' "$f"; cat "$f" 2>/dev/null; done
152     } | openssl dgst -sha256 2>/dev/null | awk '{print $NF}'
153 }
154
155 # Hash the ad-block lists the user controls: black_list.txt (custom blocks) and
156 # white_list.txt (the allow-list that governs what DNS resolves through). toggle.sh
157 # gates the ~28s rule recompilation on this editing either list changes the digest
158 # and triggers a one-off recompilation of compiled_rules.txt + ip_blocklist.ipset;
159 # an unchanged toggle skips it entirely (15.8.8). Prints a sha256 hex digest.
160 compute_rules_hash() {
161     local root="${SCRIPT_DIR:-.}"
162     { printf ':black:.'; cat "$root/black_list.txt" 2>/dev/null
163       printf ':white:.'; cat "$root/white_list.txt" 2>/dev/null
164     } | openssl dgst -sha256 2>/dev/null | awk '{print $NF}'
165 }
166
167 # Check if the gateway is active (either containers are running, or host DNS is hijacked)
168 is_gateway_active() {
169     # 1. Check if containers are running (suppress stderr to avoid errors if docker is down)
170     if run_with_timeout 15 docker compose ps --status running 2>/dev/null | grep -q 'warp'; then
171         echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: TRUE (warp container is running)" >> "
172         $LOG_FILE"
173         return 0
174     fi
175     # 2. Check if host DNS was hijacked (not 1.1.1.1 and not default/empty)
176     local current_dns=""
177     local active_svc=""
178     if [[ "$OSTYPE" == "darwin"* ]]; then
179         active_svc=$(get_active_service)
180         current_dns=$(networksetup -getdnsservers "$active_svc" 2>> output.log || true)
181     else
182         current_dns=$(resolvectl dns 2>/dev/null | awk '/Global|Link/ {for(i=4;i<=NF;i++) print $i}' | head -
183         n1)
184     fi
185     if [[ -n "$current_dns" && "$current_dns" != "1.1.1.1" && ! "$current_dns" =~ "There aren't any DNS
186     Servers" ]]; then
187         echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: TRUE (DNS was hijacked: $current_dns)" >> "
188         $LOG_FILE"
189         return 0
190     fi

```

```

190 # 3. Check if SOCKS proxy is enabled (macOS only)
191 if [[ "$OSTYPE" == "darwin"* ]]; then
192     local socks_proxy
193     socks_proxy=$(networksetup -getsocksfirewallproxy "$active_svc" 2>> output.log || true)
194     if echo "$socks_proxy" | grep -q "Enabled: Yes"; then
195         echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: TRUE (SOCKS proxy enabled)" >> "$LOG_FILE"
196         return 0
197     fi
198 fi
199
200 echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: FALSE (Containers down, DNS clean, SOCKS
    disabled)" >> "$LOG_FILE"
201 return 1
202 }
203
204 # Decide whether the gateway SHOULD be running, from persisted *intent* rather
205 # than a live health probe. This is the correct signal for toggle.sh's
206 # STOP-vs-START decision: a disable should tear down whatever the last START
207 # left behind it should NOT depend on whether Docker/Colima happens to be
208 # reachable right now.
209 #
210 # Why this exists: using is_gateway_active() for the decision means every toggle
211 # (including *disable*) runs 'docker compose ps' first. When Colima is down the
212 # socket is absent, that call blocks for the full run_with_timeout window, and
213 # under 'set -e' + the ERR trap a non-zero result at the 'if then else fi'
214 # boundary could abort the script BEFORE the START body runs so the very path
215 # that would boot Colima never executed (observed as 'EXIT: code=1 phase=start').
216 #
217 # Signal source: MARKER_FILE is written at the end of a successful START
218 # (write_gateway_active_marker) and cleared at the top of STOP and in
219 # cleanup_handler. Reading it is instant and cannot hang or abort.
220 #
221 # ECHOES "running" or "stopped" to stdout (and ALWAYS returns 0). Callers read
222 # the string: [ "$(gateway_state)" = "running" ] &&
223 #
224 # CRITICAL why this echoes instead of using a 0/1 return code: on macOS
225 # /bin/bash 3.2, with 'set -e' + an 'ERR' trap + 'set -x' (DEBUG_TRACE=true) all
226 # active, a function that does 'return 1' triggers a SPURIOUS 'set -e' abort in
227 # the caller even when the call is guarded ('if f; then', or even 'f || rc=$?').
228 # The 'return N' from the function, traced by 'set -x', trips the ERR trap. This
229 # is a genuine bash-3.2 interaction (see devref 15.11.8). Echoing a result and
230 # always returning 0 sidesteps 'return'/'set -e'/'set -x' entirely the function
231 # can never contribute a non-zero status, so no caller can be aborted by it.
232 gateway_state() {
233     # Primary: persisted intent. Marker present => a START completed and no STOP
234     # has cleared it yet.
235     #
236     # NOTE: toggle.sh runs a "defensive stale-marker clear" near the top of every
237     # invocation (before this decision) to stop the wake-watcher firing against a
238     # crashed gateway. That would wipe the marker before we read it, so toggle.sh
239     # captures the marker's existence into GATEWAY_MARKER_AT_STARTUP *before* that
240     # clear and we honor it here. If the variable is unset (other callers, or the
241     # --restart pre-check which runs before the defensive clear), fall back to the
242     # live marker file.
243     if [ -n "${GATEWAY_MARKER_AT_STARTUP:-}" ]; then
244         if [ "$GATEWAY_MARKER_AT_STARTUP" = "yes" ]; then echo "running"; return 0; fi
245     elif [ -f "$MARKER_FILE" ]; then
246         echo "running"; return 0
247     fi
248
249     # Marker absent. Normally this means "stopped". But cover the edge case where
250     # the marker was lost (e.g. /tmp cleared, or gateway started by an older build)
251     # yet containers are genuinely up reconcile via is_gateway_active, but ONLY
252     # when Docker is actually reachable, so a dead-socket probe can never hang. On
253     # macOS Docker runs in the Colima VM; if that socket is absent, the gateway
254     # definitively is not running.
255     local docker_up="no"
256     if [[ "$OSTYPE" == "darwin"* ]]; then
257         if [ -S /var/run/docker.sock ] || [ -S "$HOME/.colima/default/docker.sock" ]; then docker_up="yes";
258         fi
259     else
260         if [ -S /var/run/docker.sock ]; then docker_up="yes"; fi
261     fi
262
263     if [ "$docker_up" = "yes" ] && is_gateway_active; then
264         echo "running"; return 0
265     fi
266     echo "stopped"
267     return 0
268 }

```

```

269 # Reads an environment variable from a file (default: .env), strips quotes and whitespace
270 read_env_var() {
271     local var_name="$1"
272     local env_file="${2:-.env}"
273     grep -E "^${var_name}= " "$env_file" 2>/dev/null | cut -d'=' -f2- | tr -d "\"\`" | sed -e 's/^[:space:
:]*/'/' -e 's/^[:space:]*$//' || echo ""
274 }
275
276 # Load KILL_SWITCH variable from .env (defaults to false if not found/empty)
277 export KILL_SWITCH
278 KILL_SWITCH=$(read_env_var "KILL_SWITCH" 2>/dev/null | tr '[:upper:]' '[:lower:]')
279 if [ -z "$KILL_SWITCH" ]; then
280     KILL_SWITCH="false"
281 fi
282
283 # Function to get the active network service name (e.g., "Wi-Fi" or "USB 10/100 LAN")
284 get_active_service() {
285     if [[ "$OSTYPE" == "darwin*" ]]; then
286         local iface
287         iface=$(route get default 2>> output.log | awk '/interface:/ {print $2}')
288
289         # If the default interface is empty or a VPN tunnel (like utunX), fallback to the active physical
290         # interface
291         if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
292             for i in en0 en1 en2 en3; do
293                 if ifconfig "$i" 2>> output.log | grep -q "status: active" && ifconfig "$i" 2>> output.log | grep
294                     -q "inet "; then
295                     iface="$i"
296                     break
297                 fi
298             done
299
300             # Default fallback if still empty or not enX
301             if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
302                 iface="en0"
303             fi
304
305             # Map interface (e.g., en0) to service name (e.g., Wi-Fi)
306             local service
307             service=$(networksetup -listnetworkserviceorder 2>> output.log | grep -B 1 "Device: $iface" | head -n
308                 1 | sed -E 's/^\([0-9\*]+\)\s*//' )
309
310             if [ -n "$service" ]; then
311                 echo "$service"
312             else
313                 # Get the interface used for default internet routing
314                 local iface
315                 iface=$(ip route get 1.1.1.1 2>/dev/null | awk '{print $5; exit}')
316                 if [ -z "$iface" ]; then
317                     echo ""
318                     return 1
319                 fi
320                 echo "$iface"
321             fi
322         }
323     }
324
325     get_en0_service() {
326         local service
327         service=$(networksetup -listnetworkserviceorder 2>> output.log | grep -B1 "Device: en0" | head -1 | sed
328             -E 's/^\([0-9\*]+\)\s*//' || true)
329         if [ -z "$service" ]; then
330             echo "Wi-Fi"
331         else
332             echo "$service"
333         fi
334     }
335
336     get_warp_endpoint_1() { read_env_var "WARP_ENDPOINT_1" ".env" | grep -Eo '^[0-9\.]+' || echo "
337         $DEFAULT_WARP_ENDPOINT_1"; }
338     get_warp_endpoint_2() { read_env_var "WARP_ENDPOINT_2" ".env" | grep -Eo '^[0-9\.]+' || echo "
339         $DEFAULT_WARP_ENDPOINT_2"; }
340
341     restart_tailscaled_daemon() {
342         echo " [Tailscale Recovery] tailscaled daemon appears to be wedged/unresponsive."
343         echo " [Tailscale Recovery] Attempting to restart tailscaled service..."
344
345         if run_with_timeout 15 brew services restart tailscale >> output.log 2>&1; then

```

```

343     echo " [Tailscale Recovery] Successfully restarted tailscaled (user service)."
344 elif run_with_timeout 15 sudo -n brew services restart tailscale >> output.log 2>&1; then
345     echo " [Tailscale Recovery] Successfully restarted tailscaled (system service)."
346 else
347     echo " [Tailscale Recovery] WARNING: Failed to restart tailscaled. Manual intervention may be needed
348     ."
349 fi
350 sleep 3
351 }
352
353 disconnect_tailscale_host() {
354     local ts_bin="${1:-tailscale}"
355     if command -v "$ts_bin" >> output.log 2>&1; then
356         echo " Disconnecting host Tailscale from mesh..."
357
358         # Check if actually connected first (with timeout tailscaled could be wedged)
359         local status_ok=false
360         if run_with_timeout 5 "$ts_bin" status >> output.log 2>&1; then
361             status_ok=true
362         else
363             local status_exit=$?
364             # If it was a quick exit code 1 (disconnected), it is still reachable
365             if [ "$status_exit" -ne 143 ] && [ -S /var/run/tailscaled.socket ]; then
366                 status_ok=true
367             fi
368         fi
369
370         if [ "$status_ok" = "false" ]; then
371             restart_tailscaled_daemon
372         fi
373
374         # Explicitly reset any exit-node so it doesn't linger.
375         "$ts_bin" up >> output.log 2>&1 || true
376         if run_with_timeout 10 "$ts_bin" set --ssh=true --accept-dns=false --exit-node= >> output.log 2>&1;
377         then
378             echo " [] Exit-node preference cleared."
379         else
380             echo " [!] tailscale up didn't respond (tailscaled may be wedged)"
381         fi
382
383         local ts_args=""
384         if is_kill_switch_enabled; then ts_args="--accept-risk=lose-ssh"; fi
385         if run_with_timeout 10 "$ts_bin" down $ts_args >> output.log 2>&1; then
386             echo " [] Tailscale disconnected."
387         else
388             echo " [!] tailscale down didn't respond"
389         fi
390     else
391         echo " [!] tailscale CLI not found skipping"
392     fi
393 }
394
395 stop_pidfile_daemon() {
396     local pid_file="$1"
397     local label="$2"
398     if [ -f "$pid_file" ]; then
399         local pid
400         pid=$(cat "$pid_file")
401         if kill -0 "$pid" 2>/dev/null; then
402             echo " Stopping $label (PID $pid)..."
403             sudo -n kill "$pid" 2>> output.log || kill "$pid" 2>> output.log || true
404             sleep 0.5
405             if kill -0 "$pid" 2>/dev/null; then
406                 sudo -n kill -9 "$pid" 2>> output.log || kill -9 "$pid" 2>> output.log || true
407             fi
408         fi
409         rm -f "$pid_file"
410     fi
411 }
412
413 disable_all_proxies() {
414     local svc="$1"
415     if [ -n "$svc" ]; then
416         sudo -n networksetup -setsocksfirewallproxystate "$svc" off 2>> output.log || true
417         sudo -n networksetup -setwebproxystate "$svc" off 2>> output.log || true
418         sudo -n networksetup -setsecurewebproxystate "$svc" off 2>> output.log || true
419     fi
420 }
421
422 flush_dns_cache() {
423     if [[ "$OSTYPE" == "darwin"* ]]; then

```

```

422     sudo -n dscacheutil -flushcache 2>> output.log || true
423     sudo -n killall -HUP mDNSResponder 2>> output.log || true
424 else
425     resolvectl flush-caches 2>> output.log || true
426 fi
427 }
428
429 wait_for_dhcp_settle() {
430     # Resolve physical interface (Wi-Fi or Ethernet)
431     local physical_iface
432     physical_iface=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: (Wi-Fi|
433         Ethernet)/{getline; print $2; exit}')
434     [ -z "$physical_iface" ] && physical_iface="en0"
435
436     echo -n "Waiting for physical network interface ($physical_iface) DHCP lease to settle..."
437     local settled=false
438     for attempt in {1..60}; do
439         PHYSICAL_GW=$(ipconfig getpacket "$physical_iface" 2>> output.log | awk -F'[]' ' /router/{print $2}
440         ')
441         local local_ip
442         local_ip=$(ifconfig "$physical_iface" 2>/dev/null | awk '/inet/{print $2}')
443
444         if [ -n "$PHYSICAL_GW" ] && [[ "$PHYSICAL_GW" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] && \
445             [ -n "$local_ip" ] && [[ "$local_ip" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] && \
446             [[ "$local_ip" != "169.254.*" ]]; then
447             echo "settled (Interface IP: $local_ip, Router IP: $PHYSICAL_GW)."
448             settled=true
449             break
450         fi
451
452         if [ "$attempt" -gt 15 ] && [ -n "$local_ip" ] && [[ "$local_ip" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$
453         ]] && [[ "$local_ip" != "169.254.*" ]]; then
454             local fallback_gw
455             fallback_gw=$(route get default 2>> output.log | awk '/gateway:/ {print $2}')
456             if [ -n "$fallback_gw" ] && [[ "$fallback_gw" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]]; then
457                 PHYSICAL_GW="$fallback_gw"
458                 fi
459                 echo "static/settled detected (Interface IP: $local_ip, Gateway IP: ${PHYSICAL_GW:-unknown})."
460                 settled=true
461                 break
462             fi
463             echo -n "."
464             sleep 0.5
465         done
466         if [ "$settled" = "false" ]; then
467             echo "timed out or no active link. Proceeding anyway."
468         fi
469     }
470
471     reset_sharing_services() {
472         if [[ "$OSTYPE" == "darwin*" ]]; then
473             sudo -n killall sharingd rapportd 2>> output.log || true
474         fi
475     }
476
477     read_adguard_ip() {
478         if [ -f ".gateway_ip" ]; then
479             tr -d '\r' < ".gateway_ip" | awk 'NR==1{print $1;exit}'
480         fi
481     }
482
483     is_kill_switch_enabled() {
484         [ "$KILL_SWITCH" = "true" ]
485     }
486
487     add_warp_bypass_routes() {
488         local target="${1:-}"
489         local ep1
490         ep1=$(get_warp_endpoint_1)
491         local ep2
492         ep2=$(get_warp_endpoint_2)
493
494         if [[ "$OSTYPE" == "darwin*" ]]; then
495             local routing_arg=""
496             local msg_via=""
497             if [ -n "$target" ]; then
498                 if [[ "$target" =~ ^en[0-9]+$ || "$target" == "bridge*" ]]; then
499                     routing_arg="-interface $target"
500                     msg_via="interface $target"
501                 else
502                     routing_arg="$target"
503                 fi
504             fi
505         fi
506     }

```



```

500     msg_via="gateway $target"
501 fi
502 else
503     local gateway_ip
504     gateway_ip=$(route get default 2>> output.log | awk '/gateway:/ {print $2}')
505     if [ -z "$gateway_ip" ]; then
506         local iface
507         iface=$(route get default 2>> output.log | awk '/interface:/ {print $2}')
508         if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
509             iface="en0"
510         fi
511         routing_arg="-interface $iface"
512         msg_via="interface $iface"
513     else
514         routing_arg="$gateway_ip"
515         msg_via="gateway $gateway_ip"
516     fi
517 fi
518
519 echo -e "\nAdding host bypass routes for Cloudflare WARP endpoints via $msg_via..."
520 sudo -n route delete -host "$ep1" 2>/dev/null || true
521 sudo -n route delete -host "$ep2" 2>/dev/null || true
522 sudo -n route add -host "$ep1" $routing_arg >> output.log 2>&1 || true
523 sudo -n route add -host "$ep2" $routing_arg >> output.log 2>&1 || true
524
525 echo -e "Adding host bypass routes for Tailscale control plane via $msg_via..."
526 sudo -n route delete -net 192.200.0.0/24 2>/dev/null || true
527 sudo -n route add -net 192.200.0.0/24 $routing_arg >> output.log 2>&1 || true
528
529 echo -e "Adding host bypass routes for Tailscale DERP relays via $msg_via..."
530 local derp_ips
531 derp_ips=$(curl -s --connect-timeout 5 https://login.tailscale.com/derpmap/default | grep -oE '"IPv4
532 "[[:space:]]*:[[:space:]]*" "[0-9.]+"' | cut -d'"' -f4 | grep -E '^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$' ||
533 true)
534 if [ -n "$derp_ips" ]; then
535     echo "$derp_ips" > /tmp/nullexit-derp-ips.txt
536     for ip in $derp_ips; do
537         sudo -n route delete -host "$ip" 2>/dev/null || true
538         sudo -n route add -host "$ip" $routing_arg >> output.log 2>&1 || true
539     done
540 fi
541 else
542     local gateway_ip="${target}"
543     if [ -z "$gateway_ip" ]; then
544         gateway_ip=$(ip route show default 2>/dev/null | awk '/default/ {print $3}')
545     fi
546     if [ -n "$gateway_ip" ]; then
547         sudo ip route add "$ep1" via "$gateway_ip" >> output.log 2>&1 || true
548         sudo ip route add "$ep2" via "$gateway_ip" >> output.log 2>&1 || true
549     fi
550 fi
551 }
552
553 remove_warp_bypass_routes() {
554     local ep1
555     ep1=$(get_warp_endpoint_1)
556     local ep2
557     ep2=$(get_warp_endpoint_2)
558
559     if [[ "$OSTYPE" == "darwin"* ]]; then
560         echo -e "\nRemoving host bypass routes for Cloudflare WARP endpoints..."
561         sudo -n route delete -host "$ep1" >> output.log 2>&1 || true
562         sudo -n route delete -host "$ep2" >> output.log 2>&1 || true
563         echo -e "Removing host bypass routes for Tailscale control plane..."
564         sudo -n route delete -net 192.200.0.0/24 >> output.log 2>&1 || true
565         if [ -f /tmp/nullexit-derp-ips.txt ]; then
566             echo -e "Removing host bypass routes for Tailscale DERP relays..."
567             while read -r ip; do
568                 sudo -n route delete -host "$ip" >> output.log 2>&1 || true
569             done < /tmp/nullexit-derp-ips.txt
570             rm -f /tmp/nullexit-derp-ips.txt
571         fi
572     else
573         sudo ip route del "$ep1" >> output.log 2>&1 || true
574         sudo ip route del "$ep2" >> output.log 2>&1 || true
575     fi
576 }
577
578 # FaceTime / real-time P2P split-route (opt-in, default OFF)
579 # Mitigation for the FaceTime Double-Tunnel bug (15.12.22). Adds more-specific
580 # host routes for Apple's FaceTime media/relay /16s via the PHYSICAL gateway, so

```

```

579 # longest-prefix match beats the exit-node 0.0.0.0/1 + 128.0.0.0/1 and ONLY those
580 # /16s leave direct real-time media can then hole-punch from the real IP instead
581 # of dying behind WARP's symmetric NAT. Mirrors add_warp_bypass_routes.
582 #
583 # PRIVACY: the direct /16s expose the real ISP IP (not WARP) for that traffic
584 # a deliberate, narrow hole in the Double-Tunnel promise, scoped to Apple infra
585 # (which already ties traffic to your Apple ID). Inert unless FACETIME_SPLIT_ROUTE
586 # =true. Needs BOTH a route AND a PF pass: 'block out on en0 all' would otherwise
587 # drop the packets, so we populate the <apple_direct> table pf.conf permits.
588 facetime_split_enabled() {
589   [ "$(read_env_var FACETIME_SPLIT_ROUTE | tr '[:upper:]' '[:lower:]')" = "true" ]
590 }
591
592 facetime_direct_subnets() {
593   local s; s=$(read_env_var FACETIME_DIRECT_SUBNETS)
594   echo "${s:-17.249.0.0/16}"
595 }
596
597 add_apple_split_routes() {
598   facetime_split_enabled || return 0
599   [[ "$OSTYPE" == "darwin"* ]] || return 0
600   local gw="${1:-}"
601   if [ -z "$gw" ]; then
602     # The physical default route survives alongside the exit-node /1 override,
603     # so the real gateway is still the 'default' entry in the table.
604     gw=$(netstat -rn -f inet 2>/dev/null | awk '1=="default"{print $2; exit}')
605   fi
606   if [ -z "$gw" ] || [[ "$gw" =~ ^tun ]] || [[ "$gw" =~ ^link ]]; then
607     echo " [FaceTime split-route] could not resolve a physical gateway skipping."
608     return 0
609   fi
610   local subnets; subnets=$(facetime_direct_subnets)
611   echo -e "\n[FaceTime split-route] routing Apple subnets DIRECT via $gw (real IP exposed for these):
612     $subnets"
613   for net in $subnets; do
614     sudo -n route delete -net "$net" >> output.log 2>&1 || true
615     sudo -n route add -net "$net" "$gw" >> output.log 2>&1 || true
616     # Kill-switch pass: the <apple_direct> table is what pf.conf lets out on en0.
617     sudo -n pfctl -a com.apple/nullexit -t apple_direct -T add "$net" >> output.log 2>&1 || true
618   done
619 }
620
621 remove_apple_split_routes() {
622   [[ "$OSTYPE" == "darwin"* ]] || return 0
623   local subnets; subnets=$(facetime_direct_subnets)
624   for net in $subnets; do
625     sudo -n route delete -net "$net" >> output.log 2>&1 || true
626   done
627   # Flush the whole table so no direct Apple exception lingers after teardown.
628   sudo -n pfctl -a com.apple/nullexit -t apple_direct -T flush >> output.log 2>&1 || true
629 }
630
631 bounce_wifi_interfaces() {
632   if [[ "$OSTYPE" == "darwin"* ]]; then
633     local wifi_port
634     wifi_port=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: Wi-Fi/{getline;
635       print $2}')
636     if [ -n "$wifi_port" ]; then
637       sudo -n networksetup -setairportpower "$wifi_port" off 2>> output.log || true
638       sleep 2
639       sudo -n networksetup -setairportpower "$wifi_port" on 2>> output.log || true
640       echo "Wi-Fi bounced (interface $wifi_port)"
641       sleep 3
642     else
643       echo "Could not detect Wi-Fi interface. Bouncing fallback interfaces..."
644       set +e
645       for iface in en0 en1 en2 en3 en4 en5; do
646         if ifconfig "$iface" >> output.log 2>&1; then
647           sudo -n ifconfig "$iface" down 2>> output.log || true
648           sudo -n ifconfig "$iface" up 2>> output.log || true
649         fi
650       done
651       set -e
652       echo "Fallback interfaces bounced"
653     fi
654   fi
655 }
656
657 get_active_interface() {
658   local iface
659   iface=$(route get default 2>> output.log | awk '/interface:/ {print $2}')

```

```

658
659 # If the default interface is empty or a VPN tunnel (like utunX), fallback to the active physical
660 interface
661 if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
662     for i in en0 en1 en2 en3; do
663         if ifconfig "$i" 2>> output.log | grep -q "status: active" && ifconfig "$i" 2>> output.log | grep -
664             q "inet "; then
665             iface="$i"
666             break
667         fi
668     done
669 fi
670 # Default fallback if still empty or not enX
671 if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
672     iface="en0"
673 fi
674 echo "$iface"
675 }
676
677 enable_killswitch() {
678     if [[ "$OSTYPE" == "darwin"* ]]; then
679         if ! is_kill_switch_enabled; then
680             return 0
681         fi
682
683         local ext_if
684         ext_if=$(get_active_interface)
685         local ep1
686         ep1=$(get_warp_endpoint_1)
687         local ep2
688         ep2=$(get_warp_endpoint_2)
689         local pf_conf_path
690         pf_conf_path="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/pf.conf"
691
692         echo -e "\nEnabling macOS Packet Filter (PF) Kill-Switch on $ext_if..."
693
694         # Enable PF and capture the reference token
695         local token_out
696         token_out=$(sudo -n pfctl -E 2>> output.log || true)
697         local token
698         token=$(echo "$token_out" | awk '/Token/{print $NF}')
699         if [ -n "$token" ]; then
700             echo "$token" > /tmp/nullexit-pf-token.txt
701             echo " [] PF enabled with token $token."
702         else
703             echo " [!] WARNING: Could not capture PF reference token (may already be enabled or sudo failed)."
704         fi
705
706         # Load the rules into the anchor
707         if sudo -n pfctl -D "ext_if=$ext_if" -a com.apple/nullexit -f "$pf_conf_path" >> output.log 2>&1;
708         then
709             echo " [] Ruleset loaded into anchor com.apple/nullexit."
710         else
711             echo " [!] ERROR: Failed to load ruleset into anchor."
712         fi
713
714         # Populate tables in the anchor
715         sudo -n pfctl -a com.apple/nullexit -t vpn_endpoints -T flush >> output.log 2>&1 || true
716         sudo -n pfctl -a com.apple/nullexit -t vpn_endpoints -T add "$ep1" >> output.log 2>&1 || true
717         sudo -n pfctl -a com.apple/nullexit -t vpn_endpoints -T add "$ep2" >> output.log 2>&1 || true
718
719         if [ -f /tmp/nullexit-derp-ips.txt ]; then
720             local derp_ips
721             derp_ips=$(tr '\n' ' ' < /tmp/nullexit-derp-ips.txt)
722             if [ -n "$derp_ips" ]; then
723                 sudo -n pfctl -a com.apple/nullexit -t derp_relays -T flush >> output.log 2>&1 || true
724                 sudo -n pfctl -a com.apple/nullexit -t derp_relays -T add $derp_ips >> output.log 2>&1 || true
725             fi
726             echo " [] Kill-switch tables populated."
727         fi
728     }
729
730 disable_killswitch() {
731     if [[ "$OSTYPE" == "darwin"* ]]; then
732         if ! is_kill_switch_enabled; then
733             return 0
734         fi
735         echo -e "\nDisabling macOS PF Kill-Switch..."

```

```

736
737 # Flush rules from anchor com.apple/nullexit
738 sudo -n pfctl -a com.apple/nullexit -F all >> output.log 2>&1 || true
739
740 # Release the enable reference if token is present
741 if [ -f /tmp/nullexit-pf-token.txt ]; then
742     local token
743     token=$(cat /tmp/nullexit-pf-token.txt)
744     if [ -n "$token" ]; then
745         sudo -n pfctl -X "$token" >> output.log 2>&1 || true
746         echo " [] Released PF enable reference token $token."
747     fi
748     rm -f /tmp/nullexit-pf-token.txt
749 else
750     echo " [] Rules flushed from anchor com.apple/nullexit."
751 fi
752 fi
753 }
754
755
756 write_host_ips() {
757     # Gather all active IPv4 addresses on the host and write to .host_ips
758     # routing-fix.sh will read this file to explicitly whitelist them for direct Tailscale P2P
759     local repo_dir
760     repo_dir="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
761
762     if [[ "$OSTYPE" == "darwin"* ]]; then
763         ifconfig | awk '/inet /{print $2}' | grep -v '127.0.0.1' > "$repo_dir/.host_ips" 2>/dev/null || true
764     else
765         ip -4 addr show 2>/dev/null | awk '/inet /{print $2}' | cut -d/ -f1 | grep -v '127.0.0.1' > "
766         $repo_dir/.host_ips" 2>/dev/null || true
767     fi
768 }
769
770 start_dns_watcher() {
771     if [[ "$OSTYPE" == "darwin"* ]]; then
772         local target_ip=$1
773         stop_pidfile_daemon "/tmp/nullexit-dns-watcher.pid" "background DNS Watcher"
774         echo " Starting background DNS Watcher for seamless Wi-Fi roaming..."
775         nohup bash -c "
776             source \"\$SCRIPT_DIR/scripts/common.sh\"
777             trap 'exit 0' SIGTERM SIGINT SIGHUP
778             while true; do
779                 ACTIVE_IF=$(get_active_service)
780                 if [ -n \"\$ACTIVE_IF\" ]; then
781                     CURRENT_DNS=$(networksetup -getdnsservers \"\$ACTIVE_IF\" 2>/dev/null)
782                     if [ \"\$CURRENT_DNS\" != \"\$target_ip\" ]; then
783                         networksetup -setdnsservers \"\$ACTIVE_IF\" \"\$target_ip\" >/dev/null 2>&1
784                     fi
785                 fi
786                 sleep 30
787             done
788             " >> output.log 2>&1 &
789         echo $! > "$DNS_WATCHER_PID_FILE"
790     else
791         local target_ip=$1
792         stop_dns_watcher
793         echo " Starting background DNS Watcher for seamless Wi-Fi roaming..."
794         nohup bash -c "
795             trap 'exit 0' SIGTERM SIGINT SIGHUP
796             while true; do
797                 if [ -n \"\$ACTIVE_SERVICE\" ]; then
798                     CURRENT_DNS=$(resolvectl dns \"\$ACTIVE_SERVICE\" 2>/dev/null | grep -oE
799                     '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | head -1)
800                     if [ \"\$CURRENT_DNS\" != \"\$target_ip\" ]; then
801                         resolvectl dns \"\$ACTIVE_SERVICE\" \"\$target_ip\" >/dev/null 2>&1 || true
802                     fi
803                 fi
804                 sleep 30
805             done
806             " >> output.log 2>&1 &
807         echo $! > "$DNS_WATCHER_PID_FILE"
808     fi
809 }
810
811 # SOCKS5 Proxy (WARP Tunnel)
812 # When Tailscale's exit node can't route through WARP (due to userspace
813 # forwarding), we use a SOCKS5 proxy running inside the warp container's
814 # network namespace. Connections created by this proxy go through the
815 # kernel routing table (table 200 -> tun0 -> WARP), unlike Tailscale's
816 # userspace exit node which bypasses it.

```

```

815 #
816 # The SOCKS5 proxy is exposed on localhost:${SOCKS_PROXY_PORT} via Docker port mapping.
817 # 'networksetup -setsocksfirewallproxy' on macOS routes system TCP traffic
818 # through the proxy, which then goes through WARP.
819 #
820 # IMPORTANT: CLI tools like curl do NOT respect macOS system proxy settings.
821 # They must use --socks5-hostname or the ALL_PROXY env variable explicitly.
822
823 enable_socks_proxy() {
824     if [[ "$OSTYPE" == "darwin"* ]]; then
825         local svc="$1"
826         if [ -z "$svc" ]; then
827             svc="$ACTIVE_SERVICE"
828         fi
829
830         echo -n " Enabling system-wide SOCKS5 proxy on $svc (localhost:${SOCKS_PROXY_PORT})... "
831         sudo -n networksetup -setsocksfirewallproxy "$svc" 127.0.0.1 $SOCKS_PROXY_PORT 2>> output.log || true
832         sudo -n networksetup -setsocksfirewallproxystate "$svc" on 2>> output.log || true
833
834         # Also set on en0 service for macOS resolver consistency
835         if [ "$svc" != "$ENO_SERVICE" ]; then
836             sudo -n networksetup -setsocksfirewallproxy "$ENO_SERVICE" 127.0.0.1 $SOCKS_PROXY_PORT 2>> output.
837             log || true
838             sudo -n networksetup -setsocksfirewallproxystate "$ENO_SERVICE" on 2>> output.log || true
839         fi
840
841         # IMPORTANT: Exclude local LAN and mDNS traffic from the proxy so P2P works natively
842         # without source-IP masking from the Colima VM bridge.
843         sudo -n networksetup -setproxybypassdomains "$svc" 127.0.0.1 localhost 192.168.0.0/16 10.0.0.0/8
844         172.16.0.0/12 "$*.local" 2>> output.log || true
845         if [ "$svc" != "$ENO_SERVICE" ]; then
846             sudo -n networksetup -setproxybypassdomains "$ENO_SERVICE" 127.0.0.1 localhost 192.168.0.0/16
847             10.0.0.0/8 172.16.0.0/12 "$*.local" 2>> output.log || true
848         fi
849
850         echo "done."
851         echo " All TCP traffic now routed through gateway -> WARP tunnel -> internet."
852         echo " (CLI tools: export ALL_PROXY=socks5://127.0.0.1:$SOCKS_PROXY_PORT)"
853     else
854         local svc="$1"
855         echo " (Linux global SOCKS proxy configuration is desktop-environment specific, skipping.)"
856         echo " SOCKS5 proxy is active and listening on 127.0.0.1:$SOCKS_PROXY_PORT"
857     fi
858 }
859
860 disable_socks_proxy() {
861     if [[ "$OSTYPE" == "darwin"* ]]; then
862         for svc in "$ACTIVE_SERVICE" "$ENO_SERVICE"; do
863             sudo -n networksetup -setsocksfirewallproxystate "$svc" off 2>> output.log || true
864             done
865             echo " SOCKS5 proxy disabled."
866         else
867             local svc="$1"
868             echo " (Linux global SOCKS proxy configuration skipped.)"
869         fi
870     }
871 }
872
873 # Helper to restore host DNS to a clean state (used on script start, on failure,
874 # and after a successful STOP). Without this, a successful toggle-off leaves the
875 # host's resolver pinned to a now-dead gateway IP every lookup stalls for macOS's
876 # DNS timeout (~5s) before falling through to whatever next server is configured.
877 # With it, we always leave the host in '1.1.1.1 / no search domain'.
878 #
879 # We set DNS on BOTH the active service AND the en0 service because macOS's scutil DNS
880 # resolver is commonly scoped to en0 (Wi-Fi). A per-service change on e.g.
881 # "USB 10/100 LAN" is ignored by the system resolver unless the en0 service is also updated.
882
883 reset_dns() {
884     if [[ "$OSTYPE" == "darwin"* ]]; then
885         if [ -n "$ACTIVE_SERVICE" ]; then
886             for dns_svc in "$ACTIVE_SERVICE" "$ENO_SERVICE"; do
887                 networksetup -setsearchdomains "$dns_svc" "Empty" 2>> output.log || true
888                 networksetup -setdnsservers "$dns_svc" 1.1.1.1 2>> output.log || true
889             done
890         fi
891     else
892         if [ -n "$ACTIVE_SERVICE" ]; then
893             resolvectl domain "$ACTIVE_SERVICE" "" 2>/dev/null || true
894             resolvectl revert "$ACTIVE_SERVICE" 2>/dev/null || true
895         fi
896     fi
897 }

```

```

893
894 # Verify basic direct-internet connectivity after a recovery/teardown: DNS via
895 # 1.1.1.1, then HTTP (curl) or ping to 1.1.1.1. Sets the global INTERNET_OK to
896 # "true" on any successful check (callers read $INTERNET_OK in their summary).
897 # Always returns 0 so a bare call can't trip 'set -e'. Used by recover.sh's macOS
898 # and Linux recovery paths (extracted from a formerly-duplicated block).
899 verify_internet_connectivity() {
900     step "Verifying internet connectivity"
901
902     # Wait a moment for the network to settle
903     sleep 2
904
905     INTERNET_OK=false
906
907     # Try DNS resolution first
908     if host -W 3 google.com 1.1.1.1 >> output.log 2>&1; then
909         ok "DNS resolution works (google.com via 1.1.1.1)"
910         INTERNET_OK=true
911     elif nslookup google.com 1.1.1.1 >> output.log 2>&1; then
912         ok "DNS resolution works (nslookup google.com via 1.1.1.1)"
913         INTERNET_OK=true
914     else
915         warn "DNS resolution check failed will still try ping/curl"
916     fi
917
918     # Try actual HTTP connectivity
919     if command -v curl >> output.log 2>&1; then
920         if curl -sf --max-time 5 https://1.1.1.1 >> output.log 2>&1; then
921             ok "Internet reachable via HTTP"
922             INTERNET_OK=true
923         elif curl -sf --max-time 5 http://1.1.1.1 >> output.log 2>&1; then
924             ok "Internet reachable via HTTP (plain)"
925             INTERNET_OK=true
926         else
927             warn "HTTP check failed"
928         fi
929     elif command -v ping >> output.log 2>&1; then
930         if ping -c 1 -W 3 1.1.1.1 >> output.log 2>&1; then
931             ok "Internet reachable via ping"
932             INTERNET_OK=true
933         else
934             warn "Ping check failed"
935         fi
936     fi
937
938     return 0
939 }
940
941 # Local DNS Proxy (Python)
942 # When the tailnet data plane cannot establish, we use a Python DNS proxy.
943 # It listens on UDP:53, receives DNS queries, forwards them over TCP to
944 # AdGuard at 127.0.0.1:${DNS_PROXY_PORT} (Docker port mapping), and returns the response.
945 # The Python script handles the DNS-over-TCP wire format (2-byte length prefix)
946 # correctly unlike socat which just forwards raw bytes.
947 #
948 # Python3 is built into macOS no external dependencies.
949 DNS_PROXY_PID=""
950
951 # Kill any leftover DNS proxy process
952 stop_local_dns_proxy() {
953     if [ -n "$DNS_PROXY_PID" ]; then
954         kill "$DNS_PROXY_PID" 2>/dev/null || true
955         wait "$DNS_PROXY_PID" 2>/dev/null || true
956         DNS_PROXY_PID=""
957     fi
958     # Clean up any stale Python DNS proxy processes
959     sudo -n pkill -f "dns-proxy.py" 2>/dev/null || true
960 }
961
962 # Start local DNS proxy (Python)
963 start_local_dns_proxy() {
964     if [ -z "$DNS_PROXY_BIN" ]; then
965         echo " Python not found. Python3 is built into macOS this shouldn't happen."
966         return 1
967     fi
968
969     if [ ! -f "$DNS_PROXY_SCRIPT" ]; then
970         echo " DNS proxy script not found at $DNS_PROXY_SCRIPT"
971         return 1
972     fi
973

```



```

974 # Kill any leftover process first
975 stop_local_dns_proxy
976
977 echo -n " Starting local DNS proxy via Python (UDP:53 TCP:${DNS_PROXY_PORT})... "
978 sudo -n bash -c "TARGET_PORT=${DNS_PROXY_PORT} \"${DNS_PROXY_BIN}\" \"${DNS_PROXY_SCRIPT}\" &
979 DNS_PROXY_PID=$!
980 disown "${DNS_PROXY_PID}" 2>/dev/null || true
981
982 # Give it a moment to bind
983 sleep 0.5
984
985 if kill -0 "${DNS_PROXY_PID}" 2>/dev/null; then
986     echo "started (PID ${DNS_PROXY_PID})."
987
988     # Hijack host DNS to localhost
989     echo -n " Hijacking host DNS to 127.0.0.1 for ad-blocking... "
990     if [[ "$OSTYPE" == "darwin"* ]]; then
991         networksetup -setsearchdomains "$ACTIVE_SERVICE" "Empty" 2>/dev/null || true
992         networksetup -setsearchdomains "$ENO_SERVICE" "Empty" 2>/dev/null || true
993         if ! networksetup -setdnsservers "$ACTIVE_SERVICE" 127.0.0.1; then
994             echo "FAILED (networksetup error check permissions)."

```

```

1054 # Apply to BOTH the active service AND the en0 service the scutil DNS resolver
1055 # is scoped to en0 (Wi-Fi) and ignores per-service changes that don't include it.
1056 # Fail fast on ACTIVE_SERVICE; best-effort on ENO_SERVICE (it may not exist).
1057 networksetup -setsearchdomains "$ACTIVE_SERVICE" "ts.net" || true
1058 if ! networksetup -setdnsservers "$ACTIVE_SERVICE" "$target_ip"; then
1059     echo "FAILED (networksetup error check permissions)"
1060     sleep 1
1061     continue
1062 fi
1063 networksetup -setsearchdomains "$ENO_SERVICE" "ts.net" || true
1064 networksetup -setdnsservers "$ENO_SERVICE" "$target_ip" || true
1065
1066 local entries
1067 entries=$(networksetup -getdnsservers "$ACTIVE_SERVICE" 2>> output.log \
1068     | awk '/^(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\. (25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$/ {print}')
1069 if echo "$entries" | grep -qFx "$target_ip"; then
1070     echo "VERIFIED"
1071     return 0
1072 fi
1073
1074 local current
1075 current=$(echo "$entries" | tr '\n' ' ' | sed 's/ $//')
1076 echo "NOT YET (current DNS: [${current:-none}])"
1077
1078 if [ "$attempt" = "3" ]; then sleep 2; else sleep 1; fi
1079 done
1080
1081 echo " [force_dns] FAILED after 3 attempts"
1082 return 1
1083 else
1084     local target_ip="$1"
1085
1086     for attempt in {1..3}; do
1087         echo -n " [force_dns] Attempt $attempt/3: setting DNS to $target_ip on $ACTIVE_SERVICE... "
1088         sudo -n resolvectl domain "$ACTIVE_SERVICE" "~ts.net" 2>/dev/null || true
1089         if ! sudo -n resolvectl dns "$ACTIVE_SERVICE" "$target_ip"; then
1090             echo "FAILED (resolvectl error)"
1091             else
1092                 # Verify
1093                 entries=$(resolvectl dns "$ACTIVE_SERVICE" 2>> output.log | grep -oE '
1094 [0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' )
1095                 if [ "$entries" == "$target_ip" ]; then
1096                     echo "VERIFIED"
1097                     return 0
1098                 else
1099                     echo "MISMATCH (Resolver refused lock)"
1100                 fi
1101             fi
1102             sleep 2
1103         done
1104     return 1
1105 fi
1106 }

```

12 scripts/watcher.sh

```

1 #!/bin/bash
2 # scripts/watcher.sh nullexit post-wake / post-roam recovery watcher
3 #
4 # Long-running daemon. Launched by launchd as a LaunchAgent at
5 # ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
6 # (label com.nullexit.wake-recovery).
7 #
8 # Two independent listeners run as backgrounded pipelines:
9 #
10 # (1) WAKE listener 'log stream' live-follows the macOS unified log for
11 # power-management events (lid closewake, manual wake, DarkWake from
12 # clamshell). Each event triggers run_recover (with a global debounce so
13 # a single wake that emits 2-3 log lines only causes ONE post-wake run).
14 #
15 # (2) NETWORK listener 'scutil n.watch' on 'State:/Network/Global/IPv4'
16 # fires on every Wi-Fi roam, ethernet swap, hotspot join, captive-portal
17 # re-bind, VPN up/down, etc. Same debounce.
18 #
19 # Both listeners call run_recover, which:

```

```

20 # - checks /tmp/nullexit-gateway-active.marker (only act when toggle.sh left
21 #   the gateway up)
22 # - debounces (10s default) so multiple events don't pile up
23 # - shells out to 'bash <repo>/recover.sh --post-wake' directly (no GUI auth prompt needed due to
    NOPASSWD sudoers)
24 #
25 # See devref.md 10.29 for the full Apple power management primer and the
26 # rationale behind using 'log stream' + 'scutil n.watch' from a shell daemon.
27
28 # NOTE: errexit ('set -e') is intentionally NOT enabled. This is a long-running
29 # daemon, not a discrete-step script, and errexit actively breaks the reconnect
30 # loops below: any failed pipe (log stream on rotation, scutil on state churn)
31 # would kill the outer subshell before 'echo "reconnecting..."; sleep 5;' runs,
32 # AND the parent's final 'wait' would propagate a non-zero child exit and kill
33 # the entire watcher. Every error path is already wrapped in '|| true' /
34 # '2>/dev/null' fallbacks.
35 SCRIPT_DIR="$(cd "$(dirname "$BASH_SOURCE[0]")" && pwd)"
36 source "$SCRIPT_DIR/common.sh"
37 setup_standard_path
38
39 LOG="$SCRIPT_DIR/./output.log"
40 # Resolve our own directory; recover.sh lives one level up at the repo root.
41 # This makes the daemon install-agnostic no need to template the path
42 # in launchd, no need to keep a deploy-specific hardcoded location in sync.
43 RECOVER="$SCRIPT_DIR/./recover.sh"
44 MARKER="$MARKER_FILE"
45 DEBOUNCE_FILE="/tmp/nullexit-watcher.last-recovery"
46 DEBOUNCE_SECONDS="${NULLEXIT_DEBOUNCE_SECONDS:-10}"
47 # Post-recovery SETTLE cooldown. A post-wake run re-asserts the exit node
48 # (recover.sh: 'tailscale set --exit-node'), which re-writes the host default
49 # route and that route IS State:/Network/Global/IPv4, the exact key Listener 2
50 # watches. The self-triggered change lands ~15-45s AFTER the run finishes, past
51 # the 10s debounce, so without a longer window the watcher re-fires post-wake in
52 # a ~40s self-feedback loop that only stops once routing happens to reach a fixed
53 # point (devref 15.12.21). This cooldown is armed ONLY after a recovery runs, so
54 # a genuine first wake/roam is still handled immediately; only the change we
55 # caused ourselves gets absorbed. A real new roam during the window is delayed at
56 # most this many seconds, which is acceptable.
57 COOLDOWN_FILE="/tmp/nullexit-watcher.settle-until"
58 POSTWAKE_SETTLE_SECONDS="${NULLEXIT_POSTWAKE_SETTLE_SECONDS:-45}"
59
60 exec >> "$LOG" 2>&1
61
62 # Single-instance lock
63 # Without this, repeated lid-closewake cycles under macOS Sonoma+ accumulate
64 # orphan watcher.sh processes: launchd suspends on sleep and SIGCONT-resumes
65 # on wake, and KeepAlive.Crashed=true caused relaunch on any non-zero exit;
66 # the listener grandchildren (log stream | while ... and (echo n.add ...;
67 # sleep 86400) | scutil | while ...) stay alive because pipe producers don't
68 # reliably respond to plain SIGTERM if the consumer is in a half-closed state.
69 #
70 # We solve it with a PID-file lock (POSIX-portable; macOS does NOT ship 'flock').
71 # The trap below removes the lock file on every exit so a stale lock never
72 # outlives a crashed watcher. On launch: read the existing PID, check it is
73 # alive AND is actually a 'scripts/watcher.sh' process; if so, exit 0; else
74 # overwrite with our PID and proceed. The check-then-write window has a TOCTOU
75 # race but is microseconds wide and only matters for hand-launched duplicate
76 # test invocations; production is single-launch via launchctl load -w.
77 LOCK_FILE="/tmp/nullexit-watcher.lock"
78 LOCK_PID=""
79 if [ -e "$LOCK_FILE" ]; then
80     LOCK_PID=$(cat "$LOCK_FILE" 2>/dev/null || echo "")
81     if [ -n "$LOCK_PID" ] && kill -0 "$LOCK_PID" 2>/dev/null; then
82         # Verify the holder is actually a scripts/watcher.sh process (not some
83         # unrelated process that got the recycled PID). The exact-launchd pattern
84         # 'bash <abs-path>/scripts/watcher.sh' is hard to accidentally match with
85         # a 'grep scripts/watcher.sh .' invocation because we anchor on '^bash'
86         # followed by whitespace and end-of-line on the script path.
87         if ps -p "$LOCK_PID" -o args= 2>/dev/null | grep -qE "([[:space:]]bash[[:space:]]+.*scripts/
            watcher\.sh$"; then
88             echo "[$(date -u +%FT%TZ)] watcher.sh: another instance holds $LOCK_FILE (pid=$LOCK_PID), exiting"
89             exit 0
90         fi
91     fi
92 fi
93 echo $$ > "$LOCK_FILE"
94
95 echo "[$(date -u +%FT%TZ)] watcher.sh start (pid=$$ ppid=$PPID user=$(id -un) lock=acquired)"
96
97 # Treat launchd-resume as a wake signal. Apple's launchd SUSPENDS LaunchAgents
98 # during system sleep and restores them on wake; during that suspended gap,

```

```

99 # macOS's 'log stream' cannot catch the wake event that prompted the resume.
100 # The next thing that happens after wake IS our own startup, so we always
101 # fire one post-wake run unconditionally here. The marker check + debounce
102 # in run_recover() make this idempotent: if the gateway isn't up or we just
103 # debounced, it no-ops. Without this, the FIRST wake-after-install goes
104 # unseen and the watcher appears broken until the SECOND wake.
105 # Helper: run recover.sh --post-wake if the gateway is active
106 run_recover() {
107     local why="$1"
108     if [ ! -f "$MARKER" ]; then
109         echo "[$(date -u +%FT%TZ)] $why no $MARKER, skip"
110         return 0
111     fi
112     local now last settle_until
113     now=$(date +%s)
114     # Post-recovery settle window: ignore the Global/IPv4 change our own last
115     # post-wake run caused (exit-node re-assert) so we don't self-feedback loop.
116     settle_until=$(cat "$COOLDOWN_FILE" 2>/dev/null || echo 0)
117     if [ "$now" -lt "$settle_until" ]; then
118         echo "[$(date -u +%FT%TZ)] $why settling ($(settle_until - now)s left in post-recovery cooldown),
119             skip"
120         return 0
121     fi
122     last=$(cat "$DEBOUNCE_FILE" 2>/dev/null || echo 0)
123     if [ "$((now - last))" -lt "$DEBOUNCE_SECONDS" ]; then
124         echo "[$(date -u +%FT%TZ)] $why debounced ($(now - last)s since last, threshold ${DEBOUNCE_SECONDS}
125             s)"
126         return 0
127     fi
128     echo "$now" > "$DEBOUNCE_FILE"
129     echo "[$(date -u +%FT%TZ)] $why bash $RECOVER --post-wake"
130     bash "$RECOVER" --post-wake
131     local exit_code=$?
132     now=$(date +%s)
133     echo "$now" > "$DEBOUNCE_FILE"
134     # Arm the settle cooldown so the route change this run just caused can't re-fire us.
135     echo "$((now + POSTWAKE_SETTLE_SECONDS))" > "$COOLDOWN_FILE"
136     echo "[$(date -u +%FT%TZ)] $why exit=$exit_code (settle until ${POSTWAKE_SETTLE_SECONDS}s, now=$now)"
137     return $exit_code
138 }
139 if [ -f "$MARKER" ]; then
140     run_recover "initial post-wake (launchd resume = wake signal)"
141 fi
142 # LAN P2P Auto-Detection
143 # Detects whether the current network allows safe Tailscale LAN P2P by:
144 # 1. Checking WPA2-Enterprise (802.1x) via system_profiler SPAirPortDataType always forces P2P=false
145 # because enterprise networks almost universally have AP Client Isolation.
146 # 2. AP Isolation probe sends a broadcast ping and checks if ANY LAN host responds.
147 # If no LAN hosts respond on a non-empty network, AP Isolation is active.
148 # Writes 'true' or 'false' to .lan_p2p_detected in the repo root.
149 # routing-fix.sh reads this file dynamically every 30s loop iteration.
150 P2P_OVERRIDE_FILE="$SCRIPT_DIR/../../lan_p2p_detected"
151
152 detect_lan_p2p_mode() {
153     local reason="unknown" allow="false"
154
155     # Step 1: Detect Wi-Fi security type via system_profiler (airport binary removed in macOS 14.4+)
156     # SPAirPortDataType lists all known networks; the first "Security:" line is the current association.
157     local security
158     security=$(system_profiler SPAirPortDataType 2>/dev/null \
159         | awk '/Current Network Information:/{found=1} found && /Security:/{print $NF; exit}')
160
161     if [ -z "$security" ]; then
162         # Not on Wi-Fi or system_profiler unavailable fail safe
163         reason="no-wifi" allow="false"
164     elif echo "$security" | grep -qi "Enterprise"; then
165         # WPA2/WPA3 Enterprise = 802.1x always AP isolated
166         reason="802.1x-enterprise" allow="false"
167     elif echo "$security" | grep -qi "Personal\|None\|Open"; then
168         # WPA2/WPA3 Personal or open probe for AP isolation
169         local gw
170         gw=$(route -n get 1.1.1.1 2>/dev/null | awk '/gateway:/{print $2}')
171         if [ -z "$gw" ]; then
172             reason="no-default-route" allow="false"
173         else
174             local responses
175             responses=$(ping -c 3 -t 1 -q "$gw" 2>/dev/null | awk '/packets transmitted/{print $4}')
176             if [ "${responses:-0}" -gt 0 ]; then

```

```

178     reason="wpa-personal-lan-reachable" allow="true"
179 else
180     reason="wpa-personal-ap-isolated" allow="false"
181 fi
182 fi
183 else
184     # Unknown security type fail safe
185     reason="unknown-security-type" allow="false"
186 fi
187
188 echo "[$(date -u +%FT%TZ)] P2P detect: security='${security:-none}' allow=${allow} (reason: ${reason})"
189
190 echo "$allow" > "$P2P_OVERRIDE_FILE"
191 write_host_ips
192 }
193
194 # Run once on startup
195 detect_lan_p2p_mode
196
197 # Listener 1: WAKE events via unified log
198 # Predicate picks up:
199 # * "didWake"           regular kernel wake events (lid open, RTC alarm)
200 # * "Wake from Sleep"   powerd-driven normal wake
201 # * "DarkWake"          clamshell-display wake that didn't fully wake the
202 #                       Mac (relevant when lid opens during display sleep)
203 # * "Waking from"       generic catch-all for variations in newer macOS
204 # '[c]' makes the match case-insensitive (the unified log uses mixed-case
205 # phrases). --info reduces noise by dropping debug/trace log levels.
206 # Subsystem-based predicate catches every powermanagement emission (willSleep,
207 # didWake, didDim, didUndim, DarkWake, etc.) regardless of message phrasing
208 # across kernel versions. Phrasing-based predicates ('didWake', 'Wake from Sleep')
209 # are fragile across macOS releases. We accept the extra log noise because the
210 # run_recover() debounce + marker check filter out anything that isn't a real
211 # gateway-relevant event.
212 (
213     # Reconnect loop: macOS rotates the unified log buffer periodically; when
214     # that happens, this 'log stream' subscriber's pipe EOFs, the inner
215     # 'while read' consumer terminates, and without this wrapper the listener
216     # would be silently dead for the rest of the watcher's lifetime.
217     while ;; do
218         log stream --predicate \
219             'subsystem == "com.apple.powermanagement"' \
220             --style compact --info 2>/dev/null \
221             | while IFS= read -r line; do
222                 [ -z "$line" ] && continue
223                 run_recover "WAKE: $(echo "$line" | head -c 100)"
224             done
225         echo "[$(date -u +%FT%TZ)] log stream subshell exited; reconnecting in 5s"
226         sleep 5
227     done
228 ) &
229
230 # Listener 2: NETWORK state changes via scutil
231 # 'scutil n.watch' on a state key prints SCEventUpdate lines whenever the key
232 # changes. We watch Global/IPv4 because that's the umbrella that catches link,
233 # address, route, and DNS changes for ALL interfaces in one namespace.
234 # The pipe is kept alive by an 'sleep 86400' loop so scutil never sees EOF
235 # from us (which would silently terminate the watch).
236 (
237     # Reconnect loop: scutil can exit noisily on certain state changes (rare,
238     # but observed). Without the wrapper, the network listener's pipe EOFs and
239     # every subsequent roam goes unseen until launchd restarts us.
240     while ;; do
241         (
242             # Watch both IPv4 and IPv6 umbrella state IPv6-only or dual-stack
243             # networks never fire on the IPv4 key alone, and the user's first roam
244             # in a hotel / airport / on cellular hotspot is often IPv6-first under
245             # NAT64.
246             echo "n.add State:/Network/Global/IPv4"
247             echo "n.add State:/Network/Global/IPv6"
248             echo "n.watch"
249             while true; do sleep 86400; done
250         ) | script -q /dev/null scutil 2>/dev/null \
251         | while IFS= read -r line; do
252             case "$line" in
253                 *changedKey*|*State:/Network/Global/IPv*|*n.state*|*SCEventUpdate*)
254                     detect_lan_p2p_mode
255                     run_recover "NET: $(echo "$line" | head -c 100)"
256                 ;;
257                 *) ;;
258             esac
259         done
260     done
261 ) &

```

```

258         done
259         echo "[$(date -u +%FT%TZ)] scutil pipe exited; reconnecting in 5s"
260         sleep 5
261     done
262 ) &
263 # Listener 3: TUNNEL_FAILED_CLOSED.marker Watcher
264 # Polls for the fail-closed marker dropped by the routing-fix container.
265 (
266     last_notified=0
267     while ;; do
268         if [ -f "$SCRIPT_DIR/../TUNNEL_FAILED_CLOSED.marker" ]; then
269             now=$(date +%s)
270             # Notify once every 5 minutes (300s) if it stays failed
271             if [ "$(now - last_notified)" -ge 300 ]; then
272                 osascript -e 'display notification "VPN Tunnel Health Check Failed. Internet traffic is
blackholed to prevent IP leak." with title "NullExit Alert"' || true
273                 last_notified=$now
274             fi
275         else
276             last_notified=0
277         fi
278         sleep 3
279     done
280 ) &
281
282 # Lifetime
283 # 'wait' blocks until both background pipelines exit (which they normally
284 # never do). launchctl 'bootout' /SIGTERM triggers the trap, which kills the
285 # whole process group so the sleep-forever in the scutil pipe also dies.
286 # We catch TERM/INT/HUP for explicit signals and EXIT for any normal-exit path
287 # (so listeners always get cleaned up even if 'exit 0' runs somewhere).
288 # 'rm -f $LOCK_FILE' first so a stale lock can never block the next launch.
289 # 'pkill -TERM -P $$' then targets direct listener-children, then 'kill 0'
290 # takes care of any backgrounded group-mates not reachable via the parent.
291 trap 'echo "[$(date -u +%FT%TZ)] watcher.sh terminating (signal/exit=$?)"; rm -f "$LOCK_FILE" 2>/dev/null
|| true; pkill -TERM -P $$ 2>/dev/null || true; kill 0 2>/dev/null || true; exit 0' TERM INT HUP
EXIT
292 wait

```

13 scripts/crypto.sh

```

1  #!/usr/bin/env bash
2  # crypto.sh - Cryptographic Integrity Check
3
4  set -euo pipefail
5  cd "$(dirname "$0")/.."
6
7  if [ "$#" -ne 1 ] || { [ "$1" != "--sign" ] && [ "$1" != "--verify" ]; }; then
8      echo "Usage: $0 --sign | --verify"
9      exit 1
10 fi
11
12 if [ ! -f .env ]; then
13     echo "Error: .env not found."
14     exit 1
15 fi
16
17 SEED=$(grep "^NULLEXIT_SEED=" .env | cut -d '=' -f2)
18 if [ -z "$SEED" ]; then
19     echo "Error: NULLEXIT_SEED not found in .env."
20     exit 1
21 fi
22
23 if [ "$1" == "--sign" ]; then
24     echo "Generating cryptographic signatures using seed..."
25     rm -f .signatures
26     for file in toggle.sh recover.sh scripts/common.sh scripts/routing-fix.sh scripts/diagnose-host-leak.sh
scripts/sweep.sh scripts/noise.sh scripts/watcher.sh scripts/pf.conf post-rules.txt docker-compose.
yaml black_list.txt white_list.txt; do
27         if [ -f "$file" ]; then
28             hash=$(openssl dgst -sha256 -hmac "$SEED" "$file" | awk '{print $NF}')
29             echo "$file:$hash" >> .signatures
30             echo " [Signed] $file"
31         else
32             echo " [Skipped] $file not found"
33         fi

```

```

34 done
35 echo "Signatures saved to .signatures"
36 exit 0
37 fi
38
39 if [ "$1" == "--verify" ]; then
40     if [ ! -f .signatures ]; then
41         echo "Error: .signatures file missing. Run $0 --sign first."
42         exit 1
43     fi
44     FAIL=0
45     while IFS=: read -r file expected_hash; do
46         if [ -z "$file" ]; then continue; fi
47         if [ ! -f "$file" ]; then
48             echo "[FAIL] $file is missing!"
49             FAIL=1
50             continue
51         fi
52         actual_hash=$(openssl dgst -sha256 -hmac "$SEED" "$file" | awk '{print $NF}')
53         if [ "$actual_hash" != "$expected_hash" ]; then
54             echo "[FAIL] $file has been modified! Hash mismatch."
55             FAIL=1
56         fi
57     done < .signatures
58
59     if [ "$FAIL" -eq 1 ]; then
60         echo ""
61         echo "CRITICAL: Script integrity verification failed!"
62         echo "One or more core files have been modified without authorization."
63         echo "To authorize these changes, run: ./scripts/crypto.sh --sign"
64         echo ""
65         exit 1
66     fi
67     exit 0
68 fi

```

14 .aiignore

```

1 # Environment variables and secrets
2 .env
3 tor-bridges.txt

```

15 .claude/settings.local.json

```

1 {
2   "permissions": {
3     "allow": [
4       "Bash(grep *)",
5       "Bash(python3 /tmp/gen_nullexit_graph.py)",
6       "Bash(python3 -)",
7       "Bash(\"/Applications/Google Chrome.app/Contents/MacOS/Google Chrome\" --headless=new --disable-gpu
8       --window-size=1280,800 --virtual-time-budget=8000 --dump-dom \"file:///tmp/kg_debug.html\")",
9       "Bash(echo \"dangling-check-exit=$?\")"
10    ]
11  }
12 }

```

16 .gitignore

```

1 # Environment variables and secrets
2 .env
3
4 # wgcf generated files containing keys and account info
5 wgcf-account.toml
6 wgcf-profile.conf
7
8 # Tailscale persistent state containing node identity
9 tailscale-state/

```



```

10
11 # Binaries
12 wgcfl
13
14 # macOS App bundles
15 *.app/
16 *.app
17
18 # AdGuard Home persistent state
19 adguard/
20 .gateway_ip
21 logs.txt
22
23 #
24 stability_incident_report.md
25 output.log
26 .DS_Store
27 *.desktop
28 blocked.log
29 wtf.md
30 host-leak-diagnostic-*.txt
31 nullexit_review.md
32 nullexit_unified.tex
33 nullexit_unified.txt
34
35 nullexit_unified.synctex.gz
36 tor-bridges.txt
37
38 # Runtime state files
39 .host_ips
40 .lan_p2p_detected
41 .build_hash
42 .rules_hash
43 .signatures
44 TUNNEL_FAILED_CLOSED.marker
45
46 # Python cache
47 __pycache__/_
48 *.py[cod]
49
50 # LaTeX build artifacts
51 *.aux
52 *.log
53 *.out
54 *.toc
55 *.xdv
56 *.fdb_latexmk
57 *.fls
58 *.synctex*
59 refactor.md
60 sweep.md
61
62 # Obsidian Knowledge graph
63 nullexit-knowledge-graph/
64
65 # DNS anomaly detector per-network model (regenerated by 'learn')
66 .dns_baseline.json

```

17 Recover-Gateway.applescript

```

1 tell application "Finder" to set scriptDir to POSIX path of (container of (path to me) as alias)
2 try
3     do shell script "true" with administrator privileges
4 on error
5     return
6 end try
7 tell application "Terminal"
8     activate
9     do script "cd " & quoted form of scriptDir & "; bash recover.sh"
10 end tell

```

18 Sweep-Gateway.applescript

```

1 tell application "Finder" to set scriptDir to POSIX path of (container of (path to me) as alias)
2 try
3     do shell script "cd " & quoted form of scriptDir & " && ./scripts/crypto.sh --verify"
4 on error errMsg
5     display dialog "nullexit integrity check FAILED sweep will not run." & return & return & "A signed
        file (scripts/sweep.sh, scripts/common.sh, ) no longer matches .signatures. If you edited it on
        purpose, re-sign with:" & return & "./scripts/crypto.sh --sign" & return & return & errMsg buttons {
        "Abort"} default button "Abort" with icon stop
6     return
7 end try
8 tell application "Terminal"
9     activate
10    do script "cd " & quoted form of scriptDir & " && bash scripts/sweep.sh"
11 end tell

```

19 Toggle-Gateway.applescript

```

1 tell application "Finder" to set scriptDir to POSIX path of (container of (path to me) as alias)
2 try
3     do shell script "true" with administrator privileges
4 on error
5     return
6 end try
7 tell application "Terminal"
8     activate
9     do script "cd " & quoted form of scriptDir & "; ./toggle.sh"
10 end tell

```

20 benchmarks/benchmark.sh

```

1 #!/usr/bin/env bash
2
3 # benchmark.sh
4 # Automates the full benchmarking suite (both Python download test and Lighthouse HTML test)
5 # Runs the suite with Gateway ON, toggles OFF, runs again, then toggles back ON.
6
7 set -e
8
9 # Change to project root so we can access toggle.sh reliably
10 cd "$(dirname "$0")/.."
11
12 # Ensure jq is installed
13 if ! command -v jq &> /dev/null; then
14     echo "Error: 'jq' is not installed. Please run 'brew install jq'"
15     exit 1
16 fi
17
18 run_lighthouse() {
19     local url=$1
20     local file_prefix=$2
21     echo " [Lighthouse] Testing $url ..."
22
23     # Run lighthouse headlessly, silencing standard output
24     npx -y lighthouse "$url" --chrome-flags="--headless" --only-categories=performance --output=json --
        output-path="${file_prefix}.json" >/dev/null 2>&1
25
26     # Parse the results
27     local score=$(jq -r '.categories.performance.score' "${file_prefix}.json")
28     local tti=$(jq -r '.audits["interactive"].displayValue' "${file_prefix}.json")
29     local si=$(jq -r '.audits["speed-index"].displayValue' "${file_prefix}.json")
30
31     # Convert score to percentage
32     local score_pct=$(awk "BEGIN {print $score*100}")
33
34     echo "    -> Performance Score: ${score_pct}%"
35     echo "    -> Time to Interactive: $tti"
36     echo "    -> Speed Index: $si"
37
38     # Cleanup
39     rm -f "${file_prefix}.json"
40 }

```

```

41
42 echo "===== "
43 echo "    PHASE 1: Testing with Gateway ON"
44 echo "===== "
45 # 1. Run raw Python download test
46 python3 benchmarks/test_load.py
47
48 # 2. Run Lighthouse HTML render test
49 echo ""
50 echo "Starting Lighthouse HTML Tests (this takes ~30-60s per site)..."
51 run_lighthouse "https://www.cnet.com/" "on_cnet"
52 run_lighthouse "https://www.independent.co.uk/" "on_ind"
53
54
55 echo ""
56 echo "===== "
57 echo "    PHASE 2: Toggling Gateway OFF"
58 echo "===== "
59 ./toggle.sh
60 echo "Waiting 10 seconds for macOS Wi-Fi to stabilize after DNS reset..."
61 sleep 10
62
63
64 echo ""
65 echo "===== "
66 echo "    PHASE 3: Testing with Gateway OFF"
67 echo "===== "
68 # 1. Run raw Python download test
69 python3 benchmarks/test_load.py
70
71 # 2. Run Lighthouse HTML render test
72 echo ""
73 echo "Starting Lighthouse HTML Tests (this takes ~30-60s per site)..."
74 run_lighthouse "https://www.cnet.com/" "off_cnet"
75 run_lighthouse "https://www.independent.co.uk/" "off_ind"
76
77
78 echo ""
79 echo "===== "
80 echo "    PHASE 4: Restoring Gateway ON"
81 echo "===== "
82 ./toggle.sh
83
84 echo ""
85 echo "===== "
86 echo " BENCHMARKS COMPLETE!"
87 echo "You can scroll up to compare Phase 1 (AdGuard ON) vs Phase 3 (AdGuard OFF)."
88 echo "===== "

```

21 benchmarks/test_load.py

```

1 import urllib.request
2 import urllib.error
3 import time
4 from html.parser import HTMLParser
5 import concurrent.futures
6
7 class ResourceParser(HTMLParser):
8     def __init__(self):
9         super().__init__()
10        self.urls = set()
11
12    def handle_starttag(self, tag, attrs):
13        attrs = dict(attrs)
14        url = None
15        if tag in ['img', 'script']:
16            url = attrs.get('src')
17        elif tag == 'link' and attrs.get('rel') == 'stylesheet':
18            url = attrs.get('href')
19
20        if url and url.startswith('http'):
21            self.urls.add(url)
22
23    def fetch_url(url):
24        try:
25            req = urllib.request.Request(url, headers={'User-Agent': 'Mozilla/5.0'})

```

```

26     urllib.request.urlopen(req, timeout=3)
27     return True
28 except Exception:
29     return False
30
31 def measure_url(target_url):
32     print(f"\nTesting {target_url} Load...")
33     start_total = time.time()
34
35     # Fetch main HTML
36     try:
37         req = urllib.request.Request(target_url, headers={'User-Agent': 'Mozilla/5.0 (Macintosh; Intel
38 Mac OS X 10_15_7) AppleWebKit/537.36'})
39         response = urllib.request.urlopen(req, timeout=5)
40         html = response.read().decode('utf-8', errors='ignore')
41     except Exception as e:
42         print(f"Failed to fetch {target_url}: {e}")
43         return
44
45     parser = ResourceParser()
46     parser.feed(html)
47     urls = list(parser.urls)
48     print(f"Found {len(urls)} subresources to fetch (scripts, images, css)...")
49
50     # Concurrently fetch subresources
51     success_count = 0
52     fail_count = 0
53     max_threads = min(64, max(1, len(urls)))
54     with concurrent.futures.ThreadPoolExecutor(max_workers=max_threads) as executor:
55         results = list(executor.map(fetch_url, urls))
56
57     success_count = sum(1 for r in results if r)
58     fail_count = len(results) - success_count
59
60     total_time = time.time() - start_total
61     print(f"Time taken: {total_time:.2f} seconds")
62     print(f"Successfully fetched: {success_count}, Failed/Blocked: {fail_count}")
63
64 urls_to_test = [
65     "https://www.dailymail.co.uk/home/index.html",
66     "https://www.cnet.com/",
67     "https://www.independent.co.uk/"
68 ]
69
70 for u in urls_to_test:
71     measure_url(u)

```

22 black_list.txt

```

1 ad.doubleclick.net
2 adclick.g.doubleclick.net
3 adeventtracker.spotify.com
4 adfstat.yandex.ru
5 ads-api.tiktok.com
6 ads-api.twitter.com
7 ads-fa.spotify.com
8 ads-sg.tiktok.com
9 ads.tiktok.com
10 adserver.unityads.unity3d.com
11 adsfs.oppomobile.com
12 an.facebook.com$important
13 connect.facebook.net$important
14 pixel.facebook.com$important
15 analytics.query.yahoo.com
16 analytics.s3.amazonaws.com
17 analytics.spotify.com
18 analyticsengine.s3.amazonaws.com
19 appmetrica.yandex.ru
20 audio2.spotify.com
21 bdapi-ads.realmemobile.com
22 bdapi-in-ads.realmemobile.com
23 bounceexchange.com
24 bs.serving-sys.com
25 business-api.tiktok.com
26 ck.ads.oppomobile.com
27 crashdump.spotify.com

```

```

28 data.ads.oppomobile.com
29 data.mistat.india.xiaomi.com
30 data.mistat.rus.xiaomi.com
31 data.mistat.xiaomi.com
32 desktop.spotify.com
33 doubleclick.net
34 ds.metric.spotify.com
35 gemini.yahoo.com
36 googleads.g.doubleclick.net
37 googleadservices.com
38 googlesyndication.com
39 grs.hicloud.com
40 gtm.spotify.com
41 iot-eu-logser.realme.com
42 iot-logser.realme.com
43 log.byteoversea.com
44 log.fc.yahoo.com
45 log.spotify.com
46 m.doubleclick.net
47 mediavisor.doubleclick.net
48 metrics.spotify.com
49 metrika.yandex.ru
50 omaze.com
51 pagead2.googlesyndication.com
52 securepubads.g.doubleclick.net
53 static.doubleclick.net
54 stats.g.doubleclick.net
55 tracking.rus.miui.com
56 udcms.yahoo.com
57 v.jwpcdn.com
58 webblb-wg.gslb.spotify.com
59 webview.unityads.unity3d.com
60 diagassets.apple.com
61 iphonesubmissions.apple.com
62 radarsubmissions.apple.com
63 metrics.apple.com
64 xp.apple.com
65 heads-fa.spotify.com
66 heads4.spotify.com
67 pixel.spotify.com
68 segs.spotify.com
69 audio-fa.spotify.com
70 events.data.microsoft.com
71 datarouter-prod.ak.epicgames.com
72 nel.cloudflare.com
73 eu-mobile.events.data.microsoft.com
74 stocks-analytics-events.apple.com

```

23 cloud-init.yaml

```

1 #cloud-config
2 # Generic Cloud-Init Bootstrap Script for Remote Exit Nodes (Ubuntu/Debian)
3 # Paste this into the "User Data" or "Initialization Script" section when creating a VPS.
4
5 package_update: true
6 package_upgrade: true
7
8 packages:
9   - apt-transport-https
10  - ca-certificates
11  - curl
12  - gnupg
13  - lsb-release
14  - git
15  - python3
16  - python3-pip
17
18 runcmd:
19   # 1. Install Docker natively
20   - curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/
      docker-archive-keyring.gpg
21   - echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg
      ] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.
      list.d/docker.list > /dev/null
22   - sudo apt-get update
23   - sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

```

```

24
25 # 2. Add default user (ubuntu) to docker group
26 - usermod -aG docker ubuntu || true
27 - usermod -aG docker debian || true
28
29 # 3. Create the installation directory
30 - mkdir -p /opt/nullexit
31 - chown -R ubuntu:ubuntu /opt/nullexit || chown -R debian:debian /opt/nullexit
32
33 # 4. Enable IPv4 and IPv6 forwarding for VPN routing
34 - echo "net.ipv4.ip_forward=1" >> /etc/sysctl.d/99-tailscale.conf
35 - echo "net.ipv6.conf.all.forwarding=1" >> /etc/sysctl.d/99-tailscale.conf
36 - sysctl -p /etc/sysctl.d/99-tailscale.conf
37
38 final_message: "The remote exit node has been fully bootstrapped. SSH in, clone the repo to /opt/nullexit
, add your .env, and run ./setup-linux.sh!"

```

24 docker/tor/Dockerfile

```

1 FROM debian:bookworm-slim
2
3 RUN apt-get update && \
4     apt-get install -y tor obfs4proxy gosu bash && \
5     rm -rf /var/lib/apt/lists/*
6
7 COPY entrypoint.sh /entrypoint.sh
8 RUN chmod +x /entrypoint.sh
9
10 ENTRYPOINT ["/entrypoint.sh"]

```

25 docker/tor/entrypoint.sh

```

1 #!/bin/bash
2 set -e
3
4 TORRC="/etc/tor/torrc"
5 mkdir -p /etc/tor /var/lib/tor
6
7 # Base configuration: SOCKS 9050, Control 9051, TransPort 9040, DNSPort 5353 all bound to
8 # 0.0.0.0 (required for Docker/Colima host port-mapping). The ControlPort is secured via a
9 # dynamic HashedControlPassword (added below when TOR_PASSWORD is set); see devref.md 15.10.2.
10 cat <<EOF > $TORRC
11 SocksPort 0.0.0.0:9050
12 ControlPort 0.0.0.0:9051
13 CookieAuthentication 0
14 Log notice stdout
15 DataDirectory /var/lib/tor
16
17 # Transparent proxying & DNS resolution for .onion mapping
18 TransPort 0.0.0.0:9040
19 DNSPort 0.0.0.0:5353
20 AutomapHostsOnResolve 1
21 VirtualAddrNetworkIPv4 198.18.0.0/15
22 EOF
23
24 if [ -n "$TOR_PASSWORD" ]; then
25     HASH=$(tor --hash-password "$TOR_PASSWORD" | tail -n 1)
26     echo "HashedControlPassword $HASH" >> $TORRC
27 fi
28
29 if [ -n "$TOR_EXCLUDE_EXIT_NODES" ]; then
30     echo "ExcludeExitNodes $TOR_EXCLUDE_EXIT_NODES" >> $TORRC
31     echo "StrictNodes 1" >> $TORRC
32 fi
33
34 if [ "${TOR_USE_BRIDGES:-false}" = "true" ]; then
35     BRIDGE_FILE="${TOR_BRIDGE_LINES_FILE:-/tor-bridges.txt}"
36
37     if ! grep -vE "[[:space:]]*#" "$BRIDGE_FILE" | grep -q '[^[:space:]]'; then
38         MSG="[(date '+%Y-%m-%d %H:%M:%S')] [CRITICAL] [Tor Proxy] TOR_USE_BRIDGES is enabled, but no
bridge lines found at $BRIDGE_FILE. Proxy startup ABORTED. Get bridges from https://bridges.
torproject.org."

```

```

39     echo "$MSG"
40     if [ -w "/output.log" ]; then
41         echo "$MSG" >> /output.log
42     fi
43     # Sleep indefinitely to prevent docker-compose 'unless-stopped' from triggering a crash loop
44     exec sleep infinity
45 fi
46
47 echo "UseBridges 1" >> $TORRC
48 echo "ClientTransportPlugin obfs4 exec /usr/bin/obfs4proxy" >> $TORRC
49
50 while IFS= read -r line; do
51     # Ignore comments and empty lines
52     [[ -z "$line" || "$line" == \#* ]] && continue
53     echo "Bridge $line" >> $TORRC
54 done < "$BRIDGE_FILE"
55 fi
56
57 # Debian tor package uses 'debian-tor' user
58 chown -R debian-tor:debian-tor /var/lib/tor /etc/tor
59
60 exec gosu debian-tor /usr/bin/tor -f $TORRC

```

26 launchd/com.nullexit.noise.plist

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
3 <plist version="1.0">
4 <dict>
5     <key>Label</key>
6     <string>com.nullexit.noise</string>
7
8     <!-- Same install-agnostic pattern as com.nullexit.wake-recovery: WorkingDirectory
9          is __NULLEXIT_HOME__ (substitute your absolute install root at install time
10         via sed), and the program arg is scripts/noise.sh RELATIVE to it, so
11         BASH_SOURCE still resolves SCRIPT_DIR correctly. -->
12     <key>WorkingDirectory</key>
13     <string>__NULLEXIT_HOME__</string>
14
15     <!-- '__boot': governed SOLELY by NOISE_ENABLED in .env. It exits 0 cleanly when
16         disabled (so KeepAlive never tight-loops) and runs the padding loop in the
17         foreground when enabled. Loading this plist adds NO new option it just means
18         "auto-start cover traffic whenever NOISE_ENABLED=true". -->
19     <key>ProgramArguments</key>
20     <array>
21         <string>/bin/bash</string>
22         <string>scripts/noise.sh</string>
23         <string>__boot</string>
24     </array>
25
26     <!-- Start immediately on launchctl load and re-run at every login/boot. -->
27     <key>RunAtLoad</key>
28     <true/>
29
30     <!-- Same restart policy rationale as wake-recovery:
31         SuccessfulExit=false a clean exit 0 (the disabled-idle path) is NOT
32                             relaunched, so a disabled job doesn't spin.
33         Crashed=false       a SIGTERM from 'launchctl bootout' is not treated as
34                             a relaunch trigger, so stopping is clean and loop-free.
35         Net effect: load once padding runs at boot when enabled; bootout stops it. -->
36     <key>KeepAlive</key>
37     <dict>
38         <key>SuccessfulExit</key>
39         <false/>
40         <key>Crashed</key>
41         <false/>
42     </dict>
43
44     <!-- Cover traffic must not be suspended by App Nap when the user is idle. -->
45     <key>ProcessType</key>
46     <string>Background</string>
47
48     <key>StandardOutPath</key>
49     <string>/tmp/nullexit-noise.out.log</string>
50     <key>StandardErrorPath</key>
51     <string>/tmp/nullexit-noise.err.log</string>

```



```

52 </dict>
53 </plist>

```

27 launchd/com.nullexit.wake-recovery.plist

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
3 <plist version="1.0">
4 <dict>
5   <key>Label</key>
6   <string>com.nullexit.wake-recovery</string>
7
8   <!-- WorkingDirectory + relative program arg keeps this plist install-agnostic.
9        The source-of-truth for the install path is __NULLEXIT_HOME__, which
10       setup.sh or the user must substitute with their absolute nullexit
11       install root at install time. Program arg is 'scripts/watcher.sh'
12       RELATIVE to WorkingDirectory, so launchd's evaluated cwd IS the
13       install root and BASH_SOURCE inside scripts/watcher.sh still
14       resolves to the real <install>/scripts/watcher.sh, preserving the
15       SCRIPT_DIR pattern. After install-time 'sed -i ' 's|__NULLEXIT_HOME__|<abs_path>|' ',
16       the plist is portable to any clone location. -->
17   <key>WorkingDirectory</key>
18   <string>__NULLEXIT_HOME__</string>
19
20   <key>ProgramArguments</key>
21   <array>
22     <string>/bin/bash</string>
23     <string>scripts/watcher.sh</string>
24   </array>
25
26   <!-- Start the daemon immediately on launchctl load (instead of waiting
27        for the next user login). Combined with keepAlive below this means
28        once installed: load once, runs forever across logouts/reboots. -->
29   <key>RunAtLoad</key>
30   <true/>
31
32   <!-- Restart policy:
33        SuccessfulExit=false    SIGTERM / clean exit does NOT relaunch (so
34                                'launchctl bootout' cleanly stops, not loops).
35        Crashed=false          HARD crashes ALSO do NOT relaunch. We've seen
36                                that repeated sleep/wake cycles on macOS
37                                Sonoma+ accumulate orphan watcher.sh PIDs
38                                because SIGCONT/resume doesn't reliably kill
39                                the prior instance. The watcher.sh script
40                                now enforces single-instance via a manual
41                                PID-file and process-identity check at the
42                                top, so a re-launch on crash is
43                                actively dangerous (it would skip the lock
44                                and exit; better to surface the crash in
45                                logs than to silently 0-process). -->
46   <key>KeepAlive</key>
47   <dict>
48     <key>SuccessfulExit</key>
49     <false/>
50     <key>Crashed</key>
51     <false/>
52   </dict>
53
54   <!-- Background hint to App Nap so it doesn't get suspended when
55        the user is inactive. We're the entire reason this LaunchAgent
56        needs to NOT nap. -->
57   <key>ProcessType</key>
58   <string>Background</string>
59
60   <!-- launchd-level stdout/stderr (the bash launcher shell). The script
61        itself writes to /tmp/nullexit-watcher.log via exec >>. These
62        separate channels help us tell "launchd crashed" from "the bash
63        wrapper piped to recover.sh errored". -->
64   <key>StandardOutPath</key>
65   <string>/tmp/nullexit-watcher.out.log</string>
66   <key>StandardErrorPath</key>
67   <string>/tmp/nullexit-watcher.err.log</string>
68 </dict>
69 </plist>

```

28 plan.md

```
1 # Nullexit Cybersecurity & Performance Improvement Plan
2
3 ## 1. Context
4 *(Note: Some of the ideas outlined below have already been partially or fully explored/implemented. This
5   document serves as a master plan for ongoing and future iterations.)*
6
7 ## 2. Performance & Speed Testing
8
9 * **Noise-Based Speed Measurement (Sweep Extension):** Extend the 'sweep' functionality to measure
10   actual network throughput across all connected devices. Instead of standard ping/speed test payloads
11   , send mathematically verifiable random noise. This ensures the data cannot be compressed by
12   intermediate nodes, giving a true measure of the encrypted tunnel speed.
13
14 * **Investigate Mobile Device Slowness:** Investigate why internet speeds are fast on the Mac host but
15   noticeably slower on mobile devices like the Samsung S24 (and suspected on the iPhone).
16   *Hypothesis:* This could be related to MTU/MSS clamping over the Tailscale + WARP double tunnel
17   behaving differently on Android/iOS TCP stacks, or battery-saving Wi-Fi radio states aggressively
18   sleeping.
19
20 ## 3. Network OPSEC & Traffic Analysis
21
22 * **Device-to-Device Traffic Fingerprinting:** Analyze if a hostile/residence network can track or
23   fingerprint the traffic patterns when one device communicates with another on the same Wi-Fi (e.g.,
24   Mac to phone).
25   *Consideration:* While Tailscale (WireGuard) encrypts the payload, the network administrator can
26   still see the timing, volume, and IP endpoints of the UDP packets. We need to evaluate if this
27   metadata could expose the use of the mesh network or specific activities (like large file transfers)
28   .
29
30 ## 4. Advanced Cybersecurity Features
31
32 * **DNS Tunneling & Statistical ML Anomaly Detection:** Implement an adaptive mechanism to detect and
33   stop DNS tunneling or highly anomalous DNS requests using Shannon Entropy analysis.
34   *Methodology (Rolling Baseline & 3):* Normal DNS requests (like 'google.com') typically have a
35   Shannon entropy around 3.5 to 4.5. DNS tunneling relies on encoding data (often Base64/Hex),
36   resulting in highly random subdomains with entropy closer to 5.0 - 6.0.
37   *Implementation:* Build a rolling distribution curve based on the user's *actual* normal DNS
38   usage. Calculate the moving average and standard deviation (). If a request's entropy exceeds the
39   adaptive threshold (e.g., 'Mean + 3'), it is mathematically classified as a statistically
40   significant anomaly and dropped.
41
42 * **Micro-Honeypot (Honey-Port Expansion):** Run a lightweight honeypot container (like 'cowrie' for
43   SSH/Telnet) listening on decoy Tailscale IPs. If a compromised mesh device attempts lateral scanning
44   , it hits the honeypot, alerting you and dropping their traffic via 'iptables'.
45
46 * **Deep Packet Inspection (DPI) / IDS:** Route traffic through an IDS like Suricata or Zeek in a
47   container to detect anomalies, plain-text credential leaks, or advanced malware patterns missed by
48   basic IP/DNS blocklists.
49
50 * **Automated Ephemeral Rotating Tor Bridges:** Automatically scrape/request new 'obfs4' bridges every
51   few days and rotate them into the Tor container to ensure entry guards constantly shift.
52
53 * **Hardened Container Profiles:** Improve isolation for AdGuard, Tor, and routing containers by
54   implementing strict Docker 'seccomp' profiles and dropping unnecessary Linux capabilities.
55
56 * **WireGuard Key Rotation:** Implement auto-rotation for Cloudflare WARP keys (e.g., weekly cron) to
57   ensure perfect forward secrecy.
```

29 post-rules.txt

```
1 iptables -A FORWARD -i tailscale0 -o tun0 -j ACCEPT
2 iptables -A FORWARD -i tun0 -o tailscale0 -j ACCEPT
3 iptables -A INPUT -i tailscale0 -j ACCEPT
4 iptables -A INPUT -i eth0 -p udp --dport 41642 -j ACCEPT
5 iptables -t nat -A POSTROUTING -o tun0 -j MASQUERADE
6 # Clamp MSS to 1120. With Tailscale overhead on top of WARP (each WireGuard encapsulation adds ~60 bytes)
7 ,
8 # the safe MSS for double-tunneled traffic is 1120 to prevent strict-MSS endpoints from stalling.
9 # NOTE: this 1120 integer is rewritten at boot by toggle.sh from GATEWAY_MSS in .env
10 # (Gluetun's parser can't interpolate variables) change GATEWAY_MSS in .env, not this line.
11 iptables -t mangle -A POSTROUTING -p tcp --tcp-flags SYN,RST SYN -j TCPMSS --set-mss 1120
12
13 iptables -t nat -A PREROUTING -i tailscale0 -p udp --dport 53 -j REDIRECT --to-port 5335
14 iptables -t nat -A PREROUTING -i tailscale0 -p tcp --dport 53 -j REDIRECT --to-port 5335
15
16 # Also redirect DNS from Docker bridge interface so the host can reach AdGuard via localhost:53
17 iptables -t nat -A PREROUTING -i eth0 -p udp --dport 53 -j REDIRECT --to-port 5335
18 iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 53 -j REDIRECT --to-port 5335
```

```

19 # Block IPv6 exit-node traffic to prevent leakage (Gluetun/WARP is IPv4-only)
20 iptables -A FORWARD -i tailscale0 -j DROP
21
22 # Block DNS-over-TLS (Port 853) to force Android Private DNS to fallback to Port 53
23 iptables -A FORWARD -i tailscale0 -p tcp --dport 853 -j REJECT --reject-with tcp-reset
24 iptables -A FORWARD -i tailscale0 -p udp --dport 853 -j REJECT
25
26 # Block QUIC (UDP 443) to force Facebook/Google to fallback to TCP.
27 # UDP bypasses TCP MSS clamping, causing fragmentation. Over weak Wi-Fi (far from gateway),
28 # dropping a single fragment destroys the entire packet, stalling image downloads.
29 iptables -A FORWARD -i tailscale0 -p udp --dport 443 -j REJECT

```

30 scripts/Dockerfile.routing-fix

```

1 FROM golang:1.22-alpine AS builder
2 WORKDIR /app
3 COPY logger/ ./logger/
4 RUN cd logger && go build -o logger main.go
5
6 FROM alpine:3.20
7 RUN apk add --no-cache iptables iproute2 curl ipset
8 COPY --from=builder /app/logger/logger /usr/local/bin/nullexit-logger
9 COPY routing-fix.sh /routing-fix.sh
10 CMD ["sh", "/routing-fix.sh"]

```

31 scripts/Dockerfile.rule-compiler

```

1 FROM golang:1.22-alpine AS builder
2 WORKDIR /app
3 COPY rule-compiler/ ./rule-compiler/
4 RUN cd rule-compiler && go build -o sync-rules main.go
5
6 FROM alpine:3.20
7 COPY --from=builder /app/rule-compiler/sync-rules /usr/local/bin/sync-rules
8 WORKDIR /app
9 CMD ["sync-rules"]

```

32 scripts/Toggle-Gateway.bat

```

1 <# :
2 @echo off
3 powershell -NoProfile -ExecutionPolicy Bypass -Command "Invoke-Expression ([System.IO.File]::ReadAllText
4 ('%~f0') -replace '^(?s).*?#\s*>\s*\r?\n', '')"
5 exit /b
6 #>
7 # Toggle-Gateway.bat (Hybrid Batch-PowerShell Toggler) nullexit Gateway Toggler for Windows
8 # Mirrors the sophisticated error handling, checks, and automation of toggle.sh
9
10 # 1. Administrator Check & Elevation
11 $isAdmin = ([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).
12 IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)
13 if (-not $isAdmin) {
14     Write-Host "Requesting Administrator privileges..." -ForegroundColor Yellow
15     Start-Process PowerShell -ArgumentList "-NoProfile -ExecutionPolicy Bypass -Command 'Invoke-
16     Expression ([System.IO.File]::ReadAllText('PSCommandPath') -replace '^(?s).*?#\s*>\s*\r?\n', '') '""
17     -Verb RunAs
18     Exit
19 }
20
21 # Set console output encoding to UTF8 for clean symbols
22 [Console]::OutputEncoding = [System.Text.Encoding]::UTF8
23 Clear-Host
24
25 Write-Host "" -ForegroundColor Green
26 Write-Host "          nullexit Windows Toggler" -ForegroundColor Green
27 Write-Host "" -ForegroundColor Green
28 Write-Host ""

```

```

26 # 2. Helpers
27 function Reset-HostDNS {
28     Write-Host "Restoring default DNS (DHCP/DHCP-assigned DNS)..." -ForegroundColor Cyan
29     $activeAdapters = Get-NetAdapter | Where-Object { $_.Status -eq 'Up' }
30     foreach ($adapter in $activeAdapters) {
31         Write-Host " -> Resetting DNS on adapter: $($adapter.Name)" -ForegroundColor Gray
32         Set-DnsClientServerAddress -InterfaceIndex $adapter.InterfaceIndex -ResetServerAddresses -
        ErrorAction SilentlyContinue
33     }
34 }
35
36 function Hijack-HostDNS ($ip) {
37     Write-Host "Hijacking DNS to route through AdGuard ($ip)..." -ForegroundColor Yellow
38     $activeAdapters = Get-NetAdapter | Where-Object { $_.Status -eq 'Up' }
39     foreach ($adapter in $activeAdapters) {
40         Write-Host " -> Pointing DNS to $ip on adapter: $($adapter.Name)" -ForegroundColor Gray
41         Set-DnsClientServerAddress -InterfaceIndex $adapter.InterfaceIndex -ServerAddresses $ip -
        ErrorAction SilentlyContinue
42     }
43 }
44
45 function Get-HostTailscalePath {
46     $paths = @(
47         "$env:ProgramFiles\Tailscale\tailscale.exe",
48         "$env:ProgramFiles (x86)\Tailscale\tailscale.exe",
49         "tailscale.exe"
50     )
51     foreach ($path in $paths) {
52         if (Test-Path $path) { return $path }
53     }
54     return $null
55 }
56
57 # 3. Detect State
58 # Prevent deadlocks by resetting DNS to default temporarily during checks
59 Reset-HostDNS
60
61 Write-Host "Checking Docker daemon status..." -ForegroundColor Cyan
62 & docker info >$null 2>&1
63 if ($LASTEXITCODE -ne 0) {
64     Write-Host "Docker is not running. Starting Docker Desktop..." -ForegroundColor Yellow
65     $dockerPath = "C:\Program Files\Docker\Docker\Desktop.exe"
66     if (Test-Path $dockerPath) {
67         Start-Process $dockerPath
68         Write-Host "Waiting for Docker daemon to become responsive..." -ForegroundColor Gray
69         $timeout = 60
70         while ($timeout -gt 0) {
71             Start-Sleep -Seconds 2
72             & docker info >$null 2>&1
73             if ($LASTEXITCODE -eq 0) {
74                 Write-Host "Docker is ready!" -ForegroundColor Green
75                 break
76             }
77             $timeout -= 2
78         }
79         if ($timeout -le 0) {
80             Write-Error "Docker failed to start in time. Aborting."
81             Exit 1
82         }
83     } else {
84         Write-Error "Docker Desktop executable not found at standard path. Please start Docker manually."
85         Exit 1
86     }
87 }
88
89 # Detect if gateway is already running
90 $runningWarp = docker compose ps --status running --format json | ConvertFrom-Json -ErrorAction
    SilentlyContinue | Where-Object { $_.Service -eq "warp" }
91 $isGatewayActive = ($null -ne $runningWarp)
92
93 # 4. Execution
94 if ($isGatewayActive) {
95     # STOP PATH
96     Write-Host "Gateway is currently RUNNING. Disabling now..." -ForegroundColor Yellow
97     Write-Host ""
98
99     # Disconnect host Tailscale from exit node if installed
100     $tsCli = Get-HostTailscalePath
101     if ($null -ne $tsCli) {
102         Write-Host "Disconnecting host Tailscale from exit node..." -ForegroundColor Cyan
103         & $tsCli up --exit-node= --accept-dns=true >$null 2>&1

```

```

104 }
105
106 Write-Host "Stopping Docker containers..." -ForegroundColor Cyan
107 docker compose down -t 5
108
109 Reset-HostDNS
110 Write-Host "Gateway has been successfully STOPPED." -ForegroundColor Green
111 } else {
112     # START PATH
113     Write-Host "Gateway is currently STOPPED. Enabling now..." -ForegroundColor Yellow
114     Write-Host ""
115
116     # Clean up corrupted AdGuardHome configurations
117     $confFile = "adguard\conf\AdGuardHome.yaml"
118     if (Test-Path $confFile) {
119         $fileSize = (Get-Item $confFile).Length
120         if ($fileSize -eq 0) {
121             Remove-Item $confFile -Force
122             Write-Host "Removed corrupted empty AdGuardHome.yaml to prevent container crash loop." -
123                 ForegroundColor Yellow
124         }
125     }
126
127     Write-Host "Starting Docker containers..." -ForegroundColor Cyan
128     docker compose up -d
129
130     Write-Host "Waiting for container's Tailscale connection to offer Exit Node..." -ForegroundColor Cyan
131     Write-Host " (This can take up to 60 seconds..." -ForegroundColor Gray
132
133     $elapsed = 0
134     $maxWait = 60
135     $resolvedIP = $null
136
137     while ($elapsed -lt $maxWait) {
138         $statusJson = docker compose exec -T tailscale tailscale status --json 2>$null
139         if ($null -ne $statusJson) {
140             $status = $statusJson | ConvertFrom-Json -ErrorAction SilentlyContinue
141             if ($null -ne $status -and $null -ne $status.Self) {
142                 # Check if it has a valid Tailscale IP and is running
143                 if ($status.Self.Online -and $status.Self.TailscaleIPs.Count -gt 0) {
144                     $resolvedIP = $status.Self.TailscaleIPs[0]
145                     break
146                 }
147             }
148             Start-Sleep -Seconds 2
149             $elapsed += 2
150         }
151     }
152
153     if ($null -ne $resolvedIP) {
154         Write-Host "Tailscale IP resolved: $resolvedIP" -ForegroundColor Green
155         Write-Host ""
156
157         # Configure host Tailscale client to route through this exit node
158         $tsCli = Get-HostTailscalePath
159         if ($null -ne $tsCli) {
160             Write-Host "Configuring host Tailscale to use Exit Node ($resolvedIP)..." -ForegroundColor
161             Cyan
162             & $tsCli up --exit-node=$resolvedIP --accept-dns=false >$null 2>&1
163         }
164
165         # Hijack DNS
166         Hijack-HostDNS $resolvedIP
167         Write-Host ""
168         Write-Host "Gateway has been successfully STARTED." -ForegroundColor Green
169     } else {
170         Write-Host "Warning: Could not resolve gateway Tailscale IP. DNS hijacking skipped." -
171             ForegroundColor Red
172
173         # Fallback to .gateway_ip if it exists
174         if (Test-Path ".gateway_ip") {
175             $fallbackIP = Get-Content ".gateway_ip" -TotalCount 1
176             if (-not [string]::IsNullOrEmpty($fallbackIP)) {
177                 Write-Host "[Fallback] Using static IP from .gateway_ip: $fallbackIP" -ForegroundColor
178                 Yellow
179                 Hijack-HostDNS $fallbackIP
180                 Write-Host "Gateway STARTED (with static fallback IP)." -ForegroundColor Green
181             }
182         } else {
183             Write-Host "Gateway started in offline/unauthenticated state. Run setup.sh if this is a fresh
184                 setup." -ForegroundColor Yellow
185         }
186     }
187 }

```

```

180     }
181 }
182
183 Write-Host ""
184 Write-Host "Press any key to exit..."
185 [void]$Host.UI.RawUI.ReadKey("NoEcho,IncludeKeyDown")

```

33 scripts/check_circuits.py

```

1  #!/usr/bin/env python3
2  import socket
3  import os
4  import re
5
6  def query_tor(port, command, password=""):
7      try:
8          s = socket.socket()
9          s.settimeout(2)
10         s.connect(('127.0.0.1', port))
11         s.sendall(f"AUTHENTICATE \"{password}\"{command}\r\nQUIT\r\n".encode())
12         response = b""
13         while True:
14             chunk = s.recv(4096)
15             if not chunk:
16                 break
17             response += chunk
18         return response.decode()
19     except Exception as e:
20         return f"Error connecting to Tor Control Port: {e}"
21
22 def get_ports():
23     ports = {}
24     ports_file = "/tmp/nullexit-ports.env"
25     if os.path.exists(ports_file):
26         with open(ports_file, 'r') as f:
27             for line in f:
28                 if line.startswith("export "):
29                     # Split at first '=' to handle potential multiple '=' in value
30                     parts = line.replace("export ", "").strip().split("=", 1)
31                     if len(parts) == 2:
32                         key, val = parts
33                         try:
34                             ports[key] = int(val)
35                         except ValueError:
36                             ports[key] = val
37     return ports
38
39 def get_password(ports):
40     # 1. Try ports.env
41     password = ports.get("TOR_PASSWORD")
42     if password:
43         return password
44
45     # 2. Try .env
46     if os.path.exists(".env"):
47         try:
48             with open(".env", "r") as f:
49                 for line in f:
50                     if line.startswith("ADGUARD_PASSWORD="):
51                         return line.split("=", 1)[1].strip().strip('\n')
52                     if line.startswith("TOR_PASSWORD="):
53                         return line.split("=", 1)[1].strip().strip('\n')
54         except Exception:
55             pass
56
57     # 3. Fallback
58     return "nullexit"
59
60 def main():
61     ports = get_ports()
62     control_port = ports.get("TOR_CONTROL_PORT", 9051)
63     password = get_password(ports)
64
65     print(f"Connecting to Tor Control Port on 127.0.0.1:{control_port}...")
66     status = query_tor(control_port, "GETINFO circuit-status", password)
67

```

```

68     if "Error" in status:
69         print(status)
70         return
71
72     print("\nActive Tor Circuits:")
73     circuits = re.findall(r"(\d+)\s+BUILT\s+(.?)\s+PURPOSE", status)
74     if not circuits:
75         print("No active built circuits found yet. Tor might still be bootstrapping.")
76         return
77
78     for circ_id, path_str in circuits:
79         print(f"\nCircuit #{circ_id}:")
80         nodes = path_str.split(",")
81         for idx, node in enumerate(nodes):
82             fingerprint = node.split("~")[0].replace("$", "")
83             nickname = node.split("~")[1] if "~" in node else "unknown"
84
85             # Fetch node IP
86             ns_info = query_tor(control_port, f"GETINFO ns/id/{fingerprint}", password)
87             ip = "unknown"
88             for line in ns_info.split("\n"):
89                 if line.startswith("r "):
90                     parts = line.split(" ")
91                     if len(parts) >= 7:
92                         ip = parts[6]
93                         break
94
95             # Fetch country
96             country = "unknown"
97             if ip != "unknown":
98                 country_info = query_tor(control_port, f"GETINFO ip-to-country/{ip}", password)
99                 for line in country_info.split("\n"):
100                     if line.startswith("250-ip-to-country/"):
101                         country = line.split("=")[1].strip().upper()
102                         break
103
104             role = "Guard" if idx == 0 else ("Exit" if idx == len(nodes) - 1 else "Middle")
105             print(f" [{role}] {nickname} ({ip}) - Country: {country}")
106
107 if __name__ == "__main__":
108     main()

```

34 scripts/diagnose-host-leak.sh

```

1  #!/bin/bash
2  # scripts/diagnose-host-leak.sh nullexit host-routing diagnostic
3  #
4  # Symptom this addresses: tests like https://www.whatismyip.com on the
5  # *HOST MAC* return the underlying physical ISP/ASN
6  # local ISP / campus network) even though the nullexit gateway is active and remote
7  # tailnet clients correctly egress via Cloudflare WARP. The container and
8  # the gateway itself are healthy only the host's own routing is leaking.
9  #
10 # This script runs 7 checks in sequence, classifies the result into ONE
11 # of the three known leak scenarios, writes the full report to a
12 # timestamped file in the project root, and prints a verdict + a
13 # ready-to-run fix command on stdout. Pass --fix to apply the matching
14 # remediation in the same invocation and re-verify egress afterwards.
15 # See devref.md 10.30 for the deep-dive rationale.
16 #
17 # Usage:
18 #   bash scripts/diagnose-host-leak.sh           # diagnose + write report
19 #   bash scripts/diagnose-host-leak.sh --fix      # diagnose + fix + re-verify
20 #   bash scripts/diagnose-host-leak.sh --watch    # full baseline then loop checks every 60s
21 #   bash scripts/diagnose-host-leak.sh --watch 30 # same, with custom interval (seconds)
22 #   bash scripts/diagnose-host-leak.sh --help    # this message
23 #
24 # Multiple runs produce timestamped files (one per run) so historical
25 # reports can be diffed. The newest file is the most recent.
26
27 set -uo pipefail
28 # NOTE: errexit ('set -e') is intentionally NOT enabled. Several checks
29 # below are EXPECTED to fail and that's diagnostic data (e.g. 'tailscale
30 # status' returns non-zero when the daemon is wedged). Every command
31 # pipes to a helper that records the exit code without killing the script.
32

```



```

33 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
34 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
35 TIMESTAMP="$(date -u +%Y%m%dT%H%M%SZ)"
36 source "$SCRIPT_DIR/common.sh"
37 LOG_FILE="$PROJECT_ROOT/output.log"
38 REPORT_FILE="$PROJECT_ROOT/host-leak-diagnostic-$TIMESTAMP.txt"
39
40 # Argument parsing
41 DO_FIX=false
42 DO_WATCH=false
43 WATCH_INTERVAL=60
44 WATCH_LOG="$PROJECT_ROOT/host-leak-watch.log"
45 case "${1:-}" in
46   --fix) DO_FIX=true ;;
47   --watch)
48     DO_WATCH=true
49     # Optional second argument: custom interval in seconds
50     if [ -n "${2:-}" ] && [[ "$2" =~ ^[0-9]+$ ]]; then
51       WATCH_INTERVAL="$2"
52     fi
53     ;;
54   --help|-h)
55     sed -n '2,31p' "$0"
56     exit 0
57     ;;
58   *)
59     : # default: diagnose only
60     ;;
61   *)
62     echo "Unknown argument: $1" >&2
63     echo "Run with --help for usage." >&2
64     exit 2
65     ;;
66 esac
67
68 # Colours (auto-disabled when stdout is not a tty, e.g. piped)
69 if [ -t 1 ]; then
70   RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
71   BOLD='\033[1m'; NC='\033[0m'
72 else
73   RED=''; GREEN=''; YELLOW=''; BOLD=''; NC=''
74 fi
75
76 # Output helpers write to BOTH stdout and the report file
77 exec 3>> "$REPORT_FILE" || {
78   printf '\n[FATAL] cannot open %s for write\n' "$REPORT_FILE" >&2
79   exit 1
80 }
81 section() {
82   local title="$1"
83   printf '\n%b %s %b\n' "$BOLD" "$title" "$NC" >&3
84   printf '\n%b %s %b\n' "$BOLD" "$title" "$NC"
85 }
86 line() { printf ' %b\n' "$1" >&3; printf ' %b\n' "$1"; }
87 ok() { line "${GREEN}${NC} $1"; }
88 warn() { line "${YELLOW}${NC} $1"; }
89 fail() { line "${RED}${NC} $1"; }
90 note() { line "(no $1)"; }
91
92 # Watch-mode header (only shown when entering watch loop)
93 if [ "$DO_WATCH" = "true" ]; then
94   echo ""
95   echo ""
96   echo " n u l l e x i t   w a t c h   m o d e "
97   echo ""
98   echo " Interval:          ${WATCH_INTERVAL}s"
99   echo " Watch log:         $WATCH_LOG"
100   echo ""
101 else
102   echo ""
103   echo " n u l l e x i t   h o s t - r o u t i n g   d i a g n o s t i c "
104   echo ""
105   echo " Timestamp (UTC): $TIMESTAMP"
106   echo " Project root:    $PROJECT_ROOT"
107   echo " Report file:     $REPORT_FILE"
108   echo " Mode:            $([ "$DO_FIX" = true ] && echo "diagnose + apply fix" || echo "diagnose only (
109     pass --fix to remediate)")"
110   echo ""
111   echo "" >&3
112   echo "" >&3
113   echo " n u l l e x i t   h o s t - r o u t i n g   d i a g n o s t i c " >&3

```

```

113 echo "" >&3
114 echo " Timestamp (UTC): $TIMESTAMP" >&3
115 echo " Mode:          $([ "$DO_FIX" = true ] && echo "diagnose + apply fix" || echo "diagnose only")"
    >&3
116 fi
117
118 # Always add Homebrew to PATH (same as toggle.sh / recover.sh)
119 export PATH="/opt/homebrew/bin:/usr/local/bin:$PATH"
120
121 # Pure-bash timeout (macOS lacks GNU 'timeout')
122 run_with_timeout() {
123     local timeout_sec="$1"
124     shift
125     if [ "$#" -eq 0 ]; then return 1; fi
126     "$@" &
127     local cmd_pid=$!
128     (
129         sleep "$timeout_sec"
130         if kill -0 "$cmd_pid" 2>> "$LOG_FILE"; then
131             kill -9 "$cmd_pid" 2>> "$LOG_FILE" || true
132         fi
133     ) >> "$LOG_FILE" 2>&1 &
134     local watcher_pid=$!
135     set +e
136     wait "$cmd_pid"
137     local exit_status=$?
138     set -uo pipefail
139     kill "$watcher_pid" 2>> "$LOG_FILE" || true
140     return "$exit_status"
141 }
142
143 #
144 # Quick-check functions (used by --watch loop; return simple status strings)
145 # Each echoes its result so the caller can capture and compare.
146 #
147
148 quick_warp() {
149     # Returns: "on", "off", "unknown"
150     local out
151     out=$(run_with_timeout 8 curl -s --max-time 6 \
152         https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null || echo "")
153     local warp
154     warp=$(printf '%s' "$out" | awk -F'=' '/warp={print $2; exit}')
155     printf '%s' "${warp:-unknown}"
156 }
157
158 quick_ipv6() {
159     # Returns: the IPv6 address if leaking, or empty string if clean
160     local out
161     out=$(run_with_timeout 7 curl -6 -s --max-time 5 ifconfig.co 2>/dev/null || echo "")
162     if [ -n "$out" ] && printf '%s' "$out" | grep -qE '^[0-9a-fA-F:]+$'; then
163         printf '%s' "$out"
164     fi
165 }
166
167 quick_default_route() {
168     # Returns: "utun" if default route is via Tailscale utun*,
169     #           "wifi" if via physical Wi-Fi (192.168.x.x),
170     #           "other" otherwise
171     local route
172     route=$(netstat -rn 2>/dev/null | awk '/^default/ {print $0; exit}')
173     if [ -z "$route" ]; then
174         printf '%s' "none"
175     elif printf '%s' "$route" | grep -qE 'utun[0-9]+'; then
176         printf '%s' "utun"
177     elif printf '%s' "$route" | grep -qE '^(default|0\.0\.0\.0).*\b192\.168\.'; then
178         printf '%s' "wifi"
179     else
180         printf '%s' "other"
181     fi
182 }
183
184 # Watch loop (runs after baseline diagnostic when --watch is passed)
185 run_watch_loop() {
186     local baseline_warp="$1"
187     local baseline_default="$2"
188     local baseline_ipv6_leak="$3"
189     local interval="$4"
190
191     echo ""
192     echo ""

```

```

193 echo " Baseline established. Monitoring every ${interval}s..."
194 echo "   warp:      $baseline_warp"
195 echo "   default:    $baseline_default"
196 echo ""
197
198 # Safety: warn if interval is very short (avoids hammering APIs)
199 if [ "$interval" -lt 30 ]; then
200     printf '%b Short interval (%ss) rate-limiting risk on external APIs%b\n' "$YELLOW" "$interval" "$NC"
201     printf ' Consider 30s or longer for production use.\n'
202 fi
203 printf '[%s] WATCH START warp=%s default=%s interval=%ss\n' \
204     "$(date -u +%Y-%m-%dT%H:%M:%SZ)" "$baseline_warp" "$baseline_default" "$interval" \
205     >> "$WATCH_LOG"
206
207 local iteration=0
208 local last_warp="$baseline_warp"
209 local last_default="$baseline_default"
210 local last_ipv6_ok=$(( [ "$baseline_ipv6_leak" = "false" ] && echo true || echo false ))
211
212 trap 'printf "\n%bWatch stopped by user. Log at: %s%b\n" "$YELLOW" "$WATCH_LOG" "$NC"; exit 0' INT TERM
213
214 while true; do
215     iteration=$((iteration + 1))
216
217     local warp_now ipv6_now default_now
218     warp_now=$(quick_warp)
219     ipv6_now=$(quick_ipv6)
220     default_now=$(quick_default_route)
221
222     local ts
223     ts=$(date -u +%H:%M:%S)
224
225     # WARP check
226     if [ "$warp_now" != "$last_warp" ]; then
227         if [ "$warp_now" = "off" ] || [ "$warp_now" = "unknown" ]; then
228             printf '\n%b ALERT %b\n' "$RED$BOLD" "$NC"
229             printf '%b[%s] %bWARP FLIPPED: %s %s%b YOUR IP IS LEAKING!\n' \
230                 "$RED" "$ts" "$BOLD" "$last_warp" "$warp_now" "$NC"
231             printf '[%s] ALERT warp flipped %s%b\n' "$ts" "$last_warp" "$warp_now" >> "$WATCH_LOG"
232         else
233             printf '\n%b[%s] warp recovered: %s %s%b\n' \
234                 "$GREEN" "$ts" "$last_warp" "$warp_now" "$NC"
235             printf '[%s] OK warp recovered %s%b\n' "$ts" "$last_warp" "$warp_now" >> "$WATCH_LOG"
236         fi
237         last_warp="$warp_now"
238     fi
239
240     # IPv6 check
241     if [ -n "$ipv6_now" ]; then
242         if [ "$last_ipv6_ok" = true ]; then
243             printf '\n%b ALERT %b\n' "$RED$BOLD" "$NC"
244             printf '%b[%s] %bIPv6 LEAK DETECTED %s%b\n' \
245                 "$RED" "$ts" "$BOLD" "$ipv6_now" "$NC"
246             printf '[%s] ALERT IPv6 leak %s\n' "$ts" "$ipv6_now" >> "$WATCH_LOG"
247             last_ipv6_ok=false
248         fi
249     else
250         if [ "$last_ipv6_ok" = false ]; then
251             printf '%b[%s] IPv6 leak resolved%b\n' "$GREEN" "$ts" "$NC"
252             printf '[%s] OK IPv6 resolved\n' "$ts" >> "$WATCH_LOG"
253         fi
254         last_ipv6_ok=true
255     fi
256
257     # Default route check
258     if [ "$default_now" != "$last_default" ]; then
259         if [ "$default_now" != "utun" ]; then
260             printf '\n%b ALERT %b\n' "$RED$BOLD" "$NC"
261             printf '%b[%s] %bDEFAULT ROUTE CHANGED: %s %s%b traffic may bypass WARP!\n' \
262                 "$RED" "$ts" "$BOLD" "$last_default" "$default_now" "$NC"
263             printf '[%s] ALERT route changed %s%b\n' "$ts" "$last_default" "$default_now" >> "$WATCH_LOG"
264         else
265             printf '%b[%s] default route recovered: %s %s%b\n' \
266                 "$GREEN" "$ts" "$last_default" "$default_now" "$NC"
267             printf '[%s] OK route recovered %s%b\n' "$ts" "$last_default" "$default_now" >> "$WATCH_LOG"
268         fi
269         last_default="$default_now"
270     fi
271
272     # Heartbeat (every 10th iteration or if all OK)

```

```

273     if [ $((iteration % 10)) -eq 0 ]; then
274         printf "[%s] #d warp=%s ipv6=%s route=%s\n" \
275             "$ts" "$iteration" "$warp_now" "${ipv6_now:-none}" "$default_now" >> "$WATCH_LOG"
276         printf "\r[%s] #d warp=%s route=%s (Ctrl+C to stop)%b" \
277             "$GREEN" "$ts" "$iteration" "$warp_now" "$default_now" "$NC"
278     fi
279     sleep "$interval"
280 done
281 }
282
283 #
284 # Begin main diagnostic (skipped in non-watch modes if we're just watching)
285 #
286 #
287 ACTIVE_SERVICE=$(get_active_service)
288 ENO_SERVICE=$(networksetup -listnetworkserviceorder 2>> "$LOG_FILE" \
289     | grep -B 1 "Device: en0" | head -1 \
290     | sed -E 's/^\([0-9\*]+\)/ /' || true)
291 [ -z "$ENO_SERVICE" ] && ENO_SERVICE="Wi-Fi"
292
293 # Resolve the gateway's Tailscale IP.
294 GATEWAY_TS_IP=""
295 if [ -f "$PROJECT_ROOT/.gateway_ip" ]; then
296     GATEWAY_TS_IP=$(cat "$PROJECT_ROOT/.gateway_ip" 2>> "$LOG_FILE" \
297         | tr -d '\r' | awk 'NR==1{print $1; exit}')
298 fi
299 if [ -z "$GATEWAY_TS_IP" ] && command -v docker >> "$LOG_FILE" 2>&1 \
300     && docker compose ps --status running 2>/dev/null | grep -q 'tailscale'; then
301     GATEWAY_TS_IP=$(run_with_timeout 10 docker compose exec -T tailscale \
302         tailscale ip -4 2>> "$LOG_FILE" \
303         | tr -d '\r' | awk 'NR==1{print $1; exit}')
304 fi
305
306 # Scenario classification state
307 SCENARIO=""
308 WARP_STATUS=""
309 SOCKS5_ENABLED=false
310 TS_ON_MESH=false
311 DEFAULT_VIA_UTUN=false
312 IPV6_LEAK=false
313
314 #
315 section "1/8 Host Tailscale mesh status"
316 #
317 if ! command -v tailscale >> "$LOG_FILE" 2>&1; then
318     fail "tailscale CLI not on PATH install via 'brew install tailscale' then re-run"
319     echo " (a Tailscale daemon is required for the exit-node path to work)"
320 else
321     set +e
322     TS_OUT=$(run_with_timeout 5 tailscale status 2>&1)
323     TS_EXIT=$?
324     set -uo pipefail
325     TS_HEAD=$(printf '%s\n' "$TS_OUT" | head -5 | tr -d '\r')
326     if [ "$TS_EXIT" -eq 137 ]; then
327         fail "tailscaled daemon is wedged/unresponsive (5s timeout)"
328         echo " Fix path: 'sudo brew services restart tailscale'"
329     elif printf '%s' "$TS_OUT" | grep -qE 'Logged out|not logged in|expired'; then
330         fail "tailscale is logged out / auth expired on this host"
331         echo " Fix path: 'sudo tailscale up' (browser auth once)"
332     elif printf '%s' "$TS_OUT" | grep -q '100\.'; then
333         ok "Host is on tailnet (100.x.x.x visible in status)"
334         TS_ON_MESH=true
335         if printf '%s' "$TS_OUT" | grep -q "$GATEWAY_TS_IP"; then
336             ok "Gateway $GATEWAY_TS_IP visible in host's peer list"
337         elif [ -n "$GATEWAY_TS_IP" ]; then
338             warn "Gateway $GATEWAY_TS_IP NOT in host's peer list possible auth/key mismatch"
339         fi
340         line " first 5 lines of 'tailscale status' "
341         line "$(printf '%s' "$TS_HEAD" | awk '{print "    "$0}')"
342     else
343         fail "tailscale output unrecognized:"
344         line "$(printf '%s' "$TS_OUT" | head -3 | awk '{print "    "$0}')"
345     fi
346 fi
347
348 #
349 section "2/8 Active SOCKS5 proxy on Wi-Fi"
350 #
351 SOCKS_OUT=$(networksetup -getsocksfirewallproxy "$ACTIVE_SERVICE" 2>> "$LOG_FILE" \
352     || networksetup -getsocksfirewallproxy Wi-Fi 2>> "$LOG_FILE" \

```

```

354         || echo "Could not query SOCKS5 proxy state")
355 if printf '%s' "$SOCKS_OUT" | grep -q "Enabled: Yes"; then
356     fail "SOCKS5 proxy IS enabled on $ACTIVE_SERVICE toggle.sh fell back to SOCKS5 mode last run"
357     SOCKS5_ENABLED=true
358     line "    Full state:"
359     line "$(printf '%s' "$SOCKS_OUT" | awk '{print "    "$0}')"
360     line "    Browse on macOS ignores system SOCKS5 by default traffic bypasses WARP."
361 else
362     ok "SOCKS5 proxy is OFF on $ACTIVE_SERVICE exit-node path should be active"
363 fi
364
365 #
366 section "3/8 Egress IP + WARP tunnel verification (definitive test)"
367 #
368 CDN_OUT=$(run_with_timeout 8 curl -s --max-time 6 \
369     https://www.cloudflare.com/cdn-cgi/trace 2>&1 || echo "(curl failed)")
370 CDN_WARP=$(printf '%s' "$CDN_OUT" | awk -F=' ' '/^warp=/ {print $2; exit}')
371 CDN_IP=$(printf '%s' "$CDN_OUT" | awk -F=' ' '/^ip=/ {print $2; exit}')
372 CDN_COLO=$(printf '%s' "$CDN_OUT" | awk -F=' ' '/^colo=/ {print $2; exit}')
373 WARP_STATUS="$CDN_WARP"
374 if [ "$CDN_WARP" = "on" ]; then
375     ok "warp=on tunnel is hot, host traffic IS going through Cloudflare WARP"
376     line "    Public IP:    $CDN_IP"
377     line "    Cloudflare:    ${CDN_COLO:-unknown colo}"
378     line "    whatismyip.com's 'local ISP' result is its ISP-database error, not yours."
379 elif [ "$CDN_WARP" = "off" ]; then
380     fail "warp=off host traffic is NOT going through WARP"
381     line "    Public IP:    $CDN_IP (NOT Cloudflare)"
382     warn "This is the actual leak."
383 else
384     warn "could not determine warp status (curl failed or returned unexpected shape)"
385     line "    Raw output: $(printf '%s' "$CDN_OUT" | head -3 | tr '\n' ' ')"
386 fi
387
388 #
389 section "4/8 IPv6 leak probe (bypasses IPv4 exit-node on campus networks)"
390 #
391 IPV6_OUT=$(run_with_timeout 7 curl -s --max-time 5 ifconfig.co 2>&1 || echo "")
392 if [ -n "$IPV6_OUT" ] && printf '%s' "$IPV6_OUT" | grep -qE '[0-9a-fA-F:]+$'; then
393     fail "host has live IPv6 egress $IPV6_OUT (A6 record bypasses WARP)"
394     IPV6_LEAK=true
395     line "    Tailscale exit-node advertisement is IPv4-only, so IPv6 traffic skips it."
396     line "    Quick-fix: 'sudo networksetup -setv6off $ACTIVE_SERVICE'"
397 else
398     ok "no IPv6 egress detected (good IPv6 won't bypass the tunnel)"
399 fi
400
401 #
402 section "5/8 Host egress route for real traffic (route -n get 1.1.1.1)"
403 #
404 # WHY NOT 'netstat -rn | grep default':
405 # On macOS, Tailscale does NOT replace the physical default route installed by
406 # DHCP. Instead it adds a second interface-scoped route bound to the utun
407 # interface that the kernel prefers for real traffic forwarding. The literal
408 # "default" entry (192.168.x.x via en0) is left untouched in the table as a
409 # BSD routing quirk it's not lying, it's just answering a different question.
410 # The correct question is: "where does a real destination actually resolve?"
411 # 'route -n get 1.1.1.1' asks the kernel's forwarding logic directly.
412 ROUTE_GET=$(route -n get 1.1.1.1 2>/dev/null)
413 ROUTE_IFACE=$(echo "$ROUTE_GET" | awk '/interface:/ {print $2}')
414 ROUTE_GATEWAY=$(echo "$ROUTE_GET" | awk '/gateway:/ {print $2}')
415 if [ -z "$ROUTE_IFACE" ]; then
416     warn "route -n get 1.1.1.1 returned no interface routing table may be empty"
417 else
418     line "    interface: $ROUTE_IFACE gateway: ${ROUTE_GATEWAY:-n/a}"
419     if echo "$ROUTE_IFACE" | grep -qE '^utun[0-9]+'; then
420         ok "1.1.1.1 resolves via $ROUTE_IFACE Tailscale interface-scoped route is winning"
421         DEFAULT_VIA_UTUN=true
422     elif echo "$ROUTE_IFACE" | grep -qE '^en[0-9]+'; then
423         fail "1.1.1.1 resolves via $ROUTE_IFACE (physical Wi-Fi) exit-node interface route not active"
424     else
425         warn "1.1.1.1 resolves via unexpected interface: $ROUTE_IFACE"
426     fi
427 fi
428
429 #
430 section "6/8 Last toggle.sh START-relevant lines in output.log (if any)"
431 #
432 if [ ! -f "$LOG_FILE" ]; then
433     note "output.log"
434     line "    toggle.sh was never run from this directory. Run './toggle.sh' first,"

```

```

435 line " then re-run this diagnostic."
436 else
437   HITS=$(grep -E 'Exit node enabled|SKIP_EXIT_NODE|Pre-flight|Falling back to SOCKS|Starting|Successfully
    '\
438     "$LOG_FILE" 2>/dev/null | tail -8 || true)
439   if [ -z "$HITS" ]; then
440     note "matching lines in output.log"
441   else
442     line "$(printf '%s\n' "$HITS" | awk '{print "    "$0}')"
443   fi
444 fi
445
446 #
447 section "7/8 Gateway IP we should be pointing at"
448 #
449 if [ -n "$GATEWAY_TS_IP" ]; then
450   ok "gateway Tailscale IP: $GATEWAY_TS_IP"
451 else
452   warn "could not resolve gateway Tailscale IP"
453   line " (looked in .gateway_ip and tried docker compose exec tailscale ip -4)"
454 fi
455
456 #
457 section "8/8 Tailscale System Extension (App Store conflict check)"
458 #
459 SE_PENDING_UNINSTALL=false
460 SE_ACTIVE=false
461 if command -v systemextensionsctl >/dev/null 2>&1; then
462   SE_OUT=$(systemextensionsctl list 2>/dev/null | grep -iE '(io|com)\.tailscale')
463   if [ -n "$SE_OUT" ]; then
464     line " Tailscale System Extension(s) detected:"
465     line "$(printf '%s' "$SE_OUT" | awk '{print "    "$0}')"
466     if printf '%s' "$SE_OUT" | grep -q 'waiting to uninstall'; then
467       SE_PENDING_UNINSTALL=true
468       warn "System Extension is pending uninstall on next reboot."
469     else
470       SE_ACTIVE=true
471       warn "Active Tailscale System Extension detected. This will conflict with Homebrew Tailscale."
472     fi
473   else
474     ok "no Tailscale System Extension detected"
475   fi
476 else
477   note "systemextensionsctl not available (not on macOS or permission restricted)"
478 fi
479
480 #
481 section "CLASSIFICATION applies ONE of the four known scenarios"
482 #
483
484 if [ "$SWARP_STATUS" = "on" ] && [ "$IPV6_LEAK" = "false" ]; then
485   SCENARIO="OK"
486 elif [ "$SE_PENDING_UNINSTALL" = "true" ]; then
487   SCENARIO="SE_PENDING"
488 elif [ "$SOCKS5_ENABLED" = "true" ]; then
489   SCENARIO="A"
490 elif [ "$IPV6_LEAK" = "true" ]; then
491   SCENARIO="B"
492 elif [ "$DEFAULT_VIA_UTUN" = "false" ] && [ "$TS_ON_MESH" = "true" ]; then
493   SCENARIO="C"
494 else
495   SCENARIO="UNKNOWN"
496 fi
497
498 case "$SCENARIO" in
499   SE_PENDING)
500     echo ""
501     echo -e " ${RED}${BOLD}VERDICT: Scenario SE_PENDING Tailscale System Extension Pending Uninstall${{
502       NC}}
503     echo " A prior App Store Tailscale System Extension is pending uninstall."
504     echo " Since System Integrity Protection (SIP) is enabled, macOS blocks manual"
505     echo " uninstallation of terminated system extensions until the next reboot."
506     echo " Brew-managed tailscaled cannot engage the Network Extension slot until this is resolved."
507     echo ""
508     echo " ${BOLD}Recommended fix:${{NC}}
509     echo " sudo shutdown -r now"
510     echo " # After reboot, run: ./toggle.sh"
511     ;;
512   A)
513     echo ""
514     echo -e " ${RED}${BOLD}VERDICT: Scenario A SOCKS5 fallback lane${{NC}}

```

```

514 echo " toggle.sh's pre-flight checks failed last run. The script activated"
515 echo " SOCKS5 + local DNS proxy as a fallback. DNS gets hijacked but"
516 echo " browsers on macOS ignore the system SOCKS5 traffic egresses"
517 echo " direct through the local network."
518 echo ""
519 echo " ${BOLD}Recommended fix:${NC}"
520 echo "     echo 'GATEWAY_BYPASS_PING=true' >> $PROJECT_ROOT/.env"
521 echo "     sudo tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \\"
522 if [ -n "$GATEWAY_TS_IP" ]; then
523     echo "         --exit-node=$GATEWAY_TS_IP --exit-node-allow-lan-access=true"
524 else
525     echo "         --exit-node=$GATEWAY_TS_IP --exit-node-allow-lan-access=true # fill in $GATEWAY_TS_IP"
526 fi
527 echo "     sudo dscacheutil -flushcache && sudo killall -HUP mDNSResponder"
528 echo "     ./toggle.sh"
529 ;;
530 B)
531 echo ""
532 echo -e " ${RED}${BOLD}VERDICT: Scenario B IPv6 leak over dual-stack campus/local IPv6${NC}"
533 echo " Tailscale exit-node advertises only IPv4. macOS sends AAAA queries"
534 echo " straight out en0, which is dual-stack on most campus APs IPv6"
535 echo " traffic bypasses the WARP tunnel entirely."
536 echo ""
537 echo " ${BOLD}Recommended fix:${NC}"
538 echo "     sudo networksetup -setv6off $ACTIVE_SERVICE"
539 echo "     sudo dscacheutil -flushcache && sudo killall -HUP mDNSResponder"
540 echo "     # Re-enable IPv6 later with: sudo networksetup -setv6automatic $ACTIVE_SERVICE"
541 ;;
542 C)
543 echo ""
544 echo -e " ${RED}${BOLD}VERDICT: Scenario C Tailscale route-freeze${NC}"
545 echo " Host's default route points at the Wi-Fi gateway instead of the"
546 echo " Tailscale utun* interface. The exit-node preference is set but the"
547 echo " routing-table assertion did not take effect (devref 10.26)."

```



```

593 Scenario: $$SCENARIO
594 WARP egress: $$WARP_STATUS (cdn-cgi/trace)
595 SOCKS5 lane: $([ "$SOCKS5_ENABLED" = true ] && echo "ENABLED (browsers bypass)" || echo "off")
596 IPv6 leak: $([ "$IPV6_LEAK" = true ] && echo "YES (campus IPv6 egress)" || echo "no")
597 Default via: $([ "$DEFAULT_VIA_UTUN" = true ] && echo "utun* (Tailscale)" || echo "Wi-Fi gateway")
598 Host on mesh: $([ "$TS_ON_MESH" = true ] && echo "yes" || echo "no")
599 SE pending: $([ "$SE_PENDING_UNINSTALL" = true ] && echo "yes" || echo "no")
600 Gateway IP: ${GATEWAY_TS_IP:-unresolved}
601 EOF
602 )
603 printf '%s\n' "$SECOND_BLOCK" >&3
604
605 #
606 # --fix mode
607 #
608 if [ "$DO_FIX" = "true" ] && [ "$SCENARIO" != "OK" ] && [ "$SCENARIO" != "UNKNOWN" ]; then
609     echo ""
610     echo ""
611     echo " A P P L Y I N G   F I X   (scenario $SCENARIO)"
612     echo ""
613     echo ""
614
615     case "$SCENARIO" in
616         SE_PENDING)
617             warn "System Extension uninstall is pending on next reboot."
618             line "Please run 'sudo shutdown -r now' to reboot the system."
619             ;;
620         A)
621             if [ -n "$GATEWAY_TS_IP" ]; then
622                 line " writing GATEWAY_BYPASS_PING=true to .env"
623                 ENV="$PROJECT_ROOT/.env"
624                 [ -f "$ENV" ] || touch "$ENV"
625                 if grep -q '^GATEWAY_BYPASS_PING=' "$ENV" 2>/dev/null; then
626                     sed -i 's/^GATEWAY_BYPASS_PING=./GATEWAY_BYPASS_PING=true/' "$ENV"
627                 else
628                     printf '\nGATEWAY_BYPASS_PING=true\n' >> "$ENV"
629                 fi
630                 line " re-asserting tailscale exit-node (with --reset)"
631                 sudo -n tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \
632                     --exit-node="$GATEWAY_TS_IP" --exit-node-allow-lan-access=true \
633                     >> "$LOG_FILE" 2>&1 \
634                     && ok "tailscale exit-node re-asserted for $GATEWAY_TS_IP" \
635                     || warn "tailscale up did not return success (check $LOG_FILE)"
636             else
637                 warn "no gateway Tailscale IP resolved skipping tailscale up"
638                 warn "fix manually: 'sudo tailscale up --reset ... --exit-node=<TS_IP>'"
639             fi
640             ;;
641         B)
642             line " disabling IPv6 on $ACTIVE_SERVICE while gateway is up"
643             sudo -n networksetup -setv6off "$ACTIVE_SERVICE" >> "$LOG_FILE" 2>&1 \
644                 && ok "IPv6 disabled on $ACTIVE_SERVICE (re-enable with 'setv6automatic')" \
645                 || warn "networksetup -setv6off failed (check $LOG_FILE)"
646             ;;
647         C)
648             line " restarting tailscaled (route-table fix)"
649             run_with_timeout 15 brew services restart tailscale >> "$LOG_FILE" 2>&1 \
650                 || run_with_timeout 15 sudo -n brew services restart tailscale >> "$LOG_FILE" 2>&1
651             sleep 3
652             line " flushing stale host routes + DNS cache"
653             sudo -n route delete -net 0.0.0.0/1 >> "$LOG_FILE" 2>&1 || true
654             sudo -n route delete -net 128.0.0.0/1 >> "$LOG_FILE" 2>&1 || true
655             sudo -n dscacheutil -flushcache >> "$LOG_FILE" 2>&1 || true
656             sudo -n killall -HUP mDNSResponder >> "$LOG_FILE" 2>&1 || true
657             if [ -n "$GATEWAY_TS_IP" ]; then
658                 line " re-asserting exit-node after restart"
659                 sudo -n tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \
660                     --exit-node="$GATEWAY_TS_IP" --exit-node-allow-lan-access=true \
661                     >> "$LOG_FILE" 2>&1 \
662                     && ok "tailscale exit-node re-asserted for $GATEWAY_TS_IP" \
663                     || warn "tailscale up did not return success (check $LOG_FILE)"
664             fi
665             ;;
666     esac
667
668     echo ""
669     echo "Re-verifying after fix..."
670     sleep 3
671     CDN_WARP_AFTER=$(run_with_timeout 8 curl -s --max-time 6 \
672         https://www.cloudflare.com/cdn-cgi/trace 2>&1 \
673         | awk -F'=' '{warp=${print $2; exit}}')

```

```

674 IPV6_AFTER=$(run_with_timeout 7 curl -6 -s --max-time 5 ifconfig.co 2>&1 || echo "")
675 printf '\n[AFTER FIX]\n' >&3
676 printf '  warp status:      %s\n' "$CDN_WARP_AFTER" >&3
677 printf '  IPv6 egress:       %s\n' "${IPV6_AFTER:-none}" >&3
678 line "  warp status:      ${CDN_WARP_AFTER:-unknown}"
679 line "  IPv6 egress:       ${IPV6_AFTER:-none}"
680 if [ "$CDN_WARP_AFTER" = "on" ] && [ -z "$IPV6_AFTER" ]; then
681     echo ""
682     ok "Fix verified  host is now routing through WARP. Remote clients unaffected."
683 else
684     echo ""
685     warn "Fix did not fully take effect on first try. Likely causes:"
686     line "      tailscaled needed a longer settling window (re-run diagnostic in 30s)"
687     line "      The classified scenario was wrong; paste this report for triage."
688     line "      Tailscale needs a full auth refresh: 'sudo tailscale up'"
689 fi
690 fi
691
692 echo ""
693 echo ""
694 echo "  Full report written to:"
695 echo "    $REPORT_FILE"
696 echo ""
697 echo ""
698
699 #
700 # --watch mode: enter the continuous monitoring loop after baseline diagnostic
701 #
702 if [ "$DO_WATCH" = "true" ]; then
703     # Build baseline from the full diagnostic just completed
704     BASELINE_WARP="$WARP_STATUS"
705     BASELINE_DEFAULT=$(( [ "$DEFAULT_VIA_UTUN" = "true" ] && echo "utun" || echo "wifi/other" )
706
707     if [ "$SCENARIO" != "OK" ]; then
708         warn "Baseline diagnostic found issues (scenario: $SCENARIO)."
709         line "  Watch loop will still run  alerts fire on any STATE CHANGE from current baseline."
710         line "  Fix the issue first? Re-run with:  bash scripts/diagnose-host-leak.sh --fix"
711         echo ""
712     fi
713
714     run_watch_loop "$BASELINE_WARP" "$BASELINE_DEFAULT" "$IPV6_LEAK" "$WATCH_INTERVAL"
715 fi
716
717 exit 0

```

35 scripts/dns-proxy.py

```

1  #!/usr/bin/env python3
2  """
3  dns-proxy.py  Local DNS proxy for nullexit gateway.
4
5  Listens on UDP port 53, receives DNS queries, forwards them over TCP
6  to AdGuard at 127.0.0.1:5354 (the Docker port mapping), and returns
7  the response via UDP. Handles the DNS-over-TCP wire format (2-byte
8  length prefix) correctly  unlike socat which just forwards raw bytes.
9  """
10
11 import socket
12 import struct
13 import sys
14 import os
15 from concurrent.futures import ThreadPoolExecutor
16
17 LISTEN_ADDR = os.environ.get("LISTEN_ADDR", "127.0.0.1")
18 LISTEN_PORT = int(os.environ.get("LISTEN_PORT", 53))
19 TARGET_HOST = os.environ.get("TARGET_HOST", "127.0.0.1")
20 TARGET_PORT = int(os.environ.get("TARGET_PORT", 5354))
21
22
23 def handle_query(data: bytes, client_addr: tuple, sock: socket.socket) -> None:
24     """Forward a single DNS query over TCP and send the response back via UDP."""
25     try:
26         tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
27         tcp.settimeout(5)
28         tcp.connect((TARGET_HOST, TARGET_PORT))
29

```

```

30     # DNS over TCP: prepend 2-byte length (big-endian)
31     tcp.sendall(struct.pack("!H", len(data)) + data)
32
33     # Read the 2-byte response length
34     raw_len = tcp.recv(2)
35     if len(raw_len) < 2:
36         tcp.close()
37         return
38     resp_len = struct.unpack("!H", raw_len)[0]
39
40     # Read the full response
41     response = b""
42     while len(response) < resp_len:
43         chunk = tcp.recv(resp_len - len(response))
44         if not chunk:
45             break
46         response += chunk
47
48     tcp.close()
49
50     # Send the response back over UDP (strip TCP length prefix)
51     if response:
52         sock.sendto(response, client_addr)
53 except Exception:
54     # Silently drop failed queries (malformed, timeout, etc.)
55     pass
56
57
58 def main() -> None:
59     # Create UDP socket
60     sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
61     sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
62
63     try:
64         sock.bind((LISTEN_ADDR, LISTEN_PORT))
65     except PermissionError:
66         print(f"dns-proxy: ERROR need root to bind port {LISTEN_PORT}", flush=True)
67         sys.exit(1)
68     except OSError as e:
69         print(f"dns-proxy: ERROR {e}", flush=True)
70         sys.exit(1)
71
72     print(f"dns-proxy: listening on UDP {LISTEN_ADDR}:{LISTEN_PORT} TCP {TARGET_HOST}:{TARGET_PORT}",
73         flush=True)
74
75     # Use a thread pool to handle concurrent DNS queries efficiently without thread exhaustion
76     executor = ThreadPoolExecutor(max_workers=64)
77
78     while True:
79         try:
80             data, client_addr = sock.recvfrom(4096) # Support EDNS0 (up to 4096 bytes) to prevent
81             truncation
82             executor.submit(handle_query, data, client_addr, sock)
83         except KeyboardInterrupt:
84             break
85         except Exception:
86             pass
87
88     executor.shutdown(wait=False)
89     sock.close()
90
91 if __name__ == "__main__":
92     main()

```

36 scripts/dns_anomaly_detector.py

```

1  #!/usr/bin/env python3
2  """
3  nullexit DNS anomaly detector (v2)  report-only DNS-tunneling / DGA / C2 finder.
4
5  Replaces the v1 whole-FQDN Shannon-entropy + mean3 demo (which measured the
6  wrong scope, was fooled by short encoded labels, assumed a normal distribution
7  DNS names don't have, and ignored the volumetric signal that actually defines
8  tunneling). v2 keeps the "statistical, data-based, 100% local, no third-party
9  deps" spirit but detects far better:

```

```

10
11 1. n-gram (character bigram) improbability of the SUBDOMAIN labels only,
12 trained on YOUR querylog. Robust on short strings where Shannon entropy
13 fails (entropy is capped at log2(len)); scores "doesn't look like real
14 naming", which is what encoded tunneling labels are.
15 2. per-registered-domain AGGREGATION (unique-subdomain count, query volume,
16 suspicious record types) a stream of high-improbability names under ONE
17 domain is the real tunneling tell, not any single name.
18 3. robust thresholds (median + kMAD, resistant to CDN/cloud hash outliers)
19 instead of mean3 on a non-normal distribution.
20 4. a data-driven allowlist of YOUR frequent domains, so CloudFront/S3/telemetry
21 hashes don't false-positive.
22
23 Report-only by design: a privacy gateway must never silently drop DNS on a
24 statistical trip (a false positive = "my internet broke"). It surfaces findings;
25 promoting a domain to an actual block stays a human/AdGuard-blocklist decision.
26
27 Footprint: single streaming pass (never loads the log into RAM); the model is a
28 ~4040 bigram table + a few floats + a small allowlist (a few KB on disk); per-
29 domain state is a handful of ints plus a subdomain set capped at 64.
30
31 Exit codes (scan): 0 = ran, clean 1 = ran, anomalies found 2 = FAILED to run
32 (no/corrupt model, unreadable or empty querylog) a failure is never reported as
33 "clean". selftest exits 3 if the detection logic itself is broken.
34
35 Usage:
36 dns_anomaly_detector.py learn [--log PATH] [--out PATH] # train from querylog
37 dns_anomaly_detector.py scan [--log PATH] [--model PATH] [--recent N] [--quiet]
38 dns_anomaly_detector.py selftest # prove it detects (exit 3 if broken)
39 dns_anomaly_detector.py demo # human-readable showcase
40
41 import argparse
42 import json
43 import math
44 import os
45 import random
46 import re
47 import sys
48 from collections import defaultdict
49
50 DEFAULT_LOG = os.path.join(os.path.dirname(os.path.abspath(__file__)),
51                             "..", "adguard", "work", "data", "querylog.json")
52 DEFAULT_MODEL = os.path.join(os.path.dirname(os.path.abspath(__file__)),
53                               "..", ".dns_baseline.json")
54
55 ALPHABET = "abcdefghijklmnopqrstuvwxyz0123456789-_"
56 AIDX = {c: i for i, c in enumerate(ALPHABET)}
57 OTHER = len(ALPHABET) # bucket for any char outside the alphabet
58 NSYM = len(ALPHABET) + 1
59 BEG = NSYM # start-of-label marker
60 NSTATES = NSYM + 1
61
62 # Minimal two-level public suffixes so eTLD+1 grouping is stable without shipping
63 # the full ~200 KB Public Suffix List. Exact PSL isn't needed we only need a
64 # consistent grouping key per registered domain.
65 TWO_LEVEL = {
66     "co.uk", "org.uk", "ac.uk", "gov.uk", "co.jp", "co.kr", "co.in", "co.za",
67     "co.nz", "com.au", "net.au", "org.au", "com.br", "com.mx", "com.tr",
68     "com.cn", "com.hk", "com.sg", "com.tw", "com.ar", "co.il", "ne.jp",
69 }
70
71 def sym(ch):
72     return AIDX.get(ch, OTHER)
73
74
75 def registered_and_sub(qh):
76     """Split a query host into (registered_domain, subdomain_labels_string)."""
77     qh = qh.rstrip(".").lower()
78     parts = qh.split(".")
79     if len(parts) <= 2:
80         return qh, ""
81     last2 = ".".join(parts[-2:])
82     reg_n = 3 if last2 in TWO_LEVEL else 2
83     reg = ".".join(parts[-reg_n:])
84     sub = ".".join(parts[:-reg_n])
85     return reg, sub
86
87
88 def iter_querylog(path, recent=0):
89     """Stream (QH, QT) from an AdGuard querylog. recent>0 keeps only the last N."""

```

```

91     if recent > 0:
92         # Bounded tail without loading the file: keep a ring buffer of raw lines.
93         from collections import deque
94         buf = deque(maxlen=recent)
95         with open(path, "r", encoding="utf-8", errors="ignore") as f:
96             for line in f:
97                 if line.strip():
98                     buf.append(line)
99         lines = buf
100     else:
101         lines = _line_reader(path)
102     for line in lines:
103         try:
104             rec = json.loads(line)
105         except (json.JSONDecodeError, ValueError):
106             continue
107         qh = rec.get("QH")
108         if not qh:
109             continue
110         qh = qh.strip().lower()
111         if qh.endswith("in-addr.arpa") or qh.endswith("ip6.arpa"):
112             continue
113         yield qh, rec.get("QT", "")
114
115
116 def _line_reader(path):
117     with open(path, "r", encoding="utf-8", errors="ignore") as f:
118         for line in f:
119             if line.strip():
120                 yield line
121
122
123 # n-gram model
124 def label_improbability(label, logp):
125     """Mean -log2 P(char|prev) over a label. High = doesn't look like real naming."""
126     if not label:
127         return 0.0
128     prev = BEG
129     total = 0.0
130     for ch in label:
131         s = sym(ch)
132         total += logp[prev][s]
133         prev = s
134     return total / len(label)
135
136
137 def name_score(sub, logp):
138     """Worst-label improbability across the subdomain (tunneling hides in one label)."""
139     if not sub:
140         return 0.0
141     return max(label_improbability(lbl, logp) for lbl in sub.split(".")) if lbl
142
143
144 def finalize_logp(counts):
145     """Add-1 smoothed transition matrix -log2 probabilities."""
146     logp = [[0.0] * NSYM for _ in range(NSTATES)]
147     for a in range(NSTATES):
148         row_total = sum(counts[a]) + NSYM # +NSYM for add-1 smoothing
149         for b in range(NSYM):
150             p = (counts[a][b] + 1) / row_total
151             logp[a][b] = -math.log2(p)
152     return logp
153
154
155 def learn(log_path, out_path):
156     counts = [[0] * NSYM for _ in range(NSTATES)]
157     reg_counts = defaultdict(int)
158     reservoir = [] # sampled subdomain strings for threshold calibration
159     RES_MAX = 6000
160     seen = 0
161
162     for qh, _qt in iter_querylog(log_path):
163         reg, sub = registered_and_sub(qh)
164         reg_counts[reg] += 1
165         if not sub:
166             continue
167         for lbl in sub.split("."):
168             prev = BEG
169             for ch in lbl:
170                 s = sym(ch)
171                 counts[prev][s] += 1

```

```

172         prev = s
173         # Reservoir-sample subdomains (unbiased) for a robust score distribution.
174         seen += 1
175         if len(reservoir) < RES_MAX:
176             reservoir.append(sub)
177         else:
178             j = random.randint(0, seen - 1)
179             if j < RES_MAX:
180                 reservoir[j] = sub
181
182     if seen == 0:
183         print("learn: no subdomained queries found nothing to train on.", file=sys.stderr)
184         return 1
185
186     logp = finalize_logp(counts)
187
188     # Robust threshold from the score distribution: median + kMAD (MAD1.4826,
189     # so k=6 mean+4 but resistant to the CDN-hash outliers that wreck plain ).
190     scores = sorted(name_score(s, logp) for s in reservoir if s)
191     median = _median(scores)
192     mad = _median(sorted(abs(x - median) for x in scores)) or 1e-9
193     threshold = median + 6.0 * mad
194
195     # Data-driven allowlist: the user's frequent registered domains (their normal
196     # traffic, incl. their habitual CDNs) tunneling to a NEW domain isn't here.
197     top = sorted(reg_counts.items(), key=lambda kv: kv[1], reverse=True)
198     allow_cut = max(20, int(0.002 * seen)) # "frequent" = seen a lot for this net
199     allowlist = [r for r, c in top if c >= allow_cut][:400]
200
201     model = {
202         "version": 2,
203         "counts": counts, # compact ints; logp recomputed on load
204         "threshold": threshold,
205         "median": median,
206         "mad": mad,
207         "allowlist": allowlist,
208         "trained_on": seen,
209         "unique_registered": len(reg_counts),
210     }
211     with open(out_path, "w", encoding="utf-8") as f:
212         json.dump(model, f, separators=(",", ":"))
213     print(f"learn: trained on {seen} subdomained queries across "
214           f"{len(reg_counts)} registered domains.")
215     print(f" score median={median:.3f} MAD={mad:.3f} threshold={threshold:.3f}")
216     print(f" allowlist: {len(allowlist)} frequent domains {out_path} "
217           f"({os.path.getsize(out_path)} bytes)")
218     return 0
219
220
221 def _median(xs):
222     n = len(xs)
223     if n == 0:
224         return 0.0
225     xs = sorted(xs)
226     m = n // 2
227     return xs[m] if n % 2 else (xs[m - 1] + xs[m]) / 2.0
228
229
230 # scan
231 SUSPICIOUS_QT = {"TXT", "NULL", "CNAME", "ANY"} # classic tunneling data channels
232 SUB_CAP = 64 # bound per-domain memory
233
234 # High-entropy-but-legit CDN/cloud suffixes: their hostnames LOOK random but are
235 # structured, not encoded payload. A tiny static safety net beside the data-driven
236 # frequency allowlist (which only catches domains queried a lot in THIS log).
237 CDN_SUFFIXES = (
238     "nflxvideo.net", "nflxso.net", "googleusercontent.com", "usercontent.goog",
239     "cloudfront.net", "akamai.net", "akamaihd.net", "akamaiedge.net", "edgekey.net",
240     "edgesuite.net", "fastly.net", "fbcdn.net", "yting.com", "ggpht.com", "gvt1.com",
241     "gvt2.com", "ie100.net", "azureedge.net", "cloudflare.net", "cloudflarestream.com",
242     "digitaloceanspaces.com", "b-cdn.net", "llnwd.net", "wac.edgecastcdn.net", "cdn77.org",
243 )
244
245 _HEX = re.compile(r"^[0-9a-f]{16,}$")
246 _B32 = re.compile(r"^[a-z2-7]{16,}$")
247
248
249 def looks_encoded(label):
250     """True if a label looks like an encoded payload (hex/base32/long base64),
251     NOT just a random-ish structured CDN name or a long dictionary word. This is
252     the strongest single discriminator between DNS tunneling and legitimate

```

```

253 high-entropy hostnames."""
254 if _HEX.match(label):                                     # 16+ hex chars
255     return True
256 # base32 alphabet [a-z2-7] fully overlaps lowercase letters, so a long word
257 # like "visualstudioexptteam" would match require digits, which real base32
258 # payload always carries (~19% of chars) but dictionary words don't.
259 if _B32.match(label) and sum(c in "234567" for c in label) >= 2:
260     return True
261 # long base64-ish: high char diversity AND several digits (excludes words).
262 if len(label) >= 20 and len(set(label)) >= 13 and sum(c.isdigit() for c in label) >= 3:
263     return True
264 return False
265
266
267 def is_allowlisted(reg, allow):
268     return reg in allow or any(reg == s or reg.endswith("." + s) for s in CDN_SUFFIXES)
269
270
271 def scan(log_path, model_path, recent, quiet):
272     # Fail LOUD, never silently: a corrupt/absent model or an unreadable log must
273     # NOT look like a clean result that would be a security detector reporting
274     # "all clear" while blind. Each failure prints to stderr and exits non-zero.
275     try:
276         with open(model_path, "r", encoding="utf-8") as f:
277             model = json.load(f)
278             counts = model["counts"]
279             threshold = float(model["threshold"])
280             if (not isinstance(counts, list) or len(counts) != NSTATES
281                 or any(not isinstance(r, list) or len(r) != NSYM for r in counts)):
282                 raise ValueError("bigram matrix has the wrong shape")
283     except (OSError, json.JSONDecodeError, KeyError, ValueError, TypeError) as e:
284         print(f"scan: FAILED to load a usable baseline model at {model_path} "
285               f"({type(e).__name__}: {e}). Run 'dns_anomaly_detector.py learn' first.",
286               file=sys.stderr)
287         return 2
288     logp = finalize_logp(counts)
289     allow = set(model.get("allowlist", []))
290
291     agg = {} # reg -> [queries, {subdomains cap64}, sum_score, n_scored, susp_qt, maxlen, encoded]
292     for qh, qt in iter_querylog(log_path, recent=recent):
293         reg, sub = registered_and_sub(qh)
294         a = agg.get(reg)
295         if a is None:
296             a = agg[reg] = [0, set(), 0.0, 0, 0, 0, 0]
297         a[0] += 1
298         if qt in SUSPICIOUS_QT:
299             a[4] += 1
300         if sub:
301             if len(a[1]) < SUB_CAP:
302                 a[1].add(sub)
303             sc = name_score(sub, logp)
304             a[2] += sc
305             a[3] += 1
306             for l in sub.split("."):
307                 if len(l) > a[5]:
308                     a[5] = len(l)
309                 if not a[6] and looks_encoded(l):
310                     a[6] = 1
311
312     if not agg:
313         print(f"scan: FAILED the querylog at {log_path} yielded 0 parseable DNS "
314               f"records. The detector did NOT run; this is NOT a clean result.",
315               file=sys.stderr)
316         return 2
317
318     findings = []
319     for reg, (q, subs, ssum, sn, susp, maxlen, encoded) in agg.items():
320         if sn == 0 or is_allowlisted(reg, allow):
321             continue
322         mean_score = ssum / sn
323         uniq = len(subs)
324         improbable = mean_score > threshold
325         # Three independent tunneling signatures (any one flags), each tunneling-
326         # SHAPED so structured CDN names don't trip it. Encoded-payload volume is
327         # primary because encoded labels score like CDN hashes under the bigram
328         # model the improbability threshold alone can't separate them, but their
329         # SHAPE (pure hex/base32/base64) and per-domain multiplicity can:
330         strong_encoded = encoded and uniq >= 8 # many unique encoded payloads
331         txt_abuse = susp >= 5 # sustained TXT/NULL channel
332         improbable_vol = improbable and (uniq >= 8 or maxlen >= 24)
333         if strong_encoded or txt_abuse or improbable_vol:

```



```

334     corro = min(1.0, uniq / 30.0 + susp / 12.0 + encoded * 0.45 + (maxlen >= 40) * 0.2)
335     over = min(1.0, max(0.0, mean_score - threshold) / (threshold + 1e-9))
336     findings.append((round(0.55 * corro + 0.45 * over, 3), reg, round(mean_score, 2),
337                     uniq + (SUB_CAP if uniq >= SUB_CAP else 0), susp, maxlen, q))
338
339 findings.sort(reverse=True)
340 if quiet:
341     print(f"DNS anomaly scan: {len(findings)} suspicious domain(s) "
342           f"(threshold={threshold:.2f}, {len(agg)} domains scanned)")
343 else:
344     print(f"\n=== DNS anomaly scan {len(findings)} suspicious domain(s) "
345           f"of {len(agg)} scanned (threshold {threshold:.2f}) ===")
346     if findings:
347         print(f"{'conf':>5} {'registered domain':32} {'score':>6} {'uniqusub':>7} "
348               f"{'txt/null':>8} {'maxlbl':>6} {'queries':>7}")
349         for conf, reg, sc, uniq, susp, maxlen, q in findings[:25]:
350             cap = "+" if uniq > SUB_CAP else " "
351             print(f"{'conf':>5} {'reg':32.32} {'sc':>6} {'min(uniq, SUB_CAP):>6}{cap} "
352                   f"{'susp':>8} {'maxlen':>6} {'q':>7}")
353     else:
354         print(" clean no domain combined improbable naming with a volumetric signal.")
355 return 1 if findings else 0
356
357
358 # demo (self-test, no querylog needed)
359 def demo():
360     normal = ["google.com", "api.github.com", "mail.google.com", "cdn.jsdelivr.net",
361              "en.wikipedia.org", "outlook.office365.com", "i.redd.it", "apple.com",
362              "static.cloudflareinsights.com", "play.googleapis.com"] * 40
363     counts = [[0] * NSYM for _ in range(NSTATES)]
364     for qh in normal:
365         _reg, sub = registered_and_sub(qh)
366         for lbl in sub.split("."):
367             prev = BEG
368             for ch in lbl:
369                 s = sym(ch)
370                 counts[prev][s] += 1
371             prev = s
372     logp = finalize_logp(counts)
373     samples = [registered_and_sub(q)[1] for q in normal]
374     scores = sorted(name_score(s, logp) for s in samples if s)
375     med = _median(scores)
376     mad = _median(sorted(abs(x - med) for x in scores)) or 1e-9
377     thr = med + 6.0 * mad
378     print(f"demo threshold = {thr:.3f} (median {med:.3f}, MAD {mad:.3f})")
379     tests = [("api.github.com", "normal"), ("mail.google.com", "normal"),
380             ("A8F93BD72Q9X.attacker.com", "base64 tunnel"),
381             ("c732488a09b2e4f6d1.tunnel.evil.org", "hex tunnel"),
382             ("kZ9x-Qw82-Lp01-Vn55.dns.io", "high-entropy tunnel")]
383     for qh, label in tests:
384         _reg, sub = registered_and_sub(qh)
385         sc = name_score(sub, logp)
386         flag = "anomalous" if sc > thr else "normal"
387         print(f" {flag:14} score={sc:5.2f} {qh} ({label})")
388
389
390 def selftest():
391     """Prove the detection logic works end-to-end. Exit 0 = healthy, 3 = BROKEN
392     (loud). Lets sweep/CI catch a regression instead of silently trusting a
393     detector that no longer detects. Validates BOTH signal paths: the n-gram
394     language model (improbability) and the encoding-shape detector."""
395     normal = ["google.com", "api.github.com", "mail.google.com", "cdn.jsdelivr.net",
396              "en.wikipedia.org", "outlook.office365.com", "i.redd.it", "apple.com"] * 40
397     counts = [[0] * NSYM for _ in range(NSTATES)]
398     for qh in normal:
399         for lbl in registered_and_sub(qh)[1].split("."):
400             prev = BEG
401             for ch in lbl:
402                 s = sym(ch)
403                 counts[prev][s] += 1
404             prev = s
405     logp = finalize_logp(counts)
406     scores = sorted(name_score(registered_and_sub(q)[1], logp) for q in normal
407                     if registered_and_sub(q)[1])
408     med = _median(scores)
409     mad = _median(sorted(abs(x - med) for x in scores)) or 1e-9
410     thr = med + 6.0 * mad
411
412     fails = []
413     for g in ("mail.google.com", "api.github.com", "cdn.jsdelivr.net"):
414         sub = registered_and_sub(g)[1]

```

```

415         if name_score(sub, logp) > thr or any(looks_encoded(l) for l in sub.split(".")):
416             fails.append(f"false-positive on known-good '{g}'")
417         # hex/base32 encoded payloads must be caught by the SHAPE detector:
418         for enc in ("a8f93bd72c9e4f16.evil.com", "mzxw6ytb2do4zb3q7a.tunnel.io"):
419             lbl = registered_and_sub(enc)[1].split(".")[0]
420             if not looks_encoded(lbl):
421                 fails.append(f"encoding detector missed '{lbl}'")
422         # an unpronounceable non-encoded label must be caught by the n-gram model:
423         imp = registered_and_sub("xqzjvkwbmqprfxn.dga.net")[1].split(".")[0]
424         if name_score(imp, logp) <= thr:
425             fails.append(f"n-gram model missed improbable label '{imp}'")
426
427     if fails:
428         print("SELFTEST FAILED detector is not working:", file=sys.stderr)
429         for f in fails:
430             print(f" - {f}", file=sys.stderr)
431         return 3
432     print("selftest OK flags hex/base32 payloads + improbable labels, clears known-good.")
433     return 0
434
435 def main():
436     ap = argparse.ArgumentParser(description="nullexit DNS anomaly detector (report-only)")
437     sub = ap.add_subparsers(dest="cmd", required=True)
438     pl = sub.add_parser("learn"); pl.add_argument("--log", default=DEFAULT_LOG); pl.add_argument("--out",
439         default=DEFAULT_MODEL)
440     ps = sub.add_parser("scan")
441     ps.add_argument("--log", default=DEFAULT_LOG); ps.add_argument("--model", default=DEFAULT_MODEL)
442     ps.add_argument("--recent", type=int, default=0, help="scan only the last N records (0=all)")
443     ps.add_argument("--quiet", action="store_true")
444     sub.add_parser("demo")
445     sub.add_parser("selftest")
446     args = ap.parse_args()
447
448     if args.cmd == "learn":
449         if not os.path.exists(args.log):
450             print(f"learn: querylog not found at {args.log}", file=sys.stderr); return 1
451         return learn(args.log, args.out)
452     if args.cmd == "scan":
453         if not os.path.exists(args.log):
454             print(f"scan: querylog not found at {args.log}", file=sys.stderr); return 2
455         return scan(args.log, args.model, args.recent, args.quiet)
456     if args.cmd == "demo":
457         demo(); return 0
458     if args.cmd == "selftest":
459         return selftest()
460
461 if __name__ == "__main__":
462     sys.exit(main())
463

```

37 scripts/fix-docker-bridge-collision.sh

```

1  #!/bin/bash
2  # scripts/fix-docker-bridge-collision.sh nullexit fix for devref 9.13
3  #
4  # Symptom: host default route points at Docker's bridge gateway (172.17.0.1)
5  # instead of the real Wi-Fi gateway. ALL host internet egress fails. Remote
6  # tailnet clients route through the gateway correctly (their tunnel is
7  # independent of the host's broken route table), so they look healthy while
8  # the host itself cannot reach anything.
9  #
10 # Root cause: Docker's default bridge claims 172.17.0.0/16 by default. When the
11 # host's physical Wi-Fi also lands in the same /16 extremely common on
12 # university networks which hand out 172.16.0.0/12 the kernel
13 # installs Docker's bridge as the default route, and every packet bound for
14 # the internet is silently absorbed by docker0 instead of egressing.
15 #
16 # This script:
17 # 1. Detects the actual collision (Docker's 172.17.0.0/16 vs the host's
18 # Wi-Fi subnet, queried live via the ACTIVE interface not hardcoded en0)
19 # 2. Picks a non-colliding Docker bip using /8-family routing (works for
20 # 172.16.0.0/12 campus networks; naive prefix-match would pick 172.26/24
21 # which still lives INSIDE a /12 of 172.16.0.0/12)
22 # 3. Backs up ~/.colima/default/colima.yaml with an ISO-8601 timestamp
23 # 4. Edits the file in place, idempotent (replace existing docker.bip /

```

```

24 #         append new docker.bip without disturbing other keys; skip YAML comments)
25 #     5. Restarts Colima so the VM rebuilds with the new bridge subnet
26 #     6. Rebinds Wi-Fi DHCP so the host re-learns its real Wi-Fi gateway
27 #     7. Verifies the new default route is no longer the Docker bridge
28 #     8. Re-runs toggle.sh to bring the gateway back up cleanly
29 #     9. Re-runs scripts/diagnose-host-leak.sh and prints the new verdict
30 #
31 # Usage:
32 #     bash scripts/fix-docker-bridge-collision.sh           # apply the fix
33 #     bash scripts/fix-docker-bridge-collision.sh --dry    # show what would change, then exit
34 #     bash scripts/fix-docker-bridge-collision.sh --skip-toggle # fix but don't restart gateway
35 #     bash scripts/fix-docker-bridge-collision.sh --help # usage
36
37 set -uo pipefail
38 # NOTE: errexit ('set -e') is intentionally NOT enabled so intermediate
39 # 'command || warn' patterns can continue past non-fatal failures.
40 # Every critical failure (colima restart, sed, cp backup, toggle.sh)
41 # uses an explicit '|| die' so a half-applied state never escapes.
42
43 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
44 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
45 LOG_FILE="$PROJECT_ROOT/output.log"
46 source "$SCRIPT_DIR/common.sh"
47 TIMESTAMP="$(date -u +%Y%m%dT%H%M%SZ)"
48 COLIMA_YAML="$HOME/.colima/default/colima.yaml"
49 PLATFORM="$(uname -s)"
50
51 # Argument parsing
52 DRY_RUN=false
53 SKIP_TOGGLE=false
54 case "${1:-}" in
55     --dry|-n)          DRY_RUN=true ;;
56     --skip-toggle)     SKIP_TOGGLE=true ;;
57     --help|-h)
58         sed -n '28,37p' "$0"
59         exit 0
60     ;;
61     *)
62         : # default: apply
63     ;;
64 *)
65     printf 'Unknown argument: %s\n' "$1" >&2
66     exit 2
67 ;;
68 esac
69
70
71
72 # Helper: print "[dry-run] $cmd" or actually run $cmd. Wraps the redirect
73 # INSIDE the function so outer '>> $FILE' statements never accidentally
74 # append dry-run noise to the real file. (Earlier draft didn't and the
75 # dry-run polluted $COLIMA_YAML.)
76 run() {
77     if [ "$DRY_RUN" = "true" ]; then
78         printf '[dry-run] %s\n' "$*"
79         return 0
80     fi
81     printf '%s\n' "$*"
82     "$@"
83 }
84
85 # Platform-aware sed in-place. macOS BSD sed needs an empty argument;
86 # GNU sed (Linux) does not. Earlier draft used '-i '' everywhere and
87 # would have failed on Linux silently.
88 sed_inplace() {
89     local expr="$1"
90     local file="$2"
91     case "$PLATFORM" in
92         Darwin) run sed -i '' "$expr" "$file" ;;
93         *)      run sed -i "$expr" "$file" ;;
94     esac
95 }
96
97 # 0. Resolve the ACTIVE physical interface once (used everywhere)
98 # Replaces the earlier 'en0' hardcoding. Same algorithm recover.sh uses:
99 # 'route get default' if utun/tun, walk en0..en5 first with 'status: active'
100 # + 'inet' fall back to en0. This means the script also works on hosts
101 # whose primary NIC is en1/en2 (Thunderbolt Ethernet, USB-LAN, etc.).
102 resolve_active_iface() {
103     local iface
104     iface=$(route get default 2>> "$LOG_FILE" | awk '/interface:/{print $2; exit}')

```

```

105 if [ -z "$iface" ] || [ "${iface#utun}" != "$iface" ] || [ "${iface#tun}" != "$iface" ]; then
106     for i in en0 en1 en2 en3 en4 en5; do
107         if ifconfig "$i" 2>> "$LOG_FILE" | grep -q "status: active" \
108             && ifconfig "$i" 2>> "$LOG_FILE" | grep -q "inet"; then
109             iface="$i"; break
110         fi
111     done
112 fi
113 [ -z "$iface" ] && iface="en0"
114 printf '%s\n' "$iface"
115 }
116 ACTIVE_IFACE=$(resolve_active_iface)
117
118 # 1. Detect collision
119 step "1/9 Detecting Docker-bridge vs Wi-Fi subnet collision"
120
121 # Host subnet on the ACTIVE interface (not hardcoded en0).
122 HOST_SUBNET=$(ifconfig "$ACTIVE_IFACE" 2>> "$LOG_FILE" \
123     | awk '/inet /{print $2; exit}')
124 HOST_GATEWAY=$(route get default 2>> "$LOG_FILE" \
125     | awk '/gateway:/{print $2; exit}')
126 DEFAULT_DEV=$(route get default 2>> "$LOG_FILE" \
127     | awk '/interface:/{print $2; exit}')
128
129 printf '    active iface:  %s\n' "$ACTIVE_IFACE"
130 printf '    iface inet:    %s\n' "${HOST_SUBNET:-none}"
131 printf '    default via:    %s\n' "${HOST_GATEWAY:-none}"
132 printf '    default dev:    %s\n' "${DEFAULT_DEV:-none}"
133
134 # Collision fires when the host's default-route gateway is 172.17.0.1 (Docker's
135 # bridge) OR when the host subnet is in 172.17.0.0/16. The full-bridge-collision
136 # case (gateway is literally the docker bridge IP) is the smoking gun.
137 COLLISION=false
138 if [ -n "$HOST_GATEWAY" ] && [ "${HOST_GATEWAY#172.17.}" != "$HOST_GATEWAY" ]; then
139     COLLISION=true
140     fail "default gateway $HOST_GATEWAY collides with Docker's default bridge (172.17.0.0/16)"
141 elif [ -n "$HOST_SUBNET" ] && [ "${HOST_SUBNET#172.17.}" != "$HOST_SUBNET" ]; then
142     COLLISION=true
143     fail "host subnet $HOST_SUBNET overlaps Docker's 172.17.0.0/16 bridge"
144 else
145     ok "no collision detected between host Wi-Fi and Docker's default bridge"
146 fi
147
148 if [ "$COLLISION" = "false" ]; then
149     echo ""
150     echo "Nothing to fix. If you still see egress issues, run:"
151     echo "    bash scripts/diagnose-host-leak.sh"
152     exit 0
153 fi
154
155 # 2. Pick non-colliding bip via /8-family routing
156 step "2/9 Picking a non-colliding Docker bip for ~/.colima/default/colima.yaml"
157
158 # Earlier draft tried naive prefix-match: "if candidate's 3-octet prefix is
159 # not a literal prefix of host gateway, pick it". That breaks on /12 campus
160 # networks (172.16.0.0/12 covers 172.16-172.31): a candidate like
161 # 172.26.0.1/24 LIVES INSIDE the host's /12 even though the prefix test
162 # passes. Fix: classify the host's address family FIRST, then choose ALL
163 # candidates from a DIFFERENT RFC1918 family. /8 boundaries are coarse
164 # enough that no /24 picked this way can ever collide.
165 case "$HOST_GATEWAY" in
166     172.16.*|172.17.*|172.18.*|172.19.*|172.20.*|172.21.*|172.22.*|172.23.*|
167     172.24.*|172.25.*|172.26.*|172.27.*|172.28.*|172.29.*|172.30.*|172.31.*)
168         HOST_FAMILY="172"
169         # Host is in 172.x jump past 172.31 entirely. Pick from 10.x / 192.168.x.
170         CANDIDATES="10.200.0.1/24 10.201.0.1/24 192.168.99.1/24"
171         ;;
172     10.*)
173         HOST_FAMILY="10"
174         # Host is in 10.x pick from 172.x (well outside any commonly-deployed
175         # 10.0.0.0/8 slice) / 192.168.x.
176         CANDIDATES="172.26.0.1/24 172.28.0.1/24 192.168.99.1/24"
177         ;;
178     192.168.*)
179         HOST_FAMILY="192"
180         CANDIDATES="172.26.0.1/24 10.200.0.1/24 10.201.0.1/24"
181         ;;
182     *)
183         HOST_FAMILY="unknown"
184         warn "host gateway $HOST_GATEWAY is not in any RFC1918 range (/8 family)"
185         warn "falling back to abstract candidate set; verify collision-free manually"

```

```

186 CANDIDATES="172.26.0.1/24 10.200.0.1/24 192.168.99.1/24"
187 ;;
188 esac
189
190 printf ' host address family: %s\n' "$HOST_FAMILY"
191 printf ' candidate pool: %s\n' "$CANDIDATES"
192
193 # Sanity-check the chosen pool has at least one entry that doesn't literally
194 # share a /24 with the host gateway. Naive check (just /24 prefix) is fine
195 # here because we already picked across families.
196 CHOSEN=""
197 for cand in $CANDIDATES; do
198     PREFIX3=$(printf '%s' "$cand" | cut -d. -f1-3)
199     if [ -n "$HOST_GATEWAY" ] && [ "${HOST_GATEWAY#"$PREFIX3"}" != "$HOST_GATEWAY" ]; then
200         warn "skip $cand (literal /24 collision with host gateway $HOST_GATEWAY)"
201         continue
202     fi
203     CHOSEN="$cand"
204     ok "chose bip=$CHOSEN (cross-family, no /24 literal collision)"
205     break
206 done
207
208 if [ -z "$CHOSEN" ]; then
209     CHOSEN="172.26.0.1/24"
210     warn "could not find a clean candidate; defaulting to $CHOSEN"
211     warn "verify with 'route get default' after restart"
212 fi
213
214 # 3. Backup colima.yaml
215 step "3/9 Backing up $COLIMA_YAML"
216 mkdir -p "$(dirname "$COLIMA_YAML")"
217 BACKUP="$HOME/.colima/default/colima.yaml.bak.$TIMESTAMP"
218 if [ -f "$COLIMA_YAML" ]; then
219     # 'cp' is safe in dry-run because run() suppresses the inner command
220     # entirely no chance of accidentally writing a real backup.
221     run cp -p "$COLIMA_YAML" "$BACKUP" || die "backup failed"
222     if [ "$DRY_RUN" = "false" ]; then ok "backup: $BACKUP"; fi
223 else
224     warn "no existing $COLIMA_YAML (will create fresh)"
225 fi
226
227 # 4. Edit colima.yaml in place (idempotent)
228 step "4/9 Setting docker.bip=$CHOSEN in colima.yaml (in place)"
229
230 # Resolve the existing docker.bip (if any) without picking up commented-out
231 # YAML keys. awk-based extraction; the negated-comment check ('! /^[[:space:]]*#')
232 # ensures '# bip: 1.2.3.4/24' is treated as a comment, not the active value.
233 EXISTING_BIP=""
234 if [ -f "$COLIMA_YAML" ]; then
235     EXISTING_BIP=$(awk '
236         /^[[:space:]]*#/{next}
237         /^[[:space:]]*bip:[[:space:]]*/{print $2; exit}
238     ' "$COLIMA_YAML" 2>> "$LOG_FILE" || true)
239 fi
240
241 if [ -n "$EXISTING_BIP" ] && [ "$EXISTING_BIP" = "$CHOSEN" ]; then
242     ok "colima.yaml already has docker.bip=$CHOSEN no edit needed"
243 elif [ -n "$EXISTING_BIP" ]; then
244     printf ' (replacing existing docker.bip=%s with %s)\n' "$EXISTING_BIP" "$CHOSEN"
245     # Use sed_inplace so Linux / macOS get the right syntax. Skip YAML comment
246     # lines so existing '# bip:' comments aren't rewired.
247     sed_inplace "s|^\([[:space:]]*bip:[[:space:]]*\).*\${1}$|${1}$CHOSEN|" "$COLIMA_YAML" \
248     || die "sed replacement failed (colima.yaml malformed?)"
249     ok "colima.yaml updated"
250 else
251     # No existing 'bip:' key. Branch:
252     # - file has 'docker:' block append ' bip:' under it
253     # - file has no 'docker:' block append new docker: section
254     # - file does not exist create fresh
255     # Each branch is dry-run-aware so no branch can accidentally leak noise
256     # into $COLIMA_YAML during a preview.
257     HAS_DOCKER_BLOCK=false
258     if [ -f "$COLIMA_YAML" ] && grep -q '^docker:' "$COLIMA_YAML"; then
259         HAS_DOCKER_BLOCK=true
260     fi
261
262     if [ "$HAS_DOCKER_BLOCK" = "true" ]; then
263         if [ "$DRY_RUN" = "true" ]; then
264             printf ' [dry-run] would append to %s:\n bip: %s\n' "$COLIMA_YAML" "$CHOSEN"
265         else
266             printf '\n bip: %s\n' "$CHOSEN" >> "$COLIMA_YAML" \

```

```

267     || die "append failed"
268     ok "appended bip under existing docker: block"
269 fi
270 elif [ -f "$COLIMA_YAML" ]; then
271     if [ "$DRY_RUN" = "true" ]; then
272         printf '        [dry-run] would append to %s:\\ndocker:\\n  bip: %s\\n' "$COLIMA_YAML" "$CHOSEN"
273     else
274         printf '\\ndocker:\\n  bip: %s\\n' "$CHOSEN" >> "$COLIMA_YAML" \\
275         || die "append failed"
276         ok "appended new docker: block at end of file"
277     fi
278 else
279     if [ "$DRY_RUN" = "true" ]; then
280         printf '        [dry-run] would create %s with:\\ndocker:\\n  bip: %s\\n' "$COLIMA_YAML" "$CHOSEN"
281     else
282         printf '\\docker:\\n  bip: %s\\n' "$CHOSEN" > "$COLIMA_YAML" \\
283         || die "create failed"
284         ok "created fresh $COLIMA_YAML with docker.bip=$CHOSEN"
285     fi
286 fi
287 fi
288
289 # Always show the final colima.yaml docker section for visibility
290 printf '\\n\\b  colima.yaml 'docker:' section now reads:%b\\n' "$BOLD" "$NC"
291 awk '
292 /~docker:/{p=1}
293 p && /^[^ ]/ && !/~docker:/{exit}
294 p
295 ' "$COLIMA_YAML" 2>/dev/null | sed 's/~ / /' || true
296
297 if [ "$DRY_RUN" = "true" ]; then
298     printf '\\n  \\b(dry-run: exiting before colima restart / dhcp rebind / toggle.sh)%b\\n' "$YELLOW" "$NC"
299     exit 0
300 fi
301
302 # 5. Restart Colima
303 step "5/9 Restarting Colima VM with new bip"
304 if ! command -v colima >> "$LOG_FILE" 2>&1; then
305     die "colima not on PATH  install with 'brew install colima' first"
306 fi
307 # No timeout wrapper (would mask actual VM startup time). If colima hangs,
308 # the user kills with Ctrl+C and can manually 'colima restart' from a fresh
309 # terminal the rest of this script is idempotent so re-running works.
310 colima restart >> "$LOG_FILE" 2>&1 || die "colima restart failed; check $LOG_FILE"
311 ok "colima restarted"
312
313 # 6. Rebind DHCP on the ACTIVE interface (not hardcoded en0)
314 step "6/9 Rebinding DHCP on $ACTIVE_IFACE so host re-learns its real gateway"
315
316 # Map ifname macOS network service name (Wi-Fi, USB 10/100 LAN, etc.).
317 # Same logic as the diagnostic + toggle.sh.
318 ACTIVE_SVC=""
319 if [ "$PLATFORM" = "Darwin" ]; then
320     ACTIVE_SVC=$(get_active_service)
321 fi
322
323 case "$PLATFORM" in
324     Darwin)
325         sudo -n networksetup -setdhcp "$ACTIVE_SVC" renew >> "$LOG_FILE" 2>&1 \\
326         && ok "DHCP rebind issued on $ACTIVE_SVC" \\
327         || warn "setdhcp renew failed; try 'sudo ipconfig set $ACTIVE_IFACE DHCP' manually"
328         ;;
329     Linux)
330         sudo -n dhclient -r "$ACTIVE_IFACE" >> "$LOG_FILE" 2>&1 || true
331         sudo -n dhclient "$ACTIVE_IFACE" >> "$LOG_FILE" 2>&1 \\
332         && ok "dhclient rebind on $ACTIVE_IFACE" \\
333         || warn "dhclient rebind failed"
334         ;;
335     *)
336         warn "unsupported platform $PLATFORM; skip DHCP rebind"
337         ;;
338 esac
339
340 # 7. Verify new default route
341 step "7/9 Verifying the new default route is NOT the Docker bridge"
342 sleep 2 # give DHCP + kernel a beat
343 NEW_DEFAULT=$(netstat -rn 2>> "$LOG_FILE" | awk '/^default/ {print $0; exit}')
344 printf '  %s\\n' "${NEW_DEFAULT:-no default route detected}"
345 NEW_GATEWAY=$(printf '%s' "$NEW_DEFAULT" | awk '{print $2; exit}')
346
347 if [ -n "$NEW_GATEWAY" ] && [ "${NEW_GATEWAY#172.17.} " != "$NEW_GATEWAY" ]; then

```

```

348 fail "default route STILL points at 172.17.x.x DHCP rebind may need manual touch"
349 warn "try: sudo ipconfig set $ACTIVE_IFACE DHCP (or reconnect Wi-Fi in the menu bar)"
350 elif [ -n "$NEW_GATEWAY" ]; then
351 ok "default route is now via $NEW_GATEWAY (no longer Docker's bridge)"
352 else
353 warn "could not determine new gateway; verify with 'route get default'"
354 fi
355
356 # 8. Re-run toggle.sh to bring the gateway back up
357 if [ "$SKIP_TOGGLE" = "true" ]; then
358 step "8/9 Skipping toggle.sh (--skip-toggle)"
359 warn "gateway is currently down run ./toggle.sh manually when ready"
360 else
361 step "8/9 Re-running toggle.sh to bring the gateway back up"
362 if bash "$PROJECT_ROOT/toggle.sh" 2>&1 | tee -a "$LOG_FILE"; then
363 ok "toggle.sh completed"
364 else
365 warn "toggle.sh returned non-zero see $LOG_FILE for details"
366 fi
367 fi
368
369 # 9. Re-run the diagnostic + show verdict
370 step "9/9 Re-running host-leak diagnostic to confirm fix"
371 if bash "$SCRIPT_DIR/diagnose-host-leak.sh" 2>&1 | tee -a "$LOG_FILE"; then
372 LATEST_REPORT=$(ls -t "$PROJECT_ROOT"/host-leak-diagnostic-*.txt 2>/dev/null | head -1)
373 if [ -n "$LATEST_REPORT" ]; then
374 VERDICT=$(awk '/^ Scenario:/ {print $2; exit}' "$LATEST_REPORT" 2>/dev/null)
375 WARP=$(awk -F: '*/^ WARP egress:/ {print $2; exit}' "$LATEST_REPORT" 2>/dev/null)
376 printf '\n Most recent verdict: Scenario=%s WARP=%s\n' \
377 "${VERDICT:-?}" "${WARP:-?}"
378 if [ "$VERDICT" = "OK" ]; then
379 ok "fix confirmed by diagnostic host traffic is going through WARP"
380 fi
381 fi
382 else
383 warn "diagnostic exited non-zero; verify manually"
384 fi
385
386 printf '\n%b\n' "$BOLD" "$NC"
387 printf '%bDone.%b Full transcript at %s\n' "$BOLD" "$NC" "$LOG_FILE"
388 printf '%b\n' "$BOLD" "$NC"
389 exit 0

```

38 scripts/fix-ssh-delay.sh

```

1 #!/bin/bash
2 # scripts/fix-ssh-delay.sh Fix ~20s SSH connection delay caused by UseDNS
3 #
4 # macOS OpenSSH defaults UseDNS to "yes". When Tailscale SSH connects from
5 # a peer (phone/laptop), sshd performs a reverse-DNS (PTR) lookup on the
6 # client's 100.x.x.x Tailscale IP. Since DNS is hijacked to the nullexit
7 # gateway AdGuard Cloudflare WARP, and 100.64.0.0/10 is CGNAT private
8 # space, the PTR query times out (~15-20s) before SSH proceeds.
9 #
10 # This script uncomments "UseDNS no" in /etc/ssh/sshd_config and reloads
11 # sshd. Existing SSH sessions are NOT dropped.
12 #
13 # Usage:
14 # sudo bash scripts/fix-ssh-delay.sh
15
16 set -euo pipefail
17
18 SSHD_CONFIG="/etc/ssh/sshd_config"
19
20 # Check we're root
21 if [ "$(id -u)" -ne 0 ]; then
22 echo "ERROR: This script must be run with sudo (it edits $SSHD_CONFIG)"
23 echo "Usage: sudo bash scripts/fix-ssh-delay.sh"
24 exit 1
25 fi
26
27 # Apply config (idempotent)
28 if grep -qE '^UseDNS no' "$SSHD_CONFIG" 2>/dev/null; then
29 echo "UseDNS is already set to 'no' in $SSHD_CONFIG"
30 elif grep -qE '^#UseDNS' "$SSHD_CONFIG" 2>/dev/null; then
31 echo "Uncommenting 'UseDNS no' in $SSHD_CONFIG"

```



```

32     sed -i '' 's/^#UseDNS no/UseDNS no/' "$SSHD_CONFIG"
33 elif grep -qEi '^UseDNS' "$SSHD_CONFIG" 2>/dev/null; then
34     echo " Changing existing UseDNS line to 'UseDNS no'"
35     sed -i '' 's/^UseDNS.*/UseDNS no/' "$SSHD_CONFIG"
36 else
37     echo " Appending 'UseDNS no' to $SSHD_CONFIG"
38     echo "" >> "$SSHD_CONFIG"
39     echo "UseDNS no" >> "$SSHD_CONFIG"
40 fi
41
42 # Verify
43 if ! grep -qE '^UseDNS no' "$SSHD_CONFIG"; then
44     echo " Failed to apply UseDNS no check $SSHD_CONFIG manually"
45     exit 1
46 fi
47
48 # Reload sshd (does NOT drop existing connections)
49 echo " Reloading sshd..."
50
51 SSHD_PID=$(pgrep -f /usr/sbin/sshd 2>/dev/null | head -1 || true)
52 if [ -n "$SSHD_PID" ]; then
53     kill -HUP "$SSHD_PID"
54     echo " Sent SIGHUP to sshd (PID $SSHD_PID) config reloaded"
55 else
56     echo " No running sshd found. macOS starts sshd on-demand per connection."
57     echo " The new 'UseDNS no' config will take effect on the next SSH connection."
58 fi
59
60 echo ""
61 echo " Done. SSH connections should now connect instantly."
62 echo " Existing sessions were not interrupted."

```

39 scripts/generate_tex.py

```

1  #!/usr/bin/env python3
2  import os
3
4  PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))
5
6  IGNORE_DIRS = {'.git', '.github', 'adguard', '__pycache__', 'Recover Gateway.app', 'Toggle Gateway.app',
7  'Sweep Gateway.app', 'nullexit-knowledge-graph'}
8  IGNORE_FILES = {'.env', 'nullexit-unified.tex', 'nullexit-unified.pdf', 'output.log', 'blocked.log', '.DS_Store',
9  '.lan_p2p_detected', '.signatures', '.gateway_ip', 'TUNNEL_FAILED_CLOSED.marker', '.host_ips', 'refactor.md', 'sweep.md',
10 '.build_hash', '.rules_hash', 'knowledge-graph.html', '.dns_baseline.json'}
11 IGNORE_EXTS = {'.pdf', '.tex', '.png', '.jpg', '.jpeg', '.zip', '.tar', '.gz', '.log', '.aux', '.fls', '.fdb_latexmk',
12 '.out', '.toc', '.xdv', '.synctex(busy)'}
13 # Transient runtime reports (timestamped names) carry live egress/peer IPs never
14 # embed them in the published quine. Matched by filename prefix.
15 IGNORE_PREFIXES = ('sweep-', 'host-leak-diagnostic-')
16
17 # Exclude from code block inclusion, but keep in tree
18 SKIP_CONTENT_FILES = {'LICENSE'}
19
20 PRIORITY_FILES = [
21     'README.md',
22     'agent.md',
23     'devref.md',
24     'diagrams.md',
25     'toggle.sh',
26     'recover.sh',
27     'setup.sh',
28     'scripts/setup-linux.sh',
29     'docker-compose.yml',
30     'scripts/common.sh',
31     'scripts/watcher.sh',
32     'scripts/crypto.sh'
33 ]
34
35 def get_language(filename):
36     ext = os.path.splitext(filename)[1].lower()
37     if ext == '.sh' or ext == '.applescript':
38         return 'bash'
39     elif ext == '.py':
40         return 'Python'
41     elif ext == '.go':

```



```

117 \section{Project Directory Tree}
118 \begin{verbatim}
119 nullexit/
120 """
121
122 tree = generate_tree(PROJECT_ROOT)
123 midamble = r"""\end{verbatim}
124 \newpage
125 """
126
127 all_relpaths = []
128 for root, dirs, files in os.walk(PROJECT_ROOT):
129     dirs[:] = sorted([d for d in dirs if d not in IGNORE_DIRS])
130     for file in sorted(files):
131         if file in IGNORE_FILES or file in SKIP_CONTENT_FILES or os.path.splitext(file)[1].lower() in
IGNORE_EXTS:
132             continue
133         if file.startswith(IGNORE_PREFIXES):
134             continue
135
136         filepath = os.path.join(root, file)
137         relpath = os.path.relpath(filepath, PROJECT_ROOT)
138         all_relpaths.append(relpath)
139
140 def sort_key(path):
141     if path in PRIORITY_FILES:
142         return (0, PRIORITY_FILES.index(path))
143     return (1, path)
144
145 all_relpaths.sort(key=sort_key)
146
147 with open(tex_path, 'w', encoding='utf-8') as f:
148     f.write(preamble)
149     f.write(tree)
150     f.write(midamble)
151
152     for relpath in all_relpaths:
153         lang = get_language(relpath)
154         lang_attr = f"[language={lang}]" if lang else ""
155
156         f.write(f"\\section{{{escape_tex(relpath)}}}\n")
157         f.write(f"\\begin{{{lstlisting}}}{lang_attr}\n")
158         try:
159             with open(os.path.join(PROJECT_ROOT, relpath), 'r', encoding='utf-8', errors='ignore') as
src:
160                 content = src.read()
161                 import re
162                 # Strip all non-ASCII characters (emojis, box drawings, em dashes, etc)
163                 # AND ASCII control characters except \t\n (binary payloads, ANSI escapes,
164                 # form feeds all invisible, all fatal to LaTeX compilation)
165                 content = re.sub(r'[\x09\x0A\x20-\x7E]+' , '' , content)
166                 f.write(content)
167                 if not content.endswith('\n'):
168                     f.write('\n')
169             except Exception as e:
170                 f.write(f"Error reading file: {e}\n")
171                 f.write("\\end{lstlisting}\n\n")
172
173         f.write("\\end{document}\n")
174
175 print(f"Generated {tex_path}")
176
177 if __name__ == '__main__':
178     main()

```

40 scripts/logger/go.mod

```

1 module nullexit/logger
2
3 go 1.22

```

41 scripts/logger/main.go

```

1 package main
2
3 import (
4     "bufio"
5     "encoding/json"
6     "fmt"
7     "io"
8     "log"
9     "os"
10    "regexp"
11    "strings"
12    "time"
13 )
14
15 const (
16     queryLogPath = "/adguard_work/data/querylog.json"
17     procKmsgPath  = "/proc/kmsg"
18     outputLogPath = "/app/blocked.log"
19 )
20
21 func logMessage(msg string) {
22     timestamp := time.Now().Format("2006-01-02 15:04:05")
23     formattedMsg := fmt.Sprintf("%s %s\n", timestamp, msg)
24     fmt.Print(formattedMsg)
25
26     f, err := os.OpenFile(outputLogPath, os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)
27     if err != nil {
28         log.Printf("Error writing to log file: %v\n", err)
29         return
30     }
31     defer f.Close()
32     f.WriteString(formattedMsg)
33 }
34
35 func tailFile(filepath string, out chan<- string) {
36     for {
37         if _, err := os.Stat(filepath); os.IsNotExist(err) {
38             time.Sleep(1 * time.Second)
39             continue
40         }
41         break
42     }
43
44     f, err := os.Open(filepath)
45     if err != nil {
46         log.Printf("Error opening file %s: %v\n", filepath, err)
47         return
48     }
49     defer func() {
50         if f != nil {
51             f.Close()
52         }
53     }()
54
55     // Seek to end
56     f.Seek(0, io.SeekEnd)
57     reader := bufio.NewReader(f)
58
59     for {
60         line, err := reader.ReadString('\n')
61         if err != nil {
62             if err == io.EOF {
63                 stat, err := os.Stat(filepath)
64                 if err == nil {
65                     currentOffset, _ := f.Seek(0, io.SeekCurrent)
66                     if stat.Size() < currentOffset {
67                         // File truncated or rotated
68                         f.Close()
69                         for {
70                             if _, err := os.Stat(filepath); os.IsNotExist(err) {
71                                 time.Sleep(1 * time.Second)
72                                 continue
73                             }
74                             break
75                         }
76                     }
77                     f, _ = os.Open(filepath)
78                     reader = bufio.NewReader(f)
79                     continue
80                 }
81             }

```

```

81         time.Sleep(500 * time.Millisecond)
82         continue
83     }
84     log.Printf("Error reading file %s: %v\n", filepath, err)
85     time.Sleep(1 * time.Second)
86     continue
87 }
88 out <- line
89 }
90 }
91
92 type adguardLogEntry struct {
93     IP      string `json:"IP"`
94     QH      string `json:"QH"`
95     QT      string `json:"QT"`
96     Result  struct {
97         IsFiltered bool `json:"IsFiltered"`
98         Reason      int  `json:"Reason"`
99         Rules       []struct {
100             Text string `json:"Text"`
101         } `json:"Rules"`
102     } `json:"Result"`
103 }
104
105 func monitorDNS() {
106     logMessage("[System] Starting DNS block logger...")
107     lines := make(chan string)
108     go tailFile(queryLogPath, lines)
109
110     reasonMap := map[int]string{
111         2: "CustomRule",
112         3: "BlockList",
113         4: "SafeBrowsing",
114         5: "ParentalControl",
115         6: "SafeSearch",
116         7: "BlockedService",
117     }
118
119     for line := range lines {
120         var data adguardLogEntry
121         if err := json.Unmarshal([]byte(line), &data); err != nil {
122             continue
123         }
124
125         res := data.Result
126         if res.IsFiltered || (res.Reason != 0 && res.Reason != 1) {
127             qh := data.QH
128             if qh == "" {
129                 qh = "unknown"
130             }
131             qt := data.QT
132             if qt == "" {
133                 qt = "unknown"
134             }
135             clientIP := data.IP
136             if clientIP == "" {
137                 clientIP = "unknown"
138             }
139
140             ruleText := "unknown rule"
141             if len(res.Rules) > 0 && res.Rules[0].Text != "" {
142                 ruleText = res.Rules[0].Text
143             }
144
145             reasonStr, ok := reasonMap[res.Reason]
146             if !ok {
147                 reasonStr = fmt.Sprintf("Reason-%d", res.Reason)
148             }
149
150             logMessage(fmt.Sprintf("[DNS] Blocked %s (Type: %s) for client %s | Reason: %s | Rule: %s", qh, qt,
151                 clientIP, reasonStr, ruleText))
152         }
153     }
154 }
155
156 func monitorIPs() {
157     logMessage("[System] Starting IP block logger...")
158
159     prefixRe := regexp.MustCompile(`(IP_BLOCK_[A-Z_]+):`)
160     srcRe := regexp.MustCompile(`SRC=([0-9a-fA-F\.:]+)`)
161     dstRe := regexp.MustCompile(`DST=([0-9a-fA-F\.:]+)`)

```

```

161 protoRe := regexp.MustCompile(`PROTO=([A-Z0-9]+)`)
162 sptRe := regexp.MustCompile(`SPT=(\d+)`)
163 dptRe := regexp.MustCompile(`DPT=(\d+)`)
164 inRe := regexp.MustCompile(`IN=([a-zA-Z0-9\.\-_\+])`)
165 outRe := regexp.MustCompile(`OUT=([a-zA-Z0-9\.\-_\+])`)
166
167 f, err := os.Open(procKmsgPath)
168 if err != nil {
169     if os.IsPermission(err) {
170         logMessage("[System] ERROR: Insufficient permissions to read /proc/kmsg. Enable CAP_SYSLOG.")
171     } else {
172         logMessage(fmt.Sprintf("[System] ERROR in IP logger: %v", err))
173     }
174     return
175 }
176 defer f.Close()
177
178 reader := bufio.NewReader(f)
179 for {
180     line, err := reader.ReadString('\n')
181     if err != nil {
182         if err == io.EOF {
183             time.Sleep(100 * time.Millisecond)
184             continue
185         }
186         logMessage(fmt.Sprintf("[System] ERROR reading kmsg: %v", err))
187         time.Sleep(1 * time.Second)
188         continue
189     }
190
191     if strings.Contains(line, "IP_BLOCK") {
192         match := prefixRe.FindStringSubmatch(line)
193         if len(match) > 1 {
194             prefix := match[1]
195
196             src := "unknown"
197             if m := srcRe.FindStringSubmatch(line); len(m) > 1 {
198                 src = m[1]
199             }
200
201             dst := "unknown"
202             if m := dstRe.FindStringSubmatch(line); len(m) > 1 {
203                 dst = m[1]
204             }
205
206             proto := "unknown"
207             if m := protoRe.FindStringSubmatch(line); len(m) > 1 {
208                 proto = m[1]
209             }
210
211             spt := ""
212             if m := sptRe.FindStringSubmatch(line); len(m) > 1 {
213                 spt = m[1]
214             }
215
216             dpt := ""
217             if m := dptRe.FindStringSubmatch(line); len(m) > 1 {
218                 dpt = m[1]
219             }
220
221             inIf := ""
222             if m := inRe.FindStringSubmatch(line); len(m) > 1 {
223                 inIf = m[1]
224             }
225
226             outIf := ""
227             if m := outRe.FindStringSubmatch(line); len(m) > 1 {
228                 outIf = m[1]
229             }
230
231             direction := "inbound"
232             if strings.HasSuffix(prefix, "DST") {
233                 direction = "outbound"
234             }
235
236             listType := "MALICIOUS"
237             if !strings.Contains(prefix, "MALICIOUS") {
238                 parts := strings.Split(prefix, "_")
239                 if len(parts) >= 3 {
240                     listType = "GEO_" + parts[2]
241                 }

```

```

242     }
243
244     portStr := ""
245     if dpt != "" {
246         portStr = ":" + dpt
247     }
248
249     srcPortStr := ""
250     if spt != "" {
251         srcPortStr = ":" + spt
252     }
253
254     ifStr := ""
255     if inIf != "" && outIf != "" {
256         ifStr = fmt.Sprintf("IF: %s->%s", inIf, outIf)
257     } else if inIf != "" || outIf != "" {
258         ifStr = fmt.Sprintf("IF: %s%s", inIf, outIf)
259     }
260
261     logMessage(fmt.Sprintf("[IP] Blocked %s to %s%s (Proto: %s) from %s%s (%s) | List: %s", direction
262         , dst, portStr, proto, src, srcPortStr, ifStr, listType))
263     }
264 }
265 }
266
267 func main() {
268     logMessage("[System] Nullexit Block Logger starting...")
269
270     go monitorDNS()
271     monitorIPs()
272 }

```

42 scripts/noise.sh

```

1  #!/bin/bash
2  # scripts/noise.sh  nullexit verifiable-random noise + cover-traffic padding
3  #
4  # OPT-IN and ACTIVE. Unlike sweep.sh (read-only), this module PUTS TRAFFIC ON
5  # THE WIRE. It stays completely inert unless NOISE_ENABLED=true in .env, so the
6  # default posture is unchanged and the sweep's read-only guarantee is intact.
7  #
8  # Two jobs, one primitive (CSPRNG bytes from the OS, /dev/urandom via os.urandom):
9  #
10 # 1. verify prove a freshly generated buffer is statistically random using
11 #    three standard tests (Shannon entropy, NIST monobit, chi-square over the
12 #    256 byte values). This is the "we can verify it's truly random
13 #    mathematically" requirement it is harmless and always allowed.
14 #
15 # 2. pad a cover-traffic daemon. It emits verified-random UDP datagrams
16 #    toward the exit at a configured rate + jitter so a passive on-path
17 #    observer (residence / campus / hostile network) sees a continuously
18 #    varying encrypted flow instead of your real traffic envelope. This is
19 #    the "dummy padding" defence the Threat Model flags as the countermeasure
20 #    to metadata / traffic-flow correlation. It ONLY runs with NOISE_ENABLED=true.
21 #
22 # HONEST LIMITS: this is constant/random one-way padding, not a full
23 # traffic-analysis-resistant shaper. It blurs volume + timing on the uplink; it
24 # does not mimic a specific protocol, pad bidirectionally, or defeat a global
25 # active adversary. It costs bandwidth and (on battery devices) power hence
26 # opt-in, modest default rate. The DNS/IP anomaly detector is deliberately NOT
27 # here (left for later).
28 #
29 # Usage:
30 #   bash scripts/noise.sh verify [BYTES]    # randomness self-test (default 1 MiB)
31 #   bash scripts/noise.sh start              # launch the padding daemon (background)
32 #   bash scripts/noise.sh pad                # run the padding loop in the foreground
33 #   bash scripts/noise.sh status             # is the daemon running? target + rate
34 #   bash scripts/noise.sh stop               # stop the daemon
35 #   bash scripts/noise.sh --help            # this message
36 #
37 # Persistence: install launchd/com.nullexit.noise.plist (see README) so padding
38 # auto-starts at boot/login. It is governed by the SAME NOISE_ENABLED switch no
39 # extra knob: the launchd job is a clean no-op whenever NOISE_ENABLED=false.
40 #
41 # .env knobs (all optional; sane defaults):

```



```

42 # NOISE_ENABLED          master on/off switch (default false)
43 # NOISE_PAD_TARGET       where to aim padding (default: the gateway Tailscale IP)
44 # NOISE_PAD_PORT         UDP port on the target (default 9999; no listener needed)
45 # NOISE_PAD_RATE_KBPS    average padding bitrate (default 64)
46 # NOISE_PAD_PACKET_BYTES datagram payload size (default 1024)
47 # NOISE_PAD_JITTER_MS    +/- timing jitter per packet (default 40)
48
49 set -uo pipefail
50
51 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
52 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
53 cd "$PROJECT_ROOT" || { echo "[FATAL] cannot cd to $PROJECT_ROOT" >&2; exit 1; }
54 source "$SCRIPT_DIR/common.sh"
55 export PATH="/opt/homebrew/bin:/usr/local/bin:$PATH"
56
57 SELF="$SCRIPT_DIR/noise.sh"
58 NOISE_PID="/tmp/nullexit-noise.pid"
59 NOISE_LOG="$PROJECT_ROOT/output.log"
60
61 # .env-driven config
62 NOISE_ENABLED=$(read_env_var "NOISE_ENABLED" | tr '[:upper:]' '[:lower:]')
63 [ -z "$NOISE_ENABLED" ] && NOISE_ENABLED="false"
64
65 PAD_TARGET=$(read_env_var "NOISE_PAD_TARGET")
66 [ -z "$PAD_TARGET" ] && PAD_TARGET=$(read_adguard_ip) # gateway Tailscale IP
67 PAD_PORT=$(read_env_var "NOISE_PAD_PORT"); [ -z "$PAD_PORT" ] && PAD_PORT=9999
68 PAD_RATE=$(read_env_var "NOISE_PAD_RATE_KBPS"); [ -z "$PAD_RATE" ] && PAD_RATE=64
69 PAD_BYTES=$(read_env_var "NOISE_PAD_PACKET_BYTES"); [ -z "$PAD_BYTES" ] && PAD_BYTES=1024
70 PAD_JITTER=$(read_env_var "NOISE_PAD_JITTER_MS"); [ -z "$PAD_JITTER" ] && PAD_JITTER=40
71
72 log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] [noise] $*" >> "$NOISE_LOG" 2>/dev/null || true; }
73
74 require_enabled() {
75     if [ "$NOISE_ENABLED" != "true" ]; then
76         echo "noise: disabled (NOISE_ENABLED != true in .env) refusing to emit traffic." >&2
77         echo "Set NOISE_ENABLED=true to arm cover-traffic padding." >&2
78         exit 3
79     fi
80 }
81
82 # Randomness self-test (stdlib python3; no third-party deps)
83 # Exits 0 (PASS) / 1 (FAIL). Prints a human summary unless $2 == "quiet".
84 verify_random() {
85     local nbytes="$${1:-1048576}" mode="$${2:-loud}"
86     python3 - "$nbytes" "$mode" <<'PY'
87 import os, sys, math
88 from collections import Counter
89
90 n = int(sys.argv[1])
91 mode = sys.argv[2] if len(sys.argv) > 2 else "loud"
92 buf = os.urandom(n)
93 bits = n * 8
94
95 # 1) Shannon entropy over byte values (bits/byte, ideal = 8.0)
96 c = Counter(buf)
97 H = -sum((v / n) * math.log2(v / n) for v in c.values())
98
99 # 2) NIST monobit: 1-bit count should be ~half. z = (ones - bits/2)/sqrt(bits/4)
100 ones = sum(bin(b).count("1") for b in buf)
101 z = (ones - bits / 2) / math.sqrt(bits / 4)
102
103 # 3) Chi-square over the 256 byte values (df = 255, mean 255, sd ~22.6)
104 exp = n / 256.0
105 chi2 = sum((c.get(i, 0) - exp) ** 2 / exp for i in range(256))
106
107 # PASS windows chosen wide enough that true CSPRNG output passes ~always,
108 # yet a broken/patterned source (constant, low-entropy, biased) fails hard.
109 H_OK = H >= 7.99 if n >= 1 << 16 else H >= 7.5
110 Z_OK = abs(z) < 5.0
111 CHI_OK = 185.0 < chi2 < 345.0
112 ok = H_OK and Z_OK and CHI_OK
113
114 if mode != "quiet":
115     print(f"sample : {n} bytes ({bits} bits) from os.urandom (/dev/urandom CSPRNG)")
116     print(f"shannon H : {H:.5f} bits/byte (ideal 8.0) [{ 'ok' if H_OK else 'FAIL' }]")
117     print(f"monobit z : {z:.3f} (|z|<5, bias {ones/bits*100:.3f}% ones) [{ 'ok' if Z_OK else 'FAIL' }]")
118     print(f"chi-square : {chi2:.1f} (df=255, expect ~255, window 185-345) [{ 'ok' if CHI_OK else 'FAIL' }]")
119     print(f"verdict : {'PASS statistically random' if ok else 'FAIL not random enough'}")
120 sys.exit(0 if ok else 1)

```

```

121 PY
122 }
123
124 # The cover-traffic loop (foreground; 'start' backgrounds this)
125 pad_loop() {
126     require_enabled
127     # Own the PID file for BOTH run modes (manual 'start' and launchd '__boot'), so
128     # 'status'/'stop' behave identically however padding was launched. Cleared on exit.
129     echo $$ > "$NOISE_PID"
130     trap 'rm -f "$NOISE_PID"' EXIT INT TERM
131     # Gate: only ever emit noise we have just proven is statistically random.
132     if ! verify_random 1048576 quiet; then
133         echo "noise: randomness self-test FAILED refusing to emit non-random padding." >&2
134         log "pad aborted: randomness self-test failed"
135         exit 1
136     fi
137     log "pad start  ${PAD_TARGET}:${PAD_PORT} rate=${PAD_RATE}kbps pkt=${PAD_BYTES}B jitter=${PAD_JITTER}ms"
138     python3 - "$PAD_TARGET" "$PAD_PORT" "$PAD_RATE" "$PAD_BYTES" "$PAD_JITTER" <<'PY'
139     import os, sys, socket, time, random
140
141     target = sys.argv[1]
142     port = int(sys.argv[2])
143     rate = float(sys.argv[3]) * 1000.0 / 8.0 # kbps bytes/sec
144     pkt = int(sys.argv[4])
145     jitter = float(sys.argv[5]) / 1000.0 # ms sec
146
147     pps = max(0.1, rate / pkt) # packets per second for the target rate
148     interval = 1.0 / pps
149     s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
150     try:
151         while True:
152             try:
153                 s.sendto(os.urandom(pkt), (target, port)) # fresh CSPRNG bytes per packet
154             except OSError:
155                 pass # target unreachable / dropped is fine the bytes still left the NIC
156                 time.sleep(max(0.0, interval + random.uniform(-jitter, jitter)))
157     except KeyboardInterrupt:
158         pass
159     PY
160 }
161
162 daemon_running() {
163     [ -f "$NOISE_PID" ] || return 1
164     local pid; pid="$(cat "$NOISE_PID" 2>/dev/null)"
165     [ -n "$pid" ] && kill -0 "$pid" 2>/dev/null
166 }
167
168 cmd_start() {
169     require_enabled
170     if daemon_running; then
171         echo "noise: padding daemon already running (pid $(cat "$NOISE_PID"))."
172         exit 0
173     fi
174     echo "noise: verifying randomness before arming padding"
175     verify_random 1048576 loud || { echo "noise: aborting self-test failed." >&2; exit 1; }
176     nohup "$SELF" pad >>"$NOISE_LOG" 2>&1 &
177     local npid=$!
178     sleep 0.3 # let pad_loop write its PID file so 'status' is immediately accurate
179     echo "noise: padding daemon armed (pid $npid)  ${PAD_TARGET}:${PAD_PORT}, ${PAD_RATE} kbps."
180 }
181
182 cmd_stop() {
183     if daemon_running; then
184         local pid; pid="$(cat "$NOISE_PID")"
185         kill "$pid" 2>/dev/null && echo "noise: stopped padding daemon (pid $pid)."
186         log "pad stop (pid $pid)"
187     else
188         echo "noise: no padding daemon running."
189     fi
190     rm -f "$NOISE_PID"
191 }
192
193 cmd_status() {
194     echo "NOISE_ENABLED = $NOISE_ENABLED"
195     if daemon_running; then
196         echo "padding : RUNNING (pid $(cat "$NOISE_PID"))"
197         echo "target : ${PAD_TARGET}:${PAD_PORT}"
198         echo "rate / packet : ${PAD_RATE} kbps / ${PAD_BYTES} B, +/-${PAD_JITTER} ms jitter"
199     else
200         echo "padding : stopped"

```

```

201 fi
202 }
203
204 # launchd entry point (see launchd/com.nullexit.noise.plist). Governed by the
205 # SAME NOISE_ENABLED switch NOT a new option. When disabled it exits 0 cleanly
206 # so KeepAlive never tight-loops; when enabled it runs the padding loop in the
207 # foreground so launchd supervises it directly. This is why persistence adds no
208 # extra knob: loading the plist once means "auto-start whenever NOISE_ENABLED=true".
209 cmd_boot() {
210     if [ "$NOISE_ENABLED" != "true" ]; then
211         log "boot: NOISE_ENABLED != true idle (padding not started)"
212         exit 0
213     fi
214     log "boot: NOISE_ENABLED=true starting padding loop under launchd"
215     pad_loop
216 }
217
218 case "${1:-}" in
219     verify)    verify_random "${2:-1048576}" loud ;;
220     start)     cmd_start ;;
221     pad)       pad_loop ;;
222     __boot)    cmd_boot ;;
223     stop)      cmd_stop ;;
224     status)    cmd_status ;;
225     --help|-h|"") sed -n '2,50p' "$0" ;;
226     *) echo "Unknown argument: $1 (try --help)" >&2; exit 2 ;;
227 esac

```

43 scripts/pf.conf

```

1 # scripts/pf.conf - Packet Filter configuration for nullexit killswitch
2 # Dynamically parameterized by pfctl macro override
3
4 # ext_if must be passed via command line, e.g. -D ext_if=en0
5 # If not passed, default to en0
6 ext_if = "en0"
7 ts_if = "utun+"
8
9 # Tables populated dynamically via pfctl
10 table <vpn_endpoints> persist
11 table <derp_relays> persist
12 # FaceTime split-route (opt-in): Apple media/relay /16s that egress DIRECT via the
13 # physical gateway. EMPTY unless FACETIME_SPLIT_ROUTE=true, so the kill-switch
14 # posture is unchanged by default. Populated by add_apple_split_routes().
15 table <apple_direct> persist
16
17 # Core Rules
18 set skip on lo0 # Ensure local loopback is never blocked
19 scrub out all max-mss 1160 # TCP MSS Clamping: Prevents fragmentation on host TCP flows (like
    Tailscale control plane)
20 pass quick on $ts_if all # Allow all Tailscale traffic (essential to prevent SSH lockout)
21
22 # Default deny outbound WAN traffic
23 block out on $ext_if all
24
25 # Block inbound Screen Sharing (VNC) and SSH from local Wi-Fi, forcing them to use Tailscale
26 block in on $ext_if proto tcp from any to any port { 22, 5900 }
27
28 # Exceptions for tunnel handshakes and coordination
29 pass out quick on $ext_if proto udp to <vpn_endpoints> port 2408
30 pass out quick on $ext_if proto udp to <derp_relays>
31 pass out quick on $ext_if proto tcp to 192.200.0.0/24 port 443
32
33 # FaceTime split-route (opt-in, table empty unless FACETIME_SPLIT_ROUTE=true):
34 # let real-time media/relay traffic to Apple's /16s leave DIRECT (real IP) so
35 # FaceTime can hole-punch instead of dying behind WARP's symmetric NAT.
36 pass out quick on $ext_if proto { tcp udp } to <apple_direct>
37
38 # Allow local LAN traffic (AirDrop, local Tailscale P2P, etc)
39 pass out quick on $ext_if proto tcp to { 192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12 }
40 pass out quick on $ext_if proto udp to { 192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12 }
41 pass out quick on $ext_if proto icmp to { 192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12 }
42
43 # Allow DHCP outbound (client port 68 to server port 67) so we can obtain/renew IP address
44 pass out quick on $ext_if proto udp from any port 68 to any port 67

```

44 scripts/reinstall-host-tailscale.sh

```
1 #!/usr/bin/env bash
2 #
3 # scripts/reinstall-host-tailscale.sh
4 #
5 # Complete uninstall + reinstall of the host-side Homebrew Tailscale.
6 #
7 # Use this when the brew LaunchAgent is wedged, Login Items entries are
8 # duplicated, "Tailscale is stopped" persists despite
9 # 'brew services restart tailscale', or you simply want a clean baseline
10 # before re-engaging the gateway as exit node.
11 #
12 # Idempotent. Safe to re-run. Use --dry to preview every step with zero
13 # changes. Use --yes to skip the per-step confirmation prompts.
14 #
15 # This script ONLY touches the HOST-side Tailscale (the one installed by
16 # Homebrew 'brew install tailscale' and managed by launchd as
17 # homebrew.mxcl.tailscale). The nullexit gateway container's tailscaled
18 # (which lives inside the Colima VM at 100.100.21.8 and offers the exit
19 # node to this host) is unaffected.
20
21 set -u
22
23 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
24
25 DRY=0
26 YES=0
27
28 for arg in "$@"; do
29     case "$arg" in
30         --dry) DRY=1 ;;
31         --yes) YES=1 ;;
32         --help|-h) # NOTE: heredoc is UNQUOTED so $SCRIPT_DIR expands in the body. If you
33                   # add new $-style placeholders here, they'll expand the same way.
34                   cat <<USAGE
35 Usage: bash "$SCRIPT_DIR/reinstall-host-tailscale.sh" [--dry] [--yes]
36
37 --dry  Print every step, make zero changes. Use to audit before
38        committing.
39 --yes  Auto-confirm the prompts at every destructive step. Useful in
40        CI / fully-scripted recovery; do not use until you've read
41        the script.
42
43 After this script completes cleanly, you still must (MANUALLY):
44
45 1. Open System Settings Privacy & Security Local Network and
46    enable Tailscale. macOS will prompt automatically the first time
47    the fresh install of 'tailscale' is invoked.
48
49 2. Re-engage the tailnet + exit node:
50     sudo tailscale up --ssh=true --accept-dns=false \
51       --exit-node="$(cat .gateway_ip | tr -d '\r' | awk 'NR==1{print $1; exit}')" \
52       --exit-node-allow-lan-access=true
53
54 3. Re-run ./toggle.sh from the project root.
55
56 4. Final verification:
57     curl -s https://www.cloudflare.com/cdn-cgi/trace | grep -E '^(warp|ip|colo)=*'
58     expected: warp=on, ip=<Cloudflare>, colo=YYZ/YUL
59 USAGE
60     exit 0
61     ;;
62     *)
63     echo "Unknown flag: $arg" >&2
64     exit 2
65     ;;
66     esac
67 done
68
69 # ----- helpers -----
70
71 confirm() {
72     if [ "$YES" = 1 ] || [ "$DRY" = 1 ]; then
73         return 0
74     fi
75     printf "      %s [y/N] " "$1"
76     read -r ans
77     case "$ans" in
```

```

78     y|Y|yes|YES) return 0 ;;
79     *)
80         echo "        aborted by user"
81         exit 1
82         ;;
83     esac
84 }
85
86 header() {
87     printf '\n %s\n' "$1"
88 }
89
90 # with_timeout <seconds> <cmd...>
91 # Run a command in the background and kill it if it exceeds <seconds>.
92 # Returns the command's exit code if it finished in time, 124 (matching
93 # GNU 'timeout' convention) if it timed out. macOS doesn't ship GNU
94 # coreutils 'timeout', so this is the bash-3.2-friendly replacement.
95 # stdout + stderr are discarded (we only care about exit code); redirect
96 # or pipe externally if you need the output.
97 with_timeout() {
98     local timeout_s="${1:-30}"
99     shift
100     "$@" >/dev/null 2>&1 &
101     local cmdpid=$!
102     local elapsed=0
103     while kill -0 "$cmdpid" 2>/dev/null; do
104         if [ "$elapsed" -ge "$timeout_s" ]; then
105             kill -9 "$cmdpid" 2>/dev/null
106             wait "$cmdpid" 2>/dev/null
107             return 124
108         fi
109         sleep 1
110         elapsed=$((elapsed+1))
111     done
112     wait "$cmdpid" 2>/dev/null
113     return $?
114 }
115
116 # ----- 0. Pre-flight -----
117 header "Pre-flight current host tailscale state"
118 echo "    brew: $(command -v brew >/dev/null 2>&1 && brew --version | head -1 || echo '
119         not on PATH')"
119 echo "    tailscaled running: $(pgrep -lf tailscaled 2>/dev/null | wc -l | tr -d ' ') process(es)"
120 echo "    brew service tailscale: $(brew services list 2>/dev/null | awk '$1=="tailscale"{{print $2, $3}}
121         | tr '\n' ' ' | sed 's/ $//')'"
121 echo "    LaunchAgent plists (under ~/Library/LaunchAgents):"
122 ls -l ~/Library/LaunchAgents/ 2>/dev/null | grep -i 'tail' | sed 's/~/ /' || echo "    (none)"
123
124 # Detect Tailscale's macOS Network Extension (residual from prior App Store
125 # Tailscale.app installs). Lives under /Library/Apple/SystemExtensions/ and
126 # is managed by 'systemextensionsctl' NOT launchd. If loaded, brew
127 # tailscaled will NOT engage the Network Extension framework slot because
128 # the SE still has it claimed. Symptom: 'sudo tailscale status' returns
129 # "Tailscale is stopped" even when the daemon is alive and listening.
130 se_found=0
131 if command -v systemextensionsctl >/dev/null 2>&1; then
132     # Grab any line matching tailscale and network-extension
133     se_line=$(systemextensionsctl list 2>/dev/null | grep -iE '(io|com)\.tailscale\..*network-extension' |
134         head -n 1)
134     if [ -n "$se_line" ]; then
135         se_found=1
136         # Extract the teamID (10-char uppercase/numeric word)
137         se_teamID=$(echo "$se_line" | tr -s ' \t' '\n' | grep -E '[A-Z0-9]{10}$' | head -n 1)
138         # Extract the bundleID (starts with io.tailscale or com.tailscale)
139         se_bundleID=$(echo "$se_line" | tr -s ' \t' '\n' | grep -E '(io|com)\.tailscale\.' | head -n 1 | sed
140             's/(.*)//')
141
142         # If teamID is not found, fallback to '-'
143         if [ -z "$se_teamID" ]; then
144             se_teamID="-"
145         fi
146     fi
147
148 if [ "$se_found" = 1 ] && [ -n "$se_bundleID" ]; then
149     echo "    Tailscale Network Extension System Extension is loaded (residual from App Store Tailscale.
150         app install)"
150     echo "    Team ID: $se_teamID, Bundle ID: $se_bundleID"
151     echo "    brew tailscaled cannot engage the NE slot while this SE has it claimed."
152     if [ "$DRY" = 1 ]; then
153         echo "    [dry] (would prompt) sudo systemextensionsctl uninstall $se_teamID $se_bundleID"

```

```

154 else
155     confirm "Run 'sudo systemextensionsctl uninstall $se_teamID $se_bundleID'? (frees the NE slot for
156     brew tailscale; non-fatal if it errors)"
157     sudo systemextensionsctl uninstall "$se_teamID" "$se_bundleID" 2>&1 \
158     || echo " ! uninstall returned non-zero (may require reboot, non-fatal)"
159 fi
160 else
161     echo " no Tailscale System Extension loaded"
162 fi
163 # Pattern used by the Step-1 bootout loops, in both dry + real branches.
164 # Widened from a bare /tailscale/ so the same loops also pick up any
165 # nullexit-labeled LaunchAgents (e.g., com.nullexit.wake-recovery, the
166 # caffeinate + post-wake recovery hook). The wake-recovery LaunchAgent
167 # is re-installed by ./toggle.sh / setup.sh on re-engage, so wiping it
168 # here is non-fatal and is part of restoring a clean baseline.
169 TAILSCALE_LABEL_RE='tailscale'
170 NULLEXIT_LABEL_RE='nullexit'
171
172 # ----- 1. brew services stop -----
173 header "Step 1 Stop the brew-managed service + unload LaunchAgents"
174 # Detect whether the brew-managed service is registered as $USER (gui/$UID
175 # domain) or as root (system domain). NOPASSWD sudoers can leave a stale
176 # root-owned registration after a prior 'sudo brew services ...' round;
177 # 'brew services stop' without sudo refuses to touch a root-owned plist
178 # (Error: Service 'tailscale' is started as 'root').
179 brew_status_line="$(brew services list 2>/dev/null | awk '$1=="tailscale"')"
180 brew_status_user="$(echo "$brew_status_line" | awk '{print $3}')"
181 if [ -n "$brew_status_line" ]; then
182     if [ "$DRY" = 1 ]; then
183         echo " [dry] brew services stop tailscale (user=$brew_status_user)"
184     else
185         if [ "$brew_status_user" = "root" ]; then
186             confirm "Run 'sudo brew services stop tailscale' (service is registered as root)?"
187             sudo brew services stop tailscale 2>&1 || echo " (sudo brew services stop returned non-zero non
188             -fatal)"
189         else
190             echo " brew services stop tailscale (user=$brew_status_user, no sudo needed)"
191             brew services stop tailscale 2>&1 || echo " (brew reported it wasn't actually running)"
192         fi
193     fi
194     echo " unload any tailscale-related LaunchAgents (prevents launchd from respawning after kill)"
195     if [ "$DRY" = 1 ]; then
196         echo " [dry] for each 'tailscale|nullexit'-matched label in gui/$UID domain:"
197         while IFS= read -r label; do
198             [ -z "$label" ] && continue
199             echo " [dry] launchctl bootout gui/$UID/$label"
200             done < <(launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" 'NR
201             > 1 && ($3 ~ ts || $3 ~ nx) {print $3}')
202             echo " [dry] for each 'tailscale|nullexit'-matched label in system domain (root-owned brew plist
203             ):"
204             while IFS= read -r label; do
205                 [ -z "$label" ] && continue
206                 echo " [dry] (would prompt) sudo launchctl bootout system/$label"
207             done < <(sudo -n launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$
208             $NULLEXIT_LABEL_RE" 'NR > 1 && ($3 ~ ts || $3 ~ nx) {print $3}')
209             if launchctl print system/com.tailscale.ipn.macos >/dev/null 2>&1; then
210                 echo " [dry] (would prompt) sudo launchctl bootout system/com.tailscale.ipn.macos"
211             fi
212             if launchctl print system/com.tailscale.ipn.macsys >/dev/null 2>&1; then
213                 echo " [dry] (would prompt) sudo launchctl bootout system/com.tailscale.ipn.macsys"
214             fi
215         fi
216     else
217         # Bootout every gui-domain LaunchAgent whose label matches 'tailscale'
218         # OR 'nullexit'. The nullexit arm catches com.nullexit.wake-recovery,
219         # which holds caffeinate/watcher.sh alive across a daemon reset not
220         # desired while the script is establishing a fresh baseline. Awake-
221         # recovery is re-installed by ./toggle.sh / setup.sh on re-engage, so
222         # wiping here is non-fatal. The pattern stays narrower than a bare
223         # /tail/ or /null/ so unrelated Apple services are not unloaded.
224         while IFS= read -r label; do
225             [ -z "$label" ] && continue
226             echo " launchctl bootout gui/$UID/$label"
227             launchctl bootout "gui/$UID/$label" 2>/dev/null || true
228             done < <(launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" 'NR
229             > 1 && ($3 ~ ts || $3 ~ nx) {print $3}')
230         # Bootout every system-domain LaunchDaemon/Agent whose label matches
231         # 'tailscale' OR 'nullexit'. This catches the root-owned brew plist left
232         # behind by prior 'sudo brew services ...' invocations that was the
233         # case that left tailscaled PID 47447 57734 respawning between sudo
234         # pkill runs. Defensive nullexit catch in case the user previously

```

```

229 # 'sudo ln -s' d the wake-recovery plist into /Library/LaunchDaemons.
230 while IFS= read -r label; do
231     [ -z "$label" ] && continue
232     confirm "Run 'sudo launchctl bootout system/$label'?"
233     sudo launchctl bootout "system/$label" 2>/dev/null || true
234     done < <(sudo -n launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$
$NULLEXIT_LABEL_RE" 'NR > 1 && ($3 ~ ts || $3 ~ nx) {print $3}')
235 # Legacy Tailscale.app system-domain entries. Each one is checked and
236 # confirmed separately so we don't sudo-prompt or sudo-bootout for nothing.
237 if launchctl print system/com.tailscale.ipn.macos >/dev/null 2>&1; then
238     confirm "Run 'sudo launchctl bootout system/com.tailscale.ipn.macos'?"
239     sudo launchctl bootout system/com.tailscale.ipn.macos 2>/dev/null || true
240 fi
241 if launchctl print system/com.tailscale.ipn.macsys >/dev/null 2>&1; then
242     confirm "Run 'sudo launchctl bootout system/com.tailscale.ipn.macsys'?"
243     sudo launchctl bootout system/com.tailscale.ipn.macsys 2>/dev/null || true
244 fi
245 fi
246 echo "    waiting up to 10s for tailscaled to exit"
247 for i in 1 2 3 4 5 6 7 8 9 10; do
248     if [ "$DRY" = 1 ]; then
249         echo "    [dry] sleep 1; loop check would terminate here"
250         break
251     fi
252     sleep 1
253     if ! pgrep -lf tailscaled >/dev/null 2>&1; then
254         echo "    tailscaled exited in ${i}s"
255         break
256     fi
257     [ "$i" = 10 ] && echo "    tailscaled still alive after 10s Step 2 will deal with it"
258 done
259 else
260     echo "    brew reports no tailscale service loaded"
261 fi
262
263 # ----- 2. Kill any orphan tailscaled -----
264 header "Step 2 Kill any orphan tailscaled process(es) (may escalate to sudo)"
265 if ! pgrep -lf tailscaled >/dev/null 2>&1; then
266     echo "    no orphans"
267 else
268     echo "    Stray process(es):"
269     pgrep -lf tailscaled | sed 's/^/    /'
270     echo "    no-sudo: pkill tailscaled"
271     if [ "$DRY" = 1 ]; then
272         echo "    [dry] pkill tailscaled on failure: sudo pkill -9 tailscaled"
273     else
274         if pkill tailscaled 2>/dev/null; then
275             echo "    no-sudo pkill succeeded"
276         else
277             echo "    ! no-sudo pkill insufficient; cannot escalate yet"
278         fi
279         sleep 1
280     fi
281 fi
282 # Recheck; if still alive, offer sudo escalation
283 if { [ "$DRY" = 0 ] && pgrep -lf tailscaled >/dev/null 2>&1; } || [ "$DRY" = 1 ]; then
284     if [ "$DRY" = 1 ]; then
285         echo "    [dry] sudo pkill -9 tailscaled (escalation would fire here)"
286     else
287         confirm "Run 'sudo pkill -9 tailscaled' to escalate?"
288         sudo pkill -9 tailscaled 2>&1
289         sleep 1
290     fi
291 fi
292
293 if [ "$DRY" = 0 ]; then
294     if pgrep -lf tailscaled >/dev/null 2>&1; then
295         echo "    Failed to kill stragglers bailing out"
296         pgrep -lf tailscaled | sed 's/^/    /'
297         exit 3
298     fi
299     echo "    all tailscaled processes gone"
300 fi
301 fi
302
303 # ----- 3. brew uninstall -----
304 header "Step 3 brew uninstall tailscale"
305 if ! brew list tailscale >/dev/null 2>&1; then
306     echo "    not currently installed via brew skip"
307 else
308     # Detect whether the keg directory is owner=$USER or owner=root. If the

```



```

309 # keg is root-owned (typically a residual of some prior 'sudo brew ...'
310 # cycle that did perms fixups), 'brew uninstall' without sudo refuses
311 # to remove the files and prints: 'Error: Could not remove tailscale
312 # keg! Do so manually: sudo rm -rf /opt/homebrew/Cellar/tailscale/<v>'.
313 # Detect that and escalate to a sudoed uninstall, with a manual rm -rf
314 # fallback if 'sudo brew uninstall' itself errors out (e.g., partial
315 # state where the keg dir is gone but brew still thinks it's installed).
316 # Detect keg state. Treat '(dir absent)' AND '(dir present + empty)' as the
317 # same '<unknown>' bucket so both escalate to sudo. The empty-parent case
318 # is what you get after a manual 'sudo rm -rf /opt/homebrew/Cellar/tailscale/<v>'
319 # from inside: brew's parent dir is left present-but-empty and brew still
320 # thinks the package is installed.
321 keg_dir="/opt/homebrew/Cellar/tailscale"
322 if [ -d "$keg_dir" ] && [ -n "$(ls -A "$keg_dir" 2>/dev/null)" ]; then
323     keg_owner="$(stat -f '%Su' "$keg_dir" 2>/dev/null || echo '<unknown>')"
324 else
325     keg_owner="<unknown>"
326 fi
327 if [ "$DRY" = 1 ]; then
328     echo "    [dry] brew uninstall tailscale (keg_owner=$keg_owner)"
329 else
330     # 'root' AND '<unknown>' both require sudo escalation. The '<unknown>'
331     # case is the partial-state path: keg dir was already removed (manually
332     # or by a prior failed uninstall) but brew's internal state still says
333     # installed. Without this branch the script would fall through to a
334     # non-sudo brew uninstall that fails again with the same 'Could not
335     # remove tailscale keg!' error.
336     if [ "$keg_owner" = "root" ] || [ "$keg_owner" = "<unknown>" ]; then
337         confirm "Run 'sudo brew uninstall tailscale' (keg=$keg_owner)?"
338         sudo brew uninstall tailscale 2>&1 || {
339             echo "    ! sudo brew uninstall failed; offering manual rm -rf fallback"
340             # Guard the glob so an empty/absent dir doesn't literal-expand the
341             # '*' and confuse the user with a sudo error about a non-existent path.
342             if [ -d /opt/homebrew/Cellar/tailscale ] && [ -n "$(ls -A /opt/homebrew/Cellar/tailscale 2>/dev/
null)" ]; then
343                 confirm "Run 'sudo rm -rf /opt/homebrew/Cellar/tailscale/*'?"
344                 sudo rm -rf /opt/homebrew/Cellar/tailscale/* 2>&1
345             else
346                 echo "    keg dir already empty or absent  nothing more to rm"
347             fi
348         }
349     else
350         confirm "Run 'brew uninstall tailscale' (removes LaunchAgent plist + linked symlinks)?"
351         brew uninstall tailscale 2>&1
352     fi
353 fi
354 fi
355
356 # ----- 4. Wipe state directories -----
357 header "Step 4 Wipe stale state"
358 # User-owned
359 declare -a USER_PATHS
360 USER_PATHS=(
361     "$HOME/.local/share/tailscale"
362     "$HOME/.local/run/tailscaled.socket"
363     "$HOME/.local/run/tailscaled.state"
364     "$HOME/.local/state/tailscale"
365     "$HOME/Library/Application Support/Tailscale"
366     "$HOME/Library/Caches/com.tailscale.ipn.macos"
367     "$HOME/Library/Caches/Tailscale"
368     "$HOME/Library/Logs/Tailscale"
369 )
370 echo "    User-owned paths (no sudo):"
371 for p in "${USER_PATHS[@]}; do
372     if [ -e "$p" ]; then
373         if [ "$DRY" = 1 ]; then
374             echo "    [dry] rm -rf '$p'"
375         else
376             rm -rf "$p"
377             echo "    rm -rf '$p'"
378         fi
379     fi
380 done
381
382 # System
383 declare -a SUDO_PATHS
384 SUDO_PATHS=(
385     "/var/lib/tailscale"
386     "/var/cache/tailscale"
387 )
388 echo "    System paths under /var (will prompt for sudo if any are present):"

```

```

389 eligible_sudo=0
390 for p in "${SUDDO_PATHS[@]"; do
391     if [ -e "$p" ]; then
392         eligible_sudo=1
393         if [ "$DRY" = 1 ]; then
394             echo "        [dry] sudo rm -rf '$p'"
395         else
396             if confirm "Run 'sudo rm -rf $p'?" ; then
397                 sudo rm -rf "$p" && echo "        rm -rf '$p'"
398             fi
399         fi
400     fi
401 done
402 [ "$eligible_sudo" = 0 ] && echo "        (none of /var/lib/tailscale, /var/cache/tailscale are present)"
403
404 # ----- 5. Wipe stale LaunchAgent plists -----
405 header "Step 5 Wipe stale LaunchAgent plist files"
406 declare -a PLIST_FILES
407 PLIST_FILES=(
408     "$HOME/Library/LaunchAgents/homebrew.mxcl.tailscale.plist"
409     "$HOME/Library/LaunchAgents/com.tailscale.ipn.macos.plist"
410     "$HOME/Library/LaunchAgents/homebrew.mxcl.tailscale.plist.bak"
411     "$HOME/Library/LaunchAgents/com.nullexit.wake-recovery.plist"
412 )
413 for f in "${PLIST_FILES[@]"; do
414     if [ -e "$f" ]; then
415         if [ "$DRY" = 1 ]; then
416             echo "        [dry] rm -f '$f'"
417         else
418             rm -f "$f"
419             echo "        rm -f '$f'"
420         fi
421     fi
422 done
423 echo "        Remaining tailscale|nullexit plists (should be empty):"
424 ls -l ~/Library/LaunchAgents/ 2>/dev/null | grep -i 'tail' | sed 's/^/' /' || echo "        (none)"
425
426 # ----- 5.5. Drain Background-Items (Login Items) -----
427 header "Step 5.5 Drain Background-Items (Login Items)"
428 # macOS 13+ keeps a parallel "Background Items" database alongside
429 # classic LaunchAgents. Even when we wipe ~/Library/LaunchAgents/homebrew.mxcl.tailscale.plist
430 # in Step 5, the .btm file at
431 # ~/Library/Application Support/com.apple.backgroundtaskmanagementagent.backgrounditems.btm
432 # can still hold a Background-Items entry pointing at the (now-dead) plist
433 # path. Next login: macOS sees the entry, tries to bootstrap, finds the
434 # plist missing, and either silently regenerates it via brew's post-install
435 # hook (causing tailscaled respawn at next login) or refuses outright.
436 # 'sfltool reset-login-items' is Apple's canonical API for wiping the
437 # entire .btm file. It DOES wipe all Login Items the user re-adds what
438 # they want via System Settings General Login Items. Acceptable here
439 # because the goal of this script is "fresh baseline for Tailscale".
440 BTM="$HOME/Library/Application Support/com.apple.backgroundtaskmanagementagent/backgrounditems.btm"
441 if [ -f "$BTM" ]; then
442     if [ "$DRY" = 1 ]; then
443         echo "        [dry] found $BTM; would prompt sfltool reset-login-items"
444     else
445         confirm "Run 'sfltool reset-login-items' (wipes ALL Login Items; rebuild via System Settings General
446             Login Items)?"
447         if sfltool reset-login-items 2>&1; then
448             echo "        Login Items drained"
449         else
450             echo "        ! sfltool reset-login-items failed (non-fatal; do manually: System Settings General
451             Login Items)"
452         fi
453     fi
454 else
455     echo "        no backgrounditems.btm present"
456 fi
457
458 # ----- 6. brew install -----
459 header "Step 6 brew install tailscale (fresh)"
460 if brew list tailscale >/dev/null 2>&1; then
461     echo "        already installed skip"
462 else
463     confirm "Run 'brew install tailscale'?"
464     if [ "$DRY" = 1 ]; then
465         echo "        [dry] brew install tailscale"
466     else
467         brew install tailscale 2>&1
468     fi
469 fi

```

```

468 # ----- 6.5. Strip quarantine xattr from keg -----
469 header "Step 6.5 Strip quarantine xattr from tailscale install"
470 # brew's freshly poured bottles don't carry com.apple.quarantine (the
471 # bottles are content-addressed and Homebrew-verified), but the upstream
472 # installer (Tailscale.app from App Store) and any manual download do.
473 # Strip the xattr from the entire keg as a defensive safety net so the
474 # freshly-installed tailscaled binary and any helper binaries are not
475 # Gatekeeper-blocked on first launch. Idempotent no-op if no quarantine
476 # xattr is present.
477 quarantine_count="$(xattr -lr /opt/homebrew/Cellar/tailscale 2>/dev/null | grep -c '^[^ ]* com.apple.
478 quarantine' || echo 0)"
479 if [ "${quarantine_count:-0}" -gt 0 ]; then
480     if [ "$DRY" = 1 ]; then
481         echo " [dry] found $quarantine_count file(s) with com.apple.quarantine; would prompt xattr -dr"
482     else
483         confirm "Run 'xattr -dr com.apple.quarantine /opt/homebrew/Cellar/tailscale'? (strips quarantine
484         xattr from $quarantine_count file(s); non-fatal if it errors)"
485         if xattr -dr com.apple.quarantine /opt/homebrew/Cellar/tailscale 2>&1; then
486             echo " quarantine xattr stripped"
487         else
488             echo " ! xattr -dr failed (non-fatal; open /opt/homebrew/Cellar/tailscale in Finder, right-click
489             tailscaled Open Anyway)"
490         fi
491     fi
492 else
493     echo " no com.apple.quarantine xattr on tailscale install"
494 fi
495 # ----- 7. brew services start (NO sudo!) -----
496 header "Step 7 Start the service as $USER (NO sudo, with multi-tier self-healing)"
497 if ! command -v tailscale >/dev/null 2>&1; then
498     echo " tailscale not on PATH after install brew install errored; aborting"
499     exit 4
500 fi
501 # Aliveness fan-out: ANY one of these is taken as proof of life. None of
502 # them alone is reliable pgrep misses tailscaled if it does setproctitle
503 # or fork-detach; launchctl's reported PID can be a shim that exited
504 # within 1s; tailscale-cli hangs while macOS Local-Network permission is
505 # unresolved. Combined, they cover each other's blind spots.
506
507 tailscaled_api_up() {
508     # Strategy A: tailscale CLI can talk to the local API. Canonical proof.
509     # Bound at 8s tailscale status itself returns quickly on success OR
510     # failure, but if the post-install state is wedged it can hang.
511     with_timeout 8 /opt/homebrew/bin/tailscale status >/dev/null 2>&1
512 }
513
514 tailscaled_proc_up() {
515     # Strategy B: any process has 'tailscaled' substring in argv.
516     pgrep -lf tailscaled >/dev/null 2>&1
517 }
518
519 tailscaled_lc_up() {
520     # Strategy C: launchctl-managed service has a non-dash, non-zero PID.
521     local lc_pid
522     lc_pid="$(launchctl list 2>/dev/null | awk ' $3 == "homebrew.mxcl.tailscale" {print $1; exit}')"
523     if [ -n "$lc_pid" ] && [ "$lc_pid" != "-" ] && [ "$lc_pid" -gt 0 ] 2>/dev/null; then
524         return 0
525     fi
526     return 1
527 }
528
529 tailscaled_is_alive() {
530     tailscaled_api_up && return 0
531     tailscaled_proc_up && return 0
532     tailscaled_lc_up && return 0
533     return 1
534 }
535
536 confirm "Run 'brew services start tailscale'?"
537 if [ "$DRY" = 1 ]; then
538     echo " [dry] brew services start tailscale"
539     echo " [dry] (then poll up to 30s for tailscaled_is_alive)"
540     echo " [dry] (recovery tier 1: launchctl kickstart)"
541     echo " [dry] (recovery tier 2: launchctl bootout + bootstrap of explicit plist)"
542     echo " [dry] (recovery tier 3: brew services restart (stop+start cycle))"
543     echo " [dry] (recovery tier 4: direct /opt/homebrew/bin/tailscaled spawn (bypasses launchd))"
544 else
545     brew services start tailscale 2>&1

```

```

546 # Primary poll tailscaled_is_alive covers all 3 detection strategies.
547 echo "    polling for tailscaled to come up (up to ~30s)"
548 tailscaled_up=0
549 for i in 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15; do
550     if tailscaled_is_alive; then
551         tailscaled_up=1
552         echo "        tailscaled alive after ${i}*2s"
553         break
554     fi
555     sleep 2
556 done
557
558 # Recovery tier 1: kickstart the launchd-managed service. This is the
559 # cheapest and most-aligned-with-launchd fix when brew has loaded the
560 # LaunchAgent but tailscaled didn't fork. Kickstart tells launchd
561 # "re-spawn if not running" without forcing a teardown.
562 if [ "$tailscaled_up" = 0 ]; then
563     echo "    ! Recovery tier 1: launchctl kickstart"
564     if launchctl kickstart -kp "gui/$UID/homebrew.mxcl.tailscale" 2>&1; then
565         for i in 1 2 3 4 5 6 7 8 9 10; do
566             if tailscaled_is_alive; then
567                 tailscaled_up=1
568                 echo "        tailscaled alive after kickstart (${i}*2s)"
569                 break
570             fi
571             sleep 2
572         done
573     else
574         echo "    ! kickstart returned non-zero (LaunchAgent may not be loaded)"
575     fi
576 fi
577
578 # Recovery tier 2: explicit plist bootout + bootstrap. Forces a fresh
579 # launchd registration cycle for the brew-managed service, which fixes
580 # the wider class of "brew says started but daemon never spawned"
581 # wedge states that kickstart alone can't dig out of.
582 if [ "$tailscaled_up" = 0 ]; then
583     echo "    ! Recovery tier 2: launchctl bootout + bootstrap of explicit plist"
584     launchctl bootout "gui/$UID/homebrew.mxcl.tailscale" 2>/dev/null || true
585     sleep 1
586     # Brew installs the canonical plist template at $HOMEBREW_PREFIX/Library/.
587     # On Apple Silicon that's /opt/homebrew/Library/LaunchAgents/. On Intel
588     # it's /usr/local/Library/LaunchAgents/. Detect which and pick the
589     # right one so this script works across both architectures.
590     if [ -f /opt/homebrew/Library/LaunchAgents/homebrew.mxcl.tailscale.plist ]; then
591         plist_path="/opt/homebrew/Library/LaunchAgents/homebrew.mxcl.tailscale.plist"
592     elif [ -f /usr/local/Library/LaunchAgents/homebrew.mxcl.tailscale.plist ]; then
593         plist_path="/usr/local/Library/LaunchAgents/homebrew.mxcl.tailscale.plist"
594     else
595         plist_path=""
596     fi
597     if [ -n "$plist_path" ]; then
598         if launchctl bootstrap "gui/$UID" "$plist_path" 2>&1; then
599             for i in 1 2 3 4 5 6 7 8 9 10; do
600                 if tailscaled_is_alive; then
601                     tailscaled_up=1
602                     echo "        tailscaled alive after bootstrap (${i}*2s)"
603                     break
604                 fi
605                 sleep 2
606             done
607         else
608             echo "    ! explicit bootstrap returned non-zero"
609         fi
610     else
611         echo "    ! could not locate brew plist template; skipping bootstrap tier"
612     fi
613 fi
614
615 # Recovery tier 3: brew services restart (forces a stop + start cycle,
616 # which unsticks LaunchAgents with stale 'started' state from prior
617 # half-cycles). This is the canonical brew-doc fix for 'services don't
618 # actually start even though brew says started'.
619 if [ "$tailscaled_up" = 0 ]; then
620     echo "    ! Recovery tier 3: brew services restart (forces stop+start cycle)"
621     if brew services restart tailscale 2>&1; then
622         for i in 1 2 3 4 5 6 7 8 9 10; do
623             if tailscaled_is_alive; then
624                 tailscaled_up=1
625                 echo "        tailscaled alive after brew restart (${i}*2s)"
626                 break

```

```

627         fi
628         sleep 2
629     done
630 else
631     echo "          ! brew services restart returned non-zero"
632 fi
633 fi
634
635 # Recovery tier 4: last-resort direct spawn of the daemon binary,
636 # bypassing launchd entirely. If tailscaled crashes here too, we'll
637 # capture its actual stderr in /tmp for the user to read (NOPASSWD
638 # sudoers often have local-network permission denied the captured
639 # stderr makes it obvious what to fix in System Settings).
640 if [ "$tailscaled_up" = 0 ]; then
641     echo "          ! Recovery tier 4: direct spawn (bypasses launchd; tail stderr to /tmp)"
642     /opt/homebrew/bin/tailscaled >/tmp/nullexit-tailscaled-spawn.log 2>&1 &
643     spawned_pid=$!
644     disown "$spawned_pid" 2>/dev/null || true
645     sleep 3
646     if tailscaled_is_alive; then
647         tailscaled_up=1
648         echo "          tailscaled alive via direct spawn (pid=$spawned_pid)"
649         echo "          (NOTE: not under launchd supervision a reboot, re-run of"
650         echo "          this script, or ./toggle.sh is required to recreate."
651         echo "          Last 5 lines of stderr captured to /tmp/nullexit-tailscaled-spawn.log:)"
652         tail -n 5 /tmp/nullexit-tailscaled-spawn.log 2>/dev/null | sed 's/^/' '/' || true
653     fi
654 fi
655
656 if [ "$tailscaled_up" = 0 ]; then
657     echo "          HARD FAIL: tailscaled refuses to stay alive across every recovery tier"
658     echo "          launchctl-managed state (if available):"
659     launchctl print "gui/$UID/homebrew.mxcl.tailscale" 2>/dev/null \
660         | grep -E 'state =|last exit code =|runs =|path =' \
661         | sed 's/^/' '/' \
662         || echo "          (no launchctl state available for homebrew.mxcl.tailscale)"
663     echo "          Last 10 lines of /tmp/nullexit-tailscaled-spawn.log (if any):"
664     tail -n 10 /tmp/nullexit-tailscaled-spawn.log 2>/dev/null | sed 's/^/' '/' || echo "          (no log)"
665     echo "          Most likely remaining cause: macOS Local Network permission denied."
666     echo "          System Settings Privacy & Security Local Network toggle TailScale ON."
667     exit 7
668 fi
669 fi
670 sleep 3
671
672 # ----- 8. Verify -----
673 header "Step 8 Verify fresh install"
674 echo "          brew service tailscale: $(brew services list 2>/dev/null | awk '$1=="tailscale"{{print $2, $3}}' \
675     | tr '\n' ' ' | sed 's/ $//')"
676 echo "          tailscaled process: $(pgrep -lf tailscaled 2>/dev/null | head -1 || echo 'NONE (check brew logs)')"
677 ts_ver=""
678 if [ -x /opt/homebrew/bin/tailscale ]; then
679     ts_ver="$(/opt/homebrew/bin/tailscale version 2>/dev/null | head -1 | tr -d '\n')"
680 elif command -v tailscale >/dev/null 2>&1; then
681     ts_ver="$(tailscale version 2>/dev/null | head -1 | tr -d '\n')"
682 fi
683 echo "          tailscale version:      ${ts_ver:-binary missing}"
684 ts_status=""
685 if [ -x /opt/homebrew/bin/tailscale ]; then
686     ts_status="$(/opt/homebrew/bin/tailscale status 2>/dev/null | head -3)"
687 fi
688 if [ -n "$ts_status" ]; then
689     echo "          tailscale status:"
690     printf '%s\n' "$ts_status" | sed 's/^/' '/'
691 else
692     echo "          tailscale status:      (binary missing or errored)"
693 fi
694 if { [ "$DRY" = 0 ] && /opt/homebrew/bin/tailscale status 2>/dev/null | grep -q '^TailScale is stopped'; }; then
695     echo
696     echo "          NOTICE: 'tailscale status' still reports stopped after a fresh install."
697     echo "          Two equally likely causes:"
698     echo "          (a) macOS hasn't yet shown the first-run 'Local Network' prompt for"
699     echo "              TailScale open System Settings Privacy & Security Local"
700     echo "              Network. Toggle TailScale ON."
701     echo "          (b) macOS denied Local Network permission at some prior point."
702     echo "          Either way: System Settings Privacy & Security Local Network."
703     echo "          If the entry isn't present, invoking any tailscale command will"

```

```

704     echo "           re-trigger the prompt automatically."
705 fi
706
707 # ----- 9. Done -----
708 header "Done"
709 if [ "$DRY" = 1 ]; then
710     echo "           (dry-mode finished; no follow-up commands were issued.)"
711     exit 0
712 fi
713
714 cat <<'DONE'
715
716 Manual follow-ups (all of these; the script stops short of running sudo
717 tailscale up so you can verify the brew install is actually live first):
718
719 1. System Settings Privacy & Security Local Network Tailscale ON
720 macOS will normally pop the prompt the first time you run 'tailscale'.
721 If it didn't, trigger it via the menu bar check.
722
723 2. Re-engage the tailnet + exit node:
724     sudo tailscale up --ssh=true --accept-dns=false \
725       --exit-node="$(cat .gateway_ip | tr -d '\r' | awk 'NR==1{print $1; exit}')" \
726       --exit-node-allow-lan-access=true
727
728 3. Re-run: ./toggle.sh
729 (re-installs the com.nullexit.wake-recovery LaunchAgent wiped
730 in Step 5 to clear the slate and re-engages the gateway)
731
732 4. Final verification:
733     curl -s https://www.cloudflare.com/cdn-cgi/trace | grep -E '^(warp|ip|colo)=\'
734     expect: warp=on, ip=<Cloudflare IP>, colo=YYZ/YUL
735
736 Container-side (gateway / 100.100.21.8) Tailscale is unaffected by this
737 script. Other tailnet devices (S24, iPhone, Windows) are unaffected.
738
739 DONE

```

45 scripts/routing-fix.sh

```

1  #!/bin/sh
2  # routing-fix: Maintains routing for SOCKS5 proxy traffic through WARP (tun0)
3  #               and FORWARD rules for Tailscale exit-node return traffic.
4  #
5  # Runs inside the warp container's network namespace. This script ensures:
6  # 1. Table 200 has default via tun0 (for SOCKS5 proxy traffic from 172.18.0.2)
7  #    with the WARP endpoint exception via eth0 (prevents tunnel loop)
8  # 2. The CGNAT ip rule (100.64.0.0/10 table 52 tailscale routes) is maintained for Tailscale
9  # 3. FORWARD RELATED,ESTABLISHED rule allows return traffic for exit-node
10 #    forwarded connections (S24 tailscale0 eth0 host internet)
11 #
12 # CRITICAL: Does NOT touch the main table's default route. Gluetun handles that
13 # internally via its own routing rules and WireGuard fwmark (0xca6c table 51820).
14 # Overriding the main table default would create a loop: encrypted WireGuard
15 # packets exiting tun0 would match default dev tun0 and re-enter the tunnel.
16
17 set -e
18
19 trap 'echo "routing-fix: Caught signal, exiting..."; exit 0' TERM INT QUIT
20
21 sleep 15 &
22 wait $! || true
23
24 echo "routing-fix: Setting up routing for SOCKS5 proxy through WARP (tun0)..."
25
26 # Dynamically detect Docker subnet from eth0
27 DOCKER_NET=$(ip route show dev eth0 | grep -v default | head -1 | awk '{print $1}')
28 DOCKER_GW=$(ip route show default | head -1 | awk '{print $3}')
29 WARP_ENDPOINT="${WARP_ENDPOINT}"
30 TAILSCALE_ALLOW_LAN_P2P="${TAILSCALE_ALLOW_LAN_P2P:-false}"
31 IP_BLOCKLIST_FILE="/userfilters/ip_blocklist.ipset"
32 LAST_IP_MTIME=""
33 echo "routing-fix: Docker network: ${DOCKER_NET}, gateway: ${DOCKER_GW}"
34
35 # Table 200: Used by ip rule 100 (from 172.18.0.2) for SOCKS5 proxy traffic
36 # WARP endpoint goes via eth0 to avoid tunnel loop, everything else via tun0
37 ip route add "${WARP_ENDPOINT}" via "${DOCKER_GW}" dev eth0 table 200 2>/dev/null || \

```

```

38 ip route replace "${WARP_ENDPOINT}" via "${DOCKER_GW}" dev eth0 table 200 2>/dev/null || true
39 ip route replace default dev tun0 table 200 2>/dev/null || true
40
41 # Main table: Just ensure the Docker subnet route exists via eth0
42 # (gluetun owns the default route here do NOT touch it)
43 ip route replace "${DOCKER_NET}" dev eth0 2>/dev/null || true
44
45 # FORWARD RELATED, ESTABLISHED: Allow return traffic for exit-node connections.
46 # The container has BOTH iptables (nftables backend) and iptables-legacy
47 # loaded. Gluetun's post-rules.txt uses the nftables backend, while Tailscale
48 # uses the legacy backend. Packets go through BOTH stacks, so we add the rule
49 # to both to ensure it survives regardless of which backend has policy DROP.
50 #
51 # Packet flow: S24 outgoing (tailscale0->eth0) ACCEPTed by first rule.
52 # Return traffic (eth0->eth0 via table 52 to host) needs RELATED, ESTABLISHED
53 # to match the conntrack entry created by the outgoing flow.
54 add_fwd_related_established() {
55     # nftables backend (used by gluetun's post-rules.txt)
56     if ! iptables -C FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null; then
57         iptables -A FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null || true
58     fi
59     # legacy backend (used by Tailscale's ts-forward)
60     if command -v iptables-legacy >/dev/null 2>&1; then
61         if ! iptables-legacy -C FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null; then
62             iptables-legacy -A FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null || true
63         fi
64     fi
65 }
66
67 add_fwd_related_established
68
69 # Policy Routing for Tailscale P2P:
70 # Tailscale sends its encrypted mesh UDP packets from port 41642.
71 # If these go out tun0 (WARP), Cloudflare's strict NAT drops the returning
72 # hole-punch packets, causing Tailscale to fail P2P and fall back to DERP relays (high latency).
73 # We mark packets originating from port 41642 and force them out the main table (eth0).
74 #
75 # LAN P2P is controlled by two sources (in priority order):
76 # 1. /app/.lan_p2p_detected written by watcher.sh on macOS on every network change
77 #    using system_profiler SPAirPortDataType (802.1x detection) + AP isolation probe. Overrides .env.
78 # 2. TAILSCALE_ALLOW_LAN_P2P env var (from .env) static fallback.
79 # When disabled, we explicitly DROP local RFC1918-d destined Tailscale UDP so the
80 # phone cleanly falls back to DERP rather than getting poisoned by macOS gvproxy SNAT.
81 add_tailscale_p2p_bypass() {
82     # Dynamic override from watcher.sh (network auto-detection) takes priority over .env
83     local allow_p2p="${TAILSCALE_ALLOW_LAN_P2P:-false}"
84     if [ -f "/app/.lan_p2p_detected" ]; then
85         allow_p2p=$(cat "/app/.lan_p2p_detected" 2>/dev/null | tr -d '[:space:]')
86     fi
87
88     # Always allow P2P directly to the Docker host gateway (the Mac running this container).
89     # The gateway container is always co-located with the host Mac on the same machine,
90     # so direct Tailscale P2P via the Docker bridge is always safe and avoids DERP overhead.
91     # This RETURN rule must be inserted BEFORE the general DROP rules so it takes priority.
92     local default_gw
93     default_gw=$(ip route show default | awk '{print $3}')
94     if [ -n "$default_gw" ]; then
95         if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -d "${default_gw}/32" -j RETURN 2>/dev/null;
96         then
97             iptables -t mangle -I OUTPUT 1 -p udp --sport 41642 -d "${default_gw}/32" -j RETURN 2>/dev/null ||
98             true
99         fi
100     fi
101
102     # Also explicitly whitelist all physical IPs of the host Mac (e.g. 172.17.52.223).
103     # The Tailscale control plane registers the Mac using its physical IP, so the
104     # container will attempt P2P handshakes using that physical IP instead of the
105     # Docker bridge IP. If this physical IP falls within 172.16.0.0/12, it would be dropped.
106     if [ -f "/app/.host_ips" ]; then
107         while IFS= read -r host_ip; do
108             [ -z "$host_ip" ] && continue
109             if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -d "${host_ip}/32" -j RETURN 2>/dev/null;
110             then
111                 iptables -t mangle -I OUTPUT 1 -p udp --sport 41642 -d "${host_ip}/32" -j RETURN 2>/dev/null ||
112                 true
113             fi
114             done < "/app/.host_ips"
115     fi
116
117     if [ "$allow_p2p" != "true" ]; then
118         # Drop Tailscale UDP packets destined for local subnets to prevent macOS gvproxy SNAT

```



```

115 # endpoint poisoning. When disabled (campus/enterprise WPA2-Enterprise with AP isolation),
116 # dropping these forces the S24 to use DERP relays reliably.
117 for subnet in "192.168.0.0/16" "10.0.0.0/8" "172.16.0.0/12"; do
118     if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -d "$subnet" -j DROP 2>/dev/null; then
119         iptables -t mangle -A OUTPUT -p udp --sport 41642 -d "$subnet" -j DROP 2>/dev/null || true
120     fi
121 done
122 else
123     # Remove DROP rules if they exist (switching from restricted to allowed)
124     for subnet in "192.168.0.0/16" "10.0.0.0/8" "172.16.0.0/12"; do
125         while iptables -t mangle -D OUTPUT -p udp --sport 41642 -d "$subnet" -j DROP 2>/dev/null; do ;;
126         done
127     fi
128
129     # Bypass STUN and P2P packets to public IPs (DERP relays and remote peers) out eth0
130     if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -j MARK --set-mark 0x8888 2>/dev/null; then
131         iptables -t mangle -A OUTPUT -p udp --sport 41642 -j MARK --set-mark 0x8888 2>/dev/null || true
132     fi
133     if ! ip rule show | grep -q 'fwmark 0x8888'; then
134         ip rule add fwmark 0x8888 lookup 254 pref 98 2>/dev/null || true
135     fi
136 }
137
138 add_tailscale_p2p_bypass
139
140 add_onion_transparent_proxy() {
141     for ipt in iptables iptables-legacy; do
142         command -v "$ipt" >/dev/null 2>&1 || continue
143         if ! $ipt -t nat -C PREROUTING -d 198.18.0.0/15 -p tcp -j REDIRECT --to-ports 9040 2>/dev/null; then
144             $ipt -t nat -I PREROUTING -d 198.18.0.0/15 -p tcp -j REDIRECT --to-ports 9040 2>/dev/null || true
145         fi
146         if ! $ipt -C FORWARD -d 198.18.0.0/15 -j ACCEPT 2>/dev/null; then
147             $ipt -I FORWARD -d 198.18.0.0/15 -j ACCEPT 2>/dev/null || true
148         fi
149     done
150 }
151
152 add_tailscale_p2p_bypass
153 add_onion_transparent_proxy
154
155 create_log_drop_chain() {
156     local ipt="$1"
157     local chain="$2"
158     local prefix="$3"
159
160     if ! $ipt -N "$chain" 2>/dev/null; then
161         $ipt -F "$chain" 2>/dev/null || true
162     fi
163     $ipt -A "$chain" -j LOG --log-prefix "$prefix"
164     $ipt -A "$chain" -j DROP 2>/dev/null || true
165 }
166
167 add_country_block() {
168     # To add a country to the blocklist, simply define the 'BLOCKED_COUNTRIES' variable in .env with 2-
169     # letter ISO country codes.
170     # You can find the country code by looking at the filename on http://www.ipdeny.com/ipblocks/data/countries/
171     # For example, to block China (cn) and Russia (ru), set: BLOCKED_COUNTRIES="cn ru" in your .env file
172     for zone in $BLOCKED_COUNTRIES; do
173         set_name="block_${zone}"
174         zone_upper=$(echo "$zone" | tr 'a-z' 'A-Z')
175         chain_name="LOG_DROP_${zone_upper}"
176         log_prefix="IP_BLOCK_${zone_upper}: "
177
178         # Create the ipset if it doesn't exist
179         if ! ipset list "$set_name" >/dev/null 2>&1; then
180             ipset create "$set_name" hash:net 2>/dev/null || true
181             echo "routing-fix: Downloading ${zone_upper} IPs for blocklist..."
182             curl -sS "http://www.ipdeny.com/ipblocks/data/countries/${zone}.zone" | grep -v '^#' | while read -r subnet; do
183                 [ -n "$subnet" ] && ipset add "$set_name" "$subnet" 2>/dev/null || true
184             done
185         fi
186
187         # Apply LOG_DROP rule to both backends
188         for ipt in iptables iptables-legacy; do
189             command -v "$ipt" >/dev/null 2>&1 || continue
190             create_log_drop_chain "$ipt" "$chain_name" "$log_prefix"
191         done
192     done
193 }

```

```

192     # Clean up old plain DROP rules to ensure we hit the log-and-drop chains
193     while $ipt -D FORWARD -m set --match-set "$set_name" dst -j DROP 2>/dev/null; do ;; done
194
195     if ! $ipt -C FORWARD -m set --match-set "$set_name" dst -j "$chain_name" 2>/dev/null; then
196         $ipt -I FORWARD -m set --match-set "$set_name" dst -j "$chain_name" 2>/dev/null || true
197     fi
198 done
199 done
200 }
201
202 add_ip_blocklist() {
203     # Only reload when the compiled file has actually changed (mtime check).
204     # This prevents a pointless ipset restore on every 30-second loop tick.
205     if [ ! -f "$IP_BLOCKLIST_FILE" ]; then
206         return 0
207     fi
208
209     CURRENT_MTIME=$(stat -c %Y "$IP_BLOCKLIST_FILE" 2>/dev/null || echo "0")
210     if [ "$CURRENT_MTIME" != "$LAST_IP_MTIME" ]; then
211         echo "routing-fix: IP blocklist changed, reloading..."
212         # ipset restore handles the atomic swap internally:
213         # create_new populate swap with live destroy_new
214         if ipset restore < "$IP_BLOCKLIST_FILE" 2>/dev/null; then
215             LAST_IP_MTIME="$CURRENT_MTIME"
216             echo "routing-fix: IP blocklist loaded ($(ipset list block_malicious | grep -c '[0-9]') entries)."
217         else
218             echo "routing-fix: Warning: ipset restore failed. Retrying next cycle."
219             return 1
220         fi
221     fi
222
223     # Apply FORWARD rules in BOTH iptables backends.
224     for ipt in iptables iptables-legacy; do
225         command -v "$ipt" >/dev/null 2>&1 || continue
226
227         create_log_drop_chain "$ipt" LOG_DROP_MALICIOUS_DST "IP_BLOCK_MALICIOUS_DST: "
228         create_log_drop_chain "$ipt" LOG_DROP_MALICIOUS_SRC "IP_BLOCK_MALICIOUS_SRC: "
229
230         # Clean up old plain DROP rules to ensure we hit the log-and-drop chains
231         while $ipt -D FORWARD -m set --match-set block_malicious dst -j DROP 2>/dev/null; do ;; done
232         while $ipt -D FORWARD -m set --match-set block_malicious src -j DROP 2>/dev/null; do ;; done
233
234         if ! $ipt -C FORWARD -m set --match-set block_malicious dst -j LOG_DROP_MALICIOUS_DST 2>/dev/null;
235         then
236             $ipt -I FORWARD -m set --match-set block_malicious dst -j LOG_DROP_MALICIOUS_DST 2>/dev/null ||
237             true
238         fi
239
240         if ! $ipt -C FORWARD -m set --match-set block_malicious src -j LOG_DROP_MALICIOUS_SRC 2>/dev/null;
241         then
242             $ipt -I FORWARD -m set --match-set block_malicious src -j LOG_DROP_MALICIOUS_SRC 2>/dev/null ||
243             true
244         fi
245     done
246 }
247
248 # Start background block logger (DNS + IP)
249 if ! pgrep -f "nullexit-logger" >/dev/null; then
250     echo "routing-fix: Starting background block logger..."
251     nullexit-logger &
252 fi
253
254 add_country_block
255 add_ip_blocklist
256
257 echo 'routing-fix: Routes applied.'
258
259 UNHEALTHY_COUNT=0
260
261 # Re-assert loop (every 30 seconds)
262 while true; do
263     # Table 200 routes
264     ip route replace "${WARP_ENDPOINT}" via "${DOCKER_GW}" dev eth0 table 200 2>/dev/null || true
265
266     # Tunnel health check
267     if ! curl -m 3 -sS --interface tun0 https://cloudflare.com/cdn-cgi/trace >/dev/null 2>&1; then
268         UNHEALTHY_COUNT=$((UNHEALTHY_COUNT + 1))
269     else
270         UNHEALTHY_COUNT=0
271         if [ -f "/app/TUNNEL_FAILED_CLOSED.marker" ]; then
272             rm -f "/app/TUNNEL_FAILED_CLOSED.marker"
273         fi
274     fi
275 done

```

```

269     fi
270 fi
271
272 if [ "$UNHEALTHY_COUNT" -ge 3 ]; then
273     echo "routing-fix: Tunnel health check failed $UNHEALTHY_COUNT times. Failing closed."
274     ip route del default dev tun0 table 200 2>/dev/null || true
275     ip route replace blackhole default table 200 2>/dev/null || true
276     touch "/app/TUNNEL_FAILED_CLOSED.marker"
277 else
278     ip route replace default dev tun0 table 200 2>/dev/null || true
279 fi
280
281 # Main table: keep Docker subnet route
282 ip route replace "${DOCKER_NET}" dev eth0 2>/dev/null || true
283
284 # CGNAT ip rule for Tailscale
285 if ! ip rule show | grep -q '100.64.0.0/10'; then
286     ip rule add to 100.64.0.0/10 lookup 52 pref 99 2>/dev/null || true
287     echo 'routing-fix: CGNAT rule missing, re-injected'
288 fi
289
290 # FORWARD RELATED, ESTABLISHED: Allow return traffic in both iptables
291 # backends. Gluetun may reset the nftables ruleset on VPN reconnect,
292 # and Tailscale may reset the legacy ruleset on auth refresh.
293 add_fwd_related_established
294
295 # Ensure P2P packets bypass WARP to maintain low latency
296 add_tailscale_p2p_bypass
297
298 # Ensure transparent proxying for .onion is active
299 add_onion_transparent_proxy
300
301 # Enforce country blocklist
302 add_country_block
303
304 # Enforce IP blocklist (reloads automatically when file changes)
305 add_ip_blocklist
306
307 sleep 30 &
308 wait $! || true
309 done

```

46 scripts/rule-compiler/go.mod

```

1 module nullexit/rule-compiler
2
3 go 1.22

```

47 scripts/rule-compiler/main.go

```

1 package main
2
3 import (
4     "bufio"
5     "bytes"
6     "crypto/md5"
7     "encoding/base64"
8     "encoding/hex"
9     "fmt"
10    "io"
11    "net/http"
12    "net/netip"
13    "os"
14    "os/exec"
15    "path/filepath"
16    "regexp"
17    "sort"
18    "strings"
19    "sync"
20    "time"
21 )
22

```

```

23 const (
24     BlackListPath = "black_list.txt"
25     WhiteListPath = "white_list.txt"
26     OutputPath    = "adguard/work/userfilters/compiled_rules.txt"
27     IpOutputPath  = "adguard/work/userfilters/ip_blocklist.ipset"
28     IpCacheDir    = "adguard/work/userfilters/cache/ip"
29     CacheDir      = "adguard/work/userfilters/cache"
30 )
31
32 var CoreLists = []string{
33     "https://raw.githubusercontent.com/anudeepND/blacklist/master/adservers.txt",
34     "https://raw.githubusercontent.com/anudeepND/blacklist/master/facebook.txt",
35     "https://raw.githubusercontent.com/crazy-max/WindowsSpyBlocker/master/data/hosts/spy.txt",
36     "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/adblock/native.samsung.txt",
37     "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/adblock/native.apple.txt",
38     "https://abp.oisd.nl/basic/",
39     "https://adaway.org/hosts.txt",
40     "https://pgl.yoyo.org/adservers/serverlist.php?hostformat=adblockplus&showintro=1&mimetype=plaintext",
41     "https://raw.githubusercontent.com/nextdns/cname-cloaking-blocklist/master/domains",
42 }
43
44 var MediumAdditions = []string{
45     "https://raw.githubusercontent.com/lightswitch05/hosts/master/docs/lists/facebook-extended.txt",
46     "https://someonewhocares.org/hosts/zero/hosts",
47     "https://urihaus.abuse.ch/downloads/hostfile/",
48 }
49
50 var HeavyAdditions = []string{
51     "https://raw.githubusercontent.com/StevenBlack/hosts/master/hosts",
52     "https://raw.githubusercontent.com/Perflyst/PiHoleBlocklist/master/SmartTV-AGH.txt",
53     "https://raw.githubusercontent.com/DandelionSprout/adfilt/master/GameConsoleAdblockList.txt",
54 }
55
56 var IpBlockLists = []string{
57     "https://feodotracker.abuse.ch/downloads/ipblocklist.txt",
58     "https://www.spamhaus.org/drop/drop.txt",
59     "https://rules.emergingthreats.net/fwrules/emerging-Block-IPs.txt",
60     "https://cinsscore.com/list/ci-badguys.txt",
61 }
62
63 func getProfiles() map[string][]string {
64     profiles := make(map[string][]string)
65
66     light := append([]string(nil), CoreLists...)
67     light = append(light, "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/light.txt")
68     profiles["light"] = light
69
70     medium := append([]string(nil), CoreLists...)
71     medium = append(medium, MediumAdditions...)
72     medium = append(medium, "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/multi.txt")
73     profiles["medium"] = medium
74
75     heavy := append([]string(nil), CoreLists...)
76     heavy = append(heavy, MediumAdditions...)
77     heavy = append(heavy, HeavyAdditions...)
78     heavy = append(heavy, "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/pro.txt")
79     profiles["heavy"] = heavy
80
81     return profiles
82 }
83
84 func loadEnvProfile() string {
85     profile := "medium"
86
87     if bytesData, err := os.ReadFile(".env"); err == nil {
88         scanner := bufio.NewScanner(strings.NewReader(string(bytesData)))
89         for scanner.Scan() {
90             line := strings.TrimSpace(scanner.Text())
91             if line != "" && !strings.HasPrefix(line, "#") {
92                 parts := strings.SplitN(line, "=", 2)
93                 if len(parts) == 2 {
94                     key := strings.TrimSpace(parts[0])
95                     val := strings.TrimSpace(parts[1])
96                     if key == "GATEWAY_RULE_PROFILE" {
97                         val = strings.Trim(val, "\"'")
98                         profile = strings.ToLower(val)
99                     }
100                 }
101             }
102         }
103     }

```

```

103 } else if !os.IsNotExist(err) {
104     fmt.Printf("Warning: Failed to read .env file for profile configuration (%v). Using default.\n", err)
105 }
106
107 if envVal := os.Getenv("GATEWAY_RULE_PROFILE"); envVal != "" {
108     profile = strings.ToLower(envVal)
109 }
110
111 profiles := getProfiles()
112 if _, exists := profiles[profile]; !exists {
113     fmt.Printf("Warning: Profile '%s' is invalid. Falling back to 'medium'.\n", profile)
114     profile = "medium"
115 }
116 return profile
117 }
118
119 func loadDomains(filepath string) map[string]bool {
120     domains := make(map[string]bool)
121     if bytesData, err := os.ReadFile(filepath); err == nil {
122         scanner := bufio.NewScanner(strings.NewReader(string(bytesData)))
123         for scanner.Scan() {
124             line := strings.TrimSpace(scanner.Text())
125             if line != "" && !strings.HasPrefix(line, "#") {
126                 domains[strings.ToLower(line)] = true
127             }
128         }
129     }
130     return domains
131 }
132
133 func parseDomainsFromContent(content string) map[string]bool {
134     domains := make(map[string]bool)
135     scanner := bufio.NewScanner(strings.NewReader(content))
136     ipRegex := regexp.MustCompile(`^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$`)
137     agRegex := regexp.MustCompile(`^\| |([a-z0-9.-]+)\^$`)
138     domainRegex := regexp.MustCompile(`^[a-z0-9.-]+$`)
139
140     for scanner.Scan() {
141         line := strings.TrimSpace(scanner.Text())
142         if line == "" || strings.HasPrefix(line, "!") || strings.HasPrefix(line, "[") {
143             continue
144         }
145         parts := strings.SplitN(line, "#", 2)
146         line = strings.TrimSpace(parts[0])
147         if line == "" {
148             continue
149         }
150
151         fields := strings.Fields(line)
152         var rule string
153         if len(fields) >= 2 {
154             if ipRegex.MatchString(fields[0]) {
155                 rule = fields[1]
156             } else {
157                 rule = fields[0]
158             }
159         } else {
160             rule = fields[0]
161         }
162
163         rule = strings.TrimSpace(strings.ToLower(rule))
164
165         var domain string
166         if m := agRegex.FindStringSubmatch(rule); m != nil {
167             domain = m[1]
168         } else if domainRegex.MatchString(rule) {
169             domain = rule
170         } else {
171             domain = rule
172         }
173
174         if domain != "" && domain != "localhost" && domain != "0.0.0.0" && domain != "127.0.0.1" && domain !=
175             "broadcasthost" {
176             domains[domain] = true
177         }
178     }
179     return domains
180 }
181
182 // WARNING: Parsing AdGuardHome.yaml using regex instead of a proper YAML parser
183 // is brittle and assumes the exact format currently emitted by AdGuard.

```

```

183 // If an AdGuard version bump updates the configuration schema format,
184 // this parser will fail silently or loudly and should be refactored to use gopkg.in/yaml.v3.
185 func getAdguardNativeLists() []string {
186     yamlPath := "adguard/conf/AdGuardHome.yaml"
187     var enabledUrls []string
188     if _, err := os.Stat(yamlPath); os.IsNotExist(err) {
189         return enabledUrls
190     }
191
192     contentBytes, err := os.ReadFile(yamlPath)
193     if err != nil {
194         fmt.Printf("Warning: Failed to parse AdGuardHome.yaml for native lists (%v)\n", err)
195         return enabledUrls
196     }
197     content := string(contentBytes)
198
199     filtersRegex := regexp.MustCompile(`(?ms)^filters:(.*)"(?:[a-zA-Z_]+:|\z)` )
200     match := filtersRegex.FindStringSubmatch(content)
201     if len(match) > 1 {
202         filtersText := match[1]
203         blocks := strings.Split(filtersText, "\n - ")
204         urlRegex := regexp.MustCompile(`url:\s*(\S+)`)
205
206         for _, block := range blocks {
207             if strings.TrimSpace(block) == "" {
208                 continue
209             }
210             if strings.Contains(block, "enabled: true") {
211                 urlMatch := urlRegex.FindStringSubmatch(block)
212                 if len(urlMatch) > 1 {
213                     url := urlMatch[1]
214                     if !strings.Contains(url, "compiled_rules.txt") {
215                         enabledUrls = append(enabledUrls, url)
216                     }
217                 }
218             }
219         }
220     }
221     return enabledUrls
222 }
223
224 func getAdguardCompiledRulesCachePath() string {
225     yamlPath := "adguard/conf/AdGuardHome.yaml"
226     if _, err := os.Stat(yamlPath); os.IsNotExist(err) {
227         return ""
228     }
229
230     contentBytes, err := os.ReadFile(yamlPath)
231     if err != nil {
232         fmt.Printf("Warning: Failed to resolve compiled_rules.txt cache path (%v)\n", err)
233         return ""
234     }
235     content := string(contentBytes)
236
237     filtersRegex := regexp.MustCompile(`(?ms)^filters:(.*)"(?:[a-zA-Z_]+:|\z)` )
238     match := filtersRegex.FindStringSubmatch(content)
239     if len(match) > 1 {
240         blocks := strings.Split(match[1], "\n - ")
241         idRegex := regexp.MustCompile(`id:\s*(\d+)`)
242         for _, block := range blocks {
243             if !strings.Contains(block, "compiled_rules.txt") {
244                 continue
245             }
246             idMatch := idRegex.FindStringSubmatch(block)
247             if len(idMatch) > 1 {
248                 return fmt.Sprintf("adguard/work/data/filters/%s.txt", idMatch[1])
249             }
250         }
251     }
252     return ""
253 }
254
255 func handleFetchError(urlStr string, cacheFile string, err error) map[string]bool {
256     if _, statErr := os.Stat(cacheFile); statErr == nil {
257         fmt.Printf(" -> Warning: Failed to fetch %s (%v). Falling back to expired local cache.\n", urlStr, err)
258         content, readErr := os.ReadFile(cacheFile)
259         if readErr == nil {
260             domains := parseDomainsFromContent(string(content))
261             fmt.Printf(" -> Loaded from expired cache (%d domains).\n", len(domains))
262             return domains
263         }
264     }
265 }

```

```

263     }
264     fmt.Printf(" -> Warning: Failed to read expired cache (%v). Using local lists only.\n", readErr)
265 } else {
266     fmt.Printf(" -> Warning: Failed to fetch %s (%v). Using local lists only for this source.\n", urlStr,
267         err)
268 }
269 return make(map[string]bool)
270 }
271 func fetchRemoteDomains(urlStr string) map[string]bool {
272     os.MkdirAll(CacheDir, 0755)
273
274     hash := md5.Sum([]byte(urlStr))
275     urlHash := hex.EncodeToString(hash[:])
276     cacheFile := filepath.Join(CacheDir, fmt.Sprintf("%s.txt", urlHash))
277
278     if stat, err := os.Stat(cacheFile); err == nil {
279         fileAge := time.Since(stat.ModTime()).Seconds()
280         if fileAge < 86400 {
281             fmt.Printf("Loading remote blacklist from cache: %s\n", urlStr)
282             content, err := os.ReadFile(cacheFile)
283             if err == nil {
284                 domains := parseDomainsFromContent(string(content))
285                 fmt.Printf(" -> Loaded from cache (copy is %.1f hours old, %d domains).\n", fileAge/3600.0, len(
286                     domains))
287                 return domains
288             }
289             fmt.Printf(" -> Warning: Failed to read cache file %s (%v). Re-fetching...\n", cacheFile, err)
290         }
291     }
292     fmt.Printf("Fetching remote blacklist from: %s ...\n", urlStr)
293
294     req, err := http.NewRequest("GET", urlStr, nil)
295     if err != nil {
296         return handleFetchError(urlStr, cacheFile, err)
297     }
298     req.Header.Set("User-Agent", "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)")
299
300     client := &http.Client{Timeout: 15 * time.Second}
301     resp, err := client.Do(req)
302     if err != nil {
303         return handleFetchError(urlStr, cacheFile, err)
304     }
305     defer resp.Body.Close()
306
307     if resp.StatusCode != 200 {
308         return handleFetchError(urlStr, cacheFile, fmt.Errorf("HTTP %d", resp.StatusCode))
309     }
310
311     bodyBytes, err := io.ReadAll(resp.Body)
312     if err != nil {
313         return handleFetchError(urlStr, cacheFile, err)
314     }
315
316     content := string(bodyBytes)
317     domains := parseDomainsFromContent(content)
318
319     if len(domains) < 10 {
320         return handleFetchError(urlStr, cacheFile, fmt.Errorf("Sanity check failed: only found %d domains.
321             Possible 404 or bad URL", len(domains)))
322     }
323
324     err = os.WriteFile(cacheFile, bodyBytes, 0644)
325     if err != nil {
326         fmt.Printf(" -> Warning: Failed to save cache file (%v)\n", err)
327     }
328
329     fmt.Printf(" -> Successfully fetched and cached %d domains.\n", len(domains))
330     return domains
331 }
332 func optimizeSubdomains(domains map[string]bool, listName string) map[string]bool {
333     fmt.Printf("Optimizing %s by removing redundant subdomains...\n", listName)
334     rawCount := len(domains)
335     if rawCount == 0 {
336         return make(map[string]bool)
337     }
338
339     optimized := make(map[string]bool)
340     for domain := range domains {

```



```

341     if strings.ContainsAny(domain, "|~@$/") {
342         optimized[domain] = true
343         continue
344     }
345
346     parts := strings.Split(domain, ".")
347     hasParent := false
348
349     for i := 1; i < len(parts)-1; i++ {
350         parent := strings.Join(parts[i:], ".")
351         if domains[parent] {
352             hasParent = true
353             break
354         }
355     }
356
357     if !hasParent {
358         optimized[domain] = true
359     }
360 }
361
362 saved := rawCount - len(optimized)
363 reduction := float64(0)
364 if rawCount > 0 {
365     reduction = (float64(saved) / float64(rawCount)) * 100
366 }
367 fmt.Printf(" -> Reduced %s from %d to %d domains (-%d / %.1f%% reduction).\n", listName, rawCount, len(
    optimized), saved, reduction)
368 return optimized
369 }
370
371 func parseIpsFromContent(content string) map[string]bool {
372     ips := make(map[string]bool)
373     scanner := bufio.NewScanner(strings.NewReader(content))
374
375     for scanner.Scan() {
376         line := strings.TrimSpace(scanner.Text())
377         if line == "" || strings.HasPrefix(line, "#") || strings.HasPrefix(line, ";") {
378             continue
379         }
380
381         fields := strings.Fields(line)
382         if len(fields) > 0 {
383             token := strings.TrimRight(fields[0], ";,")
384             if _, err := netip.ParsePrefix(token); err == nil {
385                 ips[token] = true
386             } else if _, err := netip.ParseAddr(token); err == nil {
387                 ips[token] = true
388             }
389         }
390     }
391     return ips
392 }
393
394 func handleIpFetchError(urlStr string, cacheFile string, err error) map[string]bool {
395     if _, statErr := os.Stat(cacheFile); statErr == nil {
396         fmt.Printf(" -> Warning: fetch failed (%v). Falling back to stale cache.\n", err)
397         content, readErr := os.ReadFile(cacheFile)
398         if readErr == nil {
399             ips := parseIpsFromContent(string(content))
400             fmt.Printf(" -> %d IPs loaded from stale cache.\n", len(ips))
401             return ips
402         }
403         fmt.Printf(" -> Warning: stale cache also failed (%v).\n", readErr)
404     } else {
405         fmt.Printf(" -> Warning: %s failed (%v). No cache available. Skipping.\n", urlStr, err)
406     }
407     return make(map[string]bool)
408 }
409
410 func fetchRemoteIps(urlStr string) map[string]bool {
411     os.MkdirAll(IpCacheDir, 0755)
412
413     hash := md5.Sum([]byte(urlStr))
414     urlHash := hex.EncodeToString(hash[:])
415     cacheFile := filepath.Join(IpCacheDir, fmt.Sprintf("%s.txt", urlHash))
416
417     if stat, err := os.Stat(cacheFile); err == nil {
418         fileAge := time.Since(stat.ModTime()).Seconds()
419         if fileAge < 86400 {
420             fmt.Printf("Loading IP feed from cache: %s\n", urlStr)

```

```

421     content, err := os.ReadFile(cacheFile)
422     if err == nil {
423         ips := parseIpsFromContent(string(content))
424         fmt.Printf(" -> %d IPs (cache is %.1fh old).\n", len(ips), fileAge/3600.0)
425         return ips
426     }
427     fmt.Printf(" -> Warning: cache read failed (%v). Re-fetching...\n", err)
428 }
429 }
430
431 fmt.Printf("Fetching IP feed: %s ...\n", urlStr)
432 req, err := http.NewRequest("GET", urlStr, nil)
433 if err != nil {
434     return handleIpFetchError(urlStr, cacheFile, err)
435 }
436 req.Header.Set("User-Agent", "Mozilla/5.0")
437
438 client := &http.Client{Timeout: 15 * time.Second}
439 resp, err := client.Do(req)
440 if err != nil {
441     return handleIpFetchError(urlStr, cacheFile, err)
442 }
443 defer resp.Body.Close()
444
445 if resp.StatusCode != 200 {
446     return handleIpFetchError(urlStr, cacheFile, fmt.Errorf("HTTP %d", resp.StatusCode))
447 }
448
449 bodyBytes, err := io.ReadAll(resp.Body)
450 if err != nil {
451     return handleIpFetchError(urlStr, cacheFile, err)
452 }
453
454 content := string(bodyBytes)
455 ips := parseIpsFromContent(content)
456 if len(ips) < 1 {
457     return handleIpFetchError(urlStr, cacheFile, fmt.Errorf("Sanity check failed: only %d IPs found. Possible 404", len(ips)))
458 }
459
460 err = os.WriteFile(cacheFile, bodyBytes, 0644)
461 if err != nil {
462     fmt.Printf(" -> Warning: cache write failed (%v)\n", err)
463 }
464
465 fmt.Printf(" -> %d IPs fetched and cached.\n", len(ips))
466 return ips
467 }
468
469 func compileIpBlocklist() int {
470     fmt.Println("\n IP Blocklist Compilation ")
471
472     allIps := make(map[string]bool)
473     var mu sync.Mutex
474     var wg sync.WaitGroup
475
476     maxThreads := 8
477     if len(IpBlockLists) < 8 {
478         maxThreads = len(IpBlockLists)
479     }
480     semaphore := make(chan struct{}, maxThreads)
481
482     for _, urlStr := range IpBlockLists {
483         wg.Add(1)
484         go func(u string) {
485             defer wg.Done()
486             semaphore <- struct{}{}
487             defer func() { <-semaphore }()
488
489             ips := fetchRemoteIps(u)
490             mu.Lock()
491             for ip := range ips {
492                 allIps[ip] = true
493             }
494             mu.Unlock()
495         }(urlStr)
496     }
497     wg.Wait()
498
499     fmt.Printf("Total unique entries before filtering: %d\n", len(allIps))
500

```

```

501 reservedStr := []string{
502     "10.0.0.0/8",
503     "172.16.0.0/12",
504     "192.168.0.0/16",
505     "127.0.0.0/8",
506     "169.254.0.0/16",
507     "100.64.0.0/10",
508     "0.0.0.0/8",
509 }
510 var reserved []netip.Prefix
511 for _, s := range reservedStr {
512     p, _ := netip.ParsePrefix(s)
513     reserved = append(reserved, p)
514 }
515
516 cleanIps := make(map[string]bool)
517 for entry := range allIps {
518     var prefix netip.Prefix
519     p, err := netip.ParsePrefix(entry)
520     if err != nil {
521         addr, err2 := netip.ParseAddr(entry)
522         if err2 != nil {
523             continue
524         }
525         prefix = netip.PrefixFrom(addr, addr.BitLen())
526     } else {
527         prefix = p
528     }
529
530     overlaps := false
531     for _, r := range reserved {
532         if r.Overlaps(prefix) {
533             overlaps = true
534             break
535         }
536     }
537     if !overlaps {
538         cleanIps[entry] = true
539     }
540 }
541
542 removed := len(allIps) - len(cleanIps)
543 if removed > 0 {
544     fmt.Printf(" -> Removed %d private/reserved entries.\n", removed)
545 }
546 fmt.Printf(" -> Final IP blocklist: %d entries.\n", len(cleanIps))
547
548 os.MkdirAll(filepath.Dir(IpOutputPath), 0755)
549
550 f, err := os.Create(IpOutputPath)
551 if err != nil {
552     fmt.Printf("Error writing IP output file: %v\n", err)
553     return len(cleanIps)
554 }
555 defer f.Close()
556
557 f.WriteString("# nullexit Compiled IP Blocklist\n")
558 f.WriteString("# Sources: Feodo Tracker, Spamhaus DROP/EDROP, Emerging Threats, CINS\n")
559 f.WriteString(fmt.Sprintf("# Entries: %d\n", len(cleanIps)))
560
561 f.WriteString("create block_malicious_new hash:net maxelem 200000 -exist\n")
562
563 var sortedIps []string
564 for ip := range cleanIps {
565     sortedIps = append(sortedIps, ip)
566 }
567 sort.Strings(sortedIps)
568
569 for _, ip := range sortedIps {
570     f.WriteString(fmt.Sprintf("add block_malicious_new %s -exist\n", ip))
571 }
572
573 f.WriteString("create block_malicious hash:net maxelem 200000 -exist\n")
574 f.WriteString("swap block_malicious block_malicious_new\n")
575 f.WriteString("destroy block_malicious_new\n")
576
577 fmt.Printf("IP blocklist written to %s\n", IpOutputPath)
578 return len(cleanIps)
579 }
580
581 func main() {

```

```

582 profile := loadEnvProfile()
583 fmt.Printf("Active memory profile: '%s'\n", strings.ToUpper(profile))
584
585 localBlackList := loadDomains(BlackListPath)
586 whiteList := loadDomains(WhiteListPath)
587
588 blackList := make(map[string]bool)
589 for k := range localBlackList {
590     blackList[k] = true
591 }
592
593 profiles := getProfiles()
594 urlsToFetch := profiles[profile]
595
596 fmt.Printf("\nStarting concurrent downloads for %d lists...\n", len(urlsToFetch))
597 startTime := time.Now()
598
599 var mu sync.Mutex
600 var wg sync.WaitGroup
601
602 maxThreads := 16
603 if len(urlsToFetch) < 16 {
604     maxThreads = len(urlsToFetch)
605 }
606 semaphore := make(chan struct{}, maxThreads)
607
608 for _, urlStr := range urlsToFetch {
609     wg.Add(1)
610     go func(u string) {
611         defer wg.Done()
612         semaphore <- struct{}{}
613         defer func() { <-semaphore }()
614
615         domains := fetchRemoteDomains(u)
616         mu.Lock()
617         for d := range domains {
618             blackList[d] = true
619         }
620         mu.Unlock()
621     }(urlStr)
622 }
623 wg.Wait()
624
625 fmt.Printf("Finished concurrent downloads in %.2f seconds using %d threads.\n", time.Since(startTime).
        Seconds(), maxThreads)
626
627 adguardNativeUrls := getAdguardNativeLists()
628 var adguardNativeDomains map[string]bool
629 if len(adguardNativeUrls) > 0 {
630     fmt.Printf("\nFetching %d AdGuard native list(s) for deduplication...\n", len(adguardNativeUrls))
631     adguardNativeDomains = make(map[string]bool)
632
633     maxNativeThreads := 4
634     if len(adguardNativeUrls) < 4 {
635         maxNativeThreads = len(adguardNativeUrls)
636     }
637     nativeSemaphore := make(chan struct{}, maxNativeThreads)
638
639     for _, urlStr := range adguardNativeUrls {
640         wg.Add(1)
641         go func(u string) {
642             defer wg.Done()
643             nativeSemaphore <- struct{}{}
644             defer func() { <-nativeSemaphore }()
645
646             domains := fetchRemoteDomains(u)
647             mu.Lock()
648             for d := range domains {
649                 adguardNativeDomains[d] = true
650             }
651             mu.Unlock()
652         }(urlStr)
653     }
654     wg.Wait()
655
656     if len(adguardNativeDomains) > 0 {
657         fmt.Println("Deduplicating compiled rules against AdGuard native lists...")
658         originalSize := len(blackList)
659         for d := range adguardNativeDomains {
660             delete(blackList, d)
661         }

```

```

662     fmt.Printf(" -> Removed %d redundant rules already covered by AdGuard.\n", originalSize-len(
        blackList))
663 }
664 }
665
666 var contradictions []string
667 for d := range whiteList {
668     if blackList[d] {
669         contradictions = append(contradictions, d)
670     }
671 }
672
673 if len(contradictions) > 0 {
674     fmt.Printf("Detected %d contradictions. Whitelist taking priority.\n", len(contradictions))
675     sort.Strings(contradictions)
676     for _, domain := range contradictions {
677         if localBlackList[domain] {
678             fmt.Printf(" -> Removed local blacklist domain: %s\n", domain)
679         }
680         delete(blackList, domain)
681     }
682 }
683
684 blackList = optimizeSubdomains(blackList, "blacklist")
685 whiteList = optimizeSubdomains(whiteList, "whitelist")
686
687 os.MkdirAll(filepath.Dir(OutputPath), 0755)
688 outFile, err := os.Create(OutputPath)
689 if err != nil {
690     fmt.Printf("Error creating output file: %v\n", err)
691     return
692 }
693 defer outFile.Close()
694
695 outFile.WriteString("! Custom Compiled Rules (Auto-Generated)\n")
696 outFile.WriteString(fmt.Sprintf("! Memory Profile: %s\n", strings.ToUpper(profile)))
697 outFile.WriteString(fmt.Sprintf("! Total Block Rules: %d\n", len(blackList)))
698 outFile.WriteString(fmt.Sprintf("! Native AdGuard Rules: %d\n", len(adguardNativeDomains)))
699 outFile.WriteString(fmt.Sprintf("! Total Whitelist Rules: %d\n", len(whiteList)))
700
701 outFile.WriteString("! --- Blacklist Rules ---\n")
702
703 var sortedBlacklist []string
704 for d := range blackList {
705     sortedBlacklist = append(sortedBlacklist, d)
706 }
707 sort.Strings(sortedBlacklist)
708
709 for _, domain := range sortedBlacklist {
710     if strings.Contains(domain, "$") {
711         parts := strings.SplitN(domain, "$", 2)
712         dom, mod := parts[0], parts[1]
713         if strings.HasPrefix(dom, "|") {
714             if strings.HasSuffix(dom, "^") {
715                 outFile.WriteString(fmt.Sprintf("%s%s\n", dom, mod))
716             } else {
717                 outFile.WriteString(fmt.Sprintf("%s^%s\n", dom, mod))
718             }
719         } else {
720             outFile.WriteString(fmt.Sprintf("||%s^%s\n", dom, mod))
721         }
722     } else if strings.HasPrefix(domain, "/") || strings.HasPrefix(domain, "|") || strings.HasSuffix(
        domain, "|") || strings.HasSuffix(domain, "^") {
723         outFile.WriteString(fmt.Sprintf("%s\n", domain))
724     } else {
725         outFile.WriteString(fmt.Sprintf("||%s\n", domain))
726     }
727 }
728
729 outFile.WriteString("\n! --- Whitelist Rules ---\n")
730 var sortedWhitelist []string
731 for d := range whiteList {
732     sortedWhitelist = append(sortedWhitelist, d)
733 }
734 sort.Strings(sortedWhitelist)
735
736 for _, domain := range sortedWhitelist {
737     if strings.HasPrefix(domain, "/") || strings.HasPrefix(domain, "|") || strings.HasSuffix(domain, "|")
        || strings.HasSuffix(domain, "^") || strings.HasPrefix(domain, "@@") {
738         rule := domain
739         if !strings.HasPrefix(domain, "@@") {

```

```

740     rule = "@@" + domain
741 }
742 outFile.WriteString(fmt.Sprintf("%s\n", rule))
743 } else {
744     outFile.WriteString(fmt.Sprintf("@@||%s^\n", domain))
745 }
746 }
747
748 fmt.Printf("\nSuccessfully compiled %d block rules and %d allow rules to %s\n", len(blackList), len(
    whiteList), OutputPath)
749
750 cachedFilterPath := getAdguardCompiledRulesCachePath()
751 if cachedFilterPath != "" {
752     input, err := os.ReadFile(OutputPath)
753     if err == nil {
754         err = os.WriteFile(cachedFilterPath, input, 0644)
755         if err == nil {
756             fmt.Printf("Updated AdGuard filter cache: %s\n", cachedFilterPath)
757         } else {
758             fmt.Printf("Warning: Could not update AdGuard filter cache %s (%v)\n", cachedFilterPath, err)
759         }
760     } else {
761         fmt.Printf("Warning: Could not read OutputPath to copy to cache (%v)\n", err)
762     }
763 } else {
764     fmt.Println("Warning: Could not determine compiled_rules.txt cache path from AdGuardHome.yaml;
        skipping cache update.")
765 }
766
767 adguardReachable := false
768
769 dockerComposeYamlExists := false
770 if _, err := os.Stat("docker-compose.yml"); err == nil {
771     dockerComposeYamlExists = true
772 }
773 dockerPath, err := exec.LookPath("docker")
774 if err == nil && dockerComposeYamlExists {
775     cmd := exec.Command(dockerPath, "compose", "ps", "--status", "running", "-q", "adguardhome")
776     out, err := cmd.Output()
777     if err == nil && strings.TrimSpace(string(out)) != "" {
778         fmt.Println("Force-recreating AdGuard Home container to drop persisted filter state...")
779         upCmd := exec.Command(dockerPath, "compose", "up", "-d", "--force-recreate", "--no-deps", "
            adguardhome")
780         err := upCmd.Run()
781         if err == nil {
782             fmt.Println("AdGuard Home force-recreated successfully.")
783             time.Sleep(8 * time.Second)
784             adguardReachable = true
785         } else {
786             fmt.Println("Failed to restart AdGuard Home. Is it running?")
787         }
788     }
789 }
790
791 if adguardReachable {
792     agPass := "nullexit"
793     if bytesData, err := os.ReadFile(".env"); err == nil {
794         scanner := bufio.NewScanner(strings.NewReader(string(bytesData)))
795         for scanner.Scan() {
796             line := scanner.Text()
797             if strings.HasPrefix(line, "ADGUARD_PASSWORD=") {
798                 parts := strings.SplitN(line, "=", 2)
799                 if len(parts) == 2 {
800                     agPass = strings.Trim(strings.TrimSpace(parts[1]), "\"'")
801                 }
802                 break
803             }
804         }
805     }
806
807     creds := base64.StdEncoding.EncodeToString([]byte(fmt.Sprintf("admin:%s", agPass)))
808     req, err := http.NewRequest("POST", "http://127.0.0.1:3000/control/filtering/refresh", bytes.
        NewBuffer([]byte("{}")))
809     if err == nil {
810         req.Header.Set("Authorization", fmt.Sprintf("Basic %s", creds))
811         req.Header.Set("Content-Type", "application/json")
812         client := &http.Client{Timeout: 15 * time.Second}
813         resp, err := client.Do(req)
814         if err == nil {
815             fmt.Printf("Triggered AdGuard filter refresh via REST API (HTTP=%d).\n", resp.StatusCode)
816             resp.Body.Close()

```

```

817     } else {
818         fmt.Printf("Warning: AdGuard filter refresh API call failed (%v). New whitelist will not load
until next refresh tick (~24h) or manual refresh.\n", err)
819     }
820 }
821 }
822
823 compileIpBlocklist()
824 }

```

48 scripts/setup-common.sh

```

1  #!/bin/bash
2  # 1. Docker
3  step "Checking Docker"
4
5  if ! command -v docker >> output.log 2>&1; then
6      echo ""
7      if [[ "$OSTYPE" == "darwin"* ]]; then
8          echo " Docker is not installed."
9          # Note: Homebrew aggressively rolls forward and discourages version pinning.
10         # This setup is confirmed stable on Colima v0.10.3 and Docker v29.6.1.
11         read -rp " Would you like to automatically install Colima & Docker via Homebrew? [y/N]: "
12         INSTALL_DOCKER
13         if [[ "$INSTALL_DOCKER" == "y" || "$INSTALL_DOCKER" == "Y" ]]; then
14             step "Installing Colima and Docker..."
15             brew install colima docker docker-compose
16             step "Starting Colima VM..."
17             colima start --memory 0.6 --vm-type vz --network-address --network-mode bridged
18         else
19             echo ""
20             echo " Please install Docker manually. Two options:"
21             echo " A) Colima (Recommended): brew install colima docker docker-compose && colima start --
memory 0.6 --vm-type vz --network-address --network-mode bridged"
22             echo " B) Docker Desktop: https://www.docker.com/products/docker-desktop/"
23             die "Install Docker, then re-run this script."
24         fi
25     else
26         echo " Docker is not installed."
27         # Note: The Docker convenience script always fetches the latest stable release.
28         # This setup is confirmed stable on Docker Engine v29.6.1.
29         read -rp " Would you like to automatically install Docker? (Requires sudo) [y/N]: "
30         INSTALL_DOCKER
31         if [[ "$INSTALL_DOCKER" == "y" || "$INSTALL_DOCKER" == "Y" ]]; then
32             step "Installing Docker..."
33             curl -fsSL https://get.docker.com | sh
34             sudo usermod -aG docker "$USER"
35             echo -e "\n\033[0;32m[OK] Docker installed successfully!\033[0m"
36             echo -e "\033[0;33m[!] CRITICAL: You must log out of your SSH session and log back in for
Docker permissions to apply.\033[0m"
37             die "Please log out, log back in, and re-run setup.sh."
38         else
39             echo " Please install Docker manually:"
40             echo " curl -fsSL https://get.docker.com | sh"
41             echo " sudo usermod -aG docker \"$USER && newgrp docker"
42             die "Install Docker, then re-run this script."
43         fi
44     fi
45 fi
46
47 if ! docker info >> output.log 2>&1; then
48     echo ""
49     if [[ "$OSTYPE" == "darwin"* ]]; then
50         echo " Docker is installed but not running."
51         echo " If using Colima: colima start --memory 0.6 --vm-type vz --network-address --
network-mode bridged"
52         echo " If using Docker Desktop: open the app."
53     else
54         echo " Docker is installed but not running."
55         echo " Run: sudo systemctl start docker"
56     fi
57     die "Start Docker, then re-run this script."
58 fi
59
60 if ! docker compose version >> output.log 2>&1; then
61     die "Docker Compose v2 not found. Update Docker or install the compose plugin:\n https://docs.docker
.com/compose/install/"
62 fi

```



```

60 fi
61
62 ok "Docker $(docker --version | grep -oE '[0-9.]+' | head -1) is running."
63
64 # 1.5 Python 3
65 step "Checking Python 3"
66
67 if ! command -v python3 >> output.log 2>&1; then
68     echo ""
69     if [[ "$OSTYPE" == "darwin"* ]]; then
70         echo " Python 3 is not installed."
71         echo " macOS includes it via Xcode Command Line Tools, or you can install it via Homebrew:"
72         echo "         brew install python3"
73         die "Install Python 3, then re-run this script."
74     else
75         echo " Python 3 is not installed."
76         echo " Please install it using your system's package manager. For example:"
77         echo "     Debian/Ubuntu:  sudo apt install python3"
78         echo "     Fedora:         sudo dnf install python3"
79         echo "     Arch:           sudo pacman -S python"
80         die "Install Python 3, then re-run this script."
81     fi
82 fi
83
84 # Note: This setup is confirmed stable on Python 3.9.6 (macOS default).
85 PYTHON_VER=$(python3 --version 2>&1 | head -n 1)
86 ok "$PYTHON_VER is installed."
87
88 # 2. Tailscale (host)
89 step "Checking Tailscale (host)"
90 # The host needs its own Tailscale installation separate from the Docker
91 # container so Toggle-Gateway can run 'tailscale down' before stopping
92 # containers and 'tailscale up' after starting them. Without this, the
93 # exit-node deadlock that setup.sh is designed to prevent can still occur.
94
95 # Resolve the tailscale binary: CLI (brew/package) or bundled macOS app
96 TAILSCALE_BIN=""
97 if command -v tailscale >> output.log 2>&1; then
98     TAILSCALE_BIN="tailscale"
99 elif [[ -x "/Applications/Tailscale.app/Contents/MacOS/Tailscale" ]]; then
100     TAILSCALE_BIN="/Applications/Tailscale.app/Contents/MacOS/Tailscale"
101 fi
102
103 RECOMMENDED_TS_VERSION="1.98.5"
104
105 if [[ -z "$TAILSCALE_BIN" ]]; then
106     warn "Tailscale not found on host installing..."
107
108     if [[ "$OSTYPE" == "darwin"* ]]; then
109         if command -v brew >> output.log 2>&1; then
110             step "Installing Tailscale CLI formula via Homebrew (standalone, no .app GUI)..."
111             # Note: We install Tailscale via Homebrew, targeting the stable version (recommended: 1.98.5)
112             brew install tailscale -q
113             TAILSCALE_BIN="tailscale"
114
115             step "Starting tailscaled as a per-user LaunchAgent (auto-starts at login)..."
116             if brew services start tailscale 2>&1 | grep -qE 'Successfully|started'; then
117                 ok "tailscaled LaunchAgent registered."
118             else
119                 warn "Could not auto-start tailscaled. Run 'brew services start tailscale' manually."
120                 warn "For a system-wide LaunchDaemon that runs before login, run instead:"
121                 warn "    sudo brew services start tailscale"
122             fi
123             echo ""
124         else
125             die "Homebrew not found. Install Tailscale manually (recommended version: ${RECOMMENDED_TS_VERSION}):
126             \n https://tailscale.com/download/mac\n Then re-run this script."
127         fi
128     elif [[ "$OSTYPE" == "linux-gnu"* ]]; then
129         curl -fsSL https://tailscale.com/install.sh | sh
130         sudo systemctl enable --now tailscaled 2>> output.log || true
131         TAILSCALE_BIN="tailscale"
132     else
133         die "Unsupported OS. Install Tailscale manually (recommended version: ${RECOMMENDED_TS_VERSION}):
134         \n https://tailscale.com/download\n Then re-run this script."
135     fi
136
137     TAILSCALE_VER=$(("${TAILSCALE_BIN}" version 2>> output.log | head -n 1)
138     if [[ "$TAILSCALE_VER" != "$RECOMMENDED_TS_VERSION" ]]; then

```

```

139     warn "Tailscale installed, but version (${TAILSCALE_VER}) differs from recommended version (
140     $RECOMMENDED_TS_VERSION)."
141 else
142     ok "Tailscale installed (version $TAILSCALE_VER)."
143 fi
144 else
145     TAILSCALE_VER=$((${TAILSCALE_BIN} version 2>> output.log | head -n 1)
146     if [[ "$TAILSCALE_VER" != "$RECOMMENDED_TS_VERSION" ]]; then
147         warn "Tailscale is already installed ($TAILSCALE_VER), but recommended version is
148         $RECOMMENDED_TS_VERSION for host/container consistency."
149     else
150         ok "Tailscale is already installed at recommended version ($TAILSCALE_VER)."
151     fi
152 fi
153 # Authenticate the host machine if it isn't already connected.
154 # This is an interactive step it opens a browser login URL.
155 # We do it now, before containers start, so Toggle-Gateway can
156 # safely run 'tailscale down/up' from day one.
157 if ! ${TAILSCALE_BIN} status >> output.log 2>&1; then
158     echo ""
159     echo " Your host machine needs to join your Tailscale network."
160     echo " A browser window or login URL will appear authenticate, then"
161     echo " return here. The script will continue automatically."
162     echo ""
163     if [[ "$OSTYPE" == "linux-gnu*" ]]; then
164         sudo ${TAILSCALE_BIN} up
165     else
166         ${TAILSCALE_BIN} up
167     fi
168     ok "Host machine connected to Tailscale."
169 else
170     ok "Host machine is already connected to Tailscale."
171 fi
172 # 3. wgcf
173 step "Checking wgcf (Cloudflare WARP key tool)"
174
175 if ! command -v wgcf >> output.log 2>&1; then
176     warn "wgcf not found installing..."
177
178     if [[ "$OSTYPE" == "darwin*" ]]; then
179         command -v brew >> output.log 2>&1 || die "Homebrew not found.\nInstall wgcf manually: https://
180         github.com/ViRb3/wgcf/releases"
181         brew install wgcf -q
182     elif [[ "$OSTYPE" == "linux-gnu*" ]]; then
183         ARCH=$(uname -m)
184         case $ARCH in
185             x86_64) WGCF_ARCH="amd64" ;;
186             aarch64) WGCF_ARCH="arm64" ;;
187             armv7l) WGCF_ARCH="armv7" ;;
188             *) die "Unsupported architecture: $ARCH.\nInstall wgcf manually: https://github.com/ViRb3/
189             wgcf/releases" ;;
190         esac
191
192         echo " Fetching latest release..."
193         WGCF_VERSION=$(curl -sf https://api.github.com/repos/ViRb3/wgcf/releases/latest \
194         | grep 'tag_name' | cut -d'"' -f4)
195         [[ -z "$WGCF_VERSION" ]] && die "Could not fetch wgcf version. Check your internet connection."
196
197         sudo curl -sfl \
198         "https://github.com/ViRb3/wgcf/releases/download/${WGCF_VERSION}/wgcf-${WGCF_VERSION#v}
199         _linux-${WGCF_ARCH}" \
200         -o /usr/local/bin/wgcf
201         sudo chmod +x /usr/local/bin/wgcf
202     else
203         die "Unsupported OS. Install wgcf manually: https://github.com/ViRb3/wgcf/releases"
204     fi
205 fi
206 ok "wgcf is ready."
207 # 4. WARP profile
208 step "Generating Cloudflare WARP profile"
209
210 if [[ -f "wgcf-profile.conf" ]]; then
211     warn "wgcf-profile.conf already exists skipping key generation."
212 else
213     wgcf register --accept-tos 2>> output.log || die "wgcf register failed. Check your internet
214     connection."

```

```

214     wgcf generate 2>> output.log           || die "wgcf generate failed."
215     ok "WARP profile generated."
216 fi
217
218 # Parse keys (IPv4 address only  gluetun doesn't need the IPv6 one)
219 PRIVATE_KEY=$(awk '/PrivateKey/{print $3}' wgcf-profile.conf)
220 PUBLIC_KEY=$(awk '/PublicKey/{print $3}'   wgcf-profile.conf)
221 ADDRESSES=$(awk '/Address/{print $3}'      wgcf-profile.conf | grep -v ':' | head -1)
222
223 [[ -z "$PRIVATE_KEY" ]] && die "Could not parse PrivateKey from wgcf-profile.conf"
224 [[ -z "$PUBLIC_KEY"  ]] && die "Could not parse PublicKey from wgcf-profile.conf"
225 [[ -z "$ADDRESSES"   ]] && die "Could not parse IPv4 Address from wgcf-profile.conf"
226
227 ok "WARP keys extracted."
228
229 # 5. Tailscale auth key
230 step "Tailscale authentication"
231
232 TS_AUTHKEY=""
233 if [[ -f ".env" ]]; then
234     EXISTING_TS_KEY=$(read_env_var TS_AUTHKEY)
235     if [[ -n "$EXISTING_TS_KEY" ]]; then
236         TS_AUTHKEY="$EXISTING_TS_KEY"
237         warn "Found existing TS_AUTHKEY in .env. Skipping prompt."
238     fi
239 fi
240
241 if [[ -z "$TS_AUTHKEY" ]]; then
242     echo ""
243     echo "  Generate a key at: https://login.tailscale.com/admin/settings/keys"
244     echo "  'Generate auth key'  enable Reusable if you plan to re-run setup"
245     echo ""
246     read -rp "  Paste your Tailscale auth key: " TS_AUTHKEY
247     echo ""
248 fi
249
250 [[ -z "$TS_AUTHKEY" ]] && die "Tailscale auth key is required."
251 [[ "$TS_AUTHKEY" != tskey-auth-* ]] && warn "Key doesn't look like a Tailscale auth key  proceeding
252 anyway."
253 ok "Auth key accepted."
254
255 # 6. AdGuard Home admin password
256 step "AdGuard Home admin account"
257
258 # Hardcoded default credentials to prevent lockout
259 # Username: admin
260 # Password: nullexit
261 if [[ -f ".env" ]]; then
262     EXISTING_AG_PASS=$(read_env_var ADGUARD_PASSWORD)
263     if [[ -n "$EXISTING_AG_PASS" ]]; then
264         ADGUARD_PASSWORD="$EXISTING_AG_PASS"
265         ADGUARD_PWD_ESC="$EXISTING_AG_PASS"
266         ok "Found existing ADGUARD_PASSWORD in .env."
267     else
268         ADGUARD_PASSWORD="nullexit"
269         ADGUARD_PWD_ESC="nullexit"
270         ok "Default password set to 'nullexit'."
271     fi
272 else
273     ADGUARD_PASSWORD="nullexit"
274     ADGUARD_PWD_ESC="nullexit"
275     ok "Default password set to 'nullexit'."
276 fi
277
278 # 7. Write .env
279 step "Writing .env"
280
281 if [[ -f ".env" ]]; then
282     read -rp "  .env already exists. Overwrite? [y/N]: " OVERWRITE
283     if [[ "$OVERWRITE" != "y" && "$OVERWRITE" != "Y" ]]; then
284         warn "Keeping existing .env."
285     else
286         WRITE_ENV=1
287     fi
288 else
289     WRITE_ENV=1
290 fi
291
292 if [[ "${WRITE_ENV:-0}" == "1" ]]; then
293     # Preserve existing configuration flags if .env already exists

```

```

294 EXISTING_PROFILE="medium"
295 EXISTING_MSS="1120"
296 EXISTING_HIJACK="true"
297 EXISTING_EXIT="true"
298 EXISTING_THRESH="6"
299 EXISTING_KILL="false"
300 EXISTING_BLOCKED="kp il"
301 EXISTING_HOST_MTU="1200"
302
303 if [[ -f ".env" ]]; then
304     [[ -n "$(read_env_var GATEWAY_RULE_PROFILE)" ]] && EXISTING_PROFILE=$(read_env_var
GATEWAY_RULE_PROFILE)
305     [[ -n "$(read_env_var GATEWAY_MSS)" ]] && EXISTING_MSS=$(read_env_var GATEWAY_MSS)
306     [[ -n "$(read_env_var GATEWAY_HIJACK_HOST)" ]] && EXISTING_HIJACK=$(read_env_var
GATEWAY_HIJACK_HOST)
307     [[ -n "$(read_env_var GATEWAY_USE_EXIT_NODE)" ]] && EXISTING_EXIT=$(read_env_var
GATEWAY_USE_EXIT_NODE)
308     [[ -n "$(read_env_var WARP_FAIL_THRESHOLD)" ]] && EXISTING_THRESH=$(read_env_var
WARP_FAIL_THRESHOLD)
309     [[ -n "$(read_env_var KILL_SWITCH)" ]] && EXISTING_KILL=$(read_env_var KILL_SWITCH)
310     [[ -n "$(read_env_var BLOCKED_COUNTRIES)" ]] && EXISTING_BLOCKED=$(read_env_var BLOCKED_COUNTRIES
)
311     [[ -n "$(read_env_var HOST_MTU)" ]] && EXISTING_HOST_MTU=$(read_env_var HOST_MTU)
312 fi
313
314 cat > .env <<EOF
315 WIREGUARD_PRIVATE_KEY=${PRIVATE_KEY}
316 WIREGUARD_PUBLIC_KEY=${PUBLIC_KEY}
317 WIREGUARD_ADDRESSES=${ADDRESSES}
318 TS_AUTHKEY=${TS_AUTHKEY}
319 GATEWAY_RULE_PROFILE=${EXISTING_PROFILE}
320 # Safe MSS for double-tunneled traffic (Tailscale + WARP). Change to 1180 if you experience slow speeds
and have a healthy path.
321 GATEWAY_MSS=${EXISTING_MSS}
322 # Lower the host Tailscale MTU to 1200 to prevent fragmentation when passing through the 1280-byte WARP
tunnel.
323 HOST_MTU=${EXISTING_HOST_MTU}
324 # Set to false to run as a 'Headless Server' (Docker acts as an exit node, but the host Mac/Linux's own
internet is NOT hijacked).
325 GATEWAY_HIJACK_HOST=${EXISTING_HIJACK}
326 GATEWAY_USE_EXIT_NODE=${EXISTING_EXIT}
327 WARP_FAIL_THRESHOLD=${EXISTING_THRESH}
328 KILL_SWITCH=${EXISTING_KILL}
329 ADGUARD_PASSWORD=${ADGUARD_PASSWORD}
330 BLOCKED_COUNTRIES="${EXISTING_BLOCKED}"
331 EOF
332 ok ".env written."
333 fi
334
335 # 8. Prepare directories
336 step "Preparing AdGuard directories"
337
338 mkdir -p adguard/conf adguard/work/userfilters
339
340 # Remove empty AdGuardHome.yaml that would cause a crash loop (same logic as Toggle-Gateway scripts)
341 if [[ -f "adguard/conf/AdGuardHome.yaml" ]] && ! -s "adguard/conf/AdGuardHome.yaml" ]; then
342     rm "adguard/conf/AdGuardHome.yaml"
343     warn "Removed empty AdGuardHome.yaml to prevent crash loop."
344 fi
345
346 ok "Directories ready."
347
348 # 9. Compile DNS filter rules
349 step "Compiling DNS filter rules (this may take a minute)"
350
351
352 # 10. Start containers
353 step "Starting containers"
354 # Note: tailscale depends on adguardhome being healthy (port 5335 up),
355 # so Docker Compose will hold it back automatically until the wizard is done.
356 docker compose up -d
357 ok "Containers starting."
358
359 # 11. Wait for AdGuard Home setup wizard
360 step "Waiting for AdGuard Home setup wizard"
361 echo " (up to 60 seconds...)"
362
363 MAX=60; ELAPSED=0
364 until docker exec routing-fix curl -sf http://127.0.0.1:3000 >> output.log 2>&1; do
365     sleep 2; ELAPSED=$((ELAPSED + 2))

```

```

366 [[ $ELAPSED -ge $MAX ]] && die "AdGuard Home didn't come up.\nDebug with: docker compose logs
    adguardhome"
367 done
368
369 ok "AdGuard Home is up."
370
371 # 12. Configure AdGuard Home via API
372 step "Configuring AdGuard Home"
373
374 # Run all API calls in a single docker exec sh -c so the session cookie file
375 # is shared across calls without leaving the container.
376 docker exec routing-fix sh -c "
377     set -e
378
379     # Step 1: Complete the setup wizard (sets admin creds + DNS port to 5335)
380     curl -sf -X POST http://127.0.0.1:3000/control/install/configure \
381         -H 'Content-Type: application/json' \
382         -d '{"web":{"ip":"0.0.0.0","port":3000},"dns":{"ip":"0.0.0.0","port":5335},"username
    ":"admin","password":"'${ADGUARD_PWD_ESC}'"}' \
383         >/dev/null || true
384
385     # Step 2: Wait for AdGuard to restart after wizard
386     sleep 4
387     WAITED=0
388     until curl -sf http://127.0.0.1:3000 >/dev/null 2>&1; do
389         sleep 2; WAITED=$((WAITED + 2))
390         [ \ $WAITED -ge 30 ] && { echo 'AdGuard did not restart in time'; exit 1; }
391     done
392
393     # Step 3: Login to get session cookie
394     curl -sf -X POST http://127.0.0.1:3000/control/login \
395         -H 'Content-Type: application/json' \
396         -d '{"name":"'admin'","password":"'${ADGUARD_PWD_ESC}'"}' \
397         -c /tmp/agh.cookie >/dev/null
398
399     # Step 4: Point upstream DNS back through the WARP tunnel
400     curl -sf -X POST http://127.0.0.1:3000/control/dns_config \
401         -H 'Content-Type: application/json' \
402         -b /tmp/agh.cookie \
403         -d '{"upstream_dns":["127.0.0.1:53","[/onion/]127.0.0.1:5353"],"bootstrap_dns":["1.1.1.1",\
    "8.8.8.8"],"upstream_mode":"'load_balance'"}' \
404         >/dev/null
405
406     # Step 5: Register the compiled rules file as a filter list
407     curl -sf -X POST http://127.0.0.1:3000/control/filtering/add_url \
408         -H 'Content-Type: application/json' \
409         -b /tmp/agh.cookie \
410         -d '{"name":"'Custom Compiled Rules'","url":"'"/opt/adguardhome/work/userfilters/compiled_rules.
    txt'"}}' \
411         >/dev/null || true
412
413     # Step 6: Force a filter refresh to load the rules immediately
414     curl -sf -X POST http://127.0.0.1:3000/control/filtering/refresh \
415         -H 'Content-Type: application/json' \
416         -b /tmp/agh.cookie \
417         -d '{"whitelist":false}' \
418         >/dev/null || true
419
420     rm -f /tmp/agh.cookie
421 "
422
423 ok "AdGuard Home configured."
424
425 # 13. Wait for Tailscale
426 step "Waiting for Tailscale to authenticate"
427 echo " (Tailscale only starts once AdGuard's healthcheck passes up to 90s)"
428
429 MAX=90; ELAPSED=0; TS_IP=""
430 until [[ -n "$TS_IP" ]]; do
431     TS_IP=$(docker exec tailscale tailscale ip -4 2>> output.log || true)
432     if [[ -z "$TS_IP" ]]; then
433         sleep 3; ELAPSED=$((ELAPSED + 3))
434         if [[ $ELAPSED -ge $MAX ]]; then
435             warn "Tailscale hasn't authenticated yet."
436             warn "Once it does, re-run the toggle script it will resolve the IP dynamically."
437             break
438         fi
439     fi
440 done
441
442 if [[ -n "$TS_IP" ]]; then

```

```

443 ok "Tailscale IP: ${TS_IP} (resolved dynamically by the toggle script on each startup)"
444 fi

```

49 scripts/sweep.sh

```

1 #!/bin/bash
2 # scripts/sweep.sh nullexit gateway health sweep
3 #
4 # Automates the 6-check post-restart health sweep (sweep.md 1 + tailnet ping).
5 # It answers one question: "after a toggle/restart/wake, is the gateway
6 # actually healthy end-to-end no leak, direct P2P, DNS filtering live,
7 # kill-switch armed?" It is READ-ONLY: it never mutates routing, PF, DNS,
8 # or containers, so it is safe to run against a live gateway at any time
9 # (including from a remote session see the caveat in agent.md L16).
10 #
11 # The 6 checks (1-5 mirror sweep.md 1):
12 # 1. Containers warp, adguardhome, tailscale, tor, routing-fix up/healthy
13 # 2. Double-tunnel/leak container WARP exit IP == host exit IP, both warp=on
14 # 3. Hostcontainer path Tailscale exit-node is a DIRECT P2P conn, not DERP relay
15 # 4. AdGuard filtering compiled rule counts present + live DNS interception
16 # 5. PF kill-switch pfctl Enabled + anchor com.apple/nullexit carries rules
17 # 6. Tailnet reachability every ONLINE Tailscale peer answers a tailscale ping
18 #
19 # A timestamped report is written to the project root (one file per run, so
20 # runs can be diffed). stdout carries a coloured summary + a final PASS/FAIL
21 # verdict; exit code is 0 when every check passes, 1 otherwise (so it can gate
22 # CI / launchd / a post-restart hook).
23 #
24 # Usage:
25 # bash scripts/sweep.sh # run the sweep, write report, print verdict
26 # bash scripts/sweep.sh --quiet # only the final verdict line + non-zero on FAIL
27 # bash scripts/sweep.sh --help # this message
28 #
29 set -uo pipefail
30 # NOTE: errexit is intentionally OFF. Several probes are EXPECTED to fail on an
31 # unhealthy gateway and that failure IS the diagnostic signal we record each
32 # exit status rather than letting one abort the whole sweep.
33 #
34 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
35 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
36 TIMESTAMP="$(date -u +%Y%m%d%H%M%S)"
37 cd "$PROJECT_ROOT" || { echo "[FATAL] cannot cd to $PROJECT_ROOT" >&2; exit 1; }
38 source "$SCRIPT_DIR/common.sh"
39 LOG_FILE="$PROJECT_ROOT/output.log"
40 REPORT_FILE="$PROJECT_ROOT/sweep-$TIMESTAMP.txt"
41 #
42 # Argument parsing
43 QUIET=false
44 case "${1:-}" in
45 --quiet|-q) QUIET=true ;;
46 --help|-h) sed -n '2,27p' "$0"; exit 0 ;;
47 *)
48 : ;;
49 *)
50 echo "Unknown argument: $1" >&2
51 echo "Run with --help for usage." >&2
52 exit 2 ;;
53 esac
54 #
55 # Colours (auto-disabled when stdout is not a tty, e.g. piped)
56 if [ -t 1 ]; then
57 RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
58 BOLD='\033[1m'; NC='\033[0m'
59 else
60 RED=''; GREEN=''; YELLOW=''; BOLD=''; NC=''
61 fi
62 #
63 # Homebrew on PATH (same as toggle.sh / recover.sh / diagnose-host-leak)
64 export PATH="/opt/homebrew/bin:/usr/local/bin:$PATH"
65 #
66 # Output helpers write to BOTH stdout and the report file
67 exec 3>> "$REPORT_FILE" || {
68 printf '\n[FATAL] cannot open %s for write\n' "$REPORT_FILE" >&2
69 exit 1
70 }
71 emit() { [ "$QUIET" = true ] || printf '%b\n' "$1"; printf '%b\n' "$1" | sed 's/\x1b[[0-9;]*m//g' >&3; }
72 section() { emit ""; emit "${BOLD} $1 ${NC}"; }

```

```

71 line()      { emit " $1"; }
72 ok()        { line "${GREEN}${NC} $1"; }
73 warn()      { line "${YELLOW}${NC} $1"; }
74 fail()      { line "${RED}${NC} $1"; }
75
76 # common.sh redefines run_with_timeout; it is a pure-bash macOS-safe timeout
77 # (GNU 'timeout' is absent on stock macOS). We reuse the sourced version.
78
79 # Result accounting
80 # Each check appends "PASS"/"WARN"/"FAIL" so the verdict is a pure roll-up.
81 declare -a RESULTS=( )
82 record() { RESULTS+=("${1}|${2}"); } # status|label
83
84 # Header
85 if [ "$QUIET" != true ]; then
86     emit ""
87     emit ""
88     emit " n u l l e x i t   h e a l t h   s w e e p "
89     emit ""
90     emit " Timestamp (UTC): $TIMESTAMP"
91     emit " Project root:    $PROJECT_ROOT"
92     emit " Report file:     $REPORT_FILE"
93     emit ""
94 fi
95
96 GATEWAY_TS_IP="$(read_adguard_ip)" # gateway Tailscale IP from .gateway_ip
97
98 #
99 section "1/6 Containers"
100 #
101 # All five services must be running; warp + adguardhome additionally expose a
102 # healthcheck and must be 'healthy'. 'docker compose ps' is the source of truth.
103 EXPECTED_SERVICES="warp adguardhome tailscale tor routing-fix"
104 if ! command -v docker >/dev/null 2>&1; then
105     fail "docker CLI not on PATH cannot inspect containers"
106     record FAIL "containers"
107 else
108     PS_OUT=$(run_with_timeout 20 docker compose ps 2>>"$LOG_FILE")
109     missing=""; unhealthy=""
110     for svc in $EXPECTED_SERVICES; do
111         row=$(printf '%s\n' "$PS_OUT" | awk -v s="$svc" 's==s || $0 ~ ("[:space:]"s"[:space:]")')
112         if [ -z "$row" ] || ! printf '%s' "$row" | grep -qiE 'up|running'; then
113             missing="$missing $svc"
114         elif printf '%s' "$row" | grep -qi 'unhealthy'; then
115             unhealthy="$unhealthy $svc"
116         fi
117     done
118     if [ -z "$missing" ] && [ -z "$unhealthy" ]; then
119         ok "all 5 services up (warp/adguardhome healthy, tailscale/tor/routing-fix running)"
120         record PASS "containers"
121     else
122         [ -n "$missing" ] && fail "not up/running:$missing"
123         [ -n "$unhealthy" ] && fail "reporting unhealthy:$unhealthy"
124         record FAIL "containers"
125     fi
126     line " docker compose ps "
127     printf '%s\n' "$PS_OUT" | awk '{print "    "$0}' >&3
128     [ "$QUIET" = true ] || printf '%s\n' "$PS_OUT" | awk '{print "    "$0}'
129 fi
130
131 #
132 section "2/6 Double-tunnel / IP-leak"
133 #
134 # The container's WARP egress and the host's egress must be the SAME public IP
135 # with warp=on for both. Equal IPs all host traffic is exiting through the
136 # same WARP tunnel the container uses (no split / raw-ISP leak).
137 # The warp container (gluetun) ships wget, not curl fetch the trace from
138 # inside its nets with whichever client is present.
139 CTR_TRACE=$(run_with_timeout 12 docker compose exec -T warp sh -c \
140     'curl -s --max-time 8 https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null \
141     || wget -qO- --timeout=8 https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null' \
142     2>>"$LOG_FILE")
143 HOST_TRACE=$(run_with_timeout 12 curl -s --max-time 8 \
144     https://www.cloudflare.com/cdn-cgi/trace 2>>"$LOG_FILE")
145 CTR_IP=$(printf '%s' "$CTR_TRACE" | awk -F= '/^ip={print $2;exit}')
146 CTR_WARP=$(printf '%s' "$CTR_TRACE" | awk -F= '/^warp={print $2;exit}')
147 HOST_IP=$(printf '%s' "$HOST_TRACE" | awk -F= '/^ip={print $2;exit}')
148 HOST_WARP=$(printf '%s' "$HOST_TRACE" | awk -F= '/^warp={print $2;exit}')
149 line "container exit: ip=${CTR_IP:-?} warp=${CTR_WARP:-?}"
150 line "host exit:      ip=${HOST_IP:-?} warp=${HOST_WARP:-?}"
151 if [ -z "$CTR_IP" ] || [ -z "$HOST_IP" ]; then

```



```

152 fail "could not obtain both egress IPs (container or host trace failed)"
153 record FAIL "leak"
154 elif [ "$CTR_IP" = "$HOST_IP" ] && [ "$HOST_WARP" = "on" ]; then
155 ok "no leak host egress == container egress ($HOST_IP), warp=on"
156 record PASS "leak"
157 elif [ "$HOST_WARP" != "on" ]; then
158 fail "host warp=$HOST_WARP host traffic is NOT going through WARP (LEAK)"
159 record FAIL "leak"
160 else
161 fail "egress mismatch: host $HOST_IP != container $CTR_IP split/leak"
162 record FAIL "leak"
163 fi
164
165 #
166 section "3/6 Host container path (direct P2P, not DERP relay)"
167 #
168 # 'tailscale status' annotates each peer with 'direct <ip:port>' or
169 # 'relay <derp>'. The gateway peer should be DIRECT a DERP relay hop adds
170 # latency and signals NAT traversal failure between host and gateway.
171 if ! command -v tailscale >/dev/null 2>&1; then
172 fail "tailscale CLI not on PATH"
173 record FAIL "p2p"
174 else
175 TS_OUT=$(run_with_timeout 6 tailscale status 2>>"$LOG_FILE")
176 PEER_LINE=$(printf '%s\n' "$TS_OUT" | grep -E "(^|[:space:])$GATEWAY_TS_IP([[:space:]]|$)" | head -1)
177 if [ -z "$PEER_LINE" ]; then
178 warn "gateway $GATEWAY_TS_IP not found in host peer list"
179 line "$(printf '%s\n' "$TS_OUT" | head -5 | awk '{print " " "$0}')"
180 record WARN "p2p"
181 elif printf '%s' "$PEER_LINE" | grep -q 'direct '; then
182 conn=$(printf '%s' "$PEER_LINE" | grep -oE 'direct [0-9.]+:[0-9.]+')
183 ok "direct P2P to gateway $conn (no DERP relay)"
184 record PASS "p2p"
185 elif printf '%s' "$PEER_LINE" | grep -q 'relay '; then
186 relay=$(printf '%s' "$PEER_LINE" | grep -oE 'relay "[^"]+"')
187 warn "gateway reached via DERP $relay (not direct) NAT traversal degraded"
188 record WARN "p2p"
189 else
190 warn "gateway peer state indeterminate (idle/no active conn)"
191 line "$PEER_LINE"
192 record WARN "p2p"
193 fi
194 fi
195
196 #
197 section "4/6 AdGuard filtering"
198 #
199 # Two signals: (a) the compiled ruleset exists with a non-trivial block count,
200 # (b) the gateway resolver actually answers a query (live interception).
201 COMPILED="adguard/work/userfilters/compiled_rules.txt"
202 if [ -f "$COMPILED" ]; then
203 TOTAL=$(grep -m1 '! Total Block Rules:' "$COMPILED" | awk -F': ' '{print $2}' | tr -d '[:space:]')
204 NATIVE=$(grep -m1 '! Native AdGuard Rules:' "$COMPILED" | awk -F': ' '{print $2}' | tr -d '[:space:]')
205 if [ -n "$TOTAL" ] && [ "$TOTAL" -gt 0 ] 2>/dev/null; then
206 ok "compiled ruleset present Total Block Rules: $TOTAL / Native AdGuard: ${NATIVE:-?}"
207 RULES_OK=true
208 else
209 warn "compiled_rules.txt present but block-rule count unreadable"
210 RULES_OK=false
211 fi
212 else
213 warn "compiled_rules.txt missing (rule-compile may not have run)"
214 RULES_OK=false
215 fi
216 # Live interception: the gateway resolver must answer (dig via .gateway_ip).
217 DNS_OK=false
218 if command -v dig >/dev/null 2>&1 && [ -n "$GATEWAY_TS_IP" ]; then
219 DIG_OUT=$(run_with_timeout 8 dig +time=3 +tries=1 @"$GATEWAY_TS_IP" google.com A 2>>"$LOG_FILE")
220 if printf '%s' "$DIG_OUT" | grep -qE 'ANSWER SECTION|status: NOERROR'; then
221 ok "live DNS interception gateway $GATEWAY_TS_IP resolved google.com"
222 DNS_OK=true
223 else
224 warn "gateway $GATEWAY_TS_IP did not answer a test query"
225 fi
226 else
227 warn "skipped live-resolution probe (dig missing or gateway IP unknown)"
228 fi
229 if [ "$RULES_OK" = true ] && [ "$DNS_OK" = true ]; then
230 record PASS "adguard"
231 elif [ "$RULES_OK" = true ] || [ "$DNS_OK" = true ]; then
232 record WARN "adguard"

```

```

233 else
234     record FAIL "adguard"
235 fi
236
237 #
238 section "5/6 PF kill-switch"
239 #
240 # PF must be Enabled AND the com.apple/nullexit anchor must carry rules.
241 # An empty/absent anchor means the fail-closed lane is not actually armed.
242 PF_STATUS=$(run_with_timeout 6 sudo -n pfctl -s info 2>>"$LOG_FILE" | awk '/^Status:/{print $2;exit}')
243 ANCHOR_RULES=$(run_with_timeout 6 sudo -n pfctl -a com.apple/nullexit -sr 2>>"$LOG_FILE")
244 if [ "$PF_STATUS" = "Enabled" ] && [ -n "$ANCHOR_RULES" ]; then
245     RULE_N=$(printf '%s\n' "$ANCHOR_RULES" | grep -cvE '~\s*$')
246     ok "PF Enabled + anchor com.apple/nullexit armed ($RULE_N rules)"
247     line "$(printf '%s\n' "$ANCHOR_RULES" | head -4 | awk '{print "    "$0}')"
248     record PASS "pf"
249 elif [ "$PF_STATUS" = "Enabled" ]; then
250     fail "PF Enabled but anchor com.apple/nullexit has NO rules kill-switch not armed"
251     record FAIL "pf"
252 elif [ -z "$PF_STATUS" ]; then
253     warn "could not read PF status (needs passwordless sudo for 'pfctl -s info')"
254     record WARN "pf"
255 else
256     fail "PF Status: $PF_STATUS kill-switch disabled"
257     record FAIL "pf"
258 fi
259
260 #
261 section "6/6 Tailnet reachability (ping all online devices)"
262 #
263 # Every peer the coordination server reports as ONLINE should answer a
264 # 'tailscale ping' (TSMP probes the WireGuard data path; ICMP is avoided since
265 # phones/laptops often drop it). Offline peers and this host are skipped.
266 # An online-but-unreachable peer means control plane and data path disagree.
267 if ! command -v tailscale >/dev/null 2>&1; then
268     fail "tailscale CLI not on PATH"
269     record FAIL "tailnet"
270 else
271     SELF_TS_IP=$(run_with_timeout 6 tailscale ip -4 2>>"$LOG_FILE")
272     [ -n "$TS_OUT:-" ] || TS_OUT=$(run_with_timeout 6 tailscale status 2>>"$LOG_FILE")
273     # Online peers = rows starting with a Tailscale IP, minus self and 'offline'.
274     # (A '-' status column means online-but-idle, so it is kept.)
275     PEERS=$(printf '%s\n' "$TS_OUT" | awk -v self="$SELF_TS_IP" \
276         '$1 ~ /^100\./ && $1 != self && $0 !~ /^(|[:space:]))offline(,|[:space:])$/ {print $1, $2
277         }')
278     if [ -z "$PEERS" ]; then
279         warn "no online peers to ping (everything else in the tailnet is offline)"
280         record WARN "tailnet"
281     else
282         unreached=0
283         while read -r peer_ip peer_name; do
284             PONG=$(run_with_timeout 10 tailscale ping --timeout=3s --c 1 --until-direct=false "$peer_ip" 2>>"$LOG_FILE")
285             if printf '%s' "$PONG" | grep -q '~pong'; then
286                 detail=$(printf '%s\n' "$PONG" | head -1 | sed -E 's/^pong from [^ ]+ \([^\)]+\) //' )
287                 ok "$peer_name ($peer_ip) pong $detail"
288             else
289                 fail "$peer_name ($peer_ip) no pong (online per control plane, data path dead)"
290                 unreached=$((unreached+1))
291             fi
292         done <<< "$PEERS"
293         n_peers=$(printf '%s\n' "$PEERS" | wc -l | tr -d '[:space:]')
294         if [ "$unreached" -eq 0 ]; then
295             record PASS "tailnet"
296         elif [ "$unreached" -lt "$n_peers" ]; then
297             record WARN "tailnet"
298         else
299             record FAIL "tailnet"
300         fi
301     fi
302 fi
303
304 #
305 # Cover traffic (opt-in) informational only, NOT a scored check.
306 # Shown solely when NOISE_ENABLED=true so the sweep stays read-only and the
307 # 6-check verdict is unchanged. Reports whether the padding daemon is alive.
308 NOISE_ON=$(read_env_var "NOISE_ENABLED" | tr '[:upper:]' '[:lower:]')
309 if [ "$NOISE_ON" = "true" ]; then
310     section "Cover traffic (opt-in, informational)"
311     NOISE_PID_FILE="/tmp/nullexit-noise.pid"
312     npid=$(cat "$NOISE_PID_FILE" 2>/dev/null)

```

```

312 if [ -n "$npid" ] && kill -0 "$npid" 2>/dev/null; then
313     ok "dummy-padding daemon running (pid $npid) see 'bash scripts/noise.sh status'"
314 else
315     warn "NOISE_ENABLED=true but no padding daemon running start it: bash scripts/noise.sh start"
316 fi
317 fi
318
319 #
320 # DNS anomaly scan (informational, read-only, NOT a scored check the 6-check
321 # verdict is unchanged). It only READS the AdGuard querylog; never blocks DNS.
322 # It FAILS LOUD: a broken detector (crash, corrupt model, empty log, failing
323 # self-test) shows a red with the reason it is NEVER silently reported as
324 # clean, and an untrained detector is announced rather than skipped in silence.
325 QUERYLOG="$PROJECT_ROOT/adguard/work/data/querylog.json"
326 DNS_TOOL="$SCRIPT_DIR/dns_anomaly_detector.py"
327 if command -v python3 >/dev/null 2>&1 && [ -f "$DNS_TOOL" ]; then
328     section "DNS anomaly scan (informational)"
329     # 1) Prove the detection logic itself works before trusting any "clean".
330     ST_ERR=$(python3 "$DNS_TOOL" selftest 2>&1 >/dev/null); ST_RC=$?
331     if [ "$ST_RC" -ne 0 ]; then
332         fail "DNS detector SELF-TEST FAILED (rc=$ST_RC) detector is broken, do NOT trust results: ${ST_ERR:-
333             see stderr}"
334     elif [ ! -f "$PROJECT_ROOT/.dns_baseline.json" ]; then
335         warn "DNS detector NOT trained (no .dns_baseline.json) not protecting; run: python3 scripts/
336             dns_anomaly_detector.py learn"
337     elif [ ! -f "$QUERYLOG" ]; then
338         warn "DNS detector trained but querylog absent ($QUERYLOG) nothing to scan"
339     else
340         # 2) Real scan. Capture stderr + exit code; do NOT swallow failures.
341         DNS_ERR=$(python3 "$DNS_TOOL" scan --quiet 2>&1 >/tmp/nullexit-dns-scan.out); DNS_RC=$?
342         DNS_OUT=$(cat /tmp/nullexit-dns-scan.out 2>/dev/null); rm -f /tmp/nullexit-dns-scan.out
343         case "$DNS_RC" in
344             0) ok "no DNS-tunneling / DGA signatures $DNS_OUT" ;;
345             1) warn "$DNS_OUT review: python3 scripts/dns_anomaly_detector.py scan" ;;
346             *) fail "DNS scan FAILED to run (rc=$DNS_RC) NOT a clean result: ${DNS_ERR:-unknown error}" ;;
347         esac
348     fi
349 fi
350
351 #
352 section "VERDICT"
353 #
354 pass=0; warnc=0; failc=0
355 for r in "${RESULTS[@]"; do
356     case "${r%|*}" in PASS) pass=$((pass+1));; WARN) warnc=$((warnc+1));; FAIL) failc=$((failc+1));; esac
357 done
358 for r in "${RESULTS[@]"; do
359     st="${r%|*}"; lbl="${r##|}"
360     case "$st" in
361         PASS) ok "$lbl" ;;
362         WARN) warn "$lbl" ;;
363         FAIL) fail "$lbl" ;;
364     esac
365 done
366
367 emit ""
368 if [ "$failc" -eq 0 ] && [ "$warnc" -eq 0 ]; then
369     emit "${GREEN}${BOLD}SWEEP PASS gateway healthy (${pass}/${#RESULTS[@]} checks green)${NC}"
370     EXIT=0
371 elif [ "$failc" -eq 0 ]; then
372     emit "${YELLOW}${BOLD}SWEEP PASS with warnings ${pass} pass, ${warnc} warn, 0 fail${NC}"
373     EXIT=0
374 else
375     emit "${RED}${BOLD}SWEEP FAIL ${pass} pass, ${warnc} warn, ${failc} fail${NC}"
376     EXIT=1
377 fi
378
379 emit ""
380 [ "$QUIET" = true ] || emit "Full report: $REPORT_FILE"
381 exit "$EXIT"

```

50 scripts/unlock-files.sh

```

1 #!/bin/bash
2 # Resolve project root relative to this script's location, so the script
3 # works regardless of the caller's CWD.
4 SCRIPT_DIR="$(dirname "${BASH_SOURCE[0]}")" && pwd)
5 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"

```

```

6
7 echo "Unlocking .env..."
8 if [ -f "$PROJECT_ROOT/.env" ]; then
9     sudo cat "$PROJECT_ROOT/.env" > "$PROJECT_ROOT/.env.tmp" && mv "$PROJECT_ROOT/.env.tmp" "$PROJECT_ROOT
    /.env"
10     echo ".env unlocked"
11 fi
12
13 echo "Unlocking AdGuard cache files..."
14 for f in "$PROJECT_ROOT"/adguard/work/userfilters/compiled_rules.txt \
15         "$PROJECT_ROOT"/adguard/work/userfilters/ip_blocklist.ipset \
16         "$PROJECT_ROOT"/adguard/work/userfilters/cache/*.txt \
17         "$PROJECT_ROOT"/adguard/work/userfilters/cache/ip/*.txt \
18         "$PROJECT_ROOT"/adguard/work/data/filters/*.txt; do
19     if [ -f "$f" ]; then
20         cat "$f" > "$f.tmp" && mv "$f.tmp" "$f"
21         echo "Unlocked $f"
22     fi
23 done
24
25 echo ""
26 echo "All locked files have been successfully bypassed via atomic rename!"

```

51 tor-bridges.txt

52 white_list.txt

```

1 0.client-channel.google.com
2 idrv.com
3 2.android.pool.ntp.org
4 akamaihd.net
5 akamaitechnologies.com
6 akamaized.net
7 amazonaws.com
8 android.clients.google.com
9 api-adservices.apple.com
10 api.ipify.org
11 api.rlje.net
12 app-api.ted.com
13 appleid.apple.com
14 apps.skype.com
15 appsbackup-pa.clients6.google.com
16 appsbackup-pa.googleapis.com
17 appspot-preview.l.google.com
18 apt.sonarr.tv
19 aspnetcdn.com
20 attestation.xboxlive.com
21 ax.phobos.apple.com.edgesuite.net
22 books-analytics-events.apple.com
23 brightcove.net
24 c.s-microsoft.com
25 cdn.cloudflare.net
26 cdn.embedly.com
27 cdn.optimizely.com
28 cdn.vidible.tv
29 cdn2.optimizely.com
30 cdn3.optimizely.com
31 cdnjs.cloudflare.com
32 cert.mgt.xboxlive.com
33 clientconfig.passport.net
34 clients1.google.com
35 clients2.google.com
36 clients3.google.com
37 clients4.google.com
38 clients5.google.com
39 clients6.google.com
40 connectivitycheck.android.com
41 connectivitycheck.gstatic.com
42 continuum.dds.microsoft.com
43 cpms.spop10.ams.plex.bz
44 cpms35.spop10.ams.plex.bz

```

```
45 cse.google.com
46 ctldl.windowsupdate.com
47 cws.conviva.com
48 d2c8v521l5s99u.cloudfront.net
49 d2gatte9o95jao.cloudfront.net
50 dashboard.plex.tv
51 dataplicity.com
52 def-vef.xboxlive.com
53 delivery.vidible.tv
54 dev.virtualearth.net
55 device.auth.xboxlive.com
56 display.ugc.bazaarvoice.com
57 displaycatalog.mp.microsoft.com
58 dl.delivery.mp.microsoft.com
59 dl.dropbox.com
60 dl.dropboxusercontent.com
61 dns.msftncsi.com
62 download.sonarr.tv
63 drift.com
64 driftt.com
65 dynupdate.no-ip.com
66 ecn.dev.virtualearth.net
67 edge.api.brightcove.com
68 eds.xboxlive.com
69 fonts.gstatic.com
70 forums.sonarr.tv
71 g.live.com
72 geo-prod.do.dsp.mp.microsoft.com
73 geo3.ggpht.com
74 gfws1.geforce.com
75 giphy.com
76 github.com
77 github.io
78 googleapis.com
79 gravatar.com
80.gstatic.com
81 help.ui.xboxlive.com
82 hls.ted.com
83 i.ytimg.com
84 il.ytimg.com
85 iadsdk.apple.com
86 imagesak.secureserver.net
87 img.vidible.tv
88 imgix.net
89 imgs.xkcd.com
90 instantmessaging-pa.googleapis.com
91 intercom.io
92 jquery.com
93 jsdelivr.net
94 keystone.mwbsys.com
95 lastfm-img2.akamaized.net
96 licensing.xboxlive.com
97 live.com
98 livepassdl.conviva.com
99 login.live.com
100 login.microsoftonline.com
101 manifest.googlevideo.com
102 meta-db-worker02.pop.ric.plex.bz
103 meta.plex.bz
104 meta.plex.tv
105 microsoftonline.com
106 msftncsi.com
107 my.plexapp.com
108 nexusrules.officeapps.live.com
109 nine.plugins.plexapp.com
110 no-ip.com
111 node.plexapp.com
112 notes-analytics-events.apple.com
113 notify.xboxlive.com
114 npr-news.streaming.adswizz.com
115 ns1.dropbox.com
116 ns2.dropbox.com
117 o1.email.plex.tv
118 o2.sg0.plex.tv
119 ocsp.apple.com
120 office.com
121 office.net
122 office365.com
123 officeclient.microsoft.com
124 om.cbsi.com
125 onedrive.live.com
```

```

126 outlook.live.com
127 outlook.office365.com
128 pings.conviva.com
129 placeholder.it
130 placeholder.it.imgix.net
131 players.brightcove.net
132 pricelist.skype.com
133 products.office.com
134 proxy.plex.bz
135 proxy.plex.tv
136 proxy02.pop.ord.plex.bz
137 pubsub.plex.bz
138 pubsub.plex.tv
139 raw.githubusercontent.com
140 redirector.googlevideo.com
141 res.cloudinary.com
142 s.gateway.messenger.live.com
143 s.marketwatch.com
144 s.youtube.com
145 s.ytimg.com
146 s1.wp.com
147 s2.youtube.com
148 s3.amazonaws.com
149 sa.symcb.com
150 secure.avangate.com
151 secure.brightcove.com
152 secure.surveymonkey.com
153 services.sonarr.tv
154 skyhook.sonarr.tv
155 spclient.wg.spotify.com
156 ssl.p.jwpcdn.com
157 staging.plex.tv
158 status.plex.tv
159 t.co
160 t0.ssl.ak.dynamic.tiles.virtualearth.net
161 t0.ssl.ak.tiles.virtualearth.net
162 tawk.to
163 tedcdn.com
164 themoviedb.com
165 thetvdb.com
166 tinyurl.com
167 title.auth.xboxlive.com
168 title.mgt.xboxlive.com
169 traffic.libsyn.com
170 tvdb2.plex.tv
171 tvthemes.plexapp.com
172 twimg.com
173 ui.skype.com
174 video-stats.l.google.com
175 videos.vidible.tv
176 vidtech.cbsinteractive.com
177 weather-analytics-events.apple.com
178 widget-cdn.rpxnow.com
179 win10.ipv6.microsoft.com
180 wp.com
181 ws.audioscrobbler.com
182 www.dataplicity.com
183 www.googleapis.com
184 www.msftconnecttest.com
185 www.msftncsi.com
186 www.no-ip.com
187 www.youtube-nocookie.com
188 xbox.ipv6.microsoft.com
189 xboxexperiencesprod.experimentation.xboxlive.com
190 xflight.xboxlive.com
191 xkms.xboxlive.com
192 xsts.auth.xboxlive.com
193youtu.be
194 youtube-nocookie.com
195 yt3.ggpht.com
196 zee.cws.conviva.com
197
198 # YouTube family base domains so every subdomain (video stream servers on
199 # *.googlevideo.com, image hosts on *.ytimg.com, profile thumbnails on
200 # *.ggpht.com, etc.) is automatically allowlisted via the AdGuard '||domain~'
201 # wildcard. The specific s.youtube.com / manifest.googlevideo.com entries
202 # above are kept for clarity but will be optimized away by rule-compiler
203 # because their parents are now in this list.
204 youtube.com
205 googlevideo.com
206 ytimg.com

```

```
207 ggphht.com
208 gvt1.com
209 gvt2.com
210 gvt3.com
211 gvt1.net
212 gvt2.net
213 gvt3.net
214
215 edge-mqtt.facebook.com
216 gateway.instagram.com
217 i.instagram.com
218
219 facebook.com
220 fb.com
221 fbcdn.net
222 fbsbx.com
223 messenger.com
224 whatsapp.com
225 web.whatsapp.com
226 instagram.com
227 cdninstagram.com
228 static.whatsapp.net
229 media-bos5-1.cdn.whatsapp.net
230 media-yyz1-1.cdn.whatsapp.net
231 webtp.whatsapp.net
232 mmg.whatsapp.net
233 g.whatsapp.net
234 g-fallback.whatsapp.net
```